Алгоритмы

Джефф Эриксон

Русский перевод первого издания

Перевод подготовлен с помощью ИИ и прошёл выборочную проверку. В некоторых иллюстрациях сохранены английские надписи; указатели в конце приведены из оригинала.

Оригинал: Jeff Erickson, *Algorithms*, первое издание, 2019. © Jeff Erickson.

Оригинал · Лицензия Creative Commons Attribution 4.0 International

Содержание

Предисловие

- Об этой книге
- Предварительные знания
- Дополнительные источники
- Об упражнениях
- Украдите эту книгу!
- Благодарности
- Читатель, берегись!

Введение

- 0.1. Что такое алгоритм?
- 0.2. Умножение
- Умножение решёткой
- Удвоение и деление пополам
- Циркуль и линейка
- 0.3. Распределение мест в Конгрессе
- 0.4. Плохой пример
- 0.5. Описание алгоритмов
- Постановка задачи
- Описание алгоритма
- 0.6. Анализ алгоритмов
- Упражнения

Рекурсия

- 1.1. Сведения
- 1.2. Упрощайте и делегируйте
- 1.3. Ханойская башня
- 1.4. Сортировка слиянием
- 1.5. Быстрая сортировка
- 1.6. Общая схема
- 1.7. Деревья рекурсии
- ♥ 1.8. Выбор за линейное время
- 1.9. Быстрое умножение
- 1.10. Возведение в степень
- Ханойская башня
- Деревья рекурсии
- Сортировка
- Выбор
- Арифметика
- Массивы

Деревья

Поиск с возвратом

2.1. N ферзей

2.2. Игровые деревья

2.3. Сумма подмножества

2.4. Общая схема

2.5. Разбиение текста на слова (*Interpunctio Verborum*)

2.6. Наибольшая возрастающая подпоследовательность

2.7. Наибольшая возрастающая подпоследовательность: вторая попытка

2.8. Оптимальные двоичные деревья поиска

Упражнения

Динамическое программирование

3.1. Матравритта

♥ 3.2. Отступление: ещё более быстрое вычисление чисел Фибоначчи

3.3. Возвращение к interpunctio verborum

3.4. Общая схема: умная рекурсия

3.5. Предупреждение: жадность — это глупо

3.6. Наибольшая возрастающая подпоследовательность

3.7. Расстояние редактирования

3.8. Сумма подмножества

3.9. Оптимальные двоичные деревья поиска

3.10. Динамическое программирование на деревьях

Упражнения

Жадные алгоритмы

4.1. Хранение файлов на ленте

4.2. Составление расписания занятий

4.3. Общая схема

4.4. Коды Хаффмана

4.5. Устойчивое паросочетание

Упражнения

Основы графов

5.1. Введение и история

5.2. Основные определения

5.3. Представления и примеры

5.4. Структуры данных

5.5. Обход в произвольном порядке

5.6. Важные варианты

5.7. Сведения к графам: заливка

Графы

Алгоритмы обхода

Сведения

Поиск в глубину

6.1. Прямой и обратный порядок
6.2. Обнаружение циклов
6.3. Топологическая сортировка
6.4. Мемоизация и динамическое программирование
6.5. Сильная связность
6.6. Сильные компоненты за линейное время
Упражнения

Минимальные остовные деревья

7.1. Различные веса рёбер
7.2. Единственный алгоритм минимального остовного дерева
7.3. Алгоритм Борувки
7.4. Алгоритм Ярника («Прима»)
7.5. Алгоритм Краскала

Кратчайшие пути

8.1. Деревья кратчайших путей
♥ 8.2. Рёбра отрицательного веса
8.3. Единственный алгоритм кратчайших путей из одного источника
8.4. Невзвешенные графы: поиск в ширину
8.5. Ориентированные ациклические графы: поиск в глубину
8.6. Сначала лучший: алгоритм Дейкстры
8.7. Релаксируйте ВСЕ рёбра: алгоритм Беллмана — Форда
Упражнения

Кратчайшие пути между всеми парами

9.1. Введение
9.2. Много исходных вершин
9.3. Изменение весов
9.4. Алгоритм Джонсона
9.5. Динамическое программирование
9.6. Разделяй и властвуй
9.7. Занимательное умножение матриц
9.8. Алгоритм (Клини — Роя —) Флойда — Уоршелла (— Ингермана)
Упражнения

Потоки и разрезы

10.1. Потоки
10.2. Разрезы
10.3. Теорема о максимальном потоке и минимальном разрезе
10.4. Алгоритм Форда — Фалкерсона с увеличивающими путями
10.5. Сложение и разложение потоков
10.6. Алгоритмы Эдмондса и Карпа
10.7. Дальнейшие достижения
Упражнения

Применения потоков

11.1. Пути без общих рёбер
11.2. Пропускные способности вершин и пути без общих вершин
11.3. Паросочетания в двудольном графе
11.4. Выбор кортежей
11.5. Покрытия непересекающимися путями
11.6. Выбывание из бейсбольного турнира
11.7. Выбор проектов
Упражнения

NP-трудность

12.1. Игра, в которой не выиграть
12.2. P против NP
12.3. NP-трудные, принадлежащие NP и NP-полные задачи
♥ 12.4. Формальные определения (HC SVNT DRACONES)
12.5. Сведения и выполнимость формул
12.6. 3SAT (сведение из CIRCUITSAT)
12.7. Максимальное независимое множество (сведение из 3SAT)
12.8. Общая схема
12.9. Клика и вершинное покрытие (сведение из независимого множества)
12.10. Раскраска графа (сведение из 3SAT)
12.11. Гамильтонов цикл
12.12. Сумма подмножества (сведение из вершинного покрытия)
12.13. Другие полезные NP-трудные задачи
12.14. Как выбрать подходящую задачу
12.15. Несерьёзный пример из реальной жизни
♥ 12.16. И за пределами зебры
Упражнения

Источники иллюстраций

Предисловие

Здесь начинается предисловие к книге аль-Хорезми о практике арифметики.

— Иоанн Испанский [Иоанн Севильский?], Liber algorismi de pratica arismetrice (около 1135 года)

Сказать вам, мой друг, как вы придёте к пониманию? Пойдите и напишите об этом книгу.

— Генри Хоум, лорд Кеймс, в письме сэру Гилберту Эллиоту

Отдельный человек всегда ошибается. Он многое задумал, привлёк помощников, поссорился с некоторыми или со всеми, много напортачил — и что-то сделано; все немного продвинулись вперёд, но отдельный человек всегда ошибается. Получается нечто новое и совсем непохожее на то, что он себе обещал.

— Ральф Уолдо Эмерсон, «Опыт», Essays, Second Series (1844)

То, что я изложил выше, составляет содержание книги, основной план которой, возможно, невозможно осуществить и все подробности которой невозможно включить; написанное мною — второй или третий черновик предварительной версии этой книги.

— Майкл Спивак, предисловие к первому изданию Differential Geometry, том I (1970)

Об этой книге

Этот учебник вырос из конспектов лекций, которые я писал для различных курсов по алгоритмам в Иллинойсском университете в Урбане-Шампейне. С января 1999 года я преподаю такие курсы примерно раз в год. В 2016 году изменения в программе бакалаврской теоретической подготовки побудили меня существенно переработать конспекты. Книга состоит из части этих переработанных материалов, посвящённой наиболее фундаментальным темам, и в основном отражает алгоритмическое содержание нашего нового обязательного теоретического курса для студентов третьего года обучения.

Предварительные знания

У моих курсов алгоритмов в Иллинойсе есть два важных предварительных требования: курс дискретной математики и курс основных структур данных. Поэтому большинству студентов эта книга, вероятно, не подойдёт в качестве первого курса по структурам данных и алгоритмам. В частности, я предполагаю хотя бы поверхностное знакомство со следующими темами:

- **Дискретная математика:** школьная алгебра, свойства логарифмов, наивная теория множеств, булева алгебра, логика предикатов первого порядка, множества, функции,

отношения эквивалентности, частичные порядки, арифметика по модулю, рекурсивные определения, деревья как абстрактные объекты, а не структуры данных, и графы как вершины и рёбра, а не графики функций.

- **Методы доказательства:** прямое и косвенное доказательство, доказательство от противного, исчерпывающий разбор случаев и индукция, особенно сильная и структурная. В главе 0 применяется индукция; и если она применяется в главе n − 1, то она применяется и в главе n.
- **Понятия программирования:** переменные, условия, циклы, записи, косвенный доступ через адреса, указатели или ссылки, подпрограммы и рекурсия. Я не предполагаю свободного владения каким-либо конкретным языком, но ожидаю опыта работы хотя бы с одним языком, поддерживающим и косвенный доступ, и рекурсию.
- **Основные абстрактные типы данных:** скаляры, последовательности, векторы, множества, стеки, очереди, отображения или словари, упорядоченные отображения и очереди с приоритетом.
- **Основные структуры данных:** массивы, связные списки — односвязные и двусвязные, линейные и кольцевые, — двоичные деревья поиска, хотя бы один вид сбалансированных двоичных деревьев поиска, например AVL-деревья, красно-чёрные деревья, декартовы деревья, списки с пропусками или splay-деревья, хеш-таблицы, двоичные кучи и, что важнее всего, различие между этим и предыдущим списками.
- **Основные вычислительные задачи:** элементарная арифметика, сортировка, поиск, перечисление, обход деревьев в прямом, симметричном, обратном порядке, по уровням и так далее.
- **Основные алгоритмы:** элементарные арифметические методы, последовательный и двоичный поиск, сортировка выбором, вставками, слиянием, кучей, быстрая, поразрядная и другие, поиск в ширину и глубину хотя бы в двоичных деревьях и, что важнее всего, различие между этим и предыдущим списками.
- **Элементарный анализ алгоритмов:** асимптотические обозначения o, O, Θ, Ω, ω, перевод циклов в суммы и рекурсивных вызовов в рекуррентные соотношения, вычисление простых сумм и решение простых рекуррентных соотношений.
- **Математическая зрелость:** свободная работа с абстракцией, формальными, особенно рекурсивными, определениями и, особенно индуктивными, доказательствами; умение записывать и понимать математические рассуждения, распознавать и избегать синтаксической, семантической и логической бессмыслицы.

В книге некоторые из этих предварительных тем кратко рассматриваются по мере необходимости, но скорее как напоминание, а не полноценное введение. Для более основательного обзора я настоятельно рекомендую следующие свободно доступные источники:

- Margaret M. Fleck. *Building Blocks for Theoretical Computer Science*, версия 1.3 (январь 2013 года) или более поздняя: http://mfleck.cs.illinois.edu/building-blocks/.
- Eric Lehman, F. Thomson Leighton, Albert R. Meyer. *Mathematics for Computer Science*, редакция июня 2018 года: https://courses.csail.mit.edu/6.042/spring18/. Настоятельно рекомендую искать самую свежую редакцию.
- Pat Morin. *Open Data Structures*, издание 0.1Gβ (январь 2016 года) или более позднее: http://opendatastructures.org/.

- Don Sheehy. *A Course in Data Structures and Object-Oriented Design*, редакция февраля 2019 года или более поздняя: https://donsheehy.github.io/datastructures/.

Дополнительные источники

Пожалуйста, не ограничивайтесь этой книгой или каким-либо другим единственным источником. Авторы и читатели привносят собственный взгляд в любой интеллектуальный материал; ни один преподаватель не находит общий язык со всеми студентами, даже с самыми сильными. Чтобы найти автора, который наиболее эффективно передаст вам своё интуитивное понимание, нужно приложить усилия, но со временем они щедро окупятся.

Перечисленные ниже источники были особенно ценны для меня как источники интуиции, примеров, упражнений и вдохновения; список не претендует на полноту.

- Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974. Я пользовался этим учебником на бакалавриате в Университете Райса, а затем во время магистратуры в Калифорнийском университете в Ирвайне.
- Boaz Barak. *Introduction to Theoretical Computer Science*. Черновик учебника, последняя редакция — июнь 2019 года. Это не учебник теоретической информатики времён вашего дедушки — и тем лучше; бесплатный доступ к нему — приятный бонус.
- Thomas Cormen, Charles Leiserson, Ron Rivest, Cliff Stein. *Introduction to Algorithms*, третье издание. MIT Press/McGraw-Hill, 2009. Первым изданием я пользовался, когда был ассистентом преподавателя в Беркли.
- Sanjoy Dasgupta, Christos H. Papadimitriou, Umesh V. Vazirani. *Algorithms*. McGraw-Hill, 2006. Вероятно, это книга, ближайшая к моей по содержанию, но она гораздо менее многословна.
- Jeff Edmonds. *How to Think about Algorithms*. Cambridge University Press, 2008.
- Michael R. Garey, David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- Michael T. Goodrich, Roberto Tamassia. *Algorithm Design: Foundations, Analysis, and Internet Examples*. John Wiley & Sons, 2002.
- Jon Kleinberg, Éva Tardos. *Algorithm Design*. Addison-Wesley, 2005. Если сможете, возьмите её в библиотеке.
- Donald Knuth. *The Art of Computer Programming*, тома 1–4A. Addison-Wesley, 1997 и 2011. Родители подарили мне первые три тома на Рождество, когда мне было 14 лет. Увы, прочёл я их гораздо позже.
- Udi Manber. *Introduction to Algorithms: A Creative Approach*. Addison-Wesley, 1989. Я пользовался этим учебником, когда был ассистентом преподавателя в Беркли.
- Ian Parberry. *Problems on Algorithms*. Prentice-Hall, 1995; издание распродано. Книгу можно скачать с https://larc.unt.edu/ian/books/free/license.html, согласившись сделать небольшое благотворительное пожертвование. Пожалуйста, выполните своё обещание.
- Robert Sedgewick, Kevin Wayne. *Algorithms*. Addison-Wesley, 2011.
- Robert Endre Tarjan. *Data Structures and Network Algorithms*. SIAM, 1983.

- Конспекты моих собственных занятий по алгоритмам в Беркли, особенно курсов Дика Карпа и Раймунда Зайделя.
- Лекционные конспекты, слайды, домашние задания, экзамены, видеолекции, научные статьи, записи блогов, вопросы и ответы на StackExchange, подкасты и полноценные массовые открытые онлайн-курсы, бесплатно выложенные в интернете бесчисленными коллегами со всего мира.

Об упражнениях

Каждая глава заканчивается несколькими упражнениями; большинство из них я хотя бы раз использовал в домашнем задании, на семинаре, лабораторном занятии или экзамене. Упражнения расположены не по возрастанию сложности, а обычно сгруппированы по общим приёмам или темам. Некоторые задачи отмечены символами:

- ♥ Красные сердечки обозначают особенно сложные задачи; многие из них предлагались на квалификационных экзаменах аспирантам Иллинойсского университета. Небольшое число действительно трудных задач отмечено большими сердечками.
- ♦ Синие бубны отмечают задачи, для которых нужны знания из последующих глав, но которые тематически относятся к данному месту. Задачи, требующие знаний предыдущего материала, не отмечаются: книга, как и жизнь, устроена накопительно.
- ♣ Зелёные трефы обозначают редкие задачи, требующие знаний за пределами книги, например конечных автоматов, линейной алгебры, теории вероятностей или планарных графов.
- ♠ Чёрные пики обозначают редкие задачи, требующие значительного объёма кропотливой работы или программирования.
- Оранжевые звёзды указывают, что вы едите хлопья Lucky Charms, произведённые до 1998 года. Фу.

Упражнения задуманы как возможность попрактиковаться, а не как самоцель. Задача состоит не в том, чтобы решить именно это упражнение, а в том, чтобы развить определённый набор навыков или потренироваться решать определённый тип задач. Отчасти поэтому я не привожу решения: они не главное. В частности, «пособия для преподавателя» нет; если вы сами не можете решить задачу, вероятно, не стоит задавать её студентам. Тем не менее решения домашних заданий, которые я задаю в текущем семестре, скорее всего можно найти на странице соответствующего курса. И ничто не мешает вам написать пособие для преподавателя!

Украдите эту книгу!

Книга опубликована на условиях лицензии Creative Commons, позволяющей использовать, распространять, перерабатывать и комбинировать её содержимое без моего разрешения при условии указания оригинального источника. Полная электронная версия бесплатно доступна по следующим адресам:

- Сайт книги: http://jeffe.cs.illinois.edu/teaching/algorithms/.

- Короткий запоминающийся адрес: http://algorithms.wtf.
- Сайт сообщений об ошибках: https://github.com/jeffgerickson/algorithms.
- Internet Archive: https://archive.org/details/Algorithms-Jeff-Erickson.

На сайте книги также находятся сотни страниц дополнительных лекционных конспектов по смежным и более сложным темам, а также почти полный архив прежних домашних заданий, экзаменов, задач для семинаров и лабораторных и других учебных материалов. Каждый раз, когда я веду курс алгоритмов, я исправляю и обновляю материалы, а иногда удаляю часть из них, поэтому на странице текущего курса вы можете найти более свежие редакции.

Неважно, студент вы или преподаватель: вы можете использовать в своих занятиях любую часть этого учебника или других моих конспектов без разрешения — именно поэтому я выложил их в интернет! Однако, пожалуйста, указывайте эту книгу по названию или давайте ссылку на http://algorithms.wtf. Это особенно важно, если вы студент и используете материалы курса при выполнении домашнего задания. Пожалуйста, также уточните правила у своего преподавателя.

Если вы преподаватель, я настоятельно рекомендую дополнять эти материалы тем, что напишете сами. Это укрепит ваше понимание и улучшит изложение на занятиях, а значит, поможет и студентам. Кроме того, вам будет проще справиться с теми частями книги, которые вам не нравятся. Все учебники несовершенны, и этот не исключение.

Наконец, пожалуйста, выкладывайте написанные вами материалы в открытом интернете так, чтобы они были бесплатны, легко доступны и открыты всему миру, а не спрятаны за стенами системы управления обучением или платного доступа. Тогда студенты и преподаватели в других местах тоже смогут воспользоваться вашим уникальным взглядом. Если вы создадите полезные материалы, непосредственно дополняющие этот учебник, например слайды, видео или пособия с решениями, сообщите мне, чтобы я добавил ссылки на них на сайт книги.

Благодарности

Эта книга во многом опирается на вклад бесчисленных студентов, преподавателей и исследователей алгоритмов. В частности, я бесконечно благодарен более чем трём тысячам студентов Иллинойсского университета, которые пользовались моими конспектами как основным источником, высказывали полезную, хотя порой болезненную, критику и стойко переносили некоторые по-настоящему ужасные ранние черновики. Спасибо также многочисленным коллегам и студентам со всего мира, которые использовали эти конспекты на своих занятиях и присылали полезные отзывы и сообщения об ошибках.

Я особенно благодарен за отзывы и вклад, прежде всего упражнения, моим замечательным ассистентам: Aditya Ramani, Akash Gautam, Alex Steiger, Alina Ene, Amir Nayyeri, Asha Seetharam, Ashish Vulimiri, Ben Moseley, Brad Sturt, Brian Ensink, Chao Xu, Charlie Carlson, Chris Neihengen, Connor Clark, Dan Bullok, Dan Cranston, Daniel Khashabi, David Morrison, Ekta Manaktala, Erin Wolf Chambers, Gail Steitz, Gio Kao, Grant Czajkowski, Hsien-Chih Chang, Igor Gammer, Jacob Laurel, John Lee, Johnathon Fischer, Junqing Deng, Kent Quanrud, Kevin Milans, Kevin Small, Konstantinos Koiliaris, Kyle Fox, Kyle Jao, Lan Chen, Mark Idleman, Michael Bond, Mitch Harris, Naveen Arivazhagen, Nick Bachmair, Nick Hurlburt, Nirman Kumar, Nitish Korula, Patrick Lin,

Phillip Shih, Rachit Agarwal, Reza Zamani-Nasab, Rishi Talreja, Rob McCann, Sahand Mozaffari, Shalan Naqvi, Shripad Thite, Spencer Gordon, Srihita Vatsavaya, Subhro Roy, Tana Wattanawaroon, Umang Mathur, Vipul Goyal, Yasu Furakawa и Yipu Wang.

Мне также очень помогли многочисленные беседы с коллегами-преподавателями в Иллинойсе: Alexandra Kolla, Cinda Heeren, Edgar Ramos, Herbert Edelsbrunner, Jason Zych, Kim Whittlesey, Lenny Pitt, Madhu Parasarathy, Mahesh Viswanathan, Margaret Fleck, Shang-Hua Teng, Steve LaValle и особенно Chandra Chekuri, Ed Reingold и Sariel Har-Peled.

Разумеется, книга многим обязана людям, которые впервые научили меня алгоритмам: Bob Bixby и Michael Pearlman в Райсе; David Eppstein, Dan Hirschberg и George Lueker в Ирвайне; Abhiram Ranade, Dick Karp, Manuel Blum, Mike Luby и Raimund Seidel в Беркли.

Первоначальную общую структуру курса и саму идею писать собственные лекционные конспекты я украл у Herbert Edelsbrunner; идею превратить часть конспектов в книгу — у Steve LaValle; а некоторые элементы оформления книги — у Robert Ghrist.

Читатель, берегись!

Разумеется, никто из этих людей не должен отвечать за недостатки получившейся книги. Несмотря на многократные исправления и редактирование, в книге встречаются ошибки, сбои, ляпы, пропуски, неразбериха, костыли, опечатки, ошибки в математике, грамматике и мышлении, провалы памяти, неудачные проектные решения, исторические неточности, анахронизмы, противоречия, преувеличения, нерешительность, болтовня, искажения, чрезмерные упрощения, повторения, словесный понос, бессмыслица, хлам, мусор и откровенная ложь. Во всём этом целиком виноват Steve Skiena.

Я веду список замечаний по адресу https://github.com/jeffgerickson/algorithms, где читатели вроде вас могут сообщать об ошибках, предлагать новые возможности и оставлять общие отзывы о книге. Пожалуйста, сообщайте мне о любых ошибках — математических, грамматических, исторических, типографских, культурных или иных, — найденных как в основном тексте или упражнениях, так и в других моих учебных материалах. Стиву, скорее всего, всё равно. Разумеется, приветствуются и любые другие отзывы!

Приятного чтения!

— Джефф

> *По традиции автор великодушно принимает вину за все оставшиеся недостатки. Я этого делать не стану. Любые ошибки, недостатки и проблемы этой книги — чья-то чужая вина, но я был бы признателен, если бы вы сообщили мне о них, чтобы установить, кто именно виноват.*
>
> *— Steven S. Skiena, The Algorithm Design Manual (1997)*
>
> *Несомненно, за этим последует аннотированный список всех учебников и объяснения, почему каждый из них — дрянь.*
>
> *— Adam Contini, MetaFilter, 4 января 2010 года*

Глава 0. Введение

Здесь начинается алгоритм. Настоящее искусство алгоритма называется так потому, что в нём мы пользуемся десятью индийскими знаками: 0, 9, 8, 7, 6, 5, 4, 3, 2, 1.

— брат Александр из Вильдьё, Carmen de Algorismo (около 1220 года)

Вы правы, требуя от художника сознательного отношения к своему делу, но смешиваете два понятия: решение вопроса и правильную постановку вопроса.

— Антон Чехов, из письма А. С. Суворину (27 октября 1888 года)

Чем больше мы уподобляем себя машинам в низших делах, тем больше сил освобождаем для высших.

— Анна К. Брэкетт, The Technique of Rest (1892)

И вот я в половине третьего ночи пишу о технике, несмотря на твёрдое убеждение, что как только человек начинает говорить о технике, это доказывает, что у него только что кончились идеи.

— Рэймонд Чандлер, письмо Эрлу Стэнли Гарднеру (5 мая 1939 года)

Хорошим людям правила не нужны. Сегодня не тот день, когда стоит выяснять, почему у меня их так много.

— Доктор [Мэтт Смит], «Хороший человек идёт на войну», Доктор Кто (2011)

0.1. Что такое алгоритм?

Алгоритм — это явно заданная, точная, однозначная и механически выполнимая последовательность элементарных инструкций, обычно предназначенная для достижения определённой цели. Например, вот алгоритм исполнения надоедливой песни «99 бутылок пива на стене» для произвольного значения вместо 99:

```
BottlesOfBeer(n):
    Для i от n до 1 в порядке убывания
        Спеть «i бутылок пива на стене, i бутылок пива»
        Спеть «Одну возьми и передай, i – 1 бутылок пива на стене»
    Спеть «Нет бутылок пива на стене, нет бутылок пива»
    Спеть «В магазин сходи, ещё купи, n бутылок пива на стене»
```

Слово «алгоритм» происходит не от греческих корней *arithmos* (ἀριθμός), означающего «число», и *algos* (ἄλγος), означающего «боль», как могли бы предположить испытывающие страх перед алгоритмами специалисты по античности. Это искажённое имя персидского учёного IX века Мухаммада ибн Мусы аль-Хорезми.[1]

Аль-Хорезми, возможно, наиболее известен как автор трактата *Аль-китаб аль-мухтасар фи хисаб аль-джабр ва-ль-мукабала*,[2] от названия которого произошло современное слово «алгебра». В другом трактате он описал современную десятичную систему записи чисел и действий с ними — в частности, использование небольшого кружка, или *сифра*, для обозначения отсутствующего разряда. Эта система была разработана в Индии несколькими столетиями ранее. Методы, описанные во втором трактате и использующие записанные цифры или счётные камешки, стали известны по-английски как *algorism* или *augrym*, а сами знаки — как *ciphers*.

Хотя и позиционная запись, и работы аль-Хорезми уже были известны некоторым европейским учёным, «индо-арабскую» систему счисления популяризировал в Европе средневековый итальянский математик и торговец Леонардо Пизанский, более известный как Фибоначчи. Отчасти благодаря его книге *Liber Abaci*[3] 1202 года в XIII веке записанные цифры начали вытеснять счётную доску, называвшуюся тогда абаком, и счёт на пальцах[4] в качестве предпочтительного способа вычислений[5] в Европе. Причина заключалась не в том, что письменным десятичным вычислениям легче учиться или их проще выполнять, а в том, что они оставляли след для проверки. Цифры распространились в Западной Европе лишь с появлением книгопечатания, а повсеместными стали после того, как в начале XIX века дешёвая бумага стала общедоступной.

Со временем слово *algorism* превратилось в современное *algorithm* под влиянием народной этимологии от греческого *arithmos* и, возможно, упомянутого ранее *algos*.[6] Таким образом, до самого недавнего времени слово «алгоритм» обозначало исключительно механические приёмы позиционной арифметики с использованием «арабских» цифр. Людей, обученных быстрому и надёжному выполнению этих процедур, называли *algorists* или *computators*, либо проще — *computers*, то есть вычислителями.

[1] «Мухаммад, отец Абдаллы, сын Мусы, хорезмиец». Хорезм — древний город, ныне называемый Хивой и находящийся в Хорезмской области Узбекистана.

[2] «Краткая книга об исчислении путём восполнения и противопоставления».

[3] Хотя название *Liber Abaci* заманчиво перевести как «Книга абака», точнее будет «Книга вычислений». И до Фибоначчи, и после него итальянским словом *abaco* обозначали всё, что связано с численными вычислениями: устройства, методы, школы, книги и так далее, примерно как сегодня в английском используется выражение *computer science* или как китайское обозначение исследования операций буквально переводится «изучение использования счётных палочек».

[4] Счёт с помощью пальцев и цифр! В английском *digit* означает и то и другое.

[5] Английское слово *calculate* происходит от латинского *calculus* — «маленький камень» и связано с камешками на счётной доске, которые Чосер называл *augrym stones*. Около 440 года до н. э. Геродот писал в «Истории»: «Греки пишут и считают слева направо; египтяне — наоборот. И всё же они говорят, что их способ письма направлен вправо, а греческий — влево». Геродот странным образом умалчивает, с какого конца египтяне ели яйца.

[6] Некоторые средневековые источники утверждали, что греческая приставка «algo-» означает «искусство» или «введение». Другие приписывали изобретение алгоритмов греческому философу, индийскому или испанскому царю по имени Алгус, Алгор или Аргус. Некоторые авторы, возможно включая Данте Алигьери, даже отождествляли изобретателя с мифическим греческим кораблестроителем, давшим имя аргонавтам. Неясно, задумывались ли эти смехотворные утверждения как исторически точные или лишь как мнемонические приёмы.

0.2. Умножение

Хотя алгоритмы стали предметом систематического научного изучения лишь несколько десятилетий назад, они сопровождают нас с самого зарождения цивилизации. Описания пошаговых арифметических вычислений относятся к древнейшим образцам человеческой письменности: они намного старше трудов Фибоначчи и аль-Хорезми и даже позиционной записи чисел, которую те популяризировали.

Умножение решёткой

Самый знакомый способ умножения больших чисел, по крайней мере американским школьникам, — алгоритм умножения решёткой. Фибоначчи популяризировал его в «Книге абака», узнав из арабских источников, в том числе от аль-Хорезми. Тот, в свою очередь, познакомился с ним по индийским источникам, включая трактат Брахмагупты VII века «Брахмаспхута-сиддханта», который, возможно, опирался на китайские источники. Древнейшие сохранившиеся описания этого алгоритма встречаются в китайском «Математическом трактате Сунь-цзы», написанном между III и V веками, и в комментариях Евтокия Аскалонского к трактату Архимеда «Измерение круга», написанных около 500 года н. э. Но есть свидетельства, что алгоритм был известен значительно раньше. Евтокий приписывает этот способ утраченному трактату Аполлония Пергского, жившего около 300 года до н. э., под названием «Окитокион» (Ὠκυτόκιον).[7] Шумеры записывали таблицы умножения на глиняных табличках ещё около 2600 года до н. э.; возможно, они пользовались и умножением решёткой.[8]

Алгоритм предполагает, что входные числа заданы явными последовательностями цифр. Здесь мы будем использовать десятичную систему, хотя алгоритм непосредственно обобщается на любую другую. Для простоты обозначений[9] входом служит пара массивов X[0 .. m − 1] и Y[0 .. n − 1], представляющих числа

$$x = \sum_{i=0}^{m-1} X[i] \cdot 10^i, \qquad y = \sum_{j=0}^{n-1} Y[j] \cdot 10^j.$$

Аналогично, результатом служит массив Z[0 .. m + n − 1], представляющий произведение

$$z = x \cdot y = \sum_{k=0}^{m+n-1} Z[k] \cdot 10^k.$$

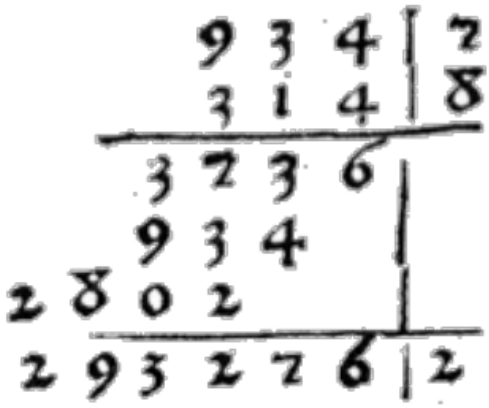
Алгоритм использует сложение и умножение однозначных чисел как элементарные операции. Сложение можно выполнить простым циклом for. На практике для умножения однозначных чисел используют таблицу: вырезанную на глиняных табличках, нарисованную на деревянных или бамбуковых полосках, записанную на бумаге, сохранённую в постоянной памяти или выученную вычислителем наизусть. Весь алгоритм можно выразить формулой

$$x \cdot y = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} \left(X[i] \cdot Y[j] \cdot 10^{i+j} \right).$$

Разные варианты алгоритма вычисляют частичные произведения X[i] · Y[j] · 10^{i+j} в разном порядке и по-разному их складывают. Например, в «Книге абака» Фибоначчи описывает вариант, рассматривающий mn частичных произведений в порядке возрастания разрядов. На современном псевдокоде он выглядит так:

```
FibonacciMultiply(X[0 .. m − 1], Y[0 .. n − 1]):
    hold ← 0
    for k ← 0 to n + m − 1
        for all i and j such that i + j = k
            hold ← hold + X[i] · Y[j]
        Z[k] ← hold mod 10
        hold ← ⌊hold/10⌋
    return Z[0 .. m + n − 1]
```

Алгоритм Фибоначчи часто выполняют, записывая все частичные произведения в двумерную таблицу, или «решётку», а затем складывая числа по диагоналям с соответствующими переносами, как показано справа на рис. 0.1. Американских школьников учат умножать один множитель (множимое) на каждую цифру другого (множителя), записывая все полученные произведения перед сложением, как показано слева на рис. 0.1. Именно этот способ описал и Евтокий, хотя он, соответственно, рассматривал цифры множителя слева направо, как показано на рис. 0.2. Оба варианта и несколько других описаны и проиллюстрированы рядом друг с другом в анонимном учебнике 1458 года «L'Arte dell'Abbaco», известном также как «Тревизская арифметика», — первой печатной математической книге на Западе.



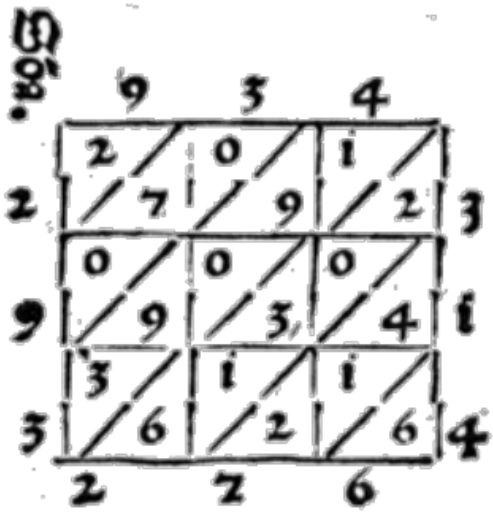


Рис. 0.1. Вычисление 934 × 314 = 293276 умножением «столбиком» (с проверкой по девяткам) и «решёткой» из L'Arte dell'Abbaco (1458). См. сведения об иллюстрациях в конце книги.

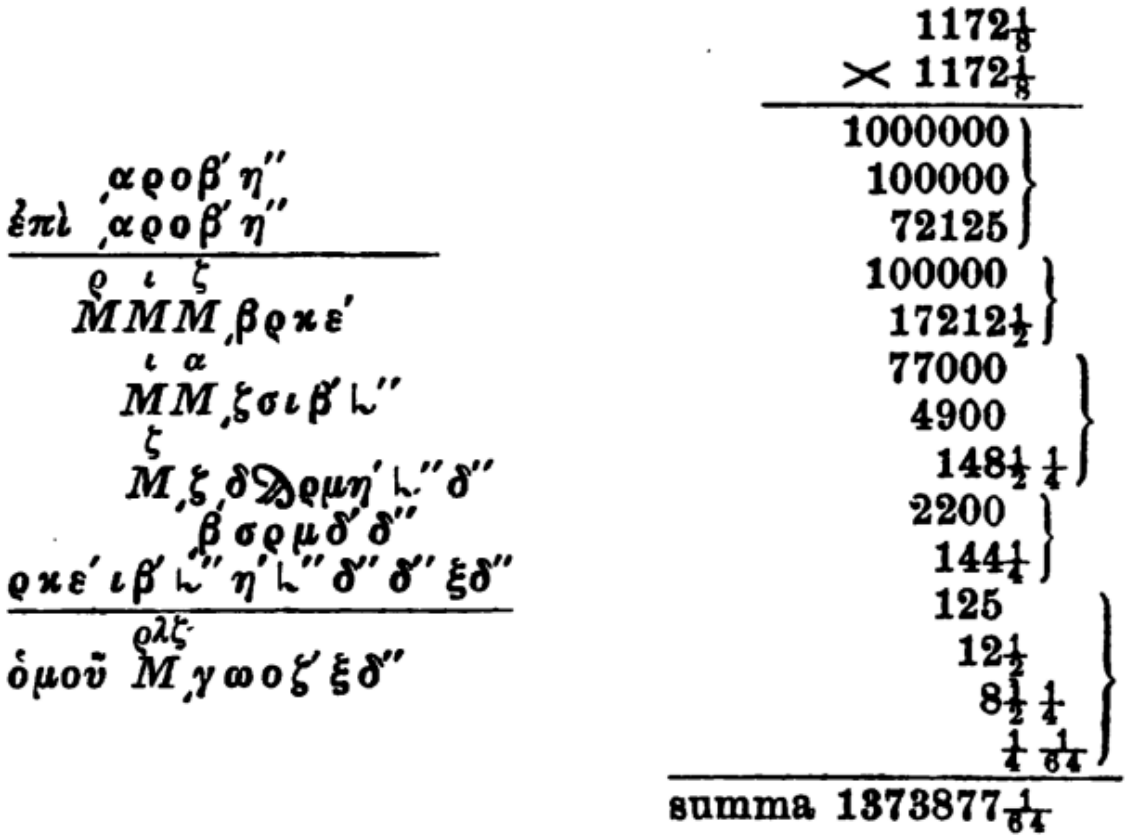


Рис. 0.2. Вычисление Евтокием в VI веке 1172⅛ × 1172⅛ = 1373877 ¹⁄₆₄ в его комментариях к «Измерению круга» Архимеда, переписанное (слева) и переведённое на современные обозначения (справа) Йоханом Хейбергом (1891). См. сведения об иллюстрациях в конце книги.

Все эти варианты умножения решёткой, а также сходные варианты, описанные Сунь-цзы, аль-Хорезми, Фибоначчи, в «Тревизской арифметике» и многих других источниках, вычисляют произведение любого m-значного и любого n-значного числа за время O(mn): время работы каждого варианта определяется числом умножений однозначных чисел.

Удвоение и деление пополам

Умножение решёткой — не древнейший алгоритм умножения, о котором сохранились прямые письменные свидетельства. Ещё более старый и, возможно, более простой алгоритм, не зависящий от позиционной записи чисел, иногда называют русским крестьянским умножением, эфиопским крестьянским умножением или просто крестьянским умножением. Один из его вариантов был переписан египетским писцом Ахмесом в папирус Ринда около 1650 года до н. э. из документа, которому, по его утверждению, было тогда около 350 лет.[10] Этот алгоритм ещё преподавали в начальных школах Восточной Европы в конце XX века; его также часто использовали первые цифровые компьютеры, в которых целочисленное умножение не было реализовано непосредственно аппаратно.

Алгоритм сводит сложную задачу умножения произвольных чисел к последовательности четырёх более простых операций: (1) определению чётности; (2) сложению; (3) удвоению числа и (4) делению числа пополам с округлением вниз.

```
PeasantMultiply(x, y):
    prod ← 0
    while x > 0
        if x is odd
            prod ← prod + y
        x ← ⌊x/2⌋
        y ← y + y
    return prod
```

x	y	prod
		0
123	+ 456	= 456
61	+ 912	= 1368
30	1824	
15	+ 3648	= 5016
7	+ 7296	= 12312
3	+ 14592	= 26904
1	+ 29184	= 56088

Рис. 0.3. Умножение удвоением и делением пополам.

Корректность алгоритма следует по индукции из следующего рекурсивного тождества, справедливого для всех неотрицательных целых x и y:

$$x \cdot y = \begin{cases} 0 & \text{если } x = 0; \\ \lfloor x/2 \rfloor \cdot (y + y) & \text{если } x \text{ чётно}; \\ \lfloor x/2 \rfloor \cdot (y + y) + y & \text{если } x \text{ нечётно}. \end{cases}$$

Можно сказать, что это рекуррентное соотношение и есть алгоритм крестьянского умножения. Не дайте итеративному псевдокоду ввести вас в заблуждение: по своей сути алгоритм рекурсивен!

В приведённом виде PeasantMultiply выполняет $O(\log x)$ операций проверки чётности, сложения и деления пополам. Однако эту оценку можно улучшить до $O(\log \min\{x, y\})$, поменяв аргументы местами, если $x > y$. Если числа представлены в любой разумной позиционной системе (например, двоичной, десятичной, вавилонской шестидесятеричной, египетской двенадцатеричной, римскими цифрами, китайскими счётными палочками, положениями косточек на абаке и так далее), каждая операция требует не более $O(\log(xy)) = O(\log \max\{x, y\})$ операций над отдельными цифрами. Поэтому общее время работы равно $O(\log \min\{x, y\} \cdot \log \max\{x, y\}) = O(\log x \cdot \log y)$.

Иными словами, для перемножения m-значного и n-значного чисел требуется O(mn) времени; с точностью до постоянного множителя это та же оценка, что и для умножения решёткой. При выполнении вручную этот алгоритм требует больше записей — на постоянный множитель! — но необходимые элементарные операции человеку, возможно, выполнять проще. Более того, при двоичной записи чисел оба алгоритма эквивалентны.

Циркуль и линейка

Классические греческие геометры отождествляли числа, а точнее величины, с отрезками соответствующей длины и работали с ними при помощи двух простых механических инструментов — циркуля и линейки. Их разновидности уже столетиями широко использовали землемеры, архитекторы и другие мастера. Применяя только эти два инструмента и начиная с одной или нескольких заданных опорных точек, учёные сводили сложные геометрические построения к следующим элементарным операциям:

- провести единственную прямую через две различные заданные точки;
- провести единственную окружность с центром в одной заданной точке через другую заданную точку;
- определить точку пересечения двух прямых, если она существует;
- определить точки пересечения прямой и окружности, если они существуют;
- определить точки пересечения двух окружностей, если они существуют.

На практике ученики греческих геометров почти наверняка выполняли построения на абаке (ἄβαξ) — столе, покрытом пылью или песком.[11] За столетия до этого египетские землемеры выполняли многие из тех же построений, используя верёвки для задания прямых и окружностей на земле.[12] Однако Евклид и другие греческие геометры рассматривали построения циркулем и линейкой как точные математические абстракции: точки — идеальные точки, прямые — идеальные прямые, окружности — идеальные окружности.

На рис. 0.4 показан алгоритм умножения или деления двух величин, описанный в «Началах» Евклида около 2500 лет назад. На вход подаются четыре различные точки A, B, C и D. Нужно построить точку Z такую, что |AZ| = |AC||AD|/|AB|. В частности, если принять |AB| за единицу длины, алгоритм вычислит произведение |AC| и |AD|.

```
⟨⟨Построить прямую, перпендикулярную ℓ и проходящую через P.⟩⟩
RightAngle(ℓ, P):
    Choose a point A ∈ ℓ
    A, B ← Intersect(Circle(P, A), ℓ)
    C, D ← Intersect(Circle(A, B), Circle(B, A))
    return Line(C, D)

⟨⟨Построить точку Z такую, что |AZ| = |AC||AD|/|AB|.⟩⟩
MultiplyOrDivide(A, B, C, D):
    α ← RightAngle(Line(A, C), A)
    E ← Intersect(Circle(A, B), α)
    F ← Intersect(Circle(A, D), α)
    β ← RightAngle(Line(E, C), F)
    γ ← RightAngle(β, F)
    return Intersect(γ, Line(A, C))
```

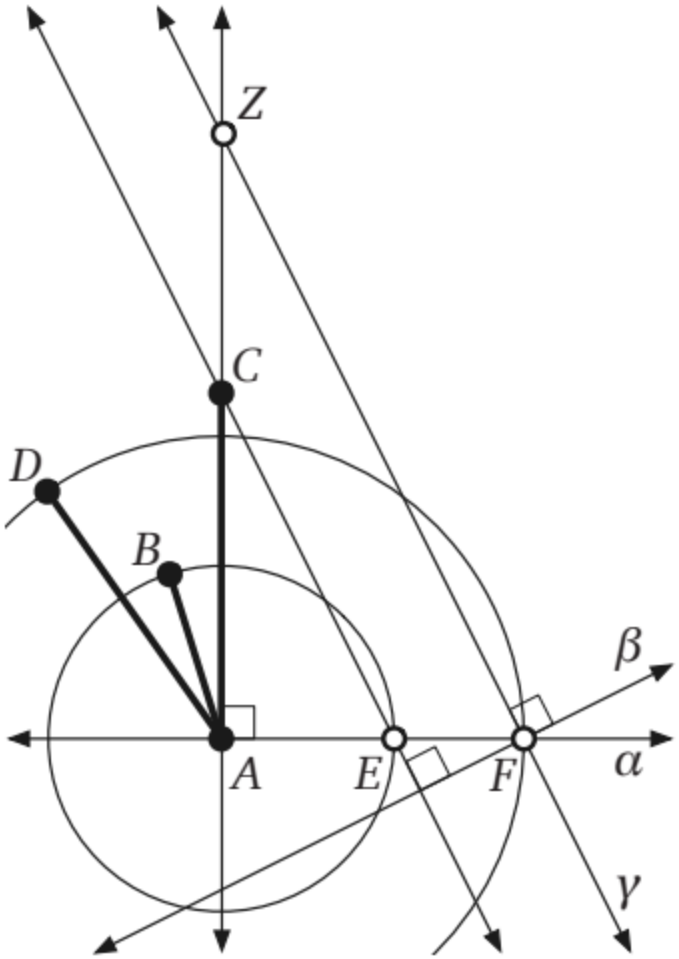


Рис. 0.4. Умножение циркулем и линейкой.

Заметим, что Евклид сначала определяет новую элементарную операцию RightAngle, написав, как выразились бы современные программисты, подпрограмму. Корректность алгоритма следует из подобия треугольников ACE и AZF. Вторая и третья строки основного алгоритма неоднозначны: прямая α пересекает любую окружность с центром A в двух различных точках. Однако алгоритм работает правильно независимо от того, какие из точек пересечения выбраны в качестве E и F.

Алгоритм Евклида сводит умножение двух величин — длин — к последовательности элементарных операций с циркулем и линейкой. На современном цифровом компьютере эти операции трудно выполнить точно, но алгоритм Евклида и не предназначался для цифрового компьютера. Он был создан для платоновского идеального геометра, вооружённого платоновскими идеальными циркулем и линейкой и способного по определению точно выполнять каждую операцию за постоянное время. В этой модели вычислений MultiplyOrDivide работает за время O(1)!

[7] Буквально — «лекарство, способствующее быстрым и лёгким родам»! Папп Александрийский воспроизвёл несколько отрывков «Окитокиона» примерно за 200 лет до Евтокия, но его описание алгоритма умножения решёткой, если оно вообще существовало, тоже утрачено.

[8] Существует множество свидетельств того, что древние шумеры точно вычисляли с исключительно большими числами в своей шестидесятеричной позиционной системе, однако автору неизвестны сохранившиеся записи непосредственно использовавшихся ими методов. Кроме обычных таблиц умножения и обратных величин, были обнаружены таблицы квадратов целых чисел от 1 до 59. Это заставило некоторых историков математики предположить, что вавилоняне умножали, пользуясь тождеством $xy = ((x + y)^2 - x^2 - y^2)/2$. Но этот приём работает, только когда $x + y < 60$; история умалчивает, как вавилоняне могли вычислять x^2 при $x \geq 60$.

[9] Хотя это грозит разжечь историческую вражду между Грецией и Египтом, Лилипутией и Блефуску, компьютерами Mac и PC или теми, кто считает ноль натуральным числом, и теми, кто ошибается.

[10] Вариант алгоритма, действительно применявшийся в Древнем Египте, не использует деление пополам или проверку чётности, зато использует сравнения. Чтобы избежать деления пополам, заранее составляют две таблицы последовательным удвоением: в одной находятся все степени двойки, не превышающие x, в другой — те же степени двойки, умноженные на y. Затем степени двойки, сумма которых равна x, находят жадным вычитанием, а соответствующие элементы второй таблицы складывают, получая произведение.

[11] Записанные цифры от 1 до 9 были известны в Европе как минимум за два столетия до «Книги абака» Фибоначчи под названием «цифры гобар» — от арабского слова ghubār, «пыль», в конечном счёте связанного с индийским обычаем вычислять на столах, покрытых песком. От греческого слова ἄβαξ произошло латинское abacus, также первоначально означавшее стол с песком.

[12] Помните, что означает слово «геометрия»? Позднее Демокрит называл этих египетских землемеров несколько пренебрежительно арпедонаптами (ἀρπεδονάπται), то есть «натягивателями верёвок».

0.3. Распределение мест в Конгрессе

Вот ещё один пример алгоритма из реальной жизни, имеющего существенное политическое значение. Статья I, раздел 2 Конституции США требует:

> *Представители и прямые налоги распределяются между штатами, которые могут войти в этот Союз, сообразно их населению. […] Число представителей не должно превышать одного на каждые тридцать тысяч человек, но каждый штат должен иметь по крайней мере одного представителя. […]*

Поскольку число мест в Палате представителей конечно, точное пропорциональное представительство требовало бы общих для нескольких штатов или дробных представителей, ни то ни другое не допускается законом. Поэтому за последующие десятилетия было предложено и применялось множество алгоритмов, призванных справедливо округлить идеальное дробное решение. Действующий алгоритм, называемый методом Хантингтона — Хилла, или методом равных пропорций, впервые предложил статистик Бюро переписи Джозеф Хилл в 1911 году, усовершенствовал гарвардский математик Эдвард Хантингтон в 1920 году, а в

1941 году он был закреплён в федеральном законе (2 U.S.C. §2a) и в 1992 году выдержал проверку Верховным судом.[13]

Метод Хантингтона — Хилла распределяет представителей между штатами по одному. Сначала на предварительном этапе каждому штату выделяется по одному представителю. Затем на каждой итерации основного цикла следующий представитель назначается штату с наивысшим приоритетом. Приоритет штата определяется как $P/\sqrt{(r(r + 1))}$, где P — население штата, а r — число уже выделенных ему представителей.

Псевдокод алгоритма приведён на рис. 0.5. Вход состоит из массива Pop[1 .. n], содержащего численность населения n штатов, и целого числа R — общего числа представителей; предполагается, что R ≥ n. (В настоящее время в США n = 50 и R = 435.) Выходной массив Rep[1 .. n] содержит число представителей, выделенных каждому штату.

```
ApportionCongress(Pop[1 .. n], R):
    PQ ← NewPriorityQueue
    ⟨⟨Дать каждому штату первого представителя⟩⟩
    for s ← 1 to n
        Rep[s] ← 1
        Insert(PQ, s, Pop[i]/√2)
    ⟨⟨Распределить оставшихся n – R представителей⟩⟩
    for i ← 1 to n – R
        s ← ExtractMax(PQ)
        Rep[s] ← Rep[s] + 1
        priority ← Pop[s]/√(Rep[s] (Rep[s] + 1))
        Insert(PQ, s, priority)
    return Rep[1 .. n]
```

Рис. 0.5. Алгоритм распределения Хантингтона — Хилла.

Примечание переводчика. В приведённом в оригинале псевдокоде, по-видимому, есть опечатки: при начальной вставке в очередь следует использовать Pop[s], а количество оставшихся представителей равно R – n, а не n – R. Здесь строки сохранены в виде оригинала; при реализации нужно учесть эти исправления.

Эта реализация метода Хантингтона — Хилла использует очередь с приоритетами, поддерживающую операции NewPriorityQueue, Insert и ExtractMax. (Сам закон, разумеется, ничего не говорит об очередях с приоритетами.) Результат алгоритма, а значит, и его корректность совершенно не зависят от реализации этой очереди. Бюро переписи использует отсортированный массив, хранящийся в одном столбце электронной таблицы Excel, который на каждой итерации пересчитывается заново. На университетском курсе структур данных вы, вероятно, изучили более эффективную реализацию.

Подобные алгоритмы распределения мест применяются на многопартийных парламентских выборах по всему миру: количество мест каждой партии должно быть пропорционально числу полученных ею голосов. Два наиболее распространённых метода — метод Д'Ондта[14] и метод Вебстера — Сент-Лагю,[15] которые используют приоритеты соответственно $P/(r + 1)$ и $P/(2r + 1)$ вместо выражения с квадратным корнем в методе Хантингтона — Хилла. Последний практически уникален для Палаты представителей США, отчасти из-за конституционного требования, чтобы каждый штат получал хотя бы одного представителя.

[13] Отменив решение федерального окружного суда, Верховный суд единогласно постановил, что любой метод распределения мест, добросовестно принятый Конгрессом, соответствует Конституции (Министерство торговли США против Монтаны). Текущее распределение подробно описано на сайте Бюро переписи США: http://www.census.gov/topics/public-sector/congressional-apportionment.html. История задачи изложена на http://www.thirty-thousand.org/pages/Apportionment.htm; доклад Исследовательской службы Конгресса о различных методах доступен по адресу http://www.fas.org/sgp/crs/misc/R41382.pdf.

[14] Метод разработал Томас Джефферсон в 1792 году; он использовался для распределения мест в Конгрессе США с 1792 по 1832 год, был заново открыт бельгийским математиком Виктором Д'Ондтом в 1878 году и усовершенствован швейцарским физиком Эдуардом Хагенбахом-Бишофом в 1888 году.

[15] Метод разработал Дэниел Вебстер в 1832 году; он использовался для распределения мест в Конгрессе США с 1842 по 1911 год, был заново открыт французским математиком Андре Сент-Лагю в 1910 году и ещё раз открыт немецким физиком Хансом Шеперсом в 1980 году.

0.4. Плохой пример

В качестве типичного примера последовательности инструкций, которая на самом деле не является алгоритмом, рассмотрим «алгоритм Мартина»:[16]

```
BeAMillionaireAndNeverPayTaxes():
    Получите миллион долларов.
    Если налоговый инспектор придёт к вам и скажет:
        «Вы никогда не платили налогов!»
    Ответьте: «Я забыл».
```

Довольно просто, если не считать первого шага: вот уж задача! Руководители компаний-миллиардеров, венчурные инвесторы Кремниевой долины или нью-йоркские дельцы от недвижимости могли бы посчитать это алгоритмом: для них первый шаг и однозначен, и тривиален.[17] Но для остальных бедолаг процедура Мартина слишком расплывчата, чтобы считаться настоящим алгоритмом. С другой стороны, это прекрасный пример сведения: задача стать миллионером и никогда не платить налоги сводится к «более простой» задаче получения миллиона долларов. В этой книге мы снова и снова будем встречать сведения. Как показали сотни бизнесменов и политиков, если вы умеете решать более простую задачу, сведение подскажет, как решить более сложную.

Алгоритм Мартина, как и некоторые предыдущие примеры, отличается от привычных специалистам по информатике алгоритмов тем, что сформулирован через операции, которые компьютеру трудно выполнить. Эта книга посвящена почти исключительно алгоритмам, которые разумно реализовать на обычном цифровом компьютере. Каждый их шаг либо непосредственно поддерживается распространёнными языками программирования — например, арифметика, присваивания, циклы или рекурсия, — либо представляет собой то, что вы уже научились делать: сортировку, двоичный поиск, обход дерева или исполнение песни «n бутылок пива на стене».

[16] Стив Мартин, «You Can Be A Millionaire», Saturday Night Live, 21 января 1978 года. Также вошло в Comedy Is Not Pretty, Warner Bros. Records, 1979.

[17] Что-то там безопасный квантовый блокчейн глубокого обучения и тому подобное.

0.5. Описание алгоритмов

Навыки эффективного проектирования и анализа алгоритмов тесно связаны с умением их описывать. По крайней мере на моих занятиях полное описание алгоритма состоит из четырёх компонентов:

- **Что:** точная постановка задачи, которую решает алгоритм.
- **Как:** точное описание самого алгоритма.
- **Почему:** доказательство того, что алгоритм решает поставленную задачу.
- **Насколько быстро:** анализ времени работы алгоритма.

Разрабатывать эти компоненты именно в таком порядке необязательно и даже нежелательно. Постановка задачи, описание алгоритма, доказательство корректности и оценка времени обычно развиваются одновременно, причём работа над каждым из компонентов влияет на остальные. Например, нам может потребоваться уточнить постановку, чтобы построить более быстрый алгоритм, или изменить алгоритм, чтобы учесть сложный случай в доказательстве корректности. Тем не менее для читателя обычно яснее представить эти компоненты по отдельности.

Как и при любом письме, важно учитывать аудиторию. Я рекомендую писать для компетентного, но скептически настроенного программиста, который не так сообразителен, как вы. Вспомните себя полгода назад. Разрабатывая новый алгоритм, вы постепенно вырабатываете интуитивное понимание задачи и того, как алгоритм её решает; именно эта интуиция направляет ваши неформальные рассуждения. Но человек, который позже прочтёт ваш алгоритм или написанный по нему код, не будет обладать вашей интуицией и опытом. Как и ваш компилятор. Как и вы сами через полгода. Всё, что у них будет, — ваше письменное описание.

Даже если вам никогда не понадобится объяснять алгоритмы кому-либо ещё, при их разработке важно помнить об аудитории. Стремление понятно излагать заставляет яснее мыслить. В частности, текст для начинающего читателя, который будет понимать ваши слова буквально, вынуждает продумать тонкости, какими бы «очевидными» или «интуитивными» ни казались общие идеи. А текст для скептически настроенной аудитории заставляет строить надёжные доказательства корректности и эффективности вместо того, чтобы полагаться на интуицию или собственный ум.[18]

Не устану повторять: главная задача разработчика алгоритмов — научить других людей понимать, как и почему работают его алгоритмы. Если вы не можете донести свои идеи до других людей, считайте, что их не существует. Создание правильного и эффективного исполняемого кода — важная, но второстепенная цель. Стремление убедить себя,

преподавателей, нынешних или будущих работодателей, коллег либо студентов в собственном уме занимает в лучшем случае далёкое третье место.

Постановка задачи

Прежде чем приступить к разработке алгоритма, нужно договориться, какую задачу он должен решать. Точно так же, прежде чем описывать алгоритм, необходимо описать решаемую им задачу.

Алгоритмические задачи часто излагаются обычным языком через объекты реального мира. Наша работа как разработчиков алгоритмов — переформулировать их в терминах формальных абстрактных математических объектов: чисел, массивов, списков, графов, деревьев и так далее, — о которых можно рассуждать строго. Нужно также выявить скрытые предположения в условии и явно их сформулировать. Например, в песне «n бутылок пива на стене» число n всегда является неотрицательным целым.[19]

По мере разработки алгоритма нам, возможно, придётся уточнять постановку. Например, алгоритм может требовать определённого представления входных данных или выдавать результат в определённом представлении, не указанном в исходном неформальном описании. Или же он может решать более общую задачу, чем та, которую нам первоначально поставили. Это часто встречается у рекурсивных алгоритмов.

Спецификация должна содержать ровно столько подробностей, чтобы другой человек мог использовать алгоритм как чёрный ящик, не зная, как и почему он работает. В частности, необходимо описать тип и смысл каждого входного параметра и точно указать, как результат зависит от входных данных. Напротив, детали, не нужные для применения алгоритма как чёрного ящика, следует намеренно скрыть. Пусть то, что не имеет значения, действительно останется в стороне.

Например, умножение решёткой и умножение удвоением и делением пополам решают одну задачу: по двум неотрицательным целым x и y, каждое из которых представлено массивом цифр, вычислить их произведение $x \cdot y$, также представленное массивом цифр. Для пользователя выбор алгоритма совершенно несущественен. Напротив, греческий алгоритм с циркулем и линейкой решает другую задачу, поскольку вход и выход представлены отрезками, а не массивами цифр.

Описание алгоритма

Компьютерные программы — конкретные представления алгоритмов, но алгоритмы не являются программами. Это абстрактные механические процедуры, которые можно реализовать на любом языке программирования, поддерживающем необходимые элементарные операции. Своеобразные синтаксические детали вашего любимого языка совершенно несущественны: сосредоточившись на них, вы лишь отвлечёте себя и читателей от сути.[20] Хорошее описание алгоритма ближе к комментариям, которые следовало бы написать в реальной программе, чем к самому коду. Код плохо подходит для повествования.

С другой стороны, описание обычным текстом тоже обычно не лучший выбор. Алгоритмы содержат много характерных структур — особенно условия, циклы, вызовы функций и рекурсию, — которые слишком легко скрыть в неструктурированном тексте. Разговорный язык полон неоднозначностей и смысловых оттенков, но алгоритмы нужно описывать как можно более однозначно. Проза плохо подходит для точного изложения.

По моему мнению, яснее всего представлять алгоритм сочетанием псевдокода и структурированного текста. Псевдокод использует структуру формальных языков программирования и математики, разбивая алгоритм на элементарные шаги. Сами шаги могут быть записаны математическими обозначениями, обычным языком или подходящим сочетанием того и другого — в зависимости от того, что понятнее. Хороший псевдокод раскрывает внутреннюю структуру алгоритма, но скрывает несущественные детали реализации, благодаря чему алгоритм легче понять, проанализировать, отладить и реализовать.

Всякий раз описание алгоритма должно включать все детали, необходимые для полного задания алгоритма, доказательства его корректности и анализа времени работы. И в то же время следует исключить детали, которые для этого не нужны. На более практическом уровне описание должно позволять компетентному, но скептически настроенному программисту, не читавшему эту книгу, быстро и правильно реализовать алгоритм на любимом языке программирования, не понимая, почему он работает.

Я не хочу утомлять вас правилами, которым следую при написании псевдокода, но обязан предостеречь от одной особенно вредной привычки. Никогда не описывайте повторяющиеся действия неформально, например: «Сначала сделайте это, затем то и так далее» или «Повторяйте процесс, пока не произойдёт нечто». Каждый, кому приходилось решать раздражающие тесты «Что будет следующим в этой последовательности?», знает: описание первых шагов почти ничего не говорит о том, что происходит дальше. Если в алгоритме есть цикл, запишите его как цикл и явно опишите произвольную итерацию. Аналогично, если алгоритм рекурсивен, запишите его рекурсивно, явно указав границы случаев и происходящее в каждом из них.

[18] В частности, я предполагаю, что вы — скептически настроенный новичок!

[19] Я никогда не слышал, чтобы кто-нибудь пел «$\sqrt{2}$ бутылок пива на стене». Порой я слышал, как специалисты по теории множеств поют «$\aleph_0$ бутылок пива на стене», но они почему-то всегда сдавались, не допев песню.

[20] Это, конечно, вопрос религиозных убеждений. Диванные лингвисты бесконечно спорят о гипотезе Сепира — Уорфа, согласно которой люди мыслят только категориями, заданными их языком. В крайней формулировке некоторые понятия одного языка просто недоступны носителям другого не только из-за технологического прогресса — как перевести на арамейский «jump the shark» или «стример Fortnite»? — но и из-за присущих языкам и культурам структурных различий. Более скептическую точку зрения см. в книге Стивена Пинкера «Язык как инстинкт». При применении к разным парадигмам программирования в этой идее, надо признать, есть доля истины. Что такое Y-комбинатор? Как работают шаблоны? Что такое абстрактная фабрика? К счастью, эти различия слишком тонки, чтобы влиять на материал

книги. Убедительный контрпример см. в монографии Криса Окасаки Functional Data Structures и более поздних работах, выросших из неё.

0.6. Анализ алгоритмов

Недостаточно просто записать алгоритм и сказать: «Любуйтесь!» Нужно ещё убедить аудиторию — и себя самих! — что алгоритм действительно делает то, что должен, причём делает это эффективно.

Корректность

В некоторых прикладных задачах допустимо, чтобы программы работали правильно большую часть времени на всех «разумных» входных данных. Но не в этой книге: нам нужны алгоритмы, которые всегда дают правильный результат для любых возможных входных данных. Более того, их корректность необходимо доказывать: полагаться на интуицию или проверить несколько примеров недостаточно. Иногда корректность действительно очевидна, особенно для алгоритмов, которые вы встречали на предыдущих курсах. Но, с другой стороны, слово «очевидно» слишком часто оказывается синонимом слова «неверно». Для доказательства корректности большинства алгоритмов этого курса приходится всерьёз потрудиться. В частности, такие доказательства обычно опираются на индукцию. Нам нравится индукция. Индукция — наш друг.[21]

Конечно, прежде чем формально доказать, что алгоритм делает то, что должен, нужно формально описать, что именно он должен делать!

[21] Если индукция вам не друг, с этой книгой вам придётся нелегко.

Время работы

Самый распространённый способ сравнить алгоритмы решения одной задачи — выяснить, как быстро они работают. В идеале для каждой задачи нужен самый быстрый алгоритм. В некоторых прикладных задачах допустимо, чтобы программы работали эффективно большую часть времени на всех «разумных» входных данных. Но не в этой книге: нам нужны алгоритмы, которые всегда работают эффективно, даже в худшем случае.

Но как измерить время работы? Например, сколько времени понадобится, чтобы спеть песню `BottlesOfBeer(n)`? Очевидно, это зависит от входного значения n, но также и от скорости пения. Один певец может потратить на куплет десять секунд, другой — двадцать. Технологии расширяют возможности ещё больше. Передача песни по телеграфу азбукой Морзе может занять целую минуту на куплет. Загрузка MP3 из интернета может занять десятую долю секунды на куплет. Копирование MP3 в оперативной памяти компьютера может занять всего несколько микросекунд на куплет.

Здесь важно, как время исполнения меняется с ростом n. Чтобы спеть `BottlesOfBeer(2n)`, потребуется примерно вдвое больше времени, чем для `BottlesOfBeer(n)`, независимо от используемой технологии. Это отражает асимптотическая оценка времени пения $\Theta(n)$.

Время можно измерять, подсчитав, сколько раз алгоритм выполняет определённую инструкцию или достигает определённого этапа в «коде». Например, слово «пиво» поётся трижды в каждом куплете `BottlesOfBeer`, так что количество его повторений хорошо характеризует общее время пения. На этот вопрос можно дать точный ответ: в `BottlesOfBeer(n)` пиво упоминается ровно $3n + 3$ раза.

Кстати, существует множество песен с квадратичным временем исполнения. Следующая, вероятно, знакома большинству англоговорящих:

```
NDaysOfChristmas(gifts[2 .. n]):
    для i ← 1 до n
        Спеть «В i-й день Рождества моя любовь подарила мне»
        для j ← i до 2 в обратном порядке
            Спеть «j gifts[j],»
        если i > 1
            Спеть «и»
        Спеть «куропатку на грушевом дереве»
```

На вход `NDaysOfChristmas` подаётся список из $n - 1$ подарков, представленный здесь массивом. Нетрудно показать, что время пения равно $\Theta(n^2)$; в частности, певец упоминает название подарка $\sum_{i=1}^{n} i = n(n+1)/2$ раз, считая куропатку на грушевом дереве. Также легко увидеть, что за первые n дней Рождества моя любовь подарила мне ровно $\sum_{i=1}^{n} \sum_{j=1}^{i} j = n(n+1)(n+2)/6 = \Theta(n^3)$ подарков.

К другим песням с квадратичным временем относятся «Old MacDonald Had a Farm», «There Was an Old Lady Who Swallowed a Fly», «Hole in the Bottom of the Sea», «Green Grow the Rushes O», «The Rattlin' Bog», «The Court Of King Caractacus», «The Barley-Mow», «If I Were Not Upon the Stage», «Star Trekkin'», «Ist das nicht ein Schnitzelbank?»,[22] «Il Pulcino Pio», «Minkurinn í hænsnakofanum», «Echad Mi Yodea» и «Το κοκοράκι». За другими примерами обратитесь к своему любимому дошкольнику.

```
Alouette(lapart[1 .. n]):
    Спеть «Alouette, gentille alouette, alouette, je te plumerai.»
    для i от 1 до n
        Спеть «Je te plumerai lapart[i]. Je te plumerai lapart[i].»
        для j от i до 1 ⟨⟨в обратном порядке⟩⟩
            Спеть «Et lapart[j] ! Et lapart[j] !»
        Спеть «Alouette! Alouette! Aaaaaa...»
        Спеть «... alouette, gentille allouette, alouette, je te plumerai.»
```

У некоторых песен время исполнения ещё причудливее. Довольно современный пример — «The TELNET Song» Гая Стила: чтобы спеть первые n куплетов, требуется $\Theta(2^n)$ времени; Стил рекомендовал $n = 4$. Наконец, есть и песни, которые никогда не заканчиваются.[23]

За исключением «The TELNET Song», все эти песни естественнее всего выразить небольшим набором вложенных циклов, поэтому время их исполнения можно вычислить с помощью вложенных сумм. Время работы рекурсивного алгоритма проще выразить рекуррентным соотношением. Например, крестьянское умножение можно записать рекурсивно:

$$x \cdot y = \begin{cases} 0 & \text{если } x = 0, \\ \lfloor x/2 \rfloor \cdot (y + y) & \text{если } x \text{ чётно}, \\ \lfloor x/2 \rfloor \cdot (y + y) + y & \text{если } x \text{ нечётно}. \end{cases}$$

Пусть $T(x, y)$ — число операций проверки чётности, сложения и деления пополам, необходимых для вычисления $x \cdot y$. Эта функция удовлетворяет рекуррентному неравенству $T(x, y) \leq T(\lfloor x/2 \rfloor, 2y) + 2$ с базовым случаем $T(0, y) = 0$. Методы следующей главы дают верхнюю оценку $T(x, y) = O(\log x)$.

Иногда время работы зависит от конкретной реализации используемой структуры данных или подпрограммы. Например, алгоритм Хантингтона — Хилла `ApportionCongress` работает за $O(N + RI + (R - n)E)$, где N — время работы `NewPriorityQueue`, I — время работы `Insert`, а E — время работы `ExtractMax`. При разумном предположении $R \geq 2n$ (в среднем на каждый штат приходится не менее двух представителей) эта оценка упрощается до $O(N + R(I + E))$.

Точное время зависит от реализации очереди с приоритетом. Бюро переписи реализует её как неотсортированный массив, что даёт $N = I = \Theta(1)$ и $E = \Theta(n)$; следовательно, реализация `ApportionCongress` Бюро переписи работает за $O(Rn)$. Если же реализовать очередь с приоритетом с помощью двоичной кучи или массива со свойством кучи, получаем $N = \Theta(1)$ и $I = E = O(\log n)$, поэтому общий алгоритм работает за $O(R \log n)$.

Наконец, иногда нас интересуют вычислительные ресурсы, отличные от времени: например, память, количество подбрасываний монеты, промахов кеша или страничных ошибок, число сообщений между процессами или число подарков, которые мне преподнесла моя любовь. Эти ресурсы анализируются теми же методами, что и время работы. Например, решёточное умножение двух n-значных чисел требует $O(n^2)$ памяти, если записать все частичные произведения до сложения, но лишь $O(n)$ памяти, если складывать их по мере получения.

[22] Да, это Отто фон Шницельпусскранкенгешайтмайер!

[23] Они просто продолжаются и продолжаются, мой друг.

Упражнения

0. Опишите и проанализируйте эффективный алгоритм, который по допустимому расположению стандартных фигур на стандартной шахматной доске определяет, кто победит из данной позиции, если оба игрока играют идеально. [Подсказка: существует тривиальное решение в одну строку!]

♥ 1.

(a) Найдите или сочините песню, исполнение первых n куплетов которой требует времени $\Theta(n^3)$.

(b) Найдите или сочините песню, исполнение первых n куплетов которой требует времени $\Theta(n \log n)$.

(c) Найдите или сочините песню, исполнение первых n куплетов которой требует какого-нибудь другого необычного количества времени.

2. Внимательные читатели могут возразить, что наш анализ песен вроде «n бутылок пива на стене» или «n дней Рождества» чрезмерно упрощён: большие числа петь дольше, чем маленькие. В более общем случае, поскольку количество слов заданной длины ограничено, в больших множествах слов обязательно встречаются более длинные слова.[24] Время пения можно оценить точнее, подсчитывая слоги, а не слова.

(a) Сколько времени нужно, чтобы пропеть целое число n?

(b) Сколько времени нужно, чтобы спеть «n бутылок пива на стене»?

(c) Сколько времени нужно, чтобы спеть «n дней Рождества»?

Как обычно, выразите ответы в форме $O(f(n))$ для некоторой функции f.

[24] Да, это машина для исследования природных свойств ускорения субатомных частиц!

3. Кумулятивная застольная песня «The Barley Mow» веками исполнялась на Британских островах. У неё много вариантов; на рисунке 0.6 приведён псевдотекст одного из вариантов, традиционных для Девона и Корнуолла. Здесь `vessel[i]` — название сосуда, вмещающего 2^i унций пива.[25]

```
BarleyMow(n):
    «Выпьем за ячменный скирд, мои славные ребята,»
    «Выпьем за ячменный скирд!»
    «Осушим весёлую коричневую чашу,»
    «Выпьем за ячменный скирд!»
    «Выпьем за ячменный скирд, мои славные ребята,»
    «Выпьем за ячменный скирд!»
    для i ← 1 до n
        «Осушим сосуд vessel[i], ребята,»
        «Выпьем за ячменный скирд!»
        для j ← i до 1 в обратном порядке
            «Сосуд vessel[j],»
        «И весёлую коричневую чашу!»
        «Выпьем за ячменный скирд!»
        «Выпьем за ячменный скирд, мои славные ребята,»
        «Выпьем за ячменный скирд!»
```

Рисунок 0.6. «The Barley Mow».

(a) Предположим, что каждое название `vessel[i]` состоит из одного слова и вы можете петь четыре слова в секунду. Сколько времени потребуется, чтобы спеть `BarleyMow(n)`? Дайте точную асимптотическую оценку.

(b) Если вы хотите петь эту песню при сколь угодно больших n, придётся придумывать собственные названия сосудов. Чтобы названия не повторялись, с ростом n они должны становиться всё длиннее. Предположим, в `vessel[n]` содержится $\Theta(\log n)$ слогов, а вы можете петь шесть слогов в секунду. Сколько теперь времени потребуется для `BarleyMow(n)`? Дайте точную асимптотическую оценку.

(c) Предположим, что всякий раз при упоминании сосуда вы действительно выпиваете соответствующее количество пива: одну унцию за весёлую коричневую чашу и 2^i унций за каждый `vessel[i]`. Считая в целях задачи, что вам не меньше 21 года, определите, сколько именно унций пива вы выпьете, исполнив `BarleyMow(n)`. Дайте точный ответ, а не просто асимптотическую оценку.

[25] На практике песня использует некоторое подмножество следующих сосудов: nipperkin, quarter-gill, half-a-gill, gill, quarter-pint, half-a-pint, pint, quart, pottle, gallon, half-anker, anker, firkin, half-barrel/kilderkin, barrel, hogshead, pipe/butt, tun, колодец, река и океан. За несколькими исключениями, особенно ближе к концу, объём каждого сосуда в списке вдвое больше предыдущего. В ирландских и шотландских версиях текст немного отличается; после галлона в них обычно переходят к людям: буфетчице, хозяину заведения, возчику и так далее.

Ранняя версия песни под названием «Give us once a drink» встречается в пьесе Джона Марстона *Jack Drum's Entertainment* (или *The Comedie of Pasquill and Katherine*), написанной около 1600 года: «Giue vs once a drinke for and the black bole. Sing gentle Butler bally moy!» Существуют разногласия, написал ли Марстон эту «верхненемецкую песню» специально для пьесы, является ли «bally moy» ослышкой вместо «barley mow» или наоборот и действительно ли это одна и та же песня. Такие вопросы лучше обсуждать за n бутылками пива.

4. Напомним, что на вход алгоритму Хантингтона — Хилла `ApportionCongress` подаются массив `Pop[1 .. n]`, где `Pop[i]` — население i-го штата, и целое число R — общее количество распределяемых мест представителей. Результат — массив `Rep[1 .. n]`, где `Rep[i]` — число мест, выделенных алгоритмом i-му штату.

Алгоритм Хантингтона — Хилла иногда описывают так, чтобы вообще обойтись без очереди с приоритетом. Алгоритм верхнего уровня «угадывает» положительное действительное число D, называемое делителем, а затем запускает следующую подпрограмму для вычисления распределения. Переменная q обозначает идеальную квоту мест штата при заданном D; фактическое число мест всегда равно либо $\lceil q \rceil$, либо $\lfloor q \rfloor$.

```
HHGuess(Pop[1 .. n], R, D):
    reps ← 0
    для i ← 1 до n
        q ← Pop[i] / D
        если q · q < ⌈q⌉ · ⌊q⌋
            Rep[i] ← ⌊q⌋
        иначе
            Rep[i] ← ⌈q⌉
        reps ← reps + Rep[i]
    вернуть reps
```

Возможны три варианта конечного значения `reps`. Если `reps < R`, мы распределили недостаточно мест, а это, по крайней мере интуитивно, означает, что делитель D был слишком мал. Если `reps > R`, мы распределили слишком много мест, а значит, по крайней мере интуитивно, делитель D был слишком велик. Наконец, если `reps = R`, можно вернуть массив `Rep[1 .. n]` как окончательное распределение. На практике допустимое распределение с `reps =`

`R` можно вычислить, вызвав `HHGuess` с небольшим количеством целочисленных делителей, близких к стандартному $D = P/R$.

> *Примечание переводчика: в рассуждении автора о направлении изменения делителя, по-видимому, перепутаны «слишком мал» и «слишком велик»: уменьшение положительного D увеличивает квоты $q = \mathrm{Pop}[i]/D$, а увеличение D их уменьшает.*

В следующих задачах пусть $P = \sum_{i=1}^{n} \mathrm{Pop}[i]$ — общая численность населения всех n штатов, и предположим, что $n \le R \le P$.

(a) Покажите, что вызов `HHGuess` со стандартным делителем $D = P/R$ не обязательно даёт допустимое распределение.

(b) Докажите, что если `HHGuess` возвращает одно и то же значение `reps` для двух разных делителей D и D', то для них вычисляется и одно и то же распределение `Rep[1 .. n]`.

(c) Докажите, что если `HHGuess` возвращает правильное значение R, то вычисленное распределение `Rep[1 .. n]` совпадает с результатом приведённого ранее алгоритма `ApportionCongress`.

(d) Докажите, что «правильный» делитель D существует не всегда! То есть укажите такие входные данные `Pop[1 .. n]` и R, где $n \le R \le P$, что ни для одного действительного $D > 0$ число мест, распределённых `HHGuess`, не равно R. [Подсказка: что произойдёт, если в четвёртой строке `HHGuess` заменить `<` на `≤`?]

Глава 1. Рекурсия

Управление многочисленной армией строится на том же принципе, что и управление горсткой людей: дело лишь в разделении их числа.

— Сунь-цзы, «Искусство войны» (около 400 г. н. э.), перевод Лайонела Джайлса (1910)

Наша жизнь растрачивается на мелочи... Упрощайте, упрощайте.

— Генри Дэвид Торо, «Уолден» (1854)

Не спрашивайте, что такое Вум. Я и сам никогда не узнаю. Но, скажу вам, ребята, снег он убирает ОТЛИЧНО!

— Доктор Сьюз [Теодор Сьюз Гейзел], The Cat in the Hat Comes Back (1958)

Сначала выполняйте трудные дела. С лёгкими они справятся сами.

— приписывается Дейлу Карнеги

1.1. Сведения

Сведение — самый распространённый приём построения алгоритмов. Свести задачу X к другой задаче Y — значит написать алгоритм для X, который использует алгоритм решения Y как чёрный ящик или подпрограмму. Принципиально важно, чтобы корректность полученного алгоритма для X никоим образом не зависела от того, как работает алгоритм для Y. Единственное, что мы вправе предполагать, — чёрный ящик правильно решает Y. Его внутреннее устройство нас совершенно не касается; это чужая проблема. Часто лучше всего буквально представлять, что чёрный ящик работает исключительно благодаря волшебству.

Например, описанный в предыдущей главе алгоритм крестьянского умножения сводит задачу умножения двух произвольных положительных целых чисел к трём более простым задачам: сложению, делению пополам и проверке чётности. Алгоритм опирается на абстрактный тип данных «положительное целое число», поддерживающий эти три операции, но корректность алгоритма умножения не зависит ни от точного представления данных (зарубки, глиняные жетоны, вавилонская шестидесятеричная запись, кипу, счётные палочки, римские цифры, положения пальцев, счётные камешки, цифры гобара, двоичная, отрицательная двоичная система, код Грея, уравновешенная троичная, финарная, четверичная мнимая система и т. д.), ни от конкретной реализации этих операций.

Разумеется, время работы алгоритма умножения зависит от времени сложения, деления пополам и проверки чётности, но это отдельный от корректности вопрос. Самое главное: мы можем получить более эффективный алгоритм умножения, просто перейдя к более эффективному представлению чисел — например, от зарубок к позиционной записи.

Аналогично алгоритм Хантингтона—Хилла сводит задачу распределения мест в Конгрессе к поддержанию очереди с приоритетом, которая допускает операции `Insert` и `ExtractMax`. Абстрактный тип данных «очередь с приоритетом» — чёрный ящик: корректность алгоритма распределения не зависит от конкретной структуры данных, реализующей очередь. Конечно, время работы алгоритма распределения зависит от времени выполнения `Insert` и `ExtractMax`, но это отдельный от корректности вопрос. Прелесть сведения в том, что мы можем получить более эффективный алгоритм распределения, просто заменив структуру данных очереди с приоритетом. Более того, разработчику этой структуры незачем знать или заботиться о том, что её будут использовать для распределения мест в Конгрессе.

При разработке алгоритмов мы можем не знать точно, как реализованы используемые нами базовые компоненты или как наши алгоритмы будут применяться в качестве компонентов для решения ещё более крупных задач. Многим новичкам такое незнание доставляет неудобство, но оно одновременно неизбежно и чрезвычайно полезно. Даже если вы точно знаете, как работают ваши компоненты, часто чрезвычайно полезно притвориться, будто вы этого не знаете.

1.2. Упрощайте и делегируйте

Рекурсия — особенно мощный вид сведения, который можно приблизительно описать так:

- Если данный экземпляр задачи можно решить непосредственно, решите его непосредственно.
- Иначе сведите его к одному или нескольким более простым экземплярам той же задачи.

Если самоссылка сбивает с толку, полезно представить, что более простые задачи решит кто-то другой, как вы предполагали бы при других видах сведения. Мне нравится называть этого кого-то Феей Рекурсии. Ваша единственная задача — упростить исходную задачу либо решить её непосредственно, когда упрощение не нужно или невозможно; все более простые подзадачи Фея Рекурсии решит за вас методами, которые вас не касаются, так что не суйте нос не в своё дело.[1] Читатели с математической подготовкой могут узнать Фею Рекурсии под её более формальным именем: предположение индукции.

Чтобы любой рекурсивный метод работал правильно, должно выполняться одно несложное техническое условие: не должно существовать бесконечной последовательности сведений ко всё более простым экземплярам. В конце концов рекурсивные сведения должны привести к элементарному базовому случаю, который можно решить иным методом; иначе рекурсивный алгоритм будет работать бесконечно. Обычно этого добиваются сведением к одному или нескольким меньшим экземплярам той же задачи. Например, если исходный объект — скрибль с n глурпами, то входом каждого рекурсивного вызова должен быть скрибль со строго меньшим числом глурпов. Конечно, это невозможно, если у скрибля совсем нет глурпов: отрицательного числа глурпов быть не может, это было бы глупо! Поэтому в этом случае мы должны отгриндлблофить скрибль каким-нибудь другим способом.

Мы уже встречали пример такой схемы в алгоритме крестьянского умножения, основанном непосредственно на следующем рекурсивном тождестве:

$$x \cdot y = \begin{cases} 0 & \text{если } x = 0, \\ \lfloor x/2 \rfloor \cdot (y + y) & \text{если } x \text{ чётно}, \\ \lfloor x/2 \rfloor \cdot (y + y) + y & \text{если } x \text{ нечётно}. \end{cases}$$

То же рекуррентное соотношение можно выразить алгоритмически:

```
PeasantMultiply(x, y):
    если x = 0
        вернуть 0
    иначе
        x′ ← ⌊x/2⌋
        y′ ← y + y
        prod ← PeasantMultiply(x′, y′)   ⟪Рекурсия!⟫
        если x нечётно
            prod ← prod + y
        вернуть prod
```

Ленивый египетский писец мог бы выполнить этот алгоритм, вычислив x' и y', попросив более младшего писца перемножить x' и y', а затем, возможно, прибавив y к его ответу. Задача младшего писца проще, поскольку $x' < x$, а последовательное уменьшение положительного целого числа в конце концов приводит к нулю. Как именно младший писец вычисляет $x' \cdot y'$, старшего писца не касается (как не касается и вас).

[1] Когда я учился в университете, я приписывал рекурсию не Фее Рекурсии, а «эльфам», имея в виду сказку братьев Гримм о старом сапожнике, который оставляет работу незавершённой, ложась спать, а проснувшись, обнаруживает, что эльфы («Wichtelmänner») за ночь всё закончили. Кто-то более опытный в энтеогенах, чем я, мог бы узнать в этих *Rekursionswichtelmänner* «самопреобразующихся машинных эльфов» Теренса Маккенны.

1.3. Ханойская башня

Головоломку «Ханойская башня» впервые опубликовал — как настоящую физическую головоломку! — французский преподаватель и любитель занимательной математики Эдуар Люка в 1883 году[2] под псевдонимом «N. Claus (de Siam)» (анаграмма имени «Lucas d'Amiens»). В следующем году Анри де Парвиль сопроводил описание головоломки такой замечательной историей:[3]

> *В великом храме Бенареса[4]... под куполом, отмечающим центр мира, покоится медная пластина, в которой закреплены три алмазные иглы, каждая локоть высотой и толщиной с пчелу. На одну из этих игл при сотворении мира Бог поместил шестьдесят четыре диска из чистого золота: самый большой лежит на медной пластине, а остальные становятся всё меньше к вершине. Это башня Брамы. День и ночь жрецы неустанно перекладывают диски с одной алмазной иглы на другую в соответствии с неизменными законами Брамы: дежурный жрец не должен перемещать больше одного диска за раз и должен помещать диск на иглу так, чтобы под ним не оказалось меньшего диска. Когда все шестьдесят четыре диска будут перенесены с иглы, на которую Бог поместил их при*

сотворении мира, на одну из двух других, башня, храм и брамины обратятся в прах, и под раскат грома мир исчезнет.

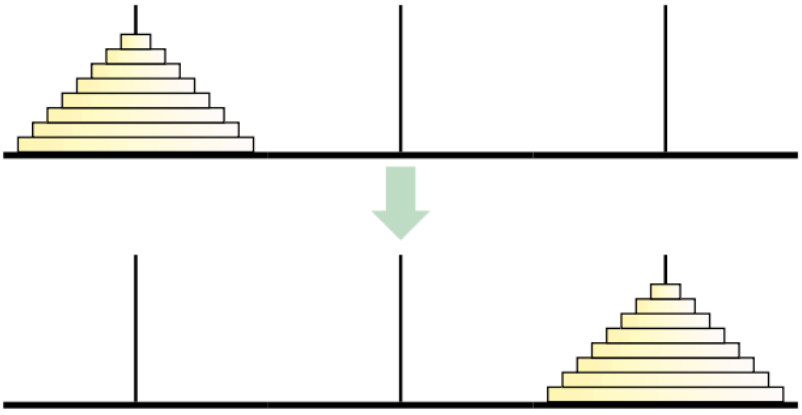


Рисунок 1.1. Головоломка «Ханойская башня» с восемью дисками.

Разумеется, как хорошие специалисты по информатике, прочитав эту историю, мы первым делом подставляем переменную n вместо жёстко заданной константы 64. А поскольку большинство физических экземпляров головоломки сделаны из дерева, а не из алмазов и золота, я буду называть три возможных места для дисков «стержнями», а не «иглами». Как переместить башню из n дисков с одного стержня на другой, используя третий как временное хранилище, и ни разу не положить больший диск на меньший?

Как указал N. Claus (de Siam) в брошюре, прилагавшейся к его головоломке, секрет решения — мыслить рекурсивно. Вместо того чтобы пытаться сразу решить всю задачу, сосредоточимся на перемещении только самого большого диска. Сразу его переместить нельзя: все остальные диски мешают. Поэтому сначала нужно перенести эти n – 1 меньших дисков на запасной стержень. После этого мы можем непосредственно переместить самый большой диск на целевой стержень. Наконец, чтобы закончить, нужно перенести n – 1 меньших дисков с запасного стержня на целевой.

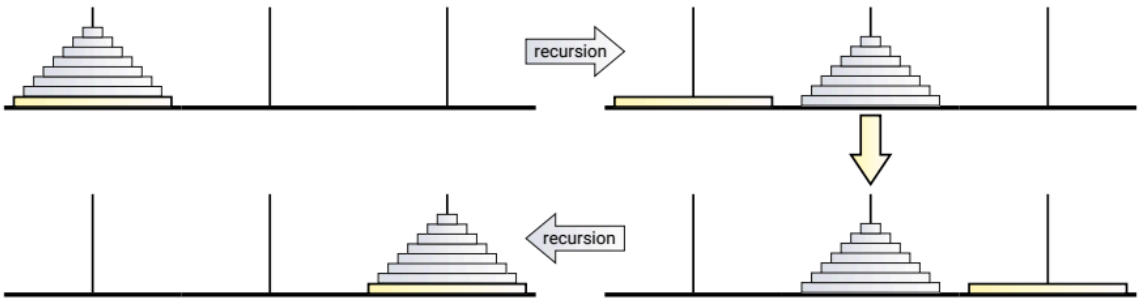


Рисунок 1.2. Алгоритм Ханойской башни; не обращайте внимания ни на что, кроме нижнего диска.

Теперь нам остаётся лишь выяснить, как...

НЕТ!! СТОП!!

Вот и всё! Мы закончили! Мы успешно свели задачу Ханойской башни для n дисков к двум экземплярам задачи для n − 1 дисков, которые с радостью можем передать Фее Рекурсии — или, продолжая метафору Люка, младшим монахам храма. Наша работа окончена. Если бы мы не доверяли младшим монахам, мы бы их не наняли; пусть спокойно делают своё дело.

Однако наше сведение опирается на одно тонкое, но чрезвычайно важное предположение: существует самый большой диск. Рекурсивный алгоритм работает для любого положительного числа дисков, но перестаёт работать при n = 0. Этот случай нужно обработать иначе. К счастью, монахи Бенареса, будучи хорошими буддистами, отлично умеют мгновенно перемещать ноль дисков с одного стержня на другой, ничего не делая.

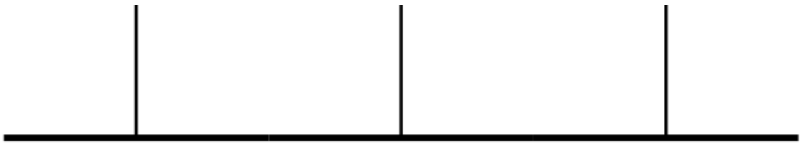


Рисунок 1.3. Пустой базовый случай алгоритма Ханойской башни. Ложки нет.

Может возникнуть соблазн обдумать, как перемещаются все меньшие диски, или, в более общем случае, что происходит при развёртывании рекурсии, — но не делайте этого. Для большинства рекурсивных алгоритмов развёртывать рекурсию не нужно и бесполезно. Наша единственная задача — свести данный экземпляр задачи к одному или нескольким более простым либо решить его непосредственно, если такое сведение невозможно. При n = 0 наш рекурсивный алгоритм Ханойской башни очевидным образом корректен. При любом n ≥ 1 Фея Рекурсии правильно перемещает верхние n − 1 дисков (формальнее, предположение индукции означает, что наш рекурсивный алгоритм правильно перемещает верхние n − 1 дисков), поэтому алгоритм корректен.

На рисунке 1.4 рекурсивный алгоритм Ханойской башни приведён в виде псевдокода. Он переносит стопку из n дисков с исходного стержня `src` на целевой стержень `dst`, используя третий стержень `tmp` для временного хранения. Заметьте: при n = 0 алгоритм правильно не делает ничего.

```
Hanoi(n, src, dst, tmp):
    если n > 0
        Hanoi(n − 1, src, tmp, dst)   ⟪Рекурсия!⟫
        переместить диск n с src на dst
        Hanoi(n − 1, tmp, dst, src)   ⟪Рекурсия!⟫
```

Рисунок 1.4. Рекурсивный алгоритм решения задачи Ханойской башни.

Пусть T(n) — число перемещений, необходимых для переноса n дисков, то есть время работы алгоритма. Из пустого базового случая следует T(0) = 0, а из общего рекурсивного алгоритма — T(n) = 2T(n − 1) + 1 при любом n ≥ 1. Выписав несколько первых значений T(n), легко предположить, что $T(n) = 2^n - 1$; простое доказательство по индукции подтверждает эту догадку. В частности, перенос башни из 64 дисков требует $2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615$ отдельных перемещений. Поэтому даже при впечатляющей скорости одного перемещения в секунду монахи Бенареса будут работать примерно 585 миллиардов лет («plus de cinq milliards

de siècles»), прежде чем башня, храм и брамины обратятся в прах, а мир исчезнет под раскат грома.

[2] Позже Люка утверждал, что изобрёл головоломку в 1876 году.

[3] Английский перевод взят из книги У. У. Роуза Болла *Mathematical Recreations and Essays* 1892 года.

[4] «Великий храм Бенареса» — почти наверняка храм Каши Вишванатх в Варанаси, Уттар-Прадеш, Индия, расположенный приблизительно в 2400 км к западу-северо-западу от Ханоя во Вьетнаме, где якобы жил вымышленный N. Claus. По совпадению французская армия вторглась в Ханой в 1883 году, в тот же год, когда Люка выпустил головоломку; в конечном итоге Ханой стал столицей Французского Индокитая.

1.4. Сортировка слиянием

Сортировка слиянием — один из самых ранних алгоритмов, разработанных для универсальных компьютеров с хранимой программой. Джон фон Нейман разработал его в 1945 году, а в 1947 году подробно описал вместе с Германом Голдстайном как одну из первых нечисловых программ для EDVAC.[5]

1. Разделить входной массив на два подмассива примерно одинакового размера.
2. Рекурсивно отсортировать каждый подмассив слиянием.
3. Слить получившиеся отсортированные подмассивы в единый отсортированный массив.

```
Вход:                 S O R T I N G E X A M P L
Разделение:           S O R T I N | G E X A M P L
Рекурсия слева:       I N O R S T | G E X A M P L
Рекурсия справа:      I N O R S T | A E G L M P X
Слияние:              A E G I L M N O P R S T X
```

Рисунок 1.5. Пример сортировки слиянием.

Первый шаг совершенно тривиален — просто разделим размер массива пополам, — а второй можно поручить Фее Рекурсии. Вся настоящая работа выполняется на заключительном шаге слияния. Полное описание алгоритма приведено на рисунке 1.6. Чтобы рекурсивная структура была понятнее, я вынес слияние в отдельную подпрограмму. Алгоритм слияния тоже рекурсивен: определите первый элемент выходного массива, а затем рекурсивно слейте оставшиеся части входных массивов.

```
MergeSort(A[1 .. n]):
    если n > 1
        m ← ⌊n/2⌋
        MergeSort(A[1 .. m])         ⟪Рекурсия!⟫
        MergeSort(A[m + 1 .. n])     ⟪Рекурсия!⟫
        Merge(A[1 .. n], m)

Merge(A[1 .. n], m):
    i ← 1; j ← m + 1
    для k ← 1 до n
        если j > n
```

```
            B[k] ← A[i]; i ← i + 1
        иначе если i > m
            B[k] ← A[j]; j ← j + 1
        иначе если A[i] < A[j]
            B[k] ← A[i]; i ← i + 1
        иначе
            B[k] ← A[j]; j ← j + 1
    для k ← 1 до n
        A[k] ← B[k]
```

Рисунок 1.6. Сортировка слиянием.

Корректность

Чтобы доказать корректность алгоритма, мы дважды применим нашу старую знакомую индукцию: сначала к подпрограмме `Merge`, затем к основному алгоритму `MergeSort`.

Лемма 1.1. `Merge` правильно сливает подмассивы A[1 .. m] и A[m + 1 .. n], если эти подмассивы во входных данных отсортированы.

Доказательство. Пусть A[1 .. n] — произвольный массив, а m — любое целое число, при котором подмассивы A[1 .. m] и A[m + 1 .. n] отсортированы. Докажем, что для всех k от 0 до n последние n − k − 1 итераций основного цикла правильно сливают A[i .. m] и A[j .. n] в B[k .. n]. Доказательство проводится индукцией по n − k + 1 — числу оставшихся для слияния элементов.

> *Примечание переводчика: в оригинале здесь указано n − k − 1 итераций и диапазон k от 0 до n. При индексации массива с 1 для слияния B[k .. n] требуется n − k + 1 итераций; базовый случай k = n + 1 соответствует пустому отрезку.*

Если k > n, алгоритм правильно сливает два пустых подмассива, не делая совершенно ничего. (Это базовый случай индукционного доказательства.) Иначе для k-й итерации основного цикла нужно рассмотреть четыре случая:

- Если j > n, подмассив A[j .. n] пуст, поэтому min(A[i .. m] ∪ A[j .. n]) = A[i].
- Если i > m, подмассив A[i .. m] пуст, поэтому min(A[i .. m] ∪ A[j .. n]) = A[j].
- Иначе, если A[i] < A[j], то min(A[i .. m] ∪ A[j .. n]) = A[i].
- Иначе A[i] ≥ A[j], и min(A[i .. m] ∪ A[j .. n]) = A[j].

Во всех четырёх случаях B[k] правильно присваивается наименьший элемент A[i .. m] ∪ A[j .. n]. В двух случаях с присваиванием B[k] ← A[i] Фея Рекурсии правильно сливает — простите, я хотел сказать, что предположение индукции означает: последние n − k итераций основного цикла правильно сливают A[i + 1 .. m] и A[j .. n] в B[k + 1 .. n]. Аналогично в двух других случаях Фея Рекурсии тоже правильно сливает оставшиеся части подмассивов. ■

Теорема 1.2. `MergeSort` правильно сортирует любой входной массив A[1 .. n].

Доказательство. Докажем теорему индукцией по n. Если n ≤ 1, алгоритм правильно не делает ничего. Иначе Фея Рекурсии правильно сортирует — простите, предположение индукции означает, что наш алгоритм правильно сортирует два меньших подмассива A[1 .. m] и A[m + 1 .. n], после чего они правильно сливаются в один отсортированный массив по лемме 1.1. ■

Анализ

Поскольку алгоритм `MergeSort` рекурсивен, время его работы естественно выражается рекуррентным соотношением. Очевидно, `Merge` занимает O(n) времени, поскольку это простой цикл с постоянным объёмом работы на итерацию. Сразу получаем для `MergeSort`:

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n).$$

Как и в большинстве рекуррентных соотношений для метода «разделяй и властвуй», знаки округления вверх и вниз можно без опасений отбросить (с помощью преобразования области определения, описанного далее в этой главе). Получим более простое соотношение T(n) = 2T(n/2) + O(n). Случай «все уровни равны» метода дерева рекурсии (также описанного далее) сразу даёт решение T(n) = O(n log n). Даже если вы (пока) не знакомы с деревьями рекурсии, это решение можно проверить индукцией.

[5] На самом деле Голдстайн и фон Нейман описали нерекурсивный вариант, который сейчас обычно называют сортировкой слиянием снизу вверх. В то время большие наборы данных сортировали специализированные машины — почти все производства IBM, — которые обрабатывали перфокарты с помощью вариантов двоичной поразрядной сортировки. Фон Нейман убедительно доказывал: поскольку EDVAC может сортировать быстрее специализированных сортировщиков IBM «без вмешательства человека и необходимости в дополнительном оборудовании», EDVAC является машиной «общего назначения», а специализированные сортировочные машины больше не нужны.

1.5. Быстрая сортировка

Быстрая сортировка — ещё один рекурсивный алгоритм сортировки, открытый Тони Хоаром в 1959 году и впервые опубликованный в 1961 году. В этом алгоритме основная работа заключается в разделении массива на меньшие подмассивы до рекурсии, благодаря чему объединение отсортированных подмассивов становится тривиальным.

1. Выбрать в массиве опорный элемент.
2. Разделить массив на три подмассива, содержащих элементы меньше опорного, сам опорный элемент и элементы больше опорного.
3. Рекурсивно отсортировать первый и последний подмассивы быстрой сортировкой.

```
Вход:                 S O R T I N G E X A M P L
Выбор опорного:       S O R T I N G E X A M P L
Разделение:           A G O E I N L M P T X S R
Рекурсия слева:       A E G I L M N O P T X S R
Рекурсия справа:      A E G I L M N O P R S T X
```

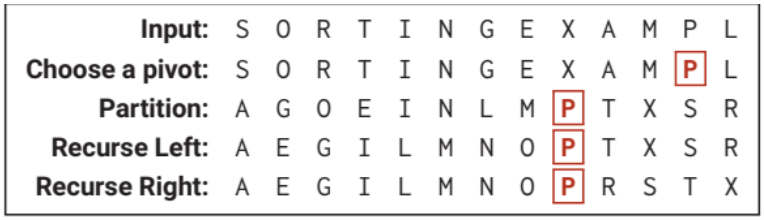


Рисунок 1.7. Пример быстрой сортировки.

Более подробный псевдокод дан на рисунке 1.8. Во входном параметре p подпрограммы `Partition` передаётся индекс опорного элемента в неотсортированном массиве; подпрограмма разделяет массив и возвращает новый индекс опорного элемента. Существует множество эффективных алгоритмов разделения; приведённый здесь приписывается Нико Ломуто.[6] Переменная ℓ считает количество элементов массива, меньших опорного.

```
QuickSort(A[1 .. n]):
    если n > 1
        выбрать опорный элемент A[p]
        r ← Partition(A, p)
        QuickSort(A[1 .. r – 1])   ⟪Рекурсия!⟫
        QuickSort(A[r + 1 .. n])   ⟪Рекурсия!⟫

Partition(A[1 .. n], p):
    обменять A[p] ↔ A[n]
    ℓ ← 0                        ⟪число элементов < опорного⟫
    для i ← 1 до n – 1
        если A[i] < A[n]
            ℓ ← ℓ + 1
            обменять A[ℓ] ↔ A[i]
    обменять A[n] ↔ A[ℓ + 1]
    вернуть ℓ + 1
```

Рисунок 1.8. Быстрая сортировка.

Корректность

Как и для сортировки слиянием, доказательство корректности `QuickSort` требует двух отдельных индукционных доказательств: одно показывает, что `Partition` правильно разделяет массив, а другое — что `QuickSort` правильно сортирует при условии корректности `Partition`. Для доказательства корректности `Partition` нужно доказать следующий инвариант цикла: в конце каждой итерации основного цикла все элементы подмассива A[1 .. ℓ] меньше A[n], а в подмассиве A[ℓ + 1 .. i] нет элементов меньше A[n]. Остальные простые, но утомительные подробности я оставлю читателю в качестве упражнений.

Анализ

Анализ быстрой сортировки тоже похож на анализ сортировки слиянием. Очевидно, `Partition` работает за O(n): это простой цикл с постоянным объёмом работы на итерацию. Для `QuickSort` получаем рекуррентное соотношение, зависящее от r — ранга выбранного опорного элемента:

$$T(n) = T(r-1) + T(n-r) + O(n).$$

Если бы нам удалось каким-то волшебным способом всегда выбирать медиану массива A как опорный элемент, то r = ⌈n/2⌉, размеры двух подзадач были бы максимально близки, а соотношение приняло бы вид

$$T(n) = T(\lceil n/2 \rceil - 1) + T(\lfloor n/2 \rfloor) + O(n) \le 2T(n/2) + O(n),$$

и мы получили бы T(n) = O(n log n), используя либо метод дерева рекурсии, либо ещё более простой метод «ага, мы уже решили это соотношение для сортировки слиянием».

На самом деле, как мы увидим далее в главе, медианный элемент неотсортированного массива действительно можно найти за линейное время, но алгоритм довольно сложен, а скрытая в обозначении O(·) константа достаточно велика, чтобы полученный алгоритм сортировки оказался непрактичным. На практике большинство программистов довольствуются чем-то простым, например выбором первого или последнего элемента массива. В этом случае r может принимать любое значение от 1 до n, поэтому

$$T(n) = \max_{1 \le r \le n} \big(T(r-1) + T(n-r) + O(n)\big).$$

В худшем случае две подзадачи совершенно несбалансированны — r = 1 или r = n, — и соотношение становится T(n) ≤ T(n − 1) + O(n). Его решение: T(n) = O(n²).

Другая распространённая эвристика называется «медиана трёх»: выбирают три элемента, обычно в начале, середине и конце массива, и в качестве опорного берут их медиану. Хотя на практике эта эвристика несколько эффективнее простого выбора одного элемента, особенно если массив уже почти отсортирован, в худшем случае всё ещё возможно r = 2 или r = n − 1. Для неё получаем T(n) ≤ T(1) + T(n − 2) + O(n), и решение по-прежнему T(n) = O(n²).

Интуитивно опорный элемент должен «обычно» оказываться где-то посередине массива, скажем, с рангом от n/10 до 9n/10. Отсюда кажется, что время работы «в среднем случае» должно быть O(n log n). Хотя эту интуицию можно формализовать, наиболее распространённая формализация исходит из совершенно нереалистичного предположения, что все перестановки входного массива равновероятны. Данные реального мира могут быть случайными, но не таким образом, который мы можем предсказать заранее, и уж точно не равномерно![7]

Иногда люди по какой-то причине рассматривают также время работы «в лучшем случае». Мы не будем.

[6] Хоар предложил более сложный алгоритм «двустороннего» разделения, у которого есть некоторые практические преимущества перед алгоритмом Ломуто. С другой стороны, алгоритм разделения Хоара — одно из мест, где погибли ошибки на единицу.

[7] С другой стороны, если выбирать индекс опорного элемента p равномерно случайным образом, быстрая сортировка работает за O(n log n) с высокой вероятностью для любого возможного входного массива. Ключевое различие в том, что случайностью управляет наш алгоритм, а не всемогущий злонамеренный противник, который даёт входные данные после прочтения нашего кода. К сожалению, анализ рандомизированной быстрой сортировки выходит за рамки этой книги, но соответствующие конспекты можно найти на http://algorithms.wtf/.

1.6. Общая схема

И сортировка слиянием, и быстрая сортировка следуют общей трёхшаговой схеме «разделяй и властвуй»:

1. Разделить данный экземпляр задачи на несколько независимых меньших экземпляров той же самой задачи.
2. Поручить каждый меньший экземпляр Фее Рекурсии.
3. Объединить решения меньших экземпляров в окончательное решение данного экземпляра.

Если размер какого-либо экземпляра становится меньше некоторого постоянного порога, мы отказываемся от рекурсии и решаем задачу непосредственно, полным перебором, за постоянное время.

Доказательство корректности алгоритма «разделяй и властвуй» почти всегда требует индукции. Для анализа времени работы необходимо составить и решить рекуррентное соотношение, что обычно, хотя, к сожалению, не всегда, можно сделать с помощью деревьев рекурсии.

1.7. Деревья рекурсии

Так что же это за «деревья рекурсии», о которых я всё время говорю? Это простой, общий и наглядный инструмент для решения рекуррентных соотношений метода «разделяй и властвуй». Дерево рекурсии — корневое дерево, в котором каждому рекурсивному экземпляру подзадачи соответствует один узел. Значение узла — время, затраченное на соответствующую подзадачу без учёта рекурсивных вызовов. Таким образом, общее время работы алгоритма равно сумме значений всех узлов дерева.

Чтобы сделать эту идею конкретнее, представьте алгоритм «разделяй и властвуй», который тратит O(f(n)) времени на нерекурсивную работу, а затем делает r рекурсивных вызовов, каждый для задачи размера n/c. С точностью до постоянных множителей (которые можно скрыть в обозначении O(·)) время работы задаётся соотношением

$$T(n) = rT(n/c) + f(n).$$

Корень дерева рекурсии для T(n) имеет значение f(n) и r дочерних узлов, каждый из которых является корнем (рекурсивно определённого) дерева рекурсии для T(n/c). Эквивалентно, дерево рекурсии — полное r-арное дерево, каждый узел на глубине d которого содержит значение $f(n/c^d)$. Можете предполагать, что n — целая степень c, так что n/c^d всегда целое, хотя на самом деле это не имеет значения.

На практике я рекомендую нарисовать первые два-три уровня дерева, как на рисунке 1.9.

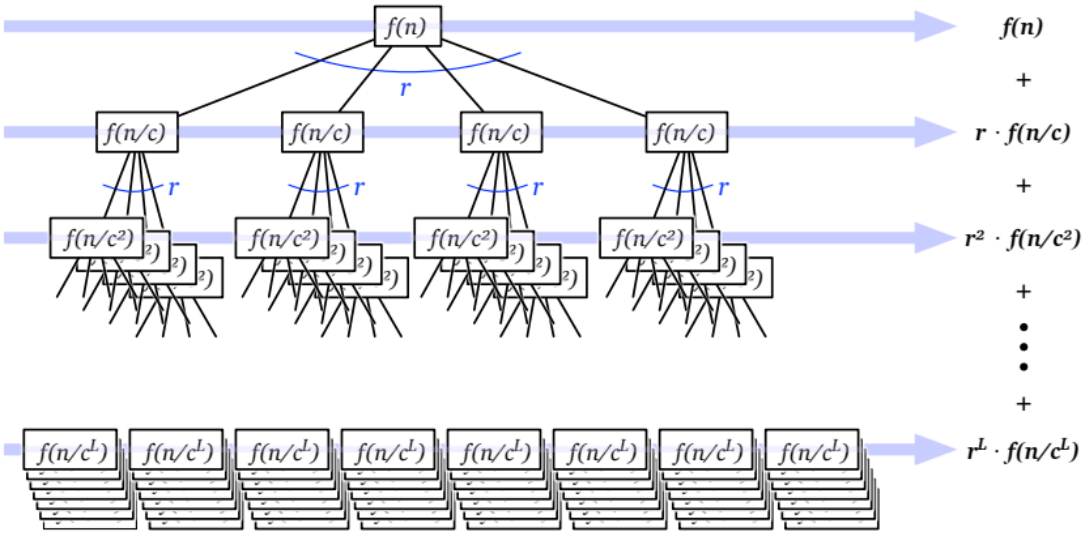


Рисунок 1.9. Дерево рекурсии для соотношения T(n) = rT(n/c) + f(n).

Листьям дерева соответствуют базовые случаи рекуррентного соотношения. Поскольку мы ищем только асимптотические оценки, точный базовый случай не важен; можно без опасений предположить T(n) = 1 для всех n ≤ n0, где n0 — произвольная положительная константа. В частности, можно выбрать любое наиболее удобное для анализа значение n0. В этом примере я выберу n0 = 1.

Теперь T(n) — сумма всех значений дерева рекурсии. Эту сумму можно вычислить, рассматривая дерево уровень за уровнем. Для каждого целого i на i-м уровне ровно r^i узлов, каждый со значением $f(n/c^i)$. Следовательно,

$$T(n) = \sum_{i=0}^{L} r^i f(n/c^i), \qquad (\Sigma)$$

где L — глубина дерева. Из базового случая n0 = 1 непосредственно следует L = log_c n, поскольку $n/c^L = n_0 = 1$. Поэтому число листьев дерева равно ровно $r^L = r^{(\log_c n)} = n^{(\log_c r)}$. Таким образом, последний член суммы по уровням (Σ) равен $n^{(\log_c r)} \cdot f(1) = O(n^{(\log_c r)})$, поскольку f(1) = O(1).

Есть три распространённых случая, когда сумму по уровням (Σ) особенно легко вычислить:

- **Убывание.** Если последовательность убывает экспоненциально — каждый член в постоянное число раз меньше предыдущего, — то T(n) = O(f(n)). В этом случае сумма определяется значением в корне дерева рекурсии.
- **Равенство.** Если все члены последовательности равны, сразу получаем T(n) = O(f(n) · L) = O(f(n) log n). (Константа c исчезает в обозначении O(·).)
- **Возрастание.** Если последовательность растёт экспоненциально — каждый член в постоянное число раз больше предыдущего, — то $T(n) = O(n^{(\log_c r)})$. В этом случае сумма определяется числом листьев дерева рекурсии.

В первом и третьем случаях важен только наибольший член геометрической прогрессии; все остальные поглощаются обозначением O(·). В случае убывания нам даже не нужно вычислять L: асимптотическая верхняя оценка оставалась бы верной, даже если бы дерево рекурсии было бесконечным!

В качестве простого примера нарисуем первые несколько уровней дерева рекурсии для упрощённого соотношения сортировки слиянием T(n) = 2T(n/2) + O(n). Мы обнаружим, что значения всех уровней равны, откуда сразу следует T(n) = O(n log n).

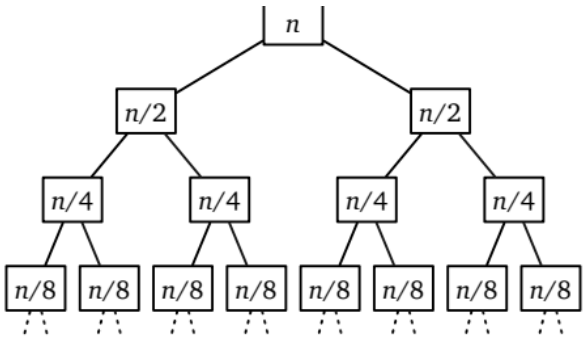


Рисунок 1.10. Дерево рекурсии сортировки слиянием.

Метод дерева рекурсии можно использовать и для алгоритмов, в которых рекурсивные подзадачи имеют разные размеры. Например, если бы удалось реализовать быструю сортировку так, чтобы опорный элемент всегда попадал в среднюю треть отсортированного массива, время работы в худшем случае удовлетворяло бы соотношению

$$T(n) \leq T(n/3) + T(2n/3) + O(n).$$

Это соотношение может выглядеть устрашающе, но на самом деле оно довольно покладистое. Нарисовав несколько уровней дерева рекурсии, мы быстро понимаем, что сумма значений на любом уровне не превосходит n (на более глубоких уровнях могут отсутствовать некоторые узлы), а глубина всего дерева равна log_(3/2) n = O(log n). Немедленно следует T(n) = O(n log n). Более того, число полных уровней дерева рекурсии равно log3 n = Ω(log n), поэтому эту осторожную оценку можно улучшить максимум в постоянное число раз, что для наших целей означает «никак». Несбалансированность дерева рекурсии попросту не имеет значения.

Более крайний пример: соотношение быстрой сортировки в худшем случае T(n) = T(n – 1) + T(1) + O(n) даёт совершенно несбалансированное дерево рекурсии, в котором один из потомков каждого внутреннего узла — лист. Сумма по уровням не попадает ни в одну из наших трёх стандартных категорий, но мы всё равно можем получить решение T(n) = O(n²), заметив, что значение каждого уровня не больше n, а всего уровней не больше n. (И снова эта осторожная оценка точна: каждый из n/2 уровней имеет значение не менее n/2.)

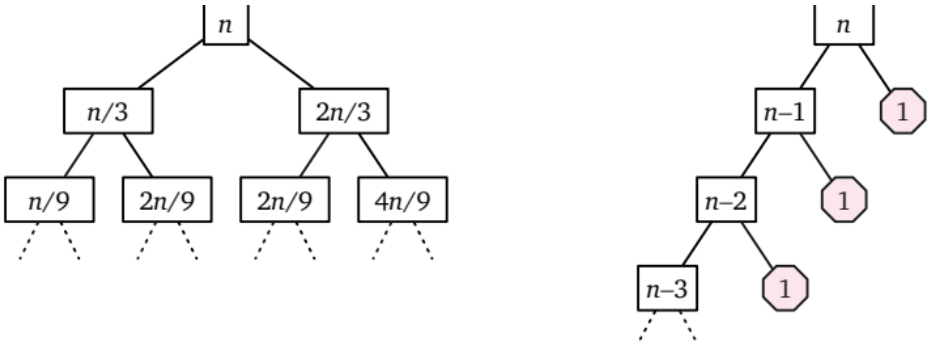


Рисунок 1.11. Деревья рекурсии быстрой сортировки при удачном выборе опорных элементов (слева) и в худшем случае (справа).

♥ Игнорировать округления вверх и вниз можно, честное слово

Внимательные читатели могут возразить, что наш анализ заметает под ковёр важную деталь. Время работы сортировки слиянием на самом деле не удовлетворяет соотношению T(n) = 2T(n/2) + O(n): в конце концов размер входа n может быть нечётным, да и что вообще значит отсортировать массив размера 42½ или 17⅞? Настоящее соотношение несколько сложнее:

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n).$$

Конечно, мы могли бы проверить T(n) = O(n log n) индукцией, но необходимые вычисления были бы ужасны. К счастью, существует простой приём удаления округлений вверх и вниз из рекуррентных соотношений — преобразование области определения.

- Во-первых, поскольку мы выводим верхнюю оценку, можно без опасений завысить T(n): сначала притвориться, что размеры двух подзадач равны, а затем избавиться от округления вверх:[8]

$$T(n) \le 2T(\lceil n/2 \rceil) + n \le 2T(n/2 + 1) + n.$$

- Во-вторых, определим новую функцию S(n) = T(n + α), выбрав константу α так, чтобы S(n) удовлетворяла более простому соотношению S(n) ≤ 2S(n/2) + O(n). Чтобы найти нужную α, выведем соотношение для S из данного соотношения для T:

$$\begin{aligned} S(n) &= T(n + \alpha) && [\text{определение } S] \\ &\le 2T(n/2 + \alpha/2 + 1) + n + \alpha && [\text{соотношение для } T] \\ &= 2S(n/2 - \alpha/2 + 1) + n + \alpha && [\text{определение } S]. \end{aligned}$$

При α = 2 это соотношение упрощается до S(n) ≤ 2S(n/2) + n + 2 — именно того, что нам требовалось.

- Наконец, метод дерева рекурсии даёт S(n) = O(n log n), а значит,

$$T(n) = S(n - 2) = O((n - 2)\log(n - 2)) = O(n \log n),$$

как мы и обещали.

Аналогичные преобразования области определения можно использовать, чтобы избавиться от округлений вверх и вниз и даже от членов более низкого порядка в любом рекуррентном соотношении метода «разделяй и властвуй». Но теперь, когда мы это поняли, больше никогда не нужно продираться через подробности! Отныне, встречая любое такое соотношение, я буду молча заметать округления и члены низкого порядка под ковёр и призываю вас поступать так же.

[8] Формально мы рассматриваем T как функцию на множестве действительных, а не только целых чисел, которая удовлетворяет данному рекуррентному соотношению с базовым случаем $T(n) = C$ для всех $n \le n_0$, где $C \ge 0$ и $n_0 > 0$ — некоторые действительные числа, конкретные значения которых не важны. Если n — целое число, $T(n)$ совпадает со временем работы алгоритма на входе размера n, но и это не важно.

♥ 1.8. Выбор за линейное время

Обсуждая быструю сортировку, я мимоходом утверждал, что медиану неотсортированного массива можно найти за линейное время. Первый такой алгоритм в начале 1970-х годов открыли Мануэль Блум, Боб Флойд, Воган Пратт, Рон Ривест и Боб Тарьян. Их алгоритм в действительности решает более общую задачу выбора k-го по величине элемента массива из n элементов, получая на вход массив и целое число k; он использует вариант алгоритма быстрого выбора, или «однорукой быстрой сортировки». Быстрый выбор впервые описал Тони Хоар в 1961 году, буквально на той же странице, на которой впервые опубликовал быструю сортировку.

Быстрый выбор

Общий алгоритм быстрого выбора выбирает опорный элемент, разделяет массив той же подпрограммой `Partition`, что и `QuickSort`, а затем рекурсивно ищет только в одном из двух подмассивов — именно в том, который содержит k-й по величине элемент исходного массива. Псевдокод дан на рисунке 1.12.

```
QuickSelect(A[1 .. n], k):
    если n = 1
        вернуть A[1]
    иначе
        выбрать опорный элемент A[p]
        r ← Partition(A[1 .. n], p)
        если k < r
            вернуть QuickSelect(A[1 .. r – 1], k)
        иначе если k > r
            вернуть QuickSelect(A[r + 1 .. n], k – r)
        иначе
            вернуть A[r]
```

Рисунок 1.12. Быстрый выбор, или однорукая быстрая сортировка.

У этого алгоритма две важные особенности. Во-первых, как и в быстрой сортировке, корректность быстрого выбора не зависит от способа выбора опорного элемента. Во-вторых, даже если нас интересует только выбор медианы (частный случай $k = n/2$), рекурсивная стратегия Хоара требует рассматривать более общую задачу выбора: медиана входного массива

A[1 .. n] почти никогда не является медианой ни одного из меньших подмассивов A[1 .. r − 1] и A[r + 1 .. n].

Время работы `QuickSelect` в худшем случае подчиняется соотношению, похожему на соотношение для `QuickSort`. Мы не знаем ни значения r, ни того, в каком из двух подмассивов будем искать рекурсивно, поэтому приходится предположить худшее:

$$T(n) \le \max_{1 \le r \le n} \max\{T(r-1), T(n-r)\} + O(n).$$

Немного упростим соотношение, обозначив длину рекурсивной подзадачи через ℓ:

$$T(n) \le \max_{0 \le \ell \le n-1} T(\ell) + O(n).$$

Если выбранный опорный элемент всегда наименьший или наибольший в массиве, соотношение упрощается до T(n) = T(n − 1) + O(n), откуда T(n) = O(n²). Дерево рекурсии для этого соотношения — просто путь.

Хорошие опорные элементы

Мы могли бы избежать квадратичного времени в худшем случае, если бы как-то волшебным образом выбирали хороший опорный элемент, то есть такой, что ℓ ≤ αn для некоторой константы α < 1. Тогда соотношение упростилось бы до

$$T(n) \le T(\alpha n) + O(n).$$

Оно разворачивается в убывающую геометрическую прогрессию, определяемую её наибольшим членом, поэтому T(n) = O(n). (И снова дерево рекурсии — простой путь. Константа во времени O(n) зависит от константы α.)

Иными словами, если бы мы могли быстро, за линейное время, найти хотя бы близкий к медиане элемент, мы могли бы за линейное время найти и точную медиану. Итак, теперь нам нужна только Фея Приближённой Медианы. Алгоритм Блума—Флойда—Пратта—Ривеста—Тарьяна выбирает хороший опорный элемент для быстрого выбора, рекурсивно вычисляя медиану тщательно выбранного подмножества входного массива. Фея Приближённой Медианы — просто переодетая Фея Рекурсии!

А именно, разделим входной массив на ⌈n/5⌉ блоков по пять элементов, кроме, возможно, последнего. (Если последний блок неполон, просто добавим несколько значений ∞.) Вычислим медиану каждого блока полным перебором, соберём эти медианы в новый массив M[1 .. ⌈n/5⌉], а затем рекурсивно вычислим медиану нового массива. Наконец, используем медиану медиан блоков (в псевдокоде ниже она называется `mom`) как опорный элемент быстрого выбора.

```
MomSelect(A[1 .. n], k):
    если n ≤ 25                       ⟪или другое значение⟫
        использовать полный перебор
    иначе
        m ← ⌈n/5⌉
        для i ← 1 до m
            M[i] ← MedianOfFive(A[5i − 4 .. 5i])  ⟪Полный перебор!⟫
        mom ← MomSelect(M[1 .. m], ⌊m/2⌋)         ⟪Рекурсия!⟫
        r ← Partition(A[1 .. n], mom)
```

```
        если k < r
            вернуть MomSelect(A[1 .. r – 1], k)     ⟪Рекурсия!⟫
        иначе если k > r
            вернуть MomSelect(A[r + 1 .. n], k – r) ⟪Рекурсия!⟫
        иначе
            вернуть mom
```

> *Примечание переводчика:* `mom` *— значение элемента, тогда как ранее определённая* `Partition` *принимает индекс опорного элемента. Для реализации нужно передать индекс элемента со значением* `mom` *либо изменить интерфейс* `Partition`*, чтобы он принимал значение.*

`MomSelect` применяет рекурсию для двух разных целей: сначала для выбора опорного элемента (`mom`), а затем для поиска среди элементов по одну сторону от него.

Анализ

Но почему это быстро? Первое ключевое наблюдение: медиана медиан — хороший опорный элемент. `mom` больше чем $\lfloor\lceil n/5\rceil/2\rfloor - 1 \approx n/10$ медиан блоков, а каждая медиана блока больше двух других элементов своего блока. Поэтому `mom` больше по меньшей мере $3n/10$ элементов входного массива; симметрично, `mom` меньше по меньшей мере $3n/10$ элементов. Значит, в худшем случае второй рекурсивный вызов ищет в массиве размера не более $7n/10$.

Можно наглядно представить поведение алгоритма, нарисовав входной массив как сетку $5 \times \lceil n/5\rceil$, в которой каждый столбец представляет пять последовательных элементов. Для иллюстрации представьте, что мы сортируем каждый столбец сверху вниз, а затем сортируем столбцы по их среднему элементу. (Подчеркну: алгоритм на самом деле этого не делает!) В таком расположении медиана медиан — ближайший к центру сетки элемент.

Левая половина первых трёх строк содержит $3n/10$ элементов, каждый из которых меньше `mom`. Если искомый элемент больше `mom`, перед рекурсией алгоритм отбросит всё, что меньше `mom`, включая эти $3n/10$ элементов. Поэтому на вход рекурсивной подзадачи попадёт не более $7n/10$ элементов. Симметричное рассуждение показывает: если искомый элемент меньше `mom`, мы отбросим не менее $3n/10$ элементов, больших `mom`, так что вход рекурсивной подзадачи будет содержать не более $7n/10$ элементов.

Хорошо, `mom` — хороший опорный элемент, но наш алгоритм всё равно выполняет два рекурсивных вызова вместо одного. Как доказать линейное время? Второе ключевое наблюдение: суммарный размер двух рекурсивных подзадач меньше размера исходного массива в постоянное число раз. Время работы алгоритма в худшем случае удовлетворяет соотношению

$$T(n) \leq T(n/5) + T(7n/10) + O(n).$$

Если нарисовать дерево рекурсии для этого соотношения, заметим, что общая работа на каждом уровне составляет не более 9/10 общей работы предыдущего уровня. Следовательно, суммы уровней убывают экспоненциально, что даёт решение $T(n) = O(n)$. (И снова несбалансированность дерева рекурсии совершенно не важна.) Ура! Спасибо, мамочка!

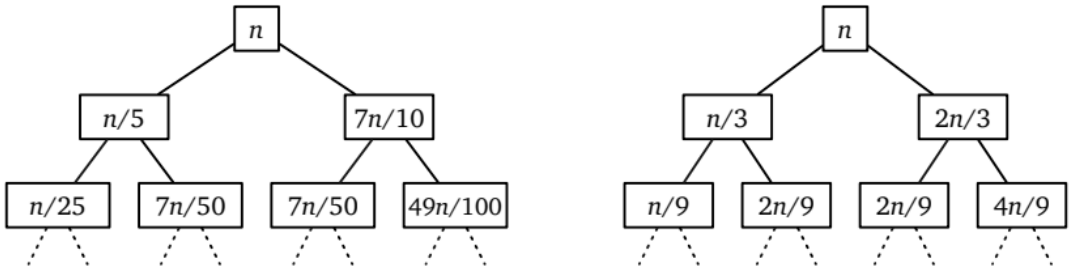


Рисунок 1.13. Деревья рекурсии для `MomSelect` и аналогичного алгоритма выбора с блоками по три элемента.

Проверка здравого смысла

На этом месте многие студенты спрашивают о волшебной константе 5. Почему выбран именно такой размер блока? Ответ: 5 — наименьший нечётный размер блока, который даёт экспоненциальное убывание в анализе дерева рекурсии! (Чётные размеры блоков вызывают дополнительные сложности.) Если бы мы использовали блоки по три элемента, соотношение времени работы было бы таким:

$$T(n) \le T(n/3) + T(2n/3) + O(n).$$

Это соотношение мы уже встречали! Каждый уровень дерева рекурсии имеет суммарное значение не более n, а глубина дерева равна log_(3/2) n = O(log n), поэтому решение — T(n) ≤ O(n log n). Более того, анализ точен, поскольку в дереве log3 n полных уровней. Выбор медианы медиан с блоками по три элемента не быстрее сортировки.

Более тонкий анализ показывает, что скрытая в обозначении O(·) константа довольно велика, даже если считать только сравнения. Для выбора медианы из пяти элементов требуется не более шести сравнений, так что для подготовки рекурсивной подзадачи нужно не более 6n/5 сравнений. Наивное разделение массива после рекурсивного вызова потребовало бы n − 1 сравнения, но нам уже известны 3n/10 элементов больше опорного и 3n/10 элементов меньше опорного, поэтому разделение на самом деле требует лишь 2n/5 дополнительных сравнений. Таким образом, более точное соотношение для числа сравнений в худшем случае:

$$T(n) \le T(n/5) + T(7n/10) + 8n/5.$$

Метод дерева рекурсии даёт верхнюю оценку

$$T(n) \le \frac{8n}{5} \sum_{i \ge 0} \left(\frac{9}{10}\right)^i = \frac{8n}{5} \cdot 10 = 16n.$$

На практике выбор медианы медиан не так медленен, как предсказывает этот анализ худшего случая: невероятно маловероятно, что на каждом уровне рекурсии будет выбран наихудший опорный элемент. Однако он всё равно медленнее сортировки даже для умеренно больших массивов.[9]

[9] На самом деле правильный способ выбора опорного элемента на практике — равномерный случайный выбор. Тогда ожидаемое число сравнений, необходимых для нахождения медианы, не превосходит 4n. Подробнее см. мои конспекты по рандомизированным алгоритмам на http://algorithms.wtf.

1.9. Быстрое умножение

В предыдущей главе мы видели два древних алгоритма умножения двух n-значных чисел за время O(n²): школьный алгоритм решёткой и египетский крестьянский алгоритм.

Возможно, более эффективный алгоритм получится, если разделить массивы цифр пополам и воспользоваться тождеством

$$(10^m a + b)(10^m c + d) = 10^{2m} ac + 10^m (bc + ad) + bd.$$

Это рекуррентное соотношение сразу подсказывает следующий алгоритм «разделяй и властвуй» для умножения двух n-значных чисел x и y. Каждое из четырёх частичных произведений ac, bc, ad и bd вычисляется рекурсивно, но умножения в последней строке не рекурсивны: умножить на степень десяти можно, сдвинув цифры влево и дописав нужное число нулей, — всё это за O(n).

```
SplitMultiply(x, y, n):
    если n = 1
        вернуть x · y
    иначе
        m ← ⌈n/2⌉
        a ← ⌊x/10ᵐ⌋; b ← x mod 10ᵐ    ⟪x = 10ᵐa + b⟫
        c ← ⌊y/10ᵐ⌋; d ← y mod 10ᵐ    ⟪y = 10ᵐc + d⟫
        e ← SplitMultiply(a, c, m)
        f ← SplitMultiply(b, d, m)
        g ← SplitMultiply(b, c, m)
        h ← SplitMultiply(a, d, m)
        вернуть 10²ᵐe + 10ᵐ(g + h) + f
```

Корректность алгоритма легко доказать индукцией. Время работы подчиняется соотношению

$$T(n) = 4T(\lceil n/2 \rceil) + O(n).$$

Метод дерева рекурсии преобразует его в возрастающую геометрическую прогрессию, откуда T(n) = O(n^(log24)) = O(n²). В действительности этот алгоритм умножает каждую цифру x на каждую цифру y — точно как алгоритм решёткой.

Похоже, ничего не вышло. Как жаль. Хорошая была идея.

В середине 1950-х годов Андрей Колмогоров, один из гигантов математики XX века, публично предположил, что алгоритма умножения двух n-значных чисел за субквадратичное время не существует. В 1960 году Колмогоров организовал семинар в Московском университете, где повторил свою «гипотезу n²» и поставил несколько связанных задач, которые планировал обсудить на следующих заседаниях. Почти ровно через неделю 23-летний студент Анатолий Карацуба представил Колмогорову замечательный контрпример. По словам самого Карацубы:

После семинара я рассказал Колмогорову о новом алгоритме и об опровержении гипотезы n². Колмогоров был очень взволнован, поскольку это противоречило его весьма правдоподобной гипотезе. На следующем заседании семинара Колмогоров сам рассказал участникам о моём методе, после чего семинар прекратился.

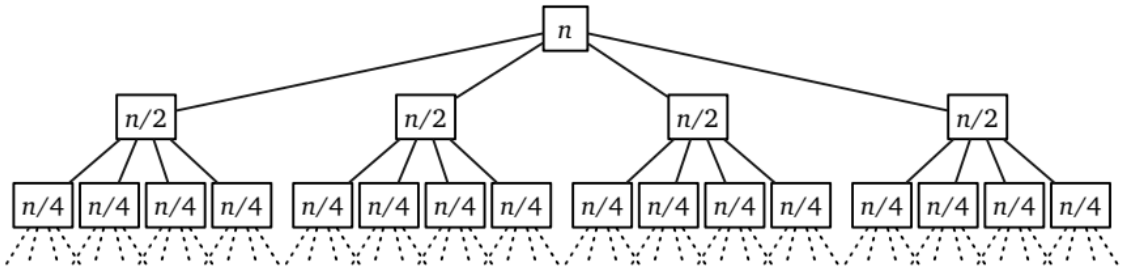


Рисунок 1.14. Дерево рекурсии наивного умножения методом «разделяй и властвуй».

Карацуба заметил, что средний коэффициент bc + ad можно вычислить по двум другим коэффициентам ac и bd всего с помощью одного дополнительного рекурсивного умножения, используя тождество

$$ac + bd - (a - b)(c - d) = bc + ad.$$

Этот приём позволяет заменить четыре рекурсивных вызова предыдущего алгоритма всего тремя:

```
FastMultiply(x, y, n):
    если n = 1
        вернуть x · y
    иначе
        m ← ⌈n/2⌉
        a ← ⌊x/10ᵐ⌋; b ← x mod 10ᵐ    ⟪x = 10ᵐa + b⟫
        c ← ⌊y/10ᵐ⌋; d ← y mod 10ᵐ    ⟪y = 10ᵐc + d⟫
        e ← FastMultiply(a, c, m)
        f ← FastMultiply(b, d, m)
        g ← FastMultiply(a – b, c – d, m)
        вернуть 10²ᵐe + 10ᵐ(e + f – g) + f
```

Время работы алгоритма Карацубы `FastMultiply` удовлетворяет соотношению

$$T(n) \leq 3T(\lceil n/2 \rceil) + O(n).$$

Метод дерева рекурсии снова преобразует это соотношение в возрастающую геометрическую прогрессию, но новое решение — всего T(n) = O(n^(log23)) = O(n^1,58496), что значительно лучше прежней квадратичной оценки.[10]

[10] Моё изложение несколько упрощает реальную историю. На самом деле Карацуба предложил алгоритм на основе формулы (a + b)(c + d) − ac − bd = bc + ad. Он тоже работает за O(n^(lg 3)), но само соотношение немного сложнее: a − b и c − d по-прежнему имеют m цифр, а a + b и c + d могут иметь по m + 1 цифре. Приведённое здесь упрощение принадлежит Дональду Кнуту.

Можно утверждать, что алгоритм Карацубы положил начало разработке и анализу алгоритмов как самостоятельной области исследования.

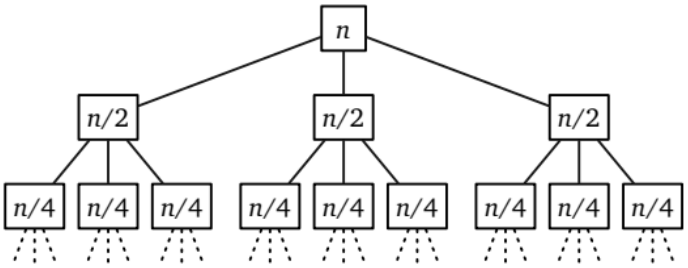


Рисунок 1.15. Дерево рекурсии алгоритма умножения Карацубы методом «разделяй и властвуй».

Идею Карацубы можно развить дальше, разбивая числа на большее число частей и объединяя их более сложными способами, чтобы получить ещё более быстрые алгоритмы умножения. Андрей Тоом открыл бесконечное семейство алгоритмов, разбивающих каждое целое число на k частей по n/k цифр и вычисляющих произведение всего с помощью 2k – 1 рекурсивных умножений; Стивен Кук далее упростил алгоритмы Тоома в своей диссертации. Для любого фиксированного k алгоритм Тоома—Кука работает за O(n^(1+1/(lg k))), где скрытая в O(·) константа зависит от k.

В конечном счёте эта стратегия «разделяй и властвуй» привела Гаусса (да, действительно) к открытию быстрого преобразования Фурье.[11] Сам базовый алгоритм БПФ работает за O(n log n); однако его использование для умножения целых чисел вызывает небольшие дополнительные накладные расходы. Первый алгоритм умножения целых чисел на основе БПФ, опубликованный Арнольдом Шёнхаге и Фолькером Штрассеном в 1971 году, работает за O(n log n log log n). Алгоритм Шёнхаге—Штрассена оставался теоретически самым быстрым алгоритмом умножения целых чисел несколько десятилетий, пока Мартин Фюрер не открыл первое из длинной серии технических усовершенствований. Наконец, в 2019 году Дэвид Харви и Йорис ван дер Хувен опубликовали алгоритм, работающий за O(n log n).[12]

[11] Конспекты по быстрому преобразованию Фурье см. на http://algorithms.wtf.

[12] На практике алгоритм Шёнхаге—Штрассена действительно самый быстрый для умножения целых чисел, имеющих более примерно 75 000 цифр; более поздние алгоритмы Фюрера, Харви, ван дер Хувена и других оказались бы быстрее «на практике» только для целых чисел, у которых цифр больше, чем частиц во Вселенной.

1.10. Возведение в степень

Допустим, нам даны число a и положительное целое n и мы хотим вычислить a^n. Стандартный наивный метод — простой цикл, выполняющий n – 1 умножений на a:

```
SlowPower(a, n):
    x ← a
    для i ← 2 до n
```

```
        x ← x · a
    вернуть x
```

Этот итеративный алгоритм требует n умножений.

Входной параметр a может быть целым, рациональным числом или числом с плавающей точкой. На самом деле ему вовсе не обязательно быть числом: достаточно, чтобы это было нечто, что мы умеем умножать. Например, тот же алгоритм можно использовать для возведения в степень по модулю некоторого конечного числа (операция, часто применяемая в криптографических алгоритмах) или возведения матриц в степень (это используется для вычисления рекуррентных соотношений и поиска кратчайших путей в графах). Поскольку мы не знаем, какого рода объекты перемножаем, мы не можем знать время одного умножения, и поэтому вынуждены анализировать время работы по числу умножений.

Существует гораздо более быстрый метод «разделяй и властвуй», первоначально предложенный индийским исследователем стихосложения Пингалой во II веке до н. э. и использующий простую рекурсивную формулу:

$$a^n = \begin{cases} 1 & \text{если } n = 0, \\ (a^{n/2})^2 & \text{если } n > 0 \text{ и } n \text{ чётно}, \\ (a^{\lfloor n/2 \rfloor})^2 \cdot a & \text{иначе.} \end{cases}$$

```
PingalaPower(a, n):
    если n = 1
        вернуть a
    иначе
        x ← PingalaPower(a, ⌊n/2⌋)
        если n чётно
            вернуть x · x
        иначе
            вернуть x · x · a
```

Общее число умножений алгоритма удовлетворяет соотношению T(n) ≤ T(n/2) + 2. Метод дерева рекурсии сразу даёт решение T(n) = O(log n).

Почти идентичный алгоритм возведения в степень можно также получить непосредственно из египетского алгоритма крестьянского умножения предыдущей главы, заменив сложение умножением (в частности, удвоение — возведением в квадрат):

$$a^n = \begin{cases} 1 & \text{если } n = 0, \\ (a^2)^{n/2} & \text{если } n > 0 \text{ и } n \text{ чётно}, \\ (a^2)^{\lfloor n/2 \rfloor} \cdot a & \text{иначе.} \end{cases}$$

```
PeasantPower(a, n):
    если n = 1
        вернуть a
    иначе если n чётно
        вернуть PeasantPower(a², n/2)
    иначе
        вернуть PeasantPower(a², ⌊n/2⌋) · a
```

Этот алгоритм, который вполне можно назвать «возведением в квадрат и делением пополам», также выполняет лишь O(log n) умножений.

Оба алгоритма асимптотически оптимальны: любой алгоритм вычисления a^n должен выполнить не менее Ω(log n) умножений, поскольку каждое умножение не более чем удваивает наибольшую вычисленную к этому моменту степень. На самом деле, когда n — степень двойки, оба алгоритма требуют ровно $\log_2 n$ умножений, что в точности оптимально. Однако для других значений n существуют немного более быстрые методы. Например, `PingalaPower` и `PeasantPower` вычисляют a^{15} за шесть умножений, но достаточно всего пяти:

- Пингала: $a \to a^2 \to a^3 \to a^6 \to a^7 \to a^{14} \to a^{15}$.
- Крестьянский метод: $a \to a^2 \to a^4 \to a^8 \to a^{12} \to a^{14} \to a^{15}$.
- Оптимальный метод: $a \to a^2 \to a^3 \to a^5 \to a^{10} \to a^{15}$.

Остаётся давним открытым вопросом, можно ли эффективно вычислить абсолютный минимум числа умножений для заданного показателя n.

Упражнения

Ханойская башня

1. Докажите, что исходный рекурсивный алгоритм Ханойской башни выполняет в точности ту же последовательность перемещений — те же диски, между теми же стержнями и в том же порядке, — что и каждый из следующих нерекурсивных алгоритмов. Стержни обозначены 0, 1 и 2; требуется переместить стопку из n дисков со стержня 0 на стержень 2 (как показано на стр. 24 оригинала).

   (a) Если n чётно, поменяйте обозначения стержней 1 и 2. На i-м шаге выполните единственный допустимый ход, не затрагивающий стержень i mod 3. Если допустимого хода нет, то все диски находятся на стержне i mod 3 и головоломка решена.

   (b) Первым ходом переместите диск 1 на стержень 1, если n чётно, или на стержень 2, если n нечётно. Затем многократно выполняйте единственный допустимый ход, в котором участвует диск, отличный от перемещённого предыдущим ходом. Если такого хода нет, головоломка решена.

   (c) Представьте, что диски n + 1, n + 2 и n + 3 лежат соответственно на дне стержней 0, 1 и 2. Многократно выполняйте единственный допустимый ход, удовлетворяющий следующим ограничениям, пока такие ходы возможны:

   - Нельзя класть нечётный диск непосредственно на другой нечётный диск.
   - Нельзя класть чётный диск непосредственно на другой чётный диск.
   - Нельзя отменять предыдущий ход.

   (d) Пусть ρ(n) — наименьшее целое k, при котором $n/2^k$ не является целым. Например, ρ(42) = 2, поскольку $42/2^1$ — целое, а $42/2^2$ — нет. Эквивалентно, ρ(n) на единицу больше позиции младшего единичного разряда двоичного представления n. Поскольку её поведение напоминает отметки на линейке, ρ(n) иногда называют функцией линейки.

```
RulerHanoi(n):
    i ← 1
    пока ρ(i) ≤ n
        если n − i чётно
            переместить диск ρ(i) вперёд    ⟪0 → 1 → 2 → 0⟫
        иначе
            переместить диск ρ(i) назад     ⟪0 → 2 → 1 → 0⟫
        i ← i + 1
```

2. Ханойская башня — сравнительно недавний потомок гораздо более старой механической головоломки, известной как китайские связанные кольца, багнодье, кольца Кардано, Меледа, «Терпение», «Утомительные кольца», «Замок узника», Spin-Out и под многими другими именами. Уже к XVI веку она была хорошо известна и в Китае, и в Европе. Итальянский математик Лука Пачоли описал головоломку из семи колец и её решение в неопубликованном трактате *De Viribus Quantitatis*, написанном между 1498 и 1506 годами;[13] всего через несколько лет поэт династии Мин Ян Шэнь назвал головоломку из девяти колец «игрушкой для женщин и детей». Согласно недостоверному преданию, её придумал китайский полководец II века, который подарил головоломку жене, чтобы занять её, пока он воюет.

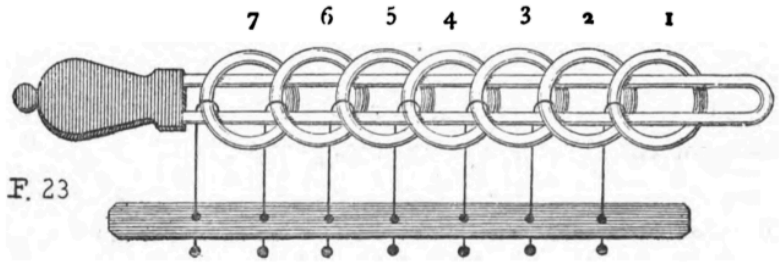


Рисунок 1.16. Багнодье из семи колец из *Récréations Mathématiques* Эдуара Люка (1891). См. сведения об источниках иллюстраций в конце книги.

Головоломка существует во многих физических вариантах, но один из самых распространённых состоит из длинной металлической петли и нескольких колец, соединённых с неподвижным основанием подвижными стержнями. Изначально петля продета через кольца, как показано на рисунке 1.16; цель — снять петлю.

Более абстрактно головоломку можно представить последовательностью битов, по одному на каждое кольцо: i-й бит равен 1, если петля проходит через i-е кольцо, и 0 в противном случае. (Кольца нумеруются справа налево, как на рисунке 1.16.) Разрешены два типа ходов:

- Первый, то есть крайний правый, бит можно менять всегда.
- Если битовая строка заканчивается ровно z нулями, можно изменить (z + 2)-й бит.

Цель — преобразовать строку из n единиц в строку из n нулей. Например, следующая последовательность из 21 хода решает головоломку из пяти колец:

```
11111 →¹ 11110 →³ 11010 →¹ 11011 →² 11001 →¹ 11000 →⁵ 01000 →¹ 01001 →² 01011 →¹ 01010 →³
01110 →¹ 01111 →² 01101 →¹ 01100 →⁴ 00100 →¹ 00101 →² 00111 →¹ 00110 →³ 00010 →¹ 00011 →²
```

`00001` $\to^1$ `00000`.

♦ (a) Назовём последовательность ходов сокращённой, если ни один ход не отменяет предыдущий. Докажите, что для любого неотрицательного целого n существует ровно одна сокращённая последовательность ходов, решающая головоломку из n колец. [Подсказка: задача намного проще, если вы уже знакомы с графами.]

(b) Опишите алгоритм решения головоломки. На входе — число колец n; алгоритм должен вывести сокращённую последовательность ходов, решающую её. Например, при входе 5 он должен вывести 1, 3, 1, 2, 1, 5, 1, 2, 1, 3, 1, 2, 1, 4, 1, 2, 1, 3, 1, 2, 1.

(c) Сколько в точности ходов выполняет ваш алгоритм в зависимости от n? Докажите правильность ответа.

[13] *De Viribus Quantitatis* («О силах чисел») — важный ранний труд по занимательной математике и, возможно, древнейший сохранившийся трактат о фокусах. Пачоли более известен книгой *Summa de Aritmetica* — почти полной энциклопедией математики конца XV века, содержащей первое описание двойной бухгалтерской записи.

3. Менее известная глава истории Ханойской башни — краткий перенос храма из Бенареса в Пизу в начале XIII века.[14] Перенос организовал богатый торговец и математик Леонардо Фибоначчи по просьбе императора Священной Римской империи Фридриха II, слышавшего о храме от солдат, вернувшихся из крестовых походов. Пизанские башни и сопровождавшие их монахи стали знаменитыми, помогая Пизе утвердиться как ведущему торговому центру Апеннинского полуострова.

   К сожалению, почти сразу после переноса храма одна из алмазных игл начала наклоняться. Чтобы избежать падения наклонной башни от слишком частого использования, Фибоначчи убедил жрецов принять более мягкое правило: за один ход с наклонной иглы можно переместить на другую любое число дисков вместе. По-прежнему запрещалось класть больший диск на меньший; на наклонную иглу или между двумя вертикальными иглами диски приходилось переносить по одному.

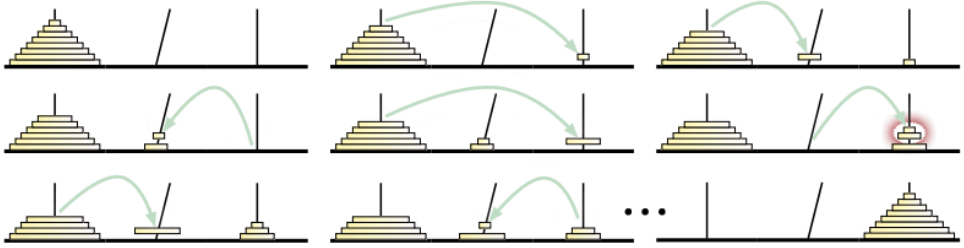


Рисунок 1.17. Пизанские башни. На пятом ходе два диска снимаются с наклонной иглы.

Благодаря новому правилу Фибоначчи жрецы могли приблизить конец Вселенной в Пизе несколько быстрее, чем в Бенаресе. К счастью, храм перенесли обратно из Пизы в Бенарес после того, как недавно избранный папа Григорий IX отлучил Фридриха II от церкви, из-за чего местные жрецы стали менее расположены принимать иностранных еретиков со странными

математическими привычками. Вскоре на месте храма возвели колокольню; она тоже почти сразу начала наклоняться.

Опишите алгоритм переноса стопки из n дисков с одной вертикальной иглы на другую за минимально возможное число ходов. Сколько в точности ходов выполняет ваш алгоритм?

[14] Некоторые части этой истории действительно правдивы.

4. Рассмотрим следующие варианты Ханойской башни с ограничениями. В каждой задаче стержни пронумерованы 0, 1 и 2, а требуется переместить стопку из n дисков со стержня 0 на стержень 2, как в задаче 1.

   (a) Допустим, непосредственно перемещать диск между стержнями 1 и 2 запрещено: каждый ход должен затрагивать стержень 0. Опишите алгоритм, решающий этот вариант головоломки за минимальное число ходов. Сколько в точности ходов он делает?

   ♣♥ (b) Допустим, разрешено перемещать диски только со стержня 0 на 2, с 2 на 1 или с 1 на 0. Иначе говоря, стержни расположены по кругу и пронумерованы по часовой стрелке, а перемещать диски можно только против часовой стрелки. Опишите алгоритм, решающий этот вариант за минимальное число ходов. Сколько ходов он делает?

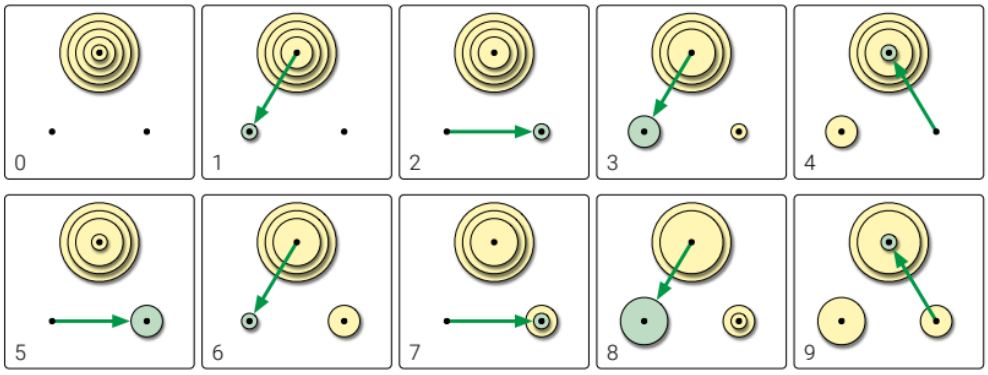


Рисунок 1.18. Первые несколько ходов решения Ханойской башни против часовой стрелки.

♣♥ (c) Наконец, допустим, единственное ограничение состоит в том, что нельзя перемещать диск непосредственно со стержня 0 на стержень 2. Опишите алгоритм решения за минимальное число ходов. Сколько ходов он делает? [Подсказка: матрицы! Анализ этого варианта значительно сложнее, чем двух предыдущих.]

5. Рассмотрим более сложный вариант Ханойской башни. В ряд расположено k стержней с номерами от 1 до k. За один ход можно переместить наименьший диск со стержня i на стержень $i - 1$ или $i + 1$ для любого индекса i; как обычно, запрещено класть больший диск на меньший. Ваша задача — переместить стопку из n дисков со стержня 1 на стержень k.

   (a) Опишите рекурсивный алгоритм для $k = 3$. Сколько в точности ходов он выполняет? (Это в точности задача 4(a).)

(b) Опишите рекурсивный алгоритм для $k = n + 1$, выполняющий не более $O(n^3)$ ходов. [Подсказка: используйте пункт (a).]

♥ (c) Опишите рекурсивный алгоритм для $k = n + 1$, выполняющий не более $O(n^2)$ ходов. [Подсказка: не используйте пункт (a).]

♥ (d) Опишите рекурсивный алгоритм для $k = \sqrt{n}$, выполняющий не более полиномиального числа ходов. (Какого именно многочлена?)

♥ (e) Опишите и проанализируйте рекурсивный алгоритм для произвольных n и k. Насколько малым должно быть k как функция от n, чтобы число ходов ограничивалось многочленом от n?

Деревья рекурсии

6. Решите каждое из следующих рекуррентных соотношений с помощью деревьев рекурсии:

$$\begin{array}{lll} A(n) = 2A(n/4) + \sqrt{n} & B(n) = 2B(n/4) + n & C(n) = 2C(n/4) + n^2 \\ D(n) = 3D(n/3) + \sqrt{n} & E(n) = 3E(n/3) + n & F(n) = 3F(n/3) + n^2 \\ G(n) = 4G(n/2) + \sqrt{n} & H(n) = 4H(n/2) + n & I(n) = 4I(n/2) + n^2. \end{array}$$

7. Решите с помощью деревьев рекурсии:

(j) $J(n) = J(n/2) + J(n/3) + J(n/6) + n$.

(k) $K(n) = K(n/2) + 2K(n/3) + 3K(n/4) + n^2$.

(l) $L(n) = L(n/15) + L(n/10) + 2L(n/6) + \sqrt{n}$.

♥ 8. Решите с помощью деревьев рекурсии:

(m) $M(n) = 2M(n/2) + O(n \log n)$.

(n) $N(n) = 2N(n/2) + O(n/\log n)$.

(p) $P(n) = \sqrt{n}\, P(\sqrt{n}) + n$.

(q) $Q(n) = \sqrt{(2n)}\, Q(\sqrt{(2n)}) + \sqrt{n}$.

Сортировка

9. Пусть дана стопка из n блинов разных размеров. Нужно отсортировать блины так, чтобы меньшие лежали поверх больших. Единственная допустимая операция — переворот: поддеть лопаткой верхние k блинов для некоторого целого k от 1 до n и перевернуть их все вместе.

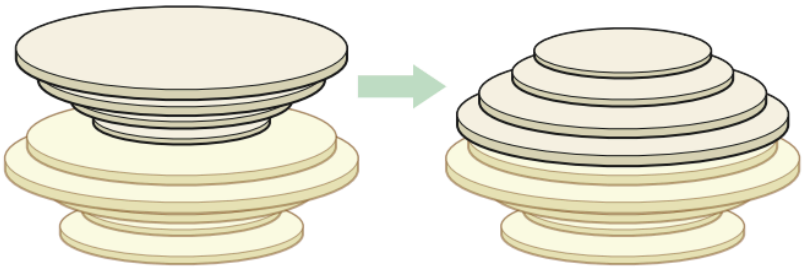


Рисунок 1.19. Переворот четырёх верхних блинов.

(a) Опишите алгоритм сортировки произвольной стопки из n блинов за O(n) переворотов. Сколько в точности переворотов выполняет ваш алгоритм в худшем случае?[15] [Подсказка: эта задача никак не связана с Ханойской башней.]

(b) Для каждого положительного целого n опишите стопку из n блинов, для сортировки которой требуется Ω(n) переворотов.

(c) Теперь допустим, что одна сторона каждого блина подгорела. Опишите алгоритм сортировки произвольной стопки из n блинов за O(n) переворотов так, чтобы подгоревшая сторона каждого блина была обращена вниз. Сколько в точности переворотов выполняет ваш алгоритм в худшем случае?

[15] Точное оптимальное в худшем случае число переворотов, необходимое для сортировки n блинов (подгоревших или нет), — давняя открытая проблема; просто сделайте как можно лучше.

10. Напомним, что эвристика «медиана трёх» рассматривает первый, последний и средний элементы массива и использует медиану этих трёх элементов как опорный элемент быстрой сортировки. Докажите, что в худшем случае быстрой сортировке с этой эвристикой требуется $\Omega(n^2)$ времени для сортировки массива размера n. А именно: для любого целого n опишите перестановку целых чисел от 1 до n, такую что при каждом рекурсивном вызове быстрой сортировки с медианой трёх опорный элемент всегда является вторым наименьшим элементом массива. Для построения такой перестановки необходимо подробно знать подпрограмму `Partition`.

(a) Для разминки предположим, что `Partition` устойчива: сохраняет исходный порядок всех элементов меньше опорного, а также исходный порядок всех элементов меньше опорного.

♥ (b) Предположим, что `Partition` использует конкретный алгоритм со страницы 29, который не является устойчивым.

11. (a) Эй, Мо! Эй, Ларри! Докажите, что следующий алгоритм действительно сортирует входные данные!

```
StoogeSort(A[0 .. n − 1]):
    если n = 2 и A[0] > A[1]
        обменять A[0] ⇔ A[1]
    иначе если n > 2
```

```
        m ← ⌈2n/3⌉
        StoogeSort(A[0 .. m − 1])
        StoogeSort(A[n − m .. n − 1])
        StoogeSort(A[0 .. m − 1])
```

(b) Будет ли `StoogeSort` по-прежнему правильно сортировать, если заменить $m = \lceil 2n/3 \rceil$ на $m = \lfloor 2n/3 \rfloor$? Обоснуйте ответ.

(c) Запишите рекуррентное соотношение, включая базовые случаи, для числа сравнений, выполняемых `StoogeSort`.

(d) Решите это соотношение и докажите правильность решения. [Подсказка: игнорируйте округление вверх.]

(e) Докажите, что число обменов `StoogeSort` не превосходит биномиального коэффициента (n над 2).

12. Следующий жестокий и необычный алгоритм сортировки предложил Гэри Миллер:

```
Cruel(A[1 .. n]):
    если n > 1
        Cruel(A[1 .. n/2])
        Cruel(A[n/2 + 1 .. n])
        Unusual(A[1 .. n])

Unusual(A[1 .. n]):
    если n = 2
        если A[1] > A[2]             ⟪единственное сравнение!⟫
            обменять A[1] ↔ A[2]
    иначе
        для i ← 1 до n/4             ⟪поменять вторую и третью четверти⟫
            обменять A[i + n/4] ↔ A[i + n/2]
        Unusual(A[1 .. n/2])          ⟪рекурсия на левой половине⟫
        Unusual(A[n/2 + 1 .. n])      ⟪рекурсия на правой половине⟫
        Unusual(A[n/4 + 1 .. 3n/4])  ⟪рекурсия на средней половине⟫
```

Выполняемые этим алгоритмом сравнения совершенно не зависят от значений входного массива; такой алгоритм сортировки называют независимым от данных. В этой задаче предполагайте, что размер входа n всегда является степенью двойки.

(a) Докажите индукцией, что `Cruel` правильно сортирует любой входной массив. [Подсказка: рассмотрите массив, содержащий n/4 единиц, n/4 двоек, n/4 троек и n/4 четвёрок. Почему достаточно этого частного случая?]

(b) Докажите, что `Cruel` перестанет правильно сортировать, если удалить цикл из `Unusual`.

(c) Докажите, что `Cruel` перестанет правильно сортировать, если поменять местами две последние строки `Unusual`.

(d) Каково время работы `Unusual`? Обоснуйте ответ.

(e) Каково время работы `Cruel`? Обоснуйте ответ.

13. Инверсия в массиве A[1 .. n] — пара индексов (i, j), для которой i < j и A[i] > A[j]. Число инверсий в массиве из n элементов находится между 0 (если массив отсортирован) и биномиальным коэффициентом (n над 2) (если массив отсортирован в обратном порядке). Опишите и проанализируйте алгоритм подсчёта инверсий за O(n log n). [Подсказка: измените сортировку слиянием.]

14. (a) Допустим, даны два множества по n точек: $\{p_1, p_2, \ldots, p_n\}$ на прямой y = 0 и $\{q_1, q_2, \ldots, q_n\}$ на прямой y = 1. Соединив каждую точку p_i с соответствующей q_i, построим n отрезков. Опишите и проанализируйте алгоритм «разделяй и властвуй», определяющий число пар пересекающихся отрезков за O(n log n). [Подсказка: см. предыдущую задачу.]

(b) Теперь допустим, что даны два множества $\{p_1, p_2, \ldots, p_n\}$ и $\{q_1, q_2, \ldots, q_n\}$ по n точек на единичной окружности. Соедините каждую точку p_i с соответствующей q_i. Опишите и проанализируйте алгоритм «разделяй и властвуй», определяющий число пар пересекающихся отрезков за $O(n \log^2 n)$. [Подсказка: используйте решение пункта (a).]

♥ (c) Опишите алгоритм для пункта (b), работающий за O(n log n). [Подсказка: используйте ваше решение пункта (b)!]

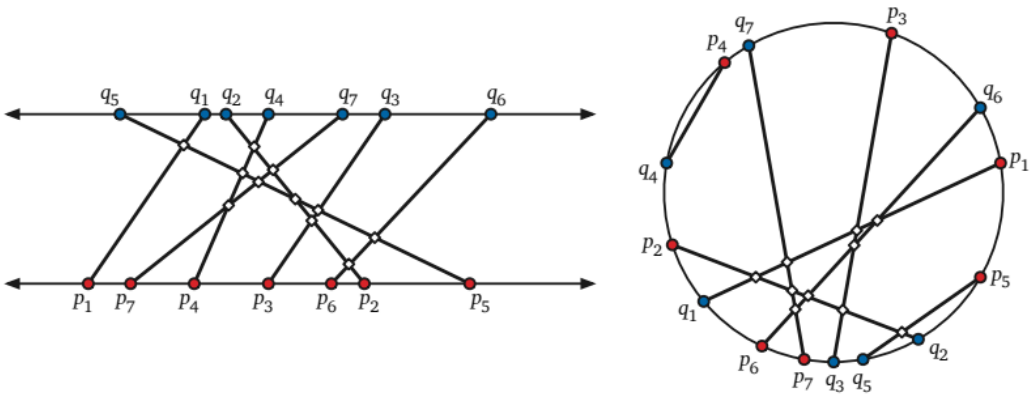


Рисунок 1.20. Одиннадцать пар пересекающихся отрезков с концами на параллельных прямых и десять пар пересекающихся отрезков с концами на окружности.

15. (a) Опишите алгоритм сортировки входного массива A[1 .. n] с помощью вызовов подпрограммы `SqrtSort(k)`, которая сортирует на месте подмассив A[k + 1 .. k + √n], получая на вход произвольное целое k от 0 до n − √n. Для упрощения считайте √n целым. Ваш алгоритм может просматривать и изменять массив только вызовами `SqrtSort`; в частности, он не должен непосредственно сравнивать, перемещать или копировать элементы массива. Сколько вызовов `SqrtSort` алгоритм выполняет в худшем случае?

♣ (b) Докажите, что ваш алгоритм из пункта (a) оптимален с точностью до постоянных множителей. Иными словами, если f(n) — число вызовов `SqrtSort` вашим алгоритмом, докажите, что никакой алгоритм не может отсортировать массив за o(f(n)) вызовов `SqrtSort`.

(c) Теперь допустим, что `SqrtSort` реализована рекурсивно вызовами вашего алгоритма сортировки из пункта (a). Например, на втором уровне рекурсии алгоритм сортирует массивы приблизительно размера n^(1/4). Каково время работы полученного алгоритма сортировки в

худшем случае? Для упрощения анализа считайте, что размер массива n имеет вид $2^{(2^k)}$, так что повторное извлечение квадратного корня всегда даёт целое число.

Выбор

16. Пусть дано множество S из n элементов, каждый со значением и весом. Для любого $x \in S$ определим два подмножества:

- $S_{<x}$ — элементы S, значения которых меньше значения x.
- $S_{>x}$ — элементы S, значения которых больше значения x.

Для любого $R \subseteq S$ обозначим через w(R) сумму весов элементов R. Взвешенная медиана R — любой элемент x, для которого $w(S_{<x}) \le w(S)/2$ и $w(S_{>x}) \le w(S)/2$.

Опишите и проанализируйте алгоритм вычисления взвешенной медианы данного взвешенного множества за O(n). На вход подаются два неотсортированных массива S[1 .. n] и W[1 .. n], где для каждого индекса i элемент имеет значение S[i] и вес W[i]. Можно считать, что все значения различны, а все веса положительны.

17. (a) Опишите алгоритм, который за O(n) определяет, содержит ли произвольный массив A[1 .. n] более n/4 экземпляров какого-либо значения.

(b) Опишите и проанализируйте алгоритм, который по произвольному массиву A[1 .. n] и целому k определяет, содержит ли A более k экземпляров какого-либо значения. Выразите время работы как функцию от n и k.

Не используйте хеширование, поразрядную сортировку или любой другой метод, зависящий от конкретных входных значений, а не от их порядка.

18. Опишите алгоритм вычисления медианы массива A[1 .. 5] различных чисел не более чем за шесть сравнений. Вместо псевдокода опишите алгоритм деревом решений: двоичным деревом, каждый внутренний узел которого содержит сравнение вида «A[i] ≷ A[j]?», а каждый лист — индекс элемента массива.

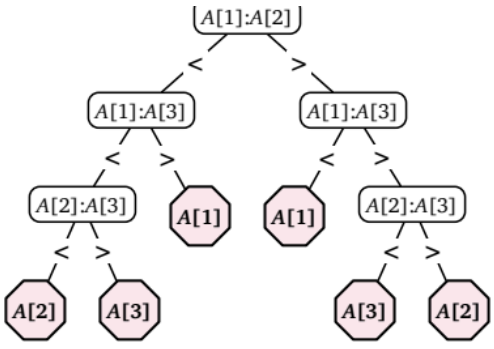


Рисунок 1.21. Нахождение медианы массива из трёх элементов не более чем за три сравнения.

19. Рассмотрим обобщение алгоритма `MomSelect` Блума—Флойда—Пратта—Ривеста—Тарьяна на рисунке 1.22: оно разбивает входной массив на ⌈n/b⌉ блоков размера b вместо ⌈n/5⌉

блоков размера 5, но во всём остальном идентично исходному алгоритму.

```
Mom_bSelect(A[1 .. n], k):
    если n ≤ b²
        использовать полный перебор
    иначе
        m ← ⌈n/b⌉
        для i ← 1 до m
            M[i] ← MedianOfB(A[b(i − 1) + 1 .. bi])
        mom_b ← Mom_bSelect(M[1 .. m], ⌊m/2⌋)
        r ← Partition(A[1 .. n], mom_b)
        если k < r
            вернуть Mom_bSelect(A[1 .. r − 1], k)
        иначе если k > r
            вернуть Mom_bSelect(A[r + 1 .. n], k − r)
        иначе
            вернуть mom_b
```

Рисунок 1.22. Параметризованное семейство алгоритмов выбора; см. задачу 19.

(a) Запишите рекуррентное соотношение для времени работы `Mom_bSelect`, предполагая, что b — константа (поэтому подпрограмма `MedianOfB` работает за O(1)). В частности, как размеры рекурсивных подзадач зависят от константы b? Рассмотрите чётные и нечётные b отдельно.

(b) Каково время работы `Mom_1Select` в худшем случае? [Подсказка: вопрос с подвохом.]

♣♥ (c) Каково время работы `Mom_2Select` в худшем случае? [Подсказка: это нечестный вопрос!]

♥ (d) Каково время работы `Mom_3Select` в худшем случае? Найти верхнюю оценку несложно; трудно показать, что она действительно точная. [Подсказка: см. задачу 10.]

♥ (e) Каково время работы `Mom_4Select` в худшем случае? И снова трудность состоит в доказательстве того, что оценку нельзя улучшить.[16]

(f) Для любой константы $b \geq 5$ алгоритм `Mom_bSelect` работает за O(n), но разные b дают разные постоянные множители. Пусть M(b) — минимальное число сравнений, необходимое для нахождения медианы b чисел. Точное значение M(b) известно только для $b \leq 13$:

b	1	2	3	4	5	6	7	8	9	10	11	12	13
M(b)	0	1	3	4	6	8	10	12	14	16	18	20	23

Для каждого b от 5 до 13 найдите верхнюю оценку времени работы `Mom_bSelect` вида $T(n) \leq \alpha_b\, n$ с явно заданной константой α_b. Например, на странице 39 мы показали $\alpha_5 \leq 16$.

(g) При каком b получается наименьшая константа α_b? [Подсказка: вопрос с подвохом!]

[16] Медиана четырёх элементов — либо второй наименьший, либо второй наибольший элемент. В 2014 году Кэ Чэнь и Адриан Думитреску доказали: если изменить `Mom_4Select` так, чтобы при $k < n/2$ находить вторые наименьшие элементы, а при $k > n/2$ — вторые наибольшие, полученный алгоритм работает за O(n)! Подробности см. в их статье «Select with Groups of 3 or 4 Takes Linear Time» (WADS 2015, arXiv:1409.3600).

20. Докажите, что вариант алгоритма выбора Блума—Флойда—Пратта—Ривеста—Тарьяна на рисунке 1.23, использующий дополнительный уровень малых медиан для выбора основного опорного элемента, работает за O(n).

```
MomomSelect(A[1 .. n], k):
    если n ≤ 81
        использовать полный перебор
    иначе
        m ← ⌈n/3⌉
        для i ← 1 до m
            M[i] ← MedianOf3(A[3i − 2 .. 3i])
        mm ← ⌈m/3⌉
        для j ← 1 до mm
            Mom[j] ← MedianOf3(M[3j − 2 .. 3j])
        momom ← MomomSelect(Mom[1 .. mm], ⌊mm/2⌋)
        r ← Partition(A[1 .. n], momom)
        если k < r
            вернуть MomomSelect(A[1 .. r − 1], k)
        иначе если k > r
            вернуть MomomSelect(A[r + 1 .. n], k − r)
        иначе
            вернуть momom
```

Рисунок 1.23. Выбор по медиане медиан медиан; см. задачу 20.

21. (a) Пусть даны два отсортированных массива A[1 .. n] и B[1 .. n]. Опишите алгоритм нахождения медианного элемента объединения A и B за Θ(log n). Можно считать, что массивы не содержат повторяющихся элементов.

(b) Пусть даны два отсортированных массива A[1 .. m] и B[1 .. n] и целое k. Опишите алгоритм нахождения k-го наименьшего элемента A ∪ B за Θ(log(m + n)). Например, при k = 1 алгоритм должен вернуть наименьший элемент A ∪ B. [Подсказка: используйте решение пункта (a).]

♥ (c) Теперь пусть даны три отсортированных массива A[1 .. n], B[1 .. n] и C[1 .. n] и целое k. Опишите алгоритм нахождения k-го наименьшего элемента A ∪ B ∪ C за O(log n).

(d) Наконец, пусть дан двумерный массив A[1 .. m, 1 .. n], каждая строка A[i, ·] которого отсортирована, и целое k. Опишите как можно более быстрый алгоритм нахождения k-го наименьшего элемента A. Как его время работы зависит от m? [Подсказка: сначала решите задачу 16.]

Арифметика

22. В 1854 году археологи обнаружили шумерские глиняные таблички, созданные около 2000 года до н. э., со списками квадратов целых чисел до 59. Это открытие побудило некоторых исследователей предположить, что древние шумеры выполняли умножение, сводя его к возведению в квадрат с помощью тождества вроде $x \cdot y = (x^2 + y^2 - (x - y)^2)/2$. К сожалению, те же исследователи умалчивают, как шумеры предположительно возводили в квадрат большие числа. Четыре тысячи лет спустя мы наконец можем избавить этих шумерских математиков от тяжёлого труда силой рекурсии!

(a) Опишите вариант алгоритма Карацубы, возводящий любое n-значное число в квадрат за O(n^(lg 3)) посредством сведения к возведению в квадрат трёх ⌈n/2⌉-значных чисел. (Карацуба действительно сделал это в 1960 году.)

(b) Опишите рекурсивный алгоритм, возводящий любое n-значное число в квадрат за O(n^(log3 6)) посредством сведения к возведению в квадрат шести ⌈n/3⌉-значных чисел.

♥ (c) Опишите рекурсивный алгоритм, возводящий любое n-значное число в квадрат за O(n^(log3 5)) посредством сведения к возведению в квадрат всего пяти чисел длины n/3 + O(1). [Подсказка: чему равно $(a + b + c)^2 + (a - b + c)^2$?]

23. (a) Опишите и проанализируйте вариант алгоритма Карацубы, умножающий любое m-значное число на любое n-значное число при n ≥ m за O(n m^(lg 3 – 1)).

(b) Опишите алгоритм вычисления десятичной записи 2^n за O(n^(lg 3)), использующий алгоритм из пункта (a) как подпрограмму. (Стандартный алгоритм, вычисляющий по одной цифре за раз, требует $\Theta(n^2)$.)

(c) Опишите алгоритм «разделяй и властвуй» для вычисления десятичной записи произвольного n-битного двоичного числа за O(n^(lg 3)). [Подсказка: избегайте дополнительного логарифмического множителя во времени работы.]

♥ (d) Пусть два n-значных числа можно умножить за O(M(n)). Опишите алгоритм вычисления десятичной записи произвольного n-битного двоичного числа за O(M(n) log n). [Подсказка: сложность — в анализе; используйте преобразование области определения.]

24. Рассмотрим классический рекурсивный алгоритм вычисления факториала n! неотрицательного целого n:

```
Factorial(n):
    если n = 0
        вернуть 1
    иначе
        вернуть n · Factorial(n – 1)
```

(a) Сколько умножений выполняет этот алгоритм?

(b) Сколько битов требуется для двоичной записи n!? Выразите ответ в виде Θ(f(n)) для некоторой знакомой функции f(n). [Подсказка: (n/2)^(n/2) < n! < n^n.]

(c) Ответ на пункт (b) должен убедить вас, что число умножений — плохая оценка фактического времени работы `Factorial`. Любое k-значное и любое ℓ-значное числа можно умножить за O(k · ℓ) с помощью алгоритма решёткой или удвоения и деления пополам. Каково время работы `Factorial`, если использовать такой алгоритм умножения как подпрограмму?

(d) Следующий рекурсивный алгоритм тоже вычисляет факториал, но группирует умножения по-другому:

```
Falling(n, m):   ⟪Вычислить n!/(n – m)!⟫
    если m = 0
```

```
        вернуть 1
    иначе если m = 1
        вернуть n
    иначе
        вернуть Falling(n, ⌊m/2⌋) · Falling(n – ⌊m/2⌋, ⌈m/2⌉)
```

Каково время работы `Falling(n, n)` при использовании школьного умножения? [Подсказка: как обычно, игнорируйте округления вверх и вниз.]

(e) Опишите и проанализируйте вариант алгоритма Карацубы, который умножает любое k-значное число на любое ℓ-значное при k ≥ ℓ за O(k · ℓ^(lg 3 – 1)) = O(k · ℓ^0,585).

♥ (f) Каково время работы `Factorial(n)` и `Falling(n, n)` при использовании изменённого умножения Карацубы из пункта (e)?

25. Наибольший общий делитель двух положительных целых x и y, обозначаемый gcd(x, y), — наибольшее целое d, на которое делятся и x, и y. В «Началах» Евклида, написанных около 300 года до н. э., описан следующий рекурсивный алгоритм вычисления gcd(x, y):[17]

```
EuclidGCD(x, y):
    если x = y
        вернуть x
    иначе если x > y
        вернуть EuclidGCD(x – y, y)
    иначе
        вернуть EuclidGCD(x, y – x)
```

(a) Докажите, что `EuclidGCD` правильно вычисляет gcd(x, y).[18] А именно:

i. Докажите, что `EuclidGCD(x, y)` делит и x, и y.

ii. Докажите, что каждый общий делитель x и y делит `EuclidGCD(x, y)`.

(b) Каково время работы `EuclidGCD(x, y)` в худшем случае как функция x и y? Считайте, что вычисление x – y требует O(log x + log y).

(c) Докажите, что следующий алгоритм тоже вычисляет gcd(x, y):

```
FastEuclidGCD(x, y):
    если y = 0
        вернуть x
    иначе если x > y
        вернуть FastEuclidGCD(y, x mod y)
    иначе
        вернуть FastEuclidGCD(x, y mod x)
```

(d) Каково время работы `FastEuclidGCD(x, y)` в худшем случае как функция x и y? Считайте, что вычисление x mod y занимает O(log x · log y).

(e) Докажите, что следующий алгоритм тоже вычисляет gcd(x, y):

```
BinaryGCD(x, y):
    если x = y
        вернуть x
    иначе если x и y оба чётны
        вернуть 2 · BinaryGCD(x/2, y/2)
    иначе если x чётно
        вернуть BinaryGCD(x/2, y)
    иначе если y чётно
        вернуть BinaryGCD(x, y/2)
    иначе если x > y
        вернуть BinaryGCD((x – y)/2, y)
    иначе
        вернуть BinaryGCD(x, (y – x)/2)
```

(f) Каково время работы `BinaryGCD(x, y)` в худшем случае как функция x и y? Считайте, что вычисление x – y занимает O(log x + log y), а вычисление z/2 — O(log z).

[17] Алгоритм Евклида иногда ошибочно называют старейшим рекурсивным или даже старейшим нетривиальным алгоритмом, хотя египетский алгоритм удвоения и деления пополам, который и нетривиален, и рекурсивен, предшествует Евклиду как минимум на 1500 лет.

[18] Евклид этого не делал. Предложение 1 книги VII «Начал» утверждает: если `EuclidGCD(x, y) = 1`, то x и y взаимно просты (то есть gcd(x, y) = 1), но доказательство рассматривает только частный случай x mod (y mod (x mod y)) = 1. Предложение 2 утверждает, что если x и y не взаимно просты, то `EuclidGCD(x, y) = gcd(x, y)`, но доказательство рассматривает только частные случаи gcd(x, y) = y и gcd(x, y) = y mod (x mod y). Наконец, эти два предложения не дают полного доказательства корректности `EuclidGCD`. Не поступайте как Евклид.

Массивы

26. Пусть дана шахматная доска $2^n \times 2^n$, из которой удалена одна произвольно выбранная клетка. Опишите и проанализируйте алгоритм покрытия доски без пропусков и наложений Г-образными плитками из трёх клеток. На вход подаются целое n и два n-битных целых числа, задающих строку и столбец отсутствующей клетки. Результат — список положений и ориентаций $(4^n – 1)/3$ плиток. Алгоритм должен работать за $O(4^n)$. [Подсказка: сначала докажите, что такое покрытие всегда существует.]

27. Вы посетитель политического съезда (или, возможно, собрания преподавателей) с n делегатами; каждый состоит ровно в одной политической партии. Невозможно определить партийную принадлежность делегата; более того, если вы спросите, вас немедленно выдворят со съезда. Однако можно определить, принадлежат ли любые два делегата одной партии, познакомив их друг с другом. Члены одной партии всегда приветствуют друг друга улыбками и дружескими рукопожатиями, а представители разных партий — сердитыми взглядами и оскорблениями.[19]

(a) Пусть больше половины делегатов принадлежат одной партии. Опишите эффективный алгоритм, выявляющий всех членов этой партии большинства.

(b) Пусть теперь партий больше двух, но одна партия имеет относительное большинство: в ней больше людей, чем в любой другой. Предложите практичную процедуру, позволяющую по возможности экономно точно определить всех представителей партии относительного большинства, предполагая, что в ней хотя бы p человек. Пожалуйста-препожалуйста.

[19] Реальная политика намного запутаннее этой упрощённой модели, но перед вами книга по теории!

28. На острове Смаллиана живут люди трёх типов: рыцари всегда говорят правду, лжецы всегда лгут, а обычные люди иногда говорят правду, а иногда нет. Все жители знают имя и тип каждого другого жителя. Вы хотите узнать тип каждого жителя.

Вы можете спросить любого жителя о типе любого другого. Если вы спросите: «Эй, X, к какому типу относится Y?», то X ответит следующим образом:

- Если X — рыцарь, он назовёт правильный тип Y.
- Если X — лжец, он может назвать любой из двух типов, к которым Y не относится.
- Если X — обычный человек, он может назвать любой из трёх типов.

Жители игнорируют вопросы любой другой формы; в частности, нельзя спрашивать человека о его собственном типе. Повторное обращение к одному и тому же жителю с одним и тем же вопросом всегда даёт тот же ответ, поэтому задавать вопрос более одного раза бессмысленно.

(a) Предположим, вы знаете, что строгое большинство жителей — рыцари. Опишите эффективный алгоритм определения типа каждого жителя.

(b) Докажите, что если рыцарей не больше половины жителей, определить тип каждого невозможно.

29. Большинство графических устройств поддерживает низкоуровневую операцию `blit`, или блочную передачу, которая быстро копирует прямоугольную область карты пикселей (двумерного массива значений пикселей) из одного места в другое. Это двумерный аналог стандартной функции C `memcpy()`.

Предположим, нужно повернуть карту n × n пикселей на 90° по часовой стрелке. Один из способов, по крайней мере когда n — степень двойки, состоит в том, чтобы разделить карту на четыре блока n/2 × n/2, переместить каждый блок на нужное место последовательностью из пяти операций `blit`, а затем рекурсивно повернуть каждый блок. (Почему пять? По той же причине, по которой Ханойской башне нужен третий стержень.) Можно и сначала рекурсивно повернуть блоки, а затем переместить их с помощью `blit`.

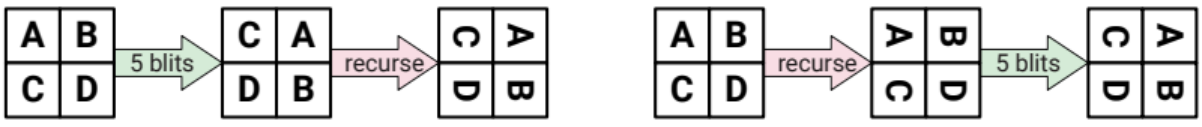


Рисунок 1.24. Два алгоритма поворота карты пикселей.

(a) Докажите корректность обеих версий алгоритма, когда n — степень двойки.

(b) Сколько в точности операций `blit` выполняет алгоритм, когда n — степень двойки?

(c) Опишите изменение алгоритма для произвольного n, а не только степени двойки. Сколько операций `blit` выполняет изменённый алгоритм?

(d) Каково время работы вашего алгоритма, если операция `blit` для блока k × k занимает $O(k^2)$?

(e) А если `blit` для блока k × k занимает только O(k)?

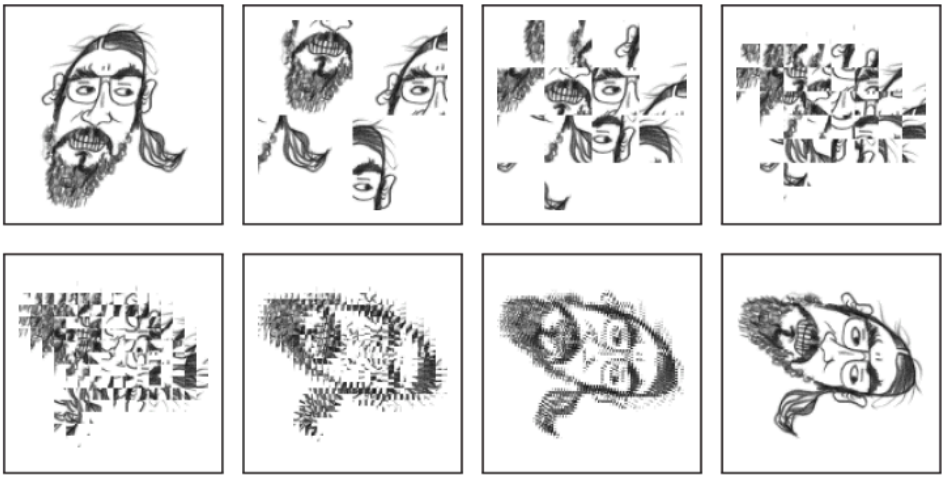


Рисунок 1.25. Первый алгоритм поворота (сначала `blit`, затем рекурсия) в действии. См. сведения об источниках иллюстраций в конце книги.

30. Массив A[0 .. n − 1] из n различных чисел называется битоническим, если существуют единственные индексы i и j, для которых A[(i − 1) mod n] < A[i] > A[(i + 1) mod n] и A[(j − 1) mod n] > A[j] < A[(j + 1) mod n]. Иными словами, битоническая последовательность либо состоит из возрастающей части, за которой следует убывающая, либо может быть циклически сдвинута в такой вид. Например:

```
4 6 9 8 7 5 1 2 3 — битоническая,
3 6 9 8 7 5 1 2 4 — не битоническая.
```

Опишите и проанализируйте алгоритм поиска наименьшего элемента в битоническом массиве из n элементов за O(log n). Можно считать числа входного массива различными.

31. Пусть дан массив A[1 .. n] из n различных целых чисел, которые могут быть положительными, отрицательными или нулём и отсортированы по возрастанию: A[1] < A[2] < ⋯ < A[n].

(a) Опишите быстрый алгоритм, который либо вычисляет индекс i такой, что A[i] = i, либо правильно сообщает, что такого индекса нет.

(b) Предположим, заранее известно, что A[1] > 0. Опишите ещё более быстрый алгоритм, который либо вычисляет индекс i такой, что A[i] = i, либо правильно сообщает, что его нет. [Подсказка: это очень просто.]

32. Пусть дан массив A[1 .. n] со специальным свойством: A[1] ≥ A[2] и A[n − 1] ≤ A[n]. Элемент A[x] называется локальным минимумом, если он не больше обоих соседей, то есть A[x − 1] ≥ A[x] и A[x] ≤ A[x + 1]. Например, в следующем массиве шесть локальных минимумов:

```
9 7 7 2 1 3 7 5 4 7 3 3 4 8 6 9
  ↑     ↑       ↑   ↑ ↑     ↑
```

Очевидно, локальный минимум можно найти за O(n), просмотрев массив. Опишите и проанализируйте алгоритм поиска локального минимума за O(log n). [Подсказка: при указанных граничных условиях в массиве обязательно есть хотя бы один локальный минимум. Почему?]

33. Пусть дан отсортированный массив из n различных чисел, который сдвинули на k позиций при неизвестном целом k от 1 до n − 1. Иными словами, дан массив A[1 .. n], у которого некоторый префикс A[1 .. k] отсортирован по возрастанию, соответствующий суффикс A[k + 1 .. n] отсортирован по возрастанию, а A[n] < A[1]. Например, может быть дан следующий массив из 16 элементов, где k = 10:

```
9 13 16 18 19 23 28 31 37 42 1 3 4 5 7 8
```

(a) Опишите и проанализируйте алгоритм вычисления неизвестного k.

(b) Опишите и проанализируйте алгоритм определения, содержит ли данный массив заданное число x.

34. В конце второго акта боевика «Форсаж и невыполнимая миссия XIII¾: Последние хранители неудержимой справедливости: Перезагрузка» злодей доктор Метафора гипнотизирует всю Лигу/Силу/Команду героев, выстраивает их в длинную шеренгу на краю обрыва и приказывает каждому по условному сигналу застрелить ближайших более высоких героев слева и справа.

Пусть рост всех n героев слева направо дан в массиве Ht[1 .. n]. Во избежание споров о гонорарах продюсеры настояли, чтобы рост всех героев различался. Тогда можно вычислить левую и правую цели каждого героя за $O(n^2)$ с помощью следующего алгоритма полного перебора:

```
WhoTargetsWhom(Ht[1 .. n]):
    для j ← 1 до n
        ⟪Найти левую цель L[j] для героя j⟫
        L[j] ← None
        для i ← 1 до j − 1
            если Ht[i] > Ht[j]
                L[j] ← i
        ⟪Найти правую цель R[j] для героя j⟫
        R[j] ← None
        для k ← n вниз до j + 1
            если Ht[k] > Ht[j]
```

```
                R[j] ← k
    вернуть L[1 .. n], R[1 .. n]
```

(a) Опишите алгоритм «разделяй и властвуй», вычисляющий результат `WhoTargetsWhom` за O(n log n).

(b) Докажите, что не менее ⌊n/2⌋ из n героев являются целями. То есть докажите, что выходные массивы R[0 .. n − 1] и L[0 .. n − 1] содержат не менее ⌊n/2⌋ различных значений, отличных от `None`.

(c) Увы, дьявольский план доктора Метафоры удался. По условному сигналу все герои одновременно стреляют в свои цели, и все цели падают с обрыва, по-видимому погибая. Метафора снова и снова повторяет свой злодейский эксперимент: после каждой бойни он заставляет оставшихся героев выбрать новые цели по тому же алгоритму, а затем стрелять по следующему сигналу. В конце концов в живых остаётся только самый низкий участник Героического экипажа/альянса/отряда.[20]

Опишите и проанализируйте алгоритм вычисления числа раундов до окончательного завершения смертельного процесса доктора Метафоры. Для полного зачёта алгоритм должен работать за O(n).

[20] В захватывающем финальном акте белка Реткон, последний выживший участник Команды/группы/общества героев, спасает всех, отправившись в прошлое и задним числом заменив остальных n − 1 героев реалистичными скульптурами из воздушных шаров. Так что да, в сущности это «Мстители: Финал».

35. Вы участник популярного телешоу «Победи соседей!». Перед вами сетка m × n из коробок, каждая с уникальным числом. Открыть коробку стоит 100 долларов. Ваша цель — найти коробку, число в которой больше чисел в соседних коробках сетки: сверху, снизу, слева и справа. Если вы потратите меньше денег, чем каждый из соперников, вы выиграете недельную поездку на двоих в Лас-Вегас и годовой запас Rice-A-Roni™, от которого безнадёжно зависимы.

(a) Пусть m = 1. Опишите алгоритм поиска числа, большего обоих соседей. Сколько коробок алгоритм открывает в худшем случае?

♥ (b) Пусть m = n. Опишите алгоритм поиска числа, большего всех соседей. Сколько коробок он открывает в худшем случае?

♣ ♥ (c) Докажите, что ваше решение пункта (b) оптимально с точностью до постоянного множителя.

36. (a) Пусть n = 2^ℓ − 1 для некоторого положительного целого ℓ. Некто утверждает, что обладает неотсортированным массивом A[1 .. n] различных ℓ-битных строк; значит, ровно одна ℓ-битная строка в A отсутствует. Кроме того, единственный способ доступа к A — вызов `FetchBit(i, j)`, возвращающий j-й бит строки A[i] за O(1). Опишите алгоритм поиска отсутствующей строки с помощью только O(n) вызовов `FetchBit`.

♥ (b) Теперь пусть n = 2^ℓ − k для положительных целых k и ℓ и снова дан массив A[1 .. n] различных ℓ-битных строк. Опишите алгоритм поиска k отсутствующих строк, использующий только O(n log k) вызовов `FetchBit`.

Деревья

37. В этой задаче поддерево двоичного дерева — любой связный подграф. Двоичное дерево называется совершенным, если у каждого внутреннего узла два потомка, а все листья имеют одинаковую глубину. Опишите и проанализируйте рекурсивный алгоритм вычисления наибольшего совершенного поддерева данного двоичного дерева. Алгоритм должен вернуть корень и глубину этого поддерева. Пример показан на рисунке 1.26.

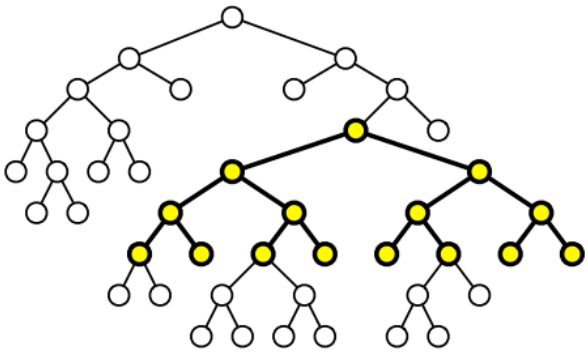


Рисунок 1.26. Наибольшее совершенное поддерево этого двоичного дерева имеет глубину 3.

38. Пусть T — двоичное дерево с n вершинами. Удаление любой вершины v разбивает T не более чем на три поддерева, содержащие левого потомка v (если есть), правого потомка v (если есть) и родителя v (если есть). Назовём v центральной вершиной, если каждое из этих меньших деревьев имеет не более n/2 вершин. Пример показан на рисунке 1.27.

Опишите и проанализируйте алгоритм поиска центральной вершины произвольного данного двоичного дерева. [Подсказка: сначала докажите, что у каждого дерева есть центральная вершина.]

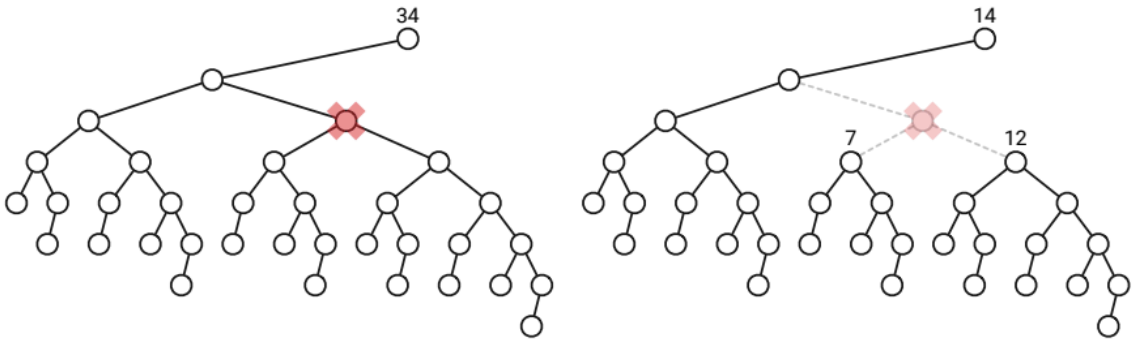


Рисунок 1.27. Удаление центральной вершины двоичного дерева из 34 узлов, после которого остаются поддеревья из 14, 7 и 12 узлов.

39. (a) У профессора Джорджа О'Джангла есть двоичное дерево из 27 узлов, каждый из которых помечен уникальной буквой латинского алфавита или символом &. Прямой и обратный обходы посещают узлы в следующем порядке:

- Прямой: `I Q J H L E M V O T S B R G Y Z K C A & F P N U D W X`.
- Обратный: `H E M L J V Q S G Y R Z B T C P U D N F W & X A K O I`.

Нарисуйте двоичное дерево Джорджа.

(b) Напомним: двоичное дерево называется полным в смысле full, если у каждого нелистового узла ровно два потомка.

i. Опишите и проанализируйте рекурсивный алгоритм восстановления произвольного такого полного двоичного дерева по последовательностям узлов при прямом и обратном обходах.

ii. Докажите, что не существует алгоритма восстановления произвольного двоичного дерева по его прямому и обратному обходам.

(c) Опишите и проанализируйте рекурсивный алгоритм восстановления произвольного двоичного дерева по последовательностям узлов при прямом и симметричном обходах.

(d) Опишите и проанализируйте рекурсивный алгоритм восстановления произвольного двоичного дерева поиска только по последовательности узлов при прямом обходе.

♥ (e) Опишите и проанализируйте рекурсивный алгоритм восстановления произвольного двоичного дерева поиска только по последовательности узлов при прямом обходе за O(n).

В пунктах (b)–(e) считайте все ключи различными, а входные данные совместимыми хотя бы с одним двоичным деревом.

40. Пусть внутри двумерного прямоугольника расположены n точек. kd-дерево[21] рекурсивно разбивает точки следующим образом. Если внутри прямоугольника нет точек, работа закончена. Иначе мы делим прямоугольник на два меньших вертикальной прямой, проходящей через медианную точку внутри прямоугольника (не на его границе), распределяя точки как можно более равномерно. Затем мы рекурсивно строим kd-дерево для точек каждого из двух меньших прямоугольников, предварительно повернув их на 90 градусов. Таким образом, на каждом уровне рекурсии мы чередуем вертикальное и горизонтальное разделение. Получившиеся пустые прямоугольники называются ячейками.

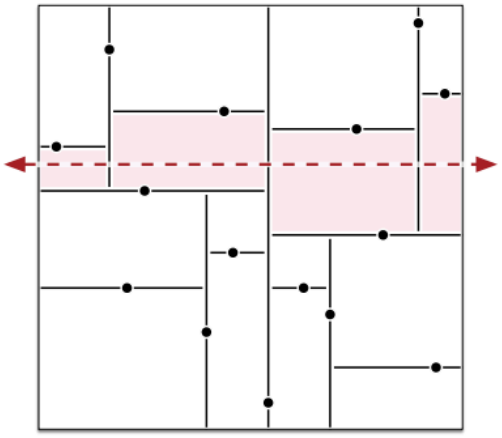


Рисунок 1.28. kd-дерево для 15 точек. Пунктирная прямая пересекает четыре закрашенные ячейки.

(a) Сколько всего ячеек как функция n? Докажите правильность ответа.

(b) Каково в точности максимальное число ячеек, которое может пересечь горизонтальная прямая, как функция n? Докажите правильность ответа. Считайте, что $n = 2^k - 1$ для некоторого целого k. [Подсказка: существует больше одной функции f, для которой f(16) = 4.]

(c) Пусть даны n точек, хранящихся в kd-дереве. Опишите и проанализируйте максимально быстрый алгоритм подсчёта точек выше горизонтальной прямой (например, пунктирной на рисунке). [Подсказка: используйте пункт (b).]

(d) Опишите и проанализируйте эффективный алгоритм, который по kd-дереву из n точек подсчитывает число точек внутри прямоугольника R с горизонтальными и вертикальными сторонами. [Подсказка: используйте пункт (c).]

[21] Термин «kd-дерево» (произносится «кей-ди-дерево») первоначально был сокращением от «k-мерное дерево», но современное употребление игнорирует эту этимологию, отчасти потому, что никто в здравом уме не стал бы обозначать размерность буквой k вместо очевидно превосходящей её d. Этимологическая последовательность требовала бы назвать структуру в этой задаче «2d-деревом», но общепринятое название теперь — «двумерное kd-дерево». См. также: B-tree, возможно; alpha shape; beta skeleton; epsilon net; река Потомак; река Миссисипи; озеро Мичиган; озеро Тахо; остров Манхэттен; асфальтовые озёра Ла-Бреа; пустыня Сахара; гора Килиманджаро; Южный Вьетнам; Восточный Тимор; галактика Млечный Путь; город Таунсвилл; и самоуправляемые автомобили.

♥ 41. Боб Ратенбур, новый студент курса CS 225, пытается написать код прямого, симметричного и обратного обходов двоичных деревьев. Он вроде бы понимает основную идею обходов, но каждый раз при реализации путает рекурсивные вызовы. За пять минут до срока сдачи Боб поспешно отправляет код следующей структуры:

```
PreOrder(v):
    если v = Null
        вернуть
    иначе
        вывести label(v)
        ?Order(left(v))
        ?Order(right(v))

InOrder(v):
    если v = Null
        вернуть
    иначе
        ?Order(left(v))
        вывести label(v)
        ?Order(right(v))

PostOrder(v):
    если v = Null
        вернуть
    иначе
        ?Order(left(v))
        ?Order(right(v))
        вывести label(v)
```

Каждый знак вопроса в псевдокоде скрывает один из префиксов `Pre`, `In` или `Post`. Более того, каждый из следующих вызовов встречается в сданном Бобом коде ровно один раз:

```
PreOrder(left(v))   PreOrder(right(v))
InOrder(left(v))    InOrder(right(v))
PostOrder(left(v))  PostOrder(right(v))
```

Следовательно, существует ровно 36 возможных вариантов кода Боба. К сожалению, после отправки исполняемого файла Боб случайно удалил исходный код, так что ни вы, ни он не знаете, какие функции где вызывались.

Пусть даны результаты алгоритмов обхода Боба для некоторого неизвестного двоичного дерева T. Результат удобно разобран в три массива Pre[1 .. n], In[1 .. n] и Post[1 .. n]. Можно считать, что эти последовательности обхода соответствуют ровно одному двоичному дереву T; в частности, метки вершин неизвестного дерева различны, а каждый внутренний узел имеет ровно двух потомков.

(a) Опишите алгоритм восстановления неизвестного дерева T по данным последовательностям обхода.

(b) Опишите алгоритм, который либо восстанавливает код Боба по данным последовательностям обхода, либо правильно сообщает, что последовательности совместимы более чем с одним набором алгоритмов.

Например, для входа

```
Pre[1 .. n]  = [H A E C B I F G D]
In[1 .. n]   = [A H D C E I F B G]
Post[1 .. n] = [A E I B F C D G H]
```

первый алгоритм должен вернуть следующее дерево:

```
        H
       / \
      A   D
         / \
        C   G
       / \
      E   B
         / \
        I   F
```

а второй должен восстановить такой код:

```
PreOrder(v):
    если v = Null
        вернуть
    иначе
        вывести label(v)
        PreOrder(left(v))
        PostOrder(right(v))

InOrder(v):
    если v = Null
        вернуть
    иначе
        PostOrder(left(v))
        вывести label(v)
        PreOrder(right(v))

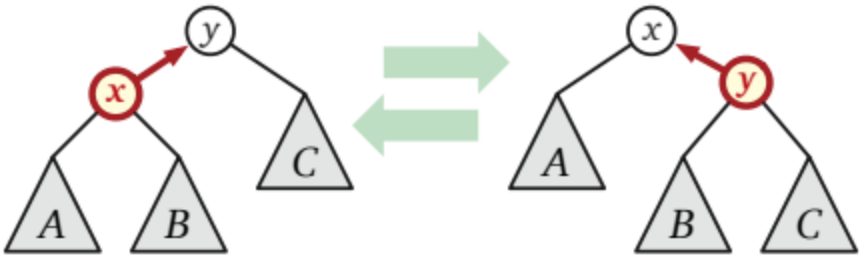
PostOrder(v):
    если v = Null
        вернуть
    иначе
        InOrder(left(v))
        InOrder(right(v))
        вывести label(v)
```

♥ 42. Пусть T — двоичное дерево, узлы которого содержат различные числовые значения. Напомним, что T является двоичным деревом поиска тогда и только тогда, когда (1) T пусто либо (2) выполнены следующие рекурсивные условия:

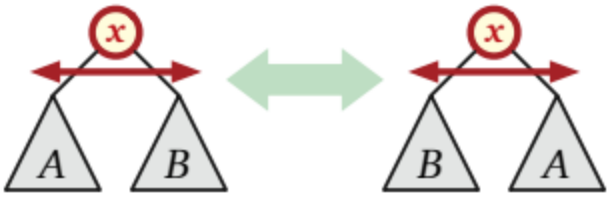
- Левое поддерево T — двоичное дерево поиска.
- Все значения левого поддерева меньше значения в корне.
- Правое поддерево T — двоичное дерево поиска.
- Все значения правого поддерева больше значения в корне.

Рассмотрим две операции над двоичными деревьями:

- Поворот произвольного узла вверх.[22]



- Обмен левого и правого поддеревьев произвольного узла.



В обеих операциях некоторые, все или ни одно из поддеревьев A, B и C могут быть пустыми.

(a) Опишите алгоритм преобразования произвольного двоичного дерева из n узлов с различными значениями в двоичное дерево поиска, используя не более $O(n^2)$ поворотов и обменов. На рисунке 1.29 показана последовательность восьми операций, преобразующая дерево из пяти узлов в двоичное дерево поиска.

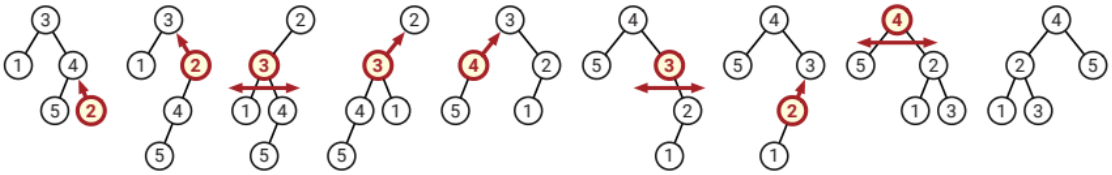


Рисунок 1.29. «Сортировка» двоичного дерева: повернуть 2, повернуть 2, обменять 3, повернуть 3, повернуть 4, обменять 3, повернуть 2, обменять 4.

Ваш алгоритм не может непосредственно изменять указатели на родителей или потомков, создавать новые или удалять старые узлы; единственный способ изменить дерево — повороты и обмены.

С другой стороны, вы можете бесплатно выполнять любые вычисления, если они не изменяют дерево; время работы алгоритма определяется числом выполненных поворотов и обменов.

♥ (b) Опишите алгоритм преобразования произвольного двоичного дерева из n узлов в двоичное дерево поиска не более чем за $O(n \log n)$ поворотов и обменов.

(c) Докажите, что любое двоичное дерево поиска из n узлов можно преобразовать в любое другое двоичное дерево поиска с теми же значениями узлов, используя только $O(n)$ поворотов и ни одного обмена.

♥ (d) Открытая задача: опишите алгоритм преобразования произвольного двоичного дерева из n узлов в двоичное дерево поиска только с помощью $O(n)$ поворотов и обменов либо докажите

невозможность такого алгоритма. [Подсказка: я не думаю, что это возможно.]

[22] Повороты сохраняют порядок узлов при симметричном обходе двоичного дерева. Отчасти поэтому они используются для поддержки нескольких видов сбалансированных двоичных деревьев поиска, включая АВЛ-деревья, красно-чёрные деревья, splay-деревья, scapegoat-деревья и декартовы деревья. Конспекты по большинству этих структур см. на http://algorithms.wtf.

2. Поиск с возвратом

Однако там, где неоднозначность нельзя устранить ни правилом веры, ни контекстом, ничто не мешает нам расставить знаки препинания любым из предлагающихся способов.

— Августин Гиппонский, О христианском учении (397 г. н. э.), перевод Маркуса Додса (1892)

Я бросил обед и побежал обратно в лабораторию. Там, в волнении, я попробовал содержимое каждого стакана и каждой выпарительной чашки на столе. К счастью, ни в одной не оказалось едкой или ядовитой жидкости.

— Константин Фальберг об открытии сахарина, Scientific American (1886)

Самая трудная задача для любого мыслителя — сформулировать проблему так, чтобы её можно было решить.

— приписывается Бертрану Расселу

Дойдя до развилки, сворачивай.

— Йоги Берра, объясняя дорогу к своему дому

В этой главе описана ещё одна важная рекурсивная стратегия — *поиск с возвратом* (backtracking). Алгоритм с возвратом пытается построить решение вычислительной задачи постепенно, добавляя по одному небольшому элементу. Всякий раз, когда нужно выбрать между несколькими вариантами следующего элемента решения, алгоритм рекурсивно оценивает каждый вариант и выбирает лучший.

2.1. N ферзей

Классический пример задачи для поиска с возвратом — задача о n ферзях. Впервые её предложил немецкий любитель шахмат Макс Безцель в 1848 году под псевдонимом «Schachfreund» для стандартной доски 8×8, а затем Франсуа-Жозеф Эсташ Лионне в 1869 году — для общего случая доски $n \times n$. Нужно расставить n ферзей на доске $n \times n$ так, чтобы никакие два ферзя не били друг друга. Для читателей, не знакомых с шахматами: это значит, что никакие два ферзя не стоят в одной строке, одном столбце или на одной диагонали.

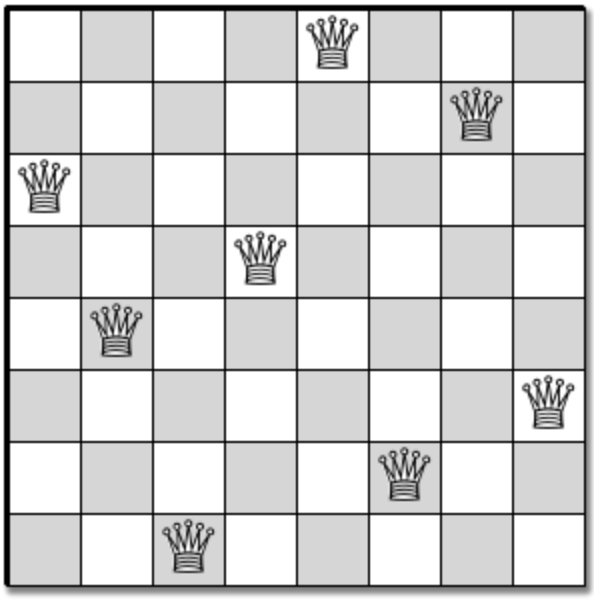


Рисунок 2.1. Первое решение Гаусса задачи о восьми ферзях, представленное массивом `[5, 7, 1, 4, 2, 8, 6, 3]`.

В письме другу Генриху Шумахеру в 1850 году выдающийся математик Карл Фридрих Гаусс писал, что утверждение Франца Наука о 92 решениях задачи о восьми ферзях можно легко проверить методом проб и ошибок за несколько часов: «Schwer ist es übrigens nicht, durch ein methodisches Tatonniren sich diese Gewissheit zu verschaffen, wenn man 1 oder ein paar Stunden daran wenden will». Употреблённое им слово *Tatonniren* происходит от французского *tâtonner* — ощупывать, двигаться на ощупь или беспомощно шарить, словно в темноте.

Письмо Гаусса описывало следующую рекурсивную стратегию решения задачи о n ферзях. Ту же стратегию в 1882 году описал французский занимательный математик Эдуар Люка, приписав её Эмманюэлю Лакьеру. Мы ставим ферзей на доску по одной строке, начиная с верхней. Чтобы поставить r-го ферзя, мы методично перебираем все n клеток строки r слева направо простым циклом `for`. Если клетка находится под боем ранее поставленного ферзя, мы её пропускаем; иначе предварительно ставим туда ферзя и рекурсивно ищем на ощупь допустимые положения ферзей в следующих строках.

На рисунке 2.2 приведён получившийся алгоритм, который рекурсивно перечисляет все полные решения задачи о n ферзях, совместимые с данным частичным решением. Следуя Гауссу, представим положения ферзей массивом `Q[1 .. n]`, где `Q[i]` указывает, в какой клетке строки i стоит ферзь. При вызове `PlaceQueens` входной параметр r — индекс первой пустой строки, а префикс `Q[1 .. r - 1]` содержит позиции первых $r - 1$ ферзей. В частности, чтобы найти все решения без ограничений, нужно вызвать `PlaceQueens(Q[1 .. n], 1)`. Внешний цикл перебирает все возможные позиции ферзя в строке r; внутренний проверяет, совместима ли рассматриваемая позиция с ферзями в первых $r - 1$ строках.

```
PlaceQueens(Q[1 .. n], r):
    если r = n + 1
        вывести Q[1 .. n]
```

```
    иначе
        для j ← 1 до n
            legal ← True
            для i ← 1 до r − 1
                если (Q[i] = j) или (Q[i] = j + r − i) или (Q[i] = j − r + i)
                    legal ← False
            если legal
                Q[r] ← j
                PlaceQueens(Q[1 .. n], r + 1)  ⟨⟨Рекурсия!⟩⟩
```

Рисунок 2.2. Алгоритм Гаусса и Лакьера для задачи о n ферзях, использующий поиск с возвратом.

Работу `PlaceQueens` можно изобразить деревом рекурсии. Каждый узел соответствует рекурсивной подзадаче, а значит, допустимому частичному решению; в частности, корню соответствует пустая доска (при $r = 0$). Рёбра дерева соответствуют рекурсивным вызовам. Листья соответствуют частичным решениям, которые нельзя продолжить: либо ферзь уже стоит в каждой строке, либо каждая клетка следующей пустой строки находится под боем ранее поставленного ферзя. Поиск полных решений с возвратом эквивалентен обходу этого дерева в глубину.

> *Примечание переводчика: в тексте оригинала для корня указано $r = 0$, но в приведённом алгоритме первый вызов выполняется с $r = 1$: именно это значение соответствует пустой доске.*

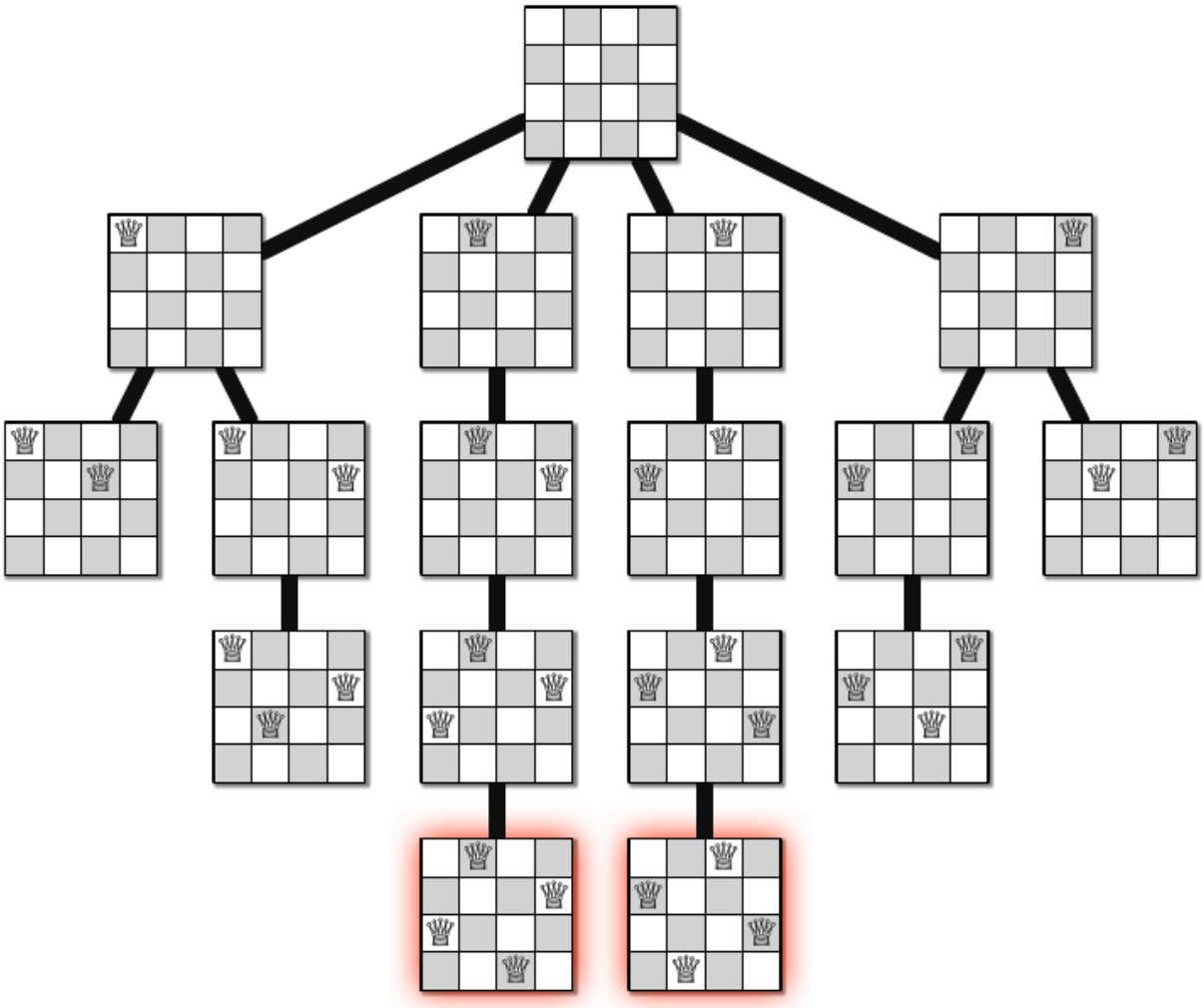


Рисунок 2.3. Полное дерево рекурсии алгоритма Гаусса и Лакьера для задачи о четырёх ферзях.

2.2. Игровые деревья

Рассмотрим простую игру для двух игроков[1] на квадратном поле $n \times n$ с рамкой из клеток. Назовём игроков Гораций Фальберг-Ремсен и Вера Ребауди.[2] У каждого по n фишек; игроки перемещают их с одной стороны доски на другую. Фишки Горация начинают на левой границе, по одной в каждой строке, и перемещаются горизонтально вправо; фишки Веры симметрично начинают на верхней границе, по одной в каждом столбце, и движутся вертикально вниз.

Игроки ходят по очереди. За ход Гораций либо передвигает одну свою фишку на одну клетку вправо на свободное место, либо перепрыгивает одной фишкой ровно через одну фишку Веры на свободную клетку, расположенную на два шага вправо. Если допустимых ходов или прыжков нет, Гораций просто пропускает ход. Аналогично, Вера за свой ход передвигает или перепрыгивает одной из своих фишек вниз, если только у неё нет допустимых ходов. Побеждает тот, кто первым выведет все свои фишки за край доски. Нетрудно доказать, что пока

на доске остаются фишки, хотя бы один игрок может сделать допустимый ход, и потому кто-то в итоге победит.

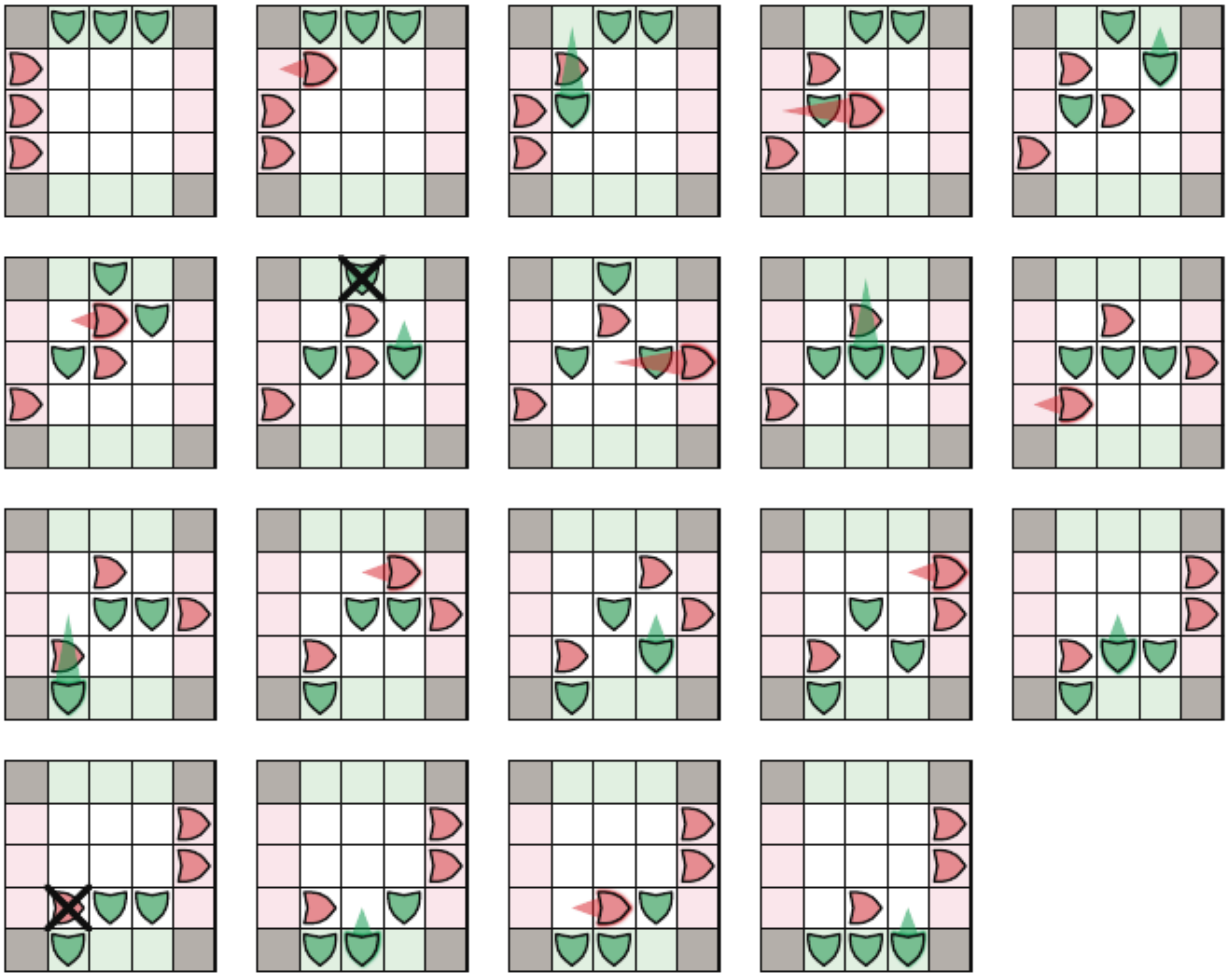


Рисунок 2.4. Вера выигрывает игру с пакетиками заменителя сахара на поле 3×3.

[1] Я не знаю названия этой игры и даже не уверен, что правильно передаю её правила. Я узнал её или нечто похожее от Ленни Питта, который рекомендовал играть пакетиками заменителя сахара в ресторанах.

[2] Константин Фальберг и Айра Ремсен впервые синтезировали сахарин в 1878 году, когда Фальберг работал в лаборатории Ремсена и изучал производные каменноугольной смолы. В 1900 году Овидио Ребауди опубликовал первый химический анализ каа-хеэ — лекарственного растения, которое гуарани выращивали более 1500 лет и которое сейчас чаще называют *Stevia rebaudiana*.

Если вы раньше не видели эту игру,[3] то, вероятно, не представляете, как хорошо в неё играть. Тем не менее существует сравнительно простой алгоритм с возвратом, который может играть безупречно — в неё или в любую другую игру двух игроков без случайности и скрытой информации, заканчивающуюся после конечного числа ходов. Иными словами, если поместить

вас посреди игры и победить другого идеального игрока возможно, алгоритм сообщит, как это сделать.

Состояние игры состоит из положения всех фишек и указания, чей сейчас ход. Эти состояния можно связать в *игровое дерево*: в нём есть ребро из состояния x в состояние y тогда и только тогда, когда игрок, чей ход в состоянии x, может по правилам перейти в y. Корень — начальная позиция, а каждый путь от корня до листа — полная партия.

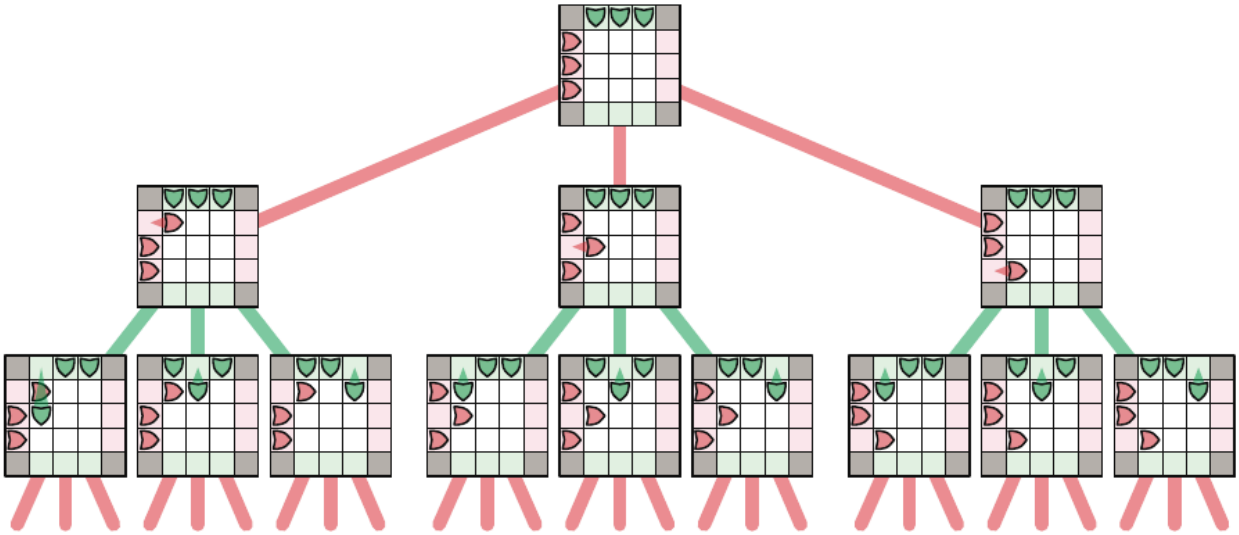


Рисунок 2.5. Первые два уровня игрового дерева игры с пакетиками заменителя сахара.

Для перемещения по этому дереву рекурсивно определим хорошее и плохое состояния:

- Состояние *хорошее*, если текущий игрок уже выиграл либо может перейти в плохое состояние для соперника.
- Состояние *плохое*, если текущий игрок уже проиграл либо каждый допустимый ход ведёт в хорошее состояние для соперника.

Иными словами, внутренний узел игрового дерева хорош, если у него есть хотя бы один плохой потомок, и плох, если все его потомки хороши. По индукции любой игрок, который застал хорошее состояние в свой ход, может победить, даже если соперник играет идеально; напротив, из плохого состояния победить можно, только если соперник ошибётся. Это рекурсивное определение предложил Эрнст Цермело в 1913 году.[4]

[3] Если видели, пожалуйста, расскажите мне где!

[4] На самом деле Цермело рассматривал более тонкий класс игр с конечным числом состояний, допускающих бесконечные последовательности ходов. Бесконечную партию он определял как ничью.

Из этого рекурсивного определения непосредственно следует алгоритм с возвратом, определяющий, хорошее или плохое заданное состояние. По сути это просто обход игрового дерева в глубину; иначе говоря, игровое дерево является деревом рекурсии алгоритма!

Несложное изменение алгоритма позволяет найти хороший ход или даже все возможные хорошие ходы, если на вход дано хорошее состояние.

```
PlayAnyGame(X, player):
    если player уже выиграл в состоянии X
        вернуть Good
    если player уже проиграл в состоянии X
        вернуть Bad
    для всех допустимых ходов X → Y
        если PlayAnyGame(Y, ¬player) = Bad
            вернуть Good  ⟨⟨X → Y — хороший ход⟩⟩
    вернуть Bad  ⟨⟨Хороших ходов нет⟩⟩
```

Все игровые программы в конечном счёте основаны на этой простой стратегии поиска с возвратом. Однако у большинства игр огромное число состояний, поэтому на практике обойти всё дерево невозможно. Вместо этого программы используют другие эвристики[5] для отсечения ветвей: игнорируют состояния, которые очевидно или «очевидно» хороши или плохи либо хотя бы лучше или хуже других, и/или обрезают дерево на определённой глубине — числе полуходов — и оценивают листья более эффективной эвристикой.

[5] Эвристика — алгоритм, который не работает. Кроме как на практике. Иногда. Возможно.

2.3. Сумма подмножества

Рассмотрим более сложную задачу `SubsetSum`: дано множество положительных целых чисел X и целое целевое значение T. Есть ли подмножество X, сумма элементов которого равна T? Таких подмножеств может быть несколько. Например, при $X = \{8, 6, 7, 5, 3, 10, 9\}$ и $T = 15$ ответ — `True`, поскольку каждое из подмножеств $\{8, 7\}$, $\{7, 5, 3\}$, $\{6, 9\}$ и $\{5, 10\}$ имеет сумму 15. С другой стороны, при $X = \{11, 6, 5, 1, 7, 13, 12\}$ и $T = 15$ ответ — `False`.

Есть два тривиальных случая. Если $T = 0$, можно сразу вернуть `True`: пустое множество является подмножеством любого X, а сумма его элементов равна нулю.[6] С другой стороны, если $T < 0$ или $T \neq 0$, но X пусто, можно сразу вернуть `False`.

В общем случае рассмотрим произвольный элемент $x \in X$; случай пустого X мы уже обработали. Подмножество X с суммой T существует тогда и только тогда, когда верно одно из двух утверждений:

- существует подмножество X, содержащее x и имеющее сумму T;
- существует подмножество X, не содержащее x и имеющее сумму T.

В первом случае должно существовать подмножество $X \setminus \{x\}$ с суммой $T - x$, во втором — подмножество $X \setminus \{x\}$ с суммой T. Поэтому `SubsetSum(X, T)` сводится к двум более простым задачам: `SubsetSum(X \ {x}, T - x)` и `SubsetSum(X \ {x}, T)`. Получившийся алгоритм:

```
⟨⟨Есть ли подмножество X с суммой T?⟩⟩
SubsetSum(X, T):
    если T = 0
        вернуть True
```

```
    иначе если T < 0 или X = ∅
        вернуть False
    иначе
        x ← любой элемент X
        with ← SubsetSum(X \ {x}, T - x)  ⟨⟨Рекурсия!⟩⟩
        wout ← SubsetSum(X \ {x}, T)      ⟨⟨Рекурсия!⟩⟩
        вернуть (with ∨ wout)
```

[6] ...а чему ещё могла бы быть равна эта сумма?

Корректность

Доказательство корректности — прямое упражнение на индукцию. Если $T = 0$, сумма элементов пустого подмножества равна T, поэтому правильный ответ — `True`. Иначе, если T отрицательно или X пусто, ни одно подмножество X не даёт в сумме T, и правильный ответ — `False`. В остальных случаях, если подходящее подмножество существует, оно либо содержит `X[n]`, либо нет, и Фея Рекурсии правильно проверяет обе возможности. Готово.

Анализ

Для анализа алгоритма нужно точнее описать некоторые детали реализации. Сначала предположим, что входное множество X задано массивом `X[1 .. n]`.

Предыдущий рекурсивный алгоритм позволяет в основном случае выбирать любой элемент $x \in X$. Исключительно ради эффективности полезно выбрать x так, чтобы оставшееся подмножество $X \setminus \{x\}$ имело краткое описание, вычисляемое быстро: тогда подготовка рекурсивных вызовов требует минимальных накладных расходов. Выберем последний элемент `X[n]`; тогда оставшееся подмножество хранится в префиксе `X[1 .. n - 1]`. Передавать рекурсивным вызовам полную копию этого префикса слишком долго: одно копирование требует $\Theta(n)$ времени. Вместо этого передадим только два значения: ссылку на массив или его начальный адрес и длину префикса. Другой вариант — сделать X глобальной переменной, чтобы не передавать ссылку при каждом вызове.

```
⟨⟨Есть ли подмножество X[1 .. i] с суммой T?⟩⟩
SubsetSum(X, i, T):
    если T = 0
        вернуть True
    иначе если T < 0 или i = 0
        вернуть False
    иначе
        with ← SubsetSum(X, i - 1, T - X[i])  ⟨⟨Рекурсия!⟩⟩
        wout ← SubsetSum(X, i - 1, T)         ⟨⟨Рекурсия!⟩⟩
        вернуть (with ∨ wout)
```

При таком выборе реализации время работы $T(n)$ удовлетворяет рекуррентному соотношению $T(n) \leq 2T(n-1) + O(1)$. Решение $T(n) = O(2^n)$ легко получить с помощью дерева рекурсии или ещё более простого метода: «Ах да, мы уже решили это соотношение для Ханойской башни». В худшем случае — например, когда T больше суммы всех элементов X, —

дерево рекурсии является полным двоичным деревом глубины n, и алгоритм рассматривает все 2^n подмножеств X.

Варианты

Небольшие изменения позволяют решить несколько вариантов `SubsetSum`. Например, алгоритм на рисунке 2.6 действительно строит подмножество X с суммой T, если такое существует, либо возвращает значение ошибки `None` в противном случае. Он использует точно такую же рекурсивную стратегию, что и предыдущие алгоритмы принятия решения. Его время работы также равно $O(2^n)$. Анализ проще всего, если предположить структуру множества со вставкой элемента за $O(1)$, например связный список; однако время остаётся $O(2^n)$, даже если вставка требует $O(n)$, как в отсортированном связном списке. Аналогичные варианты позволяют подсчитать подмножества с заданной суммой или выбрать лучшее по некоторому другому критерию.

Большинство других задач, решаемых поиском с возвратом, обладает тем же свойством: одна рекурсивная стратегия позволяет решить множество вариантов задачи. Например, стратегию из предыдущего раздела, определяющую, хороша или плоха игровая позиция, легко изменить так, чтобы она возвращала хороший ход или список всех хороших ходов. Поэтому при проектировании алгоритмов с возвратом следует стремиться к простейшему варианту задачи: вычислять число или даже единственное логическое значение вместо более сложной информации или структуры.

```
⟨⟨Вернуть подмножество X[1 .. i] с суммой T⟩⟩
⟨⟨или None, если такого подмножества нет⟩⟩
ConstructSubset(X, i, T):
    если T = 0
        вернуть ∅
    если T < 0 или n = 0
        вернуть None
    Y ← ConstructSubset(X, i – 1, T)
    если Y ≠ None
        вернуть Y
    Y ← ConstructSubset(X, i – 1, T – X[i])
    если Y ≠ None
        вернуть Y ∪ {X[i]}
    вернуть None
```

Рисунок 2.6. Рекурсивный алгоритм с возвратом для варианта `SubsetSum`, в котором требуется построить подмножество.

> *Примечание переводчика: в условии* `n = 0` *на рисунке оригинала, вероятно, опечатка: параметр функции —* `i`*, и для окончания рекурсии нужно проверять* `i = 0`.

2.4. Общая схема

Алгоритмы с возвратом часто используются для последовательного принятия решений с целью построить рекурсивно определённую структуру, удовлетворяющую заданным ограничениям.

Часто, хотя и не всегда, эта структура сама является последовательностью. Например:

- В задаче о n ферзях цель — последовательность позиций ферзей, по одной на строку, причём никакие два не бьют друг друга. Для каждой строки алгоритм решает, куда поставить ферзя.
- В задаче об игровом дереве цель — последовательность допустимых ходов, каждый из которых как можно лучше для соответствующего игрока. Для каждого состояния алгоритм выбирает наилучший следующий ход.
- В задаче `SubsetSum` цель — последовательность входных элементов с заданной суммой. Для каждого элемента алгоритм решает, включать ли его в выходную последовательность.

Постойте, почему цель задачи о сумме подмножества — найти последовательность? Это сознательное проектное решение. Мы ввели удобный порядок на входном множестве, представив его массивом вместо какой-либо более бесформенной структуры данных; этот порядок можно использовать в рекурсивном алгоритме.

В каждом рекурсивном вызове алгоритма с возвратом нужно принять ровно одно решение, согласованное со всеми предыдущими. Поэтому каждому вызову необходимы не только ещё не обработанные входные данные, но и подходящее краткое описание уже принятых решений. Ради эффективности это описание должно быть как можно меньше. Например:

- Для задачи о n ферзях необходимо передать не только количество пустых строк, но и позиции всех ранее поставленных ферзей. К сожалению, здесь приходится помнить прошлые решения полностью.
- Для игрового дерева достаточно передать текущее состояние игры, включая указание следующего игрока. Ничего о предыдущих решениях помнить не нужно: победитель из данного состояния не зависит от ходов, которые к нему привели.[7]
- Для `SubsetSum` нужно передать оставшиеся числа и оставшееся целевое значение, то есть исходную цель за вычетом суммы уже выбранных элементов. Какие именно элементы были выбраны, неважно.

При разработке новых рекурсивных алгоритмов с возвратом необходимо заранее определить, какая информация о прошлых решениях понадобится в середине работы. Если эта информация нетривиальна, рекурсивному алгоритму может потребоваться решать более общую задачу, чем исходная. Мы уже встречали подобное обобщение: чтобы найти медиану неотсортированного массива за линейное время, мы вывели алгоритм выбора k-го наименьшего элемента для произвольного k.

Наконец, выяснив, какую рекурсивную задачу действительно нужно решить, решаем её рекурсивным полным перебором: пробуем все варианты следующего решения, согласованные с предыдущими, а об остальном пусть беспокоится Фея Рекурсии. Никаких хитростей. Никакого пропуска «очевидно» глупых вариантов. Пробуйте всё. Ускорить алгоритм можно позже.

[7] Многие игры кажутся нарушающими это условие независимости. Например, стандартные правила шахмат и шашек позволяют объявить ничью при троекратном повторении одной расстановки фигур, а китайские правила го запрещают повторять любую из прежних

расстановок камней. Поэтому для таких игр состояние формально включает не только текущие положения фигур, но и всю историю предыдущих ходов.

2.5. Разбиение текста на слова (*Interpunctio Verborum*)

Предположим, дана строка букв, представляющая текст на некотором иностранном языке без пробелов и знаков препинания. Нужно разбить её на отдельные слова. Например, вам может встретиться следующий фрагмент знаменитой речи Цицерона в защиту Луция Лициния Мурены в 62 году до н. э., записанный традиционным для классической латыни способом *scriptio continua*:[8]

```
PRIMVSDIGNITASINTAMTENVISCIENTIANONPOTEST
ESSERESENIMSVNTPARVAEPROPEINSINGVLISLITTERIS
ATQVEINTERPVNCTIONIBUSVERBORVMOCCVPATAE
```

Человек, свободно читающий на латыни, разобьёт строку следующим образом, используя современную орфографию: *Primus dignitas in tam tenui scientia non potest esse; res enim sunt parvae, prope in singulis litteris atque interpunctionibus verborum occupatae*.[9] Разбиение текста важно не только для классических латыни и греческого, но и для ряда современных языков и письменностей, включая балийский, бирманский, китайский, японский, яванский, кхмерский, лаосский, тайский, тибетский и вьетнамский. Похожие задачи возникают при разбиении английского текста без знаков препинания на предложения,[10] разбиении текста на строки при наборе, распознавании речи и рукописного текста, упрощении кривых и нескольких видах анализа временных рядов. Для примера я ограничусь разбиением последовательностей букв современного английского алфавита на современные английские слова.

Некоторые строки, конечно, допускают несколько разбиений. Например, `BOTHEARTHANDSATURNSPIN` можно разбить на английские слова как `BOTH·EARTH·AND·SATURN·SPIN` или `BOT·HEART·HANDS·AT·URNS·PIN`, не говоря о множестве других вариантов. Пока рассмотрим предельно простую задачу: можно ли вообще разбить заданную строку символов на английские слова?

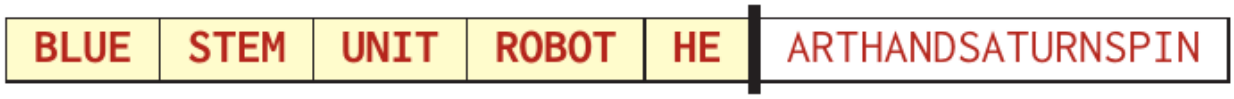
Чтобы сделать задачу конкретной и независимой от языка, предположим, что у нас есть подпрограмма `IsWord(w)`, которая получает строку w и возвращает `True`, если w — «слово», и `False` в противном случае. Например, если мы разбиваем входную строку на палиндромы, «слово» означает «палиндром»; тогда `IsWord(ROTATOR) = True`, а `IsWord(PALINDROME) = False`.

Как и в `SubsetSum`, входная структура — последовательность, на этот раз букв, а не чисел. Поэтому естественно рассмотреть процесс принятия решений, последовательно обрабатывающий символы слева направо. Выходная структура тоже является последовательностью слов, поэтому естественно строить её слева направо. Представим, что мы находимся посреди этого процесса:

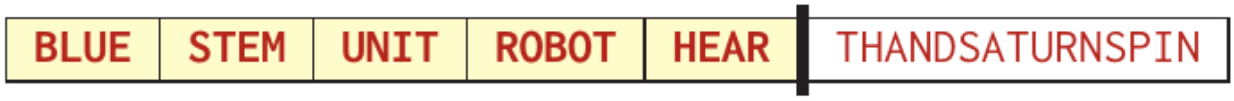


Здесь вертикальная черта отделяет принятые решения — разбиение первых 17 букв на четыре слова — от ещё не обработанной части строки. Следующий шаг — решить, где заканчивается очередное выходное слово. В данном примере возможно четыре варианта: `HE`, `HEAR`, `HEART` и `HEARTH`. Мы понятия не имеем, какой из них, если вообще какой-нибудь, совместим с полным разбиением строки. Можно попытаться быть «умными» и определить, какие варианты хороши, но для этого пришлось бы думать! Вместо этого «глупо» переберём все возможности и предоставим настоящую работу Фее Рекурсии.

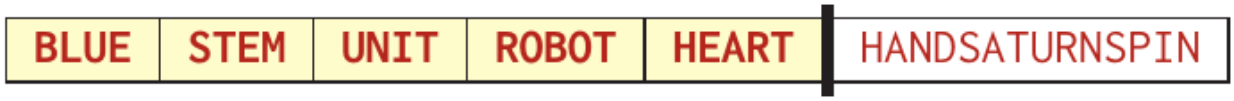
- Сначала предварительно примем `HE` как следующее слово и позволим Фее Рекурсии принять остальные решения:



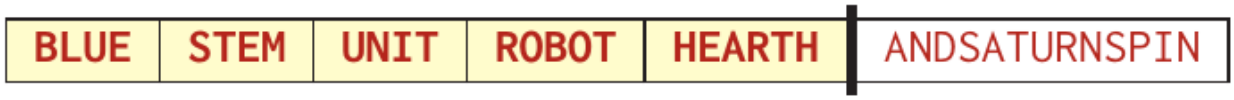
- Затем предварительно примем `HEAR` и предоставим остальное Фее:



- Затем предварительно примем `HEART` и предоставим остальное Фее:

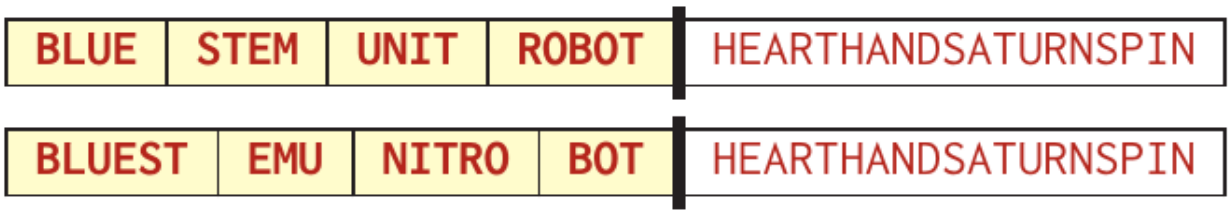


- Наконец, предварительно примем `HEARTH` и предоставим остальное Фее:



Если Фея Рекурсии хотя бы раз сообщит об успехе, мы сообщим об успехе. Если же она ни разу не сообщит об успехе — в частности, если множество возможных следующих слов пусто, — мы сообщим о неудаче.

Прошлые решения никак не влияют на доступные сейчас варианты; важен только ещё не обработанный суффикс. В частности, разные последовательности прошлых решений могут привести к одному суффиксу, но для него набор возможных решений будет совершенно одинаковым:



Поэтому можно упростить картину рекурсивного процесса, отбросив всё слева от черты:



Остаётся простая и естественная стратегия поиска с возвратом: выбрать первое выходное слово и рекурсивно разбить оставшуюся часть строки.

Для полного рекурсивного алгоритма нужен базовый случай. Наша стратегия перестаёт работать в конце входной строки, поскольку следующего слова нет. К счастью, пустая строка единственным образом разбивается на ноль слов!

Объединив всё вместе, получаем простой рекурсивный алгоритм:

```
Splittable(A[1 .. n]):
    если n = 0
        вернуть True
    для i ← 1 до n
        если IsWord(A[1 .. i])
            если Splittable(A[i + 1 .. n])
                вернуть True
    вернуть False
```

[8] На самом деле в большинстве классических латинских рукописей слова разделялись маленькими точками — интерпунктами. К III веку такое разделение почти исчезло, уступив место *scriptio continua*. Пробелы между словами ввели ирландские монахи в VIII веке; в последующие столетия они постепенно распространились по Европе. В английском языке начала XXI века *scriptio continua* сохранилась в URL и хештегах. `#octotherps4lyfe`

[9] Приблизительный перевод: «Прежде всего, достоинство в столь ничтожном знании невозможно: это мелочи, занятые преимущественно отдельными буквами и постановкой точек между словами». Цицерон открыто высмеивал юридическую компетентность своего друга (!) и известного правоведа Сервия Сульпиция Руфа, который обвинил Мурену в подкупе после того, как тот победил Руфа на выборах консула. Мурену оправдали, в том числе благодаря язвительной защите Цицерона, хотя он почти наверняка был виновен. `#librapondo` `#nunquamestfidelis`

[10] В *О христианском учении* святого Августина целая глава посвящена устранению неоднозначности латинского текста Писания с помощью знаков препинания.

Формулировка через индексы

На практике передавать массивы как входные параметры довольно медленно; лучше найти более компактное описание рекурсивных подзадач. При разработке алгоритма очень полезно считать исходный массив глобальной переменной и затем переформулировать задачу и алгоритм через индексы, а не явно заданные подмассивы.

В нашей задаче аргумент любого рекурсивного вызова — суффикс `A[i .. n]` исходного массива. Если считать `A[1 .. n]` глобальным, задача формулируется так:

> *По индексу i найти разбиение суффикса* `A[i .. n]`.

Для описания алгоритма нужны две логические функции:

- Для индексов i и j положим `IsWord(i, j) = True` тогда и только тогда, когда подстрока `A[i .. j]` является словом. Предполагается, что эта функция нам дана.
- Для индекса i положим `Splittable(i) = True` тогда и только тогда, когда суффикс `A[i .. n]` можно разбить на слова. Эту функцию нам предстоит реализовать.

Например, `IsWord(1, n) = True` тогда и только тогда, когда вся входная строка — одно слово; `Splittable(1) = True` тогда и только тогда, когда всю строку можно разбить. Прежняя рекурсивная стратегия даёт соотношение:

$$\text{Splittable}(i) = \begin{cases} \text{True} & \text{если } i > n, \\ \bigvee_{j=i}^{n} \big(\text{IsWord}(i, j) \wedge \text{Splittable}(j+1)\big) & \text{иначе.} \end{cases}$$

Это в точности тот же алгоритм: изменились только обозначения. Сходство ещё очевиднее, если переписать соотношение в псевдокоде:

```
⟨⟨Можно ли разбить суффикс A[i .. n]?⟩⟩
Splittable(i):
    если i > n
        вернуть True
    для j ← i до n
        если IsWord(i, j)
            если Splittable(j + 1)
                вернуть True
    вернуть False
```

Хотя различие в записи может показаться тривиальным, привычка использовать индексы вместо массивов важна не только для практического ускорения поиска с возвратом, но и для разработки алгоритмов динамического программирования, о которых пойдёт речь в следующей главе.

♥ Анализ

Неудивительно, что большинство алгоритмов с возвратом имеют экспоненциальное время работы в худшем случае. Для точного анализа многих из них требуются методы, выходящие за рамки этой книги. К счастью, большинство встречающихся нам алгоритмов с возвратом — лишь промежуточные результаты на пути к более эффективным, поэтому их точное время работы в худшем случае на самом деле неважно. Сначала заставьте работать; потом ускоряйте.

Но ради развлечения проанализируем время рекурсивного `Splittable`. Поскольку мы не знаем, что делает `IsWord`, мы не можем знать время каждого его вызова, поэтому вынуждены анализировать время через число вызовов `IsWord`.[11] `Splittable` вызывает `IsWord` для каждого

префикса входной строки и может рекурсивно вызвать себя для каждого суффикса выходной строки. Таким образом, «время работы» удовлетворяет пугающему соотношению:

$$T(n) \leq \sum_{i=0}^{n-1} T(i) + O(n).$$

На самом деле всё не так страшно, особенно если знать хитрость. Во-первых, заменим $O(n)$ явным выражением αn для некоторой неизвестной и в конечном счёте несущественной константы α. Во-вторых, с запасом предположим, что алгоритм действительно делает все возможные рекурсивные вызовы.[12] Тогда можно преобразовать рекуррентное соотношение с «полной историей» в соотношение с «ограниченной историей», вычтя соотношение для $T(n-1)$:

$$\begin{aligned} T(n) &= \sum_{i=0}^{n-1} T(i) + \alpha n, \\ T(n-1) &= \sum_{i=0}^{n-2} T(i) + \alpha(n-1), \\ T(n) - T(n-1) &= T(n-1) + \alpha. \end{aligned}$$

Последнее соотношение упрощается до $T(n) = 2T(n-1) + \alpha$. Теперь можно уверенно предположить — или вывести с помощью деревьев рекурсии, или вспомнить анализ Ханойской башни, — что $T(n) = O(2^n)$. В самом деле, эту верхнюю оценку нетрудно доказать индукцией по исходному соотношению с полной историей.

Более того, оценка точна. Строку длины n можно разбить ровно 2^{n-1} способами: каждый входной символ либо завершает слово, либо нет, кроме последнего, который всегда завершает последнее слово. В худшем случае `Splittable` исследует все 2^{n-1} вариантов.

[11] В действительности, пока `IsWord` работает за полиномиальное время, `Splittable` работает за $O(2^n)$.

[12] Для разбиения английского текста это предположение сильно завышает оценку, потому что большинство строк букв не являются английскими словами. Но не для похожей задачи разбиения последовательности английских слов на грамматически правильные предложения. Рассмотрим, например, последовательность из n копий слова «buffalo», или n копий слова «police», или n копий слова «can» для любого положительного n. В «Мулен Руж» танцы, которые могут быть законсервированы другими танцами в металлических цилиндрах, имеют возможность увольнять танцы, происходящие в мусорных корзинах тюремных туалетов.

Варианты

Имея основную рекурсивную схему, можно решать много вариантов задачи разбиения, как мы делали для `SubsetSum`. Здесь я опишу только один; другие рассматриваются в упражнениях. Как обычно, исходный вход — массив `A[1 .. n]`.

Если строка разбивается на слова несколькими способами, может понадобиться найти лучший по некоторому критерию. И наоборот, если её нельзя разбить на слова, может быть полезно вычислить лучшее доступное разбиение вместо простого сообщения об ошибке. Для обеих целей предположим, что у нас есть вторая функция `Score`, принимающая строку и возвращающая числовую оценку. Например, длинным или распространённым словам можно давать более высокие оценки, коротким или редким — более низкие, незначительным орфографическим ошибкам — слегка отрицательные, а очевидным не-словам — ещё более отрицательные. Наша цель — найти разбиение с максимальной суммой оценок частей.

Для каждого индекса i пусть `MaxScore(i)` — максимальная оценка разбиения суффикса `A[i .. n]`; требуется вычислить `MaxScore(1)`. Эта функция удовлетворяет соотношению:

$$\text{MaxScore}(i) = \begin{cases} 0 & \text{если } i > n, \\ \max\limits_{i \le j \le n} \big(\text{Score}(A[i..j]) + \text{MaxScore}(j+1)\big) & \text{иначе.} \end{cases}$$

По сути это то же соотношение, что мы вывели для `Splittable`; единственная разница — логические операции $\vee$ и $\wedge$ заменены числовыми операциями $\max$ и $+$.

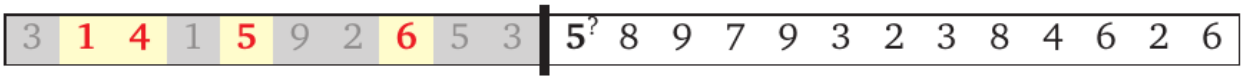
2.6. Наибольшая возрастающая подпоследовательность

Для любой последовательности S *подпоследовательность* — другая последовательность, полученная удалением нуля или более элементов из S без изменения порядка оставшихся элементов; эти элементы не обязаны идти подряд. Например, когда вы едете по главной улице любого города, вы проезжаете последовательность перекрёстков со светофорами, но останавливаться приходится только на подпоследовательности тех перекрёстков, где горит красный. Если повезёт, вы вообще нигде не остановитесь: пустая последовательность является подпоследовательностью S. Если же совсем не повезёт, придётся остановиться на каждом перекрёстке: S является подпоследовательностью самой себя.

Другой пример: строки `BENT`, `ACKACK`, `SQUARING` и `SUBSEQUENT` — подпоследовательности строки `SUBSEQUENCEBACKTRACKING`, как и пустая строка и вся строка `SUBSEQUENCEBACKTRACKING`; но строки `QUEUE`, `EQUUS` и `TALLYHO` — нет. Подпоследовательность, элементы которой идут подряд в исходной последовательности, называется *подстрокой*. Например, `MASHER` и `LAUGHTER` — подпоследовательности `MANSLAUGHTER`, но только `LAUGHTER` является подстрокой.

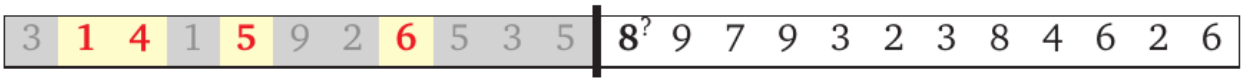
Теперь предположим, что дана последовательность целых чисел и требуется найти самую длинную подпоследовательность с возрастающими элементами. Точнее, на вход дан массив целых чисел `A[1 .. n]`, и нужно вычислить самую длинную последовательность индексов $1 \le i_1 < i_2 < \cdots < i_\ell \le n$, для которой $A[i_k] < A[i_{k+1}]$ при всех k.

Один естественный способ строить её — для каждого индекса j по порядку от 1 до n решать, включать ли `A[j]` в подпоследовательность. Представим процесс в середине принятия решений:



Как и в примерах разбиения текста, чёрная черта отделяет прошлые решения от необработанной части входа. Числа, которые мы уже решили включить, выделены жирным; те, которые решили исключить, показаны серым. Заметьте, что выбранные числа возрастают! Алгоритму нужно решить, включать ли число сразу после черты.

В данном примере число 5 включать определённо нельзя, иначе выбранные числа перестанут возрастать. Перейдём к следующему решению:

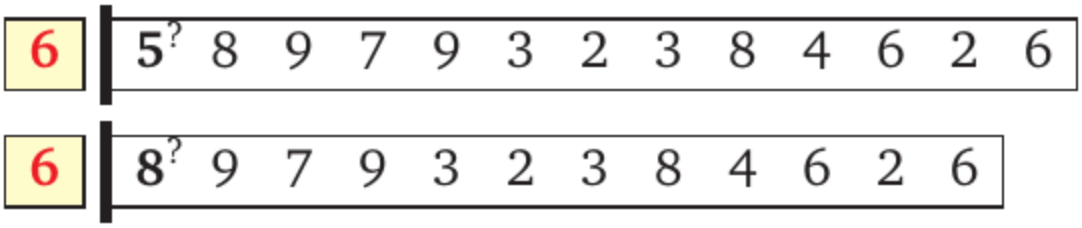


Теперь можно включить 8, но неочевидно, следует ли это делать. Вместо попытки быть «умным» алгоритм с возвратом использует простой полный перебор:

- Сначала предварительно включаем 8 и позволяем Фее Рекурсии принять оставшиеся решения.
- Затем предварительно исключаем 8 и позволяем Фее Рекурсии принять оставшиеся решения.

Правилен тот вариант, который приводит к более длинной возрастающей подпоследовательности. Это в точности схема рекурсии, использованная для `SubsetSum`.

А теперь ключевой вопрос: что нам нужно помнить о прошлых решениях? Включить `A[j]` можно, только если получившаяся последовательность возрастает. Если предположить по индукции, что числа, ранее выбранные из `A[1 .. j - 1]`, возрастают, то `A[j]` можно включить тогда и только тогда, когда оно больше последнего из этих чисел. Значит, о прошлом нужно помнить только последнее выбранное число. Теперь можно упростить рисунки, удалив всё ненужное:



Таким образом, наша рекурсивная стратегия на самом деле решает следующую задачу:

> *По целому числу* `prev` *и массиву* `A[1 .. n]` *найти наибольшую возрастающую подпоследовательность* A*, каждый элемент которой больше* `prev`.

Как обычно, нужен базовый случай. Стратегия перестаёт работать в конце массива, потому что нет «следующего числа». Но пустой массив имеет ровно одну подпоследовательность — пустую. По определению каждый элемент пустой последовательности больше любого выбранного значения и каждая пара её элементов идёт в возрастающем порядке. Следовательно, длина наибольшей возрастающей подпоследовательности пустого массива равна нулю.

Получается следующий рекурсивный алгоритм:

```
LISbigger(prev, A[1 .. n]):
    если n = 0
        вернуть 0
    иначе если A[1] ≤ prev
        вернуть LISbigger(prev, A[2 .. n])
    иначе
        skip ← LISbigger(prev, A[2 .. n])
        take ← LISbigger(A[1], A[2 .. n]) + 1
        вернуть max{skip, take}
```

Однако помните: передавать массивы через стек вызовов дорого. Попробуем переформулировать всё через индексы, считая `A[1 .. n]` глобальным массивом. Целое `prev` обычно является элементом массива `A[i]`, а оставшийся массив — всегда суффиксом `A[j .. n]` исходного. Переформулируем задачу:

> *По индексам i и j, где $i < j$, найти наибольшую возрастающую подпоследовательность* `A[j .. n]`*, каждый элемент которой больше* `A[i]`.

Пусть `LISbigger(i, j)` — длина этой подпоследовательности. Наша стратегия даёт соотношение:

$$\text{LISbigger}(i, j) = \begin{cases} 0 & \text{если } j > n, \\ \text{LISbigger}(i, j+1) & \text{если } A[i] \geq A[j], \\ \max\{\text{LISbigger}(i, j+1),\ 1 + \text{LISbigger}(j, j+1)\} & \text{иначе.} \end{cases}$$

Или, если предпочитаете псевдокод:

```
LISbigger(i, j):
    если j > n
        вернуть 0
    иначе если A[i] ≥ A[j]
        вернуть LISbigger(i, j + 1)
    иначе
        skip ← LISbigger(i, j + 1)
        take ← LISbigger(j, j + 1) + 1
        вернуть max{skip, take}
```

Наконец, нужно связать рекурсивную стратегию с первоначальной задачей: найти наибольшую возрастающую подпоследовательность массива без дополнительных ограничений. Проще всего добавить в начало массива искусственное сторожевое значение $-\infty$:

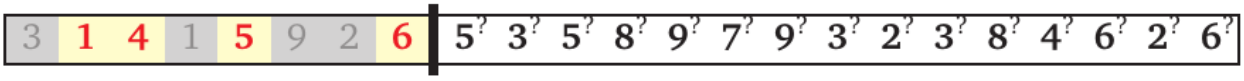
```
LIS(A[1 .. n]):
    A[0] ← -∞
    вернуть LISbigger(0, 1)
```

Время работы `LISbigger` удовлетворяет соотношению Ханойской башни $T(n) \leq 2T(n-1) + O(1)$, откуда, как обычно, $T(n) = O(2^n)$. Это не должно удивлять: в худшем случае алгоритм рассматривает каждую из 2^n подпоследовательностей входного массива.

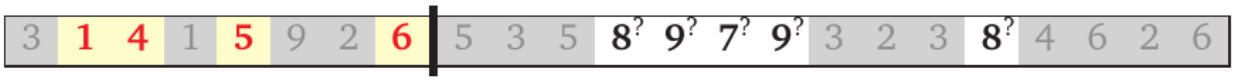
2.7. Наибольшая возрастающая подпоследовательность: вторая попытка

Это не единственная стратегия поиска с возвратом для данной задачи. Вместо рассмотрения входной последовательности по одному элементу можно строить выходную последовательность по одному элементу. То есть вместо вопроса «Является ли `A[i]` следующим элементом результата?» непосредственно спросить: «Где находится следующий элемент результата, если он есть?»

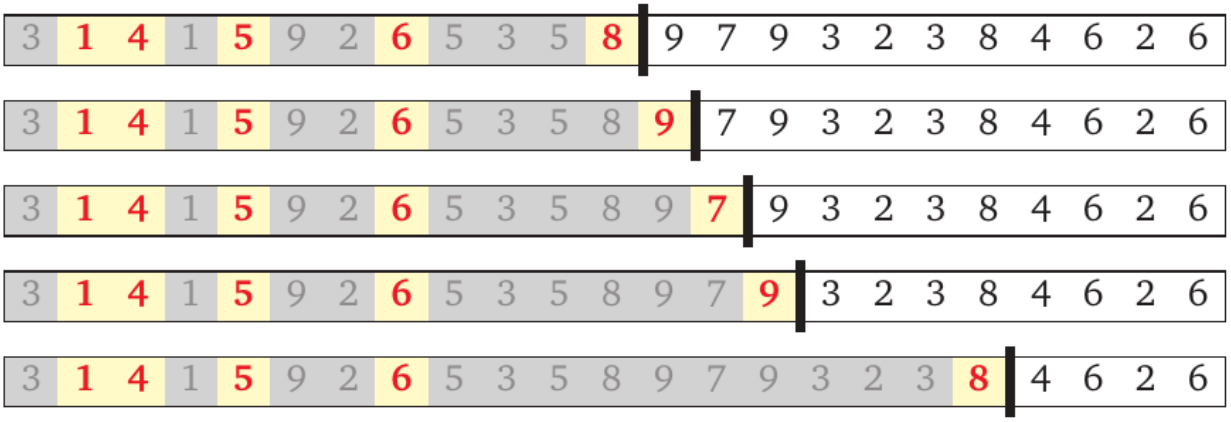
Представим, что мы находимся посреди работы: только что решили включить шестёрку слева от чёрной черты и должны выбрать следующий элемент справа.



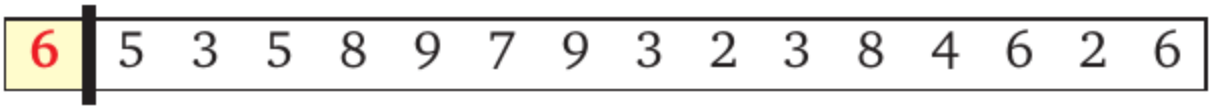
Разумеется, справа можно включить только числа больше 6, иначе выходная последовательность не будет возрастающей.



Но мы не представляем, какое из этих чисел лучше выбрать; хитроумный поиск лучшего варианта потребовал бы слишком много работы и всё равно лишь навлёк бы неприятности. Вместо этого переберём все варианты и позволим Фее Рекурсии оценить каждый.



Множество чисел, которые можно выбрать следующими, зависит только от последнего включённого числа. Поэтому упростим картину: отбросим всё слева от черты, кроме последнего выбранного числа.



Оставшаяся последовательность — суффикс исходного массива. Считая входной массив `A[1 .. n]` глобальным, формально выразим рекурсивную задачу через индексы:

> *По индексу i найти наибольшую возрастающую подпоследовательность* `A[i .. n]`*, которая начинается с* `A[i]`.

Пусть `LISfirst(i)` — её длина. Теперь стратегия записывается так:

$$\text{LISfirst}(i) = 1 + \max\{\text{LISfirst}(j) \mid j > i \text{ и } A[j] > A[i]\}.$$

Поскольку мы работаем с множествами натуральных чисел, определим $\max \varnothing = 0$. Тогда автоматически `LISfirst(i) = 1`, если $A[j] \le A[i]$ для всех $j > i$; в частности, `LISfirst(n) = 1`. Это базовые случаи.

```
LISfirst(i):
    best ← 0
    для j ← i + 1 до n
        если A[j] > A[i]
            best ← max{best, LISfirst(j)}
    вернуть 1 + best
```

Наконец, нужно снова связать рекурсивный алгоритм с исходной задачей — найти наибольшую возрастающую подпоследовательность без знания её первого элемента. Подходит естественный способ: перебрать все возможные первые элементы. Или можно добавить сторожевое значение $-\infty$ в начало массива, найти наибольшую возрастающую подпоследовательность, начинающуюся с него, а затем не учитывать его.

```
LIS(A[1 .. n]):
    best ← 0
    для i ← 1 до n
        best ← max{best, LISfirst(i)}
    вернуть best
```

```
LIS(A[1 .. n]):
    A[0] ← -∞
    вернуть LISfirst(0) - 1
```

2.8. Оптимальные двоичные деревья поиска

Наш последний пример объединяет рекурсивный поиск с возвратом со стратегией «разделяй и властвуй». Напомним, что время успешного поиска в двоичном дереве поиска пропорционально числу предков целевого узла.[13] Поэтому время поиска в худшем случае пропорционально глубине дерева. Чтобы минимизировать это время, высота дерева должна быть как можно меньше; по такому критерию идеальное дерево совершенно сбалансировано.

Однако во многих применениях важнее минимизировать суммарную стоимость нескольких поисков, а не стоимость одного поиска в худшем случае. Если элемент x ищут чаще, чем y, можно сэкономить время, построив дерево, в котором глубина x меньше глубины y, даже если

при этом общая глубина дерева увеличится. Совершенно сбалансированное дерево — не лучший выбор, если некоторые элементы гораздо популярнее других. Более того, совершенно несбалансированное дерево глубины $\Omega(n)$ может оказаться лучшим!

Это подсказывает следующую задачу. Пусть дан отсортированный массив ключей `A[1 .. n]` и массив соответствующих частот обращений `f[1 .. n]`. Нужно построить двоичное дерево поиска, минимизирующее общее время поиска, если каждый ключ `A[i]` будут искать ровно `f[i]` раз.

Прежде чем думать о решении, дадим хорошее рекурсивное определение функции, которую требуется оптимизировать! Пусть нам также дано двоичное дерево поиска T с n узлами. Обозначим его узлы через $v_1, v_2, \ldots, v_n$ в отсортированном порядке, чтобы каждый v_i содержал соответствующий ключ `A[i]`. Тогда, пренебрегая постоянными множителями, суммарная стоимость всех поисков равна

$$\text{Cost}(T, f[1..n]) := \sum_{i=1}^{n} f[i] \cdot \#\text{предков } v_i \text{ в } T. \qquad (*)$$

Пусть теперь корень $T - v_r$; по определению v_r является предком каждого узла T. Если $i < r$, все предки v_i, кроме корня, находятся в левом поддереве T. Аналогично, если $i > r$, они находятся в правом поддереве. Значит, функцию стоимости можно разбить на три части:

$$\begin{aligned}\text{Cost}(T, f[1..n]) = &\sum_{i=1}^{n} f[i] + \sum_{i=1}^{r-1} f[i] \cdot \#\text{предков } v_i \text{ в } \text{left}(T) \\ &+ \sum_{i=r+1}^{n} f[i] \cdot \#\text{предков } v_i \text{ в } \text{right}(T).\end{aligned}$$

Вторая и третья суммы выглядят точно как исходное определение $(*)$ функции `Cost(T, f[1 .. n])`. Простая подстановка даёт рекуррентное соотношение:

$$\text{Cost}(T, f[1..n]) = \sum_{i=1}^{n} f[i] + \text{Cost}(\text{left}(T), f[1..r-1]) + \text{Cost}(\text{right}(T), f[r+1..n]).$$

Базовый случай, как обычно, $n = 0$: стоимость выполнения нуля поисков в пустом дереве равна нулю.

Теперь нужно вычислить дерево T_{opt}, минимизирующее эту функцию. Предположим, что мы каким-то волшебным образом знаем: его корень — v_r. Тогда рекурсивное определение `Cost(T, f)` немедленно показывает, что левое поддерево `left(Topt)` должно быть оптимальным деревом поиска для ключей `A[1 .. r - 1]` и частот `f[1 .. r - 1]`. Аналогично, правое поддерево `right(Topt)` должно быть оптимальным деревом для `A[r + 1 .. n]` и `f[r + 1 .. n]`. Выбрав правильный ключ для корня, мы позволим Фее Рекурсии построить остальную часть оптимального дерева.

В более общем случае пусть `OptCost(i, k)` — суммарная стоимость оптимального дерева поиска для интервала частот `f[i .. k]`. Функция удовлетворяет соотношению:

$$\mathrm{OptCost}(i,k) = \begin{cases} 0 & \text{если } i > k, \\ \sum\limits_{j=i}^{k} f[i] + \min\limits_{i \le r \le k} \{\mathrm{OptCost}(i, r-1) + \mathrm{OptCost}(r+1, k)\} & \text{иначе.} \end{cases}$$

Базовый случай верно показывает: минимальная стоимость нуля поисков в пустом множестве равна нулю! Исходная задача — вычислить `OptCost(1, n)`.

> *Примечание переводчика: в сумме в оригинале напечатано $f[i]$, хотя индекс суммирования — j. В соответствии с предыдущим выводом и смыслом задачи здесь должно быть $\sum_{j=i}^{k} f[j]$.*

Это рекурсивное определение можно механически перевести в рекурсивный алгоритм с возвратом для вычисления `OptCost(1, n)`. Неудивительно, что время его работы экспоненциально. В следующей главе мы увидим, как сократить его до полиномиального, поэтому особого смысла вычислять точное время нет...

[13] Предок узла v — либо сам v, либо предок родителя v. *Собственный предок* — либо родитель v, либо собственный предок родителя v.

♥ Анализ

...если только вам это не нравится. Просто ради развлечения выясним, насколько медленный этот алгоритм. Его время работы удовлетворяет соотношению

$$T(n) = \sum_{k=1}^{n} \big(T(k-1) + T(n-k)\big) + O(n).$$

Слагаемое $O(n)$ возникает при вычислении общего числа поисков $\sum_{i=1}^{n} f[i]$. Да, соотношение страшненькое, но его можно решить ровно тем же трюком с вычитанием, что и раньше. Заменим обозначение $O(\cdot)$ явной константой, перегруппируем и соберём одинаковые слагаемые, вычтем соотношение для $T(n-1)$, чтобы избавиться от суммы, а затем снова перегруппируем:

$$\begin{aligned} T(n) &= 2\sum_{k=0}^{n-1} T(k) + \alpha n, \\ T(n-1) &= 2\sum_{k=0}^{n-2} T(k) + \alpha(n-1), \\ T(n) - T(n-1) &= 2T(n-1) + \alpha, \\ T(n) &= 3T(n-1) + \alpha. \end{aligned}$$

Эй, теперь выглядит не так уж плохо! Метод дерева рекурсии сразу даёт решение $T(n) = O(3^n)$; можно и просто угадать его, а затем подтвердить индукцией.

Из этого анализа следует, что рекурсивный алгоритм не перебирает все возможные двоичные деревья поиска! Число таких деревьев с n вершинами удовлетворяет соотношению

$$N(n) = \sum_{r=1}^{n-1} \big(N(r-1) \cdot N(n-r)\big),$$

которое имеет решение в замкнутой форме $N(n) = \Theta(4^n/\sqrt{n})$. Нет, это не очевидно. Наш алгоритм существенно экономит время, независимо ища оптимальные левое и правое поддеревья для каждого корня. Полный перебор деревьев поиска рассматривал бы все возможные пары левого и правого поддеревьев; отсюда произведение в формуле для $N(n)$.

> *Примечание переводчика: в оригинале в последней формуле, по-видимому, есть опечатки. Для числа двоичных деревьев поиска правильно* $N(n) = \sum_{r=1}^{n} N(r-1)N(n-r)$ *и* $N(n) = \Theta(4^n/n^{3/2})$.

Упражнения

1. Опишите рекурсивные алгоритмы для следующих обобщений задачи `SubsetSum`.

(a) Дан массив положительных целых чисел `X[1 .. n]` и целое T. Вычислите количество подмножеств X, сумма элементов которых равна T.

(b) Даны два массива положительных целых чисел `X[1 .. n]` и `W[1 .. n]` и целое T; `W[i]` обозначает вес соответствующего элемента `X[i]`. Найдите подмножество X максимального веса, сумма элементов которого равна T. Если ни одно подмножество не даёт сумму T, алгоритм должен вернуть $-\infty$.

2. Опишите рекурсивные алгоритмы для следующих вариантов задачи разбиения текста. Предположим, что есть подпрограмма `IsWord`, принимающая массив символов и возвращающая `True` тогда и только тогда, когда строка является «словом».

(a) Дан массив символов `A[1 .. n]`. Вычислите количество способов разбить A на слова. Например, для строки `ARTISTOIL` алгоритм должен вернуть 2: это разбиения `ARTIST·OIL` и `ART·IS·TOIL`.

(b) Даны два массива символов `A[1 .. n]` и `B[1 .. n]`. Определите, можно ли разбить оба на слова по одним и тем же индексам. Например, строки `BOTHEARTHANDSATURNSPIN` и `PINSTARTRAPSANDRAGSLAP` можно разбить на слова по одинаковым индексам так:

```
BOT·HEART·HAND·SAT·URNS·PIN
PIN·START·RAPS·AND·RAGS·LAP
```

(c) Даны два массива символов `A[1 .. n]` и `B[1 .. n]`. Вычислите число различных способов разбить их на слова по одним и тем же индексам.

3. *Аддитивная цепочка* для целого n — возрастающая последовательность целых чисел, начинающаяся с 1 и заканчивающаяся на n, каждый элемент которой после первого является суммой двух более ранних элементов. Формальнее, последовательность $x_0 < x_1 < x_2 < \cdots < x_\ell$ является аддитивной цепочкой для n тогда и только тогда, когда:

- $x_0 = 1$;
- $x_\ell = n$;
- для каждого индекса $k > 0$ существуют индексы $i \le j < k$, такие что $x_k = x_i + x_j$.

Длина аддитивной цепочки — число элементов минус 1: первый элемент мы не считаем. Например, $\langle 1, 2, 3, 5, 10, 20, 23, 46, 92, 184, 187, 374 \rangle$ — аддитивная цепочка для 374 длины 11.

(a) Опишите рекурсивный алгоритм с возвратом, который находит аддитивную цепочку минимальной длины для заданного положительного целого n. Не анализируйте и не оптимизируйте время работы, разве что ради собственного любопытства. Для полного зачёта достаточно корректного алгоритма, работающего за экспоненциальное от n время. [Подсказка: эта задача гораздо больше похожа на задачу о n ферзях, чем на разбиение текста.]

♥ (b) Опишите рекурсивный алгоритм с возвратом для нахождения аддитивной цепочки минимальной длины для заданного положительного n за субэкспоненциальное от n время. [Подсказка: возможно, пригодятся результаты некоторых египетских натягивателей верёвок, индийских исследователей стихосложения и русских крестьян.]

4.

(a) Пусть `A[1 .. m]` и `B[1 .. n]` — произвольные массивы. *Общая подпоследовательность* A и B является подпоследовательностью обоих массивов. Дайте простое рекурсивное определение функции `lcs(A, B)`, возвращающей длину наибольшей общей подпоследовательности.

(b) Пусть `A[1 .. m]` и `B[1 .. n]` — произвольные массивы. *Общая надпоследовательность* A и B — последовательность, содержащая оба массива как подпоследовательности. Дайте простое рекурсивное определение функции `scs(A, B)`, возвращающей длину кратчайшей общей надпоследовательности.

(c) Назовём последовательность чисел `X[1 .. n]` *битонической*, если существует индекс i с $1 < i < n$, такой что префикс `X[1 .. i]` возрастает, а суффикс `X[i .. n]` убывает. Дайте простое рекурсивное определение функции `lbs(A)`, возвращающей длину наибольшей битонической подпоследовательности произвольного массива целых чисел A.

(d) Назовём последовательность `X[1 .. n]` *колеблющейся*, если $X[i] < X[i+1]$ для всех чётных i и $X[i] > X[i+1]$ для всех нечётных i. Дайте простое рекурсивное определение функции `los(A)`, возвращающей длину наибольшей колеблющейся подпоследовательности произвольного массива целых чисел A.

(e) Дайте простое рекурсивное определение функции `sos(A)`, возвращающей длину кратчайшей колеблющейся надпоследовательности произвольного массива целых чисел A.

(f) Назовём последовательность `X[1 .. n]` *выпуклой*, если $2 \cdot X[i] < X[i-1] + X[i+1]$ для всех i. Дайте простое рекурсивное определение функции `lxs(A)`, возвращающей длину наибольшей выпуклой подпоследовательности произвольного массива целых чисел A.

5. В каждой из следующих задач вход состоит из двух массивов `X[1 .. k]` и `Y[1 .. n]`, где $k \leq n$.

(a) Опишите рекурсивный алгоритм с возвратом, определяющий, является ли X подпоследовательностью Y. Например, строка `PPAP` является подпоследовательностью строки `PENPINEAPPLEAPPLEPEN`.

(b) Опишите рекурсивный алгоритм с возвратом, находящий минимальное число символов, которые нужно удалить из Y, чтобы X перестала быть её подпоследовательностью. Эквивалентно алгоритм должен найти наибольшую подпоследовательность Y, которая не является надпоследовательностью X. Например, после удаления двух символов из `PENPINEAPPLEAPPLEPEN` строка `PPAP` больше не будет её подпоследовательностью.

♥ (c) Опишите рекурсивный алгоритм с возвратом, определяющий, встречается ли X как две непересекающиеся подпоследовательности Y. Например, `PPAP` встречается как две непересекающиеся подпоследовательности в `PENPINEAPPLEAPPLEPEN`.

Не анализируйте время работы алгоритмов, разве что ради собственного любопытства. Все три работают за экспоненциальное время; позже мы их улучшим, поэтому точное время не особенно важно.

6. В этой задаче требуется разработать алгоритмы с возвратом для вычисления стоимости оптимального двоичного дерева поиска с дополнительными ограничениями на сбалансированность. На вход подаются отсортированный массив ключей поиска `A[1 .. n]` и массив частот `f[1 .. n]`, где `f[i]` — число поисков `A[i]`. Функция стоимости в точности та же, что в разделе 2.8. Но теперь нужно вычислить оптимальное дерево, удовлетворяющее дополнительным ограничениям.

(a) АВЛ-деревья были первыми самобалансирующимися двоичными деревьями поиска; их впервые описали Георгий Адельсон-Вельский и Евгений Ландис в 1962 году. АВЛ-дерево — двоичное дерево поиска, в котором для каждого узла v высоты левого и правого поддеревьев отличаются не более чем на единицу.

Опишите рекурсивный алгоритм с возвратом, строящий оптимальное АВЛ-дерево для заданных ключей и частот.

(b) Симметричные двоичные B-деревья — ещё один вид самобалансирующихся двоичных деревьев, впервые описанный Рудольфом Байером в 1972 году. Они более известны как красно-чёрные деревья после несколько более простой переформулировки Лео Гибаса и Боба Седжвика в 1978 году. Красно-чёрное дерево — двоичное дерево поиска со следующими дополнительными ограничениями:

- каждый узел либо красный, либо чёрный;
- у каждого красного узла родитель чёрный;
- каждый путь от корня до листа содержит одинаковое число чёрных узлов.

Опишите рекурсивный алгоритм с возвратом, строящий оптимальное красно-чёрное дерево для заданных ключей и частот.

(c) AA-деревья предложил Арне Андерссон в 1993 году; Марк Аллен Вайс немного упростил их и дал им название в 2000 году. Они также известны как красно-чёрные деревья с наклоном влево, после симметричной переформулировки с другими алгоритмами перебалансировки, предложенной Бобом Седжвиком в 2006 году. AA-дерево — красно-чёрное дерево с дополнительным ограничением:

- ни один левый потомок не является красным.[14]

Опишите рекурсивный алгоритм с возвратом, строящий оптимальное AA-дерево для заданных ключей и частот.

Не анализируйте время работы алгоритмов, разве что ради собственного любопытства. Все три работают за экспоненциальное время; позже мы их улучшим, поэтому точное время не особенно важно.

За дополнительными упражнениями на поиск с возвратом обращайтесь к следующей главе!

[14] Переформулировка Седжвика требует, чтобы красным не был ни один *правый* потомок. Какая разница. Андерссон и Седжвик почему-то умалчивают о том, с какого конца следует разбивать яйцо.

3. Динамическое программирование

Как видно здесь на полях, мы действовали следующим образом: сложили первое число со вторым, то есть 1 с 2, затем второе с третьим, третье с четвёртым, четвёртое с пятым и так далее…

— Леонардо Пизанский, Книга абака (1202)

Те, кто не помнит прошлого, обречены его повторять.

— Хорхе Агустин Николас Руис де Сантаяна-и-Боррас, Жизнь разума, книга I: «Введение и разум в здравом смысле» (1905)

Знаете, что такое поучительный опыт? Это такая вещь, которая говорит: «Помните то, что вы только что сделали? Больше так не делайте».

— Дуглас Адамс, Лосось сомнения (2002)

3.1. Матравритта

Один из древнейших примеров рекурсии возник в Индии более 2000 лет назад при изучении стихотворного размера, или просодии. Классическая санскритская поэзия различает два вида слогов (*акшара*): лёгкие (*лагху*) и тяжёлые (*гуру*). В одном классе размеров, называемых *матравритта* или *матрачхандас*, каждая строка стихотворения состоит из фиксированного числа «долей» (*матр*): лёгкий слог длится одну долю, тяжёлый — две. Формальное изучение матравритты восходит к трактату *Чхандахшастра*, написанному учёным Пингалой между 600 и 200 годами до н. э. Пингала заметил, что существует ровно пять размеров длиной в четыре доли: —, – • •, • – •, • • – и • • • •. Здесь каждое «—» обозначает долгий слог, а каждая «•» — краткий.[3-1]

Хотя текст Пингалы намекает на систематическое правило подсчёта размеров с заданным числом долей,[3-2] прошло около тысячи лет, прежде чем это правило было сформулировано явно. В VII веке н. э. другой индийский учёный, Вираханка, написал комментарий к труду Пингалы, где заметил: число размеров из n долей равно сумме числа размеров из $n-2$ долей и числа размеров из $n-1$ долей. В современных обозначениях это наблюдение задаёт рекуррентное соотношение для общего числа $M(n)$ размеров из n долей:

$$M(n) = M(n-2) + M(n-1).$$

Нетрудно увидеть, что $M(0) = 1$ (есть только один пустой размер) и $M(1) = 1$ (единственный размер из одной доли состоит из одного краткого слога).

То же соотношение вновь появилось в Европе примерно через 500 лет после Вираханки — в трактате Леонардо Пизанского *Книга абака* 1202 года, одном из наиболее влиятельных раннеевропейских трудов по «алгоризму». В полном соответствии с законом эпонимии

Стиглера[3-3] современные *числа Фибоначчи* определяются рекуррентным соотношением Вираханки, но с другими базовыми случаями:

$$F_n = \begin{cases} 0, & n = 0, \\ 1, & n = 1, \\ F_{n-1} + F_{n-2}, & \text{в остальных случаях.} \end{cases}$$

В частности, для всех n выполняется $M(n) = F_{n+1}$.

> ***Примечание 3-1.*** *В азбуке Морзе «тире» длится в три раза дольше «точки», но за каждой точкой и каждым тире следует пауза той же длительности, что и точка. Следовательно, каждая пара «точка — пауза» представляет лёгкий слог, каждая пара «тире — пауза» — тяжёлый слог; ровно пять букв — M, D, R, U и H — имеют код длительностью в четыре матры.*

> ***Примечание 3-2.*** *Чхандахшастра содержит два систематических правила перечисления всех размеров с заданным числом слогов; они примерно соответствуют записи двоичных чисел слева направо, как у греков, или справа налево, как у египтян. Тот же текст включает рекурсивный алгоритм вычисления 2^n — числа размеров из n слогов — последовательным возведением в квадрат и, возможно, рекурсивный алгоритм вычисления биномиальных коэффициентов — числа размеров с k краткими слогами из n слогов в целом.*

> ***Примечание 3-3.*** *«Ни одно научное открытие не называется именем первооткрывателя». В статье 1980 года, давшей закону название, статистик Стивен Стиглер в шутку утверждал, что первым его предложил социолог Роберт К. Мертон. Однако сходные утверждения ранее высказывали Владимир Арнольд в 1970-х («Открытия редко приписываются нужному человеку»), Карл Бойер в 1968 году («Клио, муза истории, часто непостоянна, давая теоремам имена!»), Альфред Норт Уайтхед в 1917 году («Всё важное уже было сказано прежде кем-то, кто этого не открыл») и даже отец Стивена, Джордж Стиглер, в 1966 году («Если нам когда-либо встретится случай, когда теория названа именем нужного человека, это заметят»). В этой книге мы увидим много других примеров закона Стиглера.*

Поиск с возвратом может быть медленным

Рекурсивное определение чисел Фибоначчи сразу даёт рекурсивный алгоритм их вычисления. Тот же алгоритм на псевдокоде:

```
RecFibo(n):
    если n = 0
        вернуть 0
    иначе если n = 1
        вернуть 1
    иначе
        вернуть RecFibo(n – 1) + RecFibo(n – 2)
```

К сожалению, этот наивный рекурсивный алгоритм ужасно медленный. Если не считать рекурсивных вызовов, он выполняет лишь постоянное число шагов: одно сравнение и, возможно, одно сложение. Пусть $T(n)$ — число вызовов `RecFibo`. Эта функция удовлетворяет соотношению

$$T(0) = 1, \quad T(1) = 1, \quad T(n) = T(n-1) + T(n-2) + 1,$$

которое очень похоже на рекуррентное соотношение для самих чисел Фибоначчи! Выписав несколько первых значений $T(n)$, можно предположить явную формулу $T(n) = 2F_{n+1} - 1$ и проверить её индукцией — намёк, намёк. Таким образом, вычисление F_n этим алгоритмом занимает примерно вдвое больше времени, чем простой счёт до F_n. Методы за пределами этой книги[3-4] показывают, что $F_n = \Theta(\varphi^n)$, где $\varphi = (\sqrt{5}+1)/2 \approx 1{,}61803$ — так называемое *золотое сечение*. Итак, время работы алгоритма экспоненциально по n.

Это экспоненциальное увеличение можно увидеть и непосредственно. Рассмотрим дерево рекурсии `RecFibo` как двоичное дерево сложений, в листьях которого находятся только нули и единицы. Поскольку итог равен F_n, ровно F_n листьев должны иметь значение `1`; эти листья представляют вызовы `RecFibo(1)`. Простое рассуждение по индукции — намёк, намёк — показывает, что `RecFibo(0)` вызывается ровно F_{n-1} раз. Если нас интересует только асимптотическая оценка, достаточно заметить, что вызовов `RecFibo(0)` не больше, чем вызовов `RecFibo(1)`. Поэтому дерево рекурсии имеет ровно $F_n + F_{n-1} = F_{n+1} = O(F_n)$ листьев, а поскольку каждый внутренний узел имеет двух потомков, всего в нём $2F_{n+1} - 1 = O(F_n)$ узлов.

Мемоизация: запоминайте всё

Очевидная причина медлительности рекурсивного алгоритма состоит в том, что он снова и снова вычисляет одни и те же числа Фибоначчи. Один вызов `RecFibo(n)` приводит к одному рекурсивному вызову `RecFibo(n-1)`, двум вызовам `RecFibo(n-2)`, трём вызовам `RecFibo(n-3)`, пяти вызовам `RecFibo(n-4)` и, как указано в оригинале, к F_{k-1} вызовам `RecFibo(n-k)` для любого целого $0 \le k < n$. Каждый вызов заново вычисляет некоторое число Фибоначчи.

> *Примечание переводчика: в формуле F_{k-1} в исходном издании, вероятно, опечатка. Для перечисленных количеств и $1 \le k < n$ получается F_{k+1}; при $k = 0$ имеется один исходный вызов.*

Можно значительно ускорить рекурсивный алгоритм: записывать результаты рекурсивных вызовов и снова обращаться к ним, когда они понадобятся.

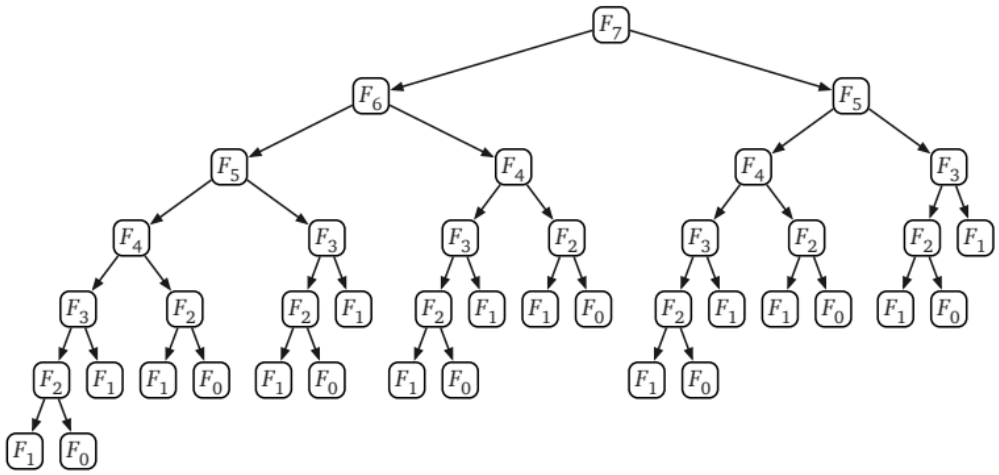


Рисунок 3.1. Дерево рекурсии для вычисления F_7; стрелки обозначают рекурсивные вызовы.

Этот приём оптимизации, известный теперь как *мемоизация* (по-английски *memoization*, без буквы r), обычно приписывают Дональду Мичи в 1967 году, но практически тот же приём в 1959 году предложил Артур Сэмюэл.[3-5]

```
MemFibo(n):
    если n = 0
        вернуть 0
    иначе если n = 1
        вернуть 1
    иначе
        если F[n] не определено
            F[n] ← MemFibo(n – 1) + MemFibo(n – 2)
        вернуть F[n]
```

Мемоизация, очевидно, сокращает время работы алгоритма, но насколько? Если проследить за рекурсивными вызовами `MemFibo`, обнаружится, что массив `F[ ]` заполняется снизу вверх: сначала `F[2]`, затем `F[3]` и так далее до `F[n]`. Этот порядок можно проверить индукцией: каждая ячейка `F[i]` заполняется только после предыдущей `F[i–1]`. Если не учитывать время рекурсивных вызовов, вычисление рекуррентного соотношения для каждого числа Фибоначчи F_i требует постоянного времени. Но по построению для каждого индекса i оно вычисляется лишь один раз. Значит, `MemFibo` выполняет только $O(n)$ сложений — экспоненциальное улучшение по сравнению с наивным рекурсивным алгоритмом!

> ***Примечание 3-4.*** *Заметки о решении рекуррентных соотношений поиска с возвратом см. на http://algorithms.wtf.*

> ***Примечание 3-5.*** *Мичи предложил языкам программирования поддерживать абстракцию, которую он называл «мемо-функцией»: обычную функцию («правило») вместе со словарём («памятью»), вместо раздельной поддержки массивов и функций. Впервые вычислив значение, такая функция «запоминает» его в словаре. Мичи вдохновило применение Сэмюэлом «механического запоминания» для ускорения рекурсивной оценки деревьев шашечной игры. Более*

общее предложение Мичи описывал как возможность для программиста «сэмюэлизировать» любые функции по желанию. Насколько мне известно, сам термин «мемоизация» Мичи не использовал. Ещё раньше этот приём применял робот Клода Шеннона «Тесей», решавший лабиринты и построенный в 1950 году.

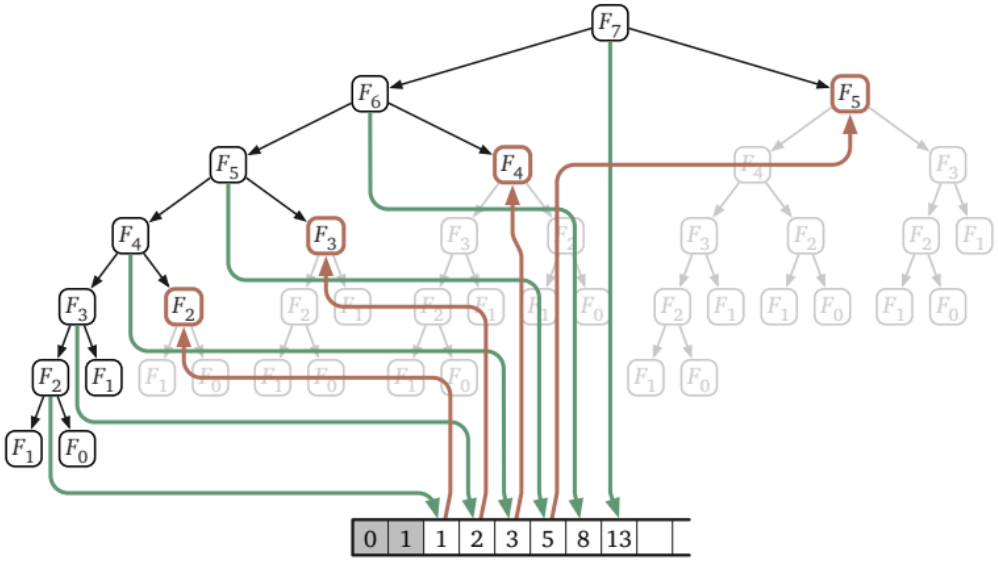


Рисунок 3.2. Дерево рекурсии для F_7, сокращённое мемоизацией. Зелёные стрелки вниз обозначают запись в массив мемоизации, красные стрелки вверх — чтение из него.

Динамическое программирование: заполняйте таблицу намеренно

Увидев, как заполняется массив `F[ ]`, можно заменить рекурсию с мемоизацией простым циклом `for`, который намеренно заполняет массив в этом порядке, вместо того чтобы полагаться на более сложный рекурсивный алгоритм, делающий это для нас как бы случайно.

```
IterFibo(n):
    F[0] ← 0
    F[1] ← 1
    для i ← 2 до n
        F[i] ← F[i – 1] + F[i – 2]
    вернуть F[n]
```

Теперь анализ времени непосредственен: очевидно, `IterFibo` выполняет $O(n)$ сложений и хранит $O(n)$ целых чисел.

Это наш первый явный алгоритм динамического программирования. Парадигму динамического программирования формализовал и популяризовал Ричард Беллман в середине 1950-х годов, работая в корпорации RAND, хотя он был далеко не первым, кто применил этот приём. В частности, данный итеративный алгоритм вычисления чисел Фибоначчи уже предлагали Вираханка и последующие санскритские просодисты XII века, а затем вновь Фибоначчи на рубеже XIII века![3-6]

Спустя много лет Беллман утверждал, что намеренно выбрал название «динамическое программирование», чтобы скрыть математический характер работы от военного начальства, активно враждебного ко всему, напоминавшему математические исследования.[3-7] Слово «программирование» здесь означает не написание кода, а планирование или составление расписания в более старом смысле, обычно путём заполнения таблицы. Например, спортивные и театральные программы — это расписания важных событий с рекламой; программирование телевидения означает заполнение каждого доступного временного интервала передачей и рекламой; учебные программы — расписания занятий с рекламой. Военно-воздушные силы финансировали работу Беллмана и других исследователей над методами составления расписаний обучения и материально-технического обеспечения, или, как они их называли, «программ». Слово «динамическое» не только отсылало к многоэтапным, меняющимся во времени процессам, которые Беллман и его коллеги пытались оптимизировать, но и было модным рекламным словом, созвучным футуристическому духу «мы всё можем»™ послевоенной Америки.[3-8] Отчасти благодаря пропаганде Беллмана динамическое программирование теперь стало стандартным инструментом многоэтапного планирования в экономике, робототехнике, теории управления и других областях.

И всё-таки не запоминайте всё

Во многих алгоритмах динамического программирования необязательно хранить все промежуточные результаты на протяжении всего вычисления. Например, можно значительно уменьшить требования `IterFibo` к памяти, сохраняя только два последних элемента массива:

Примечание 3-6. *Более общие методы динамического программирования независимо друг от друга неоднократно применялись в конце 1930-х и начале 1940-х. Например, Пьер Массе использовал их для оптимизации работы гидроэлектростанций во Франции при режиме Виши. Джон фон Нейман и Оскар Моргенштерн разработали алгоритмы динамического программирования для определения победителя в любой игре двух игроков с полной информацией, например в шашках. Алан Тьюринг и его коллеги применяли сходные методы при взломе шифров в Блетчли-парке. Работы Массе и фон Неймана с Моргенштерном впервые опубликованы в 1944 году, за шесть лет до того, как Беллман ввёл выражение «динамическое программирование». Подробности метода Тьюринга «банбуризмус» оставались секретными до середины 1980-х годов.*

Примечание 3-7. *Чарльз Эрвин Уилсон стал министром обороны в январе 1953 года после двенадцати лет на посту президента General Motors. «Моторный Чарли» реорганизовал министерство и в первый же год значительно сократил его бюджет, намеренно стремясь управлять им больше как промышленной корпорацией. В автобиографии 1984 года Беллман описал Уилсона так:*

> В Вашингтоне был очень интересный джентльмен по имени Уилсон. Он был министром обороны и действительно испытывал патологический страх и ненависть к слову «исследование». Я употребляю это слово не легкомысленно, а совершенно точно. Если кто-то произносил при нём «исследование», его лицо наливалось кровью, он краснел и приходил в ярость. Можете представить, что он чувствовал при слове «математический»… Я понимал, что должен как-то оградить Уилсона и ВВС от факта, что на

```
самом деле занимаюсь математикой внутри RAND. Какое название, какое имя мне выбрать?

Однако первое опубликованное употребление Беллманом термина «динамическое программирование»
относится уже к 1952 году, за несколько месяцев до вступления Уилсона в должность, поэтому эта
история как минимум немного приукрашена.
```

> ***Примечание 3-8.*** *...а возможно, название также обыгрывает культовую марку «Dynamic-Tension» знаменитого комплекса упражнений Чарльза Атласа, придуманную Чарльзом Романом в 1928 году. Герой пляжа!*

```
IterFibo2(n):
    prev ← 1
    curr ← 0
    для i ← 1 до n
        next ← curr + prev
        prev ← curr
        curr ← next
    вернуть curr
```

Этот алгоритм использует нестандартное, но согласованное базовое значение $F_{-1} = 1$, чтобы `IterFibo2(0)` правильно возвращал 0. Хотя экономия памяти на практике бывает чрезвычайно важна, в этой книге мы не будем сосредоточиваться на вопросах памяти.

♥ 3.2. Отступление: ещё более быстрое вычисление чисел Фибоначчи

Хотя предыдущий алгоритм прост и привлекателен, он не самый быстрый для вычисления чисел Фибоначчи. Более быстрый алгоритм можно вывести, воспользовавшись следующей матричной формой рекуррентного соотношения Фибоначчи:

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} y \\ x + y \end{bmatrix}.$$

Иными словами, умножение двумерного вектора на матрицу $\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ даёт точно такой же результат, как одна итерация внутреннего цикла `IterFibo2`. Следовательно, умножить на матрицу n раз — то же самое, что выполнить цикл n раз:

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} F_{n-1} \\ F_n \end{bmatrix}.$$

Значит, чтобы получить n-е число Фибоначчи, достаточно вычислить n-ю степень матрицы $\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$. При последовательном возведении в квадрат[3-9] для этого требуется всего $O(\log n)$ умножений. Здесь мы возводим в степень матрицу 2×2, поэтому нужно $O(\log n)$ умножений матриц 2×2, каждое из которых сводится к постоянному числу целочисленных умножений и сложений. Таким образом, F_n можно вычислить всего за $O(\log n)$ целочисленных арифметических операций.

Такого же ускорения можно добиться с помощью тождества $F_n = F_m F_{n-m-1} + F_{m+1} F_{n-m}$, которое выполняется для любых целых m и n — доказывается индукцией! В частности, оно даёт следующие взаимные рекуррентные соотношения для пар соседних чисел Фибоначчи, впервые предложенные Эдуаром Люка в 1898 году:

$$F_{2n-1} = F_{n-1}^2 + F_n^2,$$

$$F_{2n} = F_n(F_{n-1} + F_{n+1}) = F_n(2F_{n-1} + F_n).$$

Эти соотношения можно также вывести непосредственно из алгоритма возведения матрицы в квадрат. Они сразу приводят к следующему алгоритму:

```
⟨⟨Вычислить пару F[n-1], F[n]⟩⟩
FastRecFibo(n):
    если n = 1
        вернуть 0, 1
    m ← ⌊n/2⌋
    hprv, hcur ← FastRecFibo(m)  ⟨⟨F[m-1], F[m]⟩⟩
    prev ← hprv² + hcur²         ⟨⟨F[2m-1]⟩⟩
    curr ← hcur · (2 · hprv + hcur)  ⟨⟨F[2m]⟩⟩
    next ← prev + curr            ⟨⟨F[2m+1]⟩⟩
    если n чётно
        вернуть prev, curr
    иначе
        вернуть curr, next
```

Наш обычный метод дерева рекурсии показывает, что этот алгоритм выполняет всего $O(\log n)$ целочисленных арифметических операций.

Это экспоненциальное ускорение по сравнению со стандартным итеративным алгоритмом, который уже был экспоненциально быстрее первоначального рекурсивного. Верно?

Погодите! Не так быстро!

На самом деле не совсем. Числа Фибоначчи растут экспоненциально. В n-м числе Фибоначчи примерно $n \log_{10} \varphi \approx n/5$ десятичных цифр, или $n \log_2 \varphi \approx 2n/3$ бит. Поэтому мы никак не можем вычислить F_n за логарифмическое время: только чтобы записать ответ, нужно $\Omega(n)$ времени!

Выход из этого кажущегося парадокса в том, что арифметику произвольной точности нельзя выполнять за постоянное время. Пусть $M(n)$ — время, необходимое для умножения двух n-значных чисел. Время работы `FastRecFibo` удовлетворяет соотношению $T(n) = T(\lfloor n/2 \rfloor) + M(n)$, из которого методом деревьев рекурсии получается $T(n) = O(M(n))$. Самый быстрый известный на 2019 год алгоритм умножения целых чисел работает за $O(n \log n)$ времени; таково же время работы самого быстрого известного на 2019 год алгоритма вычисления чисел Фибоначчи.

Медленнее ли этот алгоритм наших итеративных алгоритмов с «линейным временем»? На самом деле нет: сложение тоже не бесплатно! Сложение двух n-значных чисел требует $O(n)$ времени, поэтому итеративные алгоритмы `IterFibo` и `IterFibo2` в действительности работают за

$O(n^2)$. Понимаете почему? Значит, `FastRecFibo` существенно быстрее итеративных алгоритмов, хотя и не экспоненциально быстрее.

В исходном рекурсивном алгоритме дополнительные затраты на арифметику произвольной точности перекрываются огромным числом рекурсивных вызовов. Правильное рекуррентное соотношение $T(n) = T(n-1) + T(n-2) + O(n)$ по-прежнему имеет решение $T(n) = O(\varphi^n)$.

> ***Примечание 3-9.*** *Как предложил Пингала для степеней двойки в другом месте Чхандахшастры.*

3.3. Возвращение к interpunctio verborum

Для следующего алгоритма динамического программирования рассмотрим задачу разбиения текста из предыдущей главы. Дана строка $A[1..n]$ и подпрограмма `IsWord`, которая определяет, является ли заданная строка словом — что бы это ни значило. Нужно узнать, можно ли разбить A на последовательность слов.

Мы решили задачу, определив функцию `Splittable(i)`, которая возвращает `True` тогда и только тогда, когда суффикс $A[i..n]$ можно разбить на последовательность слов. Нужно вычислить `Splittable(1)`. Эта функция удовлетворяет рекуррентному соотношению

$$\text{Splittable}(i) = \begin{cases} \text{True}, & i > n, \\ \displaystyle\bigvee_{j=i}^{n} \big(\text{IsWord}(i,j) \wedge \text{Splittable}(j+1)\big), & \text{иначе}, \end{cases}$$

где `IsWord(i,j)` — сокращение для `IsWord(A[i..j])`. Это соотношение непосредственно превращается в рекурсивный алгоритм с возвратом, который в худшем случае вызывает `IsWord` $O(2^n)$ раз.

Но для любой фиксированной строки $A[1..n]$ существует всего n различных способов вызвать рекурсивную функцию `Splittable(i)` — по одному для каждого i от 1 до $n+1$ — и всего $O(n^2)$ различных вызовов `IsWord(i,j)`, по одному для каждой пары $1 \le i \le j \le n$. Зачем тратить экспоненциальное время на вычисление полиномиального объёма данных?

> *Примечание переводчика: перечисленные автором значения от 1 до $n+1$ включительно дают $n+1$ возможных вызовов, а не n; асимптотическая оценка при этом не меняется.*

Каждая рекурсивная подзадача задаётся целым числом от 1 до $n+1$, поэтому можно мемоизировать функцию `Splittable` в массиве `SplitTable[1..n+1]`. Каждая подзадача `Splittable(i)` зависит только от результатов подзадач `Splittable(j)` с $j > i$, поэтому рекурсивный алгоритм с мемоизацией заполняет массив в порядке убывания индексов. Если намеренно заполнять его в этом порядке, получится алгоритм динамического программирования на рисунке 3.3. Он вызывает `IsWord` $O(n^2)$ раз — экспоненциальное улучшение по сравнению с прежним поиском с возвратом.

```
FastSplittable(A[1 .. n]):
    SplitTable[n + 1] ← True
    для i ← n вниз до 1
        SplitTable[i] ← False
        для j ← i до n
            если IsWord(i, j) и SplitTable[j + 1]
                SplitTable[i] ← True
    вернуть SplitTable[1]
```

Рисунок 3.3. Быстрое разделение слов (*Interpunctio verborum velox*).

3.4. Общая схема: умная рекурсия

В двух словах, динамическое программирование — это рекурсия без повторений. Алгоритмы динамического программирования сохраняют решения промежуточных подзадач, часто, хотя и не всегда, в каком-либо массиве или таблице. Многие студенты, преподаватели и авторы учебников ошибочно сосредоточиваются на таблице, потому что таблицы просты и знакомы, вместо гораздо более важной и трудной задачи поиска правильного рекуррентного соотношения. Пока мы мемоизируем правильное соотношение, явная таблица на самом деле необязательна; но если соотношение неверно, мы в серьёзной беде.

Динамическое программирование — не о заполнении таблиц. Оно об умной рекурсии!

Алгоритмы динамического программирования лучше всего разрабатывать в два отдельных этапа.

1. **Сформулируйте задачу рекурсивно.** Запишите рекурсивную формулу или алгоритм для всей задачи через ответы к меньшим подзадачам. Это сложная часть. Полная рекурсивная формулировка состоит из двух частей:
    - **Постановка.** Связно и точно опишите задачу, которую хотите решить рекурсивно: не как её решить, а какую именно задачу вы решаете. Без этого описания невозможно даже в принципе определить, правильно ли ваше решение.
    - **Решение.** Дайте ясную рекурсивную формулу или алгоритм для всей задачи через ответы для меньших экземпляров в точности той же задачи.
2. **Постройте решения рекуррентного соотношения снизу вверх.** Напишите алгоритм, который начинает с базовых случаев и продвигается к окончательному решению, рассматривая промежуточные подзадачи в правильном порядке. Этот этап можно разбить на несколько сравнительно механических шагов:
    - **Определите подзадачи.** Какими разными способами ваш рекурсивный алгоритм может вызвать сам себя, начиная с некоторого исходного ввода? Например, аргумент `RecFibo` всегда является целым числом от 0 до n.
    - **Выберите структуру данных для мемоизации.** Найдите структуру, которая может хранить решение каждой подзадачи из предыдущего шага. Обычно, хотя и не всегда, это многомерный массив.
    - **Определите зависимости.** От каких других подзадач зависит каждая подзадача, кроме базовых случаев? Нарисуйте свою структуру данных, выберите

произвольный элемент и проведите к нему стрелки от всех элементов, от которых он зависит. Затем формализуйте получившуюся картину.

- **Найдите подходящий порядок вычисления.** Упорядочьте подзадачи так, чтобы каждая шла после тех, от которых зависит. Сначала рассмотрите базовые случаи, затем подзадачи, зависящие только от них, и так далее, постепенно добираясь до исходной задачи верхнего уровня. Найденные зависимости задают частичный порядок подзадач; вам нужно найти его линейное расширение. Будьте внимательны!
- **Проанализируйте память и время.** Число различных подзадач определяет пространственную сложность алгоритма с мемоизацией. Чтобы вычислить общее время, сложите время решения всех возможных подзадач, предполагая, что результаты более глубоких рекурсивных вызовов уже сохранены. Это можно сделать сразу после определения подзадач.
- **Запишите алгоритм.** Вы знаете порядок рассмотрения подзадач и способ решения каждой из них. Так сделайте это! Если используется массив, обычно достаточно окружить исходное рекуррентное соотношение несколькими вложенными циклами `for` и заменить рекурсивные вызовы обращениями к массиву.

Разумеется, правильность каждого шага необходимо доказать. Если рекуррентное соотношение неверно или ответы строятся не в том порядке, алгоритм работать не будет!

3.5. Предупреждение: жадность — это глупо

Если нам невероятно повезёт, можно обойтись без всех рекуррентных соотношений, таблиц и прочего и решить задачу жадным алгоритмом. Как и алгоритм с возвратом, жадный алгоритм строит решение последовательностью выборов, но делает их непосредственно, не решая рекурсивных подзадач. Хотя такой подход кажется очень естественным, он почти никогда не работает: задачи оптимизации, правильно решаемые жадным алгоритмом, встречаются довольно редко. И всё же для многих задач, которые следует решать поиском с возвратом или динамическим программированием, первая интуиция студентов — применить жадную стратегию.

Например, жадный алгоритм разбиения текста может найти кратчайший или, если хотите, длиннейший префикс входной строки, являющийся словом, принять его за первое слово разбиения, а затем рекурсивно разбить оставшийся суффикс. Аналогично жадный алгоритм для задачи о наибольшей возрастающей подпоследовательности может найти наименьший элемент входного массива, принять его за начало искомой подпоследовательности, а затем рекурсивно искать наибольшую возрастающую подпоследовательность справа от него. Если эти приёмы кажутся вам глупыми уловками, похлопайте себя по плечу: до правильных решений им очень далеко.

Каждому стоит вытатуировать следующую фразу на тыльной стороне ладони, сразу под всеми правилами о логарифмах и обозначении большого O:

Жадные алгоритмы никогда не работают! Используйте динамическое программирование!

— Что, никогда?

— Нет, никогда!

— Что, никогда-никогда?

— Ну... почти никогда.[3-10]

Поскольку жадный подход невероятно соблазнителен, но редко бывает правильным, я настоятельно рекомендую следующее правило для любого курса алгоритмов, даже — или, возможно, особенно — если в нём обычно не требуют доказательств правильности:[3-11]

Вы не получите баллов ни за один жадный алгоритм ни в домашнем задании, ни на экзамене, даже если он правилен, без формального доказательства его правильности.

Более того, подавляющее большинство задач, для которых студентам хочется предложить жадный алгоритм, на самом деле лучше всего решаются динамическим программированием. Поэтому я всегда даю студентам такой совет:

Когда вы пишете или даже думаете слово «greeDY», ваше подсознание подсказывает использовать DYnamic programming — динамическое программирование.

Даже для задач, корректно решаемых жадными алгоритмами, обычно продуктивнее сначала разработать алгоритм с возвратом или динамическим программированием. Сначала добейтесь правильной работы, затем ускоряйте. В следующей главе мы увидим методы доказательства правильности жадных алгоритмов.

> ***Примечание 3-10.*** *Они почти никогда-никогда не работают! Тогда трижды ура и ещё раз ура строгому капитану «Пинафора»! Трижды ура и ещё раз ура капитану «Пинафора»!*

> ***Примечание 3-11.*** *Введение этого правила в моих собственных курсах алгоритмов значительно улучшило оценки студентов, потому что число неправильных жадных алгоритмов существенно сократилось.*

3.6. Наибольшая возрастающая подпоследовательность

Ещё одна задача предыдущей главы — вычислить длину наибольшей возрастающей подпоследовательности заданного массива чисел $A[1..n]$. Мы разработали для неё два разных рекурсивных алгоритма с возвратом. Оба в худшем случае работают за $O(2^n)$ времени, и оба можно значительно ускорить с помощью динамического программирования.

Первое рекуррентное соотношение: взять следующим этот элемент?

Первый алгоритм с возвратом вычислял функцию `LISbigger(i,j)` — длину наибольшей возрастающей подпоследовательности $A[j..n]$, каждый элемент которой больше $A[i]$. Мы вывели следующее соотношение:

$$\text{LISbigger}(i,j) = \begin{cases} 0, & j > n, \\ \text{LISbigger}(i, j+1), & A[i] \geq A[j], \\ \max\{\text{LISbigger}(i,j+1),\ 1 + \text{LISbigger}(j, j+1)\}, & \text{иначе.} \end{cases}$$

Чтобы решить исходную задачу, можно добавить в массив сторожевое значение $A[0] = -\infty$ и вычислить `LISbigger(0,1)`.

Каждая рекурсивная подзадача определяется двумя индексами i и j, поэтому рассмотреть нужно лишь $O(n^2)$ различных рекурсивных подзадач. Их результаты можно мемоизировать в двумерном массиве `LISbigger[0..n, 1..n]`.[3-12] Более того, каждая подзадача решается за $O(1)$ времени, не считая рекурсивных вызовов, поэтому итоговый алгоритм динамического программирования должен работать за $O(n^2)$ времени.

Порядок заполнения массива рекурсивным алгоритмом с мемоизацией не очевиден. Из соотношения известно лишь, что каждая ячейка `LISbigger[i,j]` заполняется после ячеек `LISbigger[i,j+1]` и `LISbigger[j,j+1]` следующего столбца, как показано слева на рисунке 3.4.

К счастью, этой частичной информации достаточно для выбора допустимого порядка вычислений. Если заполнять таблицу по одному столбцу справа налево, то при обращении к любой ячейке необходимые ей значения уже будут доступны. Возможно, рекурсивный алгоритм использовал бы другой порядок, но этот работает, поэтому остановимся на нём. Правая часть рисунка 3.4 иллюстрирует этот порядок: двойная стрелка обозначает внешний цикл, а одиночные — внутренний. В данном случае одиночные стрелки двунаправленные, потому что порядок заполнения ячеек внутри каждого столбца не имеет значения.

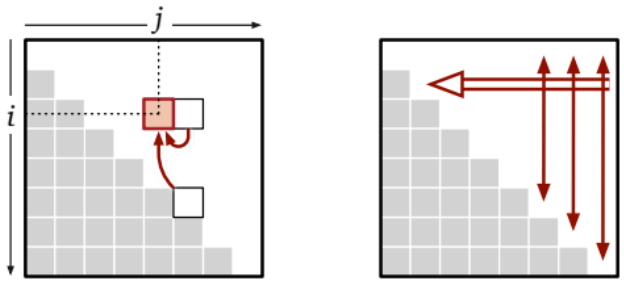


Рисунок 3.4. Зависимости подзадач для наибольшей возрастающей подпоследовательности и допустимый порядок вычисления.

Готово! Ниже приведён псевдокод алгоритма динамического программирования. Как и ожидалось, он работает за $O(n^2)$ времени. При необходимости можно уменьшить объём памяти с $O(n^2)$ до $O(n)$, сохраняя только два последних столбца таблицы: `LISbigger[·,j]` и `LISbigger[·,j+1]`.[3-13]

```
FastLIS(A[1 .. n]):
    A[0] ← -∞                  ⟨⟨Добавить сторожевой элемент⟩⟩
    для i ← 0 до n             ⟨⟨Базовые случаи⟩⟩
        LISbigger[i, n + 1] ← 0
    для j ← n вниз до 1
        для i ← 0 до j - 1     ⟨⟨... или в другом порядке⟩⟩
            keep ← 1 + LISbigger[j, j + 1]
            skip ← LISbigger[i, j + 1]
            если A[i] ≥ A[j]
                LISbigger[i, j] ← skip
            иначе
                LISbigger[i, j] ← max{keep, skip}
    вернуть LISbigger[0, 1]
```

Второе рекуррентное соотношение: что дальше?

Второй алгоритм с возвратом вычислял функцию `LISfirst(i)` — длину наибольшей возрастающей подпоследовательности $A[i..n]$, начинающейся с $A[i]$. Для неё мы вывели соотношение

$$\text{LISfirst}(i) = 1 + \max\{\text{LISfirst}(j) \mid j > i \text{ и } A[j] > A[i]\}.$$

Здесь предполагается, что $\max \varnothing = 0$, поэтому базовые случаи вроде `LISfirst(n)=1` автоматически следуют из соотношения. Для решения исходной задачи можно добавить в массив сторожевое значение $A[0] = -\infty$ и вычислить `LISfirst(0)-1`.

Теперь рекурсивные подзадачи задаются единственным индексом i, поэтому соотношение можно мемоизировать в одномерном массиве `LISfirst[1..n]`. Каждая ячейка `LISfirst[i]` зависит только от ячеек `LISfirst[j]` с $j > i$, поэтому массив можно заполнять в порядке убывания индексов. Чтобы вычислить каждое `LISfirst[i]`, нужно рассмотреть `LISfirst[j]` для всех индексов $j > i$, но делать это в каком-то определённом порядке необязательно. Полученный алгоритм динамического программирования работает за $O(n^2)$ времени и использует $O(n)$ памяти.

```
FastLIS2(A[1 .. n]):
    A[0] ← -∞                  ⟨⟨Добавить сторожевой элемент⟩⟩
    для i ← n вниз до 0
        LISfirst[i] ← 1
        для j ← i + 1 до n     ⟨⟨... или в другом порядке⟩⟩
            если A[j] > A[i] и 1 + LISfirst[j] > LISfirst[i]
                LISfirst[i] ← 1 + LISfirst[j]
    вернуть LISfirst[0] - 1    ⟨⟨Не считать сторожевой элемент⟩⟩
```

Примечание 3-12. *На самом деле нужна только половина этого массива, поскольку всегда $i < j$. Но даже если бы нас в этой книге интересовали постоянные множители — а это не так, — сейчас было бы неподходящее время о них беспокоиться. Сначала добейтесь правильной работы, затем улучшайте.*

Примечание 3-13. *Вот видите, я же говорил пока не беспокоиться о постоянных множителях!*

3.7. Расстояние редактирования

Расстояние редактирования между двумя строками — это минимальное число вставок, удалений и замен символов, необходимых для превращения одной строки в другую. Например, расстояние между `FOOD` и `MONEY` не больше четырёх:

```
FOOD → MOOD → MOND → MONED → MONEY
```

Здесь переход `MOOD → MOND` заменяет второй `O` на `N`, затем вставляется `E`, а переход `MONED → MONEY` заменяет `D` на `Y`.

Эта функция расстояния была независимо предложена Владимиром Левенштейном в 1965 году при работе над теорией кодирования, Тарасом Винцюком в 1968 году при работе над распознаванием речи и Станиславом Уламом в 1972 году при работе с биологическими последовательностями. Поэтому расстояние редактирования иногда называют расстоянием Левенштейна или расстоянием Улама, но, как ни странно, никогда — «расстоянием Винцюка».

Процесс редактирования можно представить наглядно, выровняв строки одну над другой и оставив пропуск в первом слове для каждой вставки и во втором — для каждого удаления. Столбцы с двумя разными символами соответствуют заменам. В таком представлении число операций редактирования равно числу столбцов, не содержащих два одинаковых символа.

```
F O O _ D
M O N E Y
```

Довольно очевидно, что превратить `FOOD` в `MONEY` за три операции нельзя, поэтому расстояние между ними равно ровно четырём. К сожалению, в общем случае не так просто определить, является ли последовательность правок кратчайшей. Например, следующее выравнивание показывает, что расстояние между `ALGORITHM` и `ALTRUISTIC` не больше 6. Можно ли добиться лучшего результата?

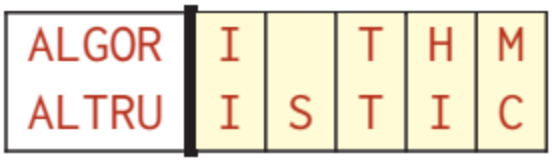
```
A L G O R   I   T H M
A L   T R U I S T I C
```

Рекурсивная структура

Чтобы разработать алгоритм динамического программирования для вычисления расстояния редактирования, сначала необходимо сформулировать задачу рекурсивно. Представление последовательностей правок с помощью выравнивания обладает важным свойством *оптимальной подструктуры*. Предположим, что у нас есть представление с пропусками для кратчайшей последовательности правок двух строк. Если удалить последний столбец, оставшиеся столбцы должны представлять кратчайшую последовательность правок для оставшихся префиксов. Это легко доказать от противного: если бы у префиксов была более короткая последовательность правок, добавив обратно последний столбец, мы получили бы более короткую последовательность и для исходных строк.

Поэтому, как только мы выясним, что должно находиться в последнем столбце, Фея рекурсии сможет определить остальную часть оптимального представления с пропусками.

Иными словами, искомое выравнивание представляет последовательность операций редактирования, упорядоченных — без особой причины — справа налево. Решая задачу расстояния редактирования, мы делаем последовательность выборов, по одному на каждый столбец итогового выравнивания. В середине этого процесса суффикс одной строки уже выровнен с суффиксом другой.



Поскольку стоимость выравнивания равна числу несовпадающих столбцов, оставшиеся решения не зависят от уже выбранных операций редактирования: они зависят только от ещё не выровненных префиксов.

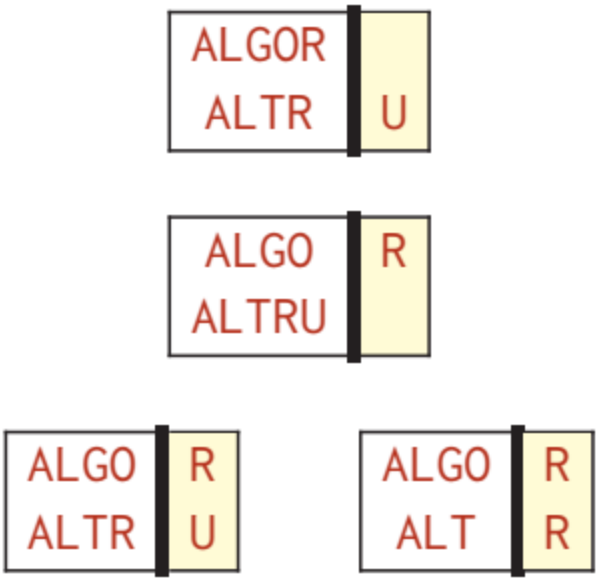


Итак, для любых входных строк $A[1..m]$ и $B[1..n]$ можно сформулировать задачу рекурсивно. Для любых индексов i и j пусть `Edit(i,j)` обозначает расстояние редактирования между префиксами $A[1..i]$ и $B[1..j]$. Нужно вычислить `Edit(m,n)`.

Рекуррентное соотношение

Если i и j положительны, для последнего столбца оптимального выравнивания $A[1..i]$ и $B[1..j] есть ровно три возможности:

- **Вставка:** последний элемент верхней строки пуст. Тогда расстояние равно `Edit(i,j-1)+1`: единица — стоимость последней вставки, а рекурсивное выражение даёт минимальную стоимость остального выравнивания.
- **Удаление:** последний элемент нижней строки пуст. Тогда расстояние равно `Edit(i-1,j)+1`: единица — стоимость последнего удаления, а рекурсивное выражение даёт минимальную стоимость остального выравнивания.
- **Замена:** в последнем столбце обеих строк есть символы. Если они различны, расстояние равно `Edit(i-1,j-1)+1`. Если одинаковы, замена бесплатна и расстояние равно `Edit(i-1,j-1)`.



Такой общий разбор перестаёт работать, если $i = 0$ или $j = 0$, но граничные случаи легко обработать непосредственно:

- Для превращения пустой строки в строку длиной j нужны j вставок, поэтому `Edit(0,j)=j`.
- Для превращения строки длиной i в пустую нужны i удалений, поэтому `Edit(i,0)=i`.

Проверка на здравый смысл: оба базовых случая правильно показывают, что расстояние между двумя пустыми строками равно нулю!

Итак, функция `Edit` удовлетворяет соотношению

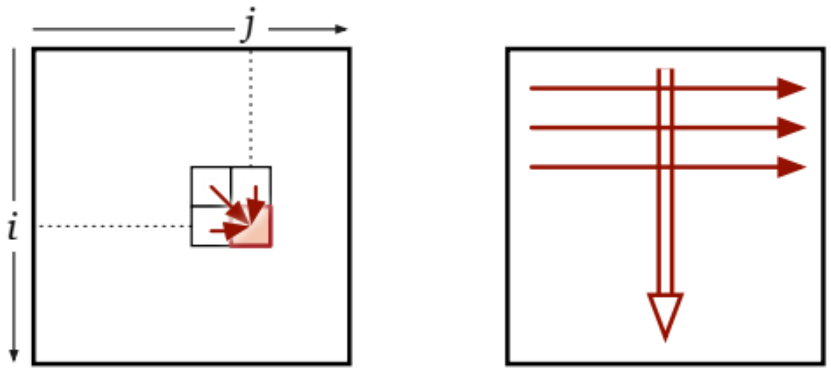
$$\text{Edit}(i,j) = \begin{cases} i, & j = 0, \\ j, & i = 0, \\ \min \begin{cases} \text{Edit}(i, j-1) + 1, \\ \text{Edit}(i-1, j) + 1, \\ \text{Edit}(i-1, j-1) + [A[i] \neq B[j]] \end{cases}, & \text{иначе.} \end{cases}$$

Динамическое программирование

Теперь, когда у нас есть рекуррентное соотношение, преобразуем его в алгоритм динамического программирования по привычной схеме.

- **Подзадачи.** Каждая рекурсивная подзадача задаётся индексами $0 \le i \le m$ и $0 \le j \le n$.
- **Структура мемоизации.** Все возможные значения `Edit(i,j)` можно сохранить в двумерном массиве `Edit[0..m,0..n]`.
- **Зависимости.** Каждая ячейка `Edit[i,j]` зависит только от трёх соседей: `Edit[i-1,j]`, `Edit[i,j-1]` и `Edit[i-1,j-1]`.
- **Порядок вычисления.** Если заполнять массив в обычном построчном порядке — строка за строкой сверху вниз, а внутри строки слева направо, — то при обращении к

любой ячейке все значения, от которых она зависит, уже доступны. Это не единственный возможный порядок, но он работает, поэтому остановимся на нём.



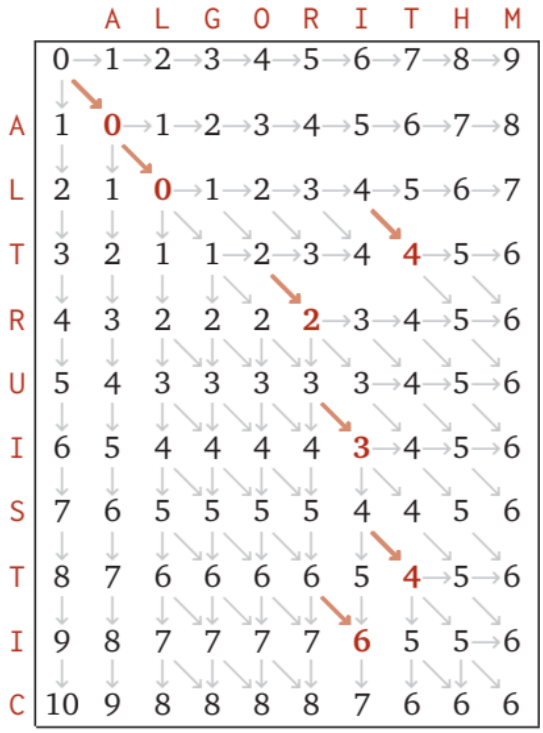
- **Память и время.** Структура мемоизации занимает $O(mn)$ памяти. Каждую ячейку `Edit[i,j]` можно вычислить за $O(1)$ времени, если известны её предшественники, поэтому весь алгоритм работает за $O(mn)$ времени.

Получаем алгоритм:

```
EditDistance(A[1 .. m], B[1 .. n]):
    для j ← 0 до n
        Edit[0, j] ← j
    для i ← 1 до m
        Edit[i, 0] ← i
        для j ← 1 до n
            ins ← Edit[i, j – 1] + 1
            del ← Edit[i – 1, j] + 1
            если A[i] = B[j]
                rep ← Edit[i – 1, j – 1]
            иначе
                rep ← Edit[i – 1, j – 1] + 1
            Edit[i, j] ← min{ins, del, rep}
    вернуть Edit[m, n]
```

Этот алгоритм чаще всего приписывают Роберту Вагнеру и Майклу Фишеру, описавшим его в 1974 году. Однако в полном соответствии с законом эпонимии Стиглера идентичные или более общие алгоритмы независимо друг от друга открыли Тарас Винцюк в 1968 году, В. М. Величко и Н. Г. Загоруйко в 1970 году, Дэвид Санкофф в 1972 году, Питер Селлерс в 1974 году и почти наверняка ещё несколько человек.[3-14] Любопытно, что никто из этих авторов не ссылается ни на Левенштейна, ни на Улама!

Таблица мемоизации для строк `ALGORITHM` и `ALTRUISTIC` показана ниже.



Жирные числа обозначают места, где символы двух строк равны. Расстояние между `ALGORITHM` и `ALTRUISTIC` действительно равно шести!

Стрелки в таблице показывают, какие предшествующие значения действительно определяют каждую ячейку. Каждому направлению соответствует операция редактирования: горизонтальному — удаление, вертикальному — вставка, диагональному — замена. Жирные красные диагональные стрелки обозначают «бесплатную» замену буквы самой собой. Любой путь по стрелкам из верхнего левого угла в нижний правый задаёт оптимальную последовательность правок двух строк. В этом примере таблица содержит ровно три ориентированных пути между углами; каждый задаёт свою последовательность из шести операций превращения `ALGORITHM` в `ALTRUISTIC`, показанную ниже.

```
A L G O R I   T H M
A L T R U I S T I C

A L G O R   I   T H M
A L   T R U I S T I C

A L G O R   I   T H M
A L T   R U I S T I C
```

Наш алгоритм `EditDistance` на самом деле не вычисляет и не хранит стрелки в таблице, но стрелки, входящие в любую ячейку, можно восстановить по числовым значениям на ходу за $O(1)$ времени. Значит, заполнив таблицу, можно восстановить кратчайшую последовательность правок за дополнительное время $O(n + m)$.

Примечание 3-14. *Этот алгоритм также иногда ошибочно приписывают Солу Нидлману и Кристиану Вуншу в 1970 году. Название «алгоритм Нидлмана — Вунша» чаще относится к стандартному алгоритму динамического программирования, вычисляющему наибольшую общую подпоследовательность двух строк — или, что равносильно, расстояние редактирования, когда разрешены только вставки и удаления, — за $O(mn)$ времени. Но и эта атрибуция неверна! На самом деле алгоритм Нидлмана и Вунша вычисляет взвешенные наибольшие общие подпоследовательности, возможно со стоимостью пропусков, за $O(m^2n^2)$ времени, используя другое рекуррентное соотношение. Санкофф явно описывает свой алгоритм времени $O(mn)$ как улучшение алгоритма Нидлмана и Вунша.*

3.8. Сумма подмножества

Напомним: задача о сумме подмножества спрашивает, есть ли среди элементов заданного массива положительных целых чисел $X[1..n]$ подмножество с суммой, равной заданному целому числу T. В предыдущей главе мы разработали рекурсивный алгоритм, который можно переформулировать следующим образом. Зафиксируем исходный массив $X[1..n]$ и определим логическую функцию: `SS(i,t) = True` тогда и только тогда, когда некоторое подмножество $X[i..n]$ имеет сумму t.

Нужно вычислить `SS(1,T)`. Функция удовлетворяет соотношению

$$\mathrm{SS}(i,t) = \begin{cases} \text{True}, & t = 0, \\ \text{False}, & t < 0 \text{ или } i > n, \\ \mathrm{SS}(i+1,t) \vee \mathrm{SS}(i+1,t-X[i]), & \text{иначе.} \end{cases}$$

Преобразуем его в алгоритм динамического программирования по обычному шаблону.

- **Подзадачи.** Каждая подзадача задаётся целым i при $1 \le i \le n+1$ и целым $t \le T$. Однако подзадачи с $t < 0$ тривиальны, поэтому сохранять их результаты кажется довольно глупым.[3-15] Более того, можно изменить соотношение так, чтобы они вообще не возникали:

$$\mathrm{SS}(i,t) = \begin{cases} \text{True}, & t = 0, \\ \text{False}, & i > n, \\ \mathrm{SS}(i+1,t), & t < X[i], \\ \mathrm{SS}(i+1,t) \vee \mathrm{SS}(i+1,t-X[i]), & \text{иначе.} \end{cases}$$

- **Структура данных.** Результаты соотношения можно сохранить в двумерном массиве `S[1..n+1,0..T]`, где `S[i,t]` хранит значение `SS(i,t)`.
- **Порядок вычисления.** Каждая ячейка `S[i,t]` зависит не более чем от двух других, обе имеют вид `S[i+1,·]`. Поэтому внешний цикл может перебирать строки снизу вверх, а внутренний — элементы каждой строки в произвольном порядке.
- **Память и время.** Структура мемоизации использует $O(nT)$ памяти. Если `S[i+1,t]` и `S[i+1,t-X[i]]` уже известны, `S[i,t]` вычисляется за постоянное время, так что алгоритм

работает за $O(nT)$ времени.

Получается алгоритм:

```
FastSubsetSum(X[1 .. n], T):
    S[n + 1, 0] ← True
    для t ← 1 до T
        S[n + 1, t] ← False
    для i ← n вниз до 1
        S[i, 0] ← True
        для t ← 1 до X[i] – 1
            S[i, t] ← S[i + 1, t]  ⟨⟨Избежать случая t < 0⟩⟩
        для t ← X[i] до T
            S[i, t] ← S[i + 1, t] ∨ S[i + 1, t – X[i]]
    вернуть S[1, T]
```

> *Примечание переводчика: в исходном псевдокоде первая внутренняя петля доходит до $X[i] - 1$, даже если это больше T. Чтобы не выйти за границы объявленной таблицы, её верхнюю границу следует ограничить значением $\min\{T, X[i] - 1\}$.*

Оценка времени в худшем случае $O(nT)$ значительно лучше, чем $O(2^n)$ у рекурсивного поиска с возвратом, когда T мало.[3-16] Однако если искомая сумма T значительно больше 2^n, этот итеративный алгоритм на самом деле медленнее наивного рекурсивного: он тратит время на подзадачи, которые рекурсивный алгоритм вообще не рассматривает. Динамическое программирование улучшает решение не всегда![3-17]

> ***Примечание 3-15.*** *Да, я нарушаю собственное правило против преждевременной оптимизации.*

> ***Примечание 3-16.*** *Хотя задача о сумме подмножества является NP-трудной, эта оценка времени не означает, что P=NP, поскольку T не обязательно ограничено полиномиальной функцией размера входа.*

> ***Примечание 3-17.*** *В исследовательском меморандуме 1967 года, где Дональд Мичи предложил мемо-функции, он написал: «Табулировать значения функции, которые не понадобятся, — пустая трата памяти, а вычислять одни и те же значения больше одного раза — пустая трата времени». Но в действительности табулирование ненужных значений — также пустая трата времени!*

3.9. Оптимальные двоичные деревья поиска

Последней задачей предыдущей главы была задача об оптимальном двоичном дереве поиска. На вход подаются отсортированный массив ключей поиска $A[1..n]$ и массив частот $f[1..n]$, где $f[i]$ — число обращений к ключу $A[i]$. Нужно построить для этого множества двоичное дерево поиска так, чтобы суммарная стоимость всех поисков была минимальной.

Зафиксируем массив частот f и обозначим через `OptCost(i,k)` суммарное время поиска в оптимальном дереве для подмассива $A[i..k]$. Для функции `OptCost` мы вывели соотношение

$$\mathrm{OptCost}(i,k) = \begin{cases} 0, & i > k, \\ \displaystyle\sum_{j=i}^{k} f[j] + \min_{i \le r \le k} \{\mathrm{OptCost}(i, r-1) + \mathrm{OptCost}(r+1, k)\}, & \text{иначе.} \end{cases}$$

Вы, наверное, догадываетесь, что мы в итоге сделаем с этим соотношением, но сначала избавимся от некрасивой суммы.

Для пары индексов $i \le k$ пусть $F(i,k)$ — суммарная частота всех ключей интервала $A[i..k]$:

$$F(i,k) := \sum_{j=i}^{k} f[j].$$

Эта функция удовлетворяет простому соотношению:

$$F(i,k) = \begin{cases} f[i], & i = k, \\ F(i,k-1) + f[k], & \text{иначе.} \end{cases}$$

Все возможные значения $F(i,k)$ можно вычислить за $O(n^2)$ времени с помощью — вы угадали! — динамического программирования. Обычные механические шаги дают следующий алгоритм:

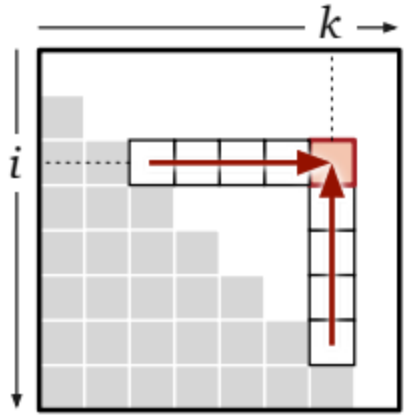
```
InitF(f[1 .. n]):
    для i ← 1 до n
        F[i, i - 1] ← 0
        для k ← i до n
            F[i, k] ← F[i, k - 1] + f[k]
```

Этот короткий алгоритм будет использоваться как подпрограмма инициализации. Она позволяет упростить исходное соотношение:

$$\mathrm{OptCost}(i,k) = \begin{cases} 0, & i > k, \\ F[i,k] + \min_{i \le r \le k} \{\mathrm{OptCost}(i, r-1) + \mathrm{OptCost}(r+1, k)\}, & \text{иначе.} \end{cases}$$

Теперь приводим механизм в действие.

- **Подзадачи.** Каждая рекурсивная подзадача задаётся двумя целыми числами i и k, где $1 \le i \le n+1$ и $0 \le k \le n$.
- **Мемоизация.** Все значения `OptCost` можно хранить в двумерном массиве `OptCost[1..n+1,0..n]`. На самом деле используются только ячейки `OptCost[i,j]` с $j \ge i - 1$, но это неважно.
- **Зависимости.** Каждая ячейка `OptCost[i,k]` зависит от ячеек `OptCost[i,j-1]` и `OptCost[j+1,k]` для всех j с $i \le j \le k$. Иными словами, каждая ячейка таблицы зависит от всех ячеек непосредственно слева или непосредственно снизу.



Следующая подпрограмма заполняет `OptCost[i,k]`, предполагая, что все необходимые ей значения уже вычислены:

```
ComputeOptCost(i, k):
    OptCost[i, k] ← ∞
    для r ← i до k
        tmp ← OptCost[i, r – 1] + OptCost[r + 1, k]
        если OptCost[i, k] > tmp
            OptCost[i, k] ← tmp
    OptCost[i, k] ← OptCost[i, k] + F[i, k]
```

- **Порядок вычисления.** Массив можно заполнять как минимум тремя способами. Большинство студентов сначала предлагает проходить по одной диагонали за раз, начиная с тривиальных базовых случаев `OptCost[i,i-1]` и двигаясь к итоговому ответу `OptCost[1,n]`:

```
OptimalBST(f[1 .. n]):
    InitF(f[1 .. n])
    для i ← 1 до n + 1
        OptCost[i, i – 1] ← 0
    для d ← 0 до n – 1
        для i ← 1 до n – d  ⟨⟨... или в другом порядке⟩⟩
            ComputeOptCost(i, i + d)
    вернуть OptCost[1, n]
```

Можно также проходить массив по строкам снизу вверх, каждую строку — слева направо, либо по столбцам слева направо, каждый столбец — снизу вверх:

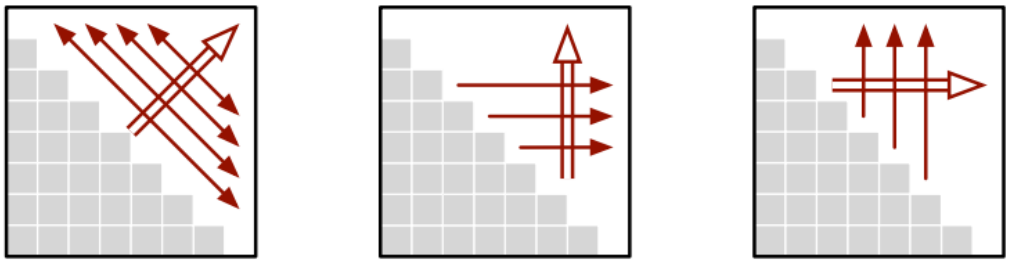
```
OptimalBST2(f[1 .. n]):
    InitF(f[1 .. n])
    для i ← n + 1 вниз до 1
        OptCost[i, i – 1] ← 0
        для j ← i до n
            ComputeOptCost(i, j)
    вернуть OptCost[1, n]

OptimalBST3(f[1 .. n]):
    InitF(f[1 .. n])
    для j ← 0 до n + 1
```

```
        OptCost[j + 1, j] ← 0
        для i ← j вниз до 1
            ComputeOptCost(i, j)
    вернуть OptCost[1, n]
```

Примечание переводчика: в `OptimalBST3` *верхняя граница* `j = n + 1` *из оригинала выходит за пределы ранее объявленного массива* `OptCost[1..n+1,0..n]`*. Для указанных границ достаточно перебирать j от 0 до n.*

Как и прежде, эти порядки можно изобразить двойной стрелкой для внешнего цикла и одинарными стрелками для внутреннего. Двунаправленные стрелки в первом порядке показывают, что порядок внутреннего цикла не важен.



- **Время и память.** Структура мемоизации занимает $O(n^2)$ памяти. Независимо от выбранного порядка вычисление каждой ячейки `OptCost[i,k]` требует $O(n)$ времени, поэтому весь алгоритм работает за $O(n^3)$ времени.

Как обычно, итоговые оценки памяти и времени можно было предсказать прямо по исходному соотношению:

$$\text{OptCost}(i,k) = \begin{cases} 0, & i > k, \\ F[i,k] + \min\limits_{i \le r \le k} \{\text{OptCost}(i, r-1) + \text{OptCost}(r+1, k)\}, & \text{иначе.} \end{cases}$$

Функция `OptCost` имеет два аргумента, каждый может принимать примерно n значений, поэтому, вероятно, нужна структура размера $O(n^2)$. С другой стороны, в теле соотношения есть три переменные — i, k и r, — каждая примерно с n возможными значениями, поэтому вычисление всего набора должно занимать $O(n^3)$ времени.

3.10. Динамическое программирование на деревьях

До сих пор во всех примерах динамического программирования мы использовали многомерные массивы для хранения результатов рекурсивных подзадач. Однако, как показывает следующий пример, это не всегда наиболее подходящая структура данных.

Независимое множество графа — подмножество вершин, между которыми нет рёбер. Найти наибольшее независимое множество произвольного графа чрезвычайно трудно: это одна из канонических NP-трудных задач, которые мы изучим в главе 12. Но для некоторых специальных классов графов наибольшее независимое множество можно найти быстро. В

частности, если входной граф — дерево с n вершинами, такое множество можно вычислить за $O(n)$ времени.

Пусть дано дерево T. Без ограничения общности предположим, что оно корневое: в T есть особая вершина, называемая корнем, а все рёбра неявно направлены от неё. Если T — некорневое дерево, то есть связный ациклический неориентированный граф, корнем можно выбрать любую вершину. Назовём вершину w *потомком* вершины v, если единственный путь от w к корню проходит через v; равносильно, потомки v — это сама v и потомки её детей. Поддерево с корнем v состоит из всех потомков v и рёбер между ними.

Для любой вершины v дерева T пусть `MIS(v)` — размер наибольшего независимого множества в поддереве с корнем v. Любое независимое множество этого поддерева, не включающее саму v, является объединением независимых множеств в поддеревьях с корнями в детях v. С другой стороны, всякое независимое множество, включающее v, обязательно исключает всех её детей и, следовательно, включает независимые множества в поддеревьях с корнями во внуках v. Поэтому функция `MIS` удовлетворяет соотношению, где нестандартная запись $w \downarrow v$ означает «w — ребёнок v»:

$$\text{MIS}(v) = \max \left\{ \sum_{w \downarrow v} \text{MIS}(w), \quad 1 + \sum_{w \downarrow v} \sum_{x \downarrow w} \text{MIS}(x) \right\}.$$

Нужно вычислить `MIS(r)`, где r — корень T.

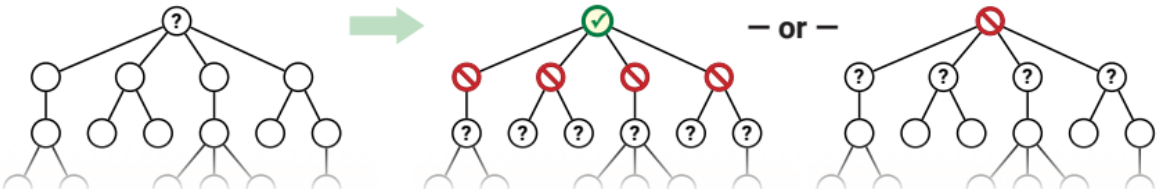


Рисунок 3.5. Вычисление наибольшего независимого множества в дереве.

Какую структуру данных выбрать для мемоизации этого соотношения? Наиболее естественный выбор — само дерево T! Для каждой вершины v будем хранить результат `MIS(v)` в новом поле `v.MIS`. В принципе, можно использовать массив, но тогда понадобятся указатели между каждым узлом и соответствующей ячейкой массива. Зачем усложнять?

Какой порядок рассмотрения подзадач подойдёт? Подзадача любой вершины v зависит от подзадач её детей и внуков. Поэтому вершины можно обходить в любом порядке, при котором каждая посещается раньше своего родителя; в частности, подойдёт стандартный обход дерева в обратном порядке, *post-order*.

Каково время работы? Время вне рекурсивных вызовов для каждой вершины v пропорционально числу её детей и внуков; это число может сильно различаться между вершинами. Но можно взглянуть на анализ иначе: каждая вершина добавляет постоянное время к работе своего родителя и деда. Поскольку у каждой вершины не более одного родителя и не более одного деда, алгоритм работает за $O(n)$ времени.

Вот получившийся алгоритм динамического программирования. Да, он всё ещё рекурсивный: это наиболее естественный способ реализовать обход дерева в обратном порядке.

```
TreeMIS(v):
    skipv ← 0
    для каждого ребёнка w вершины v
        skipv ← skipv + TreeMIS(w)
    keepv ← 1
    для каждого внука x вершины v
        keepv ← keepv + x.MIS
    v.MIS ← max{keepv, skipv}
    вернуть v.MIS
```

Ещё более простой алгоритм линейного времени можно получить, определив две разные функции на вершинах T:

- `MISyes(v)` — размер наибольшего независимого множества поддерева с корнем v, которое включает v.
- `MISno(v)` — размер наибольшего независимого множества того же поддерева, которое исключает v.

И снова требуется вычислить $\max\{\text{MISyes}(r), \text{MISno}(r)\}$, где r — корень T. Эти две функции удовлетворяют взаимным рекуррентным соотношениям:

$$\text{MISyes}(v) = 1 + \sum_{w \downarrow v} \text{MISno}(w),$$

$$\text{MISno}(v) = \sum_{w \downarrow v} \max\{\text{MISyes}(w), \text{MISno}(w)\}.$$

Эти функции опять можно мемоизировать в самом дереве, определив два новых поля для каждой вершины. Простой обход дерева в обратном порядке вычисляет обе функции во всех вершинах за $O(n)$ времени. Следующий алгоритм не только сохраняет оба значения в v, но и возвращает большее из них:

```
TreeMIS2(v):
    v.MISno ← 0
    v.MISyes ← 1
    для каждого ребёнка w вершины v
        v.MISno ← v.MISno + TreeMIS2(w)
        v.MISyes ← v.MISyes + w.MISno
    вернуть max{v.MISyes, v.MISno}
```

Во второй строке внутреннего цикла используется значение `w.MISno`, сохранённое рекурсивным вызовом на предыдущей строке.

Упражнения

Для всех следующих упражнений — и вообще при разработке любого нового алгоритма динамического программирования — я настоятельно рекомендую следовать шагам из раздела

3.4. В частности, даже не начинайте думать о таблицах или циклах `for`, пока не получите полное рекурсивное решение, включающее ясную словесную постановку тех рекурсивных подзадач, которые вы действительно решаете.[3-18] Сначала добейтесь правильной работы, затем ускоряйте.

Последовательности и массивы

1. В прошлой жизни вы работали кассиром в затерянной антарктической колонии Надирия и бо́льшую часть дня выдавали покупателям сдачу. Поскольку бумага в Антарктиде была очень редким и ценным ресурсом, закон требовал выдавать сдачу как можно меньшим числом купюр. Благодаря нумерологическим пристрастиям одного из основателей валюта Надирии — дрим-доллары — выпускалась номиналами 1, 4, 7, 13, 28, 52, 91 и 365 долларов.[3-19]

а. ♠ Жадный алгоритм выдачи сдачи каждый раз берёт крупнейшую купюру, не превышающую оставшуюся сумму. Например, чтобы выдать 122 доллара, он сначала берёт купюру в 91 доллар, затем в 28 долларов и наконец три купюры по одному доллару. Приведите пример, в котором этот алгоритм использует больше купюр, чем необходимо. Подсказка: возможно, проще написать небольшую программу, чем выяснять ответ вручную.

б. Опишите и проанализируйте рекурсивный алгоритм, который по целому числу k вычисляет минимальное количество купюр для выдачи k дрим-долларов. Не беспокойтесь о скорости: убедитесь лишь в правильности.

в. Опишите алгоритм динамического программирования, который по целому числу k вычисляет минимальное количество купюр для выдачи k дрим-долларов. Этот алгоритм должен быть быстрым.

> ***Примечание 3-18.*** *На моих курсах алгоритмов любое решение методом динамического программирования, не включающее словесного описания лежащих в его основе рекурсивных подзадач, автоматически получает ноль баллов, даже если во всём остальном оно совершенно. Введение этого правила существенно улучшило оценки студентов, поскольку они стали намного реже предлагать неверные или бессвязные алгоритмы динамического программирования.*

> ***Примечание 3-19.*** *Подробнее об истории и культуре Надирии, включая изображения разных номиналов дрим-долларов, см. http://moneyart.biz/dd/.*

2. Опишите эффективные алгоритмы для следующих вариантов задачи разбиения текста. Предположим, у вас есть подпрограмма `IsWord`, которая принимает массив символов и возвращает `True` тогда и только тогда, когда строка является «словом». Проанализируйте алгоритмы, оценив число вызовов `IsWord`.

а. Для массива символов $A[1..n]$ вычислите число разбиений A на слова. Например, для строки `ARTISTOIL` алгоритм должен вернуть 2: возможны разбиения `ARTIST·OIL` и `ART·IS·TOIL`.

б. Для двух массивов символов $A[1..n]$ и $B[1..n]$ определите, можно ли разбить обе строки на слова по одним и тем же индексам. Например:

```
BOT·HEART·HAND·SAT·URNS·PIN
PIN·START·RAPS·AND·RAGS·LAP
```

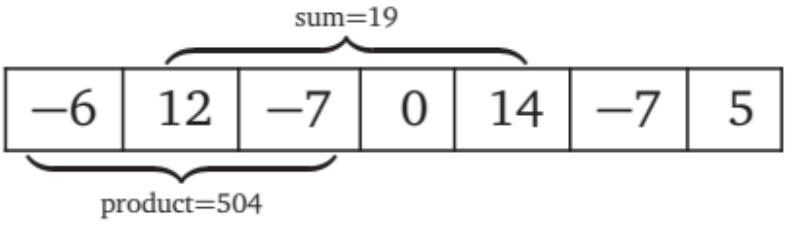
в. Для двух массивов символов $A[1..n]$ и $B[1..n]$ вычислите число различных способов разбить обе строки на слова по одним и тем же индексам.

3. Дан массив чисел $A[1..n]$, которые могут быть положительными, отрицательными или нулевыми и не обязательно целыми.

а. Опишите и проанализируйте алгоритм, который находит наибольшую сумму элементов непрерывного подмассива $A[i..j]$.

б. Опишите и проанализируйте алгоритм, который находит наибольшее произведение элементов непрерывного подмассива $A[i..j]$.

Например, для массива $[-6, 12, -7, 0, 14, -7, 5]$ первый алгоритм должен вернуть 19, а второй — 504.



Для массива из одного элемента $[-374]$ первый алгоритм должен вернуть 0, второй — 1: пустой интервал тоже считается интервалом! Для анализа предположите, что сравнение, сложение или умножение любых двух чисел занимает $O(1)$ времени.

Подсказка: пункт а — стандартный вопрос на собеседованиях по информатике по меньшей мере с середины 1980-х. В интернете есть много правильных решений; у задачи даже есть страница в Википедии! Но, по крайней мере в 2016 году, заметная часть найденных мной решений пункта б была либо медленнее необходимого, либо просто неверной.

4. Это упражнение рассматривает варианты задачи о подмассиве с максимальной суммой из упражнения 3. Во всех случаях вход — массив вещественных чисел $A[1..n]$, которые могут быть положительными, отрицательными или нулевыми, и, возможно, дополнительное целое число $X \geq 0$.

а. **С переходом через край.** Пусть A — кольцевой массив. Тогда непрерывным подмассивом считается как интервал $A[i..j]$, так и суффикс, за которым следует префикс: $A[i..n] \cdot A[1..j]$. Опишите и проанализируйте алгоритм, находящий непрерывный подмассив A с наибольшей суммой.

б. **Только длинные подмассивы.** Найдите непрерывный подмассив длиной не менее X с наибольшей суммой. Предположите, что $X \leq n$.

в. **Только короткие подмассивы.** Найдите непрерывный подмассив длиной не более X с наибольшей суммой.

г. **The Price Is Right.** Найдите непрерывный подмассив с наибольшей суммой, не превышающей X.

д. Опишите более быстрый алгоритм для пункта г, если все числа массива A неотрицательны.

5. В этом упражнении требуется разработать эффективные алгоритмы поиска оптимальных подпоследовательностей разных видов. Подпоследовательность получается выбором некоторого подмножества элементов последовательности с сохранением их порядка; элементы не обязаны располагаться подряд. Например, строки `C`, `DAMN`, `YAIOAI` и `DYNAMICPROGRAMMING` — подпоследовательности строки `DYNAMICPROGRAMMING`.

Подсказка: ровно одну из этих задач можно решить жадным алгоритмом за $O(n)$ времени.

а. Пусть $A[1..m]$ и $B[1..n]$ — произвольные массивы. Их общая подпоследовательность — последовательность, являющаяся подпоследовательностью и A, и B. Опишите эффективный алгоритм вычисления длины наибольшей общей подпоследовательности.

б. Пусть $A[1..m]$ и $B[1..n]$ — произвольные массивы. Их общая надпоследовательность — последовательность, которая содержит и A, и B как подпоследовательности. Опишите эффективный алгоритм вычисления длины кратчайшей общей надпоследовательности.

в. Назовём последовательность чисел $X[1..n]$ битонной, если существует индекс i с $1 < i < n$, при котором префикс $X[1..i]$ возрастает, а суффикс $X[i..n]$ убывает. Опишите эффективный алгоритм вычисления длины наибольшей битонной подпоследовательности произвольного массива целых чисел A.

г. Назовём последовательность чисел $X[1..n]$ колеблющейся, если $X[i] < X[i+1]$ для всех чётных i и $X[i] > X[i+1]$ для всех нечётных i. Опишите эффективный алгоритм вычисления длины наибольшей колеблющейся подпоследовательности произвольного массива целых чисел A.

д. Опишите эффективный алгоритм вычисления длины кратчайшей колеблющейся надпоследовательности произвольного массива целых чисел A.

е. Назовём последовательность чисел $X[1..n]$ выпуклой, если $2X[i] < X[i-1] + X[i+1]$ для всех i. Опишите эффективный алгоритм вычисления длины наибольшей выпуклой подпоследовательности произвольного массива целых чисел A.

ж. Назовём последовательность чисел $X[1..n]$ слабо возрастающей, если каждый элемент больше среднего двух предыдущих, то есть $2X[i] > X[i-1] + X[i-2]$ для всех $i > 2$. Опишите эффективный алгоритм вычисления длины наибольшей слабо возрастающей подпоследовательности произвольного массива целых чисел A.

з. Назовём последовательность чисел $X[1..n]$ дважды возрастающей, если $X[i] > X[i-2]$ для всех $i > 2$. Иными словами, такая последовательность является идеальным перемешиванием двух возрастающих последовательностей. Опишите эффективный алгоритм

вычисления длины наибольшей дважды возрастающей подпоследовательности произвольного массива целых чисел A.

и. Напомним, что последовательность чисел $X[1..n]$ возрастает, если $X[i] < X[i+1]$ для всех i. Опишите эффективный алгоритм вычисления длины наибольшей общей возрастающей подпоследовательности двух заданных массивов целых чисел. Например, $\langle 1, 4, 5, 6, 7, 9\rangle$ — наибольшая общая возрастающая подпоследовательность последовательностей $\langle 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3\rangle$ и $\langle 1, 4, 1, 4, 2, 1, 3, 5, 6, 2, 3, 7, 3, 0, 9, 5\rangle$.

6. *Перемешивание* двух строк X и Y получается переплетением их символов в новую строку с сохранением относительного порядка символов каждой строки. Например, строка `BANANAANANAS` является перемешиванием строк `BANANA` и `ANANAS` несколькими разными способами. Аналогично строки `PRODGYRNAMAMMIINCG` и `DYPRONGARMAMMICING` обе являются перемешиваниями строк `DYNAMIC` и `PROGRAMMING`.



а. По трём строкам $A[1..m]$, $B[1..n]$ и $C[1..m+n]$ определите, является ли C перемешиванием A и B. Опишите и проанализируйте алгоритм.

б. Назовём перемешивание X и Y *плавным*, если в нём никогда не используются более двух подряд идущих символов одной из строк. Например:

- `PRDOYGNARAMMMIICNG` — плавное перемешивание `DYNAMIC` и `PROGRAMMING`.
- `DYPRNOGRAAMMMICING` — перемешивание `DYNAMIC` и `PROGRAMMING`, но не плавное из-за подстрок `OGR` и `ING`.
- `XXXXXXXXXXXXXXXXXXX` — плавное перемешивание `XXXXXXX` и `XXXXXXXXXXX`.
- Плавного перемешивания строк `XXXX` и `XXXXXXXXXXXX` не существует.

Опишите и проанализируйте алгоритм, который по трём строкам X, Y и Z определяет, является ли Z плавным перемешиванием X и Y.

> *Примечание переводчика: в третьем примере оригинала количество символов `X` в результирующей строке не совпадает с суммой длин двух входных строк; смысл примера относится к строке соответствующей суммарной длины.*

7. В каждой следующей задаче вход состоит из двух массивов $X[1..k]$ и $Y[1..n]$, где $k \le n$.

а. Опишите и проанализируйте алгоритм, определяющий, является ли X подпоследовательностью Y. Например, `PPAP` — подпоследовательность `PENPINEAPPLEAPPLEPEN`.

б. Опишите и проанализируйте алгоритм, находящий наименьшее количество символов, которые нужно удалить из Y, чтобы X перестала быть её подпоследовательностью. Иначе говоря, алгоритм должен найти наибольшую подпоследовательность Y, не являющуюся

надпоследовательностью X. Например, после удаления двух символов из PENPINEAPPLEAPPLEPEN строка PPAP перестанет быть подпоследовательностью.

в. ♥ Опишите и проанализируйте алгоритм, определяющий, встречается ли X в Y как две непересекающиеся подпоследовательности. Например, PPAP дважды встречается непересекающимися подпоследовательностями в PENPINEAPPLEAPPLEPEN.

г. Допустим, вход также содержит третий массив чисел $C[1..n]$, которые могут быть положительными, отрицательными или нулевыми; $C[i]$ — стоимость $Y[i]$. Опишите и проанализируйте алгоритм вычисления вхождения X в виде подпоследовательности Y с минимальной стоимостью. То есть нужно найти массив $I[1..k]$ такой, что $I[j] < I[j+1]$ и $X[I[j]] = Y[j]$ для каждого индекса j, а суммарная стоимость $\sum_{j=1}^{k} C[j]$ минимальна.

> *Примечание переводчика: формулы индексов и стоимости в этом пункте воспроизведены по оригиналу, но не согласуются с описанием задачи. Для стоимости выбранных позиций естественно использовать $X[j] = Y[I[j]]$ и $\sum_{j=1}^{k} C[I[j]]$.*

д. Опишите и проанализируйте алгоритм вычисления общего числа, возможно перекрывающихся, вхождений X как подпоследовательности Y. При анализе считайте, что сложение двух произвольных целых чисел занимает $O(1)$ времени. Например, PPAP встречается как подпоследовательность PENPINEAPPLEAPPLEPEN ровно 23 раза. Если все символы X и Y равны, алгоритм должен вернуть $\binom{n}{k}$.

е. Каково время работы вашего алгоритма для пункта г, если сложение двух ℓ-битных целых чисел требует $O(\ell)$ времени?

8. Опишите и проанализируйте эффективный алгоритм нахождения длины наибольшей непрерывной подстроки, которая встречается во входной строке $T[1..n]$ как в прямом, так и в обратном направлении. Эти два вхождения не должны перекрываться. Примеры:

- Для ALGORITHM ответ — 0.
- Для RECURSION ответ — 1, подстрока R.
- Для REDIVIDE ответ — 3, подстрока EDI: прямое и обратное вхождения не должны перекрываться!
- Для DYNAMICPROGRAMMINGMANYTIMES ответ — 4, подстрока YNAM. В частности, ответом не должно быть 6 для подпоследовательности YNAMIR.

9. Палиндром — строка, которая совпадает со своей записью в обратном порядке, например I, DEED, RACECAR или AMANAPLANACATACANALPANAMA.

а. Опишите и проанализируйте алгоритм нахождения длины наибольшей подпоследовательности заданной строки, являющейся палиндромом. Например, для MAHDYNAMICPROGRAMZLETMESHOWYOUTHEM такой подпоследовательностью является MHYMRORMYHM, поэтому ответ — 11.

б. Опишите и проанализируйте алгоритм нахождения длины кратчайшей надпоследовательности заданной строки, являющейся палиндромом. Например, кратчайшая палиндромная надпоследовательность TWENTYONE — TWENTOYOTNEWT, поэтому ответ — 13.

в. Любую строку можно разбить на последовательность палиндромов. Например, BUBBASEESABANANA можно разбить следующими способами и ещё 65 другими:

```
BUB · BASEESAB · ANANA
B · U · BB · ASEESA · B · ANANA
BUB · B · A · SEES · ABA · N · ANA
B · U · BB · A · S · EE · S · A · B · A · NAN · A
B · U · B · B · A · S · E · E · S · A · B · A · N · A · N · A
```

Опишите и проанализируйте эффективный алгоритм нахождения наименьшего количества палиндромов, из которых состоит входная строка. Например, для BUBBASEESABANANA ответ равен 3.

г. Опишите и проанализируйте эффективный алгоритм нахождения наибольшего целого k, для которого заданную строку можно разбить на палиндромы длиной не менее k. Например:

- Для строки PALINDROME ответ — 1.
- Для строки BUBBASEESABANANA ответ — 3; разбиение: BUB · BASEESAB · ANANA.
- Для строки из n одинаковых символов ответ — n.

д. Опишите и проанализируйте эффективный алгоритм нахождения числа различных способов разложить заданную строку на палиндромы. Например, для PALINDROME ответ — 1, для BUBBASEESABANANA — 70, а для строки из n одинаковых символов — 2^{n-1}.

е. ♥ *Метапалиндром* — разложение строки на последовательность палиндромов, при котором последовательность длин этих палиндромов сама является палиндромом. Например,

```
BOB · S · MAM · ASEESA · UKU · L · ELE
```

— метапалиндром строки BOBSMAMASEESAUKULELE, а последовательность длин $(3, 1, 3, 6, 3, 1, 3)$ — палиндром. Опишите и проанализируйте эффективный алгоритм нахождения длины кратчайшего метапалиндрома заданной строки. Например, для BOBSMAMASEESAUKULELE алгоритм должен вернуть 11.

10. Дан массив положительных целых чисел $A[1..n]$. *Возрастающая подпоследовательность с движением туда-сюда* — последовательность индексов $I[1..\ell]$ со свойствами:

- $1 \le I[j] \le n$ для всех j;
- $A[I[j]] < A[I[j+1]]$ для всех $j < \ell$;
- если $I[j]$ чётно, то $I[j+1] > I[j]$;
- если $I[j]$ нечётно, то $I[j+1] < I[j]$.

Менее формально: имеется ряд из n клеток с положительными целыми числами. Поставьте фишку на одну из клеток, затем многократно двигайте её влево с клетки с нечётным индексом

или вправо с клетки с чётным индексом, всегда переходя от меньшего числа к большему. Последовательность позиций фишки является возрастающей подпоследовательностью с движением туда-сюда.

Опишите алгоритм вычисления длины наибольшей такой подпоследовательности для заданного массива из n целых чисел. Например, для массива

```
1 1 8 7 5 6 3 6 4 4 8 3 9 1 2 2 3 9 4 0
```



алгоритм должен вернуть 9 — длину подпоследовательности значений `0, 1, 2, 3, 4, 6, 7, 8, 9` с индексами `20, 1, 15, 18, 10, 6, 4, 3, 13`.



> *Примечание переводчика: условие оригинала говорит о положительных целых числах, но в авторском примере присутствует ноль.*

11. Допустим, мы хотим набрать абзац текста на бумаге или, если настаиваете, на экране компьютера. Текст состоит из n слов, длина i-го слова — $\ell[i]$. Нужно разбить абзац на строки полной длины ровно L. Например, согласно TeX — программе, использованной для набора этих заметок, — абзац, который вы сейчас читаете в оригинале, имеет ширину примерно 11,94794 см, или 4,7055 дюйма.

В зависимости от разбиения приходится вставлять между словами разное количество пробелов. Абзац должен быть выровнен по ширине: первый символ каждой строки начинается у левого поля, а последний символ каждой строки, кроме последней, заканчивается у правого. Между любыми соседними словами одной строки должен быть хотя бы один пробел.

Определим *небрежность* разметки абзаца как сумму по всем строкам, кроме последней, кубов количества дополнительных пробелов в каждой строке, не считая одного обязательного пробела между каждой парой соседних слов. Если строка содержит слова с i-го по j-е, её небрежность равна $(L - j + i - \sum_{k=i}^{j} \ell[k])^3$. Опишите алгоритм динамического программирования, который печатает абзац с минимальной небрежностью.

12. Вы и ваш восьмилетний племянник Элмо решили сыграть в простую карточную игру. В начале карты раскладывают лицом вверх в длинный ряд; каждая карта стоит разное число очков. Затем вы и Элмо по очереди забираете крайнюю левую или крайнюю правую карту, пока карты не закончатся. На каждом ходе можно выбрать одну из двух карт. Побеждает тот, кто к концу игры набрал больше очков.

Поскольку Элмо никогда не изучал алгоритмы, он следует очевидной жадной стратегии: на своём ходе всегда берёт карту с большим числом очков. Вам нужно найти стратегию, которая обыграет Элмо, когда это возможно. Возможно, обыгрывать маленького ребёнка кажется жестоким, но Элмо терпеть не может, когда взрослые ему поддаются.

а. Докажите, что вам не следует тоже использовать жадную стратегию: покажите игру, которую вы можете выиграть, но только отказавшись от той же стратегии, что у Элмо.

б. Опишите и проанализируйте алгоритм, определяющий по исходному ряду карт максимальное число очков, которое вы можете набрать против Элмо.

в. ♣ В четыре года Элмо использовал ещё более простую стратегию: на каждом ходе выбирал следующую карту равновероятно. Если оставалось более одной карты, левую и правую он брал с вероятностью $1/2$ каждую. Опишите алгоритм определения максимального ожидаемого числа очков, которое вы можете набрать против четырёхлетнего Элмо по исходному ряду карт.

г. Через пять лет тринадцатилетний Элмо стал гораздо более сильным игроком. Опишите и проанализируйте алгоритм определения максимального числа очков, которое вы можете набрать по исходному ряду карт против совершенного соперника.

13. Пора продемонстрировать свои потрясающие танцевальные навыки! Завтра — большой конкурс, к которому вы готовились всю жизнь, не считая того лета, когда охотились на росомах на Аляске вместе с дядей. Вам заранее достался список из n песен, которые судьи включат во время конкурса, в хронологическом порядке. Да-а-а!

Вы отлично знаете все песни, всех судей и свои способности. Для каждого k известно: если танцевать под k-ю песню, вы получите ровно `Score[k]` очков, но затем физически не сможете танцевать следующие `Wait[k]` песен — с $k+1$ по $k + \mathrm{Wait}[k]$. Победит танцор с наибольшей суммой очков к концу вечера, поэтому вы хотите набрать как можно больше.

Опишите и проанализируйте эффективный алгоритм вычисления максимально достижимой суммы очков. Вход — массивы `Score[1..n]` и `Wait[1..n]`.

14. В новой игре-головоломке с обменом сладостей *Candy Swap Saga XIII* участвуют n милых животных с номерами от 1 до n. Каждое животное держит одну из сладостей трёх видов: арахисовое лакомство circus peanuts, батончик Heath или шоколадный трюфель Cioccolateria Gardini. У вас в руке тоже есть сладость; в начале игры это circus peanut.

Чтобы набирать очки, вы посещаете животных по порядку от 1 до n. Для каждого можно оставить сладость в руке или обменять её на сладость животного:

- за обмен на сладость того же вида начисляется одно очко;
- за обмен на сладость другого вида одно очко вычитается; счёт может быть отрицательным;
- если не обмениваться, счёт не меняется.

Животных нужно посещать по порядку, и вернуться к уже посещённому нельзя. Опишите и проанализируйте эффективный алгоритм вычисления максимально возможного счёта. Вход — массив `C[1..n]`, где `C[i]` — вид сладости у i-го животного.

15. Ленни Рутенбар, первый декан нового колледжа компьютерных наук имени Максимилиана Р. Левчина, заказал серию снежных трамплинов на южном склоне саночной горки Орчард-Даунс[3-20] и вызвал Билла Кудеки, заведующего кафедрой электротехники и вычислительной техники, на соревнование. Оба съедут с горки, стараясь максимизировать время в воздухе. Победитель получает право расширить своё подразделение и в центр Зибеля, и в новое здание ECE; проигравшему придётся переселить всё подразделение в водопропускной канал Боньярд рядом с лабораторией Лумиса.

Всякий раз, когда Ленни или Билл на земле достигает трамплина, он может либо прыгнуть с него и, возможно, перелететь один или несколько следующих трамплинов, либо проехать мимо, оставаясь на земле. Разумеется, перелетая над трамплином, нельзя воспользоваться им для продления прыжка.

а. Даны массивы `Ramp[1..n]` и `Length[1..n]`, где `Ramp[i]` — расстояние от вершины горки до i-го трамплина, а `Length[i]` — расстояние, которое пролетит любой саночник, прыгнувший с него. Опишите и проанализируйте алгоритм определения максимального суммарного расстояния, которое Ленни или Билл может пролететь.

б. Университетские юристы услышали об их пари и немедленно возразили. Чтобы защитить университет от исков или резкого роста страховых взносов, они ограничили число прыжков каждого саночника. По исходным массивам `Ramp[1..n]`, `Length[1..n]` и целому числу k определите максимальное суммарное расстояние в воздухе при не более чем k прыжках. Опишите и проанализируйте алгоритм.

в. ♥ Когда юристы поняли, что ограничение не остановило соревнование, они добавили новое: каждый трамплин можно использовать не более одного раза! Возмущённые вмешательством, Ленни и Билл отказываются от пари и решают вместе устроить зрителям хорошее шоу. Опишите и проанализируйте алгоритм определения максимального общего расстояния, которое оба проведут в воздухе, если каждый делает не более k прыжков — всего не более $2k$ — и каждый трамплин используется не более одного раза.

> ***Примечание 3-20.*** *Северный склон быстрее, но слишком короток для интересного соревнования.*

16. Фермеры Боггис, Банс и Бин устроили для мистера Лиса полосу препятствий. Она состоит из длинного ряда будок, на каждой спереди яркой красной краской написано число. Формально мистеру Лису дан массив $A[1..n]$, где $A[i]$ — число на i-й будке. Каждое число может быть положительным, отрицательным или нулевым. Все согласны со следующими правилами:

- У каждой будки мистер Лис обязан сказать либо «Ринг!», либо «Динг!».
- Если у i-й будки он говорит «Ринг!», то получает награду в $A[i]$ кур. Если $A[i] < 0$, он платит штраф в $-A[i]$ кур.
- Если у i-й будки он говорит «Динг!», то платит штраф в $A[i]$ кур. Если $A[i] < 0$, он получает награду в $-A[i]$ кур.
- Нельзя произносить одно и то же слово более трёх раз подряд. Например, если у будок 6, 7 и 8 он сказал «Ринг!», то у будки 9 обязан сказать «Динг!».

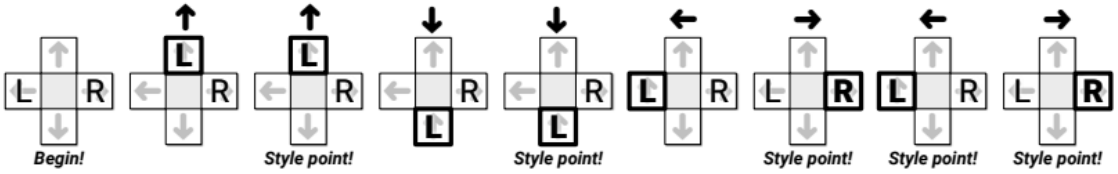
- Все расчёты производятся в конце, после посещения всех будок, когда судья объявит «Горячий ящик!». Мистеру Лису не нужно действительно нести кур через полосу препятствий.
- Наконец, если мистер Лис нарушит любое правило или останется должен фермерам кур после прохождения полосы, фермеры его застрелят.

Опишите и проанализируйте алгоритм, который по массиву чисел $A[1..n]$ вычисляет наибольшее число кур, которое мистер Лис может заработать, пройдя полосу. Подсказка: берегитесь горящей сосновой шишки!

17. *Dance Dance Revolution* — танцевальная видеоигра, впервые представленная компанией Konami в Японии в 1998 году. Игрок стоит на платформе с четырьмя стрелками, направленными вперёд, назад, влево и вправо и расположенными крестом. Во время игры звучит песня, а по экрану снизу вверх движется последовательность из n стрелок ↑, ↓, ← или →. В тот момент, когда стрелка достигает верхней части экрана, игрок должен наступить на соответствующую стрелку платформы. Стрелки рассчитаны так, чтобы шаги попадали в такт музыке.

Вы играете в разновидность этой игры под названием *Vogue Vogue Revolution*, где цель — пройти игру без ошибок, двигаясь как можно меньше. Когда стрелка достигает верха экрана, если одна нога уже стоит на соответствующей стрелке платформы, вы получаете одно очко стиля за сохранение позы. Если ни одна нога там не стоит, нужно переместить одну и только одну ногу с её текущего места на нужную стрелку. Если вы наступите не на ту стрелку, не наступите на нужную, сдвинете более одной ноги за раз или сдвинете любую ногу, когда уже стоите на нужной стрелке, все очки стиля отбираются и вы проигрываете.

Как двигать ногами, чтобы набрать максимум очков стиля? Предположим, что вы всегда начинаете с левой ногой на ←, правой на → и помните всю последовательность стрелок. Например, для последовательности ↑ ↑ ↓ ↓ ← → ← → можно получить 5 очков, двигая ногами, как показано на схеме.



а. Докажите, что для любой последовательности из n стрелок можно набрать не менее $n/4 - 1$ очков стиля.

б. Опишите эффективный алгоритм нахождения максимального числа очков стиля для заданного танца VVR. Вход — массив `Arrow[1..n]` с последовательностью стрелок.

18. Рассмотрим следующий пасьянс по мотивам Scrabble. Дана фиксированная конечная последовательность фишек; каждая содержит букву и числовую стоимость. В начале игры берём первые семь фишек и кладём их в руку. На каждом ходе составляем английское слово из некоторых или всех фишек в руке, выкладываем их на стол и получаем сумму их стоимостей в очках. Если составить английское слово невозможно, игра сразу заканчивается. Затем

последовательно добираем фишки из начала оставшейся последовательности, пока либо в руке не окажется семь фишек, либо последовательность не закончится. Увы, здесь нет удвоения или утроения стоимости слов и букв, бонусов, пустых фишек и пропуска хода. Цель — набрать как можно больше очков.

Например, рассмотрим последовательность из 20 фишек, где нижний индекс — стоимость:

```
I₂ N₂ X₈ A₁ N₂ A₁ D₃ U₅ D₃ I₂ D₃ K₈ U₅ B₄ L₂ A₁ K₈ H₅ A₁ N₂
```

Из этой последовательности можно набрать 68 очков так:

- Сначала берём `I₂ N₂ X₈ A₁ N₂ A₁ D₃`.
- Выкладываем `N₂ A₁ I₂ A₁ D₃` за 9 очков; остаются `N₂ X₈`.
- Добираем пять фишек `U₅ D₃ I₂ D₃ K₈`.
- Выкладываем `U₅ N₂ D₃ I₂ D₃` за 15 очков; остаются `K₈ X₈`.
- Добираем пять фишек `U₅ B₄ L₂ A₁ K₈`.
- Выкладываем `B₄ U₅ L₂ K₈` за 19 очков; остаются `K₈ X₈ A₁`.
- Добираем последние три фишки `H₅ A₁ N₂`.
- Выкладываем `A₁ N₂ K₈ H₅` за 16 очков; остаются `X₈ A₁`.
- Выкладываем `A₁ X₈` за 9 очков, освобождая руку и заканчивая игру.

а. Пусть последовательность задана массивами `Letter[1..n]` с буквами от A до Z и `Value[A..Z]`, где `Value[ ℓ ]` — стоимость любой фишки с буквой ℓ. Разработайте и проанализируйте эффективный алгоритм вычисления максимума очков, которые можно получить из данной последовательности.

б. Теперь пусть две фишки с одинаковой буквой могут иметь разную стоимость. Последовательность задана массивами `Letter[1..n]` и `Value[1..n]`, где `Value[i]` — стоимость i-й фишки. Разработайте и проанализируйте эффективный алгоритм вычисления максимума очков.

В обоих случаях результат — одно число: максимально возможный счёт. Предположите, поскольку это правда, что все английские слова, составимые из любого набора не более семи фишек, вместе с их стоимостью можно найти за $O(1)$ времени.

19. Дано множество L из n отрезков на плоскости; у каждого один конец лежит на прямой $y = 0$, а другой — на прямой $y = 1$. Все $2n$ концов различны.

а. Опишите и проанализируйте алгоритм вычисления наибольшего подмножества L, в котором никакие два отрезка не пересекаются.

б. Опишите и проанализируйте алгоритм вычисления наибольшего подмножества L, в котором каждая пара отрезков пересекается.

Теперь пусть дано множество L из n отрезков на плоскости, оба конца каждого лежат на единичной окружности $x^2 + y^2 = 1$, и все $2n$ концов различны.

в. Опишите и проанализируйте алгоритм вычисления наибольшего подмножества L, в котором никакие два отрезка не пересекаются.

г. Опишите и проанализируйте алгоритм вычисления наибольшего подмножества L, в котором каждая пара отрезков пересекается.

20. Пусть P — множество из n равномерно распределённых точек единичной окружности, а S — множество из m отрезков с концами в P. Концы отрезков не обязательно различны; n может быть значительно меньше $2m$.

а. Опишите алгоритм нахождения размера наибольшего подмножества отрезков S, в котором каждая пара не пересекается. Два отрезка не пересекаются, если у них нет общих точек, в том числе концов.

б. Опишите алгоритм нахождения размера наибольшего подмножества отрезков S, в котором внутренности каждой пары не пересекаются. Внутренности двух отрезков не пересекаются, если их пересечение пусто либо является общим концом обоих отрезков.

в. Опишите алгоритм нахождения размера наибольшего подмножества S, в котором каждая пара отрезков пересекается.

г. Опишите алгоритм нахождения размера наибольшего подмножества S, в котором каждая пара отрезков пересекается во внутренних точках, а не в концах.

Для полного балла каждый из четырёх алгоритмов должен работать за $O(mn)$ времени.

21. Вы ведёте по шоссе автобус, полный шумных, гиперактивных, страдающих от жажды студентов и оборудованный автоматом с газировкой. Каждую минуту, которую студент проводит в автобусе, он выпивает одну унцию газировки. Ваша цель — быстро высадить студентов, чтобы общее количество выпитой газировки было как можно меньше.

Вы знаете, сколько студентов выйдет на каждом съезде. Автобус начинает движение где-то на шоссе, вероятно, не на одном из концов, и движется с постоянной скоростью 37,4 мили в час. Он должен ехать по шоссе, но может доехать в одном направлении до съезда, затем в обратном направлении до другого, меняя направление сколько угодно раз. Остановка, высадка и разворот происходят мгновенно.

Опишите эффективный алгоритм высадки студентов с минимальным потреблением газировки. Вход состоит из маршрута — списка съездов и времени проезда между соседними съездами, — количества выходящих на каждом съезде студентов и текущего положения автобуса, которое можно считать одним из съездов.

22. Определим *сводку* двух строк A и B как конкатенацию подстрок следующего вида:

- ▲`SNA` обозначает подстроку `SNA` только первой строки A;
- ♦`FOO` обозначает общую подстроку `FOO` обеих строк;
- ▼`BAR` обозначает подстроку `BAR` только второй строки B.

Сводка допустима, если обе исходные строки можно восстановить, последовательно склеив соответствующие подстроки и отбросив разделители ▲, ♦ и ▼. Каждый обычный символ имеет длину 1, каждый разделитель — некоторую фиксированную неотрицательную длину Δ. Длина сводки — сумма длин её символов.

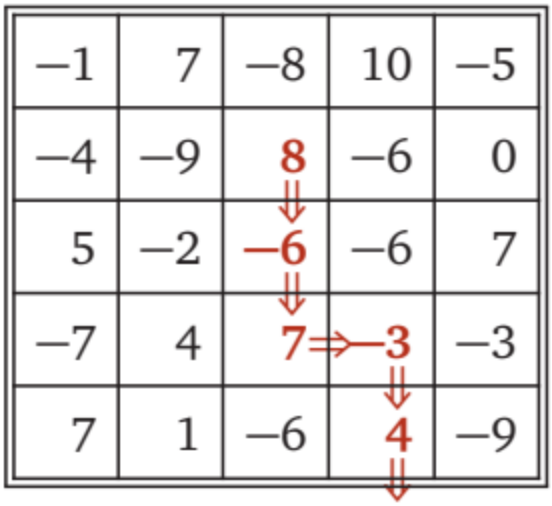
Например, каждая из следующих строк — допустимая сводка строк `KITTEN` и `KNITTING`:

- `♦K▼N♦ITT▲E▼I♦N▼G` имеет длину $9 + 7\Delta$;
- `♦K▼N♦ITT▲EN▼ING` имеет длину $10 + 5\Delta$;
- `♦K▲ITTEN▼NITTING` имеет длину $13 + 3\Delta$;
- `▲KITTEN▼KNITTING` имеет длину $14 + 2\Delta$.

Опишите и проанализируйте алгоритм вычисления длины кратчайшей сводки двух заданных строк $A[1..m]$ и $B[1..n]$. Длина разделителя Δ также входит во входные данные. Например, для `KITTEN` и `KNITTING` при $\Delta = 0$ ответ — 9, при $\Delta = 1$ — 15, при $\Delta = 2$ — 18.

23. *Миля Ванкина* — американский пасьянс на квадратной сетке $n \times n$. Игрок начинает, помещая фишку на любую клетку. Затем на каждом ходе перемещает её на одну клетку вправо или на одну клетку вниз. Игра заканчивается, когда фишка покидает доску. Каждая клетка имеет числовое значение, положительное, отрицательное или нулевое. Начальный счёт равен нулю; всякий раз, когда фишка попадает на клетку, игрок добавляет её значение к счёту. Цель — набрать как можно больше очков.

Например, на приведённой ниже сетке можно получить $8 - 6 + 7 - 3 + 4 = 10$ очков, поставив фишку на 8 во второй строке и затем сделав ходы вниз, вниз, вправо, вниз, вниз. Это не лучший счёт для этой сетки.




```
-1   7  -8  10  -5
-4  -9   8  -6   0
 5  -2  -6  -6   7
-7   4   7  -3  -3
 7   1  -6   4  -9
```

а. Опишите и проанализируйте эффективный алгоритм вычисления максимально возможного счёта в «Миле Ванкина» по заданному массиву значений $n \times n$.

б. В европейской версии игры, соответственно названной «Километр Ванкина», на каждом ходе можно переместить фишку на одну клетку вниз, вправо или влево. Однако, чтобы избежать бесконечного счёта, нельзя попадать на одну и ту же клетку более одного раза. Опишите и проанализируйте эффективный алгоритм вычисления максимально возможного счёта по заданному массиву $n \times n$.[3-21]

> ***Примечание 3-21.*** *Если бы мы разрешили также движение вверх, получившаяся игра — «Морская сажень Ванкина»? — была бы NP-трудной.*

24. Дан битовый рисунок $m \times n$ в виде массива $M[1..n, 1..n]$ из нулей и единиц. *Сплошной блок* в M — подмассив вида $M[i..i', j..j']$, в котором все биты равны. Сплошной блок является квадратным, если число его строк и столбцов одинаково.

а. Опишите алгоритм нахождения максимальной площади сплошного квадратного блока в M за $O(n^2)$ времени.

б. Опишите алгоритм нахождения максимальной площади сплошного блока в M за $O(n^3)$ времени.

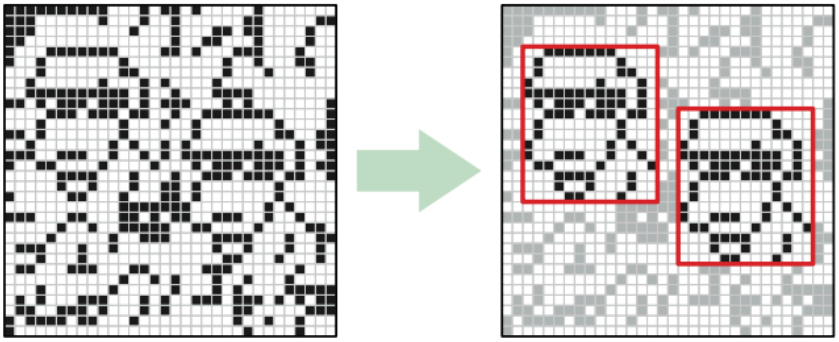
в. Опишите алгоритм нахождения максимальной площади сплошного блока в M за $O(n^2 \log n)$ времени. Подсказка: разделяй и властвуй.

г. ♥ Опишите алгоритм нахождения максимальной площади сплошного блока в M за $O(n^2)$ времени.

> *Примечание переводчика: исходное условие называет рисунок прямоугольным $m \times n$, но задаёт массив $n \times n$ и оценки только через n.*

25. Дан массив чисел $M[1..n, 1..n]$, которые могут быть положительными, отрицательными или нулевыми и не обязательно целыми. Опишите алгоритм нахождения наибольшей суммы элементов прямоугольного подмассива вида $M[i..i', j..j']$. Для полного балла алгоритм должен работать за $O(n^3)$ времени. Подсказка: см. задачу 3.

26. Опишите и проанализируйте алгоритм поиска прямоугольного узора наибольшей площади, который более одного раза встречается в заданном битовом изображении. Точнее, по двумерному массиву битов $M[1..n, 1..n]$ алгоритм должен вернуть площадь наибольшего повторяющегося прямоугольного узора. Например, для изображения на рисунке слева алгоритм должен вернуть 195 — площадь собачки размером 15×13. Хотя в этом примере такого нет, два экземпляра повторяющегося узора могут перекрываться.



а. Для полного балла опишите алгоритм времени $O(n^5)$.

б. ♥ Для дополнительных баллов опишите алгоритм времени $O(n^4)$.

в. ♣♥ Для ещё более дополнительных баллов опишите алгоритм времени $O(n^3 \operatorname{polylog} n)$.

27. Пусть P — множество точек плоскости в выпуклом положении. Интуитивно, если обернуть точки резинкой, каждая будет касаться резинки. Более формально, для любой точки $p \in P$ существует прямая, отделяющая p от остальных точек P. Кроме того, точки пронумерованы $P[1], P[2], \ldots, P[n]$ против часовой стрелки вдоль «резинки», начиная с самой левой точки $P[1]$.

Требуется решить частный случай задачи коммивояжёра: он должен посетить каждую точку P, а стоимость перемещения между точками $p, q \in P$ равна евклидову расстоянию $|pq|$.

а. Опишите простой алгоритм вычисления кратчайшего замкнутого маршрута по P.

б. Простой маршрут не пересекает сам себя. Докажите, что кратчайший маршрут по P обязательно прост.

в. Опишите и проанализируйте эффективный алгоритм вычисления кратчайшего маршрута по P, начинающегося в самой левой точке $P[1]$ и заканчивающегося в самой правой точке $P[r]$.

г. Опишите и проанализируйте эффективный алгоритм вычисления кратчайшего маршрута по P без ограничений на концы маршрута.

28. ♥ Опишите и проанализируйте алгоритм решения задачи коммивояжёра за время $O(2^n \operatorname{poly}(n))$. По заданному неориентированному графу G из n вершин со взвешенными рёбрами алгоритм должен вернуть вес наиболее лёгкого цикла в G, который посещает каждую вершину ровно один раз, либо ∞, если таких циклов нет. Подсказка: очевидный рекурсивный алгоритм с возвратом требует $O(n!)$ времени.

29. Пусть $W = \{w_1, w_2, \ldots, w_n\}$ — конечное множество строк над некоторым фиксированным алфавитом Σ. *Центр редактирования* для W — строка $C \in \Sigma^*$, для которой максимальное расстояние редактирования от C до строк из W минимально. *Радиус редактирования* W — максимальное расстояние от центра редактирования до строки из W. У множества может быть несколько центров редактирования, но его радиус единственен.

$$\text{EditRadius}(W) := \min_{C \in \Sigma^*} \max_{w \in W} \text{Edit}(w, C),$$

$$\text{EditCenter}(W) := \operatorname*{argmin}_{C \in \Sigma^*} \max_{w \in W} \text{Edit}(w, C).$$

а. Опишите и проанализируйте эффективный алгоритм вычисления радиуса редактирования трёх заданных строк.

б. ♣ ♥ Опишите и проанализируйте эффективный алгоритм приближённого вычисления радиуса редактирования произвольного множества строк с коэффициентом не хуже 2. Точное вычисление NP-трудно, если число строк не фиксировано.

30. ♥ Пусть $D[1..n]$ — массив цифр, каждая из которых является целым числом от 0 до 9. *Цифровая подпоследовательность* D — последовательность положительных целых чисел, составленных обычным способом из непересекающихся подстрок D. Например, последовательность $3, 4, 5, 6, 8, 9, 32, 38, 46, 64, 83, 279$ является цифровой подпоследовательностью первых нескольких цифр числа π:

```
3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3,
2, 3, 8, 4, 6, 2, 6, 4, 3, 3, 8, 3, 2, 7, 9
```

Длина цифровой подпоследовательности — количество входящих в неё целых чисел, а не цифр; предыдущий пример имеет длину 12. Как обычно, цифровая подпоследовательность возрастает, если каждое число больше предыдущего.

Опишите и проанализируйте эффективный алгоритм вычисления наибольшей возрастающей цифровой подпоследовательности D. Подсказка: внимательно отнеситесь к вычислительным предположениям. Сколько времени занимает сравнение двух k-значных чисел?

Для полного балла алгоритм должен работать за $O(n^4)$ времени; более быстрые алгоритмы дают дополнительные баллы. Самый быстрый из известных мне алгоритмов этой задачи работает за $O(n^{3/2} \log n)$; достижение этой оценки требует нескольких приёмов как при построении алгоритма, так и при его анализе, но ничего выходящего за рамки этого курса.[3-22]

> ***Примечание 3-22.*** *Я полагаю, что с помощью более продвинутых методов время можно уменьшить до $O(n^{3/2} \log \log n)$, но не проверял детали.*

31. ♥ Рассмотрим следующий вариант классической задачи о ханойских башнях. Как обычно, имеется n дисков разного размера на трёх стержнях с номерами 0, 1 и 2. Изначально все диски находятся на стержне 0, отсортированные по размеру: меньшие сверху, большие снизу. Цель — перенести все диски на стержень 2. За один ход можно перенести верхний диск любого стержня на другой, соблюдая два ограничения. Во-первых, никогда нельзя класть меньший диск на больший. Во-вторых, и это нестандартная часть, никогда нельзя переносить диск непосредственно со стержня 0 на стержень 2.

Опишите и проанализируйте алгоритм вычисления точного числа ходов, необходимых для переноса всех n дисков со стержня 0 на стержень 2 при указанных ограничениях. Для полного балла в худшем случае алгоритм должен использовать всего $O(\log n)$ арифметических

операций. При анализе считайте, что сложение или умножение двух k-значных чисел требует $O(k)$ времени. Подсказка: матрицы!

> *Примечание переводчика: в первом ограничении оригинала, вероятно, перепутаны размеры дисков: при стандартных правилах нельзя класть* ***больший*** *диск на меньший. Формулировка автора сохранена.*

Разбиение последовательностей и массивов

32. *Базовое арифметическое выражение* состоит из символов множества $\{1, +, \times\}$ и скобок. Почти каждое целое число можно представить более чем одним таким выражением. Например, по утверждению автора, все следующие базовые выражения представляют число 14:

```
1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1
((1 + 1) × (1 + 1 + 1 + 1 + 1)) + ((1 + 1) × (1 + 1))
(1 + 1) × (1 + 1 + 1 + 1 + 1 + 1 + 1)
(1 + 1) × (((1 + 1 + 1) × (1 + 1)) + 1)
```

Опишите и проанализируйте алгоритм, который по целому числу n вычисляет минимальное число единиц в базовом выражении со значением n. Число скобок не имеет значения, важны только единицы. Например, при $n = 14$ алгоритм должен вернуть 8 для последнего выражения выше. Время работы должно ограничиваться полиномом небольшой степени от n.

> *Примечание переводчика: первая строка примера оригинала содержит 16 единиц и равна 16, а не 14.*

33. Дана последовательность целых чисел, разделённых знаками $+$ и $-$, например:

$$1 + 3 - 2 - 5 + 1 - 6 + 7.$$

Значение выражения можно изменить, по-разному расставляя скобки:

$$1 + 3 - 2 - 5 + 1 - 6 + 7 = -1,$$

$$(1 + 3 - (2 - 5)) + (1 - 6) + 7 = 9,$$

$$(1 + (3 - 2)) - (5 + 1) - (6 + 7) = -17.$$

Опишите и проанализируйте алгоритм, который по списку целых чисел, разделённых знаками $+$ и $-$, вычисляет максимально возможное значение после расстановки скобок. Скобки разрешено использовать только для группировки сложений и вычитаний; в частности, нельзя создавать неявное умножение, как в $1 + 3(-2)(-5) + 1 - 6 + 7 = 33$.

34. Дана последовательность целых чисел, разделённых знаками $+$ и $\times$, например:

$$1 + 3 \times 2 \times 0 + 1 \times 6 + 7.$$

Значение можно изменить, расставив скобки по-разному:

$$(1 + (3 \times 2)) \times 0 + (1 \times 6) + 7 = 13,$$

$$((1 + (3 \times 2 \times 0) + 1) \times 6) + 7 = 19,$$

$$(1 + 3) \times 2 \times (0 + 1) \times (6 + 7) = 104.$$

а. Опишите и проанализируйте алгоритм вычисления максимального возможного значения после расстановки скобок, если все входные числа положительны. Подсказка: это легко.

б. Сделайте то же, если все входные числа неотрицательны.

в. Сделайте то же без ограничений на входные числа.

Считайте, что любая арифметическая операция занимает $O(1)$ времени.

35. После окончания университета Шам-Пубанана вы решили пройти собеседование на должность в банке Уолл-стрит Long Live Boole. Управляющий директор банка Элуб Эгроег ставит перед каждым новым сотрудником задачу в духе «реши или умри», которую нужно решить за 24 часа. Не справившихся немедленно увольняют!

Впервые войдя в банк, вы замечаете, что кабинеты сотрудников расположены в один ряд, а на двери каждого крупно напечатано `T` или `F`. Между каждой парой соседних кабинетов находится табличка с одним из символов $\wedge$, $\vee$ или $\oplus$. На вопрос об этих загадочных знаках Элуб подтверждает, что `T` и `F` обозначают логические значения `True` и `False`, а символы — стандартные логические операции И, ИЛИ и исключающее ИЛИ. Он также объясняет, что буквы и символы показывают, какие сочетания сотрудников могут успешно работать вместе. В начале каждого нового проекта Элуб иерархически группирует сотрудников, добавляя скобки к последовательности символов, чтобы получить однозначное логическое выражение. Проект успешен, если выражение в скобках вычисляется в `T`.

Например, если в банке три сотрудника и последовательность знаков на дверях и между ними равна $T \wedge F \oplus T$, существует ровно один успешный способ расстановки скобок: $T \wedge (F \oplus T)$. А если последовательность равна $F \wedge T \oplus F$, расставить скобки так, чтобы проект был успешным, невозможно.

Наконец Элуб задаёт вам вопрос «реши или умри»: опишите алгоритм, определяющий, можно ли расставить скобки в заданной последовательности символов так, чтобы результат равнялся `T`. Вход — массив $S[0..2n]$, где $S[i] \in \{T, F\}$ при чётном i и $S[i] \in \{\vee, \wedge, \oplus\}$ при нечётном.

36. Каждый год на ежегодной встрече Антарктических любителей улиток Верхнего Глейсиервилля проходит гонка спаривания на круглом столе. Несколько высококлассных племенных улиток размещают у края стола, пронумеровав по порядку от 1 до n. Во время гонки каждая улитка ползает по столу, оставляя слизистый след. Улитки специально обучены не падать со стола и не пересекать следы слизи, даже собственные. Когда две улитки встречаются, их объявляют парой, снимают со стола и уносят в романтическую ямку в земле, чтобы они произвели на свет маленьких улитят. Некоторые улитки могут никогда не найти пару, даже если гонка будет длиться вечно.

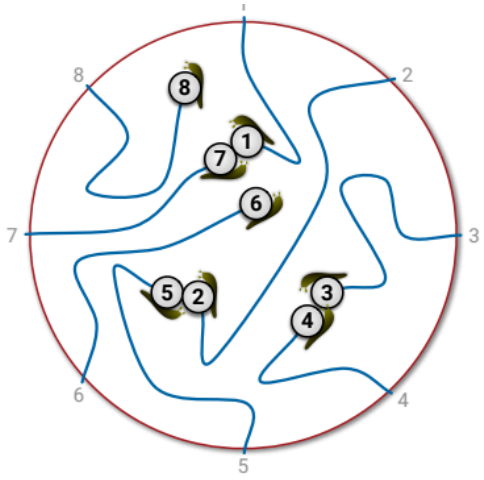


Рисунок 3.6. Конец типичной антарктической гонки SLUG. Улитки 6 и 8 не нашли пару. Организаторы должны выплатить $M[3,4] + M[2,5] + M[1,7]$.

Для каждой пары улиток организаторы объявили денежное вознаграждение, выплачиваемое владельцам, если эти улитки встретятся во время гонки. На стене за круглым столом вывешен двумерный массив $M[1..n, 1..n]$, где $M[i,j] = M[j,i]$ — награда за встречу улиток i и j.

Опишите и проанализируйте алгоритм, вычисляющий по массиву M максимальную общую сумму наград, которую организаторов могут вынудить выплатить.

37. Вы добыли в карьере большую мраморную плиту. Для простоты пусть это прямоугольник высотой n и шириной m дюймов. Вы хотите разрезать его на прямоугольники поменьше разных размеров: одни для кухонных столешниц, другие для больших скульптур, третьи для надгробий. Мраморная пила умеет делать горизонтальный или вертикальный разрез через всю прямоугольную плиту. В любой момент можно узнать текущую цену $P[x, y]$ прямоугольника размером x на y дюймов для любых положительных целых x и y. Цены зависят от спроса, а покупатели мраморных столешниц странные люди, поэтому не предполагайте о ценах ничего; в частности, большие прямоугольники могут стоить значительно дешевле.

По массиву текущих цен и целым числам m и n опишите алгоритм, вычисляющий, как разрезать плиту $n \times m$ для максимальной прибыли.

38. Требуется разработать эффективные алгоритмы построения оптимальных двоичных деревьев поиска с дополнительными ограничениями балансировки. Вход состоит из отсортированного массива ключей $A[1..n]$ и массива частот $f[1..n]$, где $f[i]$ — число поисков ключа $A[i]$. Функция стоимости та же, что в разделе 3.9. Теперь нужно построить оптимальное дерево, удовлетворяющее дополнительным ограничениям.

а. АВЛ-деревья — одни из первых самобалансирующихся двоичных деревьев поиска; их впервые описали в 1962 году Георгий Адельсон-Вельский и Евгений Ландис. В АВЛ-дереве высоты левого и правого поддеревьев любой вершины v отличаются не более чем на единицу. Опишите и проанализируйте алгоритм построения оптимального АВЛ-дерева для заданных ключей и частот.

б. Симметричные двоичные B-деревья — ещё один вид самобалансирующихся двоичных деревьев, впервые описанный Рудольфом Байером в 1972 году. После несколько более простой переформулировки Лео Гибаса и Роберта Седжвика 1978 года они более известны как красно-чёрные деревья. Красно-чёрное дерево — двоичное дерево поиска со следующими ограничениями:

- каждая вершина либо красная, либо чёрная;
- родитель каждой красной вершины чёрный;
- каждый путь от корня до листа содержит одинаковое число чёрных вершин.

Опишите рекурсивный алгоритм с возвратом для построения оптимального красно-чёрного дерева по заданным ключам и частотам.

в. AA-деревья предложил Арне Андерссон в 1993 году; в 2000 году Марк Аллен Вайс немного упростил их и дал им название. AA-деревья также известны как левосторонние красно-чёрные деревья после симметричной переформулировки с другими алгоритмами балансировки Роберта Седжвика в 2006 году. AA-дерево — красно-чёрное дерево с дополнительным ограничением: ни один левый ребёнок не является красным.[3-23] Опишите и проанализируйте алгоритм построения оптимального AA-дерева для заданных ключей и частот.

> ***Примечание 3-23.*** *Переформулировка Седжвика требует, чтобы красным не был ни один правый ребёнок. Как угодно. Андерссон и Седжвик странным образом умалчивают, следует ли измерять углы по часовой стрелке или против, является ли Плутон планетой, означает ли «более низкий ранг» «лучше» или «хуже» и лучше ли сражаться с сотней лошадей размером с утку или с одной уткой размером с лошадь.*

39. Дан битовый рисунок $m \times n$ в виде массива $M[1..m, 1..n]$ из нулей и единиц. Сплошной блок в M — подмассив вида $M[i..i', j..i']$, все биты которого равны. Нужно разбить M на как можно меньшее число непересекающихся блоков.

Одна естественная рекурсивная стратегия разбиения называется *гильотинным разрезанием*. Если весь рисунок M — сплошной блок, делать нечего. Иначе мы разрезаем M на два меньших рисунка по горизонтальной или вертикальной линии, а затем рекурсивно разбиваем их на сплошные блоки.

Любое гильотинное разрезание можно представить двоичным деревом: каждый внутренний узел хранит положение и направление разреза, а каждый лист — один бит, 0 или 1, показывающий содержимое соответствующего блока. Размер гильотинного разрезания — число листьев соответствующего двоичного дерева, то есть окончательное число сплошных блоков; глубина разрезания — глубина этого дерева.

а. Опишите и проанализируйте алгоритм вычисления гильотинного разрезания M минимального размера.

б. Покажите, что гильотинное разрезание не всегда даёт разбиение на наименьшее число сплошных блоков.

в. Опишите и проанализируйте алгоритм вычисления гильотинного разрезания M минимальной глубины.

г. Опишите и проанализируйте алгоритм определения $M[i, j]$ по дереву, представляющему гильотинное разрезание M, и двум индексам i и j.

> *Примечание переводчика: в исходном определении блока указан диапазон $M[i..i', j..i']$; вероятно, второй диапазон должен оканчиваться на j'.*

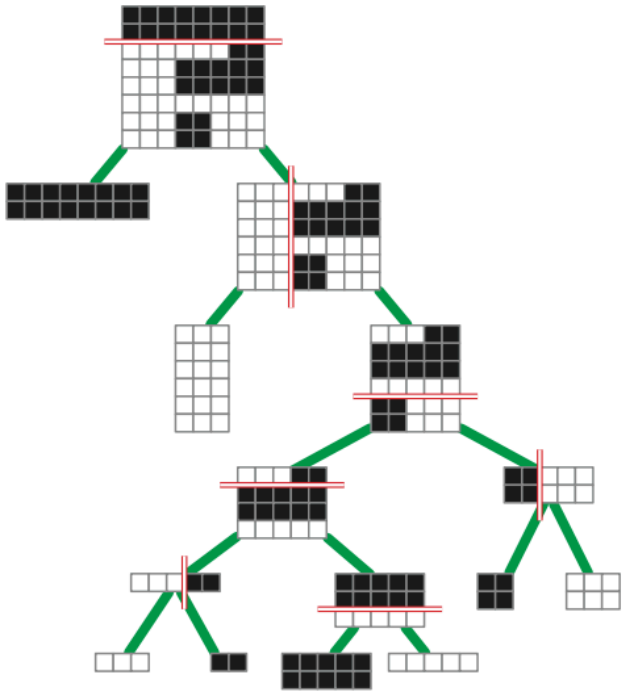


Рисунок 3.7. Гильотинное разрезание размера 8 и глубины 5.

д. Определим глубину пикселя $M[i, j]$ в гильотинном разрезании как глубину листа, содержащего этот пиксель. Опишите и проанализируйте алгоритм вычисления гильотинного разрезания M, для которого сумма глубин всех пикселей минимальна.

е. Опишите и проанализируйте алгоритм вычисления гильотинного разрезания M, для которого сумма глубин чёрных пикселей минимальна.

40. ♠ Поздравляем! Вас нанял гигантский интернет-магазин книг DeNile («Не только река в Египте!»), чтобы оптимизировать складских роботов. Каждая продаваемая книга имеет уникальный ISBN — международный стандартный книжный номер, являющийся просто числом. На каждом складе DeNile есть длинный ряд ячеек; в каждой лежит несколько экземпляров одной книги. Ячейки отсортированы по ISBN, который напечатан на их передней стороне в машиночитаемой форме. Книги извлекают роботы, движущиеся по рельсам вдоль ряда ячеек.

DeNile не хранит список соответствия ячеек и ISBN — это было бы слишком просто! Вместо этого робот сначала должен найти ячейку нужной книги с помощью двоичного поиска.

Поскольку поиск требует физического движения робота, мы больше не можем считать, что каждый шаг двоичного поиска занимает $O(1)$ времени. А именно:

- робот всегда начинает у «нулевой ячейки», где книги укладывают в коробки для отправки покупателям;
- перемещение от i-й ячейки к j-й занимает $\alpha|i - j|$ секунд при некоторой константе α;
- для чтения ISBN робот должен находиться непосредственно перед ячейкой; чтение занимает β секунд;
- изменение направления движения робота с возрастающих индексов на убывающие или наоборот занимает дополнительные γ секунд;
- найдя нужную ячейку, робот извлекает одну книгу и возвращается к нулевой ячейке.

Разработайте и проанализируйте алгоритм построения двоичного дерева поиска по ячейкам, минимизирующего суммарное время поиска книг роботом. Вход — массив целых чисел $f[1..n]$, где $f[i]$ — число запросов книги из i-й ячейки, а также временные параметры α, β и γ.

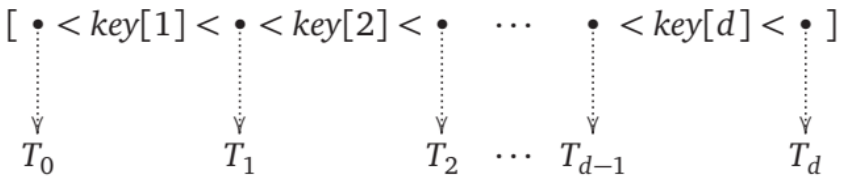
41. ♠ Стандартный способ улучшить работу деревьев поиска с кэшем — помещать в каждую вершину больше ключей и поддеревьев. *B-дерево* — корневое дерево, в каждом внутреннем узле которого хранится до B ключей и указатели не более чем на $B + 1$ детей, каждый из которых является корнем меньшего B-дерева. А именно, каждый узел v хранит три поля:

- положительное целое число $v.d \le B$;
- отсортированный массив `v.key[1..v.d]`;
- массив указателей на детей `v.child[0..v.d]`.

В частности, указателей на детей всегда ровно на один больше, чем ключей.[3-24]

Каждый указатель `v.child[i]` равен `Null` либо указывает на корень B-дерева, все ключи которого больше `v.key[i]` и меньше `v.key[i+1]`. В частности, все ключи в крайнем левом поддереве `v.child[0]` меньше `v.key[1]`, а все ключи в крайнем правом поддереве `v.child[v.d]` больше `v.key[v.d]`.

Интуитивно следует представлять чередование поддеревьев $T_0, \ldots, T_d$ и разделяющих их ключей `key[1]`, ..., `key[d]`: T_i — поддерево, на которое указывает `child[i]`.



Стоимость поиска ключа x в B-дереве — число узлов на пути от корня до узла, содержащего x среди своих ключей. 1-дерево — обычное двоичное дерево поиска.

Зафиксируем произвольное положительное целое $B > 0$. Я предлагаю $B = 8$. Дан отсортированный массив ключей поиска $A[1..n]$ и соответствующий массив частот $F[1..n]$, где $F[i]$ — число обращений к $A[i]$. Нужно описать и проанализировать эффективный алгоритм

нахождения B-дерева с минимальной общей стоимостью поиска заданных ключей с заданными частотами.

а. Опишите полиномиальный алгоритм для частного случая $B = 2$.

б. Опишите алгоритм для произвольного B, работающий за $O(n^{B+c})$ времени при некотором фиксированном целом c.

в. ♥ Опишите алгоритм для произвольного B, работающий за $O(n^c)$ времени при некотором фиксированном целом c, не зависящем от B.

> ***Примечание 3-24.*** *Обычно на B-деревья накладывают ещё два ограничения, обеспечивающие стоимость поиска $O(\log_B n)$ в худшем случае: каждый лист должен иметь одинаковую глубину, а каждый узел, за возможным исключением корня, должен содержать не менее $B/2$ ключей. Но здесь нас интересует не оптимизация стоимости поиска в худшем случае, а общая стоимость последовательности поисков, поэтому мы не будем вводить эти ограничения.*

42. Строка w, состоящая из круглых скобок `(`, `)` и квадратных скобок `[` , `]`, *сбалансирована*, если выполняется одно из условий:

- w — пустая строка;
- $w = (x)$ для некоторой сбалансированной строки x;
- $w = [x]$ для некоторой сбалансированной строки x;
- $w = xy$ для некоторых сбалансированных строк x и y.

Например, строка `([()][])) [()()] ()` сбалансирована, поскольку $w = xy$, где $x =$ `( [()] [] () )`, а $y =$ `[ () () ] ()`.

а. Опишите и проанализируйте алгоритм определения, сбалансирована ли заданная строка круглых и квадратных скобок.

б. Опишите и проанализируйте алгоритм вычисления длины наибольшей сбалансированной подпоследовательности заданной строки круглых и квадратных скобок.

в. Опишите и проанализируйте алгоритм вычисления длины кратчайшей сбалансированной надпоследовательности заданной строки круглых и квадратных скобок.

г. Опишите и проанализируйте алгоритм вычисления минимального расстояния редактирования от заданной строки круглых и квадратных скобок до некоторой сбалансированной строки круглых и квадратных скобок.

д. ♥ Опишите и проанализируйте алгоритм вычисления наибольшей общей сбалансированной подпоследовательности двух заданных строк круглых и квадратных скобок.

е. ♥ Опишите и проанализируйте алгоритм вычисления наибольшей палиндромной сбалансированной подпоследовательности заданной строки круглых и квадратных скобок.

ж. ♥ Опишите и проанализируйте алгоритм вычисления наибольшей общей палиндромной сбалансированной подпоследовательности — уф! — двух заданных строк круглых и квадратных скобок.

Для каждой задачи вход — массив $w[1..n]$, где каждый элемент принадлежит множеству $\{(,), [,]\}$. Возможно, вместо круглых и квадратных скобок вы предпочтёте другие символы, например `L`, `R`, `1`, `r`, но сообщите проверяющему, какие именно используете!

43. ♥ Поздравляем! Ваша исследовательская группа только что получила многолетний проект стоимостью 50 миллионов долларов, совместно финансируемый DARPA, Google и McDonald's, по созданию DWIM — первого компилятора, читающего мысли программистов! В заявке и многочисленных пресс-релизах вы обещали, что DWIM будет автоматически исправлять ошибки в любом коде, изменяя его как можно меньше. К сожалению, теперь пришло время заставить эту проклятую штуку действительно работать.

Для разминки вы решили заняться необходимой подзадачей. Напомним, что расстояние редактирования между строками — минимальное число вставок, удалений и замен отдельных символов, необходимое для превращения одной строки в другую. Арифметическое выражение — строка w, для которой выполнено одно из условий:

- w — строка из одной или нескольких десятичных цифр;
- $w = (x)$ для некоторого арифметического выражения x;
- $w = x \bullet y$ для некоторых арифметических выражений x и y и некоторой бинарной операции $\bullet$.

Дана строка токенов из алфавита $\{\#, \bullet, (,)\}$, где `#` представляет десятичную цифру, а $\bullet$ — бинарную операцию. Опишите и проанализируйте алгоритм вычисления минимального расстояния редактирования от данной строки до арифметического выражения.

44. Молекулы рибонуклеиновой кислоты, РНК, — длинные цепи из миллионов нуклеотидов, или оснований, четырёх видов: аденина (A), цитозина (C), гуанина (G) и урацила (U). Последовательность молекулы РНК — строка $b[1..n]$, в которой каждый символ $b[i] \in \{A, C, G, U\}$ соответствует основанию. Помимо химических связей между соседними основаниями последовательности, между некоторыми парами оснований могут возникать водородные связи. Множество связанных пар называется *вторичной структурой* молекулы РНК.

Назовём две пары оснований (i, j) и (i', j') при $i < j$ и $i' < j'$ перекрывающимися, если $i < i' < j < j'$ или $i' < i < j' < j$. На практике большинство пар не перекрываются. Перекрывающиеся пары создают во вторичной структуре так называемые псевдоузлы, важные для некоторых функций РНК, но более сложные для предсказания.

Допустим, мы хотим предсказать наилучшую вторичную структуру заданной последовательности РНК. Используем крайне упрощённую модель:

- каждое основание может связываться не более чем с одним другим;
- связываться могут только пары A–U и C–G;
- пары вида $(i, i + 1)$ и $(i, i + 2)$ связываться не могут;

- связанные пары оснований не могут перекрываться.

Последнее, наименее реалистичное ограничение позволяет изображать вторичную структуру РНК как своего рода толстое дерево.

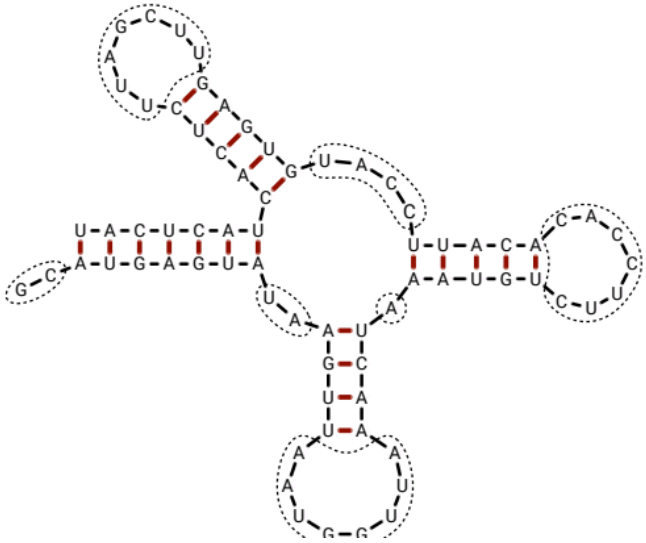


Рисунок 3.8. Пример вторичной структуры РНК с 21 связанной парой оснований, показанной толстыми красными линиями. Пропуски обозначены пунктирными кривыми. Оценка структуры равна $2^2 + 2^2 + 8^2 + 1^2 + 7^2 + 4^2 + 7^2 = 187$.

а. Опишите и проанализируйте алгоритм вычисления максимально возможного числа связанных пар оснований во вторичной структуре заданной последовательности РНК.

б. *Пропуск* во вторичной структуре — максимальная подстрока несвязанных оснований. Большие пропуски приводят к химической нестабильности, поэтому структуры с меньшими пропусками более вероятны. Чтобы учесть это предпочтение, определим оценку вторичной структуры как сумму квадратов длин пропусков; см. рисунок 3.8. Эта функция оценки полностью вымышлена: для реального предсказания структуры РНК нужны гораздо более сложные функции. Опишите и проанализируйте алгоритм вычисления минимальной возможной оценки вторичной структуры для заданной последовательности РНК.

45. ♣

а. Опишите и проанализируйте эффективный алгоритм определения, принадлежит ли заданная строка w языку $L(R)$ заданного регулярного выражения R.

б. Обобщённые регулярные выражения допускают бинарную операцию $\cap$ — пересечение — и унарную операцию $\neg$ — дополнение — помимо обычных операций $\bullet$ — конкатенации, $+$ — ИЛИ и $*$ — замыкания Клини. Конструкции недетерминированных конечных автоматов и теорема Клини показывают, что любое обобщённое регулярное выражение E представляет регулярный язык $L(E)$. Опишите и проанализируйте эффективный алгоритм определения, принадлежит ли заданная строка w языку $L(E)$.

В обеих задачах предположите, что задано дерево разбора соответствующего регулярного выражения, а не просто строка.

Деревья и поддеревья

46. Вас только что назначили организатором первого ежегодного обязательного праздничного вечера в компании Giggle, дочерней компании Abugida. Сотрудники Giggle образуют строгую иерархию — дерево с президентом компании в корне. Всезнающие оракулы из отдела кадров присвоили каждому сотруднику вещественное число, измеряющее, насколько он «весёлый». Чтобы праздник был более дружеским, на список гостей наложено ограничение: сотрудник не может присутствовать, если присутствует его непосредственный руководитель. С другой стороны, президент компании должна присутствовать, хотя её показатель веселья отрицателен: в конце концов это её компания. Дайте алгоритм составления списка гостей, максимизирующего сумму их показателей «веселья».

47. Поскольку на прошлогодний праздничный вечер пришло слишком мало людей, президент Giggle решила в этом году вместо этого подарить каждому сотруднику подарок. Каждый должен получить один из трёх подарков: полностью оплаченный шестинедельный отпуск в любой точке мира; завтрак на двоих с возможностью съесть столько блинов, сколько сумеешь отсортировать, в заведении Jumping Jack Flash's Flapjack Stack Shack; либо горящий бумажный пакет, полный собачьих экскрементов. Корпоративные правила запрещают сотруднику получать точно такой же подарок, как у его непосредственного руководителя. Любого, кто получит подарок лучше руководителя, почти наверняка уволят из зависти.

Как официальный распорядитель праздника Giggle, вы должны решить, кто какой подарок получит. Опишите алгоритм распределения подарков так, чтобы уволили как можно меньше людей. Да, вы можете отправить президенту горящий пакет собачьих экскрементов.

Более формально, дано корневое дерево T, представляющее иерархию компании. Требуется пометить его вершины числами 1, 2 или 3 так, чтобы метка каждой вершины отличалась от метки её родителя. Стоимость разметки — число вершин, у которых метка меньше метки родителя. Пример см. на рисунке 3.9. Опишите и проанализируйте алгоритм вычисления разметки T минимальной стоимости.

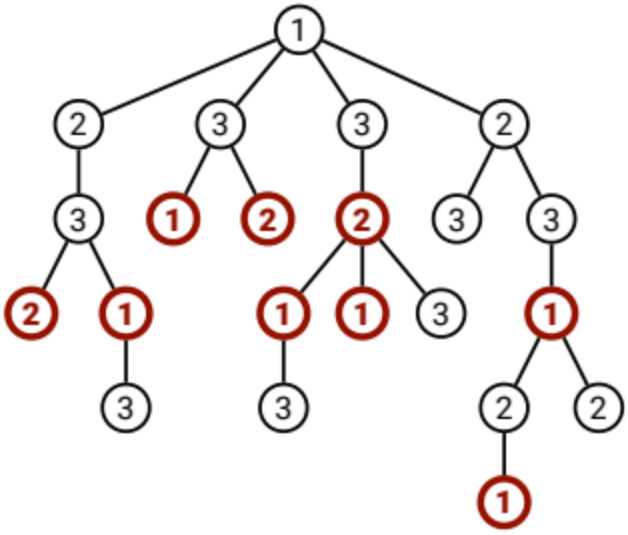


Рисунок 3.9. Разметка дерева стоимостью 9. Девять выделенных вершин имеют метки меньше меток их родителей. Это не оптимальная разметка данного дерева.

48. После праздничного фиаско с горящими собачьими экскрементами вам настоятельно посоветовали поискать другую работу, и вы ушли из Giggle в конкурирующую компанию Twitbook. К сожалению, новый президент Twitbook только что решила последовать примеру Giggle и устроить собственный праздник, а с учётом вашего прошлого опыта назначила вас официальным организатором. Президент требует пригласить ровно k сотрудников, включая её саму, и все приглашённые обязаны присутствовать. Да, будет весело.

Как и в Giggle, сотрудники Twitbook образуют строгую иерархию — дерево с президентом в корне. Всезнающие оракулы отдела кадров присвоили каждому сотруднику вещественное число, показывающее, насколько неловко одновременно приглашать его и непосредственного руководителя. Отрицательное значение означает, что сотрудник и руководитель на самом деле нравятся друг другу. Нужно выбрать ровно k сотрудников так, чтобы общая неловкость праздника была минимальной. Например, если ни одна приглашённая пара не состоит из сотрудника и его непосредственного руководителя, общая неловкость равна нулю. Вход алгоритма — дерево T, целое k и неловкость каждой вершины T.

а. Опишите алгоритм вычисления суммарной неловкости наименее неловкого набора из k сотрудников, если иерархия компании задана двоичным деревом, то есть каждый сотрудник непосредственно руководит не более чем двумя другими.

б. ♥ Опишите алгоритм вычисления суммарной неловкости наименее неловкого набора из k сотрудников без ограничений на иерархию компании.

49. Нужно распространить сообщение по всем вершинам корневого дерева. Изначально сообщение известно только корню. За один раунд любая вершина, знающая сообщение, может передать его не более чем одному своему ребёнку. Пример см. на рисунке 3.10.

а. Разработайте алгоритм вычисления минимального числа раундов, необходимых для распространения сообщения по всем вершинам двоичного дерева.

б. ♦ Разработайте алгоритм вычисления минимального числа раундов для произвольного корневого дерева. Подсказка: для доказательства правильности могут пригодиться методы следующей главы, хотя алгоритм не является жадным.

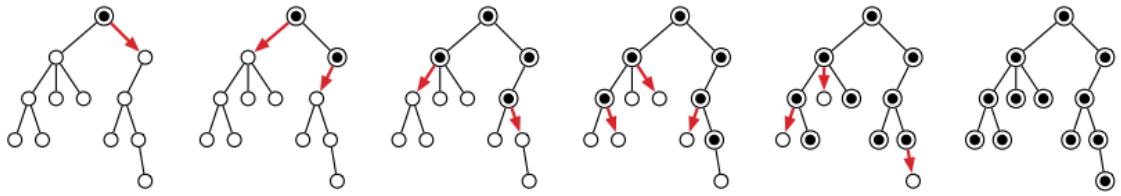


Рисунок 3.10. Распространение сообщения по дереву за пять раундов.

50. Однажды Алекс надоело лазать в зале, и она решила вывести большую группу друзей-скалолазов на природу. На месте оказался огромный широкий, но невысокий валун с отмеченными зацепами для рук и ног. Алекс быстро определила допустимое множество переходов, которые её друзья могут выполнять между зацепами.

Систему зацепов можно описать корневым деревом T из n вершин: каждой вершине соответствует зацеп, а каждому ребру — допустимый переход. По мере подъёма пути сходятся к единственному зацепу на вершине валуна, представленному корнем T.[3-25]

Алекс и её друзья — все отличные скалолазы — решили сыграть в игру: на валуне одновременно находится как можно больше участников, и каждый должен выполнить последовательность ровно из k переходов. Каждый может начать с произвольного зацепа, но двигаться только от земли вверх. Таким образом, каждый проходит путь из k рёбер дерева T, направленных к корню. Однако два участника не могут касаться одного зацепа: их пути вообще не должны пересекаться.

Опишите и проанализируйте эффективный алгоритм вычисления максимального числа участников. Более формально, по корневому дереву T и целому k нужно найти наибольшее число непересекающихся путей длины k. Не предполагайте, что T — двоичное дерево. Например, для дерева на рисунке 3.11 и $k = 3$ алгоритм должен вернуть 8.

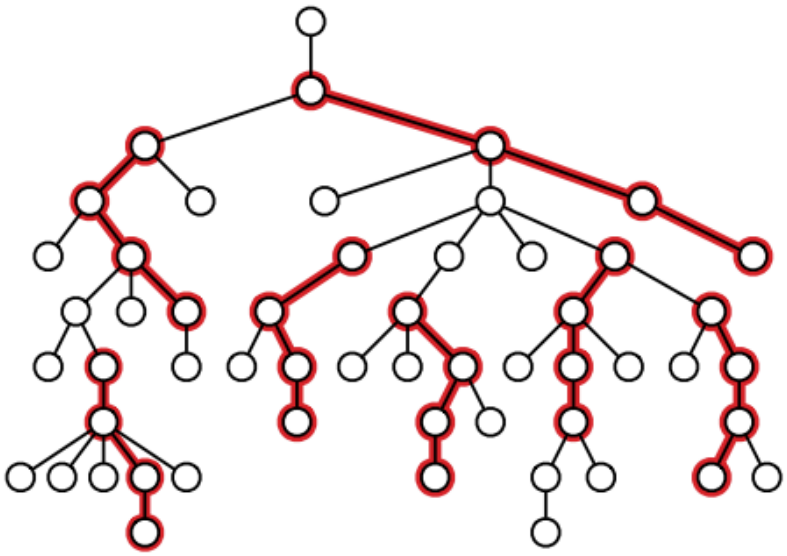


Рисунок 3.11. Семь непересекающихся путей длины $k = 3$. Это не наибольшее такое множество путей для данного дерева.

> ***Примечание 3-25.*** *Вопрос: почему преподаватели информатики считают, что корни деревьев находятся сверху? Ответ: потому что они никогда не выходили на улицу!*

51. Пусть T — корневое двоичное дерево из n вершин, а $k \le n$ — положительное целое число. Нужно пометить k вершин T так, чтобы у каждой вершины был поблизости помеченный предок. Более формально, определим стоимость кластеризации множества вершин K как

$$\operatorname{cost}(K) = \max_v \operatorname{cost}(v, K),$$

где максимум берётся по всем вершинам дерева, а $\operatorname{cost}(v, K)$ — расстояние от v до ближайшего предка из K:

$$\operatorname{cost}(v, K) = \begin{cases} 0, & v \in K, \\ \infty, & v \text{ — корень } T \text{ и } v \notin K, \\ 1 + \operatorname{cost}(\operatorname{parent}(v)), & \text{иначе.} \end{cases}$$

В частности, $\operatorname{cost}(K) = \infty$, если K не содержит корень T.

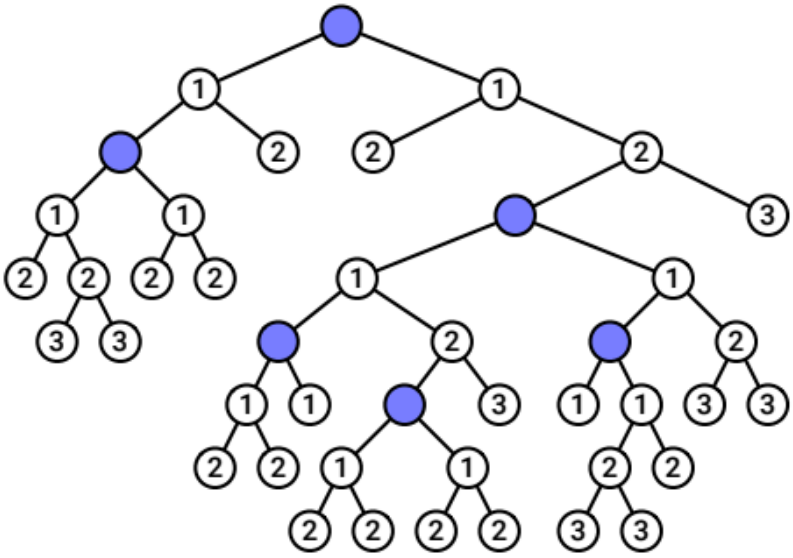


Рисунок 3.12. Подмножество пяти вершин двоичного дерева со стоимостью кластеризации 3.

а. ♥ Опишите алгоритм динамического программирования, который по дереву T и целому k вычисляет минимальную стоимость кластеризации среди всех подмножеств из k вершин. Для полного балла алгоритм должен работать за $O(n^2k^2)$ времени.

б. Опишите алгоритм динамического программирования, который по дереву T и целому r вычисляет размер наименьшего подмножества вершин со стоимостью кластеризации не более r. Для полного балла алгоритм должен работать за $O(nr)$ времени.

в. Покажите, что решение пункта б даёт алгоритм для пункта а, работающий за $O(n^2 \log n)$ времени.

52. Требуется найти эффективные алгоритмы вычисления наибольшего общего корневого поддерева двух заданных корневых деревьев. Напомним: корневое дерево — связный ациклический граф с выделенной вершиной, называемой корнем. Корневое поддерево состоит из произвольной вершины и всех её потомков. Точное значение слова «общее» зависит от того, какие пары корневых деревьев считаются изоморфными.

а. Двоичное дерево — корневое дерево, каждый узел которого имеет левое и правое поддеревья, каждое из которых может быть пустым. Два двоичных дерева изоморфны тогда и только тогда, когда оба пусты либо их левые поддеревья изоморфны и их правые поддеревья изоморфны. Опишите алгоритм поиска наибольшего общего двоичного поддерева двух заданных двоичных деревьев.

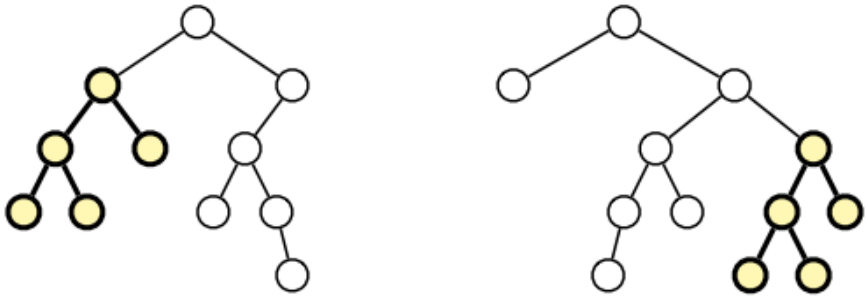


Рисунок 3.13. Два двоичных дерева с выделенным наибольшим общим корневым поддеревом.

б. В упорядоченном корневом дереве каждый узел имеет последовательность детей, являющихся корнями упорядоченных корневых поддеревьев. Два таких дерева изоморфны, если они оба пусты либо их i-е поддеревья изоморфны для каждого индекса i. Опишите алгоритм поиска наибольшего общего упорядоченного поддерева двух упорядоченных деревьев T_1 и T_2.

в. ♦♥ В неупорядоченном корневом дереве каждый узел имеет неупорядоченное множество детей, являющихся корнями неупорядоченных корневых поддеревьев. Два таких дерева изоморфны, если они оба пусты либо поддеревья каждого корня можно упорядочить так, чтобы их i-е поддеревья были изоморфны для каждого i. Опишите алгоритм поиска наибольшего общего неупорядоченного поддерева двух неупорядоченных деревьев T_1 и T_2.

53. Требуется найти эффективные алгоритмы вычисления оптимальных поддеревьев некорневых деревьев — связных ациклических неориентированных графов. Поддерево некорневого дерева — любой его связный подграф.

а. Дано некорневое дерево T с весами рёбер, которые могут быть положительными, отрицательными или нулевыми. Опишите алгоритм поиска пути с максимальным суммарным весом.

б. Дано некорневое дерево T с весами вершин, которые могут быть положительными, отрицательными или нулевыми. Опишите алгоритм поиска поддерева с максимальным суммарным весом. Это был вопрос на собеседовании в Google в 2016 году.

в. Пусть T_1 и T_2 — произвольные упорядоченные некорневые деревья, то есть для соседей каждого узла определён циклический порядок. Опишите алгоритм поиска их наибольшего общего упорядоченного поддерева.

г. ♦♥ Пусть T_1 и T_2 — произвольные неупорядоченные некорневые деревья. Опишите алгоритм поиска их наибольшего общего неупорядоченного поддерева.

54. Корневые миноры корневых деревьев — естественное обобщение подпоследовательностей. Корневой минор дерева T — любое дерево, полученное стягиванием одного или нескольких рёбер. При стягивании ребра $u \to v$, где u — родитель v, дети v становятся новыми детьми u, после чего v удаляется. В частности, корень T является корнем каждого его корневого минора.

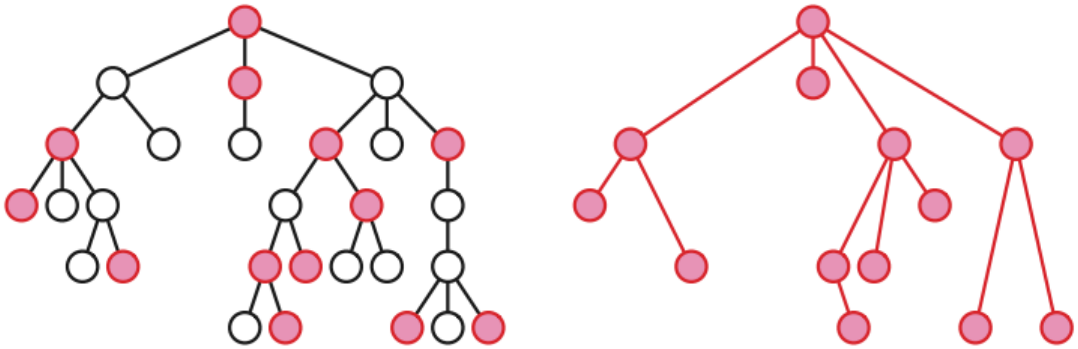


Рисунок 3.14. Корневое дерево и один из его корневых миноров.

а. Пусть T — корневое дерево с метками на вершинах. Назовём T скучным, если для каждой вершины x все её дети имеют одинаковые метки; дети разных вершин могут иметь разные метки. Опишите алгоритм поиска наибольшего скучного корневого минора заданного дерева с метками.

б. Дано корневое дерево T, вершины которого помечены числами. Опишите алгоритм поиска наибольшего корневого минора T, упорядоченного как куча: каждая вершина минора M должна иметь меньшую метку, чем её дети в M.

в. Дано двоичное дерево T, вершины которого помечены числами. Опишите алгоритм поиска наибольшего корневого минора T, упорядоченного как двоичное дерево поиска: в возвращённом миноре M каждая вершина имеет не более двух детей, а симметричный обход M является возрастающей подпоследовательностью симметричного обхода T.

г. Корневое дерево называется упорядоченным, если дети каждого узла имеют определённый порядок слева направо. Опишите алгоритм поиска наибольшего минора, упорядоченного как двоичное дерево поиска, для произвольного упорядоченного дерева T с числовыми метками вершин. Как и прежде, порядок вершин слева направо в M должен согласовываться с их порядком в T.

д. ♥ Опишите алгоритм поиска наибольшего общего упорядоченного корневого минора двух упорядоченных корневых деревьев с метками.

е. ♦♥ Опишите алгоритм поиска наибольшего общего неупорядоченного корневого минора двух неупорядоченных корневых деревьев с метками. Подсказка: объедините динамическое программирование с максимальными потоками.

4. Жадные алгоритмы

Суть, дамы и господа, в том, что жадность — это хорошо. Жадность работает, жадность правильна. Жадность проясняет, отсекает лишнее и выражает сущность духа эволюции. Жадность во всех её формах — жажда жизни, денег, любви, знаний — знаменовала восхождение человечества. И жадность, запомните мои слова, спасёт не только Teldar Paper, но и другую плохо работающую корпорацию под названием США.

— Гордон Гекко [Майкл Дуглас], «Уолл-стрит» (1987)

Для каждой человеческой проблемы всегда есть простое решение — аккуратное, правдоподобное и неправильное.

— Г. Л. Менкен, «Божественное вдохновение», New York Evening Mail (16 ноября 1917)

4.1. Хранение файлов на ленте

Предположим, у нас есть n файлов, которые требуется сохранить на магнитной ленте.[1] Впоследствии пользователи захотят читать эти файлы. Чтение файла с ленты отличается от чтения с диска: сначала нужно промотать все предыдущие файлы, а это занимает значительное время. Пусть массив `L[1 .. n]` содержит длины файлов; в частности, длина файла i равна `L[i]`. Если файлы расположены в порядке от 1 до n, стоимость доступа к k-му файлу равна

$$\text{cost}(k) = \sum_{i=1}^{k} L[i].$$

Стоимость учитывает тот факт, что перед чтением файла k нужно сначала пройти мимо всех предыдущих файлов. Если пока предположить, что все файлы запрашиваются с одинаковой вероятностью, ожидаемая стоимость поиска случайного файла равна

$$\mathbb{E}[\text{cost}] = \sum_{k=1}^{n} \frac{\text{cost}(k)}{n} = \frac{1}{n} \sum_{k=1}^{n} \sum_{i=1}^{k} L[i].$$

Если изменить порядок файлов на ленте, изменится и стоимость доступа: некоторые файлы станет дороже читать, а другие — дешевле. Разные порядки, вероятно, дадут разные ожидаемые стоимости. Пусть $\pi(i)$ обозначает индекс файла на позиции i ленты. Тогда ожидаемая стоимость перестановки π равна

$$\mathbb{E}[\text{cost}(\pi)] = \frac{1}{n} \sum_{k=1}^{n} \sum_{i=1}^{k} L[\pi(i)].$$

Какой порядок выбрать, чтобы ожидаемая стоимость была минимальной? Ответ интуитивно кажется очевидным: отсортировать файлы по возрастанию длины. Но интуиция — коварная

штука. Единственный способ убедиться, что этот порядок работает, — взлететь и уничтожить весь объект ядерным ударом с орбиты… то есть действительно доказать, что он работает!

Лемма 4.1. $\mathbb{E}[\mathrm{cost}(\pi)]$ минимальна, когда $L[\pi(i)] \le L[\pi(i+1)]$ для всех i.

Доказательство. Предположим, для некоторого i выполнено $L[\pi(i)] > L[\pi(i+1)]$. Чтобы упростить запись, обозначим $a = \pi(i)$ и $b = \pi(i+1)$. Если поменять файлы a и b местами, стоимость доступа к a увеличится на $L[b]$, а стоимость доступа к b уменьшится на $L[a]$. В целом ожидаемая стоимость изменится на $(L[b] - L[a])/n$. Но это улучшение, поскольку $L[b] < L[a]$. Значит, если файлы расположены не по порядку, ожидаемую стоимость можно уменьшить обменом некоторой неправильно упорядоченной пары. $\square$

Это наш первый пример корректного жадного алгоритма. Чтобы минимизировать суммарную ожидаемую стоимость доступа, сначала помещаем файл, доступ к которому дешевле всего, а затем рекурсивно записываем остальные; никакого возврата и динамического программирования — просто делаем лучший локальный выбор и слепо идём вперёд. При эффективной сортировке время явно равно $O(n \log n)$ плюс время записи файлов. Чтобы показать корректность, мы доказали, что результат любого другого алгоритма можно улучшить некоторым обменом.

Обобщим идею. Пусть также дан массив частот обращений `F[1 .. n]`: за срок службы ленты к файлу i обратятся ровно `F[i]` раз. Теперь суммарная стоимость доступа ко всем файлам равна

$$\Sigma\,\mathrm{cost}(\pi) = \sum_{k=1}^{n} \left(F[\pi(k)] \cdot \sum_{i=1}^{k} L[\pi(i)] \right) = \sum_{k=1}^{n} \sum_{i=1}^{k} \big(F[\pi(k)] \cdot L[\pi(i)] \big).$$

Как и раньше, перестановка файлов может изменить суммарную стоимость. Какой порядок сделает её минимальной? По духу вопрос похож на задачу об оптимальном двоичном дереве поиска, но структура данных и функция стоимости отличаются, поэтому алгоритм тоже должен быть другим.

Мы уже доказали, что при одинаковых частотах файлы нужно отсортировать по возрастанию размера. Если все частоты различны, но длины `L[i]` одинаковы, интуиция подсказывает сортировать по убыванию частоты, начиная с наиболее часто запрашиваемого файла. На самом деле это нетрудно доказать — намёк, намёк, — изменив доказательство леммы 4.1. Но если меняются и размеры, и частоты, сортировать следует по отношению L/F.

Лемма 4.2. $\Sigma\,\mathrm{cost}(\pi)$ минимальна, когда

$$\frac{L[\pi(i)]}{F[\pi(i)]} \le \frac{L[\pi(i+1)]}{F[\pi(i+1)]}$$

для всех i.

Доказательство. Предположим, что для некоторого i

$$L[\pi(i)]/F[\pi(i)] > L[\pi(i+1)]/F[\pi(i+i)].$$

Для краткости положим $a = \pi(i)$ и $b = \pi(i+1)$. При обмене этих файлов стоимость доступа к a увеличится на $L[b]$, а к b уменьшится на $L[a]$. Суммарная стоимость изменится на $L[b]F[a] - L[a]F[b]$. Это улучшение, поскольку

$$\frac{L[a]}{F[a]} > \frac{L[b]}{F[b]} \quad\Longleftrightarrow\quad L[b]F[a] - L[a]F[b] < 0.$$

Значит, если соседние файлы стоят не по порядку, обмен улучшит суммарную стоимость. □

> *Примечание переводчика: в знаменателе первого неравенства доказательства в оригинале напечатано $F[\pi(i+i)]$; по смыслу должно быть $F[\pi(i+1)]$. Для деления на частоты предполагаются положительные значения F; файлы с нулевой частотой можно размещать последними.*

[1] Читателям, намеренным возразить, что магнитные ленты устарели десятилетия назад, предлагаем посетить ближайший суперкомпьютерный центр и попросить показать ленточных роботов. Или представьте расстановку книг на библиотечной полке. Ну, знаете, тех странных похожих на кирпич предметов из мёртвых деревьев и чернил?

4.2. Составление расписания занятий

Следующий пример немного сложнее. Предположим, вы решили бросить информатику и сменить специальность на прикладной хаос. Кафедра проводит все занятия в один и тот же день недели, который студенты называют «трезводнём» — что интересно, преподаватели его так не называют. У каждого занятия своё время начала и окончания: АС 101 («Ландшафтная архитектура из туалетной бумаги») начинается в 22:27 и заканчивается в 23:51; АС 666 («Имманентизация эсхатона») начинается в 16:18 и заканчивается в 16:22, и так далее.

Чтобы закончить учёбу как можно скорее, вы хотите записаться на максимальное количество занятий. В прикладном хаосе выполнять какую-либо реальную работу не требуется. Университетская система регистрации не позволяет записаться на пересекающиеся занятия, а никто на кафедре не знает, как обойти эту «функцию». Какие занятия выбрать?

Формальнее: даны два массива `S[1 .. n]` и `F[1 .. n]`, содержащие время начала и окончания каждого занятия. Для определённости можно предположить, что $0 \le S[i] < F[i] \le M$ для каждого i и некоторого M, например числа пикосекунд в трезводне. Нужно выбрать наибольшее подмножество $X \subseteq \{1, 2, \ldots, n\}$, такое что для любой пары $i, j \in X$ выполняется $S[i] > F[j]$ либо $S[j] > F[i]$.

Задачу можно изобразить, представив каждое занятие прямоугольником, левая и правая координаты которого показывают начало и окончание. Требуется найти наибольшее подмножество прямоугольников, не перекрывающихся по вертикали.

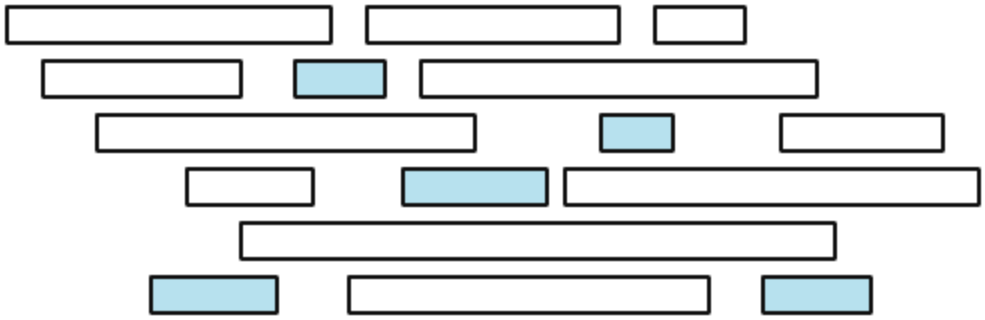


Рисунок 4.1. Максимальное расписание занятий без конфликтов.

У задачи есть довольно простое рекурсивное решение, основанное на том, что вы либо выбираете занятие 1, либо нет. Пусть B — множество занятий, заканчивающихся до начала занятия 1, а A — множество занятий, начинающихся после его окончания:

$$B := \{i \mid 2 \le i \le n \text{ и } F[i] < S[1]\}, \qquad A := \{i \mid 2 \le i \le n \text{ и } S[i] > F[1]\}.$$

Если занятие 1 входит в оптимальное расписание, то в него входят и оптимальные расписания для B и A, которые можно найти рекурсивно. Если нет — рекурсивно найдём оптимальное расписание для $\{2, 3, \ldots, n\}$. Поэтому попробуем оба варианта и выберем лучший. Вычисляя этот рекурсивный алгоритм снизу вверх, получаем динамическое программирование за $O(n^3)$. Я не стану разбирать подробности, потому что можно лучше.[2]

Интуитивно хотелось бы, чтобы первое занятие заканчивалось как можно раньше: тогда останется больше всего других занятий. Это подсказывает простой жадный алгоритм. Просматривайте занятия в порядке окончания и всякий раз, когда очередное не конфликтует с последним выбранным, берите его! Визуализация результата приведена на рисунке 4.2.

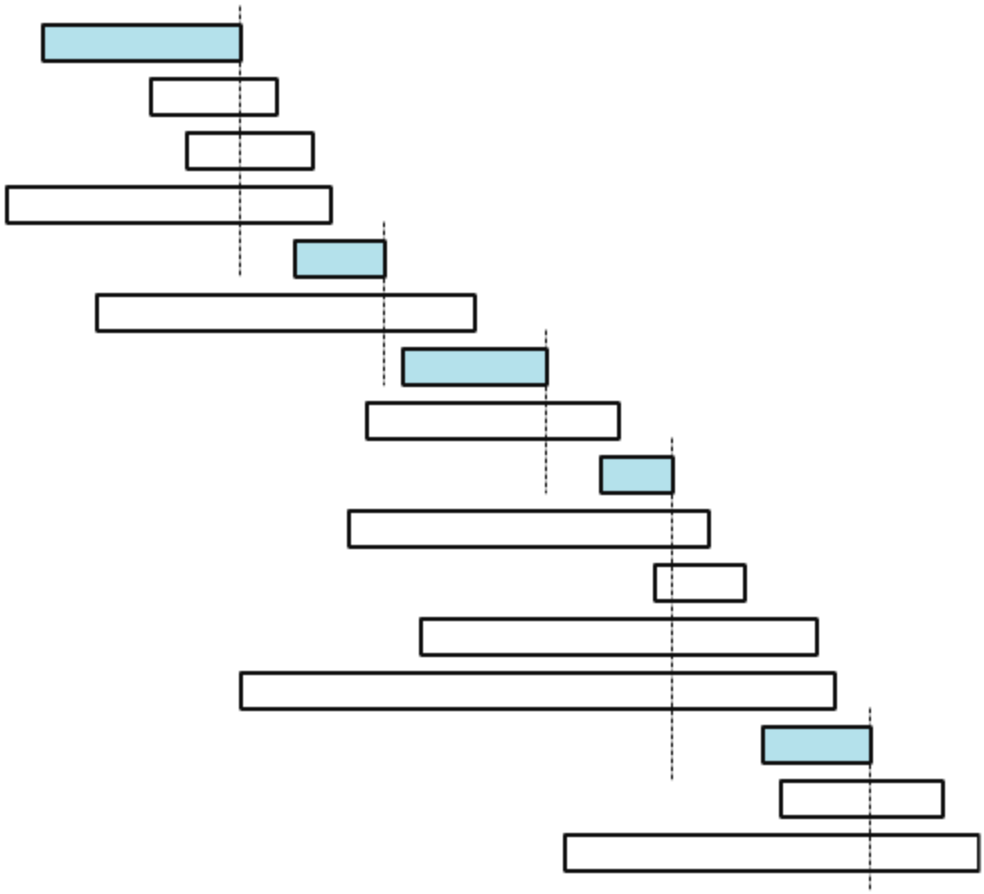


Рисунок 4.2. Те же занятия, упорядоченные по времени окончания, и жадное расписание.

Более формально алгоритм записан на рисунке 4.3. Надеюсь, первая строка понятна. После начальной сортировки остаётся простой линейный цикл, поэтому общее время работы — $O(n \log n)$.

```
GreedySchedule(S[1 .. n], F[1 .. n]):
    отсортировать F и соответствующим образом переставить S
    count ← 1
    X[count] ← 1
    для i ← 2 до n
        если S[i] > F[X[count]]
            count ← count + 1
            X[count] ← i
    вернуть X[1 .. count]
```

Рисунок 4.3. Жадный алгоритм поиска максимального набора непересекающихся занятий.

Чтобы доказать, что `GreedySchedule` действительно вычисляет наибольшее расписание без конфликтов, используем аргумент обмена, подобный доказательству для сортировки ленты. Мы не утверждаем, что жадное расписание — единственное максимальное; могут быть и другие. Сравните рисунки 4.1 и 4.2! Мы утверждаем лишь, что по крайней мере одно оптимальное расписание совпадает с результатом жадного алгоритма.

Лемма 4.3. Хотя бы одно максимальное расписание без конфликтов включает занятие, заканчивающееся первым.

Доказательство. Пусть f заканчивается первым. Предположим, есть максимальное расписание без конфликтов X, не включающее f. Пусть g — первое заканчивающееся занятие в X. Поскольку f заканчивается раньше g, оно не может конфликтовать ни с одним занятием из $X \setminus \{g\}$. Поэтому расписание $X' = X \cup \{f\} \setminus \{g\}$ тоже не содержит конфликтов. Оно имеет тот же размер, что и X, а значит, тоже максимально. □

Чтобы завершить доказательство, обратимся к нашему старому другу — индукции.

Теорема 4.4. Жадное расписание оптимально.

Доказательство. Пусть f — первое заканчивающееся занятие, а A — подмножество занятий, начинающихся после его окончания. Предыдущая лемма показывает, что некоторое оптимальное расписание содержит f, поэтому лучшее расписание, содержащее f, оптимально. Оно должно включать оптимальное расписание для занятий, не конфликтующих с f, то есть для A. Жадный алгоритм выбирает f, а затем, по предположению индукции, вычисляет оптимальное расписание для A. □

Доказательство может стать понятнее, если немного развернуть индукцию.

Доказательство. Пусть $\langle g_1, g_2, \ldots, g_k \rangle$ — последовательность занятий, выбранных жадным алгоритмом и отсортированных по времени начала. Предположим, имеется максимальное расписание без конфликтов

$$S = \langle g_1, g_2, \ldots, g_{j-1}, c_j, c_{j+1}, \ldots, c_m \rangle,$$

также отсортированное по времени начала, где c_j отличается от выбранного жадным алгоритмом g_j. Возможно, $j = 1$, и тогда расписание начинается с нежадного выбора c_1. По построению g_j не конфликтует с предыдущими $g_1, \ldots, g_{j-1}$; поскольку расписание S не содержит конфликтов, то же верно и для c_j. Более того, среди занятий, не конфликтующих с предыдущими, g_j заканчивается раньше всех; в частности, раньше c_j. Следовательно, g_j не конфликтует и с более поздними занятиями $c_{j+1}, \ldots, c_m$. Поэтому изменённое расписание

$$S' = \langle g_1, g_2, \ldots, g_{j-1}, g_j, c_{j+1}, \ldots, c_m \rangle$$

тоже не содержит конфликтов. Этот аргумент непосредственно обобщает лемму 4.3, рассматривающую $j = 1$.

По индукции существует оптимальное расписание $\langle g_1, g_2, \ldots, g_k, c_{k+1}, \ldots, c_m \rangle$, включающее все занятия, выбранные жадным алгоритмом. Но это невозможно, если $k \neq m$: если некоторое c_{k+1} не конфликтует с первыми k жадными занятиями, жадный алгоритм выбрал бы больше k занятий! □

[2] Но вам всё равно стоит самостоятельно разобрать подробности. Алгоритм динамического программирования может найти «лучшее» расписание при разных определениях слова «лучшее», а описанный здесь жадный алгоритм работает только тогда, когда «лучшее»

означает «самое большое». Кроме того, другое рекуррентное соотношение позволяет сократить время до $O(n^2)$.

4.3. Общая схема

Основная структура этого доказательства корректности в точности совпадает со структурой доказательства для сортировки ленты: индуктивный аргумент обмена.

- Предположите, что существует оптимальное решение, отличное от жадного.
- Найдите «первое» различие между ними.
- Докажите, что оптимальный выбор можно заменить жадным, не ухудшив решение, хотя обмен и не обязательно его улучшит.

По индукции отсюда следует, что некоторое оптимальное решение содержит всё жадное решение и потому совпадает с ним. Иногда, как в задаче о расписании, нужен дополнительный шаг: показать, что ни одно оптимальное решение не является строго лучше жадного.

4.4. Коды Хаффмана

Двоичный код сопоставляет каждому символу алфавита строку из нулей и единиц. Код называется *беспрефиксным*, если ни одно кодовое слово не является префиксом другого. Что несколько сбивает с толку, беспрефиксные коды часто называют также префиксными. Семибитный ASCII и UTF-8 из Unicode — беспрефиксные двоичные коды. Азбука Морзе — двоичный код с символами • и —, но она не беспрефиксна: код E (•) является префиксом кодов I (••), S (•••) и H (••••).[3]

Любой беспрефиксный двоичный код можно изобразить двоичным деревом, в листьях которого хранятся кодируемые символы. Кодовое слово символа задаётся путём от корня до соответствующего листа: `0` при движении влево, `1` — вправо. Поэтому длина кодового слова равна глубине соответствующего листа. Несмотря на внешнее сходство, двоичные кодовые деревья — не двоичные деревья поиска: порядок символов в листьях нас совершенно не интересует.

Пусть нужно закодировать сообщение алфавитом из n символов так, чтобы результат был как можно короче. Точнее, дан массив частот `f[1 .. n]`, и нужно вычислить беспрефиксный двоичный код, минимизирующий общую длину закодированного сообщения:

$$\sum_{i=1}^{n} f[i] \cdot \text{depth}(i).$$

Это та же функция стоимости, что и при оптимизации двоичных деревьев поиска, но задача оптимизации иная: в кодовых деревьях не требуется сохранять какой-либо порядок ключей.

[3] Поэтому азбуку Морзе, пожалуй, точнее описывать как беспрефиксный троичный код с тремя символами: •, — и пауза. Или можно рассматривать её как беспрефиксный двоичный код, где один такт звука/света/тока/высокого напряжения/дыма/газа обозначен ■, а один такт тишины/темноты/земли/низкого напряжения/воздуха/жидкости — □. Тогда каждая точка

кодируется как ■□, тире — как ■■■□, пауза — как □□. В стандартной азбуке Морзе после каждой буквы идёт одна пауза, а после каждого слова — ещё две дополнительные; однако □ в конце всего сообщения опускаются. Например, строка `MORSE CODE` однозначно кодируется следующей битовой строкой:



В 1951 году, будучи аспирантом MIT, Дэвид Хаффман разработал следующий жадный алгоритм построения оптимального кода:[4]

> ***Хаффман:*** *объединить два наименее частых символа и применить рекурсию.*

Лучше всего проиллюстрировать алгоритм на примере. Пусть требуется закодировать следующее полезно самоописательное предложение, найденное Ли Саллоусом:[5]

> *This sentence contains three a's, three c's, two d's, twenty-six e's, five f's, three g's, eight h's, thirteen i's, two l's, sixteen n's, nine o's, six r's, twenty-seven s's, twenty-two t's, two u's, five v's, eight w's, four x's, five y's, and only one z.*

Для простоты проигнорируем 44 пробела, 19 апострофов, 19 запятых, три дефиса и одну-единственную точку и закодируем только буквы, словно сообщение записано способом *scriptio continua*:

```
THISSENTENCECONTAINSTHREEASTHREECSTWODSTWENTYSIXESFIVEFST
HREEGSEIGHTHSTHIRTEENISTWOLSSIXTEENNSNINEOSSIXRSTWENTYSEV
ENSSTWENTYTWOTSTWOUSFIVEVSEIGHTWSFOURXSFIVEYSANDONLYONEZ
```

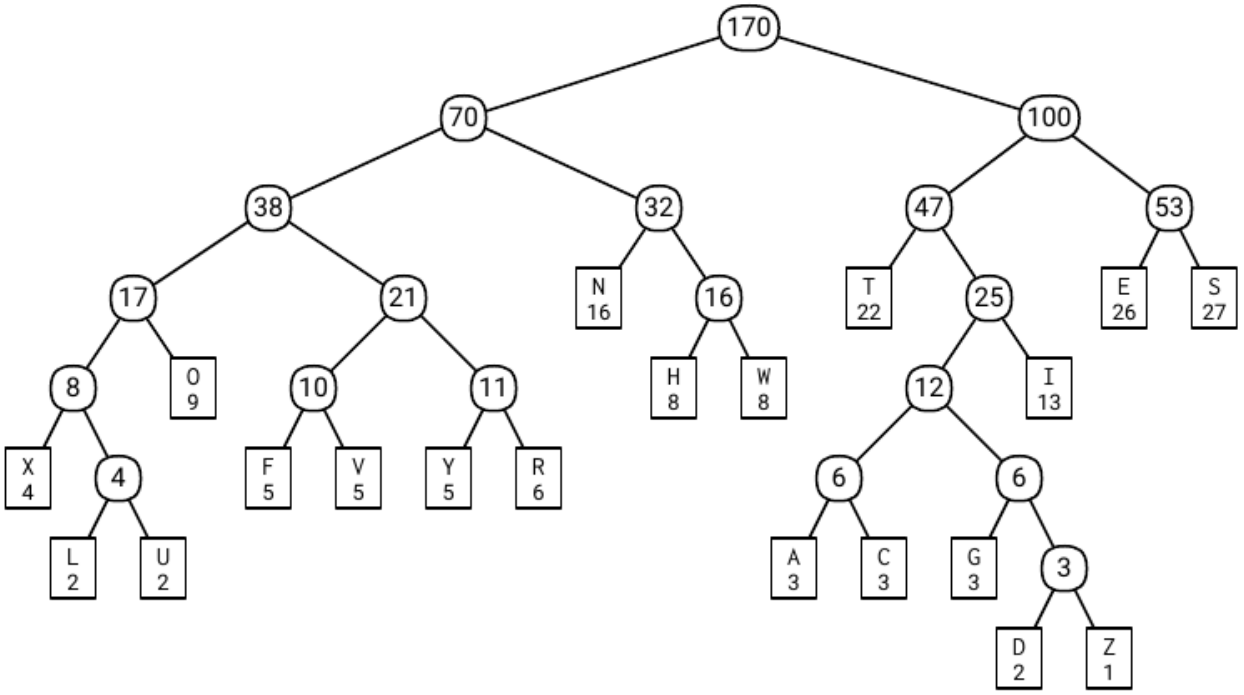
[6] Вот таблица частот предложения Саллоуса:

```
Символ  A C D  E F G H  I L  N O R  S  T U V W X Y Z
Частота 3 3 2 26 5 3 8 13 2 16 9 6 27 22 2 5 8 4 5 1
```

Алгоритм Хаффмана выбирает два самых редких символа, произвольно разрешая равенства, — в данном случае, скажем, Z и D — и объединяет в новый символ DZ частоты 3. Он становится внутренним узлом строящегося кодового дерева, а Z и D — его потомками; какой слева, неважно. Затем алгоритм рекурсивно строит код Хаффмана для новой таблицы:

```
Символ  A C  E F G H  I L  N O R  S  T U V W X Y DZ
Частота 3 3 26 5 3 8 13 2 16 9 6 27 22 2 5 8 4 5  3
```

После 19 объединений все 20 букв объединены. История объединений даёт кодовое дерево. Алгоритм неоднократно делает произвольный выбор, поэтому возможны разные коды Хаффмана. Один из них показан ниже; числа во внутренних узлах — частоты объединённых символов. Например, код A — `101000`, а код S — `111`.



Кодирование предложения Саллоуса этим кодом дало бы битовую строку, начинающуюся так:

```
T   H    I    S   S   E   N   T   E   N   C     E   C     O   N   T
100 0110 1011 111 111 110 010 100 110 010 101001 110 101001 0001 010 100 …
```

Далее приведена стоимость кодирования каждого символа и его вклад в общую длину:

```
Символ  A  C  D  E  F  G  H  I  L  N  O  R  S  T  U  V  W  X  Y Z
Частота 3  3  2 26  5  3  8 13  2 16  9  6 27 22  2  5  8  4  5 1
Глубина 6  6  7  3  5  6  4  4  6  3  4  5  3  3  6  5  4  5  5 7
Итого  18 18 14 78 25 18 32 52 12 48 36 30 81 66 12 25 32 20 25 7
```

Всего получается 649 бит. Разные коды Хаффмана представляют одни и те же символы по-разному, иногда словами разной длины, но общая длина сообщения одинакова для всех кодов Хаффмана: 649 бит.

Учитывая простоту алгоритма, удивительно, что он создаёт оптимальный беспрефиксный двоичный код.[7] Для кодирования предложения Саллоуса любым беспрефиксным кодом потребуется не менее 649 бит! К счастью, рекурсивная структура позволяет легко доказать утверждение аргументом обмена, похожим на предыдущие доказательства оптимальности. Сначала покажем, что самый первый выбор алгоритма корректен.

Лемма 4.5. Пусть x и y — два наиболее редких символа, причём равенства разрешаются произвольно. Существует оптимальное кодовое дерево, в котором x и y — соседние потомки одного узла.

Доказательство. На самом деле докажем более сильное утверждение: существует оптимальный код, в котором x и y — соседние потомки и имеют наибольшую глубину среди

всех листьев.

Пусть T — оптимальное кодовое дерево глубины d. Поскольку T — полное двоичное дерево, в нём есть не менее двух листьев на глубине d с общим родителем. Проверьте это индукцией! Предположим, что это не x и y, а другие символы a и b.

Пусть T' получается обменом x и a, а $\Delta = d - \mathrm{depth}_T(x)$. При обмене глубина x увеличивается на Δ, а глубина a на столько же уменьшается, поэтому

$$\mathrm{cost}(T') = \mathrm{cost}(T) + \Delta \cdot (f[x] - f[a]).$$

По предположению x — один из двух редчайших символов, а a — нет, поэтому $f[x] \le f[a]$; максимальная глубина a означает $\Delta \ge 0$. Следовательно, $\mathrm{cost}(T') \le \mathrm{cost}(T)$. Но T оптимально, так что должно также выполняться $\mathrm{cost}(T') \ge \mathrm{cost}(T)$. Значит, T' тоже оптимально.

Аналогично обмен y и b даёт ещё одно оптимальное дерево. В нём x и y имеют общего родителя и максимальную глубину, что и требовалось. $\square$

Теперь оптимальность гарантирует наша дорогая Фея Рекурсии! Аргумент основан на следующем нестандартном рекурсивном определении: полное двоичное дерево — это либо один узел, либо полное двоичное дерево, в котором некоторый лист заменён внутренним узлом с двумя листьями-потомками.

Теорема 4.6. Каждый код Хаффмана — оптимальный беспрефиксный двоичный код.

Доказательство. Если сообщение содержит только один или два различных символа, утверждение тривиально; предположим обратное.

Пусть `f[1 .. n]` — исходные частоты и, без ограничения общности, `f[1]` и `f[2]` — наименьшие. Для рекурсивной подзадачи определим `f[n + 1] = f[1] + f[2]`. Предыдущий аргумент обмена показывает, что `1` и `2` — соседние потомки на наибольшей глубине в некотором оптимальном коде для `f[1 .. n]`.

Пусть T' — дерево Хаффмана для `f[3 .. n + 1]`; по предположению индукции оно оптимально для меньшего набора частот. Чтобы получить итоговое дерево T, заменим лист $n + 1$ внутренним узлом с двумя потомками `1` и `2`. Утверждается, что T оптимально для исходного массива частот.

Выразим стоимость T через стоимость T'. В этих формулах `depth(i)` обозначает глубину листа i в T или T'; каждый лист, присутствующий в обоих деревьях, имеет одинаковую глубину:

$$
\begin{aligned}
\operatorname{cost}(T) &= \sum_{i=1}^{n} f[i] \cdot \operatorname{depth}(i) \\
&= \sum_{i=3}^{n+1} f[i] \cdot \operatorname{depth}(i) + f[1]\operatorname{depth}(1) + f[2]\operatorname{depth}(2) - f[n+1]\operatorname{depth}(n+1) \\
&= \operatorname{cost}(T') + (f[1] + f[2])\operatorname{depth}(T) - f[n+1](\operatorname{depth}(T) - 1) \\
&= \operatorname{cost}(T') + f[1] + f[2] + (f[1] + f[2] - f[n+1])(\operatorname{depth}(T) - 1) \\
&= \operatorname{cost}(T') + f[1] + f[2].
\end{aligned}
$$

Это равенство показывает, что минимизация стоимости T эквивалентна минимизации стоимости T'; в частности, присоединив листья 1 и 2 к листу $n+1$ в T', получаем оптимальное дерево для исходных частот. $\square$

> *Примечание переводчика: при произвольном выборе между равными частотами объединённый узел не обязан оказаться на максимальной глубине конкретного построенного дерева. В приведённом вычислении вместо* $\operatorname{depth}(T)$ *достаточно использовать фактическую общую глубину листьев 1 и 2; окончательное равенство сохраняется.*

Для эффективного построения кода храним символы в очереди с приоритетом по частоте. Дерево можно представить тремя массивами индексов: `Left` и `Right` для потомков, `Parent` для родителя. Листья итогового дерева имеют индексы от 1 до n, корень — $2n-1$. Псевдокод приведён на рисунке 4.4. `BuildHuffman` выполняет $O(n)$ операций очереди: ровно $2n-1$ вставок `Insert` и $2n-2$ извлечений минимума `ExtractMin`. В стандартной двоичной куче каждая операция требует $O(\log n)$, поэтому весь алгоритм работает за $O(n \log n)$.

```
BuildHuffman(f[1 .. n]):
    для i ← 1 до n
        L[i] ← 0; R[i] ← 0
        Insert(i, f[i])
    для i ← n до 2n − 1
        x ← ExtractMin()  ⟨⟨найти два самых редких символа⟩⟩
        y ← ExtractMin()
        f[i] ← f[x] + f[y]  ⟨⟨объединить в новый символ⟩⟩
        Insert(i, f[i])
        L[i] ← x; P[x] ← i  ⟨⟨обновить указатели дерева⟩⟩
        R[i] ← y; P[y] ← i
    P[2n − 1] ← 0
```

Рисунок 4.4. Построение кода Хаффмана.

> *Примечание переводчика: во втором цикле оригинала указан диапазон от n до $2n-1$, но при нумерации листьев $1, \dots, n$ новые узлы должны получать индексы от $n+1$ до $2n-1$. Это согласуется и с указанными автором $n-1$ объединениями.*

Наконец, простые алгоритмы кодирования и декодирования фиксированным кодом Хаффмана приведены на рисунке 4.5; оба работают за $O(m)$, где m — длина закодированного сообщения.

```
HuffmanEncode(A[1 .. k]):
    m ← 1
    для i ← 1 до k
        HuffmanEncodeOne(A[i])

HuffmanEncodeOne(x):
    если x < 2n − 1
        HuffmanEncodeOne(P[x])
        если x = L[P[x]]
            B[m] ← 0
        иначе
            B[m] ← 1
        m ← m + 1

HuffmanDecode(B[1 .. m]):
    k ← 1
    v ← 2n − 1
    для i ← 1 до m
        если B[i] = 0
            v ← L[v]
        иначе
            v ← R[v]
        если L[v] = 0
            A[k] ← v
            k ← k + 1
            v ← 2n − 1
```

Рисунок 4.5. Алгоритмы кодирования и декодирования с кодами Хаффмана.

[4] Хаффман учился на курсе теории информации Роберта Фано, близкого коллеги Клода Шеннона, отца теории информации. Фано и Шеннон ранее разработали другой жадный алгоритм построения префиксных кодов: разделить массив частот на две части как можно равномернее и рекурсивно построить код для каждой. Но было известно, что коды Фано — Шеннона не оптимальны. Фано предложил студентам найти оптимальный префиксный код. Хаффман решил взяться за задачу как за учебный проект вместо выпускного экзамена, не зная, что задача остаётся открытой и Фано с Шенноном уже безуспешно пытались её решить. После нескольких месяцев тщетных усилий он сдался и всё же решил идти на экзамен. Когда он выбрасывал заметки в корзину, на него снизошло решение. Позже Хаффман описал это озарение как «совершенную молнию внезапного понимания».

[5] Впервые об этом предложении сообщил Александр Дьюдни в октябрьской колонке «Computer Recreations» журнала *Scientific American* за 1984 год. Сам Саллоус в 1985 году опубликовал замечательную историю его открытия вместе с несколькими другими самоописательными предложениями; статья доступна на его сайте. Недовольный медленной работой программы на VAX 11/780, Саллоус спроектировал и построил специальное оборудование для полного перебора самоописательных предложений с разными вводными фразами: «Эта панграмма содержит…», «Это предложение содержит ровно…» и так далее. Тщательный теоретический анализ сократил пространство поиска до чуть более шести миллиардов вариантов, которые его машина для панграмм с частотой 1 МГц перебрала менее чем за два часа.

[6] …и он говорил сорок пять минут, и никто не понял ни слова из сказанного, но нам было весело заполнять бланки и играть карандашами на той скамейке.

[7] Это, безусловно, удивило и Хаффмана, и Фано!

4.5. Устойчивое паросочетание

Ежегодно тысячи начинающих врачей должны получить места интернатуры в больницах США. В первой половине XX века конкуренция за лучших врачей заставляла больницы предлагать места всё раньше — порой уже на втором курсе медицинского факультета — и устанавливать всё более короткие сроки ответа. В 1940-х медицинские школы договорились не выдавать информацию до общей даты на четвёртом курсе. В ответ больницы стали требовать решений ещё быстрее. К 1950 году они обычно звонили врачам, предлагали место и требовали немедленного ответа. Если первой звонила больница, занимавшая третье место в списке предпочтений, интерну приходилось рисковать: согласиться и, возможно, упустить лучший вариант или отказаться и, возможно, остаться совсем без места.[8]

Наконец, в начале 1950-х для распределения мест был создан центральный координационный орган, ныне называемый Национальной программой распределения резидентов (NRMP). Ежегодно врачи подают упорядоченный по предпочтению список больниц, где они согласны работать, а каждая больница — упорядоченный список врачей, которых согласна принять. Затем NRMP вычисляет *паросочетание* врачей и больниц, удовлетворяющее требованию устойчивости. Паросочетание неустойчиво, если существуют врач α и больница B, которые предпочли бы друг друга своим нынешним партнёрам:

- α распределена в другую больницу A, хотя предпочитает B;
- B получила другого врача β, хотя предпочитает α.

В таком случае (α, B) называется *неустойчивой парой*. Цель распределения — устойчивое паросочетание, то есть паросочетание без неустойчивых пар.

Для простоты далее будем считать, что врачей и больниц ровно поровну; каждая больница предлагает ровно одно место; каждый врач ранжирует все больницы и каждая больница — всех врачей; наконец, одинаковых мест в списках предпочтений нет.[9]

[8] На американском рынке академических вакансий приходится идти на похожий риск, по крайней мере в информатике. Одни кафедры начинают делать предложения в феврале и требуют ответа за две недели; другие приступают к собеседованиям лишь в марте; MIT, как известно, ждёт мая, когда все собеседования закончены, прежде чем делать предложения преподавателям. Излишне говорить, что разнобой дат предложений и сроков ответа вызывает огромный стресс у кандидатов и кафедр. По схожим причинам с 1965 года большинство американских университетов договорились об общем сроке 15 апреля для принятия будущими аспирантами предложений финансовой поддержки и, следовательно, зачисления.

[9] В действительности большинство больниц предлагают несколько мест, врачи и больницы ранжируют лишь часть возможных партнёров, а мест обычно больше, чем заинтересованных врачей. А дальше всё становится сложнее.

Несколько плохих идей

На первый взгляд даже не ясно, всегда ли существует устойчивое паросочетание! Конечно, не всякое распределение врачей по больницам устойчиво. Пусть есть три врача — доктор Куинси, доктор Ротванг и доктор Шепард, обозначенные строчными буквами, — и три больницы: лечебница Аркхем, Королевская больница Бетлем и городская больница, обозначенные прописными буквами. Их списки предпочтений таковы:

```
Врачи            Больницы
q  r  s          A  B  C
A  C  A          r  s  q
C  A  B          q  q  r
B  B  C          s  r  s
```

Паросочетание $\{Aq, Br, Cs\}$ неустойчиво: Аркхем предпочла бы нанять Ротванг вместо Куинси, а Ротванг предпочла бы работать в Аркхеме, а не в Бетлеме. Пара (A, r) неустойчива.

Можно вообразить постепенный алгоритм, начинающий с произвольного паросочетания и затем жадно выполняющий обмены для устранения неустойчивостей. К сожалению, устранение одной может создать новые; более того, такое «улучшение» может привести к бесконечному циклу. Например, начиная с прежнего паросочетания $\{Aq, Br, Cs\}$, каждый из следующих обменов устраняет одну неустойчивую пару, указанную над стрелкой, но вся последовательность возвращает нас к началу:[10]

$$\{Aq, Br, Cs\} \xrightarrow{Ar} \{Ar, Bq, Cs\} \xrightarrow{Cr} \{As, Bq, Cr\} \xrightarrow{Cq} \{As, Br, Cq\} \xrightarrow{Aq} \{Aq, Br, Cs\}.$$

Другой возможный жадный протокол состоит из раундов. В каждом раунде каждая больница без пары предлагает место любимому врачу из ещё свободных, затем каждый свободный врач с предложением принимает лучшее из них. Нетрудно доказать, что в каждом раунде появляется хотя бы одна новая пара, поэтому алгоритм всегда завершается полным распределением. Для предыдущего примера уже после первого раунда получаем устойчивое паросочетание $\{Ar, Bs, Cq\}$! Но рассмотрим другой вход:

```
Врачи            Больницы
q  r  s          A  B  C
C  A  A          q  q  s
B  C  B          s  r  r
A  B  C          r  s  q
```

В первом раунде Шепард принимает предложение городской больницы, а Куинси — Бетлема, отклоняя предложение Аркхема. Без пары остаются только Ротванг и Аркхем. После двух раундов протокол возвращает $\{Ar, Bq, Cs\}$. К сожалению, оно неустойчиво: Аркхем и Шепард предпочитают друг друга своим партнёрам.

[10] Этот пример нашёл Дональд Кнут.

Алгоритмы Boston Pool и Гейла — Шепли

В 1952 году NRMP приняла алгоритм *Boston Pool* для распределения интернов. Он получил имя потому, что ранее применялся региональным координационным центром в районе Бостона. Десять лет спустя Дэвид Гейл и Ллойд Шепли описали и формально проанализировали обобщение этого алгоритма, доказав, что оно вычисляет устойчивое паросочетание. Гейл и Шепли использовали метафору приёма в колледжи. По существу тот же алгоритм в 1972 году независимо разработал Эллиотт Перансон для приёма в медицинские школы. С тех пор аналогичные алгоритмы применяются на многих рынках распределения: при найме преподавателей во Франции и новых докторов экономики в США, поступлении в университеты Германии и государственные школы Нью-Йорка и Бостона, распределении должностей моряков ВМС США и подборе доноров почек.

В 2012 году Шепли получил Нобелевскую премию по экономике за исследования устойчивых паросочетаний совместно с Элвином Ротом, который значительно расширил его работу и применил её к реальным системам распределения. Гейл премию не получил, поскольку умер в 2008 году.

Подобно нашему последнему неудачному жадному алгоритму, алгоритм Гейла — Шепли действует по раундам, пока не будут приняты все предложения о местах. Каждый раунд состоит из двух этапов:

1. Произвольная больница A без пары предлагает место лучшему по своему списку врачу α, который ещё не отверг её.
2. Если у α нет пары, она предварительно принимает предложение A. Если пара уже есть, но она предпочитает A, она отвергает прежнего партнёра и предварительно принимает новое предложение. Иначе она отклоняет новое предложение.

В конечном счёте каждый врач принимает лучшее предложение из полученных согласно своему списку.[11] Иными словами, больницы жадно предлагают места, а врачи жадно принимают предложения. Возможность врача отвергнуть текущую пару ради лучшего предложения — ключ к успеху этой взаимной жадной стратегии.

Например, пусть есть четыре врача — Куинси, Ротванг, Шепард и Там — и четыре больницы: Аркхем, Бетлем, городская больница и «Инициатива Дхарма». Списки предпочтений:

```
Врачи               Больницы
q  r  s  t          A  B  C  D
A  A  B  D          t  r  t  s
B  D  A  B          s  t  r  r
C  C  C  C          r  q  s  q
D  B  D  A          q  s  q  t
```

Для них алгоритм может действовать так:

1. Аркхем предлагает место Там.
2. Бетлем предлагает место Ротванг.
3. Городская больница предлагает место Там; та отвергает прежнее предложение Аркхема.

4. Дхарма предлагает место Шепард. С этого момента только одна больница остаётся без пары, поэтому у алгоритма больше нет выбора.
5. Аркхем предлагает место Шепард; та отвергает прежнее предложение Дхармы.
6. Дхарма предлагает место Ротванг; та отвергает прежнее предложение Бетлема.
7. Бетлем предлагает место Там; та отвергает прежнее предложение городской больницы.
8. Городская больница предлагает место Ротванг; та отказывает.
9. Городская больница предлагает место Шепард; та отказывает.
10. Городская больница предлагает место Куинси.

После десятого раунда все оставшиеся предложения приняты, и алгоритм возвращает $\{As, Bt, Cq, Dr\}$. Можно и следует проверить полным перебором, что это паросочетание устойчиво, хотя ни один врач не попал в свою любимую больницу и ни одна больница не наняла любимого врача; более того, городской больнице достался наименее предпочтительный врач. Это не единственное устойчивое паросочетание: $\{Ar, Bs, Cq, Dt\}$ также устойчиво.

[11] Алгоритм Boston Pool 1952 года — частный случай алгоритма Гейла — Шепли, в котором предложения делаются в определённом порядке. Грубо говоря, каждый раз предлагает больница X, чей любимый из ещё не отвергнувших её врач ставит X выше всех. Поскольку порядок зависит от всех списков, алгоритм должен выполняться центральным органом. Напротив, алгоритму Гейла — Шепли даже не требуется, чтобы участники заранее знали собственные предпочтения, пока они ведут себя согласованно с некоторым фиксированным порядком.

Время работы

Проанализировать количество предложений сравнительно просто, поэтому начнём с него. Каждая больница делает каждому врачу не более одного предложения; значит, всего их не более n^2.

Но для анализа фактического времени нужно подробнее определить алгоритм. Как ему передаются списки предпочтений? Как он определяет наличие больницы без пары и находит её? Как хранит предварительные пары? Как выясняет, предпочитает ли врач новое предложение текущему? И фундаментальный вопрос: как вообще представляются врачи и больницы?

Один вариант — представить каждого врача и каждую больницу уникальным целым числом от `1` до n, а предпочтения — массивами `Dpref[1 .. n, 1 .. n]` и `Hpref[1 .. n, 1 .. n]`, где `Dpref[i, r]` — r-я больница в списке врача i, а `Hpref[j, r]` — r-й врач в списке больницы j. При таком входе алгоритм Boston Pool может после предварительной обработки выполнять каждое предложение за постоянное время; общая реализация работает за $O(n^2)$. Остальные детали оставляем как прямолинейное упражнение.

Несколько более сложное упражнение — доказать существование входов и порядков предложений, при которых до остановки алгоритма требуется $\Omega(n^2)$ предложений. Таким образом, верхняя оценка $O(n^2)$ для худшего случая точна.

Корректность

Но почему алгоритм вообще корректен? Откуда мы знаем, что он всегда вычисляет устойчивое или хотя бы полное паросочетание?

Получив предложение, врач навсегда сохраняет хотя бы предварительную пару. Эквивалентно: если некоторый врач без пары, ни одна больница ещё не предлагала ему работу; значит, больницы не исчерпали свои списки. Поэтому при завершении алгоритма, не позднее чем через n^2 раундов, у каждого врача есть пара и каждое место занято. Иными словами, алгоритм всегда вычисляет совершенное паросочетание врачей и больниц. Фух! Осталось доказать устойчивость.

Пусть алгоритм назначил врача α в больницу A, хотя она предпочитает B. Поскольку каждый врач принимает лучшее полученное предложение, α не получила предложения, которое ей понравилось бы больше A; в частности, B ей не предлагала. С другой стороны, B предложила место каждому врачу, которого предпочитает своему итоговому партнёру β. Следовательно, B предпочитает β врачу α, а значит, (α, B) не является неустойчивой парой. Таких пар нет, поэтому паросочетание устойчиво!

Оптимальность!

Удивительно, но корректность алгоритма Гейла — Шепли не зависит от того, какая больница делает предложение в каждом раунде. Более того, каков бы ни был порядок выбора свободных больниц, алгоритм всегда вычисляет одно и то же паросочетание! Назовём врача α *допустимым* для больницы A, если существует устойчивое паросочетание, в котором α назначена в A.

Лемма 4.7. Во время алгоритма Гейла — Шепли каждая больница A получает отказы только от врачей, недопустимых для A.

Доказательство. Индукция по числу раундов. Рассмотрим произвольный раунд, в котором врач α отказывает больнице A ради B. Отказ означает, что α предпочитает B больнице A. Все врачи, стоящие в списке B выше α, уже отказали B ранее и, по предположению индукции, недопустимы для B.

Теперь рассмотрим произвольное паросочетание тех же врачей и больниц, назначающее α в A. Мы уже знаем, что α предпочитает B. Если B предпочитает α своему партнёру, паросочетание неустойчиво. Если же B предпочитает своего партнёра врачу α, то по предыдущему рассуждению этот партнёр недопустим для B, и паросочетание снова неустойчиво. Следовательно, устойчивого паросочетания с назначением α в A не существует. □

Пусть `best(A)` — наиболее предпочтительный допустимый врач из списка A. Лемма 4.7 показывает: каждый врач, которого A предпочитает итоговому партнёру, недопустим для A. С другой стороны, итоговое паросочетание устойчиво, поэтому назначенный врач допустим. Немедленно получаем:

Следствие 4.8. Алгоритм Гейла — Шепли назначает `best(A)` в A для каждой больницы A.

Иными словами, алгоритм вычисляет наилучшее возможное устойчивое паросочетание с точки зрения больниц. Оказывается, с точки зрения врачей оно одновременно наихудшее! Пусть `worst(α)` — наименее предпочтительная допустимая больница из списка врача α.

Следствие 4.9. Алгоритм Гейла — Шепли назначает α в `worst(α)` для каждого врача α.

Доказательство. Пусть алгоритм назначил α в A; нужно показать, что $A = \text{worst}(\alpha)$. Рассмотрим произвольное устойчивое паросочетание, в котором A соединена не с α, а с другим врачом β. Предыдущее следствие показывает, что A предпочитает $\alpha = \text{best}(A)$ врачу β. Поскольку паросочетание устойчиво, α должна предпочитать назначенную ей больницу больнице A. Аргумент работает для любого устойчивого паросочетания, поэтому любой другой допустимый вариант для α предпочтительнее A. Иными словами, $A = \text{worst}(\alpha)$. □

Тонкое следствие этих двух утверждений, обнаруженное Лестером Дубинсом и Дэвидом Фридманом в 1981 году: врач потенциально может улучшить своё распределение, солгав о предпочтениях, а больница — нет. Однако группа больниц может сговориться, чтобы улучшить некоторые свои пары. Отчасти по этой причине в 1998 году NRMP поменяла направление алгоритма: теперь будущие резиденты предлагают больницам свои услуги согласно своим предпочтениям, а каждая больница принимает лучшее предложение. Новый алгоритм тем самым вычисляет наилучшее возможное устойчивое паросочетание для врачей и наихудшее для больниц. На практике, однако, изменение затронуло менее 1% распределений. Насколько мне известно, точное влияние этого изменения на пациентов остаётся открытой проблемой.

Упражнения

Предостережение читателю. Некоторые упражнения нельзя решить жадными алгоритмами! Всякий раз, описывая и анализируя жадный алгоритм, необходимо также доказать его корректность; обычно это доказательство принимает форму аргумента обмена. Такие доказательства особенно важны на занятиях, подобных моим, где обычно не требуется доказывать корректность.

1. Описанный выше `GreedySchedule` — не единственная возможная жадная стратегия составления расписания. Для каждого из следующих вариантов либо докажите, что алгоритм всегда строит оптимальное расписание, либо приведите небольшой вход, на котором результат не оптимален. Считайте, что все алгоритмы разрешают равенства произвольно, то есть совершенно вне вашего контроля. [Подсказка: три из этих алгоритмов действительно корректны.]

(a) Выбрать занятие x, заканчивающееся последним, отбросить конфликтующие с x и применить рекурсию.

(b) Выбрать занятие x, начинающееся первым, отбросить все конфликтующие с ним и применить рекурсию.

(c) Выбрать занятие x, начинающееся последним, отбросить все конфликтующие с ним и применить рекурсию.

(d) Выбрать самое короткое занятие x, отбросить все конфликтующие с ним и применить рекурсию.

(e) Выбрать занятие x, конфликтующее с наименьшим числом других занятий, отбросить конфликтующие с ним и применить рекурсию.

(f) Если конфликтов нет, выбрать все занятия. Иначе отбросить самое длинное занятие и применить рекурсию.

(g) Если конфликтов нет, выбрать все занятия. Иначе отбросить занятие, конфликтующее с наибольшим числом других, и применить рекурсию.

(h) Пусть x начинается раньше всех, а y — вторым по времени начала.

- Если x и y не пересекаются, выбрать x и рекурсивно обработать остальные.
- Если x полностью содержит y, отбросить x и применить рекурсию.
- Иначе отбросить y и применить рекурсию.

(i) Если какое-то занятие x полностью содержит другое, отбросить x и применить рекурсию. Иначе выбрать занятие y, заканчивающееся последним, отбросить конфликтующие с ним и применить рекурсию.

2. Теперь рассмотрим взвешенный вариант задачи о расписании: разные занятия дают разное число зачётных часов, совершенно не связанное с продолжительностью. Даны массивы времени начала, окончания и зачётных часов; нужно выбрать набор непересекающихся занятий с максимальным суммарным числом часов.

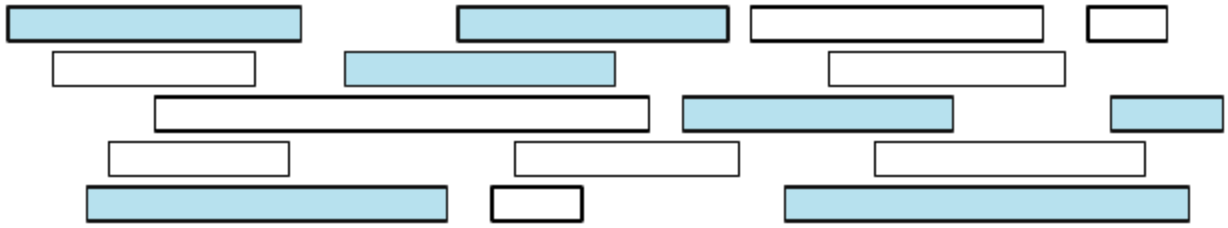
(a) Докажите, что жадный алгоритм начала главы — выбрать первое заканчивающееся занятие и применить рекурсию — не всегда возвращает оптимальное расписание.

(b) Докажите, что ни один из жадных алгоритмов упражнения 1 не всегда возвращает оптимальное расписание. [Подсказка: сначала решите упражнение 1; алгоритмы, не работающие там, не работают и здесь.]

(c) Опишите и проанализируйте алгоритм, который всегда находит оптимальное расписание. [Подсказка: он не будет жадным.]

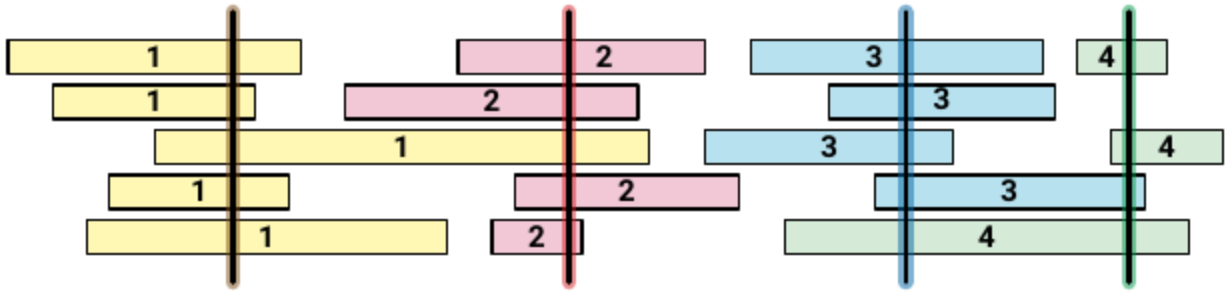
3. Пусть X — множество из n интервалов на числовой прямой. Подмножество $Y \subseteq X$ *покрывает* X, если объединение интервалов Y равно объединению всех интервалов X. Размер покрытия — число интервалов.

Опишите и проанализируйте эффективный алгоритм вычисления наименьшего покрытия X. На вход даны массивы `L[1 .. n]` и `R[1 .. n]` левых и правых концов интервалов. Если используете жадный алгоритм, докажите его корректность.



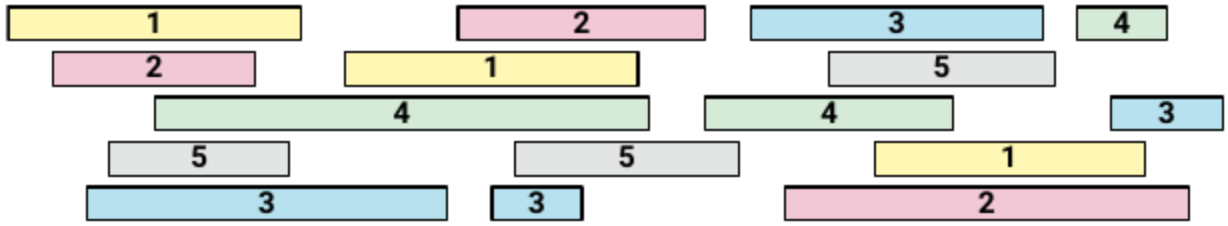
Множество интервалов с покрытием размера 7, показанным заливкой.

4. Пусть X — множество из n интервалов на прямой. Множество точек P *прокалывает* X, если каждый интервал X содержит хотя бы одну точку P. Опишите и проанализируйте эффективный алгоритм вычисления наименьшего такого множества точек. На вход даны массивы `L[1 .. n]` и `R[1 .. n]` левых и правых концов интервалов. Как обычно, если используете жадный алгоритм, докажите корректность.



Множество интервалов, проколотое четырьмя точками, показанными вертикальными отрезками.

5. Пусть X — множество из n интервалов на прямой. *Правильная раскраска* назначает каждому интервалу цвет так, чтобы любые два пересекающихся интервала имели разные цвета. Опишите и проанализируйте эффективный алгоритм вычисления минимального числа цветов для правильной раскраски X. На вход даны массивы `L[1 .. n]` и `R[1 .. n]` левых и правых концов интервалов. Если используете жадный алгоритм, докажите корректность.



Правильная раскраска множества интервалов пятью цветами.

6. (a) Для каждого целого n найдите массив частот `f[1 .. n]`, дерево кода Хаффмана которого имеет глубину $n - 1$, причём наибольшая частота минимальна.

(b) Пусть общая длина N незакодированного сообщения ограничена полиномом от размера алфавита n. Докажите, что любое дерево Хаффмана для частот `f[1 .. n]` имеет глубину

$O(\log n)$.

♥ **7.** Назовём массив частот `f[1 .. n]` *α-тяжёлым*, если выполнены два условия:

- $f[1] > f[i]$ для всех $i > 1$: символ 1 единственный наиболее частый;
- $f[1] \geq \alpha \sum_{i=1}^{n} f[i]$: не менее доли α всех символов равны 1.

Найдите наибольшее действительное α, такое что в каждом коде Хаффмана для каждого α -тяжёлого массива символ 1 представлен одним битом. [Подсказка: сначала докажите, что $1/3 \leq \alpha \leq 1/2$.]

8. Опишите и проанализируйте алгоритм построения оптимального троичного беспрефиксного кода для заданного массива частот `f[1 .. n]`. Не забудьте доказать корректность для всех n.

9. Подробно опишите реализацию алгоритма устойчивого паросочетания Гейла — Шепли, работающую за $O(n^2)$ в худшем случае, как утверждалось выше.

10. (a) Докажите, что алгоритм Гейла — Шепли может сделать $\Omega(n^2)$ предложений до завершения. Нужно описать подходящий вход и последовательность из $\Omega(n^2)$ допустимых предложений.

(b) Для любого целого n опишите предпочтения n врачей и n больниц, вынуждающие алгоритм выполнять $\Omega(n^2)$ раундов независимо от выбора допустимого предложения в каждом раунде. [Подсказка: пункт (b) влечёт пункт (a).]

11. Опишите и проанализируйте эффективный алгоритм, определяющий, имеет ли заданный набор предпочтений больниц и врачей единственное устойчивое паросочетание.

12. Рассмотрим обобщение задачи, в котором некоторые врачи ранжируют не все больницы и некоторые больницы — не всех врачей, а врач может быть назначен в больницу, только если они входят в списки друг друга. Дополнительно возможны три неустойчивые ситуации:

- больница с партнёром предпочитает свободного врача своему партнёру;
- врач с партнёром предпочитает свободную больницу своей;
- свободные врач и больница присутствуют в списках друг друга.

Устойчивое паросочетание может оставлять врачей и/или больницы без пары, даже когда их списки непусты. Например, если каждый врач указывает только Гарвард как приемлемую больницу, а каждая больница — только доктора Хауса как приемлемого интерна, пару составят лишь Хаус и Гарвард.

Опишите и проанализируйте эффективный алгоритм нахождения устойчивого паросочетания в этом общем случае. [Подсказка: сведите задачу к случаю, где каждый врач ранжирует все больницы и наоборот, а затем вызовите алгоритм Гейла — Шепли.]

13. Скандинавская мебельная компания Fürni наняла n водителей, чтобы доставить n одинаковых заказов по n адресам в Уилмингтоне, штат Делавэр. У каждого водителя есть собственный устоявшийся маршрут по городу, проходящий через все n адресов. Если они едут

привычными маршрутами, никакие два водителя не посещают один и тот же адрес одновременно.

В принципе любой водитель может доставить мебель по любому адресу, но есть осложнение. Один из водителей тайно установил датчики приближения и взрывчатку на диваны Johannshamn с зелёным полосатым узором Strinne. Если два дивана окажутся по одному адресу одновременно, оба взорвутся, уничтожив грузовик и здание. Это может произойти, только если один водитель доставил заказ, а затем другой посетил тот же адрес с мебелью в грузовике.

Ваша задача как диспетчера Fürni — назначить каждому водителю адрес доставки. Опишите алгоритм, обеспечивающий доставку по всем n адресам без взрывов.

Например, Джек посещает 537 Paper Street в 18:00 и 1888 Franklin Street в 20:00, а Марла — 537 Paper в 19:00 и 1888 Franklin в 21:00. Тогда Джек должен доставить заказ на Franklin, а Марла — на Paper; иначе на Franklin в 20:00 произойдёт взрыв. Звучит Pixies. [Подсказка: Джек и Марла несколько неустойчивы.]

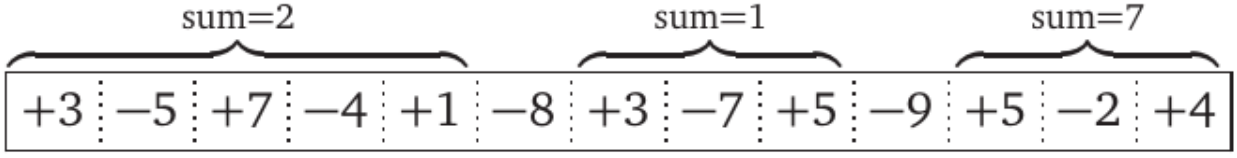
14. Вы — простой лавочник в стране с n видами монет достоинством $1 = c[1] < c[2] < \cdots < c[n]$. Например, в США $n = 6$, а номиналы — 1, 5, 10, 25, 50 и 100 центов. Любимый и великодушный диктатор Эль Генералиссимус постановил выдавать сдачу минимальным числом монет, чтобы не стирать его изображение, любовно выгравированное слугами королевской казны на каждой монете.

(a) В США простой жадный алгоритм всегда даёт минимальное число монет: вычесть максимальный возможный номинал и рекурсивно выдать остаток. Эль Генералиссимус не одобряет американскую капиталистическую жадность. Покажите систему номиналов, для которой жадный алгоритм не всегда выдаёт минимальное число монет.

(b) Пусть теперь Эль Генералиссимус ввёл систему с номиналами $b^0, b^1, b^2, \ldots, b^k$ для целого $b \geq 2$. Докажите, что, несмотря на неодобрение, жадный алгоритм из пункта (a) выдаёт оптимальную сдачу.

(c) Опишите и проанализируйте эффективный алгоритм, вычисляющий по сумме T и отсортированному массиву номиналов `c[1 .. n]` минимальное число монет для сдачи T центов. Предположим $c[1] = 1$, чтобы любую сумму можно было выдать.

15. Дан массив целых чисел `A[1 .. n]`, каждое из которых может быть положительным, отрицательным или нулём. Непрерывный подмассив `A[i .. j]` называется *положительным интервалом*, если сумма его элементов положительна. Опишите и проанализируйте алгоритм, вычисляющий минимальное число положительных интервалов, покрывающих каждый положительный элемент A. Например, для следующего массива ответ должен быть 3. Если все элементы отрицательны, ответ должен быть 0.



```
+3 -5 +7 -4 +1 -8 +3 -7 +5 -9 +5 -2 +4
```

16. Рассмотрим процесс: имеется одно положительное целое x, изначально равное 1. На каждом шаге можно увеличить x на единицу или удвоить его. Цель — получить заданное n. Например, число 10 получается за четыре шага:

$$1 \xrightarrow{+1} 2 \xrightarrow{\times 2} 4 \xrightarrow{+1} 5 \xrightarrow{\times 2} 10.$$

Очевидно, любое n можно получить ровно за $n - 1$ прибавлений единицы, но почти для всех n это ужасно неэффективно. Опишите и проанализируйте алгоритм вычисления минимального числа шагов для заданного n.

17. Даны n лыжников с ростом в массиве `P[1 .. n]` и n лыж с длинами в `S[1 .. n]`. Опишите эффективный алгоритм назначения одной лыжи каждому лыжнику так, чтобы средняя разница роста лыжника и длины назначенной лыжи была минимальна. Алгоритм должен вычислить перестановку σ, минимизирующую

$$\frac{1}{n} \sum_{i=1}^{n} \left| P[i] - S[\sigma(i)] \right|.$$

18. Алиса хочет устроить вечеринку и решает, кого пригласить. У неё есть n кандидатов, и она знает, какие пары знакомы. Она хочет пригласить как можно больше людей при двух условиях:

- у каждого гостя должно быть не менее пяти других гостей, с которыми он уже знаком;
- у каждого гостя должно быть не менее пяти других гостей, с которыми он ещё не знаком.

Опишите и проанализируйте алгоритм вычисления максимального числа гостей по списку n людей и списку знакомых пар.

19. Даны два массива положительных целых чисел `C[1 .. n]` и `R[1 .. n]`. Матрица $n \times n$ из нулей и единиц *согласуется* с R и C, если для каждого i строка i содержит `R[i]` единиц, а столбец i — `C[i]` единиц. Опишите и проанализируйте алгоритм, который строит согласованную матрицу или правильно сообщает, что она не существует.

20. Вы только что получили работу у Илона Маска: доставлять буррито из Сан-Франциско в Нью-Йорк. Вы поведёте автомобиль доставки буррито по новому трансконтинентальному подземному тоннелю доставки буррито Илона, проложенному по прямой между этими городами.[12]

Автомобиль работает на одноразовых батареях, которые нужно заменять не реже чем через 100 миль. Само топливо почти бесплатно, но батареи дорогие и хрупкие, поэтому устанавливать их могут только официальные члены профсоюза техников по замене батарей транспортных средств трансконтинентальной подземной доставки буррито.[13] Поэтому даже при досрочной замене нужно полностью оплатить установку новой батареи. Кроме того, автомобиль слишком мал, чтобы возить одновременно больше одной батареи.

Вдоль тоннеля расположены несколько заправочных станций; каждая берёт свою цену за установку новой батареи. Перед поездкой вы тщательно распечатали страницу Википедии с

расположением и ценами всех станций тоннеля. Как по этой информации выбрать лучшие места остановки?

Формально даны массивы `D[1 .. n]` и `C[1 .. n]`, где `D[i]` — расстояние от начала тоннеля до станции i, а `C[i]` — стоимость замены батареи на ней. Предположим, поездка начинается и заканчивается на станциях: `D[1] = 0`, а `D[n]` — полная длина пути. Автомобиль начинает с разряженной батареей, поэтому новую нужно установить на станции `1`.

(a) Опишите и проанализируйте жадный алгоритм нахождения минимального числа остановок для завершения поездки. Не забудьте доказать корректность.

(b) На самом деле нужно минимизировать общую стоимость поездки. Покажите, что жадный алгоритм пункта (a), распространённый на этот случай, не даёт оптимального решения.

(c) Опишите эффективный алгоритм вычисления станций, на которых следует остановиться, чтобы минимизировать общую стоимость поездки.

[12] ...который явно построен по образцу вымышленного «тоннеля буррито Аламеда — Уихокен» Мацея Цегловского.

[13] Или, как они называют себя по-немецки, *Die Transkontinentaluntergrundburritolieferfahrzeugbatteriewechseltechnikervereinigung*.

21. Вас наняли расставить последовательность из n книг по полкам библиотеки. Порядок книг фиксирован каталогом и не может меняться; на каждой полке должен находиться непрерывный отрезок последовательности. Даны массивы `H[1 .. n]` и `T[1 .. n]`, где `H[i]` и `T[i]` — высота и толщина книги i. Все полки одинаковой длины L; суммарная толщина книг на одной полке не может превышать L.

(a) Пусть высота всех книг одинакова и равна h, а высота полок больше h, поэтому любая книга помещается на любую полку. Опишите и проанализируйте жадный алгоритм размещения книг на минимальном числе полок. [Подсказка: алгоритм очевиден, но почему он корректен?]

(b) Разминка была приятной, а теперь настоящая задача. Книги имеют разную высоту, но высоту каждой полки можно подогнать под самую высокую книгу на ней. В частности, высоту любой пустой полки можно сделать нулевой. Требуется разместить книги с минимальной суммой высот полок. Покажите, что жадный алгоритм пункта (a) не всегда даёт лучшее решение.

(c) Опишите и проанализируйте алгоритм наилучшего распределения книг по полкам, как в пункте (b).

22. Строка w из круглых скобок `(` и `)` *сбалансирована*, если выполняется одно из условий:

- w — пустая строка;
- $w = (x)$ для некоторой сбалансированной строки x;
- $w = xy$ для некоторых сбалансированных строк x и y.

Например, строка `w = (((())()))(()())()` сбалансирована, поскольку $w = xy$, где

> `x = ((())()())` и `y = (()())()`.

(a) Опишите и проанализируйте алгоритм проверки сбалансированности заданной строки скобок.

(b) Опишите и проанализируйте жадный алгоритм вычисления длины наибольшей сбалансированной подпоследовательности заданной строки скобок. Как обычно, докажите его корректность.

Для обеих задач вход — массив `w[1 .. n]`, где каждый `w[i]` равен `(` или `)`. Оба алгоритма должны работать за $O(n)$.

23. Однажды Алекс устала лазать в зале и решила вывести большую группу друзей-скалолазов на природу. Они отправились к широкому невысокому валуну с несколькими отмеченными зацепами для рук и ног. Алекс быстро определила «разрешённые» движения между зацепами, доступные её друзьям.

Всю систему можно описать корневым деревом T с n вершинами: вершина соответствует зацепу, ребро — разрешённому движению между зацепами. Маршруты сходятся по мере подъёма к единственному зацепу на вершине, который соответствует корню T.

Алекс и её друзья — все отличные скалолазы — решили сыграть: на валуне одновременно находится как можно больше скалолазов, каждый должен выполнить ровно k движений. Начинать можно с любого зацепа, но все движения направлены от земли. Поэтому каждый проходит путь из k рёбер дерева T, направленный к корню. При этом никакие два скалолаза не могут касаться одного зацепа: пути не должны пересекаться.

(a) Опишите и проанализируйте жадный алгоритм вычисления максимального числа игроков. На вход даны корневое дерево T и целое k; алгоритм должен найти наибольшее число непересекающихся путей длины k. Не предполагайте, что дерево двоичное. Например, для дерева ниже ответ должен быть 8.

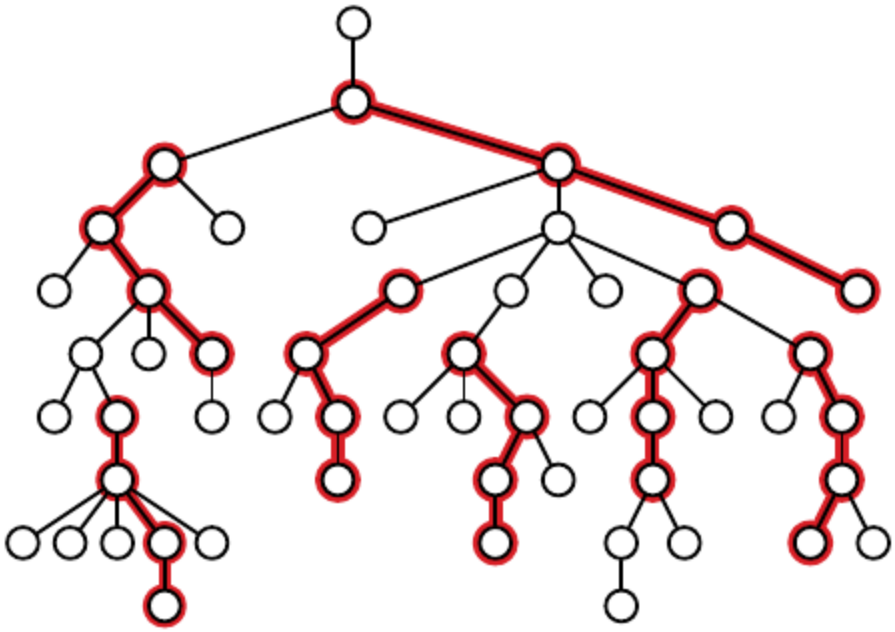


Рисунок 4.6. Семь непересекающихся путей длины $k = 3$. Это не наибольшее такое множество путей в данном дереве.

(b) Пусть теперь каждой вершине T соответствует награда, а цель — максимизировать суммарную награду вершин путей, а не число путей. Покажите, что жадный алгоритм не всегда даёт оптимальную награду.

(c) Опишите эффективный алгоритм вычисления максимальной награды из пункта (b).

24. Поздравляем! Вы успешно завоевали Камелот, превратив бывшую покрытую боевыми шрамами наследственную монархию в анархо-синдикалистскую коммуну, где граждане по очереди исполняют обязанности своего рода исполнительного секретаря недели, но все его решения утверждаются на специальном собрании раз в две недели простым большинством по чисто внутренним делам и большинством в две трети по более важным…

В качестве последнего символического акта вы приказываете разделить Круглый стол — на удивление, действительно круглый — на клинья, как пиццу, и раздать их гражданам Камелота в качестве трофеев. Каждый гражданин подал запрос на определённый угловой сектор, заданный двумя углами. Например, храбрый сэр Робин может запросить сектор от $17{,}23^\circ$ до 42°, а чистый сэр Ланселот — сектор в 2° от 359° до 1°. Гражданин будет счастлив тогда и только тогда, когда получит ровно запрошенный сектор. К сожалению, некоторые сектора пересекаются, поэтому удовлетворить всех невозможно. Добро пожаловать в политику.

Опишите и проанализируйте алгоритм нахождения максимального числа запросов, которые можно удовлетворить. [Подсказка: результат не должен меняться при повороте стола. Не предполагайте, что углы целые.]

25. Вы стоите в поле в окружении нескольких больших воздушных шаров. Нужно лопнуть все шары новым устройством Acme Zap-O-Matic™, не сходя с места. Оно стреляет мощным лазерным лучом, который лопает все задетые шары. Поскольку на каждый выстрел требуется

достаточно энергии, чтобы год обеспечивать небольшую страну, стрелять хочется как можно реже.

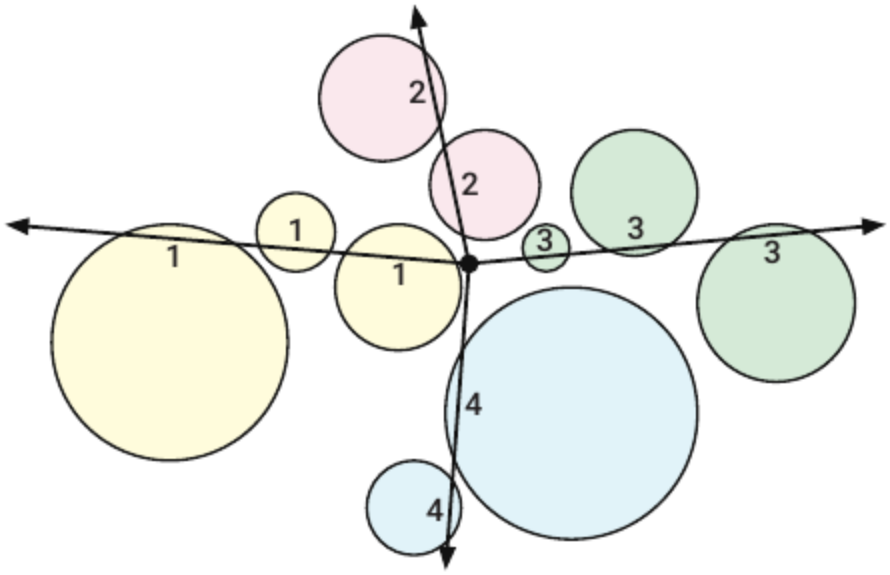


Рисунок 4.7. Девять шаров, лопнувших от четырёх выстрелов Zap-O-Matic™.

Формально задача минимального числа выстрелов выглядит так. Дано множество C из n окружностей на плоскости, каждая задана радиусом и координатами центра (x, y). Найдите минимальное число лучей из начала координат, пересекающих каждую окружность из C. Нужен эффективный алгоритм.

(a) Предположим, существует луч, не пересекающий ни одного шара. Опишите и проанализируйте жадный алгоритм решения в этом частном случае. [Подсказка: см. упражнение 4.]

(b) Опишите и проанализируйте жадный алгоритм, результат которого отличается от оптимального не более чем на 1: если минимальное число лучей равно m, алгоритм должен вернуть m или $m + 1$. Разумеется, это нужно доказать.

(c) Опишите алгоритм решения задачи за $O(n^2)$.

♥ (d) Опишите алгоритм решения задачи за $O(n \log n)$.

Считайте, что дана подпрограмма, определяющая за $O(1)$ диапазон углов лучей, пересекающих произвольную окружность c. Написать её несложно, но интерес задачи не в этом.

Глава 5. Основные алгоритмы на графах

Распределение и разделение знания не похоже на множество линий, которые сходятся под одним углом и соприкасаются лишь в точке; оно подобно ветвям дерева, сходящимся в стволе, который имеет определённую цельность и непрерывность, прежде чем разделиться и разветвиться.

— Фрэнсис Бэкон, «О достоинстве и приумножении наук» (1605)

Итак, достопочтенный господин, вы видите, как мало этот способ решения связан с математикой, и я не понимаю, почему вы ожидаете его от математика, а не от кого-либо другого.

— Леонард Эйлер о задаче кёнигсбергских мостов, в письме Карлу Леонарду Готлибу Элеру (3 апреля 1736 года)

Ну, у пожарной станции в деревне поверните налево и езжайте по старой почтовой дороге мимо водохранилища и… нет, так не получится.

Лучше продолжайте прямо по асфальтированной дороге до школы, а там поверните налево на дорогу к озеру Беннетта и… нет, так тоже не выйдет.

Восточный Миллинокет, говорите? Если подумать, отсюда туда не добраться.

— Роберт Брайан и Маршалл Додж, Bert and I and Other Stories from Down East (1961)

5.1. Введение и история

Граф — это совокупность пар: пар целых чисел, людей, городов, звёзд, стран, научных статей, веб-страниц, игровых позиций, рекурсивных подзадач, даже пар графов. Отражая наиболее распространённый способ изображения графов, сами объединяемые в пары объекты обычно называют вершинами или узлами, а пары — рёбрами или дугами, хотя в действительности объекты и пары могут быть какими угодно.

Один из древнейших примеров графов — дорожные сети и их карты. В период расцвета Римской империи римские инженеры построили сеть из более чем 400 000 км общественных дорог по Европе, Западной и Центральной Азии и Северной Африке. Путешественники по этой сети брали с собой *itineraria* — простые списки либо более наглядные изображения ориентиров и расстояний вдоль различных дорог. Певтингерова таблица, свиток XIII века с изображением всей римской государственной дорожной сети *cursus publicus*, по распространённому мнению, является средневековой копией переработанного в V веке иллюстрированного маршрута (*itinerarium pictum*) I века, заказанного во времена правления Цезаря Августа. Певтингерова таблица не является географически точной картой — историки спорят даже о том, можно ли вообще считать её «картой»! — а представляет собой абстрактное изображение дорожной сети, похожее на современную схему метро. Города вдоль каждой дороги обозначены изгибами

изображающей её линии; также указаны названия городов и длины отрезков дороги между ними. Таким образом, карта содержит достаточно информации, чтобы найти кратчайший маршрут между любыми двумя городами Римской империи V века. См. рисунок 5.1.



Рисунок 5.1. Небольшой фрагмент реставрации Певтингеровой таблицы, выполненной Конрадом Миллером в 1872 году: римская дорога от современного Биртена (Veteribus, вверху слева) через Кёльн (Agripina) и Бонн (Bonnae) до Майнца (Mogontiaco, вверху справа), с ответвлениями к Триру (Avg Tresvirorvm, в центре) и Мецу (Matricorvm, внизу в центре). См. сведения об источниках изображений в конце книги.

Одно из старейших классических применений графов, а точнее деревьев, — представление родословных. Сложные семейные «деревья» столетиями использовались для решения правовых вопросов о браке, наследовании и престолонаследии. Гражданское право Римской империи, впоследствии принятое ранней католической церковью как каноническое, запрещало браки между двоюродными братьями и сёстрами или более близкими родственниками. В начале IX века церковь изменила и требуемую степень родства, и способ её вычисления. Если римская *computatio legalis* требовала, чтобы сумма расстояний до ближайшего общего предка составляла не менее четырёх, то новая *computatio canonica* требовала, чтобы максимум двух расстояний составлял не менее семи. В 1215 году, уступая практическим соображениям и фактической практике, церковь снизила минимальное расстояние, необходимое для вступления в брак, до четырёх.[1] Левая схема на рисунке 5.2 показывает особенно запутанный случай: Тирий и Тебурга заключают брак, у них рождается сын Гай, после чего Тирий умирает; затем Тебурга выходит замуж за Лотаря, рожает ему сына и умирает; наконец, Лотарь и Берта женятся, и у них рождается дочь Гемма. Может ли сын Гая законно жениться на дочери Геммы?

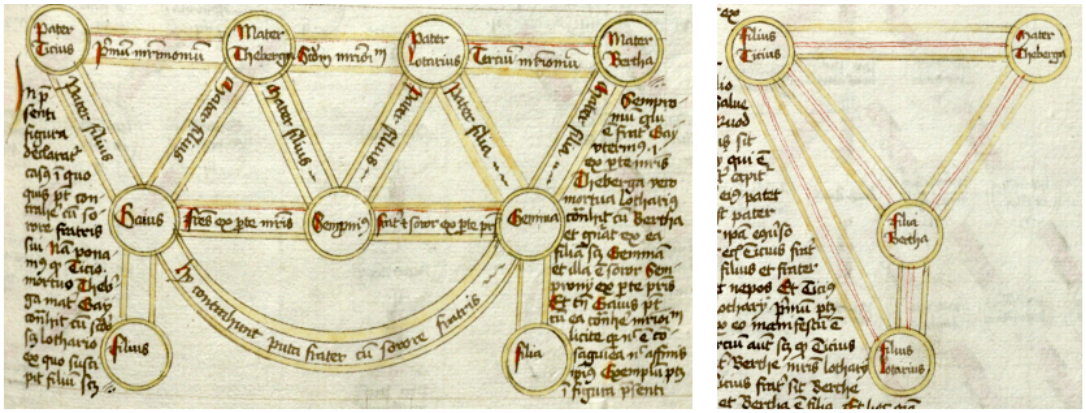


Рисунок 5.2. Две схемы сложного брачного случая из анонимного трактата XV века, посвящённого *Super arboribus consanguinitatis et affinitatis* Иоанна Андреа — трактату по каноническому праву начала XIV века. См. сведения об источниках изображений в конце книги.

В конце XVII века французский математик Пьер Вариньон разработал графический метод нахождения положения равновесия древовидной сети натянутых канатов, опираясь на более раннюю работу Симона Стевина, опубликованную столетием ранее. Вариньон заметил, что, когда канаты находятся в равновесии, можно нарисовать граф, рёбра которого параллельны отрезкам канатов и имеют длины, равные силам натяжения вдоль этих канатов, причём канаты, сходящиеся в любой точке сети, определяют замкнутый цикл графа. Метод «графической статики» Вариньона был полностью опубликован лишь в 1725 году, через два года после его смерти. Сейчас такие графы называют взаимными диаграммами сил или диаграммами Максвелла—Кремоны, по именам Джеймса Клерка Максвелла и Луиджи Кремоны, которые вместе с Карлом Кульманом и другими в конце XIX века разработали развитую теорию взаимных диаграмм.

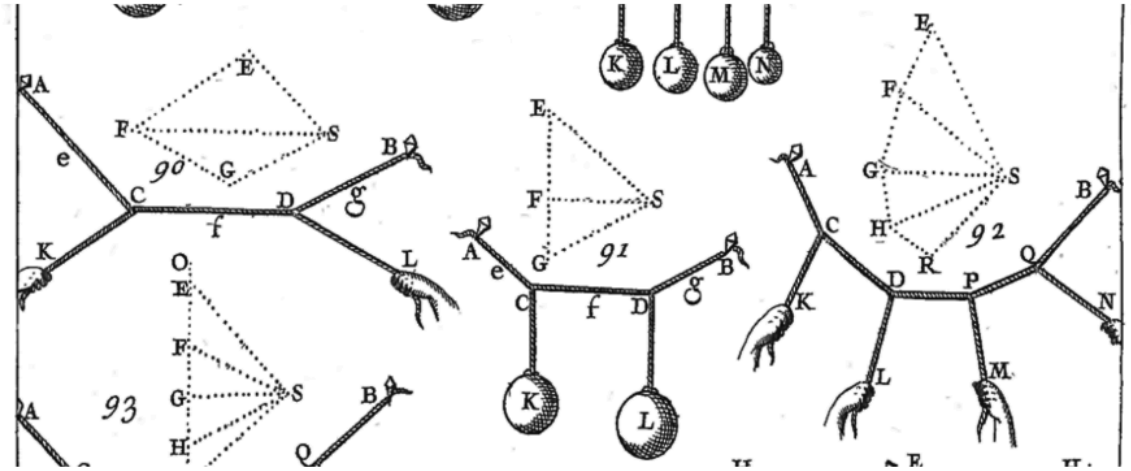


Рисунок 5.3. Взаимные диаграммы сил (пунктир) из посмертного труда Вариньона *Nouvelle mécanique, ou statique, dont le projet fut donné en MDCLXXXVII* («Новая механика, или статика, проект которой был представлен в 1687 году»). См. сведения об источниках изображений в конце книги.

[1] В XI и XII веках это ограничение постепенно распространилось на связи свойства длиной до четырёх звеньев: сначала возникающие через брак, затем через внебрачную половую связь, помолвку и даже крёстное родство. Например, формально запрещался брак между мужчиной и сестрой мужа сестры мужа его сестры, а также между вдовцом и овдовевшей матерью жены его сына. В 1215 году эти ограничения по свойству были значительно сокращены, но не устранены; церковь отказалась от понятия свойства *ex copula illicita* лишь в 1917 году.

Разумеется, есть множество других знакомых примеров графов: настольные игры (известные с древности); вершины и рёбра выпуклых многогранников (которые формально изучали древнегреческие философы, хотя они гораздо старше); изображения звёздных узоров (разработанные в Восточной Азии уже к VII веку н. э.); маршруты коня (описанные аль-Адли,

Рудратой, ас-Сули и другими в IX–X веках); лабиринты (введённые в современном виде Джованни Фонтаной около 1420 года); геодезические триангуляции (предложенные Геммой Фризиусом в 1533 году и использованные Виллебрордом Снеллиусом для вычисления длины окружности Земли в 1615 году и для определения метра в 1799 году); знаменитое частичное[2] решение Леонардом Эйлером задачи о кёнигсбергских мостах (1735); телеграфные и другие сети связи (впервые предложенные в 1753 году, разработанные Рональдсом, Шиллингом, Гауссом, Вебером и другими в начале XIX века и распространённые по всему миру к концу XIX века); электрические цепи (формализованные в начале XIX века Омом, Максвеллом, Кирхгофом и другими); структурные формулы молекул (независимо предложенные Августом Кекуле в 1857 году и Арчибальдом Купером в 1858 году); социальные сети (впервые изученные в середине 1930-х социологом Якобом Морено); цифровые электронные схемы (предложенные Чарльзом Сандерсом Пирсом в 1886 году и приведённые к современному виду Клодом Шенноном в 1937 году); и да, ладно уж, если вы настаиваете, современный интернет.

Слово «граф» для абстрактного математического объекта ввёл Джеймс Сильвестр в 1878 году, по предложению коллеги Уильяма Клиффорда приспособив «хемикографы» Кекуле для описания некоторых алгебраических инвариантов. Слово «дерево» впервые использовал для связных ациклических графов Артур Кэли в 1857 году, хотя абстрактное понятие дерева уже применяли Густав Кирхгоф и Карл фон Штаудт десятью годами ранее. Нулевую книгу по теории графов опубликовал Андре Сент-Лагю в 1926 году; Дёнеш Кёниг опубликовал первую книгу по теории графов десять лет спустя.

[2] Эйлер счёл очевидным последний шаг своего рассуждения — фактическое нахождение эйлерова обхода графа, когда степень каждой вершины чётна. Он также не заметил, что граф с эйлеровым обходом должен быть связным. Первое полное доказательство того, что граф имеет эйлеров обход тогда и только тогда, когда он связен и степени всех его вершин чётны, опубликовал Карл Гиерхольцер в 1873 году.

5.2. Основные определения

Формально (простой) граф — пара множеств (V, E), где V — произвольное непустое конечное множество, элементы которого называются вершинами[3] или узлами, а E — множество пар элементов V, которые мы называем рёбрами. В неориентированном графе рёбра — неупорядоченные пары, или просто множества из двух элементов; неориентированное ребро между u и v я обычно обозначаю uv вместо {u, v}. В ориентированном графе рёбра — упорядоченные пары вершин; ориентированное ребро из u в v я обычно обозначаю u → v вместо (u, v).

Следуя общепринятой, хотя и несколько запутанной практике, я буду также обозначать через V число вершин графа, а через E — число рёбер. Таким образом, в любом неориентированном графе $0 \le E \le (V \text{ над } 2)$, а в любом ориентированном графе $0 \le E \le V(V - 1)$.

Концы ребра uv или u → v — его вершины u и v. Концы ориентированного ребра u → v различают, называя u началом, а v концом.

Определение графа как пары множеств исключает несколько неориентированных рёбер с одинаковыми концами и несколько ориентированных рёбер с одинаковыми началом и концом.

(Один и тот же ориентированный граф может содержать и дугу $u \to v$, и обратную дугу $v \to u$.) Аналогично определение неориентированного ребра как множества вершин исключает ребро из вершины в неё саму. Графы без петель и кратных рёбер часто называют простыми графами; непростые графы иногда называют мультиграфами. Несмотря на формальное различие определений, большинство алгоритмов для простых графов распространяются на мультиграфы практически без изменений, поэтому я не вижу необходимости приводить здесь формальное определение.

Для любого ребра uv неориентированного графа вершину u называют соседом v и наоборот, а u и v — смежными. Степень узла — число его соседей. В ориентированных графах различают два вида соседей. Для любой дуги $u \to v$ вершину u называют предшественником v, а v — преемником u. Полустепень захода вершины — число её предшественников; полустепень исхода — число преемников.

Граф $G' = (V', E')$ является подграфом $G = (V, E)$, если $V' \subseteq V$ и $E' \subseteq E$. Собственный подграф G — любой его подграф, отличный от самого G.

Маршрут в неориентированном графе G — последовательность вершин, в которой каждая соседняя пара вершин смежна в G; неформально маршрут можно также представить как последовательность рёбер. Маршрут называется путём, если каждая вершина посещается не более одного раза. Для любых двух вершин u и v графа G говорят, что v достижима из u, если G содержит маршрут, а следовательно, и путь между u и v. Неориентированный граф связен, если каждая вершина достижима из любой другой. Каждый неориентированный граф состоит из одной или нескольких компонент — своих максимальных связных подграфов; две вершины находятся в одной компоненте тогда и только тогда, когда между ними есть путь.[4]

Маршрут замкнут, если начинается и заканчивается в одной вершине; цикл — замкнутый маршрут, который входит в каждую вершину и выходит из неё не более одного раза. Неориентированный граф ацикличен, если никакой его подграф не является циклом; ациклические графы также называют лесами. Дерево — связный ациклический граф или, что равносильно, одна компонента леса. Остовное дерево неориентированного графа G — его подграф, который является деревом и содержит каждую вершину G. Граф имеет остовное дерево тогда и только тогда, когда он связен. Остовный лес G — совокупность остовных деревьев, по одному для каждой компоненты G.

Для ориентированных графов определения несколько отличаются. Ориентированный маршрут — последовательность вершин $v_0 \to v_1 \to v_2 \to \cdots \to v_\ell$, такая что $v_{i-1} \to v_i$ является дугой для каждого индекса i; ориентированные пути и циклы определяются аналогично. Вершина v достижима из u в ориентированном графе G тогда и только тогда, когда G содержит ориентированный маршрут, а следовательно, и ориентированный путь из u в v. Ориентированный граф сильно связен, если каждая его вершина достижима из любой другой. Ориентированный граф ацикличен, если не содержит ориентированного цикла; ориентированные ациклические графы часто называют DAG.

[3] Единственное число английского слова *vertices* — *vertex*. Аналогично единственное число *matrices* — *matrix*, а *indices* — *index*. Если только вы не говорите по-итальянски, слов *vertice, matrice, indice, appendice, helice, apice, vortice, radice, simplice, codice, directrice, dominatrice,*

Unice, Kleenice, Asterice, Obelice, Dogmatice, Getafice, Cacofonice, Vitalstatistice, Geriatrice и *Jimi Hendrice* не существует! Если вам трудно запомнить это правило, используйте слово *node*.

[4] Компоненты часто называют «компонентами связности», но такое название избыточно: по определению компоненты связны.

5.3. Представления и примеры

Самый распространённый способ визуального представления графов — нарисовать их. Рисунок графа сопоставляет каждой вершине точку на плоскости (обычно изображаемую маленьким кружком или другой фигурой), а каждому ребру — кривую или отрезок прямой между двумя вершинами. Граф планарен, если его можно нарисовать без пересечений рёбер; такой рисунок также называют вложением.[5] Один и тот же граф может иметь множество разных рисунков, поэтому важно не путать конкретный рисунок с самим графом. В частности, планарные графы могут иметь непланарные рисунки!

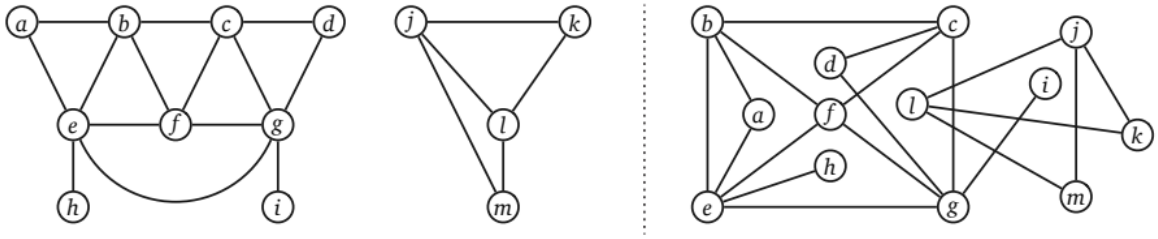


Рисунок 5.4. Два рисунка одного и того же несвязного планарного графа с 13 вершинами, 19 рёбрами и двумя компонентами. Только рисунок слева является вложением.

Однако рисунки — далеко не единственное полезное представление графов. Например, граф пересечений семейства геометрических объектов содержит узел для каждого объекта и ребро для каждой пересекающейся пары объектов. Можно ли представить конкретный граф как граф пересечений, зависит от того, какие объекты используются в качестве вершин. Различные типы объектов — отрезки, прямоугольники, окружности и т. д. — определяют различные классы графов. Особенно полезный тип графов пересечений — интервальный граф, вершины которого — интервалы на вещественной прямой, а рёбра соединяют любые два перекрывающихся интервала.

[5] К сожалению, слово «вложение» часто используют как синоним «рисунка», даже когда рёбра пересекаются. Пожалуйста, не делайте так.

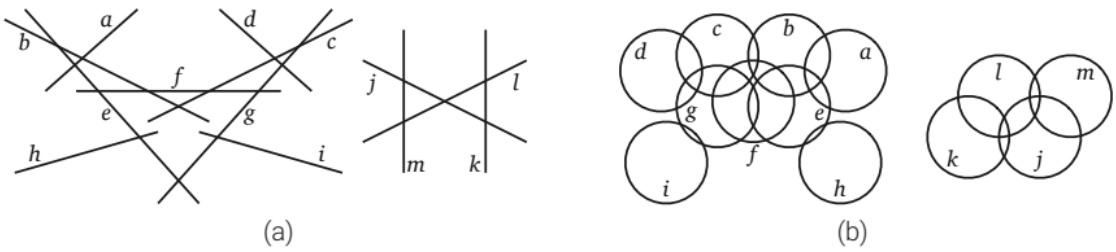


Рисунок 5.5. Граф с рисунка 5.4 также является графом пересечений (a) семейства отрезков и (b) семейства окружностей.

Другой хороший пример — граф зависимостей рекурсивного алгоритма. Графы зависимостей являются ориентированными ациклическими графами. Их вершины — все различные рекурсивные подзадачи, возникающие при выполнении алгоритма на конкретном входе. Ребро ведёт от одной подзадачи к другой, если вычисление второй требует рекурсивного вычисления первой. Например, для рекуррентного соотношения Фибоначчи

$$F_n = \begin{cases} 0 & \text{если } n = 0, \\ 1 & \text{если } n = 1, \\ F_{n-1} + F_{n-2} & \text{иначе,} \end{cases}$$

вершины графа зависимостей — целые числа 0, 1, 2, ..., n, а рёбра — пары (i – 1) → i и (i – 2) → i для каждого целого i от 2 до n.

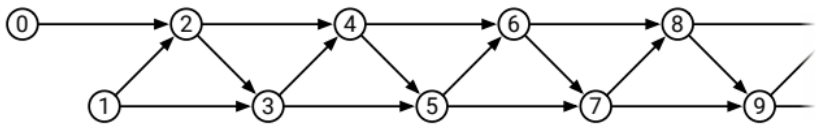


Рисунок 5.6. Граф зависимостей рекуррентного соотношения Пингалы—Фибоначчи.

Более сложный пример: вспомним рекуррентное соотношение для задачи редакционного расстояния из главы 3:

$$\text{Edit}(i, j) = \begin{cases} i & \text{если } j = 0, \\ j & \text{если } i = 0, \\ \min \begin{cases} \text{Edit}(i-1, j) + 1, \\ \text{Edit}(i, j-1) + 1, \\ \text{Edit}(i-1, j-1) + [A[i] \neq B[j]] \end{cases} & \text{иначе.} \end{cases}$$

Граф зависимостей этого соотношения — решётка m × n вершин (i, j), соединённых вертикальными рёбрами (i – 1, j) → (i, j), горизонтальными рёбрами (i, j – 1) → (i, j) и диагональными рёбрами (i – 1, j – 1) → (i, j). Динамическое программирование эффективно для любого рекуррентного соотношения с достаточно небольшим графом зависимостей: правильный порядок вычисления гарантирует, что каждая подзадача посещается после своих предшественников.

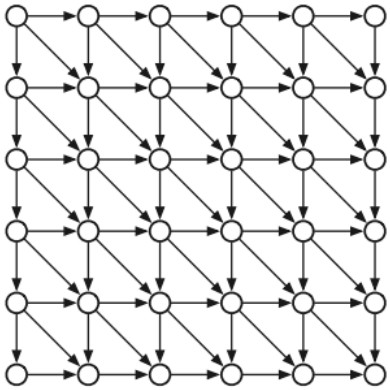


Рисунок 5.7. Граф зависимостей рекуррентного соотношения редакционного расстояния.

Ещё один интересный пример — граф конфигураций игры, головоломки или механизма, например крестиков-ноликов, шашек, кубика Рубика, Ханойской башни или машины Тьюринга. Вершины графа конфигураций — все допустимые конфигурации головоломки; ребро ведёт от одной конфигурации к другой, если первую можно преобразовать во вторую одним простым «ходом». (Очевидно, точное определение зависит от разрешённых ходов.) Даже для достаточно простых механизмов граф конфигураций может быть чрезвычайно сложным, и обычно нам доступна только локальная информация о нём.

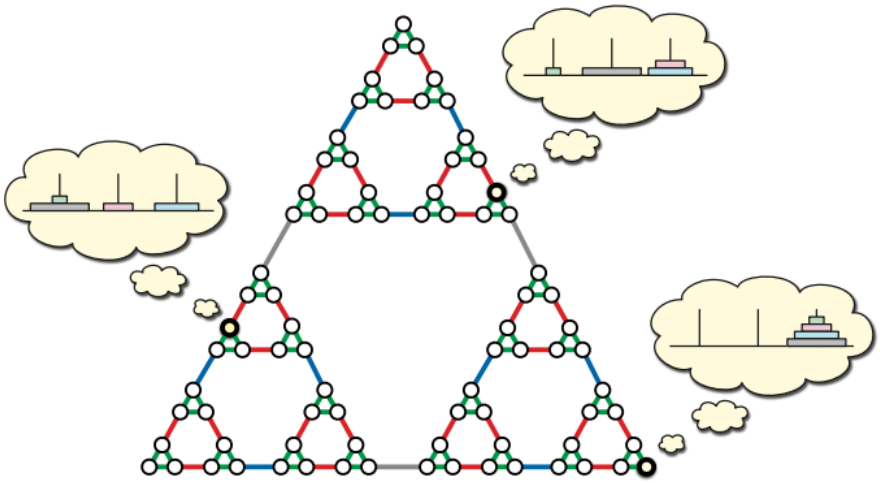


Рисунок 5.8. Граф конфигураций Ханойской башни из четырёх дисков.

Графы конфигураций — близкие родственники игровых деревьев, рассмотренных в главе 2, но с одним принципиальным отличием. Каждое состояние игры встречается в её графе конфигураций ровно один раз, тогда как в игровом дереве может появляться многократно. Короче говоря, графы конфигураций — это мемоизированные игровые деревья!

Конечные автоматы, используемые в теории формальных языков, можно моделировать размеченными ориентированными графами. Напомним: детерминированный конечный автомат формально определяется пятёркой $M = (\Sigma, Q, s, A, \delta)$, где Σ — конечное множество, называемое алфавитом, Q — конечное множество состояний, $s \in Q$ — начальное состояние, $A \subseteq Q$ — множество допускающих состояний, а $\delta: Q \times \Sigma \to Q$ — функция переходов. Но часто удобнее считать M ориентированным графом G_M, вершины которого — состояния Q, а рёбра имеют вид $q \to \delta(q, a)$ для каждого состояния $q \in Q$ и символа $a \in \Sigma$. Основные вопросы о языке $L(M)$, распознаваемом M, можно тогда сформулировать как вопросы о графе G_M. Например, $L(M) = \emptyset$ тогда и только тогда, когда из начального состояния-вершины s не достижимо ни одно допускающее состояние-вершина.

Наконец, иногда один граф можно использовать для неявного представления других, более крупных графов. Хороший пример такого неявного представления — построение подмножеств, которое обычно применяется для преобразования недетерминированных конечных автоматов в детерминированные, но может применяться к произвольным ориентированным графам следующим образом. Для любого ориентированного графа $G = (V, E)$ можно определить новый

ориентированный граф G′ = (2^V, E′), вершинами которого являются все подмножества V, а рёбра E′ определены так:

$$E' = \{A \to B \mid u \to v \in E \text{ для некоторых } u \in A \text{ и } v \in B\}.$$

Это определение можно механически перевести в алгоритм построения G′ по G, но, строго говоря, такое построение необязательно: G уже является неявным представлением G′.

Важно не путать ни один из этих примеров или представлений с настоящим формальным определением: граф — это пара множеств (V, E), где V — произвольное непустое конечное множество, а E — множество пар элементов V, упорядоченных или неупорядоченных. Коротко: граф — множество пар объектов.

5.4. Структуры данных

На практике графы обычно представляют одной из двух стандартных структур данных: списками смежности или матрицами смежности. На высоком уровне обе структуры — массивы, индексируемые вершинами; для этого каждая вершина должна иметь уникальный целочисленный идентификатор от 1 до V. Формально эти целые числа и являются вершинами.

Списки смежности

Самая распространённая структура данных для хранения графов — список смежности. Список смежности — массив списков, каждый из которых содержит соседей одной из вершин (или исходящих соседей, если граф ориентированный).[6] Для неориентированных графов каждое ребро uv хранится дважды: один раз в списке соседей u и один раз в списке соседей v; для ориентированных графов каждая дуга u → v хранится только один раз — в списке соседей начальной вершины u. В обоих случаях общая память, необходимая списку смежности, составляет O(V + E).

[6] Внимательные студенты могут заметить, что вопреки названию список смежности — не список. Эта терминология — пример принципа ложного следа: в информатике, как и в математике, ложный след (*red herring*, буквально «красная сельдь») необязательно красный и необязательно рыба.

Существует несколько способов представления списков соседей, но стандартная реализация использует простой односвязный список. Полученная структура позволяет перечислить исходящих соседей узла v за O(1 + deg(v)): достаточно просмотреть список соседей v. Аналогично можно определить, является ли u → v ребром, за O(1 + deg(u)), просматривая список соседей u. Для неориентированных графов время можно улучшить до O(1 + min{deg(u), deg(v)}), одновременно просматривая списки соседей u и v и останавливаясь, когда ребро найдено или достигнут конец одного из списков.

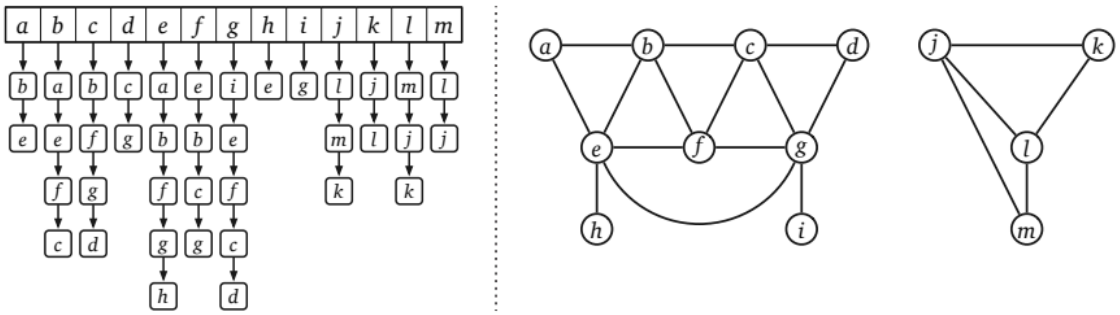


Рисунок 5.9. Список смежности для нашего примера графа.

Конечно, связанные списки — не единственная подходящая структура; подойдёт любая другая, поддерживающая поиск, перечисление, вставку и удаление. Например, можно сократить время проверки наличия ребра uv до O(1 + log(deg(u))), используя сбалансированное двоичное дерево поиска для хранения соседей u, или даже до O(1), используя подходящим образом построенную хеш-таблицу.[7]

Одна из распространённых реализаций списков смежности — массив смежности: все записи рёбер хранятся в одном массиве, записи рёбер, инцидентных каждой вершине, занимают непрерывный участок, а в отдельном массиве хранится индекс первого ребра каждой вершины. Кроме того, полезно сохранять участки каждой вершины в отсортированном порядке, как на рисунке 5.10, чтобы проверять смежность двух вершин u и v за O(log deg(u)).

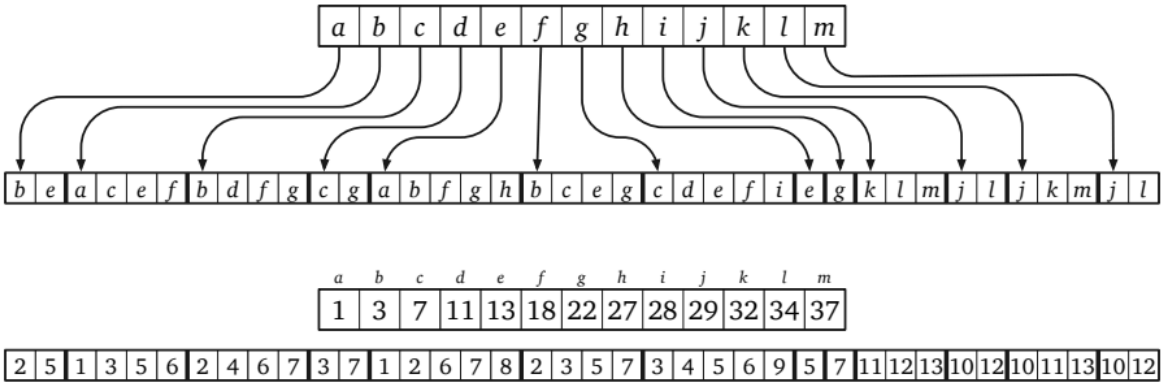


Рисунок 5.10. Абстрактный массив смежности для примера графа и его фактическая реализация в виде пары целочисленных массивов.

Матрицы смежности

Другая стандартная структура для графов — матрица смежности,[8] впервые предложенная Жоржем Брюнелем в 1894 году. Матрица смежности графа G — матрица V × V из нулей и единиц, обычно представленная двумерным массивом A[1 .. V, 1 .. V], где каждый элемент показывает наличие соответствующего ребра в G. А именно, для всех вершин u и v:

- если граф неориентированный, A[u, v] := 1 тогда и только тогда, когда uv ∈ E;
- если граф ориентированный, A[u, v] := 1 тогда и только тогда, когда u → v ∈ E.

Для неориентированных графов матрица смежности всегда симметрична, то есть A[u, v] = A[v, u] для всех вершин u и v, поскольку uv и vu — лишь разные названия одного и того же ребра, а диагональные элементы A[u, u] все равны нулю. Для ориентированных графов матрица смежности может быть симметричной или несимметричной, а диагональные элементы могут быть как нулевыми, так и ненулевыми.

	a	*b*	*c*	*d*	*e*	*f*	*g*	*h*	*i*	*j*	*k*	*l*	*m*
a	0	1	0	0	1	0	0	0	0	0	0	0	0
b	1	0	1	0	1	1	0	0	0	0	0	0	0
c	0	1	0	1	0	1	1	0	0	0	0	0	0
d	0	0	1	0	0	0	1	0	0	0	0	0	0
e	1	1	0	0	0	1	1	1	0	0	0	0	0
f	0	1	1	0	1	0	1	0	0	0	0	0	0
g	0	0	1	1	1	1	0	0	1	0	0	0	0
h	0	0	0	0	1	0	0	0	0	0	0	0	0
i	0	0	0	0	0	0	1	0	0	0	0	0	0
j	0	0	0	0	0	0	0	0	0	0	1	1	1
k	0	0	0	0	0	0	0	0	0	1	0	1	0
l	0	0	0	0	0	0	0	0	0	1	1	0	1
m	0	0	0	0	0	0	0	0	0	1	0	1	0

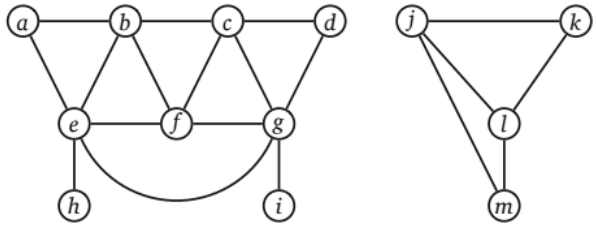


Рисунок 5.11. Матрица смежности для нашего примера графа.

Имея матрицу смежности, можно за Θ(1) определить, соединены ли две вершины ребром, просто посмотрев соответствующую ячейку. Можно также перечислить всех соседей вершины за Θ(V), просмотрев соответствующую строку или столбец. Это время оптимально в худшем случае, но даже если соседей мало, для их поиска всё равно придётся просмотреть всю строку. Аналогично матрицы смежности требуют $\Theta(V^2)$ памяти независимо от фактического числа рёбер, поэтому экономны по памяти только для очень плотных графов.

[7] Это гораздо тоньше, чем кажется. Большинство популярных методов хеширования не гарантирует быстрого времени запроса, а даже большинство хороших методов гарантируют лишь ожидаемое время O(1). Более подробное обсуждение хеширования см. на http://algorithms.wtf/.

[8] См. сноску 3.

Сравнение

В таблице 5.1 приведена сводка эффективности различных стандартных структур данных для графов. Звёздочки ∗ обозначают ожидаемые амортизированные оценки времени поддержания динамических хеш-таблиц.[9]

Операция	**Стандартный список смежности (связанные списки)**	**Быстрый список смежности (хеш-таблицы)**	**Матрица смежности**
Память	$\Theta(V + E)$	$\Theta(V + E)$	$\Theta(V^2)$
Проверить $uv \in E$	$O(1 + \min\{\deg(u), \deg(v)\}) = O(V)$	$O(1)$	$O(1)$

Операция	Стандартный список смежности (связанные списки)	Быстрый список смежности (хеш-таблицы)	Матрица смежности
Проверить $u \to v \in E$	$O(1 + \deg(u)) = O(V)$	$O(1)$	$O(1)$
Перечислить исходящих соседей v	$\Theta(1 + \deg(v)) = O(V)$	$\Theta(1 + \deg(v)) = O(V)$	$\Theta(V)$
Перечислить все рёбра	$\Theta(V + E)$	$\Theta(V + E)$	$\Theta(V^2)$
Вставить ребро uv	$O(1)$	$O(1)*$	$O(1)$
Удалить ребро uv	$O(\deg(u) + \deg(v)) = O(V)$	$O(1)*$	$O(1)$

Таблица 5.1. Время основных операций для стандартных структур представления графов.

Глядя на это сравнение, можно разумно спросить, зачем вообще использовать матрицу смежности: списки смежности с хеш-таблицами поддерживают те же операции за то же время, занимая меньше памяти. Основная причина в том, что для достаточно плотных графов матрицы смежности проще и эффективнее на практике: они избегают накладных расходов на переходы по указателям и вычисление хеш-функций и представляют собой просто непрерывные блоки памяти.

Точно так же можно спросить, зачем хранить соседей в структуре списка смежности связными списками, а не сбалансированными двоичными деревьями поиска или хеш-таблицами. Хотя основная практическая причина почти наверняка — традиция («Если они подходили для кода Дональда Кнута, то и вам подойдут!»), есть и более принципиальные доводы. Например, стандартные списки смежности вполне достаточны для большинства применений. Большинство стандартных алгоритмов на графах никогда или редко проверяют наличие произвольного ребра либо пытаются вставлять или удалять рёбра, поэтому оптимизировать структуры данных для этих операций не нужно.

Но, по моему мнению, наиболее убедительная причина использовать обе стандартные структуры состоит в том, что многие графы неявно представлены матрицами смежности и стандартными списками смежности. Например:

- Графы пересечений обычно представлены списком лежащих в их основе геометрических объектов. Если мы можем за постоянное время проверять пересечение двух объектов, то можем применять любой алгоритм на графах к графу пересечений, притворяясь, что входной граф явно хранится как матрица смежности.
- Любую структуру данных, состоящую из записей с указателями друг на друга, можно рассматривать как ориентированный граф. К ней можно применять алгоритмы на графах, притворяясь, что граф хранится как стандартный список смежности.

[9] Не беспокойтесь, если вы не понимаете выражение «ожидаемое амортизированное».

- Аналогично можно применять любой алгоритм на графах к графу конфигураций так, словно он представлен стандартным списком смежности, если все возможные ходы из данной конфигурации можно перечислять за постоянное время на каждый ход.

В последних двух примерах можно перечислить рёбра, исходящие из любой вершины, за время, пропорциональное её степени, но не обязательно можно за постоянное время определить, смежны ли две вершины. (Есть ли указатель из этой записи в ту? Можно ли перейти из этой конфигурации в ту одним ходом?) Более того, мы обычно не можем позволить себе перестроить указатели каждой записи или ходы из данной конфигурации в более эффективную структуру данных. Поэтому подходящей моделью является стандартный список смежности с соседями, хранящимися в связанных списках.

В оставшейся части книги, если явно не указано иное, все оценки времени для алгоритмов на графах предполагают представление входного графа стандартным списком смежности. Аналогично, если в упражнении вас просят разработать и проанализировать алгоритм на графе, считайте, что входной граф представлен стандартным списком смежности, если явно не указано иное.

5.5. Обход в произвольном порядке

До сих пор мы обсуждали только локальные операции над графами; пожалуй, самый фундаментальный глобальный вопрос о графах — достижимость. Для данного графа G и вершины s в G вопрос достижимости состоит в том, какие вершины достижимы из s, то есть для каких вершин v существует путь из s в v. Пока рассмотрим только неориентированные графы; ориентированные я кратко рассмотрю в конце раздела. В неориентированном графе достижимые из s вершины — в точности вершины той же компоненты, что и s.

Возможно, самый естественный алгоритм достижимости, по крайней мере для привыкших мыслить рекурсивно людей вроде нас, — поиск в глубину. Его можно записать рекурсивно или итеративно. В обоих случаях это один и тот же алгоритм; единственная разница в том, что в нерекурсивной версии мы действительно видим стек «рекурсии».

```
RecursiveDFS(v):
    если v не помечена
        пометить v
        для каждого ребра vw
            RecursiveDFS(w)

IterativeDFS(s):
    Push(s)
    пока стек не пуст
        v ← Pop
        если v не помечена
            пометить v
            для каждого ребра vw
                Push(w)
```

Поиск в глубину — лишь один, возможно наиболее распространённый, вид общего семейства алгоритмов обхода графа, которое я называю обходом в произвольном порядке (*whatever-first search*). Общий алгоритм хранит множество рёбер-кандидатов в некоторой структуре данных, которую я назову «мешком». Единственные важные свойства мешка: в него можно положить предметы, а позже извлечь их обратно. Стек — конкретная разновидность мешка, но далеко не единственная. Вот общий алгоритм:

```
WhateverFirstSearch(s):
    положить s в мешок
    пока мешок не пуст
        извлечь v из мешка
        если v не помечена
            пометить v
            для каждого ребра vw
                положить w в мешок
```

Я утверждаю, что `WhateverFirstSearch` помечает все узлы, достижимые из s, и никакие другие. Очевидно, каждая вершина G помечается не более одного раза. Чтобы показать, что каждая вершина связного графа посещается хотя бы раз, немного изменим алгоритм; в оригинале изменения выделены красным жирным шрифтом. Вместо вершин изменённый алгоритм хранит в мешке пары вершин. Благодаря этому при первом посещении вершины v мы можем запомнить, какая ранее посещённая соседняя вершина положила v в мешок. Эту предыдущую вершину назовём родителем v.

```
WhateverFirstSearch(s):
    положить (∅, s) в мешок
    пока мешок не пуст
        извлечь (p, v) из мешка              (?)
        если v не помечена
            пометить v
            parent(v) ← p
            для каждого ребра vw             (†)
                положить (v, w) в мешок      (??)
```

Лемма 5.1. `WhateverFirstSearch(s)` помечает все вершины, достижимые из s, и только их. Более того, множество всех пар (v, parent(v)), для которых parent(v) ≠ ∅, определяет остовное дерево компоненты, содержащей s.

Доказательство. Сначала покажем индукцией по длине кратчайшего пути от s до v, что алгоритм помечает каждую вершину v, достижимую из s. Алгоритм помечает s. Пусть v — любая другая достижимая из s вершина и s → ⋯ → u → v — любой путь из s в v с минимальным числом рёбер. (Такой путь существует, поскольку v достижима из s.) Путь-префикс s → ⋯ → u короче кратчайшего пути из s в u, поэтому предположение индукции означает, что алгоритм помечает u. Когда алгоритм помечает u, он должен сразу положить пару (u, v) в мешок, а значит, позже должен извлечь (u, v) из мешка; в этот момент алгоритм сразу пометит v, если она ещё не помечена.

Каждая пара (v, parent(v)) при parent(v) ≠ ∅ действительно является ребром исходного графа G. Мы утверждаем, что для любой помеченной вершины v путь по родительским рёбрам v → parent(v) → parent(parent(v)) → ⋯ в конце концов приводит обратно к s; докажем это индукцией

по порядку пометки вершин. Очевидно, s достижима из s; пусть v — любая другая помеченная вершина. Родитель v должен быть помечен до v, поэтому предположение индукции означает, что родительский путь parent(v) → parent(parent(v)) → ⋯ приводит к s; добавление ещё одного родительского ребра s → parent(s) доказывает утверждение.

Примечание переводчика. В исходном доказательстве имеются описки: префикс кратчайшего пути до v является кратчайшим путём до u и короче пути до v; последним к родительскому пути добавляется ребро v → parent(v), а не s → parent(s).

Предыдущее утверждение означает, что каждая помеченная алгоритмом вершина достижима из s, а множество всех родительских рёбер образует связный граф. Поскольку у каждой помеченной вершины, кроме s, ровно один родитель, число родительских рёбер ровно на единицу меньше числа помеченных вершин. Следовательно, родительские рёбра образуют дерево. ■

Анализ

Время работы алгоритма обхода зависит от структуры данных, используемой для «мешка», но можно сделать несколько общих наблюдений. Пусть T — время, необходимое для вставки одного элемента в мешок или извлечения одного элемента из него. Цикл `for` (†) выполняется ровно один раз для каждой помеченной вершины и, следовательно, не более V раз. Каждое ребро uv в компоненте s кладётся в мешок ровно дважды: один раз как пара (u, v) и один раз как пара (v, u), поэтому строка (??) выполняется не более 2E раз. Наконец, нельзя извлечь из мешка больше предметов, чем в него положили, поэтому строка (?) выполняется не более 2E + 1 раз. Таким образом, если исходный граф G хранится стандартным списком смежности, `WhateverFirstSearch` работает за O(V + ET). (Если G хранится матрицей смежности, время работы `WhateverFirstSearch` увеличивается до $O(V^2 + ET)$.)

5.6. Важные варианты

Стек: поиск в глубину

Если реализовать «мешок» стеком, мы вернёмся к исходному алгоритму поиска в глубину. Вставка (`push`) и извлечение (`pop`) из стека занимают O(1) каждая, поэтому алгоритм работает за O(V + E). Остовное дерево, образованное родительскими рёбрами, называется остовным деревом поиска в глубину. Его точная форма зависит от начальной вершины и порядка посещения соседей в цикле `for` (†), но обычно такие деревья длинные и узкие. В главе 6 мы рассмотрим несколько важных свойств и применений поиска в глубину.

Очередь: поиск в ширину

Если реализовать «мешок» очередью, получится другой алгоритм обхода графа — поиск в ширину. Вставка (`push`) и извлечение (`pull`) из очереди занимают O(1) каждая, поэтому алгоритм работает за O(V + E). В этом случае остовное дерево поиска в ширину, образованное родительскими рёбрами, содержит кратчайшие пути из начальной вершины s ко всем остальным вершинам её компоненты; подробно кратчайшие пути мы рассмотрим в главе 8. И

здесь точная форма дерева зависит от начальной вершины и порядка посещения соседей в цикле `for` (†), но обычно такие деревья короткие и ветвистые.

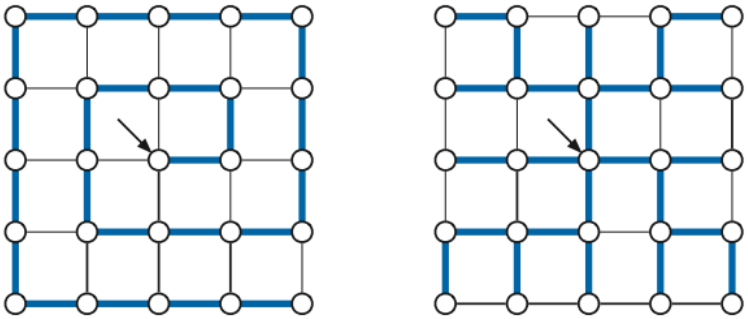


Рисунок 5.12. Остовное дерево поиска в глубину и остовное дерево поиска в ширину одного графа; оба обхода начинаются в центральной вершине.

Очередь с приоритетом: поиск по наилучшему первому

Наконец, если реализовать «мешок» очередью с приоритетом, получится ещё одно семейство алгоритмов — поиск по наилучшему первому. Поскольку очередь с приоритетом хранит не более одной копии каждого ребра, вставка ребра или извлечение ребра с минимальным приоритетом занимает O(log E), откуда время работы поиска равно O(V + E log E).

Я называю поиск по наилучшему первому «семейством алгоритмов», а не одним алгоритмом, потому что приоритеты рёбрам можно назначать разными способами, и эти решения приводят к разному поведению. Ниже я опишу три известных варианта, но есть и многие другие. Во всех трёх примерах предполагается, что каждое ребро uv или u → v входного графа имеет неотрицательный вес w(uv) или w(u → v).

Во-первых, если входной граф неориентирован и мы используем вес каждого ребра как его приоритет, поиск по наилучшему первому строит минимальное остовное дерево компоненты s. Удивительно, но если все веса рёбер различны, полученное дерево не зависит от начальной вершины или порядка посещения соседей; в этом случае минимальное остовное дерево действительно единственно. Эту реализацию поиска обычно, хотя, как обычно, ошибочно, называют алгоритмом Прима; его и другие алгоритмы минимальных остовных деревьев мы подробнее обсудим в главе 7.

Определим длину пути как сумму весов его рёбер. Можно также вычислять кратчайшие пути во взвешенных графах с помощью поиска по наилучшему первому следующим образом. Каждая помеченная вершина v хранит расстояние dist(v). Вначале задаём dist(s) = 0. Для каждой другой вершины v, когда мы устанавливаем parent(v) ← p, также задаём dist(v) ← dist(p) + w(p → v), а при вставке ребра v → w в очередь с приоритетом используем приоритет dist(v) + w(v → w). Если все веса рёбер положительны, dist(v) — длина кратчайшего пути из s в v. Этот вариант поиска обычно, хотя, строго говоря и как обычно, ошибочно, называют алгоритмом Дейкстры; мы снова встретим его в главе 8.

Наконец, определим ширину пути как минимальный вес любого ребра пути. Простое изменение алгоритма поиска «Дейкстры» вычисляет самые широкие пути из s ко всем

остальным достижимым вершинам; такие пути также называют кратчайшими путями по узкому месту. Каждая помеченная вершина v хранит значение width(v). Изначально задаём width(s) = ∞. Для каждой другой вершины v при установке parent(v) ← p задаём width(v) ← min{width(p), w(p → v)}, а при вставке ребра v → w в очередь используем приоритет min{width(v), w(v → w)}. Самые широкие пути полезны в алгоритмах вычисления максимальных потоков, которые мы, как вы догадались, рассмотрим в главе 10.

Несвязные графы

`WhateverFirstSearch(s)` посещает только вершины, достижимые из одной начальной вершины s. Чтобы посетить каждую вершину G, можно использовать следующую простую функцию-«обёртку»:

```
WFSAll(G):
    для всех вершин v
        снять пометку с v
    для всех вершин v
        если v не помечена
            WhateverFirstSearch(v)
```

Постойте, слышу я ваш вопрос, зачем[10] так всё усложнять? Почему бы просто не просмотреть массив вершин?

```
MarkEveryVertexDuh(G):
    для всех вершин v
        пометить v
```

Конечно, если у вас есть полный список вершин, так можно сделать, но помните: не все графы представлены настолько явно.[11] Что важнее, даже при наличии явного списка вершин порядок посещения в этом наивном алгоритме определяется их порядком в структуре данных, а не абстрактной структурой графа.

В частности, в отличие от наивного просмотра вершин, `WFSAll` посещает сначала все вершины одной компоненты, затем все вершины следующей и так далее для каждой компоненты входного графа. Такой покомпонентный обход позволяет, например, подсчитать компоненты несвязного графа одним счётчиком:

```
CountComponents(G):
    count ← 0
    для всех вершин v
        снять пометку с v
    для всех вершин v
        если v не помечена
            count ← count + 1
            WhateverFirstSearch(v)
    вернуть count
```

Приложив лишь немного больше усилий, можно записать, какой компоненте принадлежит каждая вершина, вместо простой пометки:

```
CountAndLabel(G):
    count ← 0
    для всех вершин v
        снять пометку с v
    для всех вершин v
        если v не помечена
            count ← count + 1
            LabelOne(v, count)
    вернуть count

⟪Пометить одну компоненту⟫
LabelOne(v, count):
    пока мешок не пуст
        извлечь v из мешка
        если v не помечена
            пометить v
            comp(v) ← count
            для каждого ребра vw
                положить w в мешок
```

Примечание переводчика. В исходном псевдокоде `LabelOne` перед циклом не помещает начальную вершину v в мешок. Чтобы обход компоненты начался, её необходимо туда положить.

`WFSAll` помечает каждую вершину один раз, кладёт каждое ребро в мешок один раз и извлекает каждое ребро один раз, поэтому общее время работы — O(V + ET), где T — время операции с мешком. В частности, если выполнять поиск в глубину или поиск в ширину из каждой вершины, полученный алгоритм по-прежнему требует всего O(V + E).

Более того, поскольку `WhateverFirstSearch` вычисляет остовное дерево одной компоненты, можно использовать `WFSAll` для вычисления остовного леса всего графа.

[10] Это слово почти всегда служит сигналом, что вы упускаете нечто важное.

[11] С другой стороны, если хранить в каждой вершине временную метку её последней «пометки», можно «снять пометки со всех вершин» за O(1), записав время начала обхода и считая вершину «помеченной», если её временная метка позднее записанного времени начала.

В частности, поиск по наилучшему первому, использующий веса рёбер в качестве приоритетов, вычисляет остовный лес минимального веса за O(V + E log E).

Поразительно, но по крайней мере один чрезвычайно популярный учебник по алгоритмам утверждает, что эту обёртку можно использовать только с поиском в глубину.[12] Это утверждение совершенно неверно. На самом деле самая первая реализация поиска в ширину, написанная около 1945 года Конрадом Цузе на его протоязыке Plankalkül, была создана именно для подсчёта и разметки компонент неориентированного графа.

Ориентированные графы

Обход в произвольном порядке легко приспособить к ориентированным графам; единственное отличие — когда мы помечаем вершину, мы кладём в мешок всех её исходящих соседей. Более

того, при использовании стандартных списков смежности или матриц смежности код вообще не нужно менять!

```
WhateverFirstSearch(s):
    положить s в мешок
    пока мешок не пуст
        извлечь v из мешка
        если v не помечена
            пометить v
            для каждого ребра v → w
                положить w в мешок
```

Предыдущее доказательство показывает, что алгоритм помечает все вершины, достижимые из s, а ориентированные рёбра parent(v) → p определяют корневое дерево, в котором все рёбра направлены от корня s. Однако даже если граф связен, мы уже не обязательно получим остовное дерево всего графа: достижимость больше не симметрична.

Примечание переводчика. В обозначении рёбер `parent(v) → p` выше, по смыслу и определению родителя, имеется в виду `parent(v) → v`.

С другой стороны, `WhateverFirstSearch` всё же определяет остовное дерево вершин, достижимых из s. Более того, меняя реализацию «мешка», можно получить для этих достижимых вершин остовное дерево поиска в глубину, остовное дерево поиска в ширину, ориентированное остовное дерево минимального веса, дерево кратчайших путей или дерево самых широких путей.

[12] Буквальная цитата: «В отличие от поиска в ширину, подграф предшественников которого образует дерево, подграф предшественников поиска в глубину может состоять из нескольких деревьев, поскольку поиск может повторяться из нескольких источников».

5.7. Сведения к графам: заливка

Один из самых ранних современных примеров обхода в произвольном порядке предложил Эдвард Мур в середине 1950-х годов. Карта пикселей — двумерный массив, значения которого обозначают цвета; отдельные элементы массива называются пикселями, что является сокращением от *picture elements* — «элементы изображения».[13] Связная область карты пикселей — связное подмножество пикселей одного цвета, причём два пикселя считаются соседними, если они непосредственно соседствуют по горизонтали или вертикали. Операция заливки, обычно обозначаемая ведром с краской в редакторах растровой графики, меняет цвет всех пикселей связной области на новый; вход операции состоит из индексов i и j одного пикселя нужной области и нового цвета.

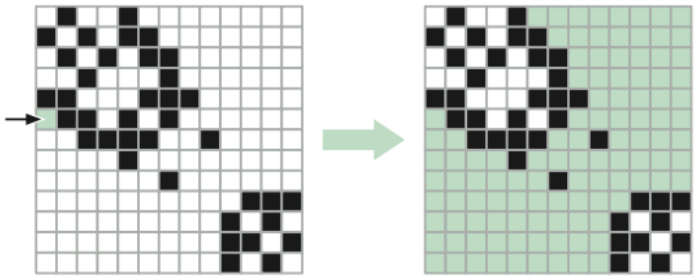


Рисунок 5.13. Пример заливки.

Задачу заливки можно свести к задаче достижимости, последовательно раскрыв определения. Определим неориентированный граф G = (V, E), вершинами которого являются отдельные пиксели, а рёбра соединяют соседние пиксели одного цвета. Каждая связная область карты пикселей — компонента G; таким образом, задача заливки сводится к задаче достижимости в G. Её можно решить обходом G в произвольном порядке, начиная с данного пикселя (i, j), с небольшим изменением: каждый раз при пометке вершины мы сразу меняем её цвет. Для карты пикселей n × n граф G имеет n^2 вершин и не более $2n^2$ рёбер, поэтому обход в произвольном порядке работает за $O(V + E) = O(n^2)$.

Этот простой пример демонстрирует основные составляющие сведения. Вместо того чтобы решать задачу заливки с нуля, мы используем существующий алгоритм как подпрограмму — чёрный ящик. То, как именно работает обход в произвольном порядке, здесь совершенно не важно; важна только его спецификация: по данному графу G и начальной вершине s пометить каждую вершину G, достижимую из s. Как и с любой подпрограммой, нам всё равно нужно описать построение входа и использование результата. Также необходимо проанализировать полученный алгоритм в терминах параметров нашего входа, а не вершин и рёбер промежуточного графа, построенного алгоритмом.

Теперь, когда у нас есть работающий алгоритм, — и только теперь — можно применить две простые оптимизации: практическую и теоретическую.

- В реальной реализации мы бы не строили отдельную структуру данных графа G. Вместо этого можно непосредственно использовать карту пикселей так, словно она является стандартным списком смежности, поскольку соседей любого пикселя того же цвета можно перечислять за O(1) на каждого. В частности, нет необходимости отдельно «помечать» вершины: можно использовать цвет пикселей.
- Более внимательный анализ показывает, что время работы пропорционально числу пикселей заливаемой области — или, что равносильно, числу вершин компоненты G, содержащей вершину (i, j), — и может быть значительно меньше $O(n^2)$.

[13] До появления современных растровых дисплеев в 1960-х годах пиксели обычно назывались стежками или тессерами в зависимости от того, были ли они сделаны из ниток или очень маленьких камней. Слово *pix* стало стандартным сокращением для *picture(s)* в начале XX века, вскоре после того как *sox* стало распространённым множественным числом от *sock*, вытеснив более раннее разговорное *piccy*. См. также *voxel* (элемент объёма), *texel* (элемент текстуры) и *taxel* (осязательный элемент и/или барсук).

Упражнения

Графы

1. Докажите эквивалентность следующих определений:

    - Дерево — связный ациклический граф.
    - Дерево — одна компонента леса. (Лес — ациклический граф.)
    - Дерево — связный граф, содержащий не более V – 1 ребра.
    - Дерево — минимально связный граф: удаление любого ребра нарушает связность.
    - Дерево — ациклический граф, содержащий не менее V – 1 ребра.
    - Дерево — максимально ациклический граф: добавление ребра между любыми двумя вершинами создаёт цикл.
    - Дерево — граф, в котором между каждой парой вершин существует единственный путь.

2. Докажите, что любой связный ациклический граф с $n \geq 2$ вершинами имеет не менее двух вершин степени 1. Не используйте слова «дерево» и «лист» или общеизвестные свойства деревьев; доказательство должно целиком следовать из определений связности и ацикличности.

3. Граф (V, E) называется двудольным, если множество вершин V можно разбить на два подмножества L и R так, что у каждого ребра один конец находится в L, а другой — в R.

    (a) Докажите, что каждое дерево является двудольным графом.

    (b) Докажите, что граф G двудолен тогда и только тогда, когда каждый его цикл содержит чётное число рёбер.

    (c) Опишите и проанализируйте эффективный алгоритм определения, является ли данный неориентированный граф двудольным.

4. Когда собираются группы голубей, они инстинктивно устанавливают порядок подчинения. Из каждой пары голубей один всегда клюёт другого, отгоняя его от еды или потенциальных партнёров. Одна и та же пара всегда придерживается одного порядка подчинения даже после многих лет разлуки и независимо от того, какие голуби находятся рядом. Удивительно, но общий порядок может содержать циклы: например, голубь i клюёт голубя j, тот клюёт k, k клюёт ℓ, а ℓ клюёт i.

    (a) Докажите, что любую конечную популяцию голубей можно выстроить в шествие (а может, в парад?) так, что каждый голубь клюёт сзади идущего перед ним голубя. Пожалуйста-препожалуйста.

    (b) Пусть дан ориентированный граф отношений подчинения n голубей. В графе одна вершина на голубя, а ребро $i \to j$ существует тогда и только тогда, когда голубь i клюёт голубя j. Опишите и проанализируйте алгоритм вычисления порядка голубей, существование которого гарантирует пункт (a).

(c) Докажите, что для любой группы из не менее трёх голубей либо порядок из пункта (a) единственен, либо существуют три голубя i, j и k, такие что i клюёт j, j клюёт k, а k клюёт i.

5. Эйлеров обход графа G — замкнутый маршрут по G, проходящий по каждому ребру G ровно один раз.

   (a) Докажите, что если связный граф G имеет эйлеров обход, то каждая его вершина имеет чётную степень. (Эйлер это доказал.)

   (b) Докажите, что если каждая вершина связного графа G имеет чётную степень, то G имеет эйлеров обход. (Эйлер этого не доказал.)

   (c) Опишите и проанализируйте алгоритм вычисления эйлерова обхода данного графа либо правильного сообщения о том, что такой обход отсутствует. (Здесь Эйлер лишь неопределённо махнул руками.)

6. d-мерный гиперкуб — граф, определяемый следующим образом: имеются 2^d вершин, каждая помечена своей d-битной строкой. Две вершины соединены ребром, если их метки различаются ровно в одном бите.

   (a) Гамильтонов цикл графа G — цикл рёбер G, входящий в каждую вершину G ровно один раз. Докажите, что при всех $d \geq 2$ d-мерный гиперкуб имеет гамильтонов цикл.

   (b) Какие гиперкубы имеют эйлеров обход — замкнутый маршрут, проходящий по каждому ребру ровно один раз? [Подсказка: это очень просто.]

Алгоритмы обхода

7. Напомним, что ориентированный граф G сильно связен, если для любых двух вершин u и v в G есть путь из u в v и путь из v в u.

   Опишите алгоритм, который по данному неориентированному графу G определяет, можно ли придать направление каждому ребру G так, чтобы полученный ориентированный граф был сильно связным.

8. Пусть G — связный граф, а T — остовное дерево поиска в глубину графа G с корнем в узле v. Докажите: если T также является остовным деревом поиска в ширину графа G с корнем v, то G = T.

9. Профессора Эпприч и Гудстайн предлагают следующую оптимизацию общего алгоритма обхода в произвольном порядке. Вместо проверки пометки вершин, извлекаемых из мешка, их алгоритм проверяет её ещё до помещения вершины в мешок, тем самым гарантируя, что каждая вершина попадает в мешок не более одного раза. Алгоритм также назначает родителя каждой вершине при её пометке.

```
EagerWFS(s):
    пометить s
    положить s в мешок
    пока мешок не пуст
        извлечь v из мешка
```

```
        для каждого ребра vw
            если w не помечена
                пометить w
                parent(w) ← v
                положить w в мешок
```

(a) Докажите, что `EagerWFS(s)` помечает все узлы, достижимые из s, и никакие другие. Эквивалентно докажите, что родительские рёбра v → parent(v), вычисленные `EagerWFS(s)`, определяют остовное дерево компоненты, содержащей s.

(b) Докажите, что если мешок реализован очередью, `EagerWFS` эквивалентен поиску в ширину: два алгоритма помечают одни и те же вершины в одном порядке и строят одно и то же остовное дерево. [Подсказка: каково определение очереди?]

(c) Докажите, что `EagerWFS` никогда не эквивалентен поиску в глубину, независимо от структуры данных мешка, в том числе если мешок — стек.

Ни `EagerWFS`, ни `RecursiveDFS` не задают порядок рассмотрения рёбер vw в каждой вершине v, а разные порядки могут давать разные остовные деревья. Поэтому вам нужно показать для некоторого конкретного графа G, что ни одно остовное дерево G, созданное `RecursiveDFS`, не может быть построено `EagerWFS` при любой структуре данных мешка, или наоборот.

10. Одно из самых ранних опубликованных описаний обхода в произвольном порядке как общего класса алгоритмов принадлежит Эдсгеру Дейкстре, Лесли Лэмпорту, Алену Мартину, Карелу Схолтену и Элизабет Стеффенс; оно появилось в 1975 году в рамках разработки автоматического сборщика мусора. Вместо помеченных и непомеченных вершин их алгоритм поддерживает цвет каждой вершины: белый, серый или чёрный. Как обычно, в следующем алгоритме мы предполагаем фиксированный исходный граф G.

```
ThreeColorSearch(s):
    окрасить все узлы в белый
    окрасить s в серый
    пока есть хотя бы одна серая вершина
        ThreeColorStep()

ThreeColorStep():
    v ← любая серая вершина
    если у v нет белых соседей
        окрасить v в чёрный
    иначе
        w ← любой белый сосед v
        parent(w) ← v
        окрасить w в серый
```

(a) Докажите, что `ThreeColorSearch` постоянно поддерживает следующий инвариант: ни одна чёрная вершина не соседствует с белой. [Подсказка: это должно быть просто.]

(b) Докажите, что после завершения `ThreeColorSearch(s)` все вершины, достижимые из s, чёрные, все недостижимые из s вершины белые, а родительские рёбра v → parent(v) задают корневое остовное дерево компоненты, содержащей s.

[Подсказка: интуитивно чёрные узлы «помечены», а серые находятся «в мешке». В отличие от нашей формулировки `WhateverFirstSearch`, однако, трёхцветный алгоритм не обязан обрабатывать все исходящие рёбра узла одновременно.]

(c) Докажите, что следующий вариант `ThreeColorSearch`, поддерживающий множество серых вершин в обычном стеке, эквивалентен поиску в глубину. [Подсказка: порядок двух последних строк `ThreeColorStackStep` важен!]

```
ThreeColorStackSearch(s):
    окрасить все узлы в белый
    окрасить s в серый
    положить s в стек
    пока есть хотя бы одна серая вершина
        ThreeColorStackStep()

ThreeColorStackStep():
    извлечь v из стека
    если у v нет белых соседей
        окрасить v в чёрный
    иначе
        w ← любой белый сосед v
        parent(w) ← v
        окрасить w в серый
        положить v в стек
        положить w в стек
```

(d) Докажите, что следующий вариант `ThreeColorSearch`, поддерживающий множество серых вершин в обычной очереди, не эквивалентен поиску в ширину. [Подсказка: порядок двух последних строк `ThreeColorQueueStep` не важен!]

```
ThreeColorQueueSearch(s):
    окрасить все узлы в белый
    окрасить s в серый
    положить s в очередь
    пока есть хотя бы одна серая вершина
        ThreeColorQueueStep()

ThreeColorQueueStep():
    извлечь v из очереди
    если у v нет белых соседей
        окрасить v в чёрный
    иначе
        w ← любой белый сосед v
        parent(w) ← v
        окрасить w в серый
        положить v в очередь
        положить w в очередь
```

♥ (e) Теперь предположим, что другой процесс добавляет рёбра в G во время работы `ThreeColorSearch`. Эти новые рёбра могут нарушить цветовой инвариант из пункта (a) и тем самым нарушить корректность алгоритма: в частности, при завершении `ThreeColorSearch` некоторые достижимые из s вершины могут остаться белыми. Это было бы катастрофой, если бы мы считали, что «белая» означает «недостижима и, следовательно, может быть безопасно удалена».

Однако если другой процесс явно сохраняет цветовой инвариант, мы по-прежнему можем безопасно использовать трёхцветный алгоритм для выявления недостижимых вершин. Смоделируем два параллельных алгоритма следующим образом: выбор одной из двух ветвей в `GarbageCollect` и выбор вершин u и w в `Mutate` полностью находятся вне контроля основного алгоритма.[14]

```
GarbageCollect(s):
    окрасить все вершины в белый
    окрасить s в серый
    пока есть хотя бы одна серая вершина
        либо
            CollectStep()
        либо
            Mutate()

CollectStep():
    v ← любая серая вершина
    если у v нет белых соседей
        окрасить v в чёрный
    иначе
        w ← любой белый сосед v
        окрасить w в серый

Mutate():
    u ← любая вершина
    w ← любая вершина
    если uw не является ребром
        добавить ребро uw
        если u чёрная и w белая
            окрасить u в серый
        если u белая и w чёрная
            окрасить w в серый
```

Докажите, что `GarbageCollect` в конце концов завершится, причём все достижимые из s вершины будут чёрными, а все недостижимые из s — белыми.

♥ (f) Пусть вместо перекрашивания чёрных вершин в серый `Mutate` поддерживает цветовой инвариант, окрашивая некоторые белые вершины в серый:

```
Mutate():
    u ← любая вершина
    w ← любая вершина
    если uw не является ребром
        добавить ребро uw
        если u чёрная и w белая
            окрасить w в серый
        если u белая и w чёрная
            окрасить u в серый
```

Докажите, что `GarbageCollect` в конце концов завершится, причём s будет чёрной, каждая вершина, достижимая из чёрной вершины, будет чёрной, а каждая вершина, недостижимая из чёрной вершины, будет белой.

[14] Это сильнейшее упрощение алгоритмов сборки мусора «пометка и очистка», действительно используемых в многопоточных языках вроде Lua и Go. К сожалению, более подробное обсуждение многопоточного управления динамической памятью выходит за рамки этой книги, за исключением первой заповеди: не пиши собственный сборщик мусора.

Сведения

11. Числовой лабиринт — сетка n × n из положительных целых чисел. Фишка начинает в левом верхнем углу; цель — переместить её в правый нижний угол. За каждый ход можно передвинуть фишку вверх, вниз, влево или вправо; допустимое расстояние определяется числом в её текущей клетке. Например, если фишка находится на клетке с числом 3, её можно передвинуть на три клетки вверх, вниз, влево или вправо. При этом никогда нельзя выводить фишку за край доски.

Опишите и проанализируйте эффективный алгоритм, который либо возвращает минимальное число ходов для решения данного числового лабиринта, либо правильно сообщает, что решения нет. Например, для лабиринта на рисунке 5.14 алгоритм должен вернуть 8.

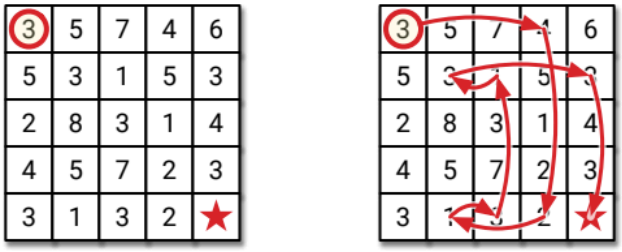


Рисунок 5.14. Числовой лабиринт 5 × 5, решаемый за восемь ходов.

12. «Змеи и лестницы» — классическая настольная игра, появившаяся в Индии не позднее XVI века. Доска — сетка n × n клеток, последовательно пронумерованных от 1 до n^2 начиная с нижнего левого угла, строка за строкой снизу вверх, причём направление нумерации в строках чередуется. Некоторые пары клеток, всегда находящихся в разных строках, соединены «змеями» (ведут вниз) или «лестницами» (ведут вверх). Каждая клетка может быть концом не более одной змеи или лестницы.

Изначально фишка стоит на клетке 1 в нижнем левом углу. За каждый ход вы продвигаете её не более чем на k позиций для некоторой фиксированной константы k. Если фишка завершает ход на верхнем конце змеи, она скользит к её нижнему концу. Аналогично если она завершает ход у нижнего конца лестницы, то поднимается к её верхнему концу.

Опишите и проанализируйте алгоритм вычисления наименьшего числа ходов, за которое фишка достигнет последней клетки доски.

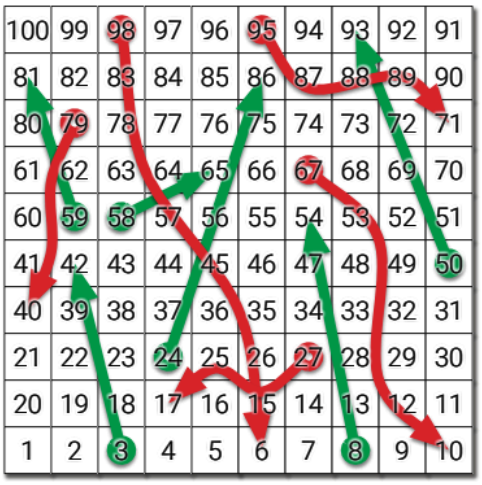


Рисунок 5.15. Доска «Змеи и лестницы». Прямые стрелки вверх — лестницы; волнистые стрелки вниз — змеи.

13. Печально известный монгольский воин-головоломщик Видрач Итки Леда изобрёл следующую головоломку в 1473 году. Она состоит из сетки n × n клеток, каждая с положительным целым числом, и двух фишек — красной и синей. Фишки всегда находятся в разных клетках. Начальные позиции — левый верхний и правый нижний углы; цель — поменять фишки местами.

За один ход можно переместить любую фишку вверх, вправо, вниз или влево на расстояние, определяемое другой фишкой. Например, если красная фишка стоит на клетке с числом 3, синюю можно передвинуть на три клетки вверх, влево, вправо или вниз. Нельзя выводить фишку за пределы сетки; в конце хода две фишки не могут находиться на одной клетке.

Опишите и проанализируйте эффективный алгоритм, который либо возвращает минимальное число ходов, необходимое для решения данной головоломки Видрача Итки Леды, либо правильно сообщает, что решения нет. Например, для головоломки на рисунке 5.16 алгоритм должен вернуть 5.

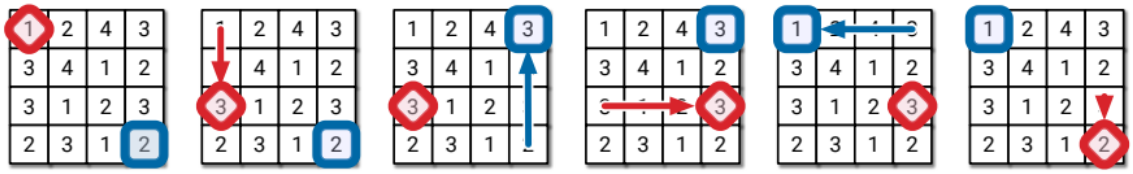
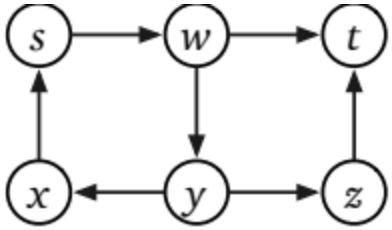


Рисунок 5.16. Решение головоломки Видрача Итки Леды 4 × 4 за пять ходов.

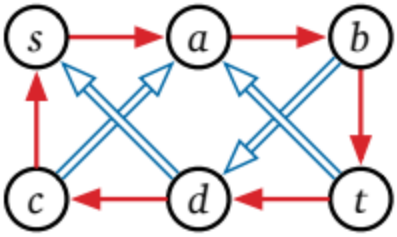
14. Пусть даны ориентированный граф G = (V, E) и две вершины s и t. Опишите и проанализируйте алгоритм, определяющий, существует ли в G маршрут из s в t, возможно с повторением вершин и/или рёбер, длина которого делится на 3.

Например, для графа ниже и указанных вершин s и t алгоритм должен вернуть `True`, поскольку маршрут s → w → y → x → s → w → t имеет длину 6.



15. Пусть дан ориентированный граф G, у которого некоторые рёбра красные, а остальные синие. Опишите алгоритм поиска кратчайшего маршрута из вершины s в вершину t, в котором никакие три последовательных ребра не имеют одного цвета. То есть после двух красных рёбер подряд следующее должно быть синим, а после двух синих подряд — красным.

Например, для показанного графа алгоритм должен вернуть 7, поскольку s → a → b ⇒ d → c ⇒ a → b → t — кратчайший допустимый маршрут из s в t.



16. Рассмотрим ориентированный граф G, каждое ребро которого окрашено в красный, белый или синий. Маршрут в G назовём маршрутом французского флага, если последовательность цветов его рёбер — красный, белый, синий, красный, белый, синий и так далее. Формально маршрут $v_0 \to v_1 \to \cdots \to v_k$ является маршрутом французского флага, если для каждого целого i ребро $v_i \to v_{i+1}$ красное при $i \bmod 3 = 0$, белое при $i \bmod 3 = 1$ и синее при $i \bmod 3 = 2$.

Опишите алгоритм поиска всех вершин G, достижимых из данной вершины v по маршруту французского флага.

17. Имеются n галактик, соединённых m межгалактическими телепортными путями. Каждый путь соединяет две галактики и может использоваться в обоих направлениях. Кроме того, с каждым путём e связана стоимость c(e) долларов, где c(e) — положительное целое число. Путём можно воспользоваться несколько раз, но каждый раз нужно платить.

Джуди хочет попасть из галактики s в галактику t, но телепортация не очень приятна, поэтому она хочет минимизировать число телепортаций. Однако она хочет, чтобы общая стоимость была кратна пяти долларам: носить мелочь тоже неприятно.

(a) Опишите и проанализируйте алгоритм вычисления минимального числа телепортаций Джуди из s в t, при котором общая стоимость кратна пяти долларам.

(b) Решите пункт (a), предполагая, что у Джуди есть купон на ровно одно бесплатное использование телепортного пути.

18. «Три ракушки» — игра для одного игрока на связном неориентированном графе G. Изначально три фишки стоят на различных начальных вершинах a, b, c. Каждый ход необходимо переместить все три фишки, передвигая каждую по ребру из текущей вершины в соседнюю. В конце каждого хода три фишки должны находиться в трёх разных вершинах. Цель — переместить фишки в три целевые вершины x, y, z; какая именно фишка окажется на какой целевой вершине, значения не имеет.

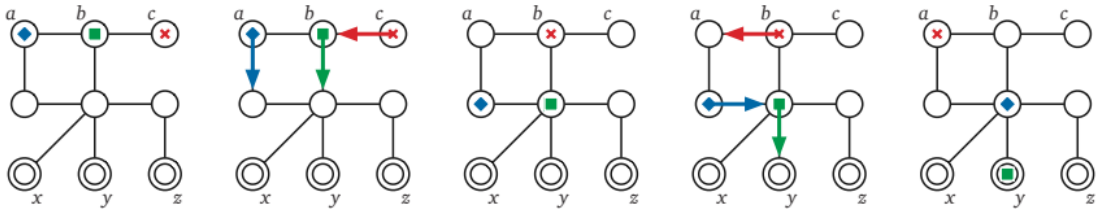


Рисунок 5.17. Начальная конфигурация головоломки «Три ракушки» и первые два хода одного из решений.

Опишите и проанализируйте алгоритм, определяющий, разрешима ли данная головоломка «Три ракушки». На входе — граф G, начальные вершины a, b, c и целевые вершины x, y, z. Результат — один бит: `True` или `False`.

19. Пусть G — связный неориентированный граф. Допустим, мы поставили две монеты в две произвольно выбранные вершины G и хотим переместить их в одну вершину за минимальное число ходов. На каждом шаге каждая монета должна переместиться в соседнюю вершину.

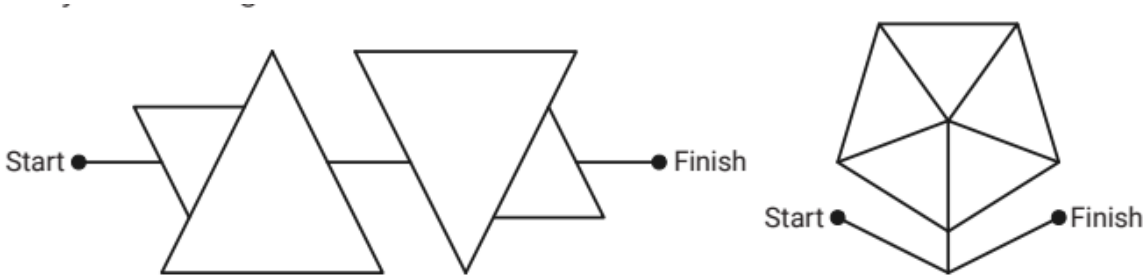
(a) Опишите и проанализируйте алгоритм вычисления минимального числа шагов до конфигурации, в которой обе монеты находятся в одной вершине, либо правильного сообщения о том, что такая конфигурация недостижима. Вход алгоритма — граф $G = (V, E)$ и две вершины $u, v \in V$, которые могут совпадать или не совпадать.

(b) Теперь предположим, что монет три. Опишите и проанализируйте алгоритм вычисления минимального числа шагов до конфигурации, в которой обе монеты находятся в одной вершине, либо правильного сообщения о том, что такая конфигурация недостижима.

(c) Наконец, предположим, что монет сорок две. Опишите и проанализируйте алгоритм определения, можно ли переместить все 42 монеты в одну вершину. Каждая монета снова обязана перемещаться на каждом шаге. Для полного зачёта алгоритм должен работать за $O(V + E)$.

20. В одной из математических рабочих тетрадей моей дочери для начальной школы[15] есть несколько задач следующего типа:

    Пройдите каждый угловой лабиринт ниже, проведя путь от начала до конца, содержащий только острые углы.



Опишите и проанализируйте алгоритм решения произвольных лабиринтов острых углов. Дан связный неориентированный граф G, вершины которого являются точками плоскости, а рёбра — отрезками. Рёбра не пересекаются, кроме как в концах. Например, рисунок буквы X имел бы пять вершин и четыре ребра, а первый приведённый лабиринт имеет 18 вершин и 21 ребро. Также даны две вершины Start и Finish.

Алгоритм должен возвращать `True`, если G содержит маршрут из Start в Finish, имеющий только острые углы, и `False` в противном случае. Формально маршрут по G допустим, если для любых двух последовательных рёбер $u \to v \to w$ выполняется либо $\angle uvw = \pi$, либо $0 < \angle uvw < \pi/2$. Считайте, что есть подпрограмма, которая за O(1) определяет, является ли угол между двумя данными отрезками развёрнутым, тупым, прямым или острым.

[15] Джейсон Баттерсон и Шэннон Роджерс, *Beast Academy Math: Practice 3A*, 2012. Другие примеры см. на https://www.beastacademy.com/resources/printables.php.

21. Пусть даны n горизонтальных и вертикальных отрезков и две точки s и t на плоскости. Опишите эффективный алгоритм определения, существует ли путь из s в t, не пересекающий ни один из данных отрезков.

Каждый горизонтальный отрезок задан левой и правой координатами x и единственной координатой y; аналогично каждый вертикальный отрезок задан единственной координатой x и верхней и нижней координатами y. Точки s и t заданы своими координатами x и y.

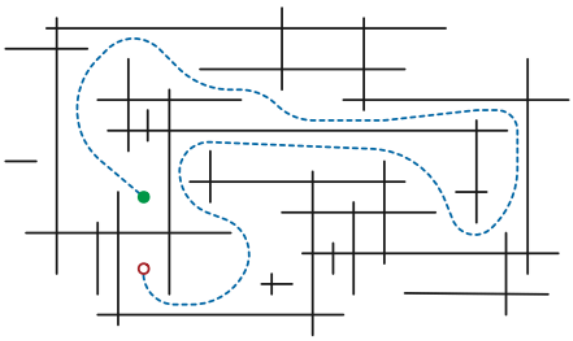


Рисунок 5.18. Путь между двумя точками в лабиринте из горизонтальных и вертикальных отрезков.

22. В каждом пошлом романтическом фильме есть сцена, где после долгой и мучительной разлуки влюблённые внезапно видят друг друга издалека, а затем медленно сближаются, не отводя глаз, пока звучит музыка, заканчивается дождь, солнце выходит из-за облаков,

музыка становится громче, все начинают танцевать, появляются радуги, котята и шоколадные единороги и...[16]

Допустим, влюблённая пара — по славной традиции информатики их зовут Алиса и Боб — входит в любимый парк через восточный и западный входы и немедленно встречается взглядами. Они не могут просто побежать навстречу: им приходится оставаться на зигзагообразной дорожке, проходящей через парк между восточным и западным входами. Для поддержания должного драматического напряжения Алиса и Боб должны идти так, чтобы всегда находиться на одной прямой восток—запад.

Зигзагообразную дорожку можно описать двумя массивами X[0 .. n] и Y[0 .. n], содержащими координаты x и y её поворотов по порядку от юго-западного до юго-восточного конца. Массив X отсортирован по возрастанию, а Y[0] = Y[n]. Дорожка состоит из последовательности отрезков, соединяющих эти повороты.

(a) Пусть Y[0] = Y[n] = 0, а Y[i] > 0 для всех остальных индексов i; то есть концы дорожки находятся строго ниже всех остальных её точек.

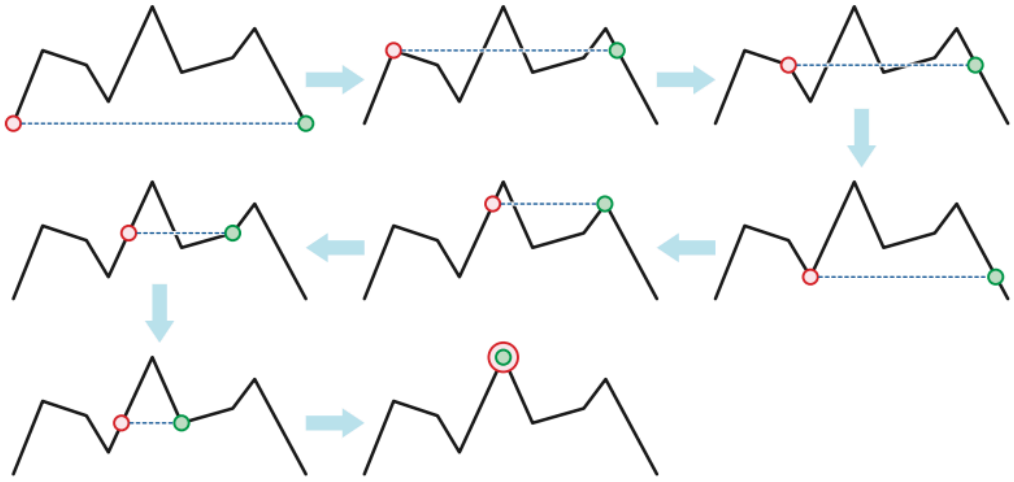


Рисунок 5.19. Алиса и Боб встречаются. Алиса идёт назад на шаге 2, а Боб — на шагах 5 и 6.

Докажите, что для любой дорожки P, удовлетворяющей этим условиям, Алиса и Боб всегда могут встретиться. [Подсказка: опишите граф, моделирующий все возможные положения пары на дорожке. Что является вершинами графа? Что является рёбрами? Используйте лемму о рукопожатиях: в каждом графе чётное число вершин нечётной степени.]

(b) Если концы дорожки не находятся ниже всех остальных её вершин, Алиса и Боб могут встретиться, но не обязательно. Опишите алгоритм, который по массивам X[0 .. n] и Y[0 .. n] определяет, смогут ли они встретиться, не нарушая зрительного контакта по линии восток—запад и не сходя с дорожки.

♥ (c) Опишите алгоритм для пункта (b), работающий за O(n).

[16] Забавный факт: Дэмьен Шазелл, режиссёр фильмов «Одержимость» и «Ла-Ла Ленд», — сын профессора информатики Принстонского университета и электрогитариста Бернара Шазелла.

23. Знаменитый создатель головоломок Каниэль Датчанин изобрёл одиночную игру с двумя фишками на квадратной сетке n × n. Некоторые клетки помечены как препятствия, а одна — как цель. За каждый ход игрок должен передвинуть одну из фишек с текущей позиции как можно дальше вверх, вниз, вправо или влево, остановив её непосредственно перед тем, как она столкнётся с (1) краем доски, (2) клеткой-препятствием или (3) другой фишкой. Цель — привести любую фишку на целевую клетку.

Например, головоломку на рисунке 5.20 можно решить, переместив красную фишку вниз до препятствия, затем зелёную влево до красной, а затем красную влево, вниз, вправо и вверх. На шестом ходе красная фишка остановится на цели, поскольку зелёная находится непосредственно над целевой клеткой.

Опишите и проанализируйте алгоритм определения разрешимости экземпляра этой головоломки. На входе — целое n, список положений препятствий, положение цели и начальные положения фишек. Выход — единственное логическое значение: `True`, если головоломка разрешима, и `False` иначе. [Подсказка: не забудьте время, необходимое для построения графа.]

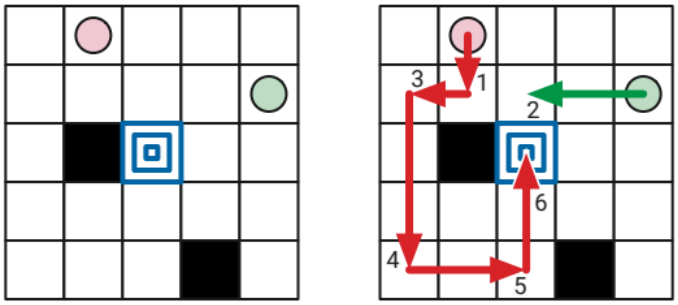


Рисунок 5.20. Головоломка Каниэля Датчанина, решаемая за шесть ходов. Круги обозначают начальные положения фишек; чёрные клетки — препятствия; центральная клетка — цель.

♥ 24. Rectangle Walk — новая абстрактная головоломка, доступная всего за 99 центов на Steam, iOS, Android, Xbox One, PlayStation 5, Nintendo Wii U, Atari 2600, Palm Pilot, Commodore 64, TRS-80, Sinclair ZX-1, DEC PDP-8, PLATO, Zuse Z3, Duramesc, арифмометре Однера, аналитической машине, жаккардовом ткацком станке, Horologium Mirabile Lundense, ступенчатом вычислителе Лейбница, оркестре роботов аль-Джазари, автоматоне Янь Ши, Антикитерском механизме, узелковой верёвке, ишангской кости и куче камней.

Игра проходит на сетке n × n из чёрных и белых клеток. Игрок перемещает по сетке прямоугольник при следующих условиях:

- Прямоугольник должен быть выровнен по сетке: его верхняя, нижняя, левая и правая координаты должны быть целыми.
- Прямоугольник должен помещаться внутри сетки n × n и содержать хотя бы одну клетку.
- Прямоугольник не должен содержать чёрных клеток.
- За один ход можно заменить текущий прямоугольник r любым прямоугольником r′, который либо содержит r, либо содержится в r.

Изначально прямоугольник игрока — клетка 1 × 1 в верхнем правом углу. Цель — достичь клетки 1 × 1 в нижнем левом углу за минимальное число ходов.

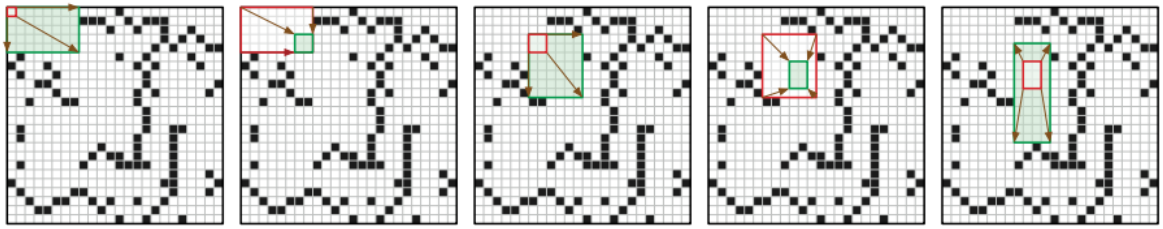


Рисунок 5.21. Первые пять шагов Rectangle Walk.

Опишите и проанализируйте алгоритм вычисления длины кратчайшего маршрута Rectangle Walk по данной растровой карте. На входе — массив M[1 .. n, 1 .. n], где M[i, j] = 1 означает чёрную клетку, а M[i, j] = 0 — белую. Считайте, что допустимый маршрут существует; в частности, M[1, 1] = M[n, n] = 0. Например, для карты выше алгоритм должен вернуть 18. [Подсказка: не забудьте время, необходимое для построения графа!!]

25. Racetrack, также известная как Graph Racers и Vector Rally, — гонка для двух игроков с бумагой и карандашом, в которую Джефф играл в автобусе в пятом классе.[17] Игра проходит на трассе, нарисованной на клетчатой бумаге. Игроки по очереди выбирают последовательность узлов сетки, представляющих движение автомобиля по трассе с соблюдением ограничений ниже.

velocity	position
(0,0)	(1,5)
(1,0)	(2,5)
(2,−1)	(4,4)
(3,0)	(7,4)
(2,1)	(9,5)
(1,2)	(10,7)
(0,3)	(10,10)
(−1,4)	(9,14)
(0,3)	(9,17)
(1,2)	(10,19)
(2,2)	(12,21)
(2,1)	(14,22)
(2,0)	(16,22)
(1,−1)	(17,21)
(2,−1)	(19,20)
(3,0)	(22,20)
(3,1)	(25,21)

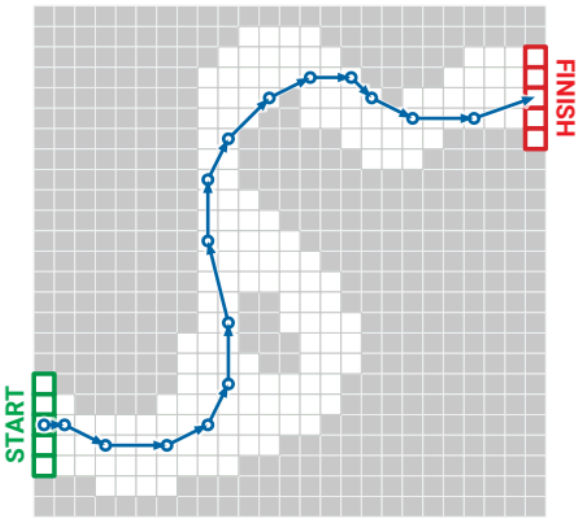


Рисунок 5.22. Прохождение трассы 25 × 25 за 16 шагов. Это не кратчайшее прохождение данной трассы.

Каждый автомобиль имеет положение и скорость, каждое из которых задано целыми координатами x и y. Одно подмножество клеток отмечено как стартовая зона, другое — как финишная. Игрок выбирает начальное положение автомобиля где-нибудь в стартовой зоне; начальная скорость всегда равна (0, 0). На каждом шаге игрок может увеличить или уменьшить

одну или обе координаты скорости на единицу; иначе говоря, каждая компонента скорости может измениться не более чем на 1 за шаг. Новое положение автомобиля определяется прибавлением новой скорости к предыдущему положению. Оно должно находиться внутри трассы; иначе автомобиль разбивается и игрок проигрывает гонку. Гонка заканчивается, когда первый автомобиль достигает позиции внутри финишной зоны.

Пусть трасса представлена массивом битов n × n, в котором каждый бит 0 обозначает точку сетки внутри трассы, каждый бит 1 — точку вне трассы, «стартовая зона» — первый столбец, а «финишная зона» — последний.

Опишите и проанализируйте алгоритм нахождения минимального числа шагов, необходимых автомобилю для прохождения данной трассы от старта до финиша.

[17] Настоящая игра несколько сложнее описанной здесь версии. Отличная онлайн-версия доступна на http://harmmade.com/vectorracer/.

26. Лабиринт с катящимся кубиком — головоломка со стандартной шестигранной костью и сеткой клеток. Представьте сетку лежащей на столе: кость всегда стоит на одной клетке и точно её закрывает. За один шаг можно перекатить кость на 90 градусов через одно из нижних рёбер, переместив её на соседнюю клетку к северу, югу, востоку или западу.

Некоторые клетки заблокированы: на них кость никогда не может остановиться. Другие клетки могут быть помечены числом; когда кость находится на такой клетке, число на её верхней грани должно совпадать с меткой. Клетки без меток и блокировок свободны. За края сетки выкатывать кость нельзя. Лабиринт разрешим, если можно поставить кость на нижнюю левую клетку и докатить до верхней правой, соблюдая эти ограничения.

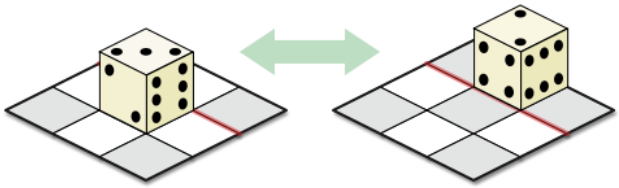


Рисунок 5.23. Перекатывание кости.

На рисунке 5.24 приведены четыре лабиринта. Если использовать стандартную кость с единицей и шестёркой на противоположных гранях, разрешимы только первые два. Например, первый можно решить, поставив кость на нижнюю левую клетку единицей вверх и перекатив её на восток, затем на север, снова на север и на восток.

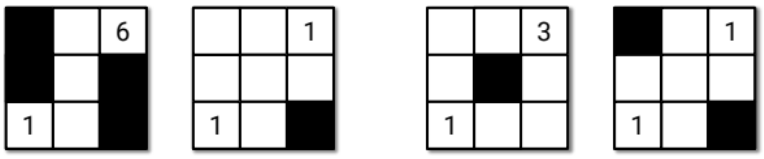


Рисунок 5.24. Четыре лабиринта с катящейся костью; разрешимы только первые два.

(a) Пусть вход — двумерный массив L[1 .. n, 1 .. n], где L[i, j] содержит метку клетки в i-й строке и j-м столбце; 0 означает свободную клетку, а −1 — заблокированную. Опишите и проанализируйте полиномиальный алгоритм определения разрешимости лабиринта.

♥ (b) Теперь пусть лабиринт задан неявно списком помеченных и заблокированных клеток. А именно, вход состоит из целого M, задающего высоту и ширину лабиринта, и массива S[1 .. n], где каждый элемент S[i] — тройка (x, y, L), указывающая, что клетка (x, y) имеет метку L. Как и в явной записи, −1 обозначает заблокированную клетку; свободные клетки в S вообще не перечисляются. Опишите и проанализируйте эффективный алгоритм определения разрешимости. Для полного зачёта время работы должно быть полиномиальным от размера входа n.

[Подсказка: у вас есть некоторый выбор начальной ориентации кости. Существуют лабиринты, разрешимые только при правильном начальном положении.]

♥ 27. Пусть дан произвольный ориентированный граф G, каждое ребро которого окрашено в красный или синий, а также две специальные вершины s и t.

(a) Опишите алгоритм, который либо вычисляет маршрут из s в t с палиндромной последовательностью цветов рёбер, либо правильно сообщает, что такого маршрута нет.

(b) Опишите алгоритм, который либо вычисляет кратчайший маршрут из s в t с палиндромной последовательностью цветов рёбер, либо правильно сообщает, что такого маршрута нет.

[Подсказка: где мы в последний раз видели палиндромы?]

♠♥ 28. Шашки, известные в США как *checkers*, — игра на доске m × m, клетки которой попеременно окрашены в светлый и тёмный цвета.[18] Обычно играют на доске 8 × 8 или 10 × 10, но правила легко обобщить на любой размер. На каждой тёмной клетке находится не более одной игровой фигуры, чёрной или белой; светлые клетки всегда пусты. Один игрок («Белые») ходит белыми фигурами, другой («Чёрные») — чёрными. Игрок проигрывает, когда с доски снята его последняя фигура.

Рассмотрим следующий простой вариант игры, по существу американские или британские шашки, но в котором каждая фигура является дамкой.[19] Фигуры можно перемещать в любом из четырёх диагональных направлений. За каждый ход игрок либо передвигает одну фигуру на одну клетку по диагонали на пустую клетку, либо выполняет ею серию прыжков. При каждом прыжке фигура перемещается на пустую клетку на расстоянии двух шагов по диагонали в любом направлении, но только если промежуточная клетка занята фигурой противоположного цвета; эта вражеская фигура берётся и немедленно удаляется с доски. Все прыжки за один ход должны выполняться одной и той же фигурой.

Опишите алгоритм определения, могут ли Белые за один ход взять все чёрные фигуры и тем самым выиграть. На входе — ширина доски m, список положений белых фигур и список положений чёрных фигур. Для полного зачёта алгоритм должен работать за O(n), где n — общее число фигур. [Подсказка: жадная стратегия — выполнять произвольные прыжки, пока они возможны, — не всегда находит выигрышную последовательность, даже если она существует. См. задачу 5. Чётность, чётность и ещё раз чётность.]

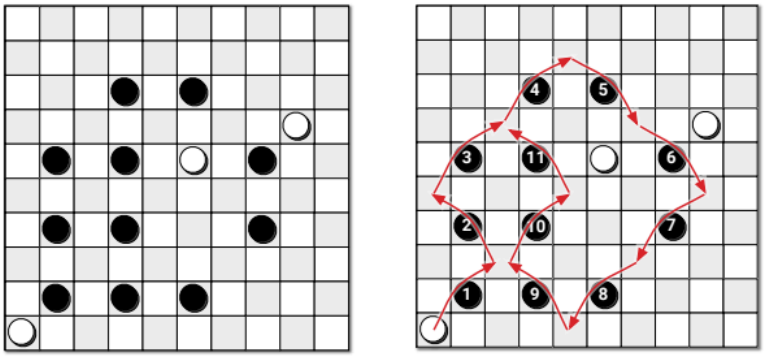


Рисунок 5.25. Белые выигрывают за один ход.

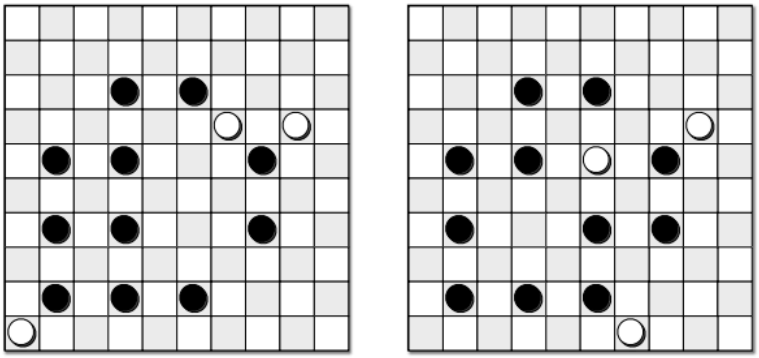


Рисунок 5.26. Белые не могут выиграть за один ход ни из одной из этих позиций.

[18] Счётные столы средневековых английских государственных бухгалтеров покрывали зелёной тканью с чёрными клетками в шахматном порядке; на клетки помещали дискообразные жетоны, обозначавшие величины. Поэтому британских государственных бухгалтеров с X века коллективно называют *Exchequer* — казначейством. Сами счётные столы применялись казначейством для подсчёта налоговых платежей вплоть до XIX века.

[19] В большинстве других разновидностей шашек есть «дальнобойные дамки», которые ходят совсем иначе, чем дамки в британской или американской игре, и значительно усложняют эту задачу, как мы увидим в главе 12.

Глава 6. Поиск в глубину

> *А дом извилист, полон странных ходов и устроен словно лабиринт. Но у меня на уме есть средство: пусть он возьмёт с собой клубок нити и вернётся тем же путём, по которому пришёл, следуя за нитью.*
>
> *— Джеффри Чосер, «Легенда о добрых женщинах» (около 1385 года)*
>
> *«Как прекрасен мир и как безобразны лабиринты!» — с облегчением сказал я.*
>
> *«Как прекрасен был бы мир, существуй правило, по которому можно проходить лабиринты», — ответил мой учитель.*
>
> *— Умберто Эко, «Имя розы» (1980)*

В предыдущей главе мы рассмотрели общий алгоритм обхода произвольных графов, как неориентированных, так и ориентированных, — поиск в произвольном порядке. В этой главе мы сосредоточимся на одном конкретном варианте этого алгоритма, называемом *поиском в глубину*, и прежде всего на его поведении в ориентированных графах.

Хотя поиск в глубину можно точно описать как «поиск в произвольном порядке со стеком», обычно этот алгоритм реализуют рекурсивно, а не с помощью явно заданного стека:

```
DFS(v):
    если v не помечена
        пометить v
        для каждого ребра v → w
            DFS(w)
```

На практике алгоритм можно немного ускорить, проверяя, помечена ли вершина, прежде чем рекурсивно её исследовать. Это изменение гарантирует, что `DFS(v)` будет вызван для каждой вершины v только один раз. Можно также дополнить алгоритм вычислением другой полезной информации о вершинах и рёбрах: для этого введём две подпрограммы — чёрные ящики `PreVisit` и `PostVisit`, которые пока не будем определять.

```
DFS(v):
    пометить v
    PreVisit(v)
    для каждого ребра v → w
        если w не помечена
            parent(w) ← v
            DFS(w)
    PostVisit(v)
```

Напомним, что в ориентированном графе G вершина w *достижима* из вершины v — или, короче, v может достичь w — тогда и только тогда, когда G содержит ориентированный путь из v в w. Пусть reach(v) обозначает множество вершин, достижимых из v, включая саму v. Если

снять пометки со всех вершин G, а затем вызвать `DFS(v)`, множество помеченных вершин будет в точности равно reach(v).

В неориентированных графах достижимость симметрична: v достигает w тогда и только тогда, когда w достигает v. Поэтому, если снять пометки со всех вершин неориентированного графа G, вызов `DFS(v)` обойдёт всю компоненту вершины v, а указатели на родителей определят остовное дерево этой компоненты.

В ориентированных графах ситуация тоньше, как показано на рисунке ниже. Хотя граф «связен», из разных вершин могут быть достижимы разные, возможно пересекающиеся, части графа. Указатели на родителей, назначенные вызовом `DFS(v)`, определяют дерево с корнем v, вершины которого в точности составляют reach(v), но это не обязательно остовное дерево всего графа.

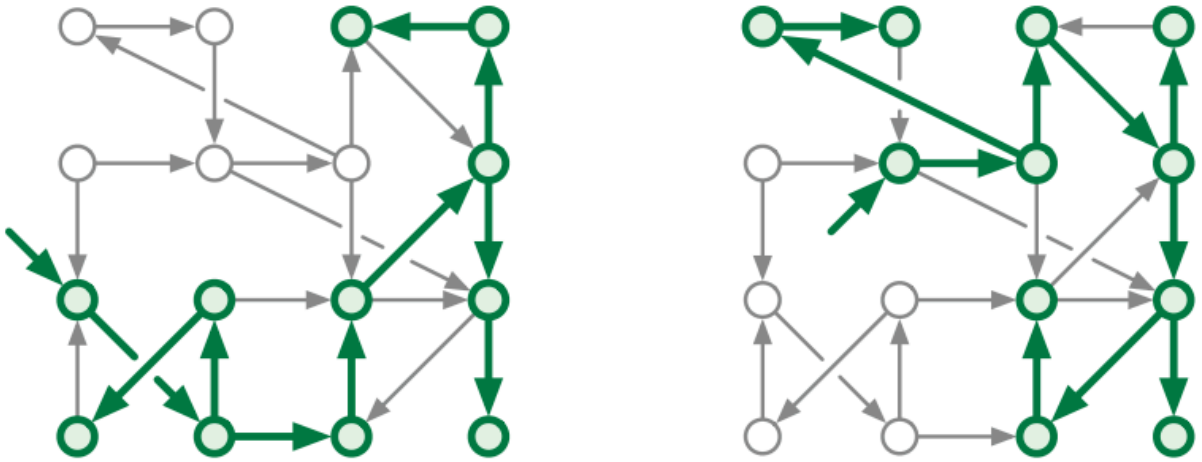


Рис. 6.1. Деревья поиска в глубину с разными корнями в одном ориентированном графе.

Как обычно, мы можем расширить алгоритм проверки достижимости так, чтобы он обходил весь входной граф, даже если тот несвязен, используя стандартную обёртку слева на рис. 6.2. Здесь мы добавляем общую подпрограмму-чёрный ящик `Preprocess`, выполняющую предварительную обработку, необходимую функциям `PreVisit` и `PostVisit`.

```
DFSAll(G):
    Preprocess(G)
    для всех вершин v
        снять пометку с v
    для всех вершин v
        если v не помечена
            DFS(v)
```

```
DFSAll(G):
    Preprocess(G)
    добавить вершину s
    для всех вершин v
        добавить ребро s → v
```

```
        снять пометку с v
    DFS(s)
```

Рис. 6.2. Две формулировки стандартной обёртки поиска в глубину.

В качестве альтернативы, если разрешено изменять граф, можно добавить новую исходную вершину s с рёбрами ко всем остальным вершинам G, а затем один раз вызвать `DFS(s)`, как показано справа на рис. 6.2. Полученные указатели на родителей всегда определяют остовное дерево дополненного входного графа, но не исходного. В остальном обе обёртки ведут себя практически одинаково; выбор между ними — вопрос удобства.[1]

И вновь алгоритм ведёт себя несколько по-разному на неориентированных и ориентированных графах. Как мы видели в предыдущей главе, для неориентированных графов `DFSAll` легко приспособить к подсчёту компонент; в частности, вычисленные ею указатели на родителей определяют остовный лес входного графа, содержащий по остовному дереву для каждой компоненты. В ориентированном графе `DFSAll` может обнаружить любое количество «компонент» от 1 до V, даже если граф «связен»: это зависит от точной структуры графа и порядка, в котором обёртка посещает вершины.

[1] Эквивалентность этих двух обёрток — особое свойство поиска в глубину. В частности, оборачивание поиска в ширину циклом for для посещения каждой вершины не даёт того же порядка обхода, что добавление исходной вершины и вызов поиска в ширину из s.

6.1. Прямой и обратный порядок

Надеюсь, вы уже знакомы с обходом корневых деревьев в прямом и обратном порядке; оба порядка можно вычислить с помощью поиска в глубину. Аналогичные порядки обхода можно определить и для произвольных ориентированных графов, даже несвязных, передавая счётчик, как показано на рис. 6.3. Эквивалентный способ — использовать наш общий алгоритм поиска в глубину со следующими подпрограммами `Preprocess`, `PreVisit` и `PostVisit`.

```
Preprocess(G):
    clock ← 0

PreVisit(v):
    clock ← clock + 1
    v.pre ← clock

PostVisit(v):
    clock ← clock + 1
    v.post ← clock
```

```
DFSAll(G):
    clock ← 0
    для всех вершин v
        снять пометку с v
    для всех вершин v
        если v не помечена
            clock ← DFS(v, clock)

DFS(v, clock):
```

```
    пометить v
    clock ← clock + 1; v.pre ← clock
    для каждого ребра v → w
        если w не помечена
            w.parent ← v
            clock ← DFS(w, clock)
    clock ← clock + 1; v.post ← clock
    вернуть clock
```

Рис. 6.3. Определение прямого и обратного порядка с помощью поиска в глубину.

В любой из двух формулировок алгоритм присваивает `v.pre` и увеличивает счётчик сразу после помещения v в стек рекурсии, а `v.post` присваивает и счётчик увеличивает непосредственно перед извлечением v из стека рекурсии. Отсюда для любых двух вершин u и v интервалы `[u.pre, u.post]` и `[v.pre, v.post]` либо не пересекаются, либо вложены один в другой. Более того, первый интервал содержит второй тогда и только тогда, когда `DFS(v)` вызывается во время выполнения `DFS(u)`, или, что равносильно, u является предком v в итоговом лесе указателей на родителей.

После того как `DFSAll` пометит каждую вершину графа, значения `v.pre` определяют прямой порядок вершин, а значения `v.post` — обратный.[2] За немногими тривиальными исключениями у каждого графа есть несколько различных прямых и обратных порядков, зависящих от порядка рассмотрения рёбер, исходящих из каждой вершины, в `DFS`, а также от порядка рассмотрения вершин в `DFSAll`.

В оставшейся части главы мы будем называть `v.pre` *временем начала* вершины v, или неформально моментом, когда v начинается, `v.post` — её *временем завершения*, или моментом, когда v заканчивается, а интервал между ними — *активным интервалом* v, или временем, пока v активна.

Классификация вершин и рёбер

Во время работы `DFSAll` каждая вершина v входного графа находится в одном из трёх состояний:

- **новая**, если `DFS(v)` ещё не вызывалась, то есть `clock < v.pre`;
- **активная**, если `DFS(v)` уже вызвана, но ещё не вернулась, то есть `v.pre ≤ clock < v.post`;
- **завершённая**, если `DFS(v)` уже вернулась, то есть `v.post ≤ clock`.

Поскольку времена начала и завершения соответствуют помещениям в стек рекурсии и извлечениям из него, вершина активна тогда и только тогда, когда она находится в стеке рекурсии. Отсюда активные вершины всегда образуют ориентированный путь в G.

[2] К сожалению, оба этих порядка иногда называют «порядком поиска в глубину». Пожалуйста, не делайте этого.

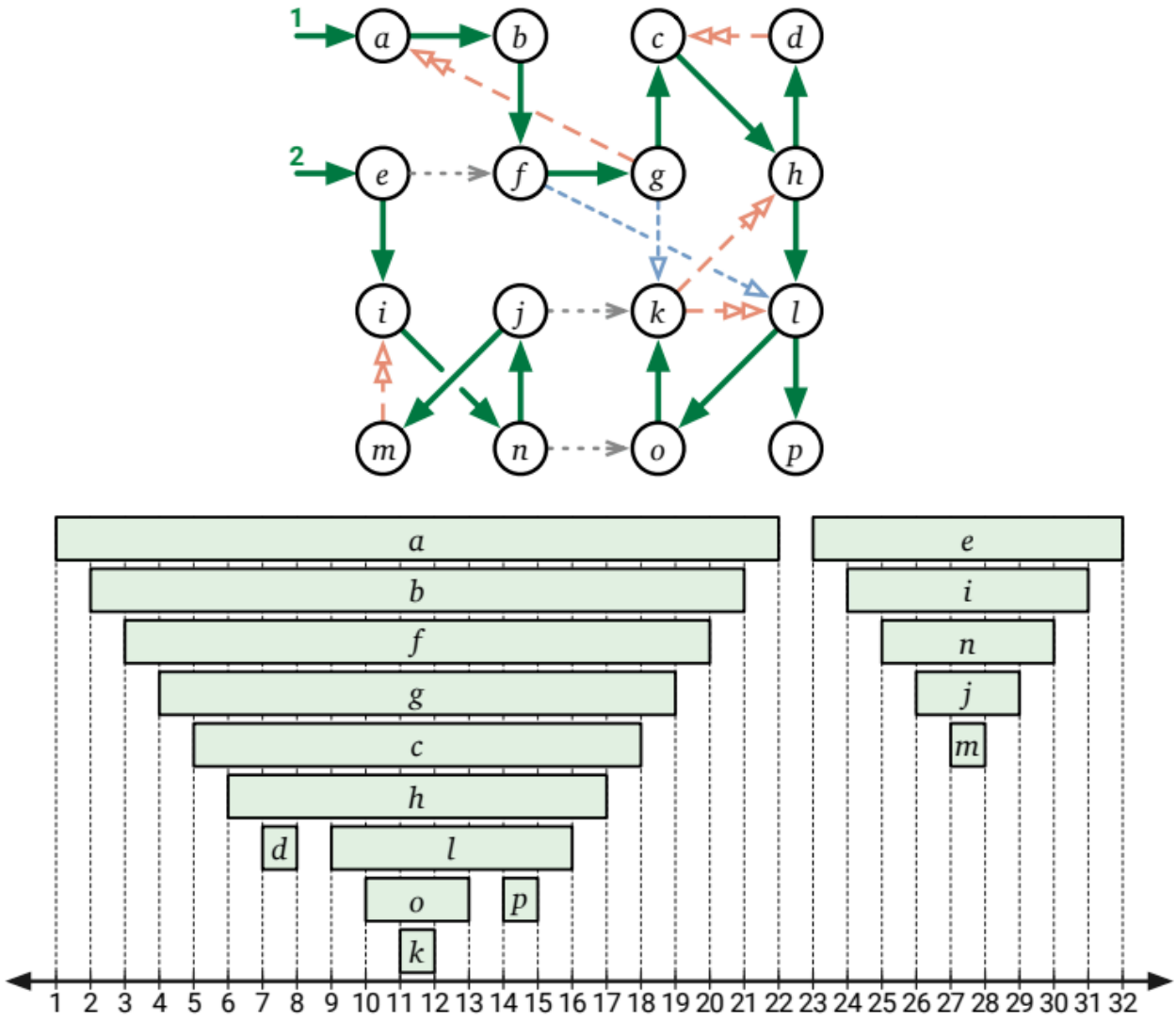


Рис. 6.4. Лес поиска в глубину ориентированного графа и соответствующие активные интервалы вершин, определяющие прямой порядок abfgchdlokpeinjm и обратный порядок dkoplhcgfbamjnie. Рёбра леса показаны сплошными линиями; пунктирные рёбра объясняются на рис. 6.5.

Рёбра входного графа делятся на четыре класса в зависимости от того, как пересекаются активные интервалы их концов. Выберем произвольное ребро u → v.

- Если v — новая вершина в начале `DFS(u)`, то `DFS(v)` обязательно будет вызвана во время выполнения `DFS(u)`, непосредственно или через промежуточные рекурсивные вызовы. В обоих случаях u — собственный предок v в лесе поиска в глубину и u.pre < v.pre < v.post < u.post.
    - Если `DFS(u)` вызывает `DFS(v)` непосредственно, то u = v.parent, а ребро u → v называется **ребром дерева**.
    - В противном случае u → v называется **прямым ребром**.
- Если v активна в начале `DFS(u)`, то v уже находится в стеке рекурсии, а интервалы вложены в обратном порядке: v.pre < u.pre < u.post < v.post. Более того, G должен

содержать ориентированный путь из v в u. Такие рёбра называются **обратными**.
- Если v завершена в начале `DFS(u)`, то непосредственно получаем v.post < u.pre. Такие рёбра называются **поперечными**.
- Наконец, четвёртый порядок u.post < v.pre невозможен.

Эти классы рёбер показаны на рис. 6.5. Фактическая классификация рёбер опять-таки зависит от порядка, в котором `DFSAll` рассматривает вершины, и порядка, в котором `DFS` рассматривает исходящие из каждой вершины рёбра.

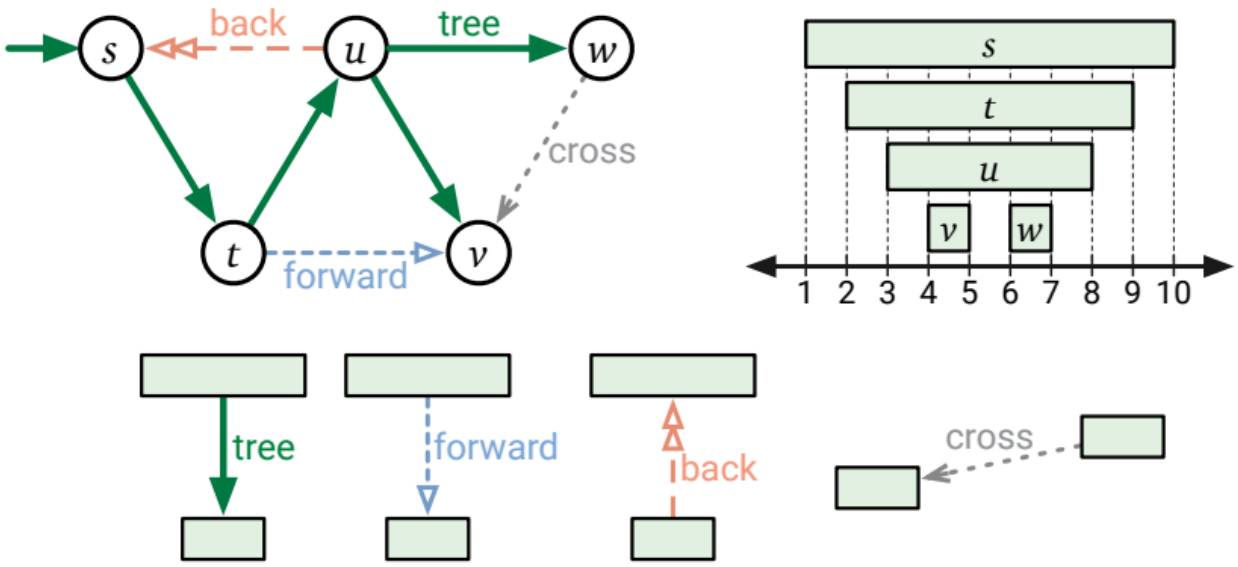


Рис. 6.5. Классификация рёбер при поиске в глубину: tree — ребро дерева, forward — прямое, back — обратное, cross — поперечное.

Наконец, следующая ключевая лемма характеризует предков и потомков в любом лесе поиска в глубину через состояния вершин во время обхода.

Лемма 6.1. Зафиксируем произвольный обход в глубину ориентированного графа G. Для любых вершин u и v графа G следующие утверждения эквивалентны:

1. u — предок v в лесе поиска в глубину.
2. u.pre ≤ v.pre < v.post ≤ u.post.
3. Сразу после вызова `DFS(v)` вершина u активна.
4. Непосредственно перед вызовом `DFS(u)` существует путь из u в v, каждая вершина которого, включая u и v, является новой.

Доказательство. Сначала предположим, что u — предок v в лесе поиска в глубину. Тогда по определению существует путь P из рёбер дерева от u до v. По индукции по длине пути для каждой вершины w на P имеем u.pre ≤ w.pre < w.post ≤ u.post, поэтому каждая вершина P является новой перед вызовом `DFS(u)`. В частности, u.pre ≤ v.pre < v.post ≤ u.post, откуда следует, что u активна во время выполнения `DFS(v)`.

Поскольку указатели на родителей соответствуют рекурсивным вызовам, из u.pre ≤ v.pre < v.post ≤ u.post следует, что u — предок v.

Предположим, что u активна сразу после вызова `DFS(v)`. Тогда u.pre ≤ v.pre < v.post ≤ u.post, откуда следует существование пути из нуля или более рёбер дерева от u через промежуточные вершины стека рекурсии, если они есть, к v.

Наконец, предположим, что u не является предком v. Выберем произвольный путь P из u в v, обозначим через x первую вершину P, не являющуюся потомком u, а через w — предшественника x на P. Ребро w → x гарантирует x.pre < w.post; поскольку w — потомок u, имеем w.post < u.post, а значит, x.pre < u.post. Отсюда x.pre < u.pre: в противном случае x была бы потомком u. Поскольку активные интервалы правильно вложены друг в друга, есть лишь две возможности:

- Если u.post < x.post, то x активна в момент вызова `DFS(u)`.
- Если x.post < u.pre, то x уже завершена в момент вызова `DFS(u)`.

Следовательно, каждый путь из u в v содержит вершину, которая не является новой при вызове `DFS(u)`. □

6.2. Обнаружение циклов

Ориентированный ациклический граф, или DAG, — это ориентированный граф без ориентированных циклов. Вершина DAG, в которую не входит ни одно ребро, называется **истоком**; вершина, из которой не выходит ни одного ребра, — **стоком**. Изолированная вершина, не имеющая инцидентных рёбер, одновременно является истоком и стоком. У любого DAG есть хотя бы один исток и хотя бы один сток, но их может быть больше. Например, в графе с n вершинами и без рёбер каждая вершина является и истоком, и стоком.

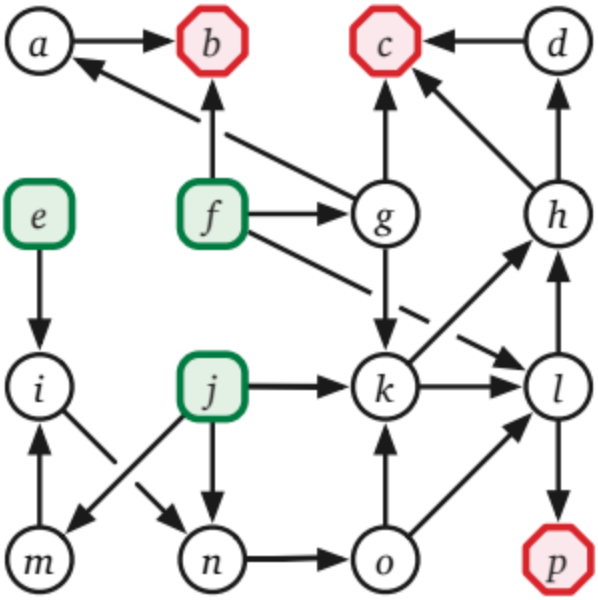


Рис. 6.6. Ориентированный ациклический граф. Вершины e, f и j — истоки; b, c и p — стоки.

Напомним из предыдущего разбора случаев: если для ребра u → v выполнено u.post < v.post, граф содержит ориентированный путь из v в u и, следовательно, ориентированный цикл, проходящий через ребро u → v. Поэтому можно определить, является ли ориентированный граф G ациклическим, за время O(V + E): вычислить обратный порядок вершин, затем перебором проверить каждое ребро.

Вместо нумерации вершин можно явно поддерживать состояние каждой вершины и немедленно возвращать `False`, обнаружив ребро к активной вершине. Этот алгоритм также работает за O(V + E); см. рис. 6.7.

```
IsAcyclic(G):
    для всех вершин v
        v.status ← New
    для всех вершин v
        если v.status = New
            если IsAcyclicDFS(v) = False
                вернуть False
    вернуть True

IsAcyclicDFS(v):
    v.status ← Active
    для каждого ребра v → w
        если w.status = Active
            вернуть False
        иначе если w.status = New
            если IsAcyclicDFS(w) = False
                вернуть False
    v.status ← Finished
    вернуть True
```

Рис. 6.7. Алгоритм линейной сложности для определения ацикличности графа.

6.3. Топологическая сортировка

Топологический порядок ориентированного графа G — это полный порядок ≺ на вершинах, такой что u ≺ v для каждого ребра u → v. Менее формально, вершины располагаются на горизонтальной линии так, чтобы все рёбра были направлены слева направо. Очевидно, топологический порядок невозможен, если граф G содержит ориентированный цикл: самое правое ребро цикла должно было бы указывать влево!

Теперь рассмотрим произвольный обратный порядок произвольного ориентированного графа G. Из предыдущего анализа следует, что если для ребра u → v выполнено u.post < v.post, то G содержит ориентированный путь из v в u и, следовательно, ориентированный цикл через ребро u → v. Равносильно, если G ацикличен, то u.post > v.post для каждого ребра u → v. Следовательно, любой ориентированный ациклический граф G имеет топологический порядок; в частности, обращение любого обратного порядка G является топологическим порядком G.

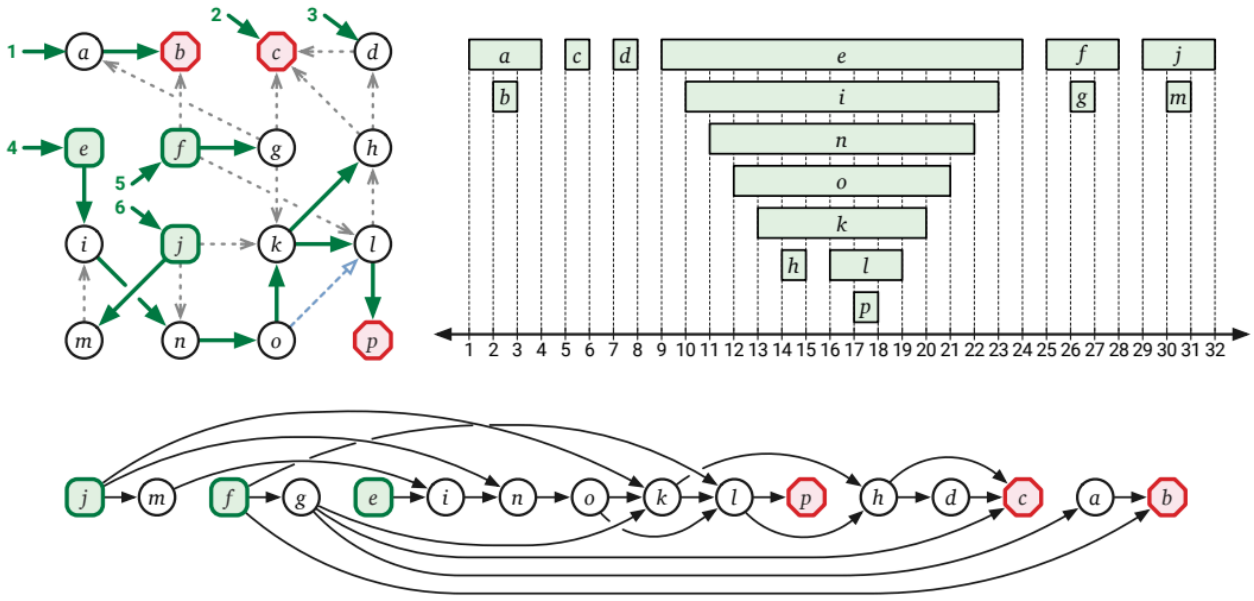


Рис. 6.8. Обращённый обратный порядок DAG с рис. 6.6.

Если топологический порядок нужен в отдельной структуре данных, достаточно записать вершины в массив в порядке, обратном обратному, за время O(V + E), как показано на рис. 6.9.

```
TopologicalSort(G):
    для всех вершин v
        v.status ← New
    clock ← V
    для всех вершин v
        если v.status = New
            clock ← TopSortDFS(v, clock)
    вернуть S[1 .. V]

TopSortDFS(v, clock):
    v.status ← Active
    для каждого ребра v → w
        если w.status = New
            clock ← TopSortDFS(w, clock)
        иначе если w.status = Active
            корректно завершить с ошибкой
    v.status ← Finished
    S[clock] ← v
    clock ← clock – 1
    вернуть clock
```

Рисунок 6.9. Явная топологическая сортировка.

Неявная топологическая сортировка

Однако запись топологического порядка в отдельную структуру данных обычно излишня. В большинстве применений топологической сортировки упорядоченный список вершин сам по себе не является целью; вместо этого мы хотим выполнить некоторое фиксированное

вычисление в каждой вершине графа — в топологическом порядке или в обратном ему. Для таких применений записывать топологический порядок вообще не нужно!

Чтобы обработать ориентированный ациклический граф в обратном топологическом порядке, достаточно обработать каждую вершину в конце её рекурсивного поиска в глубину. Ведь топологический порядок — это обращённый обратный порядок!

```
PostProcess(G):
    для всех вершин v
        v.status ← New
    для всех вершин v
        если v не помечена
            PostProcessDFS(v)

PostProcessDFS(v):
    v.status ← Active
    для каждого ребра v → w
        если w.status = New
            PostProcessDFS(w)
        иначе если w.status = Active
            корректно завершить с ошибкой
    v.status ← Finished
    Process(v)
```

Если заранее известно, что входной граф ацикличен, можно дополнительно упростить алгоритм, помечая вершины вместо записи состояния поиска:

```
PostProcessDag(G):
    для всех вершин v
        снять пометку с v
    для всех вершин v
        если v не помечена
            PostProcessDagDFS(s)

PostProcessDagDFS(v):
    пометить v
    для каждого ребра v → w
        если w не помечена
            PostProcessDagDFS(w)
    Process(v)
```

> *Примечание переводчика: в вызове* `PostProcessDagDFS(s)` *в оригинале, вероятно, опечатка: следует передать текущую вершину* `v`*. В предыдущем варианте проверка непомеченности означает состояние* `New`*.*

Это обычный поиск в глубину, в котором `PostVisit` переименован в `Process`!

Поскольку такая обработка ориентированных ациклических графов используется очень часто, иногда я записываю её идиоматически так:

```
PostProcessDag(G):
    для всех вершин v в обратном порядке обхода
        Process(v)
```

Например, прежний алгоритм явной топологической сортировки можно записать так:

```
TopologicalSort(G):
    clock ← V
    для всех вершин v в обратном порядке обхода
        S[clock] ← v
        clock ← clock – 1
    вернуть S[1 .. V]
```

Чтобы обработать DAG в прямом топологическом порядке, можно записать порядок вершин в массив и затем выполнить простой цикл. Или применить поиск в глубину к обращению G, обозначаемому $\mathrm{rev}(G)$ и получаемому заменой каждого ребра $v \to w$ на обратное $w \to v$. Обращение ориентированного цикла даёт ориентированный цикл противоположного направления, поэтому обращение DAG тоже является DAG. Каждый источник в G — сток в $\mathrm{rev}(G)$ и наоборот; отсюда по индукции любой топологический порядок $\mathrm{rev}(G)$ является обращением некоторого топологического порядка G.[3] Обращение любого ориентированного графа, представленного стандартными списками смежности, можно вычислить за $O(V + E)$; детали оставляются как простое упражнение.

[3] Обратный порядок обхода обращения G не обязательно совпадает с обращением обратного порядка обхода G, хотя оба являются топологическими порядками G.

6.4. Мемоизация и динамическое программирование

Наш алгоритм топологической сортировки, пожалуй, является моделью для широкого класса алгоритмов динамического программирования. Напомним: *граф зависимостей* рекуррентного соотношения содержит вершину для каждой рекурсивной подзадачи и ребро от одной подзадачи к другой, если для вычисления первой требуется рекурсивно вычислить вторую. Граф зависимостей должен быть ацикличен, иначе наивный рекурсивный алгоритм никогда не завершится.

Вычисление любого рекуррентного соотношения с мемоизацией в точности соответствует поиску в глубину по графу зависимостей. В частности, вершина графа «помечена», если значение соответствующей подзадачи уже вычислено. Подпрограммы-чёрные ящики `PreVisit` и `PostVisit` представляют фактическое вычисление значения. См. рисунок 6.10.

```
Memoize(x):                         DFS(v):
    если value[x] не определено         если v не помечена
        инициализировать value[x]           пометить v
        для всех подзадач y задачи x        PreVisit(x)
            Memoize(y)                      для всех рёбер v → w
            обновить value[x]                   DFS(w)
                на основе value[y]          PostVisit(x)
        завершить вычисление value[x]
```

Рисунок 6.10. Рекурсия с мемоизацией — это поиск в глубину. Поиск в глубину — это рекурсия с мемоизацией.

Примечание переводчика: в правом фрагменте оригинала в вызовах `PreVisit(x)` *и* `PostVisit(x)`*, вероятно, опечатка: параметр функции называется* `v`.

Продолжая аналогию, вычисление соотношения динамическим программированием соответствует вычислению всех подзадач графа зависимостей в обратном топологическом порядке: каждая рассматривается после подзадач, от которых зависит. Поэтому любой алгоритм динамического программирования эквивалентен обходу графа зависимостей лежащего в его основе рекуррентного соотношения в обратном порядке!

```
DynamicProgramming(G):
    для всех подзадач x в обратном порядке обхода
        инициализировать value[x]
        для всех подзадач y задачи x
            обновить value[x] на основе value[y]
        завершить вычисление value[x]
```

Рисунок 6.11. Динамическое программирование — это обход в обратном порядке.

Однако большинство алгоритмов динамического программирования немного отличается от топологической сортировки. Во-первых, граф зависимостей обычно неявный: вершины и рёбра не хранятся явно в памяти, а закодированы самим соотношением. Но это небольшое отличие: если каждую рекурсивную подзадачу можно перечислить за постоянное время, граф можно обойти точно так же, как граф, явно заданный списками смежности.

Важнее то, что графы зависимостей большинства соотношений имеют чёткую структуру. Например, как обсуждалось в главе 5, граф зависимости для редакционного расстояния — регулярная сетка с диагоналями, а для оптимальных двоичных деревьев поиска — верхняя треугольная сетка со всеми возможными рёбрами вправо и вверх. Благодаря этой структуре подходящий порядок вычисления можно непосредственно встроить в алгоритм, обычно в виде вложенных циклов; сортировать граф зависимостей при выполнении не нужно. Ранее мы называли обратный топологический порядок *порядком вычисления*.

Динамическое программирование на DAG

И наоборот, поиск в глубину позволяет строить алгоритмы динамического программирования для менее структурированных графов зависимостей. Рассмотрим задачу о длиннейшем пути: требуется найти путь максимального суммарного веса между вершинами s и t ориентированного графа G со взвешенными рёбрами. В произвольном ориентированном графе задача NP-трудна — это легко показать сведением задачи коммивояжёра, см. главу 12, — но для ацикличного входного графа длиннейший путь можно найти за линейное время следующим образом.

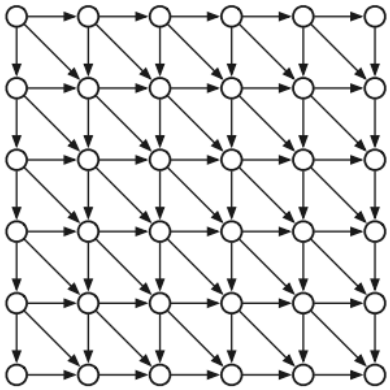


Рисунок 6.12. Ориентированный ациклический граф зависимостей рекуррентного соотношения для редакционного расстояния.

Зафиксируем конечную вершину t и для любой вершины v обозначим через $\mathrm{LLP}(v)$ длину длиннейшего пути из v в t. Если G — DAG, функция удовлетворяет соотношению

$$\mathrm{LLP}(v) = \begin{cases} 0 & \text{если } v = t, \\ \max\{\ell(v \to w) + \mathrm{LLP}(w) \mid v \to w \in E\} & \text{иначе,} \end{cases}$$

где $\ell(v \to w)$ — заданный вес, или «длина», ребра, а $\max \varnothing = -\infty$. В частности, если v — сток, не равный t, то $\mathrm{LLP}(v) = -\infty$.

Граф зависимостей этого соотношения — сам входной граф G: подзадача `LLP(v)` зависит от `LLP(w)` тогда и только тогда, когда $v \to w$ является ребром G. Следовательно, функцию можно вычислить за $O(V + E)$ поиском в глубину из s. Алгоритм мемоизирует длину `LLP(v)` в дополнительном поле соответствующей вершины v.

```
LongestPath(v, t):
    если v = t
        вернуть 0
    если v.LLP не определено
        v.LLP ← −∞
        для каждого ребра v → w
            v.LLP ← max{v.LLP, ℓ(v → w) + LongestPath(w, t)}
    вернуть v.LLP
```

В принципе этот рекурсивный алгоритм с мемоизацией можно преобразовать в динамическое программирование через топологическую сортировку:

```
LongestPath(s, t):
    для каждой вершины v в обратном порядке обхода
        если v = t
            v.LLP ← 0
        иначе
            v.LLP ← −∞
            для каждого ребра v → w
                v.LLP ← max{v.LLP, ℓ(v → w) + w.LLP}
    вернуть s.LLP
```

Можно утверждать, что эти алгоритмы одинаковы: рекурсия первого и цикл второго представляют «один и тот же» поиск в глубину! Выбор формулировки — исключительно вопрос удобства.

Почти любую задачу динамического программирования об оптимальной последовательности решений можно переформулировать как поиск оптимального пути в некотором DAG. Например, рассмотренные в главах 2 и 3 задачи разбиения текста, суммы подмножества, наибольшей возрастающей подпоследовательности и редакционного расстояния сводятся к поиску длиннейшего или кратчайшего пути в DAG, возможно, со взвешенными вершинами или рёбрами. В каждом случае этот DAG — граф зависимостей соответствующего соотношения. С другой стороны, «древовидные» задачи динамического программирования, например об оптимальных двоичных деревьях поиска или максимальных независимых множествах деревьев, нельзя переформулировать как поиск оптимального пути в DAG.

6.5. Сильная связность

Вернёмся к правильному определению связности в ориентированных графах. Вершина u может *достичь* вершины v в ориентированном графе G, если в нём есть ориентированный путь из u в v; $\text{reach}(u)$ обозначает множество вершин, достижимых из u. Вершины u и v *сильно связны*, если u достигает v, а v достигает u. Ориентированный граф сильно связен тогда и только тогда, когда сильно связна каждая пара его вершин.

Утомительное разбирательство с определениями показывает: сильная связность — отношение эквивалентности на множестве вершин любого ориентированного графа, как обычная связность для неориентированных графов. Классы эквивалентности называются *компонентами сильной связности*, или просто сильными компонентами, графа G. Эквивалентно сильная компонента — максимальный сильно связный подграф G. Граф сильно связен тогда и только тогда, когда у него ровно одна сильная компонента. На другом полюсе: граф является DAG тогда и только тогда, когда каждая сильная компонента состоит из одной вершины.

Граф сильных компонент $\text{scc}(G)$ — другой ориентированный граф, получаемый стягиванием каждой сильной компоненты в одну вершину и объединением параллельных рёбер. Его также называют метаграфом или конденсацией G. Нетрудно доказать — намёк, намёк, — что $\text{scc}(G)$ всегда является DAG. Поэтому хотя бы принципиально сильные компоненты G можно топологически упорядочить: вершины можно расположить так, чтобы каждое обратное ребро соединяло две вершины одной сильной компоненты.

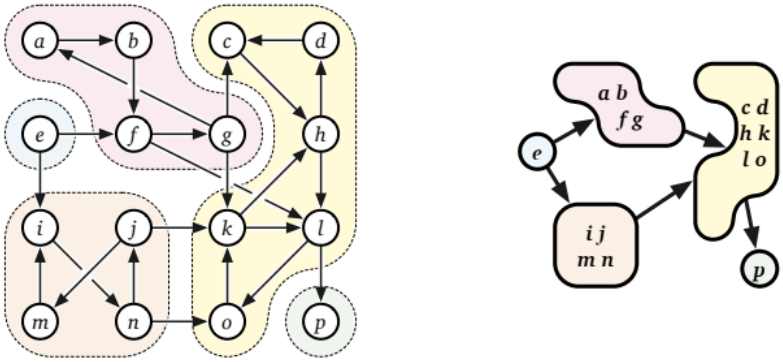


Рисунок 6.13. Сильные компоненты графа G и граф сильных компонент $\mathrm{scc}(G)$.

Вычислить сильную компоненту отдельной вершины v за $O(V + E)$ несложно. Сначала произвольным поиском находим $\mathrm{reach}(v)$. Затем поиском в обращённом графе находим $\mathrm{reach}^{-1}(v) = \{u \mid v \in \mathrm{reach}(u)\}$. Наконец, компонента v — пересечение $\mathrm{reach}(v) \cap \mathrm{reach}^{-1}(v)$. В частности, за $O(V + E)$ можно определить, сильно ли связен весь граф.

Аналогично можно вычислить все сильные компоненты, объединив предыдущий алгоритм с нашей стандартной обёрткой. Но результат работает за $O(VE)$: компонент не более V, и на обнаружение каждой требуется $O(E)$, даже если граф — DAG. Наверняка можно лучше! Ведь чтобы проверить, состоит ли каждая компонента из одной вершины, требуется лишь $O(V + E)$.

6.6. Сильные компоненты за линейное время

На самом деле есть несколько алгоритмов вычисления сильных компонент за $O(V + E)$; все они основаны на следующем наблюдении.

Лемма 6.2. Зафиксируем обход в глубину произвольного ориентированного графа G. Каждая его сильная компонента C содержит ровно одну вершину, у которой нет родителя в C. У этой вершины либо родитель в другой компоненте, либо родителя нет вообще.

Доказательство. Пусть C — произвольная сильная компонента. Рассмотрим любой путь из $v \in C$ в $w \in C$. Каждая вершина пути достигает w, а значит, и любой вершины C; симметрично, каждая вершина пути достижима из v, а значит, и из любой вершины C. Поэтому все вершины пути также принадлежат C.

Пусть v — вершина C с самым ранним временем начала. Если у неё есть родитель, он начинается раньше v и потому не принадлежит C.

Пусть w — другая вершина C. Непосредственно перед вызовом `DFS(v)` все вершины C новые, поэтому из v в w есть путь из новых вершин. По лемме 6.1 w — потомок v в лесу поиска. Каждая вершина пути из рёбер дерева от v к w лежит в C; в частности, $\mathrm{parent}(w) \in C$. □

Из леммы следует, что каждая сильная компонента ориентированного G задаёт связное поддерево любого леса поиска в глубину. В частности, для каждой компоненты C вершина с

самым ранним временем начала является наименьшим общим предком всех вершин C; назовём её *корнем* компоненты.

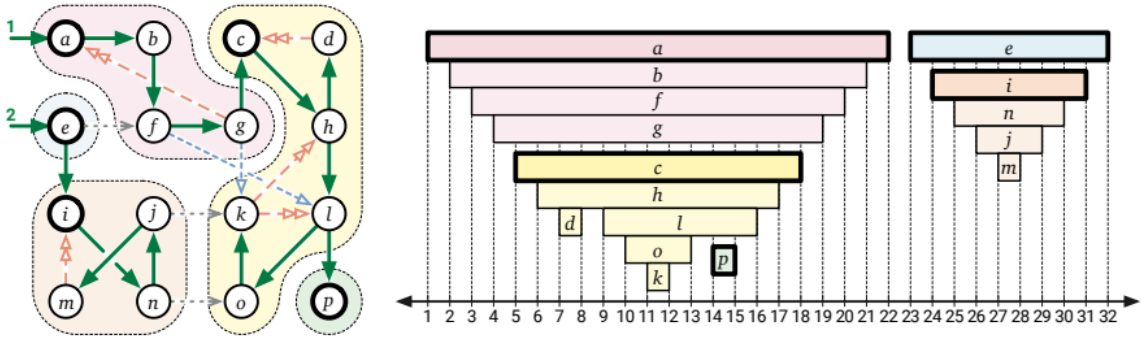


Рисунок 6.14. Сильные компоненты непрерывны в лесе поиска в глубину.

Я представлю два алгоритма, следующих одной интуитивной схеме. Пусть C — сильная компонента G, являющаяся стоком в $\mathrm{scc}(G)$; назовём её *стоковой компонентой*. Эквивалентно, множество достижимых вершин для любой вершины C равно в точности C. Все компоненты можно найти, повторяя: найти некоторую вершину v в стоковой компоненте каким-то способом, найти достижимые из неё вершины и удалить эту компоненту из входного графа. Остановиться, когда вершин не останется. Это ещё не вполне алгоритм: неясно, как найти вершину стоковой компоненты!

```
StrongComponents(G):
    count ← 0
    пока G не пуст
        C ← ∅
        count ← count + 1
        v ← любая вершина стоковой компоненты G  ⟨⟨Волшебство!⟩⟩
        для всех вершин w из reach(v)
            w.label ← count
            добавить w в C
        удалить C и входящие в неё рёбра из G
```

Рисунок 6.15. Почти алгоритм вычисления сильных компонент.

Алгоритм Косарайю — Шарира

На первый взгляд быстро найти вершину стоковой компоненты сложно. Но с помощью поиска в глубину на самом деле довольно просто найти вершину *исходной компоненты* — сильной компоненты G, соответствующей источнику в $\mathrm{scc}(G)$.

Лемма 6.3. Последняя вершина любого обратного порядка обхода G находится в исходной компоненте G.

Доказательство. Зафиксируем обход G в глубину; пусть v — последняя вершина его обратного порядка. Тогда `DFS(v)` — последний прямой вызов `DFS` алгоритмом-обёрткой `DFSAll`. Более того, v — корень одного из деревьев леса, поэтому любой узел x с $x.\mathrm{post} > v.\mathrm{pre}$ является потомком v. Наконец, v — корень своей сильной компоненты C.

Предположим для рассуждения, что есть ребро $x \to y$, где $x \notin C$, а $y \in C$. Тогда x достигает y, а y достигает v, значит, x достигает v. Поскольку v — корень C, вершина y является её потомком, так что $v.\mathrm{pre} < y.\mathrm{pre}$. Из ребра $x \to y$ следует $y.\mathrm{pre} < x.\mathrm{post}$ и, следовательно, $v.\mathrm{pre} < x.\mathrm{post}$. Значит, x — потомок v. Но тогда v достигает x по рёбрам дерева, что противоречит $x \notin C$. $\square$

Легко проверить — намёк, намёк, — что для любого ориентированного G выполнено $\mathrm{rev}(\mathrm{scc}(G)) = \mathrm{scc}(\mathrm{rev}(G))$. Значит, последняя вершина обратного порядка обхода $\mathrm{rev}(G)$ лежит в стоковой компоненте исходного G. Если обойти граф второй раз, а обёртка будет следовать обращённому обратному порядку обхода $\mathrm{rev}(G)$, каждый вызов `DFS` посетит ровно одну сильную компоненту G.[4]

Объединив всё, получаем алгоритм на рисунке 6.16: он подсчитывает и помечает сильные компоненты произвольного ориентированного графа за $O(V + E)$. Его открыл, но не опубликовал, Рао Косарайю в 1978 году; затем в 1981 году Миха Шарир независимо открыл его вновь.[5] Алгоритм состоит из двух фаз. Сначала поиск в глубину обходит $\mathrm{rev}(G)$ и помещает каждую завершённую вершину в стек. На второй фазе произвольный поиск обходит исходный G, рассматривая вершины в порядке извлечения из стека. Алгоритм помечает каждую вершину корнем её сильной компоненты относительно второго обхода в глубину.

На рисунке 6.17 алгоритм показан на нашем примере. Небольшие изменения позволяют также вычислить граф компонент $\mathrm{scc}(G)$ за $O(V + E)$.

```
KosarajuSharir(G):
    S ← новый пустой стек
    для всех вершин v
        снять пометку с v
        v.root ← None
    ⟨⟨Фаза 1: поместить в стек в обратном порядке обхода rev(G)⟩⟩
    для всех вершин v
        если v не помечена
            PushPostRevDFS(v, S)
    ⟨⟨Фаза 2: повторный DFS в порядке стека⟩⟩
    пока S не пуст
        v ← Pop(S)
        если v.root = None
            LabelOneDFS(v, v)

PushPostRevDFS(v, S):
    пометить v
    для каждого ребра u → v  ⟨⟨Обращено!⟩⟩
        если u не помечена
            PushPostRevDFS(u, S)
    Push(v, S)

LabelOneDFS(v, r):
    v.root ← r
    для каждого ребра v → w
        если w.root = None
            LabelOneDFS(w, r)
```

Рисунок 6.16. Алгоритм сильных компонент Косарайю — Шарира.

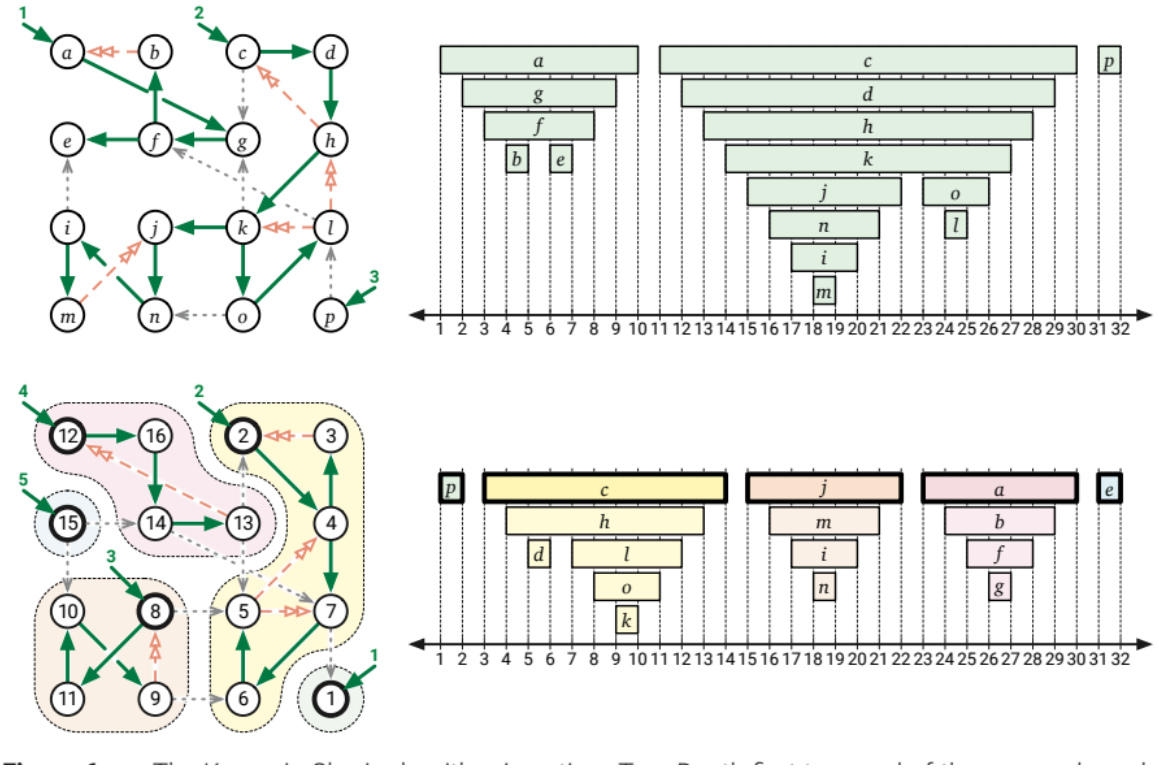


Рисунок 6.17. Работа алгоритма Косарайю — Шарира. Сверху: обход обращённого графа в глубину. Снизу: обход исходного графа в глубину; корневые вершины рассматриваются в порядке, обратном обратному порядку первого обхода.

[4] Ещё раз: обращённый обратный порядок обхода $\mathrm{rev}(G)$ — не то же самое, что обратный порядок обхода G.

[5] Ходят слухи, что тот же алгоритм встречается в русскоязычной литературе ещё до Косарайю, но я пока не нашёл надёжного источника.

♥ Алгоритм Тарьяна

Более ранний, но значительно более тонкий алгоритм вычисления сильных компонент за линейное время опубликовал Боб Тарьян в 1972 году.[6] Интуитивно он определяет исходную компоненту G, «удаляет» её и затем «рекурсивно» находит остальные компоненты; однако всё вычисление происходит во время одного поиска в глубину.

Зафиксируем произвольный обход в глубину ориентированного G. Для каждой вершины v пусть $\mathrm{low}(v)$ — наименьшее время начала среди всех вершин, достижимых из v по пути из рёбер дерева, за которым следует не более одного ребра вне дерева. Очевидно, $\mathrm{low}(v) \le v.\mathrm{pre}$: v достигает себя по пути из нуля рёбер дерева и нуля остальных рёбер. Тарьян заметил, что стоковые компоненты можно охарактеризовать этой функцией.

Лемма 6.4. Вершина v является корнем стоковой компоненты G тогда и только тогда, когда $\mathrm{low}(v) = v.\mathrm{pre}$ и $\mathrm{low}(w) < w.\mathrm{pre}$ для каждого собственного потомка w вершины v.

Доказательство. Сначала пусть $\mathrm{low}(v) = v.\mathrm{pre}$. Тогда нет ребра $w \to x$, где w — потомок v, а $x.\mathrm{pre} < v.\mathrm{pre}$. С другой стороны, v не может достичь вершины y с $y.\mathrm{pre} > v.\mathrm{post}$. Следовательно, из v достижимы только её потомки; значит, любой потомок v также может достичь только потомков v. В частности, v не достигает своего родителя, если он есть, поэтому является корнем своей сильной компоненты.

Теперь дополнительно предположим, что $\mathrm{low}(w) < w.\mathrm{pre}$ для каждого потомка w вершины v. Тогда каждый такой w достигает другой вершины x, которая тоже должна быть потомком v, с $x.\mathrm{pre} < w.\mathrm{pre}$. Следовательно, по индукции каждый потомок v достигает v. Значит, потомки v составляют сильную компоненту C с корнем v. Более того, это стоковая компонента: v не может достичь вершины вне C.

Обратно, пусть v — корень стоковой компоненты C. Тогда v достигает другой вершины w тогда и только тогда, когда $w \in C$. Но v достигает всех своих потомков, а каждая вершина C является потомком v, поэтому потомки v составляют C. Если бы $\mathrm{low}(w) = w.\mathrm{pre}$ для другой вершины $w \in C$, она была бы ещё одним корнем C, что невозможно. □

Вычислить $\mathrm{low}(v)$ для всех вершин поиском в глубину несложно; см. рисунок 6.18.

```
FindLow(G):
    clock ← 0
    для всех вершин v
        снять пометку с v
    для всех вершин v
        если v не помечена
            FindLowDFS(v)

FindLowDFS(v):
    пометить v
    clock ← clock + 1
    v.pre ← clock
    v.low ← v.pre
    для каждого ребра v → w
        если w не помечена
            FindLowDFS(w)
            v.low ← min{v.low, w.low}
        иначе
            v.low ← min{v.low, w.pre}
```

Рисунок 6.18. Вычисление $\mathrm{low}(v)$ для всех вершин v.

Лемма 6.4 показывает, что после `FindLow` можно определить корень каждой стоковой компоненты за $O(V + E)$ глобальным произвольным поиском, затем пометить и удалить эти компоненты ещё за $O(V + E)$, вызывая произвольный поиск из каждого корня, и применить рекурсию. К сожалению, может потребоваться V итераций, каждая из которых удаляет лишь одну вершину, что при наивном подходе даёт общее время $O(VE)$.

Чтобы ускорить стратегию, алгоритм Тарьяна поддерживает вспомогательный стек вершин отдельно от стека рекурсии. Начиная обработку новой вершины v, помещаем её в стек. Заканчивая v, сравниваем `v.low` и `v.pre`. Когда впервые обнаружено равенство `v.low = v.pre`, мы знаем три вещи:

- v — корень стоковой компоненты C;
- все вершины C расположены подряд на вершине вспомогательного стека;
- глубже всех в стеке среди вершин C находится v.

Теперь вершины C можно определить, извлекая их по одной из стека до извлечения v.

Можно удалить вершины C и рекурсивно вычислить компоненты оставшегося графа, но это расточительно: придётся в точности повторить все вычисления до посещения v. Вместо этого пометим каждую вершину C, указав v как корень её компоненты, и до конца поиска будем игнорировать помеченные вершины. Формально это меняет определение $\mathrm{low}(v)$: теперь это наименьшее время начала среди вершин той же сильной компоненты, достижимых из v по пути из рёбер дерева и не более чем одного ребра вне дерева. Но для доказательства корректности проще заметить, что игнорирование помеченных вершин приводит к тому же поведению, что и их удаление.

Итоговый алгоритм Тарьяна приведён на рисунке 6.19; необходимые изменения относительно `FindLow` на рисунке 6.18 в оригинале выделены жирным красным. Его время делится на две части. Каждая вершина один раз помещается в S и один раз извлекается, поэтому поддержание вспомогательного стека — красная часть — занимает в сумме $O(V)$. Если её игнорировать, остальное — обычный поиск в глубину. Следовательно, время алгоритма $O(V + E)$.

```
Tarjan(G):
    clock ← 0
    S ← новый пустой стек
    для всех вершин v
        снять пометку с v
        v.root ← None
    для всех вершин v
        если v не помечена
            TarjanDFS(v)

TarjanDFS(v):
    пометить v
    clock ← clock + 1
    v.pre ← clock
    v.low ← v.pre
    Push(S, v)
    для каждого ребра v → w
        если w не помечена
            TarjanDFS(w)
            v.low ← min{v.low, w.low}
        иначе если w.root = None
            v.low ← min{v.low, w.pre}
    если v.low = v.pre
        повторять
            w ← Pop(S)
```

```
            w.root ← v
        пока w ≠ v
```

Рисунок 6.19. Алгоритм Тарьяна для сильных компонент.

[6] Согласно легенде, Косарайю открыл свой алгоритм во время лекции по алгоритмам. Он должен был объяснять алгоритм Тарьяна, но забыл записи и потому на ходу придумал другой. Единственное, что меня удивляет в этой истории, — почему её не рассказывают про Шарира или Тарьяна.

Упражнения

Поиск в глубину, топологическая сортировка и сильные компоненты

0. (а) Опишите алгоритм, вычисляющий обращение $\mathrm{rev}(G)$ ориентированного графа за время $O(V + E)$.

(б) Докажите, что граф сильных компонент $\mathrm{scc}(G)$ любого ориентированного графа G ацикличен.

(в) Докажите, что $\mathrm{scc}(\mathrm{rev}(G)) = \mathrm{rev}(\mathrm{scc}(G))$ для любого ориентированного графа G.

(г) Пусть G — произвольный фиксированный ориентированный граф, а $S(v)$ — его сильная компонента, содержащая вершину v. Докажите для любых вершин u, v, что v достижима из u в G тогда и только тогда, когда $S(v)$ достижима из $S(u)$ в $\mathrm{scc}(G)$.

1. Ориентированный граф G называется *полусвязным*, если для любых двух вершин u, v вершина u достижима из v или v достижима из u (возможно, выполнено и то и другое).

(а) Приведите пример ориентированного ациклического графа с единственным истоком, который не является полусвязным.

(б) Опишите и проанализируйте алгоритм, определяющий, является ли заданный ориентированный ациклический граф полусвязным.

(в) Опишите и проанализируйте алгоритм, определяющий, является ли произвольный ориентированный граф полусвязным.

2. Полиция города Шам-Пубанана сделала все улицы города односторонними. Несмотря на многочисленные жалобы растерянных водителей, мэр утверждает, что от любого перекрёстка Шам-Пубананы можно доехать до любого другого, не нарушая правил.

(а) Городу нужно подтвердить или опровергнуть утверждение мэра. Сформулируйте задачу в терминах графов, затем опишите и проанализируйте алгоритм её решения.

(б) После выполнения алгоритма из пункта (а) мэр неохотно признала, что лгала, то есть была дезинформирована. Назовём перекрёсток x *хорошим*, если с любого перекрёстка y, до которого можно законно доехать из x, можно законно вернуться в x. Теперь мэр утверждает, что более

95% перекрёстков Шам-Пубананы хорошие. Опишите и проанализируйте эффективный алгоритм проверки этого утверждения.

Для полного балла оба алгоритма должны работать за линейное время.

3. Пусть задан ориентированный ациклический граф G с единственным истоком s и единственным стоком t. Вершина $v \notin \{s, t\}$ называется *разделяющей вершиной для* (s, t), если каждый путь из s в t проходит через v, или, что равносильно, если после удаления v вершина t становится недостижимой из s. Опишите и проанализируйте алгоритм поиска всех таких вершин в G.

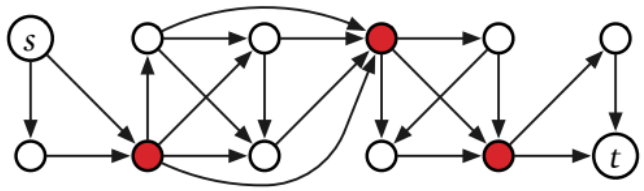


Рисунок 6.20. Ориентированный ациклический граф с тремя разделяющими вершинами для (s, t).

4. Вершина v связного неориентированного графа G называется *точкой сочленения*, если подграф $G - v$, полученный удалением v, несвязен.

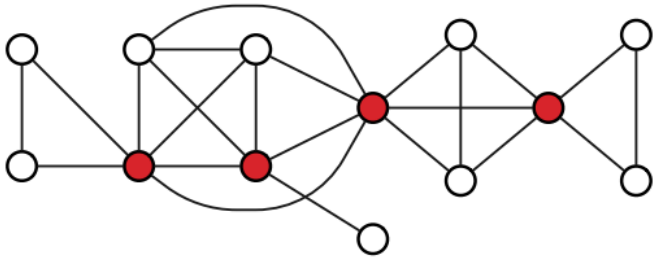


Рисунок 6.21. Неориентированный граф с четырьмя точками сочленения.

(а) Опишите линейный алгоритм, который по неориентированному графу G и вершине v определяет, является ли v точкой сочленения. Сколько времени займёт поиск всех точек сочленения, если применить алгоритм к каждой вершине?

(б) Пусть T — остовное дерево поиска в глубину неориентированного графа G.

i. Докажите, что корень T является точкой сочленения G тогда и только тогда, когда у него более одного ребёнка в T.

ii. Докажите, что отличная от корня вершина v является точкой сочленения G тогда и только тогда, когда хотя бы один потомок каждого ребёнка v в T является соседом в G некоторого собственного предка v в T.

[Подсказка: эти утверждения больше не верны, если T не является остовным деревом поиска в глубину и/или G ориентирован.] **Примечание переводчика:** в пункте (ii) условие воспроизведено по оригиналу; как сформулировано, оно ошибочно: условие наличия таких соседей для *каждого* ребёнка характеризует отсутствие точки сочленения.

(в) Опишите алгоритм, находящий все точки сочленения заданного неориентированного графа за $O(V + E)$.

5. Ребро e связного неориентированного графа G называется *мостом*, если подграф $G - e$, полученный удалением ребра, несвязен.

(а) По заданным G и e опишите линейный алгоритм, определяющий, является ли e мостом. Сколько времени потребуется, чтобы найти все мосты, применив этот алгоритм к каждому ребру?

(б) Пусть T — произвольное остовное дерево G. Докажите, что каждый мост G также является ребром T. Отсюда следует, что в G не более $V - 1$ мостов. Как это улучшает алгоритм поиска всех мостов из пункта (а)?

(в) Теперь выберем в T произвольный корень r. Для вершины v обозначим через T_v поддерево T с корнем v; например, $T_r = T$. Пусть uv — произвольное ребро T, где u — родитель v. Докажите, что uv является мостом G тогда и только тогда, когда это единственное ребро G, ровно один конец которого находится в T_v.

(г) Опишите линейный алгоритм поиска всех мостов G. [Подсказка: пусть T — остовное дерево поиска в глубину в G.]

6. *Транзитивное замыкание* G^T ориентированного графа G — ориентированный граф с теми же вершинами, содержащий ребро $u \to v$ тогда и только тогда, когда в G есть ориентированный путь из u в v. *Транзитивная редукция* G — граф с наименьшим возможным числом рёбер, транзитивное замыкание которого равно G^T. У одного графа может быть несколько транзитивных редукций.

(а) Опишите эффективный алгоритм вычисления транзитивного замыкания заданного ориентированного графа.

(б) Докажите, что ориентированный граф G имеет единственную транзитивную редукцию тогда и только тогда, когда он ацикличен.

(в) Опишите эффективный алгоритм вычисления транзитивной редукции заданного ориентированного графа.

7. Один из старейших алгоритмов обхода произвольных связных графов предложил Гастон Тарри в 1895 году как систематическую процедуру решения лабиринтов.[7] На вход алгоритма Тарри подаётся неориентированный граф G; однако для простоты изложения формально заменим каждое неориентированное ребро uv двумя ориентированными рёбрами $u \to v$ и $v \to u$. (В реальной реализации такое разделение тривиально: алгоритм просто использует заданный список смежности G, как если бы граф был ориентированным.)

```
Tarry(G):
    снять пометки со всех вершин G
    окрасить все рёбра G в белый цвет
    s ← произвольная вершина G
    RecTarry(s)

RecTarry(v):
    пометить v                  ⟨⟨«посетить v»⟩⟩
    если есть белая дуга v → w
        если w не помечена
            окрасить w → v в зелёный
        окрасить v → w в красный
        ⟨⟨«пройти по v → w»⟩⟩ RecTarry(w)
    иначе если есть зелёная дуга v → w
        окрасить v → w в красный
        ⟨⟨«пройти по v → w»⟩⟩ RecTarry(w)
```

Неформально будем говорить, что алгоритм Тарри *посещает* вершину v каждый раз, когда помечает её, и *проходит* по ребру $v \to w$, когда окрашивает это ребро в красный цвет и рекурсивно вызывает `RecTarry(w)`. В отличие от рассмотренного ранее алгоритма обхода графа, алгоритм Тарри может помечать одну вершину несколько раз.

(а) Опишите реализацию алгоритма Тарри, работающую за $O(V + E)$.

(б) Докажите, что ни одно ориентированное ребро G не проходится больше одного раза.

(в) Когда алгоритм посещает вершину v в k-й раз, сколько ровно входящих в v рёбер красные и сколько ровно исходящих из v рёбер красные? [Подсказка: рассмотрите начальную вершину s отдельно от остальных.]

(г) Докажите, что каждая вершина v, кроме начальной s, посещается не более $\deg(v)$ раз, а s — не более $\deg(s) + 1$ раз. Из этого сразу следует, что `Tarry(G)` завершается.

(д) Докажите, что последняя вершина, посещаемая `Tarry(G)`, — начальная вершина s.

(е) Докажите, что при завершении `Tarry(G)` все входящие и исходящие рёбра каждой посещённой им вершины v красные. [Подсказка: рассмотрите вершины в порядке их первой пометки, начиная с s, и докажите утверждение по индукции.]

(ж) Докажите, что `Tarry(G)` посещает каждую вершину G. Это утверждение вместе с предыдущим означает, что `Tarry(G)` проходит каждое ребро G ровно один раз.

[7] Ещё более ранние алгоритмы обхода графов описали Шарль Тремо в 1882 году, Кристиан Винер в 1873 году, Карл Хиерхольцер в 1873 году и, неявно, Леонард Эйлер в 1736 году. В частности, алгоритм Винера эквивалентен поиску в глубину в связных неориентированных графах.

8. Рассмотрим следующий вариант алгоритма обхода графа Тарри: он проходит по зелёным рёбрам, не перекрашивая их в красный цвет, и присваивает каждой вершине две числовые метки.

```
Tarry2(G):
    снять пометки со всех вершин G
    окрасить все рёбра G в белый цвет
    s ← произвольная вершина G
    RecTarry2(s, 1)

RecTarry2(v, clock):
    если v не помечена
        v.pre ← clock; clock ← clock + 1
    пометить v
    если есть белая дуга v → w
        если w не помечена
            окрасить w → v в зелёный
        окрасить v → w в красный
        RecTarry2(w, clock)
    иначе если есть зелёная дуга v → w
        v.post ← clock; clock ← clock + 1
        RecTarry2(w, clock)
```

Докажите или опровергните утверждение: когда `Tarry2(G)` завершается, зелёные рёбра задают остовное дерево, а метки `v.pre` и `v.post` — прямую и обратную нумерации, согласованные с одним и тем же поиском в глубину в G. Иными словами, докажите или опровергните, что `Tarry2` выдаёт тот же результат, что и поиск в глубину, хотя посещает рёбра в совершенно другом порядке.

9. У вас есть n запирающихся ящиков и m золотых ключей. Каждый ключ открывает не более одного ящика. При этом ящик может открываться одним ключом, несколькими ключами или не открываться ни одним. Запертый ящик можно открыть только двумя способами: отпереть подходящим ключом, который находится у вас в руках, или разбить молотком.

Ваш младший брат, любящий играть с блестящими предметами, сумел запереть все ваши ключи в ящиках! К счастью, домашняя система безопасности всё записала, поэтому вы точно знаете, какие ключи, если они есть, лежат в каждом ящике. Вам нужно достать все ключи, потому что они золотые. Ясно, что хотя бы один ящик придётся разбить.

(а) Ваш младший брат нашёл молоток и с интересом поглядывает на один из ящиков. Опишите и проанализируйте алгоритм, определяющий, можно ли достать все ключи, не разбивая никаких ящиков, кроме выбранного братом.

(б) Опишите и проанализируйте алгоритм вычисления минимального числа ящиков, которые нужно разбить, чтобы достать все ключи.

10. Предположим, вы преподаёте курс алгоритмов. На второй контрольной вы даёте студентам рисунок графа и просите указать дерево поиска в ширину и дерево поиска в глубину с заданной корневой вершиной. К сожалению, приступив к проверке, вы понимаете, что у выданного графа несколько таких остовных деревьев — слишком много, чтобы перечислить. Вместо этого нужен способ проверить каждый студенческий ответ!

В каждом из следующих пунктов вам заданы связный граф G, начальная вершина s и остовное дерево T графа G.

(а) Пусть G неориентирован. Опишите и проанализируйте алгоритм, определяющий, является ли T остовным деревом поиска в глубину с корнем s.

(б) Пусть G неориентирован. Опишите и проанализируйте алгоритм, определяющий, является ли T остовным деревом поиска в ширину с корнем s. [Подсказка: недостаточно того, что T является деревом кратчайших путей во невзвешенном графе. Да, эта задача находится в правильной главе!]

(в) Пусть G ориентирован. Опишите и проанализируйте алгоритм, определяющий, является ли T остовным деревом поиска в ширину с корнем s. [Подсказка: сначала решите пункт (б).]

(г) Пусть G ориентирован. Опишите и проанализируйте алгоритм, определяющий, является ли T остовным деревом поиска в глубину с корнем s.

11. Несколько современных языков программирования, включая JavaScript, Python, Perl и Ruby, поддерживают *параллельное присваивание*, позволяющее записать несколько операций присваивания в одной строке. Например, код Python `x,y = 0,1` одновременно присваивает `x` значение 0 и `y` значение 1. Все значения правой части вычисляются по старым значениям переменных. Поэтому код Python `a,b = b,a` меняет значения `a` и `b` местами, а следующий код вычисляет n-е число Фибоначчи:

```
def fib(n):
    prev, curr = 1, 0
    while n > 0:
        prev, curr, n = curr, prev+curr, n-1
    return curr
```

Предположим, создаваемому вами интерпретатору необходимо преобразовать каждое параллельное присваивание в эквивалентную последовательность отдельных присваиваний. Например, параллельное присваивание `a,b = 0,1` можно выполнить последовательно в любом порядке — `a=0; b=1` либо `b=1; a=0`, — но `x,y = x+1,x+y` можно выполнить только как `y=x+y; x=x+1`. Для последовательного выполнения может потребоваться одна или несколько дополнительных временных переменных; например, для `a,b = b,a` требуется одна временная переменная, а для `x,y = x+y,x-y` — две.

(а) Опишите алгоритм, определяющий, можно ли выполнить заданное параллельное присваивание последовательно без дополнительных временных переменных.

(б) Опишите алгоритм, определяющий, можно ли выполнить заданное параллельное присваивание последовательно с ровно одной дополнительной временной переменной.

Считайте, что заданное параллельное присваивание содержит только простые целочисленные переменные (без обращения через указатели или массивы); в левой части каждая переменная встречается не более одного раза; выражения в правой части не имеют побочных эффектов. Не беспокойтесь о подробностях разбора оператора присваивания: просто предположите подходящее представление графом (но опишите его!).

12. Пусть задан ориентированный ациклический граф G, вершины которого представляют работы, а рёбра — ограничения порядка: каждое ребро $u \to v$ означает, что работа u должна быть завершена до начала работы v. Каждая вершина v также имеет вес $T(v)$ — время выполнения работы v.

(а) Опишите алгоритм нахождения кратчайшего промежутка времени, за который можно выполнить все работы в G.

(б) Предположим, первая работа начинается в момент 0. Опишите алгоритм нахождения для каждой вершины v самого раннего времени начала работы v.

(в) Теперь опишите алгоритм нахождения для каждой вершины v самого позднего времени начала работы v, при котором ограничения порядка не нарушаются и общее время завершения, вычисленное в пункте (а), не увеличивается, если все работы, кроме v, начинаются в самые ранние моменты, вычисленные в пункте (б).

13. Пусть G — ориентированный ациклический граф с единственным истоком s и единственным стоком t.

(а) *Гамильтонов путь* в G — ориентированный путь, содержащий каждую вершину G. Опишите алгоритм, определяющий, есть ли в G гамильтонов путь.

(б) Пусть вершины G имеют веса. Опишите эффективный алгоритм поиска пути из s в t с максимальным суммарным весом.

(в) Пусть дополнительно задано целое число ℓ. Опишите эффективный алгоритм поиска пути максимального веса из s в t, содержащего не более ℓ рёбер. (Предположите, что хотя бы один такой путь существует.)

(г) Пусть некоторые вершины G помечены как важные и задано целое число k. Опишите эффективный алгоритм поиска пути максимального веса из s в t, посещающего не менее k важных вершин. (Предположите, что хотя бы один такой путь существует.)

(д) Опишите алгоритм подсчёта числа путей из s в t в графе G. (Предположите, что целые числа произвольной величины можно складывать за $O(1)$.)

14. Пусть вершины ориентированного ациклического графа G имеют метки из некоторого фиксированного алфавита, а $A[1..\ell]$ — строка над тем же алфавитом. *Метка* любого ориентированного пути в G — строка, полученная последовательным соединением меток его вершин.

(а) Опишите алгоритм, который находит путь в G с меткой A либо правильно сообщает об отсутствии такого пути.

(б) Опишите алгоритм подсчёта числа путей в G с меткой A. (Предположите, что целые числа произвольной величины можно складывать за $O(1)$.)

(в) Опишите алгоритм поиска самого длинного пути в G, метка которого является подпоследовательностью A.

(г) Опишите алгоритм поиска самого короткого пути в G, метка которого содержит A как подпоследовательность.

(д) Опишите алгоритм поиска пути в G, метка которого имеет минимальное расстояние редактирования до A.

15. *Ломаная* — последовательность отрезков, соединённых концами; концы этих отрезков называются вершинами ломаной. Длина ломаной — сумма длин её отрезков. Ломаная с вершинами $(x_1, y_1), (x_2, y_2), \ldots, (x_k, y_k)$ называется *монотонно возрастающей*, если $x_i < x_{i+1}$ и $y_i < y_{i+1}$ для каждого индекса i: неформально каждая вершина находится выше и правее предыдущей.

vertex of the path is above and to the right of its predecessor.

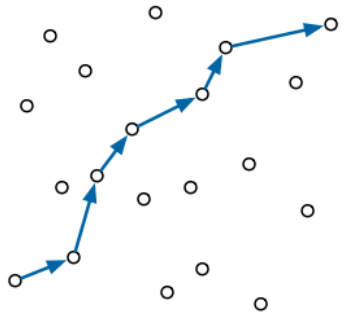


Рисунок 6.22. Монотонно возрастающая ломаная с семью вершинами из заданного множества точек.

Пусть задано множество S из n точек плоскости, представленное двумя массивами $X[1..n]$ и $Y[1..n]$. Опишите и проанализируйте алгоритм вычисления длины самой длинной монотонно возрастающей ломаной с вершинами в S. Предположите, что у вас есть подпрограмма `Length(x,y,x',y')`, возвращающая длину отрезка от (x, y) до (x', y').

16. Для любых двух вершин u и w ориентированного ациклического графа G *интервал* $G[u, w]$ — объединение всех ориентированных путей в G из u в w. Иными словами, $G[u, w]$ состоит из всех вершин v, таких что $v \in \text{reach}(u)$ и $w \in \text{reach}(v)$, вместе со всеми рёбрами G, соединяющими эти вершины.

Пусть задан ориентированный ациклический граф G, в котором каждая вершина имеет числовой вес, который может быть положительным, отрицательным или нулевым.

(а) Опишите эффективный алгоритм поиска интервала максимального веса в G, где вес интервала — сумма весов его вершин.

(б) Опишите эффективный алгоритм нахождения наибольшего веса вершины в каждом интервале G. Алгоритм должен вычислять двумерный массив `MaxWt[1..V, 1..V]`, в котором

каждый элемент `MaxWt[u,w]` — максимальный вес среди вершин интервала $G[u, w]$. В частности, если $G[u, w]$ пуст, `MaxWt[u,w]` должен равняться $-\infty$.

17. Пусть G — ориентированный ациклический граф, вершины которого помечены символами некоторого фиксированного алфавита. Метка любого ориентированного пути в G — строка, полученная последовательным соединением меток его вершин. Напомним, что *палиндром* — строка, совпадающая со своей обратной записью.

(а) Опишите и проанализируйте алгоритм нахождения длины самого длинного палиндрома, являющегося меткой пути в G. Например, для графа на рисунке 6.23 алгоритм должен вернуть число 6 — длину палиндрома `HANNAH`.

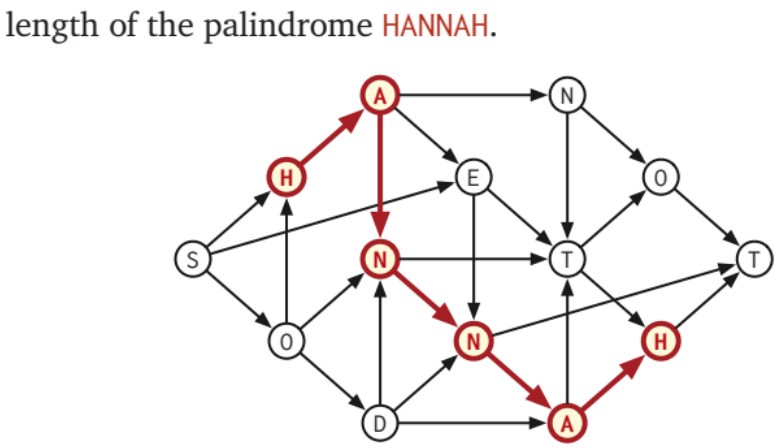


Рисунок 6.23. Ориентированный ациклический граф, самая длинная палиндромная метка пути которого имеет длину 6.

(б) Опишите алгоритм поиска самого длинного палиндрома, являющегося подпоследовательностью метки некоторого пути в G.

(в) Пусть G имеет единственный исток s и единственный сток t. Опишите алгоритм поиска самого короткого палиндрома, содержащего в качестве подпоследовательности метку некоторого пути в G из s в t.

18. Пусть заданы два ориентированных ациклических графа G и H, каждая вершина которых имеет метку из некоторого конечного алфавита; разные вершины могут иметь одинаковые метки. Метка пути в любом из этих графов — строка, полученная последовательным соединением меток его вершин.

(а) Опишите и проанализируйте алгоритм вычисления длины самой длинной строки, которая является одновременно меткой пути в G и меткой пути в H.

(б) Опишите и проанализируйте алгоритм вычисления длины самой длинной строки, являющейся одновременно подпоследовательностью метки пути в G и подпоследовательностью метки пути в H.

(в) Опишите и проанализируйте алгоритм вычисления длины самой короткой строки, которая содержит в качестве подпоследовательности и метку пути в G, и метку пути в H. [Подсказка:

задача проще, чем кажется.]

19. Пусть G — произвольный, не обязательно ациклический, ориентированный граф, в котором каждая вершина v имеет целочисленный вес $w(v)$.

(а) Опишите алгоритм поиска самого длинного ориентированного пути в G, веса вершин которого образуют возрастающую последовательность.

(б) Опишите и проанализируйте алгоритм нахождения вершины максимального веса, достижимой из каждой вершины G. То есть для каждой вершины v алгоритм должен вычислить $\text{maxreach}(v) := \max\{w(x) \mid x \in \text{reach}(v)\}$.

20. (а) Пусть задан ориентированный ациклический граф G с n вершинами и целое число $k \leq n$. Опишите эффективный алгоритм поиска множества не более чем из k попарно вершинно-непересекающихся путей, посещающих каждую вершину G.

(б) Теперь предположим, что рёбра входного ориентированного ациклического графа G имеют веса, которые могут быть положительными, отрицательными или нулевыми. Опишите эффективный алгоритм поиска множества не более чем из k попарно вершинно-непересекающихся путей минимального суммарного веса, посещающих каждую вершину G.

Ваши алгоритмы должны работать за $O(n^{k+c})$ для некоторой небольшой константы c. Одна вершина является путём нулевого веса. (В главе 11 мы увидим более эффективный алгоритм для пункта (а).)

21. Крис — профессиональный скалолаз, участвующий в национальном чемпионате США. Правила требуют использовать как можно больше зацепок на скалодроме, переходя между ними только способами, явно разрешёнными постановщиком трассы.

На стене n зацепок. Крису дан список из m пар зацепок (x, y), каждая из которых означает, что прямой переход от x к y разрешён; однако обратный переход от y к x не разрешён, если список не содержит также пару (y, x). Крису нужно найти последовательность разрешённых переходов, использующую как можно больше зацепок, поскольку каждая новая зацепка увеличивает его счёт на одно очко. Правила позволяют Крису выбрать первую и последнюю зацепки своего маршрута. Также разрешается использовать каждую зацепку сколько угодно раз, но очко начисляется только за первое использование.

(а) Определите естественный граф для представления входных данных. Опишите и проанализируйте алгоритм решения задачи Криса, если гарантируется, что входной граф ориентирован и ацикличен.

(б) Опишите и проанализируйте алгоритм решения задачи Криса без ограничений на входной граф.

Оба алгоритма должны выдавать максимально возможный счёт Криса.

22. Есть n галактик, соединённых m межгалактическими телепортными путями. Каждый путь соединяет две галактики, и по нему можно перемещаться в обоих направлениях. Однако компания, управляющая путями, установила чрезвычайно прибыльную систему оплаты: любой

может телепортироваться дальше от своей домашней галактики совершенно бесплатно, но телепортация в направлении дома стоит непомерно дорого.

Джуди решила взять творческий отпуск и отправиться в путешествие по Вселенной, посетив как можно больше галактик и начав с домашней. Чтобы сэкономить на путешествии, она хочет на каждом шаге телепортироваться дальше от дома, за исключением самого последнего прыжка домой.

(а) Опишите и проанализируйте алгоритм вычисления максимального числа галактик, которые может посетить Джуди. На вход подаются неориентированный граф G с n вершинами и m рёбрами, описывающий сеть телепортных путей, целое число $1 \le s \le n$, обозначающее домашнюю галактику Джуди, и массив $D[1..n]$ с расстояниями каждой галактики от s.

♥ (б) Непосредственно перед отправлением Джуди выигрывает космическую лотерею и получает ровно столько денег, чтобы позволить себе две телепортации в направлении дома. Опишите новый алгоритм вычисления максимального числа различных галактик, которые она может посетить. Одну галактику можно посещать несколько раз, но в общее количество засчитывается только первое посещение.

23. Доктор и Ривер Сонг решили сыграть в игру на ориентированном ациклическом графе G с одним истоком s и одним стоком t.[8]

У каждого игрока есть фишка, находящаяся на одной из вершин G. В начале фишка Доктора находится в истоке s, а фишка Ривер — в стоке t. Игроки ходят по очереди, первым ходит Доктор. На каждом своём ходу Доктор перемещает свою фишку вперёд по ориентированному ребру; на каждом своём ходу Ривер перемещает свою фишку назад по ориентированному ребру.

Если обе фишки когда-либо оказываются в одной вершине, выигрывает Ривер. («Привет, милый!») Если фишка Доктора достигает t или фишка Ривер достигает s до встречи фишек, выигрывает Доктор.

Опишите и проанализируйте алгоритм, определяющий победителя при условии, что оба игрока играют идеально. То есть, если Доктор может выиграть независимо от ходов Ривер, алгоритм должен выдать «Доктор», а если Ривер может выиграть независимо от ходов Доктора — «Ривер». (Почему других возможностей нет?) Вход алгоритма — граф G.

[8] Обозначения s и t — сокращения от Нетемперированного Разлома и Вихря времени, или Сияющего мира Семи систем (также известного как Галлифрей) и Трензалора, или Скаро и Телоса, или чего-то ещё такого времени-шмемени. Это всё очень сложно, неважно.

♣♥ **24.** Пусть задана строка $x = x_1 x_2 \dots x_n$ из n символов над конечным алфавитом Σ и детерминированный конечный автомат A с m состояниями над тем же алфавитом.

(а) Опишите и проанализируйте алгоритм вычисления длины самой длинной подпоследовательности x, допускаемой автоматом A. Например, если A допускает язык $(AR)^*$ и $x = \texttt{ABRACADABRA}$, алгоритм должен выдать 4 — длину подпоследовательности ARAR.

(б) Опишите и проанализируйте алгоритм вычисления длины самой короткой строки, содержащей x как подпоследовательность и допускаемой автоматом A. Например, если A

допускает язык $(ABCDR)^*$ и $x = \texttt{ABRACADABRA}$, алгоритм должен выдать 25 — длину строки ABCDRABCDRABCDRABCDRABCDR.

Проанализируйте алгоритмы в зависимости от длины n входной строки, числа m состояний автомата и размера алфавита Σ.

25. Не каждый алгоритм динамического программирования можно представить как поиск оптимального пути в ориентированном ациклическом графе, но каждый алгоритм динамического программирования обрабатывает некоторый лежащий в его основе граф зависимостей в обратном порядке обхода.

(а) Пусть задан ориентированный ациклический граф G, в каждой вершине которого хранится числовой ключ поиска. Опишите и проанализируйте алгоритм поиска наибольшего двоичного дерева поиска, являющегося подграфом G.

(б) Пусть задан ориентированный ациклический граф G и две вершины s и t. Опишите алгоритм подсчёта числа ориентированных путей из s в t в графе G. (Предположите, что любая арифметическая операция занимает $O(1)$.)

(в) Пусть ориентированный ациклический граф G обладает следующими свойствами:

- В G единственный исток s и несколько стоков $t_1, t_2, \ldots, t_k$.
- Каждому ребру $v \to w$ соответствует вес $p(v \to w)$ от 0 до 1.
- Для каждой вершины v, не являющейся стоком, суммарный вес исходящих рёбер равен 1: $\sum_w p(v \to w) = 1$.

Веса $p(v \to w)$ задают случайное блуждание в G из истока s к некоторому стоку t_i: достигнув вершины v, не являющейся стоком, блуждание следует по ребру $v \to w$ с вероятностью $p(v \to w)$. Все вероятности взаимно независимы. Опишите и проанализируйте алгоритм вычисления для каждого индекса i вероятности того, что это случайное блуждание достигнет стока t_i. (Предположите, что каждая арифметическая операция занимает только $O(1)$.)

Глава 7. Минимальные остовные деревья

Нам всем нужно держаться вместе, господа, иначе нас наверняка повесят по отдельности.

— Бенджамин Франклин при подписании Декларации независимости (4 июля 1776 года)

Помню, как я спросил у кого-то совета — у кого же? — стоит ли отдавать эту работу в печать: рассуждения в ней настолько просты... К счастью, мне посоветовали продолжить, и прошло немало лет, прежде чем какая-либо другая моя публикация стала столь же известной, как эта очень простая работа.

— Джозеф Краскал о своём алгоритме поиска кратчайшего остовного поддерева (1997)

Убрать ВСЕ вещи!

— Элли Брош, «Вот почему я никогда не стану взрослой», Hyperbole and a Half, 17 июня 2010 года

Пусть дан связный неориентированный взвешенный граф. Это граф G = (V, E) вместе с функцией w: E → $\mathbb{R}$, которая каждому ребру e ставит в соответствие вещественный вес w(e); он может быть положительным, отрицательным или нулевым. В этой главе описаны несколько алгоритмов поиска *минимального остовного дерева* графа G, то есть остовного дерева T, минимизирующего функцию

$$w(T) := \sum_{e \in T} w(e).$$

Пример приведён на рис. 7.1.

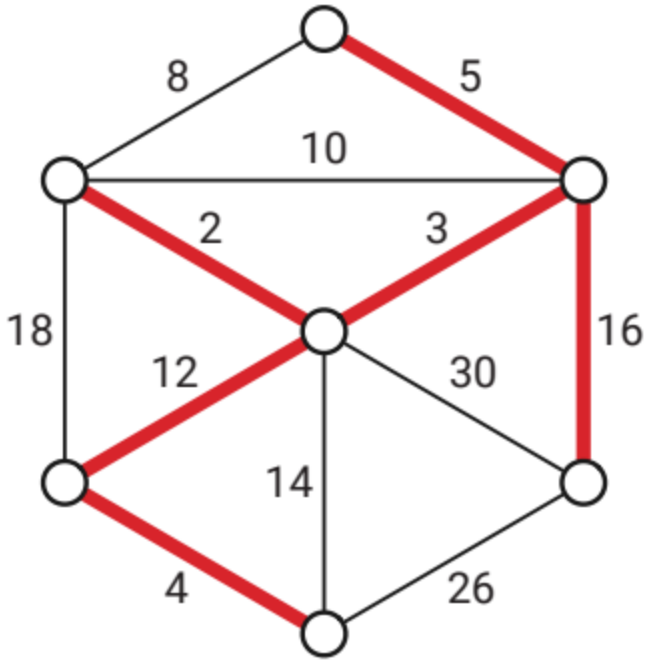


Рис. 7.1. Взвешенный граф и его минимальное остовное дерево.

7.1. Различные веса рёбер

Досадная тонкость в постановке задачи состоит в том, что у взвешенного графа может быть несколько остовных деревьев с одинаковым минимальным весом. В частности, если каждое ребро G имеет вес 1, то любое остовное дерево G является минимальным и имеет вес V – 1. Эта неоднозначность усложняет разработку алгоритмов; всё было бы намного проще, если бы мы могли считать минимальное остовное дерево единственным.

К счастью, существует простое условие, гарантирующее нужную единственность.

Лемма 7.1. Если веса всех рёбер связного графа G различны, то у G существует единственное минимальное остовное дерево.[1]

Доказательство. Пусть G — произвольный связный граф с двумя минимальными остовными деревьями T и T′; нужно доказать, что некоторые два ребра G имеют одинаковый вес. По существу, это жадное доказательство с помощью обмена.

Каждое из двух остовных деревьев содержит ребро, которого нет в другом дереве. Пусть e — ребро минимального веса в T \ T′, а e′ — ребро минимального веса в T′ \ T; при равенстве выбор произволен. Без ограничения общности предположим, что w(e) ≤ w(e′).

Подграф T′ ∪ {e} содержит ровно один цикл C, проходящий через ребро e. Пусть e″ — любое ребро этого цикла, не принадлежащее T. Такое ребро обязательно существует, поскольку T — дерево. Возможно, e″ = e′, но не обязательно. Поскольку e ∈ T, непосредственно получаем e″ ≠ e и, следовательно, e″ ∈ T′ \ T. Отсюда w(e″) ≥ w(e′) ≥ w(e).

Теперь рассмотрим остовное дерево T″ = T′ + e − e″. Оно может совпадать с T. Непосредственно получаем w(T″) = w(T′) + w(e) − w(e″) ≤ w(T′). Но T′ — минимальное остовное дерево, поэтому должно выполняться w(T″) = w(T′); иными словами, T″ тоже является минимальным остовным деревом. Следовательно, w(e) = w(e″), что завершает доказательство. □

Если у нас уже есть алгоритм, предполагающий различные веса рёбер, его всё равно можно применять к графам с одинаковыми весами, если использовать согласованный способ разрешения равенств. Один из таких способов заменяет простое сравнение весов следующим алгоритмом. `ShorterEdge` получает четыре целых числа i, j, k, l, представляющих четыре не обязательно различные вершины, и определяет, какое из двух рёбер (i, j) и (k, l) имеет «меньший» вес. Поскольку входной граф неориентирован, пары (i, j) и (j, i) задают одно и то же ребро.

```
ShorterEdge(i, j, k, l):
    если w(i, j) < w(k, l), вернуть (i, j)
    если w(i, j) > w(k, l), вернуть (k, l)
    если min(i, j) < min(k, l), вернуть (i, j)
    если min(i, j) > min(k, l), вернуть (k, l)
    если max(i, j) < max(k, l), вернуть (i, j)
    ⟨⟨если max(i, j) > max(k, l)⟩⟩ вернуть (k, l)
```

Ввиду леммы 7.1 и этого правила разрешения равенств в оставшейся части главы можно без опасений считать веса рёбер всегда различными, а минимальные остовные деревья — всегда единственными. В частности, мы можем говорить о минимальном остовном дереве без неоднозначности.

[1] Обратное утверждение неверно: у связного графа с одинаковыми весами некоторых рёбер всё равно может быть единственное минимальное остовное дерево. В качестве тривиального примера возьмём граф G, который сам является деревом!

7.2. Единственный алгоритм минимального остовного дерева

Существует много алгоритмов построения минимального остовного дерева, но почти все они являются частными случаями следующей общей стратегии. Ситуация похожа на обход графа: несколько различных алгоритмов оказываются вариантами общего алгоритма поиска в произвольном порядке.

Общий алгоритм минимального остовного дерева поддерживает ациклический подграф F входного графа G, который мы будем называть *промежуточным остовным лесом*. Во все моменты F удовлетворяет следующему инварианту:

> *F является подграфом минимального остовного дерева G.*

Изначально F состоит из V деревьев, каждое из которых содержит одну вершину. Общий алгоритм соединяет деревья в F, добавляя между ними определённые рёбра. Когда алгоритм останавливается, F состоит из одного остовного дерева; из инварианта следует, что оно должно быть минимальным остовным деревом графа G. Очевидно, выбирать рёбра для добавления в

растущий лес нужно осторожно: не каждое ребро принадлежит минимальному остовному дереву.

На каждом этапе построения промежуточный остовный лес F выделяет в остальной части графа два особых вида рёбер:

- Ребро **бесполезно**, если оно не принадлежит F, но оба его конца лежат в одной компоненте F.
- Ребро **безопасно**, если оно имеет минимальный вес среди рёбер, ровно один конец которых принадлежит некоторой компоненте F.

Одно и то же ребро может быть безопасным для двух различных компонент F. Некоторые рёбра из G \ F не являются ни безопасными, ни бесполезными; назовём их **неопределёнными**.

Все алгоритмы построения минимального остовного дерева основаны на двух простых наблюдениях. Первое доказал Роберт Прим в 1957 году, хотя оно неявно содержалось в нескольких более ранних алгоритмах; второе непосредственно очевидно.

Лемма 7.2 (Прим). Минимальное остовное дерево G содержит каждое безопасное ребро.

Доказательство. Мы докажем даже более сильное утверждение: для любого подмножества S вершин G минимальное остовное дерево G содержит ребро минимального веса, ровно один конец которого лежит в S. Как и предыдущую лемму, это утверждение мы докажем жадным обменом.

Пусть S — произвольное подмножество вершин G, а e — самое лёгкое ребро, ровно один конец которого находится в S. Предположение о различных весах рёбер гарантирует единственность e. Пусть T — произвольное остовное дерево, не содержащее e; нужно доказать, что T не является минимальным остовным деревом G.

Поскольку T связно, оно содержит путь от одного конца e до другого. Поскольку этот путь начинается в вершине S, а заканчивается вне S, он должен содержать хотя бы одно ребро, ровно один конец которого принадлежит S; пусть e' — любое такое ребро. Так как T ациклично, удаление e' из T даёт остовный лес ровно с двумя компонентами, каждая из которых содержит один конец e. Поэтому добавление e к этому лесу даёт новое остовное дерево $T' = T - e' + e$. По определению e имеем $w(e') > w(e)$, откуда суммарный вес T' меньше веса T. Следовательно, T не является минимальным остовным деревом G, что и требовалось доказать. □

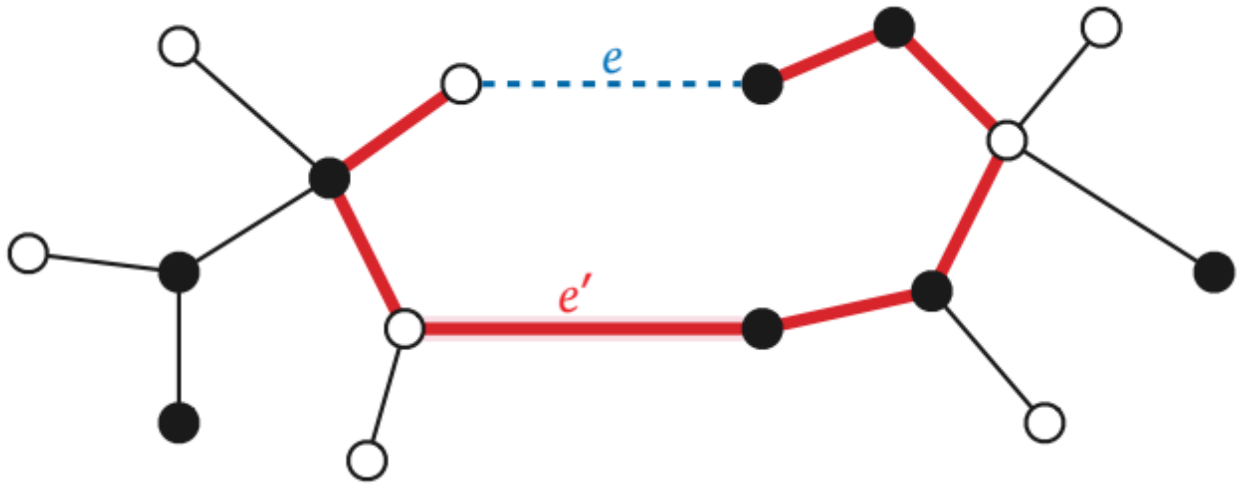


Рис. 7.2. Каждое безопасное ребро входит в минимальное остовное дерево. Чёрные вершины принадлежат подмножеству S.

Лемма 7.3. Минимальное остовное дерево не содержит бесполезных рёбер.

Доказательство. Добавление любого бесполезного ребра к F создаёт цикл. □

Наш общий алгоритм построения минимального остовного дерева последовательно добавляет безопасные рёбра к растущему лесу F. Если F ещё не связен, то существует хотя бы одно безопасное ребро, поскольку входной граф G связен. Поэтому, какие бы безопасные рёбра мы ни добавляли на каждом шаге, общий алгоритм в конце концов свяжет F. По индукции лемма 7.2 гарантирует, что получившееся дерево действительно является минимальным остовным деревом. При добавлении рёбер к F некоторые неопределённые рёбра могут стать безопасными, а другие — бесполезными. Став бесполезным, ребро остаётся таким навсегда. Чтобы полностью задать конкретный алгоритм, необходимо описать, какие безопасные рёбра добавлять на каждой итерации и как их находить.

7.3. Алгоритм Борувки

Старейший и, возможно, простейший алгоритм построения минимального остовного дерева открыл чешский математик Отакар Борувка в 1926 году, примерно через год после того, как Индржих Саксель спросил его, как построить электрическую сеть, соединяющую несколько городов минимальным количеством провода.[2] Алгоритм заново открыл Гюстав Шоке в 1938 году, затем группа польских математиков под руководством Юзефа Лукашевича в 1951 году и ещё раз Жорж Соллен в 1961 году. Хотя Соллен никогда не публиковал своё переоткрытие, его аккуратно описали и указали авторство в одном из первых учебников по алгоритмам на графах; поэтому этот алгоритм иногда называют «алгоритмом Соллена».

Алгоритм Борувки / Шоке / Флорека — Лукашевича — Перкаля — Штейнгауза — Зубжицкого / Прима / Соллена / Брош[3] можно выразить одной строкой:

> ***Борувка:*** *добавь ВСЕ безопасные рёбра и вызови себя рекурсивно.*

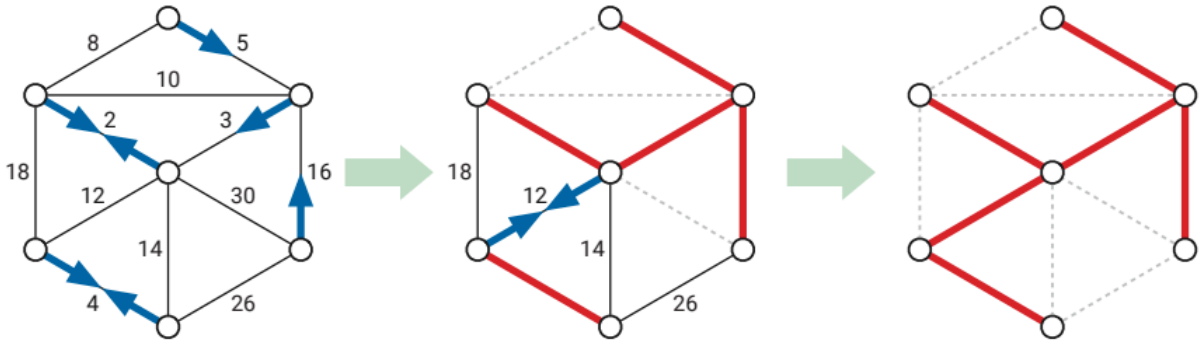


Рис. 7.3. Работа алгоритма Борувки на графе из примера. Толстые красные рёбра принадлежат F, пунктирные рёбра бесполезны. Стрелки указывают вдоль безопасного ребра каждой компоненты. Алгоритм завершается всего после двух итераций.

Рассмотрим алгоритм Борувки подробнее. Он вызывает алгоритм `CountAndLabel` из главы 5, стр. 204 оригинала, чтобы подсчитать компоненты F и пометить каждую вершину v целым числом comp(v), указывающим её компоненту.

```
Borůvka(V, E):
    F ← (V, ∅)
    count ← CountAndLabel(F)
    пока count > 1
        AddAllSafeEdges(E, F, count)
        count ← CountAndLabel(F)
    вернуть F
```

Остаётся лишь описать, как находить все безопасные рёбра и добавлять их в F. Предположим, что F содержит более одной компоненты: иначе работа уже завершена. Следующая подпрограмма вычисляет массив безопасных рёбер `safe[1 .. V]`, где `safe[i]` — ребро минимального веса с одним концом в i-й компоненте F. Для этого она перебирает все рёбра G. Для каждого ребра uv, если u и v лежат в одной компоненте, uv либо бесполезно, либо уже принадлежит F. В противном случае мы сравниваем вес uv с весами `safe[comp(u)]` и `safe[comp(v)]` и при необходимости обновляем элементы массива. Найдя все безопасные рёбра, мы добавляем каждое ребро `safe[i]` в F.

```
AddAllSafeEdges(E, F, count):
    для i ← 1 до count
        safe[i] ← Null
    для каждого ребра uv ∈ E
        если comp(u) ≠ comp(v)
            если safe[comp(u)] = Null или w(uv) < w(safe[comp(u)])
                safe[comp(u)] ← uv
            если safe[comp(v)] = Null или w(uv) < w(safe[comp(v)])
                safe[comp(v)] ← uv
    для i ← 1 до count
        добавить safe[i] в F
```

Каждый вызов `CountAndLabel` работает за время O(V), поскольку лес F содержит не более V – 1 рёбер. `AddAllSafeEdges` работает за время O(V + E), поскольку мы затрачиваем постоянное время на каждую вершину, каждое ребро G и каждую компоненту F. Входной граф связен,

поэтому $V \leq E + 1$. Отсюда каждая итерация цикла while в алгоритме Борувки занимает $O(E)$ времени.

Каждая итерация уменьшает число компонент F хотя бы вдвое: в худшем случае компоненты F сливаются попарно. Изначально F содержит V компонент, поэтому цикл while выполняется не более $O(\log V)$ раз. Следовательно, общее время работы алгоритма Борувки составляет $O(E \log V)$.

[2] Саксель был сотрудником Западно-Моравской энергетической компании; Борувка описывал его как «очень талантливого и трудолюбивого». Позднее нацисты казнили его как человека еврейского происхождения.

[3] Идите и прочитайте всё в *Hyperbole and a Half*. А потом купите книгу. И ещё один экземпляр — для вашей кошки. Что? У вас нет кошки? Что вы за чудовище? Заведите кошку, а потом купите ей дополнительный экземпляр *Hyperbole and a Half*.

Вот какой алгоритм минимального остовного дерева вам нужен

Несмотря на относительно малоизвестное происхождение алгоритма Борувки, первые западные исследователи алгоритмов знали о нём, но отвергли как «слишком сложный». Поэтому, к сожалению, большинство учебников по алгоритмам и структурам данных даже не упоминает алгоритм Борувки, несмотря на его простоту и эффективность. Этот пропуск — серьёзная ошибка: у алгоритма Борувки есть несколько явных преимуществ перед другими классическими алгоритмами построения минимального остовного дерева.

- Алгоритм Борувки часто работает быстрее оценки $O(E \log V)$ для худшего случая. За одну итерацию число компонент F может уменьшиться значительно больше чем вдвое, так что число итераций будет меньше $\lceil \log_2 V \rceil$, необходимого в худшем случае.
- Небольшое изменение формулировки алгоритма Борувки, на самом деле более близкое к первоначальному изложению автора, работает за $O(E)$ времени на широком классе интересных графов, включая графы, которые можно нарисовать на плоскости без пересечения рёбер. Для сравнения: оценки времени двух других алгоритмов относятся ко всем графам.
- Алгоритм Борувки допускает существенное распараллеливание: на каждой итерации каждую компоненту F можно обрабатывать в отдельном независимом потоке. Такой неявный параллелизм позволяет ещё ускорить работу на многоядерных и распределённых системах. Два других классических алгоритма по своей природе последовательны.
- Некоторые более новые алгоритмы построения минимального остовного дерева быстрее описанных здесь классических алгоритмов даже в худшем случае. Все эти более быстрые алгоритмы являются обобщениями алгоритма Борувки.

Короче говоря, если вам понадобится реализовать алгоритм минимального остовного дерева, используйте алгоритм Борувки. С другой стороны, чтобы эффективно доказывать утверждения о минимальных остовных деревьях, вам действительно нужно знать и два следующих алгоритма.

7.4. Алгоритм Ярника («Прима»)

Следующий по возрасту алгоритм построения минимального остовного дерева чешский математик Войтех Ярник впервые описал в письме Борувке в 1929 году и опубликовал в следующем году. Алгоритм независимо переоткрыли Джозеф Краскал в 1956 году, возможно Роберт Прим в 1957 году, Гарри Лоберман и Арнольд Вайнбергер в 1957 году и, наконец, Эдсгер Дейкстра в 1958 году. Прим, Лоберман и Вайнбергер, а также Дейкстра в конечном счёте знали о работе Краскала и даже ссылались на неё. Но поскольку Краскал в той же статье описал ещё два алгоритма, этот обычно называют «алгоритмом Прима», иногда «алгоритмом Прима — Дейкстры», хотя к 1958 году именем Дейкстры уже был неподобающим образом назван другой алгоритм.

В алгоритме Ярника промежуточный лес F имеет лишь одну нетривиальную компоненту T; все остальные компоненты — изолированные вершины. Вначале T состоит из одной произвольной вершины графа. Затем алгоритм повторяет следующий шаг, пока T не охватит весь граф:

> ***Ярник:*** *многократно добавляй к T её безопасное ребро.*

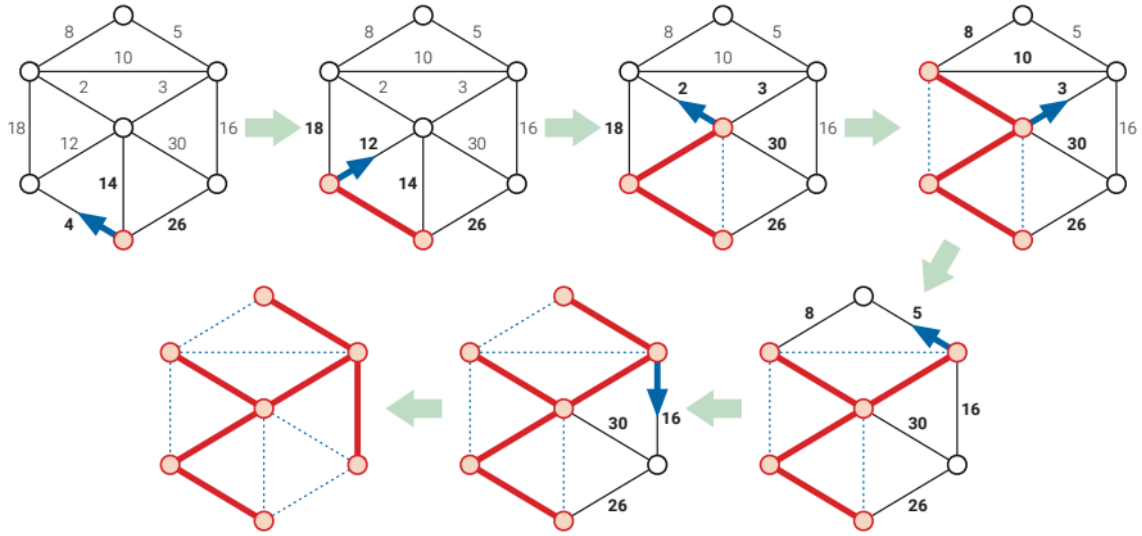


Рис. 7.4. Работа алгоритма Ярника на графе из примера, начиная с нижней вершины. На каждом этапе толстые красные рёбра принадлежат T; стрелка указывает вдоль безопасного ребра T, а пунктирные рёбра бесполезны.

Для реализации алгоритма Ярника храним все рёбра, соседние с T, в очереди с приоритетом. Извлекая из очереди ребро минимального веса, сначала проверяем, принадлежат ли оба его конца T. Если нет, добавляем ребро к T, а затем добавляем в очередь новые соседние рёбра. Иными словами, алгоритм Ярника — вариант поиска «сначала лучший», описанного в конце главы 5. Если реализовать очередь с приоритетом стандартной двоичной кучей, алгоритм Ярника работает за O(E log E) = O(E log V) времени.

♥ Улучшение алгоритма Ярника

Алгоритм Ярника можно улучшить, применив более сложную структуру очереди с приоритетом — *фибоначчиеву кучу*, впервые описанную Майклом Фредманом и Робертом Тарьяном в 1984 году. Подобно двоичным кучам, фибоначчиевы кучи поддерживают стандартные операции очереди с приоритетом `Insert`, `ExtractMin` и `DecreaseKey`. Однако в отличие от обычных двоичных куч, которым требуется O(log n) времени на каждую операцию, фибоначчиевы кучи выполняют `Insert` и `DecreaseKey` за постоянное амортизированное время. Амортизированная стоимость `ExtractMin` по-прежнему равна O(log n).[4]

Чтобы воспользоваться этой более быстрой структурой, храним в очереди вершины G вместо рёбер. Приоритет каждой вершины v — вес ребра минимального веса между v и растущим деревом T или ∞, если такого ребра нет. В начале алгоритма можно вставить все вершины в очередь; затем всякий раз, когда к T добавляется новое ребро, может потребоваться уменьшить приоритеты некоторых соседних вершин.

Для удобства разделим алгоритм на две части. `JarníkInit` инициализирует очередь с приоритетом; `JarníkLoop` выполняет основной алгоритм. На вход подаются вершины и рёбра графа вместе с начальной вершиной s. Для каждой вершины v мы поддерживаем как её приоритет priority(v), так и инцидентное ребро edge(v), для которого w(edge(v)) = priority(v).

```
Jarník(V, E, s):
    JarníkInit(V, E, s)
    JarníkLoop(V, E, s)

JarníkInit(V, E, s):
    для каждой вершины v ∈ V \ {s}
        если vs ∈ E
            edge(v) ← vs
            priority(v) ← w(vs)
        иначе
            edge(v) ← Null
            priority(v) ← ∞
        Insert(v)

JarníkLoop(V, E, s):
    T ← ({s}, ∅)
    для i ← 1 до |V| – 1
        v ← ExtractMin
        добавить v и edge(v) в T
        для каждого соседа u вершины v
            если u ∉ T и priority(u) > w(uv)
                edge(u) ← uv
                DecreaseKey(u, w(uv))
```

Рис. 7.5. Алгоритм Ярника для минимального остовного дерева, подготовленный для использования с фибоначчиевой кучей.

Операции `Insert` и `ExtractMin` вызываются по O(V) раз — по разу для каждой вершины, кроме s, — а `DecreaseKey` вызывается O(E) раз, не более двух раз на каждое ребро. Поэтому с фибоначчиевой кучей улучшенный алгоритм работает за O(E + V log V), что быстрее алгоритма Борувки, если только E не равно O(V).

Однако на практике это улучшение редко оказывается быстрее простой реализации с двоичной кучей, если только граф не исключительно велик и плотен. Алгоритмы фибоначчиевой кучи довольно сложны, а скрытые константы как во времени, так и в памяти существенны: они не запредельны, но явно больше скрытой единичной константы в оценке O(log n) для операций с двоичной кучей.

[4] Амортизированное время — бухгалтерский приём, позволяющий не учитывать редкие колебания времени отдельной операции над структурой данных. Фибоначчиева куча может выполнить любую перемежающуюся последовательность из I операций `Insert`, D операций `DecreaseKey` и X операций `ExtractMin` за O(I + D + X log n) времени в худшем случае. Таким образом, средние `Insert` и `DecreaseKey` занимают постоянное время, а средняя `ExtractMin` — O(log n), хотя некоторые отдельные операции в худшем случае могут занимать больше времени. Амортизация использует усреднение по последовательности операций; случайность ни входных данных, ни алгоритма не предполагается.

7.5. Алгоритм Краскала

Последний рассматриваемый нами алгоритм построения минимального остовного дерева впервые описал Джозеф Краскал в 1956 году в той же статье, где он заново открыл алгоритм Ярника. Краскала вдохновил «машинописный перевод неизвестного происхождения» оригинальной статьи Борувки, который «ходил по рукам» на математическом факультете Принстона. Алгоритм Борувки Краскал считал «излишне сложным».[5] В 1957 году тот же алгоритм заново открыли Гарри Лоберман и Арнольд Вайнбергер, но каким-то образом его так и не переименовали в их честь.

Как и предыдущие алгоритмы, алгоритм Краскала можно описать одной запоминающейся строкой:

> ***Краскал:*** *просматривай все рёбра в порядке возрастания веса; если ребро безопасно, добавь его в F.*

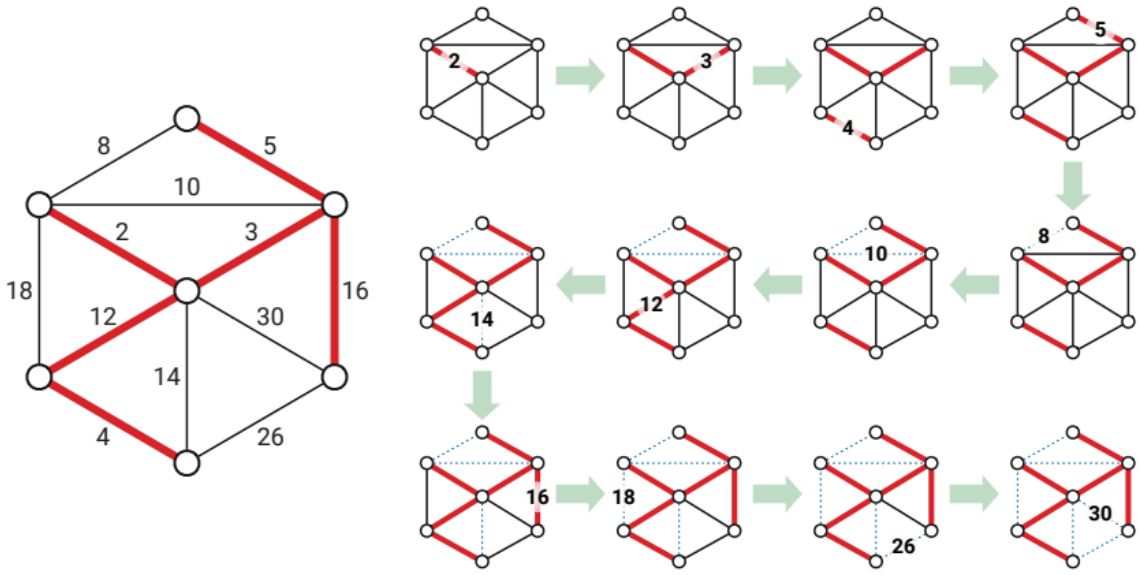


Рис. 7.6. Работа алгоритма Краскала на графе из примера. Толстые красные рёбра принадлежат F; тонкие пунктирные рёбра бесполезны.

Проще всего просматривать рёбра в порядке возрастания веса, отсортировав их за время O(E log E), а затем перебрав отсортированный список простым циклом for. Как мы вскоре увидим, именно эта предварительная сортировка определяет время работы алгоритма.

Поскольку мы рассматриваем рёбра от лёгких к тяжёлым, рассматриваемое ребро безопасно тогда и только тогда, когда его концы лежат в разных компонентах леса F. Предположим, встретилось ребро e, соединяющее компоненты A и B, но не являющееся безопасным. Тогда должно существовать более лёгкое ребро e′, ровно один конец которого лежит в A. Но это невозможно, поскольку по индукции оба конца каждого ранее рассмотренного ребра находятся в одной компоненте F.

Как и в алгоритме Борувки, каждая вершина F должна «знать», какой компоненте F она принадлежит. Однако, в отличие от алгоритма Борувки, мы не вычисляем все метки компонент заново после добавления каждого ребра. Вместо этого, когда ребро соединяет две компоненты, меньшая компонента наследует метку большей: мы обходим меньшую компоненту поиском в произвольном порядке. Этот обход требует O(1) времени на каждую её вершину. При каждом изменении метки вершины содержащая её компонента F увеличивается как минимум вдвое; следовательно, метка каждой вершины изменяется не более O(log V) раз. Поэтому общее время обновления меток составляет лишь O(V log V).

В более общем виде алгоритм Краскала поддерживает разбиение вершин G на непересекающиеся подмножества — в нашем случае компоненты F — с помощью структуры данных, поддерживающей следующие операции:

- `MakeSet(v)` — создать множество, содержащее только вершину v.
- `Find(v)` — вернуть уникальный идентификатор множества, содержащего v.
- `Union(u, v)` — заменить множества, содержащие u и v, их объединением. Эта операция уменьшает число множеств.

Вот полное описание алгоритма Краскала через эти операции:

```
Kruskal(V, E):
    отсортировать E по возрастанию веса
    F ← (V, ∅)
    для каждой вершины v ∈ V
        MakeSet(v)
    для i ← 1 до |E|
        uv ← i-е по весу ребро E
        если Find(u) ≠ Find(v)
            Union(u, v)
            добавить uv в F
    вернуть F
```

После начальной сортировки алгоритм выполняет ровно V операций `MakeSet`, по одной на вершину, 2E операций `Find`, по две на ребро, и V − 1 операций `Union`, по одной на ребро минимального остовного дерева. Мы только что описали структуру непересекающихся множеств, где `MakeSet` и `Find` требуют O(1) времени, а `Union` — O(log V) амортизированного

времени. При такой реализации общее время поддержания разбиения составляет O(E + V log V).[6]

Но вспомним, что одна лишь сортировка рёбер уже требует O(E log E) = O(E log V) времени. Поскольку это больше времени поддержания структуры `Union-Find`, общее время алгоритма Краскала равно O(E log V) — в точности как у алгоритма Борувки или алгоритма Ярника с обычной, не фибоначчиевой, кучей.

[5] Справедливости ради, первая статья Борувки действительно была излишне сложной, отчасти потому, что написана для математиков на формальном языке линейной алгебры, а не на языке графов. Его следующая статья, также опубликованная в 1927 году, но в электротехническом журнале, была написана обычным языком для гораздо более широкой аудитории и излагала алгоритм практически в современном виде. По-видимому, Краскал не знал о второй статье Борувки. Глупый железный занавес.

[6] Другая структура непересекающихся множеств, использующая объединение по рангу со сжатием путей, выполняет каждую операцию `Union` или `Find` за $O(\alpha(V))$ амортизированного времени, где α — почти, но не совсем постоянная обратная функция Аккермана. Если вам не хочется обращаться к Википедии, просто считайте $\alpha(V)$ равной 4. При такой реализации общее время поддержания разбиения составляет $O(E\alpha(V))$; это немного быстрее, когда V велико, а E очень близко к V.

Упражнения

1. Пусть G = (V, E) — произвольный связный граф со взвешенными рёбрами.
    - (а) Докажите, что для любого цикла в G минимальное остовное дерево G не содержит ребро наибольшего веса этого цикла.
    - (б) Докажите или опровергните: минимальное остовное дерево G содержит ребро наименьшего веса каждого цикла G.
2. Во всей главе предполагалось, что никакие два ребра входного графа не имеют одинакового веса, из чего следует единственность минимального остовного дерева. На самом деле единственность гарантирует более слабое условие на веса рёбер.
    - (а) Опишите взвешенный граф, у которого есть единственное минимальное остовное дерево, хотя два ребра имеют одинаковый вес.
    - (б) Докажите, что взвешенный граф G имеет единственное минимальное остовное дерево тогда и только тогда, когда выполняются следующие условия: для любого разбиения вершин G на два подмножества ребро минимального веса с одним концом в каждом подмножестве единственно; ребро максимального веса в любом цикле G единственно.
    - (в) Опишите и проанализируйте алгоритм, определяющий, имеет ли граф единственное минимальное остовное дерево.

    Примечание переводчика. Утверждение пункта (б) в исходном издании неверно без дополнительных условий. Например, в полном графе на вершинах A, B, C, D пусть все рёбра из D имеют вес 1, а три остальные — вес 2. Единственное минимальное остовное

дерево — звезда с центром D, однако в цикле A–B–C максимальное ребро не единственно, а в разрезе {A, D} / {B, C} два ребра минимального веса.

3. Большинство классических алгоритмов минимального остовного дерева использует описанные в тексте понятия «безопасного» и «бесполезного» ребра, но возможна другая формулировка. Пусть G — взвешенный неориентированный граф с различными весами рёбер. Назовём ребро e **опасным**, если оно является самым длинным ребром некоторого цикла в G, и **полезным**, если оно не принадлежит ни одному циклу G.
    - (а) Докажите, что минимальное остовное дерево G содержит каждое полезное ребро.
    - (б) Докажите, что минимальное остовное дерево G не содержит ни одного опасного ребра.
    - (в) Опишите и проанализируйте эффективную реализацию следующего алгоритма, впервые описанного Джозефом Краскалом в той же статье 1956 года, где он предложил «алгоритм Краскала»: просмотрите рёбра G в порядке убывания веса; если ребро опасно, удалите его из G. Подсказка: алгоритм не будет столь же быстрым, как обычный алгоритм Краскала.
4. (а) Опишите и проанализируйте алгоритм построения остовного дерева максимального веса заданного взвешенного графа.
    - (б) **Разрушающее множество рёбер** неориентированного графа G — подмножество F рёбер, такое что каждый цикл G содержит хотя бы одно ребро из F. Иными словами, удаление всех рёбер F делает G ацикличным. Опишите и проанализируйте быстрый алгоритм вычисления разрушающего множества минимального веса заданного взвешенного графа.
5. Пусть даны неориентированный граф G со взвешенными рёбрами и его минимальное остовное дерево T.
    - (а) Опишите алгоритм обновления минимального остовного дерева, когда вес одного ребра e уменьшается.
    - (б) Опишите алгоритм обновления минимального остовного дерева, когда вес одного ребра e увеличивается.

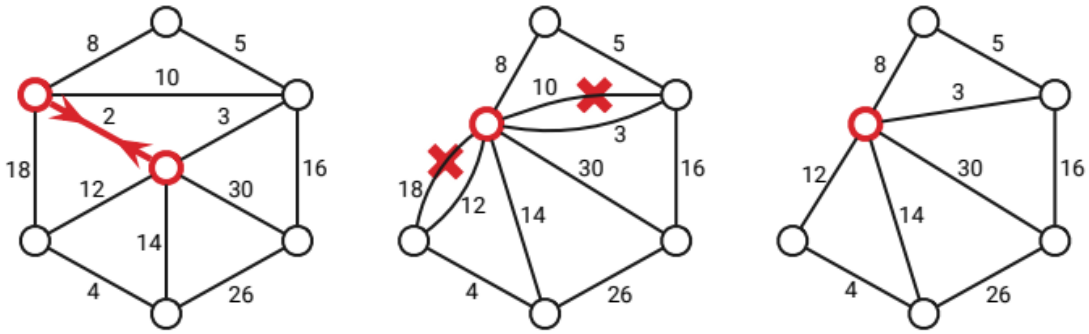
    В обоих случаях вход алгоритма — ребро e и его новый вес; алгоритм должен изменить T так, чтобы оно по-прежнему было минимальным остовным деревом. Подсказка: рассмотрите отдельно случаи $e \in T$ и $e \notin T$.

6. (а) Опишите и проанализируйте алгоритм поиска второго по величине снизу остовного дерева графа G, то есть остовного дерева с наименьшим суммарным весом после минимального остовного дерева.
    - ♥ (б) Опишите и проанализируйте эффективный алгоритм, который по взвешенному неориентированному графу G и целому k находит k остовных деревьев G наименьшего веса.
7. Граф $G = (V, E)$ называется плотным, если $E = \Theta(V^2)$. Опишите модификацию алгоритма Ярника, работающую за $O(V^2)$ времени независимо от E, если входной граф плотный, и

использующую только элементарные структуры данных, в частности без фибоначчиевых куч. Этот вариант алгоритма Ярника впервые описал Эдсгер Дейкстра в 1958 году.

8. Алгоритмы построения минимального остовного дерева часто формулируют с помощью операции **стягивания ребра**. Чтобы стянуть ребро uv, добавим новую вершину, перенаправим в неё все рёбра, инцидентные u или v, кроме uv, а затем удалим u и v. После стягивания между новой и другими вершинами могут возникнуть кратные параллельные рёбра; для каждой пары вершин удалим все такие рёбра, кроме самого лёгкого.

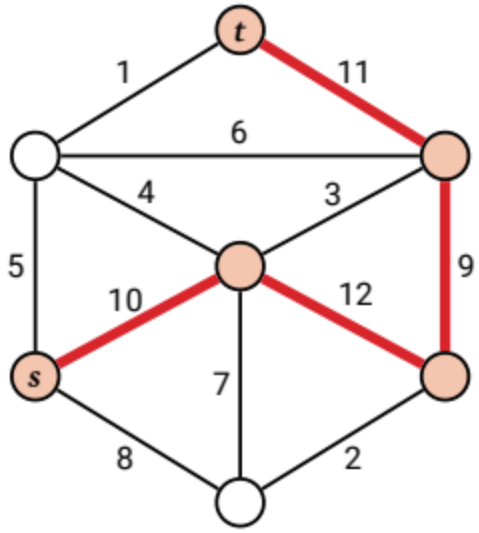
   Три описанных в этой главе классических алгоритма удобно выражаются через стягивание. Все они начинают с создания отдельной копии G′ входного графа G, а затем многократно стягивают безопасные рёбра в G′; минимальное остовное дерево состоит из стянутых рёбер.



   - **Борувка:** отметь самое лёгкое ребро, выходящее из каждой вершины, стяни все отмеченные рёбра и вызови алгоритм рекурсивно.
   - **Ярник:** многократно стягивай самое лёгкое ребро, инцидентное некоторой фиксированной корневой вершине.
   - **Краскал:** многократно стягивай самое лёгкое ребро графа.
   - (а) Опишите алгоритм выполнения одного прохода алгоритма Борувки со стягиванием за O(V + E) времени. Входной граф представлен списком смежности.
   - (б) Рассмотрим алгоритм, который сначала выполняет k проходов алгоритма Борувки со стягиванием, а затем запускает алгоритм Ярника с фибоначчиевой кучей на полученном стянутом графе. (i) Каково время работы этого гибридного алгоритма как функция V, E и k? (ii) При каком k это время минимально и чему оно равно?
   - (в) Назовём семейство графов **хорошим**, если стягивание ребра хорошего графа снова даёт хороший граф и каждый хороший граф с V вершинами имеет лишь O(V) рёбер. Например, планарные графы, которые можно нарисовать на плоскости без пересечений рёбер, хороши: стягивание ребра планарного графа даёт меньший планарный граф, а формула Эйлера гарантирует, что в планарном графе с V вершинами не более 3V − 6 рёбер. Докажите, что алгоритм Борувки со стягиванием вычисляет минимальное остовное дерево любого хорошего графа за время O(V).

9. Рассмотрим путь между вершинами s и t неориентированного взвешенного графа G. **Шириной** пути назовём минимальный вес ребра на этом пути. **Бутылочное расстояние** между s и t — ширина самого широкого пути от s до t. Если путей от s до t нет, бутылочное расстояние равно −∞; напротив, бутылочное расстояние от s до самой себя равно ∞.

    - (а) Докажите, что максимальное остовное дерево G содержит самые широкие пути между каждой парой вершин.



    - (б) Опишите алгоритм, решающий следующую задачу за O(V + E): по неориентированному взвешенному графу G, вершинам s и t и весу W определить, не превышает ли бутылочное расстояние между s и t значение W.
    - (в) Пусть B — бутылочное расстояние между s и t. (i) Докажите, что удаление любого ребра веса меньше B не изменяет это расстояние. (ii) Докажите, что стягивание любого ребра веса больше B не изменяет это расстояние. Если после стягивания возникли параллельные рёбра, удалите все, кроме самого тяжёлого ребра между каждой парой вершин.
    - ♥ (г) Опишите алгоритм построения пути с минимальным бутылочным расстоянием между s и t за O(V + E). Подсказка: начните с поиска ребра медианного веса в G.

10. Алгоритм Борувки можно переформулировать так, чтобы он определял безопасные рёбра с помощью стандартной структуры непересекающихся множеств, подобно алгоритму Краскала, вместо явного пересчёта и перемаркировки компонент растущего остовного леса F на каждой итерации.

    В этом варианте каждая компонента F представлена деревом с указателями к корню; каждая вершина v хранит указатель parent(v) на родителя или на саму себя, если она является корнем дерева. Подпрограмма `Find(v)` возвращает корень дерева вершины v, но также выполняет *сжатие пути*: переназначает все указатели родителей на пути от v к корню так, чтобы они указывали непосредственно на корень, ускоряя будущие операции

`Find`.[7] Подпрограмма `Union` объединяет два дерева, делая один из корней родителем другого.[8]

```
Find(v):
    если parent(v) = v
        вернуть v
    иначе
        v̄ ← Find(parent(v))
        parent(v) ← v̄
        вернуть v̄

Union(u, v):
    ū ← Find(u)
    v̄ ← Find(v)
    либо
        parent(ū) ← v̄
    либо
        parent(v̄) ← ū
```

```
В изменённом алгоритме Борувки корневая вершина v̄ каждой компоненты F, помимо указателей на
родителей, хранит ребро safe(v̄), которое по завершении `FindSafeEdges` является самым лёгким
ребром с одним концом в этой компоненте.
```

```
FindSafeEdges(V, E):
    для каждой вершины v ∈ V
        safe(v) ← Null
    found ← False
    для каждого ребра uv ∈ E
        ū ← Find(u)
        v̄ ← Find(v)
        если ū ≠ v̄
            если safe(ū) = Null или w(uv) < w(safe(ū))
                safe(ū) ← uv
            если safe(v̄) = Null или w(uv) < w(safe(v̄))
                safe(v̄) ← uv
            found ← True
    вернуть found

AddSafeEdges(V, E, F):
    для каждой вершины v ∈ V
        если safe(v) ≠ Null
            xy ← safe(v)
            если Find(x) ≠ Find(y)
                Union(x, y)
                добавить xy в F

Borůvka(V, E):
    F ← ∅
    для каждой вершины v ∈ V
        parent(v) ← v
    пока FindSafeEdges(V, E)
        AddSafeEdges(V, E, F)
    вернуть F
```

```
Докажите, что каждый вызов `FindSafeEdges` и `AddSafeEdges` требует только O(E) времени.
Подсказка: какова глубина деревьев с указателями к корню, когда `FindSafeEdges` завершает работу?
```

> Отсюда следует, что и этот вариант алгоритма Борувки работает за O(E log V).

[7] Сжатие путей — разновидность мемоизации!

[8] Обычно `Union` реализуют аккуратнее, чтобы корень большего или более старого дерева не менялся; но здесь эти подробности не имеют значения.

Глава 8. Кратчайшие пути

Я изучаю Библию так же, как собираю яблоки. Сначала я трясу всё дерево, чтобы упали самые спелые плоды. Затем взбираюсь на дерево и трясу каждую большую ветвь, потом каждую ветку, затем каждый прутик и, наконец, заглядываю под каждый лист.

— приписывается Мартину Лютеру (около 1500 года)

Жизнь постепенно раскрывается, и чем дальше мы идём, тем больше истины способны понять. Понять то, что у нашего порога, — лучшая подготовка к пониманию того, что находится за ним.

— приписывается Гипатии Александрийской Элбертом Хаббардом, Little Journeys to the Homes of Great Teachers (1908)

Ваш разум ответит на большинство вопросов, если вы научитесь расслабляться и ждать ответа. Как в одну из этих вычислительных машин: вводите вопрос, откидываетесь назад и ждёте...

— Уильям С. Берроуз, «Голый завтрак» (1959)

Приведённые в этой статье методы не требуют ни предвидения, ни изобретательности и потому заслуживают называться алгоритмами.

— Эдвард Р. Мур, «Кратчайший путь через лабиринт» (1959)

Пусть задан взвешенный ориентированный граф G = (V, E, w) с двумя особыми вершинами, и мы хотим найти кратчайший путь от исходной вершины s к целевой вершине t. Иными словами, мы хотим найти ориентированный путь P, начинающийся в s и заканчивающийся в t, который минимизирует функцию

$$w(P) := \sum_{u \to v \in P} w(u \to v).$$

Например, чтобы ответить на вопрос «Как быстрее всего доехать от моей старой квартиры в Шампейне, штат Иллинойс, до старой квартиры моей жены в Колумбусе, штат Огайо?», я мог бы использовать граф, вершины которого — города, рёбра — дороги, веса — время в пути, s — Шампейн, а t — Колумбус.[1] Граф ориентирован, поскольку время движения по одной и той же дороге может различаться в разных направлениях. В своё время на трассе I-70 немного восточнее границы Индианы и Огайо была ловушка для нарушителей скорости, но только для движения на восток.

8.1. Деревья кратчайших путей

Почти каждый известный алгоритм вычисления кратчайшего пути между двумя вершинами на самом деле решает значительную часть следующей более общей **задачи кратчайших путей из одного источника**, или SSSP: найти кратчайшие пути от исходной вершины s ко всем остальным вершинам графа. Обычно эту задачу решают, находя дерево кратчайших путей с корнем s, которое содержит все искомые кратчайшие пути.

Нетрудно увидеть, что если кратчайшие пути единственны, они образуют дерево, поскольку любой подпуть кратчайшего пути сам является кратчайшим путём. Если к некоторым вершинам ведёт несколько кратчайших путей, всегда можно выбрать по одному кратчайшему пути к каждой вершине так, чтобы объединение этих путей было деревом. Если кратчайшие пути от s к двум вершинам u и v расходятся, затем встречаются и снова расходятся, один из путей можно изменить без изменения его длины так, чтобы они расходились только один раз.

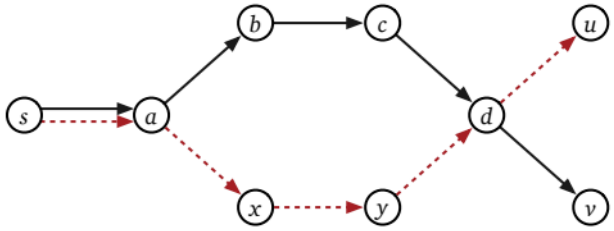


Figure 8.1. If $s \to a \to b \to c \to d \to v$ (solid) and $s \to a \to x \to y \to d \to u$ (dashed) are shortest paths, then

Рис. 8.1. Если s → a → b → c → d → v — кратчайший путь, показанный сплошной линией, а s → a → x → y → d → u — кратчайший путь, показанный пунктиром, то s → a → b → c → d → u, проходящий сверху, также является кратчайшим путём.

Хотя оба вида являются оптимальными остовными деревьями, деревья кратчайших путей и минимальные остовные деревья очень различаются. Деревья кратчайших путей имеют корень и ориентированы; минимальные остовные деревья не имеют корня и не ориентированы. Первые наиболее естественно определять для ориентированных графов, вторые — для неориентированных. Если веса рёбер различны, минимальное остовное дерево единственно, но каждая исходная вершина порождает своё дерево кратчайших путей; более того, может оказаться, что в каждом дереве кратчайших путей используется набор рёбер, отличный от набора минимального остовного дерева.

[1] По Church на запад, по Prospect на север, по I-74 на восток, по I-465 на юг, по Airport Expressway на восток, по I-65 на север, по I-70 на восток, по Grandview на север, по 5th на восток, по Olentangy River на север, по Dodridge на восток, по High на север, по Kelso на запад и по Neil на юг. С поправкой на пробки. Теперь мы живём в Урбане.

♥ 8.2. Рёбра отрицательного веса

В большинстве задач о кратчайших путях, где веса рёбер соответствуют расстоянию, длине или времени, естественно считать все веса неотрицательными или даже положительными. Однако во многих применениях алгоритмов кратчайших путей естественно рассматривать рёбра с отрицательным весом. Например, вес ребра может представлять стоимость перехода между

вершинами; тогда отрицательные веса соответствуют переходам с отрицательной стоимостью или, что равносильно, приносящим прибыль.

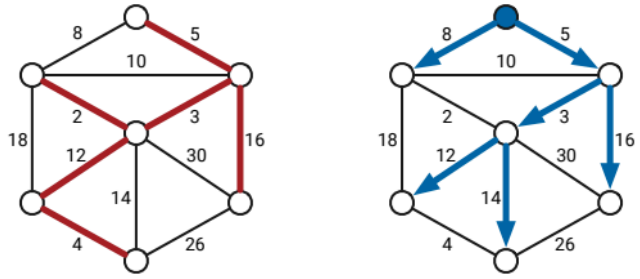


Рис. 8.2. Минимальное остовное дерево и дерево кратчайших путей одного неориентированного графа.

Отрицательные рёбра создают сложности для большинства задач о кратчайших путях, поскольку наличие отрицательного цикла может означать, что кратчайшие пути не определены корректно. Точнее, кратчайший путь от s к t существует тогда и только тогда, когда от s к t есть хотя бы один путь, но нет ни одного пути от s к t, проходящего через отрицательный цикл. Для любого пути от s к t, проходящего через отрицательный цикл, существует более короткий путь от s к t, который обходит этот цикл ещё один раз.[2] Поэтому, если хотя бы один путь от s к t проходит через отрицательный цикл, кратчайшего пути от s к t нет.

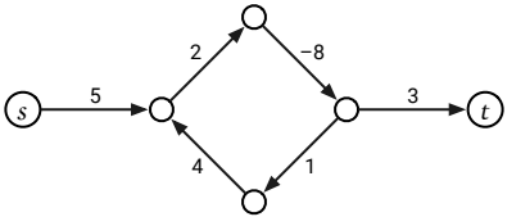


Рис. 8.3. Кратчайшего маршрута от s к t не существует.

Отчасти из-за необходимости учитывать отрицательные веса в этой главе явно рассматриваются только ориентированные графы. Все описанные алгоритмы также работают для неориентированных графов с практически тривиальными изменениями тогда и только тогда, когда отрицательные рёбра запрещены. Правильная обработка отрицательных рёбер в неориентированных графах гораздо тоньше. Нельзя просто заменить каждое неориентированное ребро парой ориентированных, поскольку это превратило бы любое отрицательное ребро в короткий отрицательный цикл. Подпути неориентированного кратчайшего пути, содержащего отрицательное ребро, не обязательно являются кратчайшими; следовательно, множество всех неориентированных кратчайших путей из одной исходной вершины может не образовывать дерево, даже если кратчайшие пути единственны.

[2] Строго говоря, здесь следует говорить о кратчайших маршрутах, а не путях, но такое злоупотребление терминологией общепринято. Если из s достижима t, кратчайший простой путь от s к t обязательно существует; просто вычислить его NP-трудно при наличии отрицательных циклов, что следует из несложного сведения от задачи о гамильтоновом пути. С

другой стороны, если кратчайший маршрут от s к t существует, он должен быть простым путём и, следовательно, является кратчайшим простым путём от s к t. Брр.

Примечание переводчика. Утверждение, что каждый кратчайший маршрут обязательно прост, требует уточнения: если маршрут содержит цикл нулевого веса, его можно удалить без изменения длины. При существовании кратчайшего маршрута существует и кратчайший простой путь, но некоторые кратчайшие маршруты могут повторять вершины.

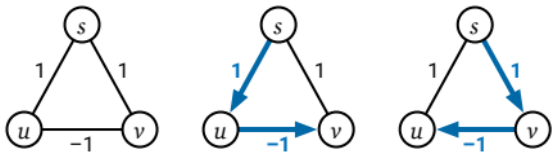


Figure 8.4. An undirected graph where shortest paths from *s* are unique but do not define a tree.

Рис. 8.4. Неориентированный граф, в котором кратчайшие пути из s единственны, но не образуют дерева.

Полное рассмотрение неориентированных графов с отрицательными рёбрами выходит за рамки этой книги. Упомяну лишь для тех, кто захочет поискать подробности, что один кратчайший путь в неориентированном графе с отрицательными рёбрами можно вычислить за время $O(VE + V^2 \log V)$ сведением к задаче о максимальном взвешенном паросочетании.

8.3. Единственный алгоритм кратчайших путей из одного источника

Как и в случае обхода графа и минимальных остовных деревьев, множество различных алгоритмов поиска кратчайших путей из одного источника можно описать как частные случаи единого общего алгоритма. Впервые его предложил Лестер Форд в 1956 году; независимо его описали Джордж Данциг в 1957 году[3] и Джордж Минти в 1958 году. Каждая вершина v хранит два значения, которые индуктивно задают предварительный кратчайший путь из s в v:

- $\text{dist}(v)$ — длина предварительного кратчайшего пути из s в v или ∞, если такой путь отсутствует;
- $\text{pred}(v)$ — предшественник v в этом пути или `Null`, если такой вершины нет.

Указатели предшественников автоматически задают предварительное дерево кратчайших путей с корнем s; эти указатели в точности соответствуют указателям родителей в общем алгоритме обхода графа. В начале алгоритма расстояния и предшественники инициализируются так:

```
InitSSSP(s):
    dist(s) ← 0
    pred(s) ← Null
    для всех вершин v ≠ s
        dist(v) ← ∞
        pred(v) ← Null
```

Во время выполнения алгоритма ребро $u \to v$ называется *напряжённым*, если $\mathrm{dist}(u) + w(u \to v) < \mathrm{dist}(v)$. Если ребро напряжено, предварительный кратчайший путь из s в v явно неверен, поскольку путь $s \rightsquigarrow u \to v$ короче. Эту очевидную переоценку можно исправить или хотя бы улучшить, выполнив *релаксацию* ребра:

```
Relax(u → v):
    dist(v) ← dist(u) + w(u → v)
    pred(v) ← u
```

Теперь общий алгоритм Форда описывается одной строкой:

Повторять релаксацию напряжённых рёбер, пока таких рёбер не останется.

```
FordSSSP(s):
    InitSSSP(s)
    пока есть хотя бы одно напряжённое ребро
        Relax(любое напряжённое ребро)
```

Если `FordSSSP` в конце концов завершается, потому что напряжённых рёбер не осталось, указатели предшественников правильно задают дерево кратчайших путей, а каждое значение $\mathrm{dist}(v)$ равно настоящему кратчайшему расстоянию от s до v. В частности, если v недостижима из s, то $\mathrm{dist}(v) = \infty$; если же из s достижим отрицательный цикл, алгоритм никогда не завершится.

Корректность общего алгоритма Форда следует из последовательности более простых утверждений:

1. В любой момент работы алгоритма для каждой вершины v значение $\mathrm{dist}(v)$ либо равно ∞, либо является длиной некоторого маршрута из s в v. Это доказывается индукцией по числу релаксаций.
2. Если в графе нет отрицательных циклов, $\mathrm{dist}(v)$ равно либо ∞, либо длине некоторого простого пути из s в v. В частности, если $\mathrm{dist}(v)$ — длина маршрута из s в v, содержащего ориентированный цикл, этот цикл обязательно имеет отрицательную длину. Отсюда следует, что при отсутствии отрицательных циклов алгоритм релаксации в конце концов останавливается, поскольку простых путей в G конечное число.
3. Если в G нет напряжённых рёбер, то для каждой вершины v расстояние $\mathrm{dist}(v)$ равно длине пути предшественников $s \to \cdots \to \mathrm{pred}(\mathrm{pred}(v)) \to \mathrm{pred}(v) \to v$. В частности, если v нарушает это условие, а её предшественник $\mathrm{pred}(v)$ — нет, то ребро $\mathrm{pred}(v) \to v$ напряжено.
4. Если в G нет напряжённых рёбер, то для каждой вершины v путь предшественников $s \to \cdots \to \mathrm{pred}(\mathrm{pred}(v)) \to \mathrm{pred}(v) \to v$ действительно является кратчайшим путём из s в v. В частности, если v нарушает это условие, а её предшественник u в некотором кратчайшем пути — нет, то ребро $u \to v$ напряжено. Отсюда также следует, что если в G имеется отрицательный цикл, некоторое ребро всегда напряжено, поэтому общий алгоритм никогда не останавливается.

Примечание переводчика. Утверждение о незавершении относится к отрицательным циклам, достижимым из источника s; недостижимые из s циклы релаксации не вызывают. Утверждения о путях предшественников подразумевают достижимые вершины.

До сих пор я ничего не сказал о том, как находить напряжённые рёбра или какие именно из них релаксировать, если таких рёбер несколько. Как и в случае поиска в произвольном порядке, существуют разные конкретные реализации общего алгоритма релаксации Форда. Но, в отличие от поиска в произвольном порядке, эффективность и корректность каждой стратегии зависят от структуры входного графа.

В оставшейся части главы рассматриваются четыре наиболее распространённые реализации алгоритма Форда, каждая из которых лучше всего подходит для определённого класса входных графов. Остальные подробности общего доказательства корректности я оставляю в упражнениях, а вместо этого привожу более содержательные и самостоятельные доказательства корректности каждого из четырёх конкретных алгоритмов.

[3] Если точнее, Данциг показал, что задачу о кратчайших путях можно сформулировать как задачу линейного программирования, а затем описал интерпретацию своего симплекс-метода в терминах исходного графа. Его описание по существу эквивалентно стратегии релаксации Форда.

8.4. Невзвешенные графы: поиск в ширину

В простейшем частном случае задачи о кратчайших путях все рёбра имеют вес 1, а длина пути равна числу его рёбер. Этот случай решается разновидностью нашего общего алгоритма обхода графа, которая называется *поиском в ширину*. Поиск в ширину часто приписывают Эдварду Муру, который описал его в 1957 году как «алгоритм A» — первый опубликованный метод поиска кратчайшего пути через лабиринт.[4] Особенно часто применительно к разводке сверхбольших интегральных схем и планированию маршрутов роботов поиск в ширину приписывают Чин Яну Ли, описавшему несколько применений «алгоритма A» Мура с должной ссылкой на Мура в 1961 году. Однако ещё в 1945 году, более чем за десятилетие до работы Мура над лабиринтами, Конрад Цузе описал реализацию поиска в ширину для подсчёта и пометки компонент несвязного графа.[6]

Поиск в ширину поддерживает очередь вершин с дисциплиной «первым вошёл — первым вышел», первоначально содержащую только исходную вершину s. На каждой итерации алгоритм извлекает вершину u из начала очереди и рассматривает каждое исходящее из неё ребро $u \to v$. Обнаружив напряжённое ребро $u \to v$, алгоритм релаксирует его и помещает вершину v в конец очереди. Алгоритм завершается, когда очередь становится пустой.

```
BFS(s):
    InitSSSP(s)
    Push(s)
    пока очередь не пуста
        u ← Pull()
        для всех рёбер u → v
            если dist(v) > dist(u) + 1       ⟨⟨если u → v напряжено⟩⟩
                dist(v) ← dist(u) + 1        ⟨⟨релаксация u → v⟩⟩
```

```
                pred(v) ← u
                Push(v)
```

Поиск в ширину несколько проще анализировать, если разделить его выполнение на фазы с помощью воображаемого жетона. Перед тем как извлекать вершины, помещаем жетон в очередь. Текущая фаза заканчивается, когда мы извлекаем жетон из очереди; следующая начинается, когда снова помещаем жетон в очередь. Таким образом, первая фаза состоит целиком из просмотра исходной вершины s. Алгоритм завершается, когда в очереди остаётся только жетон. Изменённый алгоритм приведён на рисунке 8.5, а пример его работы — на рисунке 8.6. Подчеркну: эти изменения нужны только для удобства анализа; с жетоном или без него алгоритм помещает и извлекает вершины в одинаковом порядке, просматривает рёбра в одинаковом порядке и выдаёт совершенно одинаковые расстояния и предшественников.

```
BFSWithToken(s):
    InitSSSP(s)
    Push(s)
    Push(✠)                          ⟨⟨начать первую фазу⟩⟩
    пока в очереди есть хотя бы одна вершина
        u ← Pull()
        если u = ✠
            Push(✠)                  ⟨⟨начать следующую фазу⟩⟩
        иначе
            для всех рёбер u → v
                если dist(v) > dist(u) + 1  ⟨⟨если u → v напряжено⟩⟩
                    dist(v) ← dist(u) + 1   ⟨⟨релаксация u → v⟩⟩
                    pred(v) ← u
                    Push(v)
```

Рисунок 8.5. Поиск в ширину с жетоном конца фазы ✠; в оригинале строки, добавленные только для анализа, выделены красным полужирным шрифтом.

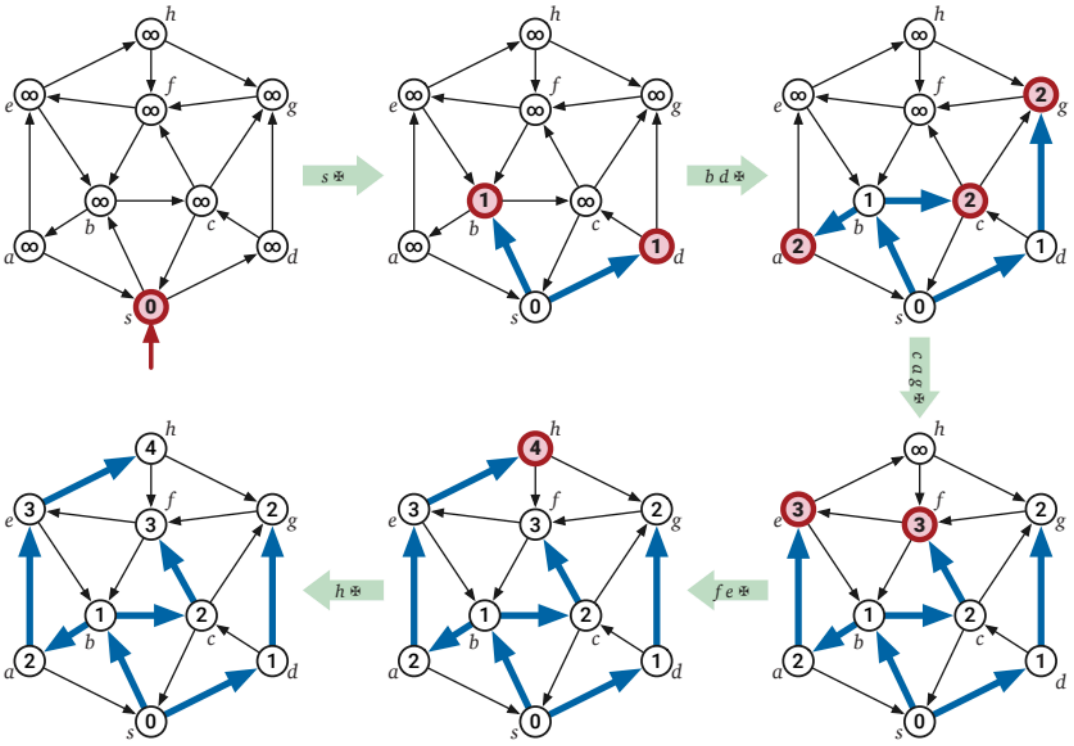


Figure 8.6 A complete run of breadth-first search in a directed graph. Vertices are pulled from the

Рисунок 8.6. Полное выполнение поиска в ширину в ориентированном графе. Вершины извлекаются из очереди в порядке `s, ✠, b, d, ✠, c, a, g, ✠, f, e, ✠, h, ✠, ✠`, где ✠ — жетон конца фазы. Полужирные вершины находятся в очереди в конце каждой фазы; полужирные рёбра задают строящееся дерево кратчайших путей.

[4] Мура побудил к работе недостаток робота Клода Шеннона «Тесей», предназначенного для прохождения лабиринтов и построенного в 1950 году. («Тесей» использовал мемоизированный поиск в глубину, реализованный на электромеханических реле; это почти наверняка была первая реализация поиска в глубину в графах.) По словам Мура, когда робота использовали на лабиринте с несколькими решениями, посетитель спросил, почему тот не всегда находит кратчайший путь. Мур и Шеннон независимо искали экономичные машинные методы; Шеннон нашёл несколько методов для аналоговых вычислений,[5] а Мур получил эти алгоритмы.

[5] Аналоговые методы поиска кратчайших путей через лабиринты предлагалось реализовывать с помощью шарикоподшипников, потоков жидкости или плазмы, волн химической реакции, хемотаксиса, резисторных сетей, электрических цепей со светодиодами, сетей мемристоров, тлеющего разряда в микрофлюидных чипах, растущих растений, слизевиков, амёб, муравьёв, пчёл, нематод и туристов.

[6] Конрад Цузе был одним из первых пионеров вычислительной техники: в конце 1930-х годов он спроектировал и построил свой первый программируемый компьютер, позднее названный Z1, из металлических полос и стержней в гостиной родителей. Z1 и его первоначальные чертежи были уничтожены во время британского авианалёта в 1944 году. В диссертации Цузе 1945 года описан самый первый высокоуровневый язык программирования Plankalkül. Первый

полный пример программы на Plankalkül в диссертации — реализация поиска в ширину для подсчёта компонент, с объяснением в псевдокоде и пошаговой иллюстрацией работы алгоритма на несвязном графе из восьми вершин. Из-за краха нацистского правительства Цузе не смог защитить диссертацию, а Plankalkül оставался неопубликованным до 1972 года. Первый компилятор Plankalkül создал Йоахим Хоман в 1975 году.

Подчеркну, что в следующей лемме $\mathrm{dist}(v)$ — всего лишь переменная, поддерживаемая алгоритмом. Хотя интуитивно она представляет предварительное кратчайшее расстояние, мы пока не можем считать, что она когда-либо действительно равна кратчайшему расстоянию из s в v. Не беспокойтесь, мы до этого доберёмся.

Лемма 8.1. Для любого целого $i \ge 0$ и любой вершины v в конце i-й фазы либо $\mathrm{dist}(v) = \infty$, либо $\mathrm{dist}(v) \le i$; при этом v находится в очереди тогда и только тогда, когда $\mathrm{dist}(v) = i$.

Доказательство. Докажем утверждение индукцией по i. База $i = 0$ очевидна: в начале первой фазы («в конце нулевой фазы») очередь содержит только начальную вершину s и жетон ✠, а `InitSSSP` только что присвоил $\mathrm{dist}(s) \leftarrow 0$ и $\mathrm{dist}(v) \leftarrow \infty$ для всех $v \ne s$.

Зафиксируем целое $i > 0$. По предположению индукции в начале i-й фазы очередь содержит все вершины u с $\mathrm{dist}(u) = i - 1$, за которыми следует жетон ✠. Следовательно, до извлечения жетона и завершения i-й фазы мы извлекаем каждую вершину u с $\mathrm{dist}(u) = i - 1$.

Для каждой такой вершины u мы рассматриваем все исходящие рёбра $u \to v$. Если $u \to v$ напряжено, присваиваем $\mathrm{dist}(v) \leftarrow \mathrm{dist}(u) + 1$, так что $\mathrm{dist}(v) = i$, и сразу помещаем v в очередь. Это единственные присваивания меткам расстояний во время i-й фазы. Следовательно, по индукции на всём протяжении i-й фазы очередь содержит несколько вершин с меткой расстояния $i - 1$, затем жетон и несколько вершин с меткой расстояния i. Непосредственно перед окончанием фазы очередь содержит жетон, за которым следуют вершины с меткой i. Более того, вершина v находится в этой очереди тогда и только тогда, когда $\mathrm{dist}(v)$ изменилось во время i-й фазы. Значит, в конце фазы очередь содержит все вершины v с $\mathrm{dist}(v) = i$. ∎

Лемма 8.1 означает, что основной цикл `BFS` присваивает метки расстояний в неубывающем порядке; с другой стороны, метка $\mathrm{dist}(v)$ каждой вершины никогда не увеличивается. Отсюда для каждой v строка `dist(v) ← dist(u) + 1` выполняется не более одного раза — на фазе $\mathrm{dist}(v)$. Аналогично:

- каждый указатель предшественника $\mathrm{pred}(v)$ изменяется не более одного раза, на фазе $\mathrm{dist}(v)$;
- каждая вершина v помещается в очередь не более одного раза, на фазе $\mathrm{dist}(v)$;
- каждая вершина u извлекается из очереди не более одного раза, на фазе $\mathrm{dist}(u) + 1$;
- для каждого ребра $u \to v$ сравнение `dist(v) > dist(u) + 1` выполняется не более одного раза, на фазе $\mathrm{dist}(u) + 1$.

В совокупности эти наблюдения означают, что поиск в ширину работает за $O(V + E)$. Интуитивно вершины очереди можно представлять как «волновой фронт», монотонно

расширяющийся от источника s и проходящий по каждой вершине и каждому ребру графа не более одного раза. Такую аналогию предложил ещё Чин Ян Ли в 1961 году, вдохновлённый визуализациями работы своей реализации алгоритма А Мура.

Эти наблюдения также показывают, что условие `dist(v) > dist(u) + 1` можно заменить, возможно, более простой проверкой `dist(v) = ∞`. Тогда расстояния играют ту же роль, что и пометки в других алгоритмах обхода графов, обеспечивающие посещение каждой вершины только один раз. А именно, вершина «помечена» тогда и только тогда, когда её метка расстояния конечна.

Но нам ещё нужно доказать корректность окончательных меток расстояния!

Теорема 8.2. Когда `BFS` завершается, $\mathrm{dist}(v)$ для каждой вершины v равно длине кратчайшего пути в G из s в v.

Доказательство. Зафиксируем произвольную вершину v и рассмотрим произвольный путь $v_0 \to v_1 \to \cdots \to v_\ell$ в G, где $v_0 = s$ и $v_\ell = v$. Утверждается, что $\mathrm{dist}(v_j) \le j$ для каждого индекса j; в частности, $\mathrm{dist}(v) \le \ell$. Докажем это по индукции:

- Очевидно, $\mathrm{dist}(v_0) = \mathrm{dist}(s) = 0$.
- Для любого $j > 0$ предположение индукции даёт $\mathrm{dist}(v_{j-1}) \le j - 1$. Сразу после извлечения v_{j-1} из очереди либо уже выполнено $\mathrm{dist}(v_j) \le \mathrm{dist}(v_{j-1}) + 1$, либо мы присваиваем $\mathrm{dist}(v_j) \leftarrow \mathrm{dist}(v_{j-1}) + 1$. В любом случае $\mathrm{dist}(v_j) \le \mathrm{dist}(v_{j-1}) + 1 \le j$.

Мы доказали, что $\mathrm{dist}(v)$ не превосходит длину произвольного пути из s в v; следовательно, оно не превосходит длину кратчайшего пути из s в v. Аналогичное доказательство по индукции показывает, что $\mathrm{dist}(v)$ равно длине пути предшественников $s \to \cdots \to \mathrm{pred}(\mathrm{pred}(v)) \to \mathrm{pred}(v) \to v$, а значит, этот путь кратчайший. ■

8.5. Ориентированные ациклические графы: поиск в глубину

В ориентированных ациклических графах кратчайшие пути тоже легко вычислить, даже если рёбра имеют веса и, в частности, некоторые веса отрицательны. (Об отрицательных циклах беспокоиться не приходится, ведь по определению в ориентированных ациклических графах нет никаких циклов!) По сути это совершенно стандартный алгоритм динамического программирования.

Пусть G — ориентированный граф со взвешенными рёбрами, а s — фиксированная начальная вершина. Для каждой вершины v пусть $\mathrm{dist}(v)$ обозначает длину кратчайшего пути в G из s в v. Эта функция удовлетворяет простому рекуррентному соотношению:

$$\mathrm{dist}(v) = \begin{cases} 0, & v = s, \\ \min_{u \to v}(\mathrm{dist}(u) + w(u \to v)), & \text{иначе.} \end{cases}$$

Вообще говоря, это равенство выполняется для всех ориентированных графов, однако является рекуррентным соотношением только для ориентированных ациклических графов. Если бы

входной граф G содержал цикл, рекурсивное вычисление функции зациклилось бы; но поскольку G ацикличен, каждый рекурсивный вызов посещает более раннюю вершину в топологическом порядке.

Граф зависимостей этой рекуррентной формулы — обращённый входной граф G: подзадача $\text{dist}(v)$ зависит от $\text{dist}(u)$ тогда и только тогда, когда $u \to v$ — ребро G. Поэтому расстояния для всех вершин вычисляются за $O(V + E)$ поиском в глубину в обращённом графе G и рассмотрением вершин в обратном порядке обхода. Равносильно можно рассматривать вершины исходного графа G в топологическом порядке, как на рисунке 8.7.

Полученный алгоритм динамического программирования — ещё один пример общего алгоритма релаксации Форда! Чтобы сделать эту связь яснее, можно вынести инициализацию $\text{dist}(v)$ из основного цикла и добавить вычисление указателей предшественников, как на рисунке 8.8. Пример работы алгоритма приведён на рисунке 8.9.

```
DagSSSP(s):
    для всех вершин v в топологическом порядке
        если v = s
            dist(v) ← 0
        иначе
            dist(v) ← ∞
            для всех рёбер u → v
                если dist(v) > dist(u) + w(u → v)  ⟨⟨если u → v напряжено⟩⟩
                    dist(v) ← dist(u) + w(u → v)    ⟨⟨релаксация u → v⟩⟩
```

Рисунок 8.7. Вычисление кратчайших путей в ориентированном ациклическом графе с помощью динамического программирования.

```
DagSSSP(s):
    InitSSSP(s)
    для всех вершин v в топологическом порядке
        для всех рёбер u → v
            если u → v напряжено
                Relax(u → v)
```

Рисунок 8.8. Вычисление кратчайших путей в ориентированном ациклическом графе с помощью алгоритма Форда. (Это один и тот же алгоритм.)

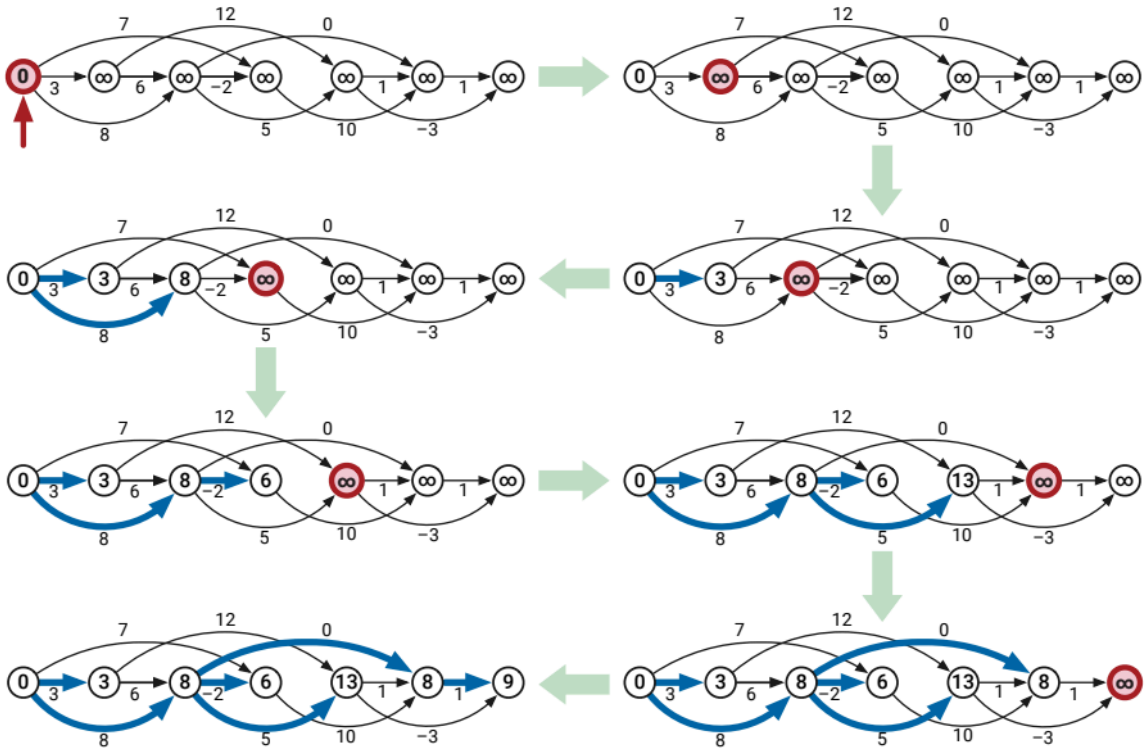


Figure 8.9. Computing shortest paths in a dag, by relaxing **incoming** edges in topological order. In

Рисунок 8.9. Вычисление кратчайших путей в ориентированном ациклическом графе релаксацией входящих рёбер в топологическом порядке. На каждой итерации полужирные рёбра обозначают предшественников, а полужирная вершина будет просмотрена следующей. Сравните с рисунком 8.10.

`DagSSSP` немного отличается от поиска в ширину и других реализаций стратегии релаксации Форда. Когда другие алгоритмы рассматривают вершину, они пытаются релаксировать каждое исходящее из неё ребро, интуитивно выталкивая волновой фронт вперёд от источника; `DagSSSP` же пытается релаксировать каждое входящее в вершину ребро, интуитивно вытягивая фронт вперёд.

Однако если изменить `DagSSSP`, релаксируя исходящие рёбра вместо входящих, мы получим другой алгоритм, который вычисляет кратчайшие пути в ориентированных ациклических графах за $O(V + E)$ и больше похож на остальные алгоритмы кратчайших путей.

```
PushDagSSSP(s):
    InitSSSP(s)
    для всех вершин u в топологическом порядке
        для всех исходящих рёбер u → v
            если u → v напряжено
                Relax(u → v)
```

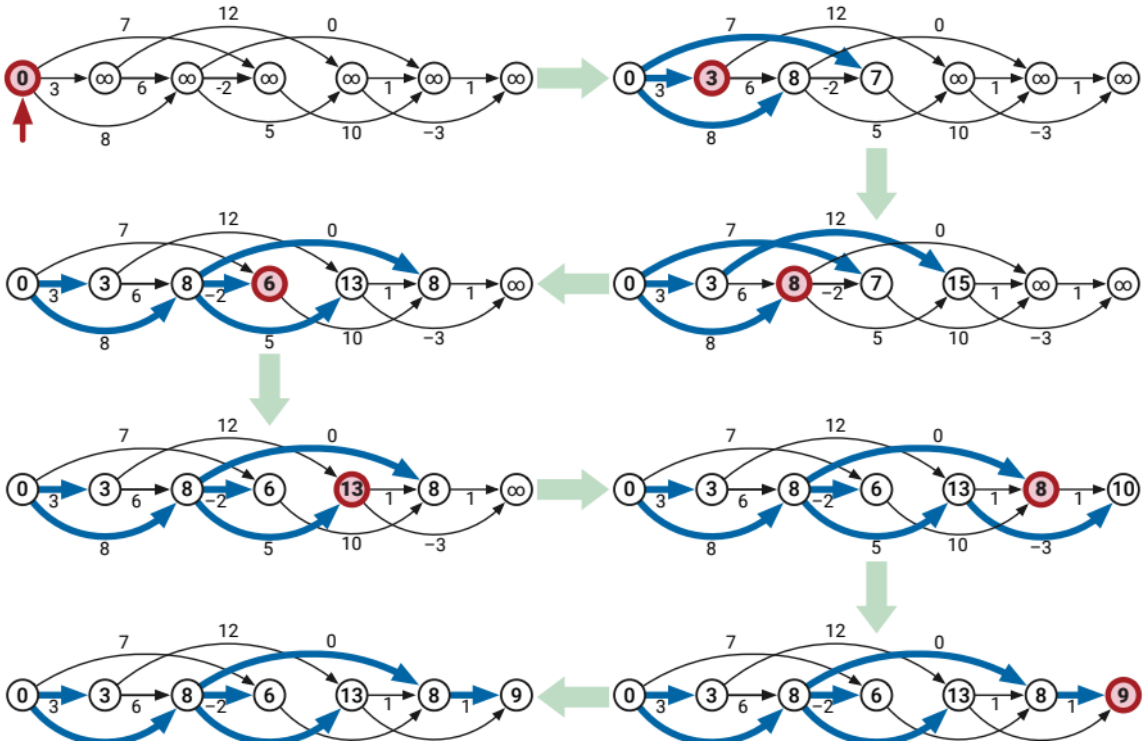


Рисунок 8.10. Вычисление кратчайших путей в ориентированном ациклическом графе релаксацией исходящих рёбер в топологическом порядке. На каждой итерации полужирные рёбра обозначают предшественников, а полужирная вершина будет просмотрена следующей. Сравните с рисунком 8.9.

На рисунке 8.10 показано выполнение изменённого алгоритма на том же графе, что и на рисунке 8.9. Корректность `PushDagSSSP` непосредственно следует из корректности общей стратегии релаксации Форда, но её несложно доказать и напрямую, индукцией по вершинам в топологическом порядке.

8.6. Сначала лучший: алгоритм Дейкстры

Если заменить очередь FIFO в поиске в ширину очередью с приоритетом, в которой ключ вершины v — её предварительное расстояние $\mathrm{dist}(v)$, получится алгоритм, впервые «опубликованный» в 1957 году группой исследователей Технологического института Кейса под руководством Майкла Лейзорека в ежегодном отчёте об исследовании для отдела боевой разработки испытательного полигона радиоэлектроники армии США. Тот же алгоритм независимо открыл Эдсгер Дейкстра в 1956 году (но опубликовал лишь в 1959-м), затем Джордж Минти некоторое время до 1960 года, а затем Питер Уайтинг и Джон Хиллер в 1960 году. Почти идентичный алгоритм описал также Джордж Данциг в 1958 году. Хотя некоторые ранние источники называли его «алгоритмом Минти», сейчас этот подход повсеместно

известен как «алгоритм Дейкстры», в полном соответствии с законом Стиглера.[7] Псевдокод приведён на рисунке 8.11.

```
Dijkstra(s):
    InitSSSP(s)
    Insert(s, 0)
    пока очередь с приоритетом не пуста
        u ← ExtractMin()
        для всех рёбер u → v
            если u → v напряжено
                Relax(u → v)
                если v находится в очереди с приоритетом
                    DecreaseKey(v, dist(v))
                иначе
                    Insert(v, dist(v))
```

Рисунок 8.11. Алгоритм Дейкстры.

Простое доказательство по индукции показывает, что на протяжении выполнения алгоритма ребро $u \to v$ напряжено только тогда, когда вершина u находится в очереди с приоритетом или является последней извлечённой из неё вершиной. Таким образом, алгоритм Дейкстры — реализация общей стратегии Форда, а значит, он правильно вычисляет кратчайшие пути при условии отсутствия отрицательных циклов в G.

[7] Я буду следовать этому общепринятому соглашению, несмотря на историческую неточность: отчасти потому, что вряд ли кто-то хочет читать об «алгоритме Лейзорека — Грея — Джонсона — Лэдью — Микера — Петри — Сейтца — Данцига — Дейкстры — Минти — Уайтинга — Хиллера», а отчасти потому, что работы, которые на самом деле не опубликованы, не считаются.

Без отрицательных рёбер

Алгоритм Дейкстры особенно хорошо ведёт себя, когда во входном графе нет рёбер отрицательного веса. В этом случае алгоритм, подобно поиску в ширину, интуитивно расширяет волновой фронт от источника s, проходя вершины в порядке возрастания расстояния от s. На рисунке 8.12 показан пример его работы.

shows the algorithm in action.

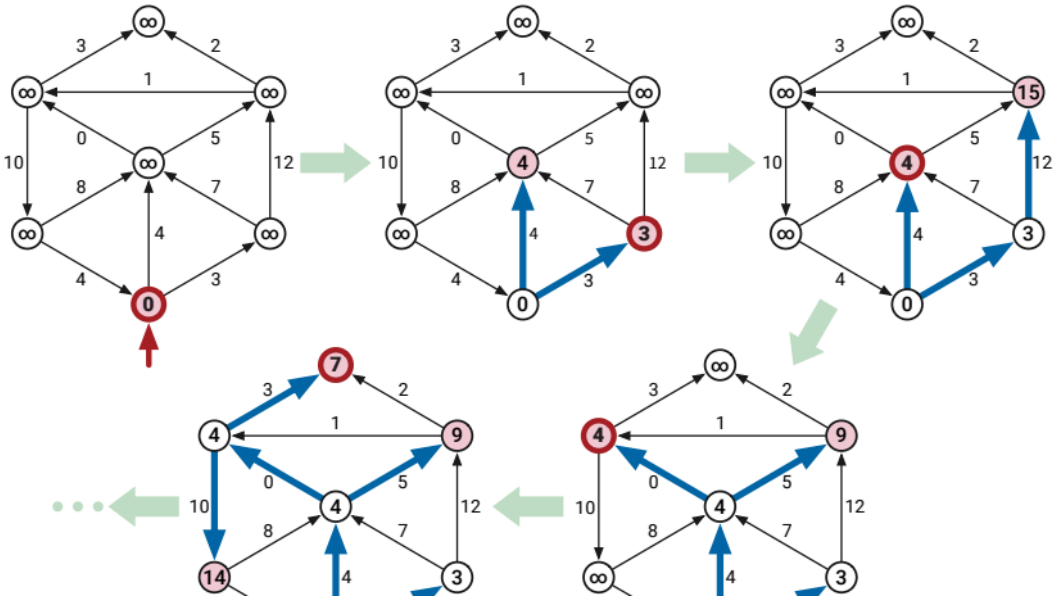


Рисунок 8.12. Первые четыре итерации алгоритма Дейкстры на графе без отрицательных рёбер. На каждой итерации полужирные рёбра обозначают предшественников; затенённые вершины находятся в очереди с приоритетом; полужирная вершина будет просмотрена следующей. Остальные итерации не меняют расстояния или дерево кратчайших путей.

На основе этой интуиции о волновом фронте можно построить самостоятельное доказательство корректности алгоритма. Для каждого целого i обозначим через u_i вершину, возвращённую i-м вызовом `ExtractMin`, а через d_i — значение $\mathrm{dist}(u_i)$ сразу после извлечения. В частности, $u_1 = s$ и $d_1 = 0$. Пока нельзя считать, что вершины u_i различны: в принципе одна вершина может извлекаться несколько раз.

Лемма 8.3. Если в G нет рёбер отрицательного веса, то для любых $i < j$ выполняется $d_i \le d_j$.

Доказательство. Пусть в G нет рёбер отрицательного веса. Зафиксируем произвольный индекс i; достаточно доказать, что $d_{i+1} \ge d_i$. Рассмотрим два случая.

- Если G содержит ребро $u_i \to u_{i+1}$ и это ребро релаксируется на i-й итерации основного цикла, в конце итерации $\mathrm{dist}(u_{i+1}) = \mathrm{dist}(u_i) + w(u_i \to u_{i+1}) \ge \mathrm{dist}(u_i)$, поскольку все веса неотрицательны.
- Иначе в начале i-й итерации u_{i+1} уже должна находиться в очереди с приоритетом, причём $\mathrm{dist}(u_{i+1}) \ge \mathrm{dist}(u_i)$, поскольку `ExtractMin` вернула вершину u_i. Кроме того, $\mathrm{dist}(u_{i+1})$ не меняется на i-й итерации.

В обоих случаях $d_{i+1} \ge d_i$. Лемма сразу следует по индукции. ■

Лемма 8.4. Если в G нет рёбер отрицательного веса, каждая вершина G извлекается из очереди с приоритетом не более одного раза.

Доказательство. Пусть v извлекается более одного раза. А именно, пусть v извлекается на i-й итерации основного цикла, снова добавляется в очередь на j-й и снова извлекается на k-й, где

$i < j < k$. В обозначениях предыдущего доказательства $v = u_i = u_k$.

Метка расстояния $\mathrm{dist}(v)$ никогда не увеличивается. Более того, на j-й итерации непосредственно перед повторным добавлением v она строго уменьшается. Поэтому $d_i > d_k$. Следовательно, по предыдущей лемме в G есть хотя бы одно ребро отрицательного веса. ■

Из леммы 8.4 непосредственно следует, что каждая вершина просматривается не более одного раза и каждое ребро релаксируется не более одного раза. Однако, в отличие от поиска в ширину, каждая метка $\mathrm{dist}(v)$ может изменяться многократно. Когда $\mathrm{dist}(v)$ впервые меняется с ∞, мы помещаем v в очередь; после этого за каждым изменением $\mathrm{dist}(v)$ следует вызов `DecreaseKey`. После извлечения v из очереди её метка расстояния больше никогда не меняется.

Остальная часть доказательства корректности почти совпадает с доказательством для поиска в ширину.

Теорема 8.5. Если в G нет рёбер отрицательного веса, то после завершения `Dijkstra` значение $\mathrm{dist}(v)$ для каждой вершины v равно длине кратчайшего пути в G из s в v.

Доказательство. Зафиксируем произвольную вершину v и рассмотрим произвольный путь $v_0 \to v_1 \to \cdots \to v_\ell$ в G, где $v_0 = s$ и $v_\ell = v$. Для каждого j пусть L_j — длина подпути $v_0 \to v_1 \to \cdots \to v_j$. Докажем индукцией, что $\mathrm{dist}(v_j) \le L_j$ для всех j.

- Очевидно, $\mathrm{dist}(v_0) = \mathrm{dist}(s) = 0 = L_0$.
- Для любого $j > 0$ предположение индукции даёт $\mathrm{dist}(v_{j-1}) \le L_{j-1}$. Сразу после извлечения v_{j-1} из очереди либо уже выполнено $\mathrm{dist}(v_j) \le \mathrm{dist}(v_{j-1}) + w(v_{j-1} \to v_j)$, либо мы присваиваем $\mathrm{dist}(v_j) \leftarrow \mathrm{dist}(v_{j-1}) + w(v_{j-1} \to v_j)$. В обоих случаях $\mathrm{dist}(v_j) \le \mathrm{dist}(v_{j-1}) + w(v_{j-1} \to v_j) \le L_j$.

Итак, $\mathrm{dist}(v)$ не превосходит длину любого пути из s в v, а значит, не превосходит длину кратчайшего такого пути. С другой стороны, аналогичная индукция показывает, что $\mathrm{dist}(v)$ равно длине пути предшественников $s \to \cdots \to \mathrm{pred}(\mathrm{pred}(v)) \to \mathrm{pred}(v) \to v$. ■

Остаётся только оценить время работы. Всего `Dijkstra` выполняет не более E операций `DecreaseKey` и не более V операций `Insert` и `ExtractMin`. Поэтому при реализации очереди с приоритетом стандартной двоичной кучей, поддерживающей каждую операцию за $O(\log V)$, `Dijkstra` работает за $O(E \log V)$.[8]

Если заранее известно, что входные графы не содержат отрицательных рёбер, алгоритм можно немного упростить: на этапе инициализации поместить все вершины в очередь с приоритетом, а в основном цикле вызывать только `DecreaseKey`, как на рисунке 8.13. Именно эту версию алгоритма Дейкстры приводят большинство учебников по алгоритмам, Википедия и даже оригинальная статья Дейкстры; именно её я описал как «поиск по наилучшему совпадению» в главе 5.

```
NonnegativeDijkstra(s):
    InitSSSP(s)
    для всех вершин v
```

```
        Insert(v, dist(v))
    пока очередь с приоритетом не пуста
        u ← ExtractMin()
        для всех рёбер u → v
            если u → v напряжено
                Relax(u → v)
                DecreaseKey(v, dist(v))
```

Рисунок 8.13. Алгоритм Дейкстры, немного упрощённый для графов без отрицательных рёбер. Отличия от `Dijkstra` в оригинале выделены красным полужирным шрифтом.

[8] В работах 1950-х годов о кратчайших путях очереди с приоритетом не упоминались. Дейкстра предложил на каждой итерации перебором просматривать все вершины волнового фронта; исходный алгоритм работает за $O(V^2)$, что при $E = \Omega(V^2)$ быстрее реализации с двоичной кучей! Минти предложил перебором просматривать все рёбра $u \to v$, для которых $\text{dist}(u)$ конечно, а $\text{dist}(v)$ нет; таким образом, его первоначальный алгоритм работает за $O(VE)$. Использование очереди с приоритетом на двоичной куче для получения почти линейного времени предложил Дональд Джонсон в 1977 году. Время можно улучшить до $O(E + V \log V)$ с помощью более сложной структуры очереди с приоритетом — фибоначчиевой кучи. Для целочисленных весов рёбер существуют ещё более быстрые алгоритмы, использующие ещё более сложные очереди с приоритетом.

♥ Отрицательные рёбра

Однако `NonnegativeDijkstra` неправильно вычисляет кратчайшие пути в графах с отрицательными рёбрами. Более того, даже если все веса рёбер положительны, `NonnegativeDijkstra` не быстрее `Dijkstra` ни теоретически, ни практически. По обеим причинам я считаю, что `Dijkstra` больше заслуживает названия «алгоритм Дейкстры». Даже Эдсгер Дейкстра согласился бы, что правильный алгоритм, который иногда, а на практике редко, медленный, лучше быстрого алгоритма, который работает не всегда!

К сожалению, при наличии отрицательных рёбер привычная интуиция «расширяющегося волнового фронта» больше не работает. Одну вершину можно извлечь несколько раз, одно ребро — несколько раз релаксировать, а расстояния могут обнаруживаться не в порядке возрастания. На рисунке 8.15 приведён пример выполнения, где верхняя левая вершина извлекается шесть раз, а каждое из трёх верхних рёбер релаксируется дважды.

Для графов без отрицательных циклов, но без других ограничений на веса рёбер, время работы `Dijkstra` в худшем случае на самом деле экспоненциально. На рисунке 8.14 показано особенно простое семейство графов, предложенное Дугласом Широм и Кристофом Вицгаллем, которое заставляет `Dijkstra` выполнять $\Theta(2^{V/2})$ релаксаций.[9] Более сложное семейство графов, которое я оставлю в качестве упражнения, заставляет выполнить $\Theta(2^V)$ релаксаций — наихудший возможный случай. Однако на практике алгоритм Дейкстры обычно быстр даже для графов с отрицательными рёбрами.

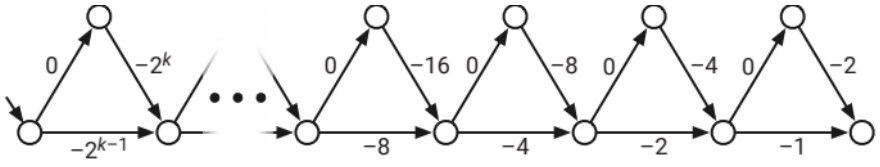


Figure 8.14. A directed graph with negative edges that forces DIJKSTRA to run in exponential time

Рисунок 8.14. Ориентированный граф с отрицательными рёбрами, заставляющий `Dijkstra` работать экспоненциальное время.

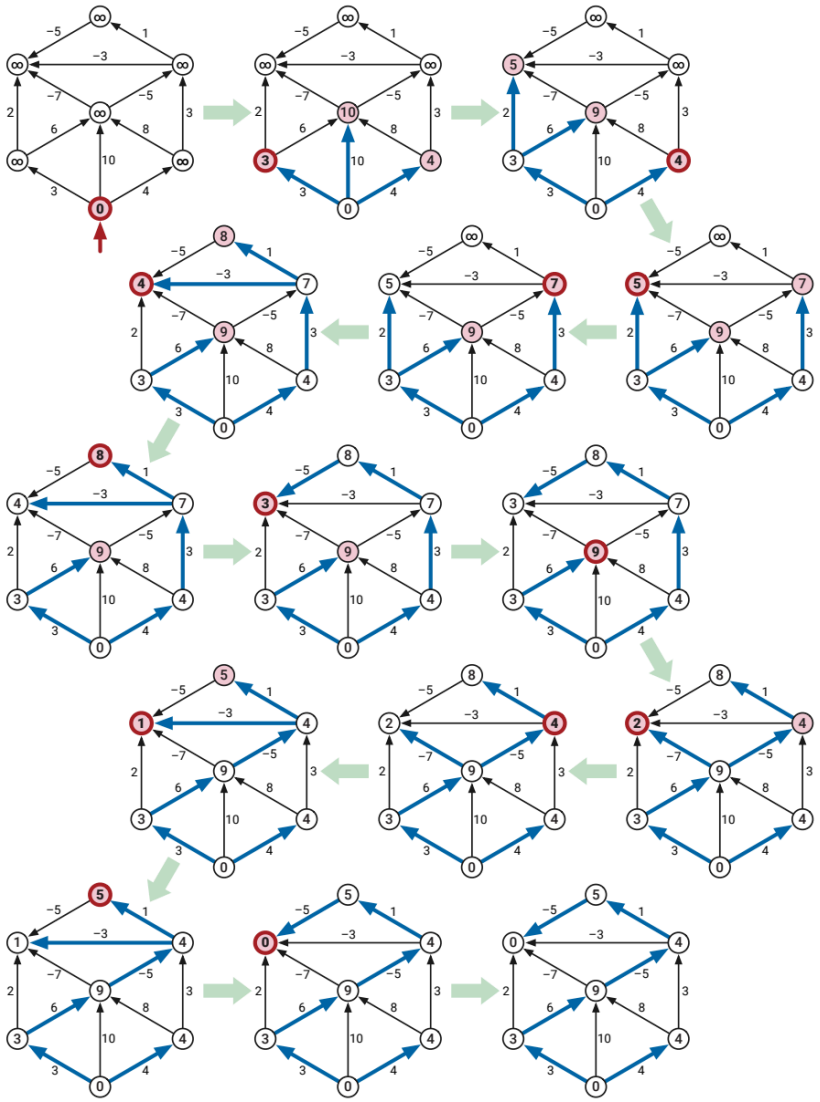


Figure 8.15. A complete run of Dijkstra's algorithm on a graph with negative edges. At each iteration,

Рисунок 8.15. Полное выполнение алгоритма Дейкстры на графе с отрицательными рёбрами. На каждой итерации полужирные рёбра обозначают предшественников, затенённые вершины находятся в очереди с приоритетом, а полужирная вершина будет просмотрена следующей. Сравните с рисунком 8.17.

[9] Забавно, что пример Шира и Вицгалля — ориентированный ациклический граф лишь с $O(V)$ рёбрами, поэтому кратчайшие пути можно вычислить всего за $O(V)$, даже если мы не заметили, что зигзагообразный путь вверху является деревом кратчайших путей.

8.7. Релаксируйте ВСЕ рёбра: алгоритм Беллмана — Форда

Простейшую реализацию общего алгоритма Форда для кратчайших путей впервые набросал Альфонсо Шимбел в 1954 году, подробнее описал Эдвард Мур в 1957 году, а независимо заново открыли Макс Вудбери и Джордж Данциг в 1957 году, Ричард Беллман в 1958 году и Джордж Минти в 1958 году. (Ни Вудбери и Данциг, ни Минти свои алгоритмы не опубликовали.) В полном соответствии с законом Стиглера алгоритм почти повсеместно известен как алгоритм Беллмана — Форда,[10] поскольку Беллман явно использовал формулировку релаксации рёбер, предложенную Фордом в 1956 году, хотя некоторые авторы называют его алгоритмом «Беллмана — Калабы»[11], а несколько ранних источников — алгоритмом «Беллмана — Шимбела».

Алгоритм Шимбела / Мура / Вудбери — Данцига / Беллмана — Форда / Калабы / Минти / Брош[12] можно описать одной строкой:

Беллман — Форд: релаксируйте ВСЕ напряжённые рёбра, затем повторите.

```
BellmanFord(s):
    InitSSSP(s)
    пока есть хотя бы одно напряжённое ребро
        для каждого ребра u → v
            если u → v напряжено
                Relax(u → v)
```

Следующая лемма — ключ к доказательству и корректности, и эффективности алгоритма Беллмана — Форда. Для каждой вершины v и неотрицательного целого i обозначим через $\text{dist}_{\le i}(v)$ длину кратчайшего маршрута в G из s в v, содержащего не более i рёбер. В частности, $\text{dist}_{\le 0}(s) = 0$, а $\text{dist}_{\le 0}(v) = \infty$ для всех $v \neq s$.

Лемма 8.6. Для любой вершины v и неотрицательного целого i после i итераций основного цикла `BellmanFord` выполняется $\text{dist}(v) \le \text{dist}_{\le i}(v)$.

Доказательство. Докажем индукцией по i. База $i = 0$ очевидна, поэтому пусть $i > 0$. Зафиксируем вершину v и выберем кратчайший маршрут W из s в v, содержащий не более i рёбер, произвольно разрешая равенства. По определению длина W равна $\text{dist}_{\le i}(v)$. Возможны два случая.

- Если W не содержит рёбер, это тривиальный маршрут из s в s, поэтому $v = s$ и $\text{dist}_{\le i}(s) = 0$. В `InitSSSP` мы присвоили $\text{dist}(s) \leftarrow 0$, а $\text{dist}(s)$ никогда не увеличивается, поэтому всегда $\text{dist}(s) \le 0$.
- Иначе пусть $u \to v$ — последнее ребро W. По предположению индукции после $i-1$ итераций $\text{dist}(u) \le \text{dist}_{\le i-1}(u)$. Когда на i-й итерации внешнего цикла внутренний цикл рассматривает ребро $u \to v$, либо уже выполнено $\text{dist}(v) \le \text{dist}(u) + w(u \to v)$,

либо мы присваиваем $\text{dist}(v) \leftarrow \text{dist}(u) + w(u \to v)$. В обоих случаях $\text{dist}(v) \leq \text{dist}_{\leq i-1}(u) + w(u \to v) = \text{dist}_{\leq i}(v)$. Как обычно, $\text{dist}(v)$ не может увеличиться, хотя до конца i-й итерации внешнего цикла может ещё уменьшиться.

В обоих случаях в конце i-й итерации $\text{dist}(v) \leq \text{dist}_{\leq i}(v)$. ■

Если во входном графе нет отрицательных циклов, кратчайший маршрут из s в любую другую достижимую вершину можно выбрать простым путём не более чем из $V - 1$ рёбер; следовательно, алгоритм Беллмана — Форда останавливается с правильными расстояниями не более чем через $V - 1$ итераций. Иными словами, если после $V - 1$ итераций некоторое ребро всё ещё напряжено, во входном графе обязательно есть отрицательный цикл, достижимый из s! Поэтому алгоритм можно переписать конкретнее:

```
BellmanFord(s):
    InitSSSP(s)
    повторить V – 1 раз
        для каждого ребра u → v
            если u → v напряжено
                Relax(u → v)
    для каждого ребра u → v
        если u → v напряжено
            вернуть «Отрицательный цикл!»
```

Каждая итерация внутреннего цикла очевидно требует $O(E)$ времени, поэтому общий алгоритм работает за $O(VE)$. Таким образом, алгоритм Беллмана — Форда всегда эффективен, даже когда граф содержит отрицательные рёбра и даже отрицательные циклы. Однако при неотрицательных весах алгоритм Дейкстры быстрее, по крайней мере в худшем случае. (На практике алгоритм Дейкстры часто быстрее и на графах с отрицательными рёбрами.)

[10] Я следую общепринятому соглашению, несмотря на историческую неточность: отчасти потому, что вряд ли кто-то захочет читать об «алгоритме Шимбела / Мура / Вудбери — Данцига / Беллмана — Форда / Калабы / Минти», а отчасти потому, что мне надоело, когда на меня странно смотрят при упоминании «алгоритма Шимбела».

[11] Вероятнее всего, это название отсылает к монографии Ричарда Беллмана и Роберта Калабы 1965 года о динамическом программировании и теории управления, где описан алгоритм Беллмана. В 1960 году Беллман и Калаба также опубликовали расширение алгоритма, вычисляющее k-е кратчайшие пути для любого постоянного k.

[12] Перечитайте всё в *Hyperbole and a Half*. А потом возьмите ещё одного кота, чтобы купить ему ещё один экземпляр книги.

Улучшение Мура

Ни Мур, ни Беллман не описывали алгоритм Беллмана — Форда в приведённой выше форме. Мур представил свою версию («алгоритм D») в той же статье, где предложил поиск в ширину («алгоритм A») для невзвешенных графов; фактически оба алгоритма почти идентичны. Хотя алгоритм Мура имеет то же время $O(VE)$ в худшем случае, что и `BellmanFord`, на практике он

часто значительно быстрее, поскольку интуитивно избегает проверки рёбер, которые «очевидно» не напряжены.

Мур получил алгоритм кратчайших путей во взвешенном графе двумя изменениями поиска в ширину. Во-первых, каждое `+1` в самом внутреннем цикле заменяется на `+w(u → v)` с учётом весов. Во-вторых, перед добавлением вершины проверяется, нет ли её уже в очереди FIFO, так что очередь всегда содержит не более одного экземпляра каждой вершины.[13]

Как и при предыдущем анализе поиска в ширину, введём жетон ✠, чтобы разбить выполнение на фазы. Каждая фаза начинается, когда жетон помещается в очередь, и заканчивается при его очередном извлечении. Как и `BFS`, алгоритм завершается, когда в очереди остаётся только жетон. Полученный алгоритм показан на рисунке 8.16.

```
Moore(s):
    InitSSSP(s)
    Push(s)
    Push(✠)                              ⟨⟨начать первую фазу⟩⟩
    пока очередь содержит хотя бы одну вершину
        u ← Pull()
        если u = ✠
            Push(✠)                      ⟨⟨начать следующую фазу⟩⟩
        иначе
            для всех рёбер u → v
                если u → v напряжено
                    Relax(u → v)
                    если v ещё нет в очереди
                        Push(v)
```

Рисунок 8.16. Алгоритм Мура для кратчайших путей. Строки с жетоном ✠, выделенные в оригинале красным полужирным шрифтом, нужны только для анализа.

Поскольку очередь в любой момент содержит не более одного экземпляра каждой вершины, каждая вершина извлекается из очереди не более одного раза за фазу и, следовательно, каждое ребро $u \to v$ проверяется на напряжённость не более одного раза за фазу. Кроме того, каждое ребро, напряжённое в начале фазы, релаксируется в течение этой фазы. (Некоторые рёбра, ставшие напряжёнными во время фазы, тоже могут быть релаксированы в той же фазе, а некоторые релаксированные рёбра могут снова стать напряжёнными.) Таким образом, `Moore` можно рассматривать как уточнение `BellmanFord`, которое использует очередь для поддержания напряжённых рёбер вместо перебора всех рёбер. В частности, аналогичное индуктивное доказательство даёт следующий аналог леммы 8.6.

Лемма 8.7. Для каждой вершины v и неотрицательного целого i после i фаз алгоритма `Moore` выполняется $\text{dist}(v) \le \text{dist}_{\le i}(v)$.

Следовательно, если во входном графе нет отрицательных циклов, `Moore` завершается не более чем за $V - 1$ фаз. На каждой фазе мы просматриваем каждую вершину не более одного раза и, значит, релаксируем каждое ребро не более одного раза, поэтому время одной фазы в худшем случае — $O(E)$, а общее время — $O(VE)$. На практике `Moore` часто вычисляет кратчайшие пути значительно быстрее `BellmanFord`, поскольку просматривает ребро $u \to v$ только если $\text{dist}(u)$ изменилось на предыдущей фазе.

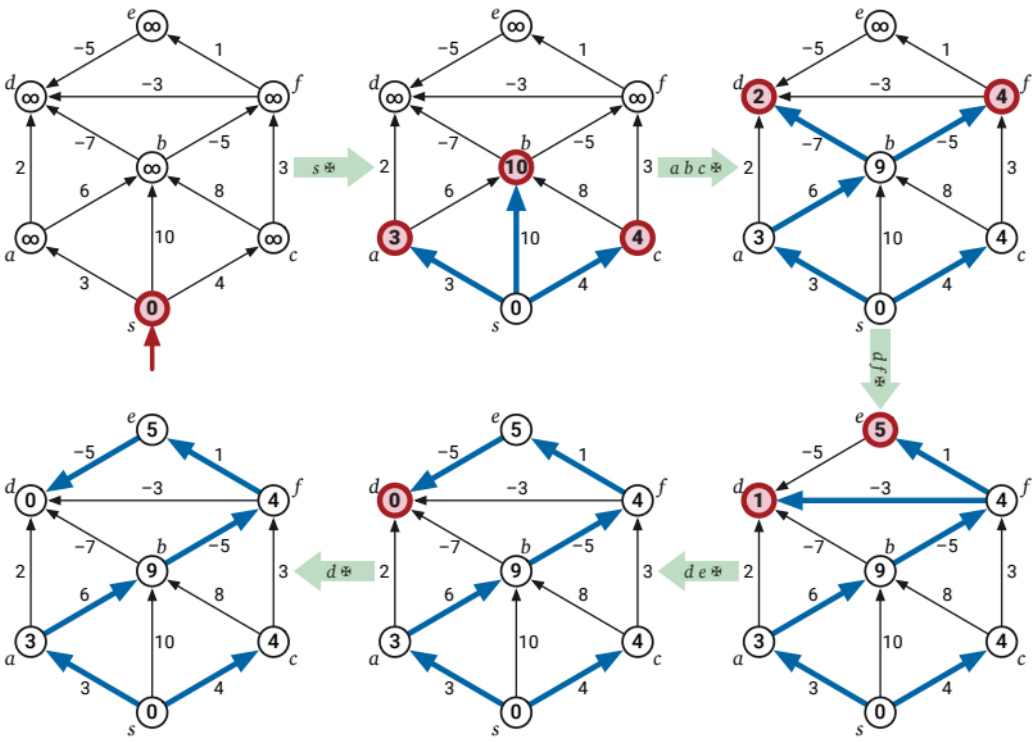


Figure 8.17 A complete run of Moore's algorithm on a directed graph with negative edges. Nodes are

Рисунок 8.17. Полное выполнение алгоритма Мура на ориентированном графе с отрицательными рёбрами. Вершины извлекаются в порядке `s`, ✠, `a`, `b`, `c`, ✠, `d`, `f`, ✠, `d`, `e`, ✠, `d`, ✠, ✠, где ✠ — жетон конца фазы. В начале каждой фазы полужирные рёбра обозначают предшественников, а затенённые вершины находятся в очереди. Сравните с рисунками 8.6 и 8.15.

Если входной граф содержит отрицательный цикл, алгоритм `Moore` никогда не завершится. К счастью, как и алгоритм Беллмана — Форда, его несложно изменить так, чтобы он сообщал об отрицательных циклах. Возможно, проще всего действительно поддерживать жетон и считать, сколько раз он извлечён из очереди. Тогда входной граф содержит отрицательный цикл тогда и только тогда, когда очередь непуста непосредственно после $(V - 1)$-го извлечения жетона. *Примечание переводчика:* речь идёт об отрицательном цикле, достижимом из s.

[13] Без этой проверки алгоритм Мура всё ещё корректен, но оценка времени $O(VE)$ уже не действует.

Формулировка через динамическое программирование

Как и почти всё остальное, носящее его имя, алгоритм кратчайших путей «Беллмана — Форда» Ричард Беллман вывел с помощью динамического программирования. Как обычно, начнём с рекурсивного определения кратчайших расстояний. Возникает соблазн использовать то же тождество, что и для ориентированных ациклических графов:

$$\operatorname{dist}(v) = \begin{cases} 0, & v = s, \\ \min_{u \to v}(\operatorname{dist}(u) + w(u \to v)), & \text{иначе.} \end{cases}$$

К сожалению, если входной граф не ацикличен, это соотношение не работает! Пусть входной граф содержит ориентированный цикл $u \to v \to w \to u$. Чтобы вычислить $\operatorname{dist}(w)$, сначала нужно $\operatorname{dist}(v)$; для $\operatorname{dist}(v)$ нужно $\operatorname{dist}(u)$; а для $\operatorname{dist}(u)$ снова нужно $\operatorname{dist}(w)$. При наличии любого ориентированного цикла мы застрянем в бесконечной рекурсии!

Чтобы получить корректное рекуррентное соотношение, добавим к функции расстояния структурный параметр, монотонно уменьшающийся при каждом рекурсивном вызове, так чтобы функция легко вычислялась, когда параметр достигает 0. Беллман выбрал максимальное число рёбер.[14]

Как и раньше, $\operatorname{dist}_{\le i}(v)$ — длина кратчайшего маршрута из s в v, содержащего не более i рёбер. Беллман заметил, что функция подчиняется следующему *уравнению Беллмана*:

$$\operatorname{dist}_{\le i}(v) = \begin{cases} 0, & i = 0 \text{ и } v = s, \\ \infty, & i = 0 \text{ и } v \neq s, \\ \min\left\{\operatorname{dist}_{\le i-1}(v),\ \min_{u \to v}\left(\operatorname{dist}_{\le i-1}(u) + w(u \to v)\right)\right\}, & \text{иначе.} \end{cases}$$

Предположим отсутствие отрицательных циклов: тогда нужно вычислить $\operatorname{dist}_{\le V-1}(v)$ для каждой вершины v. Ниже — прямое вычисление рекуррентной формулы динамическим программированием, где `dist[i,v]` хранит значение $\operatorname{dist}_{\le i}(v)$. Корректность итоговых кратчайших расстояний следует из корректности формулы, а время $O(VE)$ очевидно. По существу именно так Беллман представил свой алгоритм.

```
BellmanFordDP(s):
    dist[0,s] ← 0
    для каждой вершины v ≠ s
        dist[0,v] ← ∞
    для i ← 1 до V − 1
        для каждой вершины v
            dist[i,v] ← dist[i-1,v]
            для каждого ребра u → v
                если dist[i,v] > dist[i-1,u] + w(u → v)
                    dist[i,v] ← dist[i-1,u] + w(u → v)
```

Эту программу можно преобразовать в нашу первоначальную формулировку `BellmanFord` с помощью нескольких небольших оптимизаций. Во-первых, каждая итерация внешнего цикла рассматривает каждое ребро $u \to v$ ровно один раз, но порядок рассмотрения несуществен. Поэтому можно безопасно убрать один уровень отступа у последних трёх строк! Изменённый алгоритм может рассматривать рёбра в другом порядке, но по-прежнему правильно вычисляет $\operatorname{dist}_{\le i}(v)$ для всех i и v.

```
BellmanFordDP2(s):
    dist[0,s] ← 0
    для каждой вершины v ≠ s
        dist[0,v] ← ∞
    для i ← 1 до V − 1
        для каждой вершины v
```

```
            dist[i,v] ← dist[i-1,v]
        для каждого ребра u → v
            если dist[i,v] > dist[i-1,u] + w(u → v)
                dist[i,v] ← dist[i-1,u] + w(u → v)
```

Затем заменим индексы $i - 1$ в двух последних строках на i. Из-за этого изменения значения `dist[i,v]` могут приближаться к истинным кратчайшим расстояниям быстрее, но алгоритм по-прежнему правильно вычисляет окончательные расстояния. Вместо $\text{dist}[i, v] = \text{dist}_{\le i}(v)$ теперь для всех i и v выполняется $\text{dist}[i, v] \le \text{dist}_{\le i}(v)$, как в леммах 8.6 и 8.7.

```
BellmanFordDP3(s):
    dist[0,s] ← 0
    для каждой вершины v ≠ s
        dist[0,v] ← ∞
    для i ← 1 до V - 1
        для каждой вершины v
            dist[i,v] ← dist[i-1,v]
        для каждого ребра u → v
            если dist[i,v] > dist[i,u] + w(u → v)   ⟨⟨не i-1!⟩⟩
                dist[i,v] ← dist[i,u] + w(u → v)    ⟨⟨не i-1!⟩⟩
```

Но этот алгоритм несколько нелеп. На i-й итерации внешнего цикла мы сначала копируем $(i - 1)$-ю строку массива `dist[·,·]` в i-ю, а затем изменяем элементы i-й строки. Поэтому двумерный массив вообще не нужен: индекс итерации i совершенно избыточен! В последнем варианте храним только одномерный массив предварительных расстояний.

```
BellmanFordFinal(s):
    dist[s] ← 0
    для каждой вершины v ≠ s
        dist[v] ← ∞
    для i ← 1 до V - 1
        для каждого ребра u → v
            если dist[v] > dist[u] + w(u → v)
                dist[v] ← dist[u] + w(u → v)
```

Этот итоговый алгоритм динамического программирования почти идентичен исходной формулировке `BellmanFord`! Первые три строки инициализируют кратчайшие расстояния, а две последние релаксируют ребро $u \to v$, если оно напряжено. В `BellmanFordFinal` отсутствуют лишь две возможности прежней версии: поддержка указателей предшественников и обнаружение отрицательных циклов. К счастью, добавить их несложно.

[14] Как мы увидим в следующей главе, это не единственный разумный выбор.

Упражнения

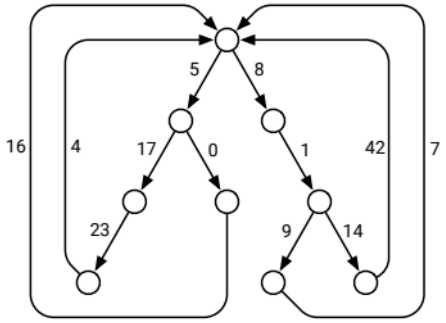
0. Пусть G — ориентированный граф с произвольными весами рёбер, которые могут быть положительными, отрицательными или нулевыми, возможно содержащий отрицательные циклы; s — произвольная вершина G.

(а) Пусть каждая вершина v хранит число $\mathrm{dist}(v)$, но не указатель предшественника. Опишите и проанализируйте алгоритм, определяющий, равно ли $\mathrm{dist}(v)$ кратчайшему расстоянию от s до v для каждой вершины v.

(б) Пусть вместо этого каждая вершина $v \neq s$ хранит указатель $\mathrm{pred}(v)$ на другую вершину G, но не расстояние. Опишите и проанализируйте алгоритм, определяющий, задают ли эти указатели дерево кратчайших путей из источника s.

1. *Дерево с обратными рёбрами* — взвешенный ориентированный граф, полученный из двоичного дерева добавлением ребра от каждого листа к корню. Все веса рёбер неотрицательны.

weight.



A looped tree.

(а) Сколько времени потребуется алгоритму Дейкстры для вычисления кратчайшего пути между двумя вершинами u и v такого дерева с n вершинами?

(б) Опишите и проанализируйте более быстрый алгоритм.

2. Пусть задан ориентированный граф G со взвешенными рёбрами и две вершины s и t.

(а) Опишите и проанализируйте алгоритм поиска кратчайшего пути из s в t, если ровно одно ребро G имеет отрицательный вес. [Подсказка: измените алгоритм Дейкстры. Или не меняйте.]

(б) Опишите и проанализируйте алгоритм поиска кратчайшего пути из s в t, если ровно k рёбер G имеют отрицательный вес. Как время работы зависит от k?

3. Пусть задан неориентированный граф G, каждая вершина которого имеет положительный вес.

(а) Опишите и проанализируйте алгоритм поиска остовного дерева G минимального суммарного веса. (Суммарный вес дерева — сумма весов его вершин.)

(б) Опишите и проанализируйте алгоритм поиска пути из заданной вершины s в другую заданную вершину t минимального суммарного веса. (Суммарный вес пути — сумма весов его вершин.)

[Подсказка: одна из этих задач тривиальна.]

4. Для любого ребра e графа G обозначим через $G \setminus e$ граф, полученный удалением e из G. Пусть заданы граф G и вершины s, t. *Задача о замещающих путях* состоит в вычислении кратчайшего расстояния от s до t в $G \setminus e$ для каждого ребра e графа G. Результат — массив из E расстояний, по одному для каждого ребра.

(а) Пусть G ориентирован, а кратчайший путь из s в t проходит через каждую вершину G. Опишите алгоритм решения этого частного случая за $O(E \log V)$.

♥ (б) Опишите алгоритм решения задачи о замещающих путях для произвольных неориентированных графов за $O(E \log V)$.

В обоих пунктах можно считать все веса рёбер неотрицательными. [Подсказка: если удалить ребро исходного кратчайшего пути, как пересекаются старый и новый кратчайшие пути?]

5. Пусть $G = (V, E)$ — связный ориентированный граф с неотрицательными весами рёбер, s, t — вершины G, а H — подграф G, полученный удалением некоторых рёбер. Мы хотим вернуть в H ровно одно ребро из G, чтобы кратчайший путь из s в t в полученном графе был как можно короче. Опишите и проанализируйте алгоритм выбора лучшего ребра за $O(E \log V)$.

6. (а) Опишите и проанализируйте модификацию алгоритма Беллмана — Форда, которая действительно возвращает отрицательный цикл, если он достижим из s, или дерево кратчайших путей, если такого цикла нет. Время работы должно остаться $O(VE)$.

(б) Опишите и проанализируйте модификацию алгоритма Беллмана — Форда, которая вычисляет правильные кратчайшие расстояния из s до всех остальных вершин даже при наличии отрицательных циклов. А именно, если какой-либо маршрут из s в v содержит отрицательный цикл, алгоритм должен завершиться со значением $\text{dist}(v) = -\infty$; иначе $\text{dist}(v)$ должно содержать длину кратчайшего пути из s в v. Время работы должно остаться $O(VE)$.

♥ (в) Повторите пункты (а) и (б) для общего алгоритма релаксации Форда. Можно считать, что неизменённый алгоритм завершается за $O(2^V)$ шагов при отсутствии отрицательного цикла; модифицированные алгоритмы также должны работать за $O(2^V)$.

7. Рассмотрим следующий ещё менее строгий вариант общего алгоритма релаксации Форда:

```
FellmanBored(s):
    InitSSSP(s)
    для i ← 1 до сколько угодно, чувак, мне всё равно
        e_i ← произвольное ребро G
        если e_i напряжено
            Relax(e_i)
```

Докажите: если `FellmanBored` рассматривает рёбра некоторого маршрута W, начинающегося в s, в их порядке вдоль W, то последняя метка расстояния в W не превосходит длину W. Формальнее: если рёбра маршрута $v_0 \to v_1 \to \cdots \to v_\ell$, где $v_0 = s$, образуют подпоследовательность рёбер $e_1, e_2, e_3, \ldots$, рассматриваемых `FellmanBored`, то $\text{dist}(v_\ell) \leq$

$\sum_{i=1}^{\ell} w(v_{i-1} \to v_i)$. [Подсказка: это свойство почти проще доказать, чем правильно сформулировать.]

8. В этой задаче рассматривается несколько способов обнаружения отрицательных циклов общим алгоритмом релаксации Форда.

(а) Докажите, что если $\mathrm{pred}(s)$ когда-либо меняется после `InitSSSP`, то входной граф содержит отрицательный цикл через s.

(б) Покажите, что $\mathrm{pred}(s)$ может никогда не измениться после `InitSSSP`, даже если входной граф содержит отрицательный цикл через s.

(в) Пусть P — текущий граф рёбер предшественников $\mathrm{pred}(v) \to v$, а X — множество всех напряжённых в данный момент рёбер; оба множества изменяются во время алгоритма. Докажите, что входной граф не содержит отрицательных циклов тогда и только тогда, когда $P \cup X$ всегда ацикличен.

(г) Пусть R — множество всех уже релаксированных рёбер; это множество растёт во время работы алгоритма. Докажите, что входной граф не содержит отрицательных циклов тогда и только тогда, когда R всегда ацикличен.

♥ **9.** Докажите, что при допущении отрицательных весов рёбер алгоритм Дейкстры в худшем случае выполняет $\Omega(2^V)$ релаксаций, даже если граф ацикличен. А именно: для каждого положительного целого n постройте взвешенный ориентированный ациклический граф G_n с n вершинами, на котором алгоритм Дейкстры вызывает `Relax` $\Omega(2^n)$ раз. [Подсказка: двоичный счётчик.]

♥ **10.** Докажите, что общий алгоритм релаксации Форда, а значит, и алгоритм Дейкстры, завершается не более чем после $O(2^V)$ релаксаций, если входной граф не содержит отрицательного цикла. [Подсказка: см. упражнение 8(г).]

11. Пусть задан ориентированный граф G, каждое ребро которого имеет отрицательный вес, и исходная вершина s. Опишите и проанализируйте эффективный алгоритм вычисления кратчайших расстояний из s до всех других вершин. А именно для каждой вершины t:

- если t недостижима из s, алгоритм должен выдать $\mathrm{dist}(t) = \infty$;
- если G содержит цикл, достижимый из s, и t достижима из этого цикла, кратчайшее расстояние от s до t не определено, поскольку существуют маршруты сколь угодно большой отрицательной длины; тогда алгоритм должен выдать $\mathrm{dist}(t) = -\infty$;
- если ни одно из предыдущих условий не выполняется, алгоритм должен выдать правильное кратчайшее расстояние от s до t.

12. Хотя мы обычно говорим о «кратчайшем» пути между двумя вершинами, граф может содержать несколько путей минимальной длины с одинаковыми концами. Даже во взвешенных графах часто желательно выбрать среди путей минимального веса путь с наименьшим числом рёбер; назовём его *лучшим путём* из s в t. Пусть задан ориентированный граф G с

положительными весами рёбер и исходная вершина s. Опишите и проанализируйте алгоритм вычисления лучших путей из s во все остальные вершины.

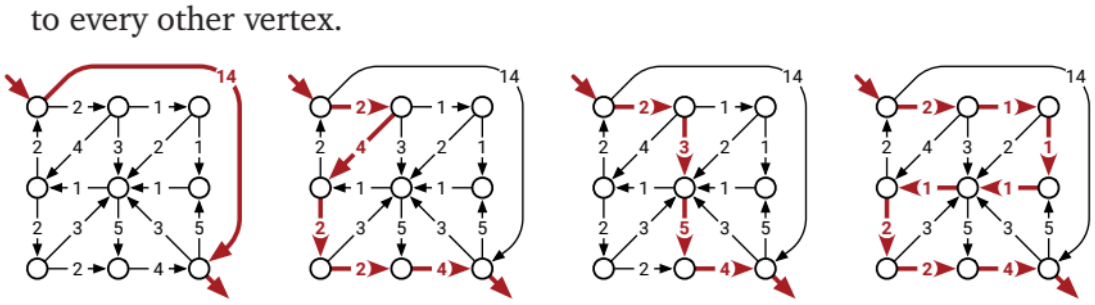


Рисунок 8.18. Четыре из многих кратчайших путей одинаковой длины. Первый путь — «лучший» из кратчайших.

13. Опишите и проанализируйте алгоритм подсчёта числа кратчайших путей из исходной вершины s в целевую t в произвольном ориентированном графе G со взвешенными рёбрами. Можно считать все веса положительными и все необходимые арифметические операции выполняемыми за $O(1)$. [Подсказка: вычислите кратчайшие расстояния от s до остальных вершин. Удалите все рёбра, которые не могут входить в кратчайший путь из s к другой вершине. Что останется?]

14. Вы только что нашли в Твитбуке лучшего друга начальной школы. Вы оба хотите встретиться как можно скорее, но живёте в разных городах далеко друг от друга. Чтобы сократить время в пути, вы договариваетесь встретиться в промежуточном городе, одновременно садитесь в машины и выезжаете навстречу друг другу. Но где именно встретиться?

Вам задан взвешенный граф $G = (V, E)$, вершины которого — города, а рёбра — дороги, непосредственно соединяющие города. Вес $w(e)$ каждого ребра e равен времени поездки между двумя городами. Заданы также вершина p, обозначающая ваше начальное положение, и вершина q, обозначающая начальное положение друга.

Опишите и проанализируйте алгоритм поиска целевой вершины t, позволяющей вам с другом встретиться как можно раньше.

15. Вас наняли велосипедистом в проект Giggle Highway View, который предоставит панорамные изображения всей национальной системы шоссе США. В качестве пилотного проекта вас просят проехать на велосипеде Giggle Highway-View Fixed-Gear Carbon-Fiber Bicycle от «Гигглплекса» в Портленде, штат Орегон, до «Гигглсбурга» в Уильямсберге, Бруклин, Нью-Йорк.

Вы безнадёжно зависимы от кофеина, но, как и большинство сотрудников Giggle, вы кофейный сноб: пьёте только органический эспрессо одного происхождения, приготовленный вручную из зёрен независимой обжарки, выращенных в тени и закупленных напрямую, без молока и сахара, спасибо. После каждой порции эспрессо вы можете проехать до L миль, прежде чем у вас начнётся мигрень от недостатка кофеина.

Giggle услужливо предоставляет карту США в виде неориентированного графа G: вершины представляют кофейни, продающие подходящий эспрессо, а рёбра — соединяющие их шоссе. Каждое ребро e помечено длиной $\ell(e)$ соответствующего участка. Разумеется, подходящие кофейни есть в обоих офисах Giggle, представленных вершинами s и t графа G.

(а) Опишите и проанализируйте алгоритм, определяющий, можно ли доехать от Гигглплекса до Гигглсбурга без мигрени от недостатка кофеина.

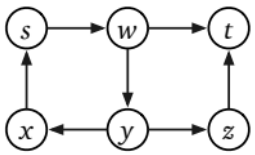
(б) Вы обнаруживаете, что более дорогая шляпа-федора позволяет увеличить расстояние L, которое вы проезжаете между порциями эспрессо. Опишите и проанализируйте алгоритм поиска наименьшего L, позволяющего проехать от Гигглплекса до Гигглсбурга без мигрени.

(в) Когда вы сообщаете руководительнице, недавно переманенной Giggle у конкурента Ünter, что поездка невозможна, она требует показать карту. «А, понятно, в чём проблема: на карте нет Starbucks!» Пока вы с ужасом смотрите, она вручает вам обновлённый граф G', содержащий вершину для каждого Starbucks в США, услужливо помеченную фирменным зелёным цветом Starbucks (Pantone® 3425 C).

Опишите и проанализируйте алгоритм поиска минимального числа заведений Starbucks, которые придётся посетить на пути от Гигглплекса до Гигглсбурга без мигрени. Формально алгоритм должен найти минимальное число зелёных вершин на любом пути в G' из s в t, использующем только рёбра длиной не более L.

16. Пусть задан ориентированный граф $G = (V, E)$ с неотрицательными весами рёбер и две вершины s, t. Опишите и проанализируйте алгоритм поиска кратчайшего маршрута в G из s в t, возможно с повторением вершин и/или рёбер, число рёбер которого делится на 3.

Например, для показанного ниже графа с указанными вершинами s, t, где все веса равны 1, алгоритм должен вернуть 6 — длину маршрута $s \to w \to y \to x \to s \to w \to t$.

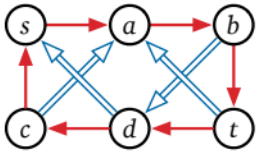


17. Suppose you are given a directed graph G with non-negatively weighted

17. Пусть задан ориентированный граф G с неотрицательными весами рёбер, причём некоторые рёбра красные, а остальные синие. Опишите алгоритм поиска кратчайшего маршрута в G из вершины s в вершину t, в котором никакие три последовательных ребра не имеют одного цвета. То есть после двух последовательных красных рёбер должно идти синее, а после двух последовательных синих — красное.

Например, для приведённого графа, где вес каждого красного ребра равен 1, а каждого синего — 2, алгоритм должен выдать 9, поскольку кратчайший допустимый маршрут из s в t — $s \to a \to b \Rightarrow d \to c \Rightarrow a \to b \to c$.

$s \to a \to b \Rightarrow d \to c \Rightarrow a \to b \to c.$



18. Рассмотрим ориентированный граф G, в котором каждое ребро имеет неотрицательный вес и окрашено в красный, белый или синий цвет. Маршрут называется *маршрутом французского флага*, если цвета его рёбер идут в последовательности красный, белый, синий, красный, белый, синий и так далее. Формально маршрут $v_0 \to v_1 \to \cdots \to v_k$ — маршрут французского флага, если для каждого целого i ребро $v_i \to v_{i+1}$ красное при $i \bmod 3 = 0$, белое при $i \bmod 3 = 1$ и синее при $i \bmod 3 = 2$.

Опишите алгоритм поиска кратчайших маршрутов французского флага из исходной вершины s во все остальные вершины G.

19. Существуют n галактик, соединённых m межгалактическими телепортными путями. Каждый путь соединяет две галактики и может использоваться в обоих направлениях. Каждому пути e соответствует стоимость $c(e)$ долларов, где $c(e)$ — положительное целое число. Путь можно использовать несколько раз, но за каждое использование нужно платить.

Джуди хочет добраться из галактики s в галактику t как можно дешевле. При этом она хочет, чтобы общая стоимость была кратна пяти долларам, потому что носить мелочь тоже неприятно.

(а) Опишите и проанализируйте алгоритм вычисления минимальной стоимости путешествия из s в t при условии, что общая стоимость кратна пяти долларам.

(б) Решите пункт (а), предположив теперь, что у Джуди есть купон, позволяющий воспользоваться ровно одним телепортным путём бесплатно.

20. Переехав в новый город, вы решили выбрать пеший маршрут от дома до нового офиса. Чтобы получать хорошую ежедневную нагрузку, маршрут должен состоять из подъёма в гору, за которым следует спуск, или только из подъёма, или только из спуска. (Домой вы пойдёте по тому же пути, так что нагрузка будет в любом случае.) Но среди таких маршрутов вам нужен кратчайший, чтобы вовремя приходить на работу.

На вход подаётся неориентированный граф G, вершины которого обозначают перекрёстки, а рёбра — участки дорог, а также начальная вершина s и целевая вершина t. Каждой вершине v соответствует высота $h(v)$ над уровнем моря, а каждому ребру uv — длина участка $\ell(uv)$.

(а) Опишите и проанализируйте алгоритм поиска кратчайшего маршрута из s в t, состоящего из подъёма и спуска. Считайте высоты всех вершин различными.

(б) Теперь допустим, что некоторые или все высоты вершин равны. Опишите и проанализируйте алгоритм поиска кратчайшего маршрута «сначала вверх, затем вниз» из s в t; горизонтальные рёбра можно использовать как на участке подъёма, так и на участке спуска.

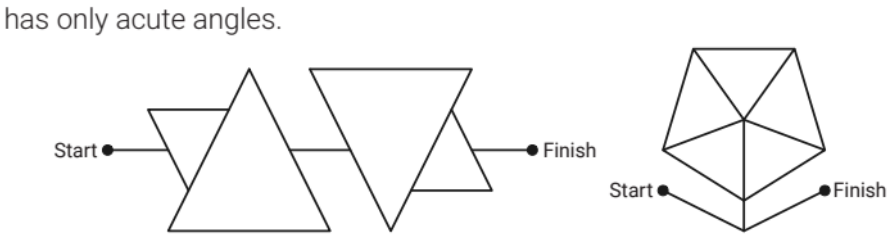
(в) Наконец, предположим, что пути желаемой структуры из s в t нет. Опишите алгоритм поиска пути из s в t, в котором подъёмы и спуски чередуются как можно меньше раз, а среди таких путей длина минимальна.

21. Окончив университет Шам-Пубананы, вы поступаете на работу в Aerophobes-R-Us — ведущее туристическое агентство для тех, кто ненавидит летать. Вам поручено построить систему помощи клиентам в планировании перелётов между городами. Все клиенты боятся летать и, соответственно, аэропортов, поэтому любая поездка должна быть как можно короче. Вы знаете времена отправления и прибытия всех рейсов планеты.

Предположим, клиент хочет перелететь из города X в город Y. Опишите алгоритм поиска последовательности рейсов, минимизирующей общее время в пути — промежуток от первого отправления до окончательного прибытия, включая ожидание пересадок в промежуточных аэропортах.

22. В упражнении 20 главы 5 вы разработали алгоритм, определяющий, можно ли пройти заданный лабиринт острых углов. Теперь нужно разработать алгоритмы поиска кратчайшего маршрута через такой лабиринт для двух разных определений «длины».

Пройдите каждый из лабиринтов ниже от начала до конца, выполняя только острые повороты.



Вход — связный неориентированный граф G, вершины которого являются точками плоскости, а рёбра — отрезками. Рёбра не пересекаются, кроме как в концах. Например, рисунок буквы X имеет пять вершин и четыре ребра; первый лабиринт выше содержит 14 вершин и 21 ребро. Также заданы вершины `Start` и `Finish`.

Маршрут от `Start` до `Finish` допустим, если он содержит только острые углы, или, формальнее, для любых двух последовательных рёбер $u \to v \to w$ выполняется $\angle uvw = \pi$ либо $0 < \angle uvw < \pi/2$. Считайте, что можно за $O(1)$ определить, является ли угол между двумя отрезками развёрнутым, тупым, прямым или острым.

(а) Опишите алгоритм поиска допустимого маршрута от `Start` до `Finish`, проходящего минимальное число отрезков. (Повторное прохождение отрезка считается повторно.)

(б) Опишите алгоритм поиска допустимого маршрута от `Start` до `Finish` с минимальным числом поворотов. [Подсказка: это не то же самое, что пункт (а).]

(в) Опишите алгоритм поиска допустимого маршрута от `Start` до `Finish` минимальной суммарной евклидовой длины. (Считайте, что длину любого отрезка тоже можно вычислить за $O(1)$.)

23. После изнурительной контрольной в Центре выявления фальшивых новостей «Си-Булл» вы решили ехать домой на автобусе. Поскольку вы предусмотрительны, у вас есть расписание со временем и местоположением каждой остановки каждого автобуса Шам-Пубананы. К сожалению, ни один автобус не посещает и Центр, и ваш дом, поэтому нужна хотя бы одна пересадка. Всего имеется ровно b автобусов. Каждый начинает движение в 00:00:01, делает ровно n остановок и заканчивает работу в 23:59:59. Автобусы всегда ходят точно по расписанию, а ваши часы точны. Наконец, вы слишком устали, чтобы ходить между остановками пешком.

(а) Опишите и проанализируйте алгоритм поиска последовательности автобусных поездок, с которой вы доберётесь домой как можно раньше. Нужно минимизировать время прибытия, а не время, проведённое в дороге.

(б) О нет! Контрольная проходила на Хэллоуин, и улицы кишат зомби! У транспортного управления Шам-Пубананы нет средств на дополнительные автобусы или защищённые от зомби остановки, особенно ради одной ночи в году. Опишите и проанализируйте алгоритм поиска последовательности поездок, минимизирующей общее время ожидания на остановках; вас не волнуют позднее прибытие домой и время, проведённое в автобусах. (Предположите, что до самого отправления первого автобуса можно ждать внутри Центра «Си-Булл».)

24. Первым утром после возвращения с замечательных весенних каникул Алиса обнаруживает, что её машина не заводится, поэтому ей придётся добираться на занятия в университет Шам-Пубананы общественным транспортом. У неё есть полное расписание транспорта округа Пубанана. Автобусные маршруты представлены ориентированным графом G, вершины которого обозначают остановки, а рёбра — маршруты между ними. Для каждого ребра $u \to v$ в расписании указаны три положительных вещественных числа:

- $\ell(u \to v)$ — длительность поездки от остановки u до остановки v в минутах;
- $f(u \to v)$ — время отправления первого автобуса из u в v в минутах после полуночи;
- $\Delta(u \to v)$ — промежуток между последовательными отправлениями из u в v в минутах.

Таким образом, первый автобус отправляется из u в момент $f(u \to v)$ и прибывает в v в момент $f(u \to v) + \ell(u \to v)$; второй отправляется в момент $f(u \to v) + \Delta(u \to v)$ и прибывает в момент $f(u \to v) + \Delta(u \to v) + \ell(u \to v)$; третий отправляется в момент $f(u \to v) + 2\Delta(u \to v)$ и прибывает в момент $f(u \to v) + 2\Delta(u \to v) + \ell(u \to v)$ и так далее.

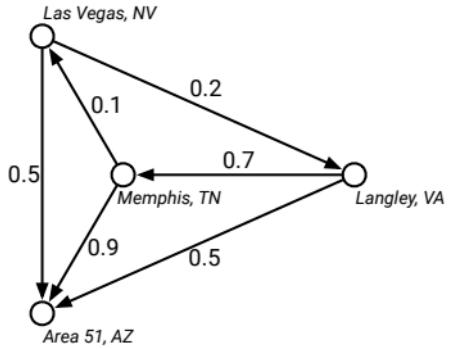
Алиса хочет выйти из остановки s у дома в определённое время и как можно скорее добраться до остановки t у Центра «Си-Булл». Если Алиса прибывает на остановку на одном автобусе ровно в момент отправления другого, она успевает на второй. Будучи студенткой университета, Алиса ездит бесплатно, поэтому число пересадок её не волнует.

Опишите и проанализируйте алгоритм поиска самого раннего времени прибытия Алисы. На вход подаются ориентированный граф $G = (V, E)$, вершины s, t, значения $\ell(e), f(e), \Delta(e)$ каждого ребра $e \in E$ и время выхода Алисы в минутах после полуночи.

[Подсказка: в этом редком случае может быть проще изменить алгоритм, а не входной граф.]

25. Малдер и Скалли вычислили для каждой дороги США точную вероятность того, что человек, едущий по этой дороге, не будет похищен инопланетянами. Агенту Малдеру нужно доехать из Лэнгли, штат Виргиния, до Зоны 51, штат Невада. Какой путь ему выбрать, чтобы вероятность похищения была наименьшей?

Формальнее: задан ориентированный граф $G = (V, E)$, каждое ребро e которого имеет независимую вероятность безопасности $p(e)$. Безопасность пути — произведение вероятностей безопасности его рёбер. Разработайте и проанализируйте алгоритм поиска самого безопасного пути из заданной вершины s в заданную вершину t. Можно считать, что все необходимые арифметические операции выполняются за $O(1)$.



For example, with the probabilities shown above, if Mulder tries to drive

Например, при указанных выше вероятностях, если Малдер поедет напрямую из Лэнгли в Зону 51, вероятность добраться без похищения составит 50%. Если он заедет в Мемфис, вероятность безопасного прибытия составит $0{,}7 \times 0{,}9 = 63\%$. А если он заедет сначала в Мемфис, а затем в Лас-Вегас, вероятность похищения составит $1 - 0{,}7 \times 0{,}1 \times 0{,}5 = 96{,}5\%$! (Так, знаете ли, они добрались до Элвиса.)

♣ **26.** Во время ночной стоянки в национальном парке Саннидейл вас будит крик. Пока вы выбираетесь из палатки, чтобы выяснить причину, из леса выбегает перепуганный смотритель парка, весь в крови и прижимающий к груди скомканный лист бумаги. Добежав до палатки, он хрипит: «Уходите… пока… ещё…», суёт вам бумагу и падает на землю. Проверив пульс, вы обнаруживаете, что смотритель мёртв.

Вы смотрите на лист и узнаёте карту парка, нарисованную в виде неориентированного графа: вершины обозначают ориентиры в парке, а рёбра — тропы между ними. (Тропы начинаются и заканчиваются у ориентиров и не пересекаются.) В одной из вершин вы узнаёте своё текущее положение; несколько вершин на границе карты обозначены `EXIT` — выход.

Присмотревшись, вы замечаете, что кто-то, возможно покойный смотритель, написал вещественное число от 0 до 1 рядом с каждой вершиной и каждым ребром. Небрежная записка на обратной стороне сообщает, что число рядом с ребром — вероятность встретить вампира на соответствующей тропе, а число рядом с вершиной — вероятность встретить вампира у соответствующего ориентира. (Вампиры не выносят общества друг друга, поэтому на одной тропе или у одного ориентира вы никогда не встретите больше одного вампира.) Записка предупреждает: если сойти с обозначенных троп, вас ждёт медленная и мучительная смерть.

Вы бросаете взгляд на труп у своих ног. Да, его смерть определённо выглядела мучительной. Постойте, он только что пошевелился? Его зубы удлиняются? Вбив колышек от палатки в сердце ожившего смотрителя, вы благоразумно решаете немедленно покинуть парк как можно быстрее.

Опишите и проанализируйте эффективный алгоритм поиска пути от вашего текущего положения до произвольной вершины `EXIT`, минимизирующего ожидаемое общее число вампиров, встреченных на пути. Обязательно учитывайте вероятности и у вершин, и у рёбер. [Подсказка: даже без вероятностей у вершин эта задача отличается от предыдущей!]

Глава 9. Кратчайшие пути между всеми парами вершин

Дерево, которое не обхватить руками, вырастает из крошечного ростка; девятиэтажная башня начинается с горстки земли; путь в тысячу ли начинается с первого шага.

— Лао-цзы, «Дао дэ цзин», глава 64, перевод Джеймса Легга (1891)

Я прошёл бы пятьсот миль, И ещё пятьсот прошёл бы, Чтобы стать тем, кто прошёл тысячу миль И упал у твоей двери.

— The Proclaimers, «I'm Gonna Be (500 Miles)», Sunshine on Leith (2001)

Почти у цели… Почти у цели…

— Красный лидер, «Звёздные войны» (1977)

9.1. Введение

В предыдущей главе мы рассмотрели несколько алгоритмов поиска кратчайших путей от одной исходной вершины s до всех остальных вершин графа, строящих дерево кратчайших путей с корнем s. Для каждой вершины v это дерево задаёт два значения:

- dist(v) — длину кратчайшего пути от s до v;
- pred(v) — предпоследнюю вершину на кратчайшем пути от s до v.

В этой главе мы рассмотрим более общую задачу поиска кратчайших путей между всеми парами вершин: требуется найти путь от каждой возможной исходной вершины к каждой возможной конечной. Для каждой пары вершин u и v мы хотим вычислить:

- dist(u, v) — длину кратчайшего пути от u до v;
- pred(u, v) — предпоследнюю вершину на кратчайшем пути от u до v.

Эти интуитивные определения не учитывают несколько граничных случаев, которые уже встречались нам в предыдущей главе.

- Если пути от u до v нет, то нет и кратчайшего пути; в этом случае определим dist(u, v) = ∞ и pred(u, v) = Null.
- Если между u и v есть отрицательный цикл, существуют маршруты[1] от u до v со сколь угодно большой по модулю отрицательной длиной; определим dist(u, v) = −∞ и pred(u, v) = Null.
- Наконец, если u не лежит на отрицательном цикле, кратчайший путь из u в саму себя не содержит рёбер и, следовательно, последнего ребра; в этом случае dist(u, u) = 0 и pred(u, u) = Null.

Искомый результат задачи — пара массивов V × V: в одном хранятся все V^2 расстояний кратчайших путей[2], в другом — все V^2 предшественников. В этой главе я почти полностью сосредоточусь на вычислении массива расстояний. Массив предшественников, с помощью которого можно восстановить сами кратчайшие пути, вычисляется с небольшими изменениями алгоритмов (намёк, намёк).

[1] Формально — маршруты, допускающие повторение вершин и рёбер.

[2] Когда дорожные карты печатали на бумаге и просматривали вручную, на них нередко помещали треугольную «таблицу расстояний». Чтобы узнать расстояние, например, от Шампейна до Колумбуса, нужно было найти строку «Champaign» и столбец «Columbus».

9.2. Много исходных вершин

Самое очевидное решение — запустить алгоритм поиска кратчайших путей от одного источника V раз, по одному разу для каждой возможной исходной вершины. А именно, чтобы заполнить одномерный подмассив dist[s, ·], запускаем алгоритм с исходной вершиной s.

```
ObviousAPSP(V, E, w):
    для каждой вершины s
        dist[s, ·] ← SSSP(V, E, w, s)
```

Время работы этого алгоритма, очевидно, зависит от выбора алгоритма для одного источника. Как и в задаче с одним источником, имеются четыре естественных варианта в зависимости от структуры графа и весов его рёбер:

- Если рёбра графа невзвешенные, поиск в ширину даёт общее время $O(VE) = O(V^3)$.
- Если граф ацикличен, обход вершин в топологическом порядке также даёт $O(VE) = O(V^3)$.
- Если все веса рёбер неотрицательны, алгоритм Дейкстры даёт $O(VE \log V) = O(V^3 \log V)$.[3]
- Наконец, в самом общем случае алгоритм Беллмана — Форда даёт $O(V^2E) = O(V^4)$.

[3] Напомню: если в нашей реализации алгоритма Дейкстры заменить двоичную кучу неотсортированным массивом, общее время составит $O(V^3)$, независимо от числа рёбер графа. Если же заменить её фибоначчиевой кучей, получим $O(V(E + V \log V)) = O(VE + V^2 \log V) = O(V^3)$.

9.3. Изменение весов

Отрицательные рёбра значительно замедляют работу; можно ли от них избавиться? Многим приходит в голову простая идея: увеличить веса всех рёбер на одну и ту же величину, сделав все веса положительными, чтобы вместо алгоритма Беллмана — Форда использовать алгоритм Дейкстры. К сожалению, эта идея не работает. Интуитивно причина в том, что два естественных понятия «длины» несовместимы: путь с большим числом рёбер может иметь меньший суммарный вес, чем путь с меньшим числом рёбер. Если увеличивать веса всех рёбер одинаково, более длинные по числу рёбер пути растут быстрее; поэтому кратчайший путь между двумя вершинами может измениться.

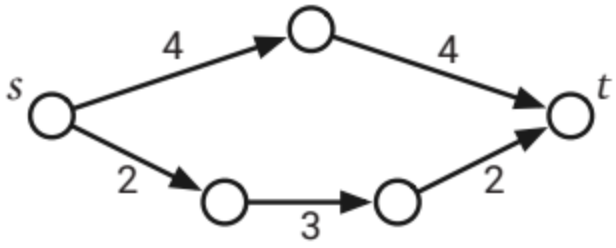


Рис. 9.1. Увеличение весов всех рёбер на 2 изменяет кратчайший путь от s до t.

Однако существует более тонкий способ изменения весов, который сохраняет кратчайшие пути. Этот способ часто приписывают Дональду Джонсону, описавшему его применение к алгоритмам кратчайших путей в 1973 году. Но сам Джонсон сослался на статью Джека Эдмондса и Ричарда Карпа 1972 года. Тот же метод описал Нобуаки Томидзава в 1971 году, а в несколько иной форме — Делберт Фалкерсон в 1961 году.

Предположим, с каждой вершиной v связана некоторая цена π(v), которая может быть положительной, отрицательной или нулевой. Определим новую функцию весов w′:

$$w\prime(u \to v) = \pi(u) + w(u \to v) - \pi(v).$$

Для наглядности представим, что при выходе из вершины u мы платим налог на выход π(u), а при входе в v получаем подарок π(v).

Нетрудно показать, что кратчайшие пути при новой функции весов w′ совпадают с кратчайшими путями при исходной функции w. Действительно, для любого пути u ⇝ v между вершинами u и v имеем

$$w\prime(u \rightsquigarrow v) = \pi(u) + w(u \rightsquigarrow v) - \pi(v).$$

Мы платим налог на выход π(u), прибавляем исходный вес пути и вычитаем подарок за вход π(v). В каждой промежуточной вершине x мы получаем подарок π(x), но сразу возвращаем его как налог на выход! Поскольку длина всех путей из u в v меняется на одну и ту же величину, кратчайший путь между ними не меняется. (Длины путей между разными парами вершин могут меняться на разные величины, так что их порядок может измениться.)

9.4. Алгоритм Джонсона

Алгоритм Джонсона вычисляет цену π(v) для каждой вершины так, чтобы новый вес каждого ребра был неотрицательным, а затем находит кратчайшие пути по новым весам алгоритмом Дейкстры.

Сначала предположим, что во входном графе есть вершина s, из которой достижимы все остальные вершины. Алгоритм Джонсона находит кратчайшие пути из s к остальным вершинам алгоритмом Беллмана — Форда, которому не мешают отрицательные веса, а затем изменяет веса графа, используя функцию цен π(v) = dist(s, v). Новый вес каждого ребра равен

$$w\prime(u \to v) = dist(s, u) + w(u \to v) - dist(s, v).$$

Эти новые веса неотрицательны, поскольку алгоритм Беллмана — Форда завершился! Напомню, ребро u → v называется напряжённым, если dist(s, u) + w(u → v) < dist(s, v), а алгоритмы кратчайших путей от одного источника устраняют все напряжённые рёбра. Если алгоритм Беллмана — Форда обнаруживает отрицательный цикл, алгоритм Джонсона прекращает работу, поскольку кратчайшие пути не определены.

Если подходящей вершины s, из которой всё достижимо, нет, то независимо от начальной вершины Беллмана — Форда некоторые полученные цены будут бесконечными. Чтобы избежать этой проблемы, всегда добавляем в граф новую вершину s и рёбра нулевого веса из s ко всем остальным вершинам, но не добавляем рёбер, ведущих обратно в s. Это не меняет кратчайшие пути между исходными вершинами, поскольку путей в s нет.

Полный псевдокод алгоритма Джонсона приведён на рисунке 9.2. Время работы определяется вызовами алгоритма Дейкстры. Точнее, мы тратим O(VE) на один запуск `BellmanFord`, O(VE log V) на V запусков `Dijkstra` и O(V + E) на прочие операции. Поэтому общее время работы равно O(VE log V) = O(V³ log V).[4] Отрицательные рёбра всё-таки нас не замедляют!

[4] При стандартной реализации с двоичной кучей; см. предыдущее примечание.

```
JohnsonAPSP(V, E, w):
    ⟨⟨Добавляем искусственный источник⟩⟩
    добавить новую вершину s
    для каждой вершины v
        добавить новое ребро s → v
        w(s → v) ← 0
    ⟨⟨Вычисляем цены вершин⟩⟩
    dist[s, ·] ← BellmanFord(V, E, w, s)
    если BellmanFord обнаружил отрицательный цикл
        корректно завершить работу с ошибкой
    ⟨⟨Меняем веса рёбер⟩⟩
    для каждого ребра u → v ∈ E
        w'(u → v) ← dist[s, u] + w(u → v) − dist[s, v]
    ⟨⟨Находим расстояния кратчайших путей с новыми весами⟩⟩
    для каждой вершины u
        dist'[u, ·] ← Dijkstra(V, E, w', u)
    ⟨⟨Вычисляем исходные расстояния кратчайших путей⟩⟩
    для каждой вершины u
        для каждой вершины v
            dist[u, v] ← dist'[u, v] − dist[s, u] + dist[s, v]
```

Рис. 9.2. Алгоритм Джонсона для кратчайших путей между всеми парами вершин.

9.5. Динамическое программирование

Задачу кратчайших путей между всеми парами можно решить непосредственно динамическим программированием, не вызывая алгоритм для одного источника. Для плотных графов, где E = Ω(V²), такой подход в конечном счёте приводит к алгоритму, который и проще, и немного быстрее алгоритма Джонсона. В оставшейся части главы будем считать, что входной граф не содержит отрицательных циклов.

Как обычно в динамическом программировании, сначала нам нужна рекуррентная формула. Как и для одного источника, «очевидное» рекурсивное определение

$$dist(u, v) = \begin{cases} 0, & u = v, \\ \min_{x \to v}\{dist(u, x) + w(x \to v)\}, & \text{иначе} \end{cases}$$

работает только тогда, когда входной граф ориентирован и ацикличен: любой ориентированный цикл вызывает бесконечную рекурсию.

Этот бесконечный цикл можно разорвать, введя дополнительный параметр, как мы делали в алгоритме Беллмана — Форда. Пусть dist(u, v, ℓ) обозначает длину кратчайшего пути из u в v, содержащего не более ℓ рёбер. Кратчайший путь между любыми двумя вершинами проходит не более V − 1 рёбер, так что настоящее расстояние равно dist(u, v, V − 1). Рекуррентная формула Беллмана для одного источника сразу обобщается:

$$dist(u, v, \ell) = \begin{cases} 0, & \ell = 0,\ u = v, \\ \infty, & \ell = 0,\ u \neq v, \\ \min\{dist(u, v, \ell - 1), \min_{x \to v}(dist(u, x, \ell - 1) + w(x \to v))\}, & \text{иначе.} \end{cases}$$

Преобразовать эту формулу в алгоритм динамического программирования несложно; его время работы равно O(V²E) = O(V⁴).

```
ShimbelAPSP(V, E, w):
    для всех вершин u
        для всех вершин v
            если u = v
                dist[u, v, 0] ← 0
            иначе
                dist[u, v, 0] ← ∞
    для ℓ ← 1 до V − 1
        для всех вершин u
            для всех вершин v ≠ u
                dist[u, v, ℓ] ← dist[u, v, ℓ − 1]
                для всех рёбер x → v
                    если dist[u, v, ℓ] > dist[u, x, ℓ − 1] + w(x → v)
                        dist[u, v, ℓ] ← dist[u, x, ℓ − 1] + w(x → v)
```

Этот алгоритм впервые набросал Альфонсо Шимбел в 1954 году.[5] Как и в формулировке Беллмана для алгоритма Беллмана — Форда, внутренний цикл по вершинам v и индекс итерации ℓ нам не нужны. Изменённый алгоритм приведён ниже.

```
AllPairsBellmanFord(V, E, w):
    для всех вершин u
        для всех вершин v
            если u = v
                dist[u, v] ← 0
            иначе
                dist[u, v] ← ∞
    для ℓ ← 1 до V − 1
        для всех вершин u
            для всех рёбер x → v
                если dist[u, v] > dist[u, x] + w(x → v)
                    dist[u, v] ← dist[u, x] + w(x → v)
```

Учитывая, как мы его получили, неудивительно, что этот алгоритм в точности совпадает с чередованием V выполнений алгоритма Беллмана — Форда, каждого со своей исходной вершиной. В частности, для любых u и v после ℓ-й итерации основного цикла dist[u, v] не превосходит длины кратчайшего пути из u в v, содержащего не более ℓ рёбер.

[5] Шимбел предполагал, что вход задан полной матрицей расстояний V × V, поэтому его первоначальный алгоритм работает за O(V⁴) независимо от числа рёбер графа.

9.6. Разделяй и властвуй

Но можно добиться и более существенного улучшения; его предложили Майкл Фишер и Альберт Мейер в 1971 году. Рекуррентная формула Беллмана разбивает кратчайший путь на немного более короткий путь и одно ребро, перебирая всех возможных предшественников конечной вершины. Вместо этого разобьём кратчайший путь в средней вершине на два более коротких кратчайших пути. Эта идея даёт другую рекуррентную формулу для той же функции dist(u, v, ℓ). Здесь базовым случаем должно быть ℓ = 1, а не ℓ = 0: у пути не более чем с одним ребром нет «средней» вершины. Чтобы немного упростить формулу, положим w(v → v) = 0 для каждой вершины v.

$$dist(u, v, \ell) = \begin{cases} w(u \to v), & \ell = 1, \\ \min_x \{dist(u, x, \ell/2) + dist(x, v, \ell/2)\}, & \text{иначе.} \end{cases}$$

В таком виде формула работает, только когда ℓ — степень двойки: в противном случае мы могли бы попытаться найти кратчайший путь, содержащий не более дробного числа рёбер! Но это не проблема: dist(u, v, ℓ) равно настоящему кратчайшему расстоянию от u до v при любом ℓ ≥ V − 1; в частности, можно взять ℓ = 2^⌈lg V⌉ < 2V.

И снова решение с динамическим программированием получается непосредственно. Ещё до записи алгоритма видно, что его время работы равно O(V³ log V): нужно рассмотреть V возможных значений каждого из u, v и x, но лишь ⌈lg V⌉ возможных значений ℓ. В следующем псевдокоде алгоритма Фишера — Мейера элемент dist[u, v, i] хранит значение dist(u, v, 2ⁱ).

```
FischerMeyerAPSP(V, E, w):
    для всех вершин u
        для всех вершин v
            dist[u, v, 0] ← w(u → v)
    для i ← 1 до ⌈lg V⌉     ⟨⟨ℓ = 2^i⟩⟩
        для всех вершин u
            для всех вершин v
                dist[u, v, i] ← ∞
                для всех вершин x
                    если dist[u, v, i] > dist[u, x, i − 1] + dist[x, v, i − 1]
                        dist[u, v, i] ← dist[u, x, i − 1] + dist[x, v, i − 1]
```

В отличие от предыдущих алгоритмов, `FischerMeyerAPSP` не совпадает с V вызовами какого-либо алгоритма поиска кратчайших путей от одного источника; в частности, его самый внутренний цикл не просто ослабляет напряжённые рёбра. Тем не менее последнее измерение таблицы можно убрать, всюду используя dist[u, v] вместо dist[u, v, i], как в алгоритме Беллмана — Форда и нашем предыдущем алгоритме динамического программирования. Это сокращает расход

памяти с $O(V^3)$ до $O(V^2)$. Этот более изящный алгоритм описали Лейзорек и соавторы в 1957 году, в той же статье, где описан алгоритм Дейкстры.

```
LeyzorekAPSP(V, E, w):
    для всех вершин u
        для всех вершин v
            dist[u, v] ← w(u → v)
    для i ← 1 до ⌈lg V⌉     ⟨⟨ℓ = 2^i⟩⟩
        для всех вершин u
            для всех вершин v
                для всех вершин x
                    если dist[u, v] > dist[u, x] + dist[x, v]
                        dist[u, v] ← dist[u, x] + dist[x, v]
```

9.7. Занимательное умножение матриц

Между вычислением кратчайших путей в ориентированном графе и возведением квадратной матрицы в степень существует очень тесная связь; первым её заметил Шимбел, а позднее независимо от него — Беллман. Сравните следующий алгоритм возведения матрицы A размера $n \times n$ в квадрат с внутренним циклом `FischerMeyerAPSP`. Я немного изменил обозначения во втором алгоритме, чтобы сходство было заметнее.

```
MatrixSquare(A):
    для i ← 1 до n
        для j ← 1 до n
            A'[i, j] ← 0
            для k ← 1 до n
                A'[i, j] ← A'[i, j] + A[i, k] · A[k, j]

FischerMeyerInnerLoop(D):
    для всех вершин u
        для всех вершин v
            D'[u, v] ← ∞
            для всех вершин x
                D'[u, v] ← min{D'[u, v], D[u, x] + D[x, v]}
```

Единственное различие между этими алгоритмами в том, что второй использует сложение вместо умножения и минимум вместо сложения. Поэтому внутренний цикл поиска кратчайших путей иногда называют «min-plus», «дистанционным» или «занимательным» умножением матриц.

Наш более медленный алгоритм `ShimbelAPSP` — стандартный итерационный алгоритм вычисления $(V - 1)$-й «min-plus-степени» матрицы весов w. Первые циклы создают единичную min-plus-матрицу: нули на главной диагонали и ∞ во всех остальных ячейках; каждая итерация второго основного цикла вычисляет следующую «min-plus-степень». `FischerMeyerAPSP` заменяет этот итерационный способ возведения в степень последовательным возведением в квадрат, как мы уже видели в конце главы 1. И снова мы видим влияние древнеегипетских ἀρπεδονάπται!

Существуют более быстрые алгоритмы обычного умножения матриц методом «разделяй и властвуй», похожие на алгоритм Карацубы для умножения целых чисел. Первый такой алгоритм, описанный Фолькером Штрассеном в 1969 году, сводит умножение двух матриц $n \times n$

к семи умножениям матриц n/2 × n/2; алгоритм Штрассена работает за O(n^lg 7) = O(n^2,807355). За последние пятьдесят лет его много раз улучшали; по состоянию на 2018 год самый быстрый известный алгоритм умножения матриц работает за O(n^2,372864).[6] К сожалению, все эти быстрые алгоритмы используют вычитание, а у него нет «занимательного» аналога. Какая операция обратна минимуму? Поэтому, по крайней мере для общих графов, очевидного способа ускорить внутренний цикл наших алгоритмов динамического программирования нет.

Но «не очевиден» не значит «невозможен»! Для частных случаев задачи кратчайших путей между всеми парами действительно существует несколько значительно более быстрых алгоритмов. Один из самых красивых — простой рандомизированный алгоритм, открытый Цви Галилом и Одедом Маргалитом в 1991 году и упрощённый Раймундом Зейделем в 1992 году. Он вычисляет расстояния кратчайших путей между всеми парами в невзвешенных неориентированных графах за ожидаемое время O(M(V) log V), где M(n) = O(n^2,372864) — время, необходимое для настоящего умножения двух целочисленных матриц n × n.[7] Позднее подход Галила, Маргалита и Зейделя расширили: он позволяет детерминированно находить сами кратчайшие пути в ориентированных графах с небольшими целочисленными весами рёбер за время строго меньше кубического.

С другой стороны, несмотря на существенный прогресс для небольших целых весов, никто не знает, как вычислить кратчайшие пути между всеми парами при более общих весах за O(V^2,999999), сколько бы девяток мы ни написали. Более того, есть основания предполагать, что такого алгоритма не существует! Так что, возможно, «не очевиден» всё-таки означает «невозможен».

[6] Определение минимального времени умножения двух произвольных матриц n × n — давняя открытая задача. Многие полагают, что существует ещё не открытый алгоритм с временем O(n^(2+ε)) для любого ε > 0 или даже O(n²).

[7] Raimund Seidel. On the all-pairs-shortest-path problem in unweighted undirected graphs. Journal of Computer and System Sciences, 51(3):400–403, 1995. Это одна из немногих работ об алгоритмах, в которой алгоритм полностью описан и проанализирован прямо в аннотации — по крайней мере, в конференционной версии 1992 года. См. также: Noga Alon, Zvi Galil, Oded Margalit. On the exponent of the all pairs shortest path problem. Journal of Computer and System Sciences 54(2):255–262, 1997.

9.8. Алгоритм (Клини — Роя —) Флойда — Уоршелла (— Ингермана)

Наш быстрый алгоритм динамического программирования всё ещё в O(log V) раз медленнее в худшем случае, чем стандартная реализация алгоритма Джонсона. Другое описание кратчайших путей, устраняющее этот логарифмический множитель, в 1962 году дважды предложили Роберт Флойд и затем независимо Питер Ингерман. Оба слегка обобщили алгоритм Стивена Уоршелла, опубликованный ранее в том же году. На самом деле алгоритм Уоршелла ранее открыл Бернар Рой в 1959 году, а лежащую в его основе схему рекурсии использовал Стивен Клини[8] ещё в 1951 году.

Идея Уоршелла, а также Роя и Клини, состояла в использовании другого третьего параметра в рекуррентной формуле динамического программирования. Вместо путей с ограниченным числом рёбер они рассматривали пути, которые могут проходить только через определённые вершины. «Проходить через» означает и войти, и выйти: например, путь w → x → y → z начинается в w, проходит через x и y и заканчивается в z.

Произвольно пронумеруем вершины от 1 до V. Для каждой пары вершин u, v и каждого целого r определим путь π(u, v, r):

> *π(u, v, r) — кратчайший путь, если он существует, из u в v, проходящий только через вершины с номерами не больше r.*

В частности, π(u, v, V) — настоящий кратчайший путь из u в v. Клини, Рой и Уоршелл заметили, что эти пути имеют простую рекурсивную структуру.

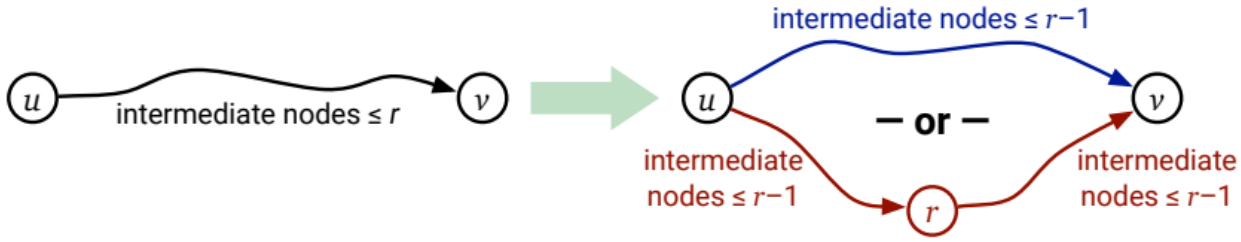


Рис. 9.3. Рекурсивная структура ограниченного кратчайшего пути π(u, v, r). Надписи на схеме: промежуточные вершины ≤ r; или; промежуточные вершины ≤ r − 1.

- Путь π(u, v, 0) не может проходить через промежуточные вершины, поэтому должен быть ребром из u в v, если оно есть.
- Для любого целого r > 0 путь π(u, v, r) либо проходит через вершину r, либо нет.
  - Если он проходит через r, то состоит из подпути от u до r и подпути от r до v. Оба подпути проходят только через вершины с номерами не больше r − 1; кроме того, они кратчайшие при этом ограничении. Поэтому это должны быть пути π(u, r, r − 1) и π(r, v, r − 1).
  - Если же π(u, v, r) не проходит через r, он проходит только через вершины с номерами не больше r − 1 и должен быть кратчайшим при таком ограничении. В этом случае π(u, v, r) = π(u, v, r − 1).

[8] Фамилия Kleene произносится примерно как «клей-ни», а не «клин», «клини», «клей-на» или «Димаджио». Клини описал индуктивное доказательство того, что каждому конечному автомату соответствует регулярное выражение; структура его индукции по существу совпадает с рекуррентной формулой Флойда — Уоршелла.

Пусть теперь dist(u, v, r) — длина пути π(u, v, r). Его рекурсивная структура немедленно даёт следующую формулу:

$$dist(u, v, r) = \begin{cases} w(u \to v), & r = 0, \\ \min\{dist(u, v, r-1),\ dist(u, r, r-1) + dist(r, v, r-1)\}, & \text{иначе.} \end{cases}$$

Наша цель — вычислить dist(u, v, V) для всех вершин u и v. Эту формулу снова можно непосредственно вычислить алгоритмом динамического программирования за $O(V^3)$.

```
KleeneAPSP(V, E, w):
    для всех вершин u
        для всех вершин v
            dist[u, v, 0] ← w(u → v)
    для r ← 1 до V
        для всех вершин u
            для всех вершин v
                если dist[u, v, r − 1] < dist[u, r, r − 1] + dist[r, v, r − 1]
                    dist[u, v, r] ← dist[u, v, r − 1]
                иначе
                    dist[u, v, r] ← dist[u, r, r − 1] + dist[r, v, r − 1]
```

Как и во всех предыдущих алгоритмах динамического программирования для кратчайших путей, можно упростить `KleeneAPSP`, убрав третье измерение таблицы мемоизации. Кроме того, поскольку нумерацию вершин мы выбрали произвольно, в псевдокоде незачем явно к ней обращаться. Наконец мы получаем улучшение алгоритма Уоршелла, предложенное Флойдом:

```
FloydWarshall(V, E, w):
    для всех вершин u
        для всех вершин v
            dist[u, v] ← w(u → v)
    для всех вершин r
        для всех вершин u
            для всех вершин v
                если dist[u, v] > dist[u, r] + dist[r, v]
                    dist[u, v] ← dist[u, r] + dist[r, v]
```

Интересно сравнить `FloydWarshall` с нашим предыдущим, несколько более медленным алгоритмом `LeyzorekAPSP`. Вместо $O(\log V)$ проходов по всем тройкам вершин `FloydWarshall` требует только одного прохода — но лишь потому, что использует другой порядок вложения трёх циклов!

Упражнения

1. (a) Опишите изменение `LeyzorekAPSP`, которое помимо массива расстояний кратчайших путей возвращает массив указателей на предшественников и по-прежнему работает за $O(V^3 \log V)$.

   (b) Опишите изменение `FloydWarshall`, которое помимо массива расстояний кратчайших путей возвращает массив указателей на предшественников и по-прежнему работает за $O(V^3)$.

2. Все рассмотренные в главе алгоритмы не справляются с графом, содержащим отрицательный цикл. Алгоритм Джонсона обнаруживает его при инициализации с помощью Беллмана — Форда и прекращает работу; алгоритмы динамического программирования просто возвращают неверные результаты. Однако все эти алгоритмы

можно изменить так, чтобы они возвращали правильные расстояния кратчайших путей даже при наличии отрицательных циклов. А именно для любых вершин u и v:

- Если v недостижима из u, алгоритм должен вернуть dist[u, v] = ∞.
- Если из u достижим отрицательный цикл, из которого достижима v, алгоритм должен вернуть dist[u, v] = −∞.
- Иначе существует кратчайший путь из u в v, и алгоритм должен вернуть его длину.

(a) Опишите, как изменить алгоритм Джонсона для возврата правильных расстояний даже при отрицательных циклах.

(b) Опишите, как аналогичным образом изменить `LeyzorekAPSP`.

(c) Опишите, как аналогичным образом изменить алгоритм Флойда — Уоршелла.

3. Алгоритмы этой главы можно изменить и так, чтобы они возвращали явное описание некоторого отрицательного цикла входного графа G, если он есть, вместо простого сообщения о его наличии.

   (a) Опишите, как изменить алгоритм Джонсона, чтобы он возвращал либо массив всех расстояний кратчайших путей, либо отрицательный цикл.

   (b) Опишите аналогичное изменение `LeyzorekAPSP`.

   (c) Опишите аналогичное изменение алгоритма Флойда — Уоршелла.

   Во всех случаях, если входной граф содержит более одного отрицательного цикла, алгоритм может выбрать любой из них.

4. Пусть G = (V, E) — ориентированный граф со взвешенными рёбрами; веса могут быть положительными, отрицательными или нулевыми, но отрицательных циклов нет.

   (a) Опишите эффективный алгоритм, который либо находит в G цикл длины ноль, либо правильно сообщает, что такого цикла нет.

   (b) Опишите эффективный алгоритм построения подграфа H графа G со следующими свойствами:

   - Каждая вершина G является вершиной H.
   - Каждый ориентированный цикл в H имеет длину 0.
   - Каждый ориентированный цикл длины 0 в G является также циклом в H.

   В частности, если в G нет нулевых циклов, в H нет рёбер.

5. Пусть G = (V, E) — ориентированный граф со взвешенными рёбрами; веса рёбер могут быть положительными, отрицательными или нулевыми. Предположим, вершины G разбиты на k непересекающихся подмножеств V_1, V_2, …, V_k: каждая вершина G принадлежит ровно одному V_i. Для каждой пары i, j обозначим через δ(i, j) минимальное расстояние кратчайшего пути между вершиной из V_i и вершиной из V_j:

$$\delta(i, j) = \min\{dist(v_i, v_j) \mid v_i \in V_i,\ v_j \in V_j\}.$$

Опишите алгоритм вычисления $\delta(i, j)$ для всех i и j. Для полного зачёта время работы должно составлять $O(VE + kV \log V)$.

6. В этой задаче мы узнаем, как вы, да-да, именно вы, можете устроиться на Уолл-стрит и вызвать крупный экономический кризис! Арбитраж — способ заработать на различиях курсов валют. Допустим, за один доллар США можно купить 120 японских иен, за одну иену — 0,01 евро, а за один евро — 1,2 доллара. Тогда трейдер, начавший с одного доллара и обменявший доллары на иены, иены на евро, а евро обратно на доллары, получит 1,44 доллара! Цикл валют «доллар → иена → евро → доллар» называют арбитражным циклом. Разумеется, для поиска и использования таких циклов до исправления цен нужны чрезвычайно быстрые алгоритмы.

   Допустим, на валютном рынке торгуются n различных валют. Вам дана матрица курсов $Exch[1 .. n, 1 .. n]$: для любых i и j одну единицу валюты i можно обменять на $Exch[i, j]$ единиц валюты j. Не предполагайте, что $Exch[i, j] \cdot Exch[j, i] = 1$.

   (a) Опишите алгоритм, возвращающий массив $MaxAmt[1 .. n]$, где $MaxAmt[i]$ — максимальное количество валюты i, которое можно получить обменами, начав с одной единицы валюты 1, если арбитражных циклов нет.

   (b) Опишите алгоритм, определяющий, создаёт ли заданная матрица курсов арбитражный цикл.

   (c) Измените алгоритм из пункта (b) так, чтобы он действительно возвращал арбитражный цикл, если тот существует.

7. Морти должен достать стабилизированный плюмбус из лабиринта Клакспайр. Он должен войти в лабиринт с помощью межпространственного портального пистолета Рика, пройти к плюмбусу, затем отнести его через лабиринт к флибу для стабилизации и, наконец, вернуть стабилизированный плюмбус к исходному порталу, чтобы попасть домой. Плюмбусы стабилизируются соком флиба, который выделяется сразу после извлечения флиба из его флибоноры. Нестабилизированный плюмбус взорвётся, если отнести его дальше чем на 137 флинков от исходного хранилища. В лабиринте Клакспайр пахнет пуками, поэтому Морти хочет провести там как можно меньше времени.

   Рик дал Морти подробную карту лабиринта: ориентированный граф $G = (V, E)$ с неотрицательными весами рёбер, указывающими расстояния во флинках, и два непересекающихся подмножества $P \subset V$ и $F \subset V$, обозначающие соответственно хранилища плюмбусов и флибоноры. Морти нужно выбрать стартовую вершину s, хранилище $p \in P$ и флибонору $f \in F$ так, чтобы расстояние кратчайшего пути от p до f не превышало 137 флинков, а длина кратчайшего маршрута $s \rightsquigarrow p \rightsquigarrow f \rightsquigarrow s$ была как можно меньше.

   Опишите и проанализируйте алго(рык)ритм решения задачи Морти. Можно считать, что Морти действительно может выполнить задание.

8. Пусть $G = (V, E)$ — ориентированный граф со взвешенными рёбрами; веса могут быть положительными, отрицательными или нулевыми.

(a) Как удалить произвольную вершину v, не изменяя расстояний кратчайших путей между остальными вершинами? Опишите алгоритм, который за O(V²) строит ориентированный граф G′ = (V \ {v}, E′) со взвешенными рёбрами, в котором расстояния между любыми двумя вершинами совпадают с расстояниями между теми же вершинами в G.

(b) Теперь предположим, что все расстояния кратчайших путей в G′ уже вычислены. Опишите алгоритм, который за O(V²) вычисляет расстояния в исходном графе G от v до каждой другой вершины и от каждой другой вершины до v.

(c) Объедините пункты (a) и (b) в ещё один алгоритм поиска кратчайших путей между всеми парами за O(V³). Полученный алгоритм почти совпадает с алгоритмом Флойда — Уоршелла!

9. Пусть A и B — булевы матрицы n × n. Их булево, или and-or, произведение — матрица C размера n × n, определённая формулой

$$C[i,j] := \bigvee_k (A[i,k] \wedge B[k,j]).$$

(a) Сведите умножение булевых матриц к min-plus-умножению. Иными словами, имея подпрограмму `MinPlusMultiply`, которая вычисляет min-plus-произведение двух матриц n × n за T(n), опишите и проанализируйте алгоритм `BooleanMatrixMultiply`, умножающий две булевы матрицы за O(T(n)).

(b) Сведите умножение булевых матриц к обычному умножению матриц. Иными словами, имея подпрограмму `MatrixMultiply`, которая вычисляет обычное произведение двух матриц n × n за T(n), опишите и проанализируйте алгоритм `BooleanMatrixMultiply`, умножающий две булевы матрицы за O(T(n)).

10. Транзитивное замыкание ориентированного графа G содержит ребро u → v тогда и только тогда, когда в G существует ориентированный путь от u до v. В этой задаче предположим, что две булевы матрицы n × n можно перемножить за O(n^ω), где 2 ≤ ω < 3 — некоторая константа. Из задачи 9(b) следует ω ≤ 2,372864.

(a) Опишите алгоритм вычисления транзитивного замыкания ориентированного графа с n вершинами за O(n^ω log n).

(b) Теперь предположим, что G — ориентированный ациклический граф. Опишите алгоритм вычисления его транзитивного замыкания за O(n^ω). Подсказка: сделайте то, что вы всегда делаете с ориентированными ациклическими графами, а затем используйте метод «разделяй и властвуй». Воспользуйтесь тем, что ω ≥ 2.

(c) Наконец, опишите алгоритм вычисления транзитивного замыкания произвольного ориентированного графа за O(n^ω). Подсказка: сделайте то, что вы всегда делаете, чтобы превратить произвольный ориентированный граф в ациклический.

(d) Теперь обратим предыдущее сведение. Имея подпрограмму `TransitiveClosure`, которая вычисляет транзитивное замыкание ориентированного графа с n вершинами за O(n^α), где 2 ≤

$\alpha < 3$ — константа, опишите и проанализируйте алгоритм умножения булевых матриц, работающий за $O(n^{\alpha})$.

11. Докажите, что следующий рекурсивный алгоритм правильно вычисляет расстояния кратчайших путей между всеми парами за $O(n^3)$. Для простоты можно считать, что n — степень двойки. Как обычно, массив D передаётся вспомогательной функции `RecAPSP` по ссылке. Подсказка: это переставленная версия алгоритма Флойда — Уоршелла, работающая с кэшем значительно эффективнее.[9]

```
RecursiveAPSP(V, E, w):
    n ← |V|
    для i ← 1 до n
        для j ← 1 до n
            если i = j
                D[i, j] ← 0
            если i → j ∈ E
                D[i, j] ← w(i → j)
            иначе
                D[i, j] ← ∞
    RecAPSP(D, n, 1, 1, 1)
    вернуть D[1 .. n, 1 .. n]

RecAPSP(D, n, i, j, k):
    если n = 1
        D[i, j] ← min{D[i, j], D[i, k] + D[j, k]}
    иначе
        m ← n/2
        RecAPSP(D, n/2, i,     j,     k)
        RecAPSP(D, n/2, i,     j,     k + m)
        RecAPSP(D, n/2, i,     j + m, k)
        RecAPSP(D, n/2, i,     j + m, k + m)
        RecAPSP(D, n/2, i + m, j,     k)
        RecAPSP(D, n/2, i + m, j,     k + m)
        RecAPSP(D, n/2, i + m, j + m, k)
        RecAPSP(D, n/2, i + m, j + m, k + m)
```

Примечание переводчика. Псевдокод воспроизведён по исходному изданию. В нём, вероятно, есть опечатки: второе условие инициализации перезаписывает диагональ, а в базовом случае для ориентированных графов ожидается D[k, j] вместо D[j, k]. Эти места следует проверить при решении упражнения.

[9] Joon-Sang Park, Michael Penner, Viktor K. Prasanna. Optimizing graph algorithms for improved cache performance. IEEE Transactions on Parallel and Distributed Systems 15(9):769–782, 2004. О существенном обобщении на более широкий класс задач динамического программирования см. Rezaul Alam Chowdhury, Vijaya Ramachandran. Cache-oblivious dynamic programming. Proc. 17th SODA 591–600, 2006.

12. ♥ Пусть $G = (V, E)$ — неориентированный невзвешенный связный граф с n вершинами, представленный матрицей смежности $A[1 .. n, 1 .. n]$. В этой задаче мы выведем субкубический алгоритм Зейделя для вычисления матрицы $D[1 .. n, 1 .. n]$ расстояний кратчайших путей в G с помощью быстрого умножения матриц. Предположим, что

подпрограмма `MatrixMultiply` вычисляет обычное произведение двух матриц n × n за O(n^ω), где ω ≥ 2 — неизвестная константа.

(a) Пусть G^2 — граф с теми же вершинами, что и G, в котором две вершины соединены ребром тогда и только тогда, когда в G они соединены путём длины не больше 2. Опишите алгоритм вычисления матрицы смежности G^2 с одним вызовом `MatrixMultiply` и $O(n^2)$ дополнительного времени.

(b) Предположим, мы выяснили, что G^2 — полный граф. Опишите алгоритм вычисления матрицы D расстояний кратчайших путей в G за $O(n^2)$ дополнительного времени.

(c) Предположим, мы рекурсивно вычислили матрицу D^2 расстояний кратчайших путей в G^2. Докажите, что расстояние кратчайшего пути в G от вершины i до вершины j равно либо $2 \cdot D^2[i, j]$, либо $2 \cdot D^2[i, j] - 1$.

(d) Теперь предположим, что G^2 не является полным графом. Пусть $X = D^2 \cdot A$, а deg(i) обозначает степень вершины i в исходном графе G. Докажите, что расстояние кратчайшего пути от i до j в G равно $2 \cdot D^2[i, j]$ тогда и только тогда, когда $X[i, j] \geq D^2[i, j] \cdot \deg(i)$.

Примечание переводчика. В пункте (d) формула воспроизведена по оригиналу, однако при заданном порядке произведения $X = D^2 \cdot A$ следует использовать степень вершины j: $X[i, j] \geq D^2[i, j] \cdot \deg(j)$. Например, в пути из четырёх вершин при i на конце пути и j в соседней вершине исходное неравенство выполняется, хотя расстояние равно 1, а не 2.

(e) Опишите алгоритм вычисления матрицы D расстояний кратчайших путей в G за $O(n^\omega \log n)$.

13. В 1976 году Гидеон Юваль предложил следующее сведение min-plus-умножения матриц к обычному умножению. Допустим, нам даны две целочисленные матрицы A и B размера n × n, каждый элемент которых находится между 0 и M, и мы хотим вычислить матрицу их min-plus-произведения C:

$$C[i, k] = \min_j (A[i, j] + B[j, k])$$

для всех индексов i и k. Определим две новые матрицы A′ и B′ размера n × n:

$$A'[i, j] = n^{M - A[i,j]}, \qquad B'[i, j] = n^{M - B[i,j]}.$$

Наконец, пусть C′ — обычное произведение A′ и B′, то есть $C'[i, k] = \sum_j A'[i, j] \cdot B'[j, k]$.

(a) Опишите алгоритм построения A′ из A, использующий только обычные целочисленные арифметические операции (+, −, ×).

(b) Опишите алгоритм извлечения min-plus-произведения C из C′, использующий только обычные целочисленные арифметические операции (+, −, ×).[10]

(c) Предположим, обычное произведение двух целочисленных матриц n × n можно вычислить за $O(n^\omega)$ арифметических операций, где $2 \leq \omega < 3$ — константа. Сколько арифметических операций требуется алгоритму Юваля для вычисления min-plus-произведения C?

(d) Сколько арифметических операций нужно алгоритму Ювaля, чтобы вычислить n-ю «занимательную» степень одной целочисленной матрицы A размера n × n? Напомню, что если A — взвешенная матрица смежности графа, то n-я «занимательная» степень A является матрицей расстояний кратчайших путей.

(e) Почему алгоритм Ювaля не даёт алгоритма поиска кратчайших путей между всеми парами с произвольными весами рёбер, более быстрого, чем алгоритм Флойда — Уоршелла? В чём состоит наша хитрость?

[10] В частности, не используйте логарифмы, деление или функцию пола ⌊x⌋. Поверьте, вы не захотите открывать этот ящик Пандоры.

10. Максимальные потоки и минимальные разрезы

> *Процесс невозможно понять, остановив его. Понимание должно двигаться вместе с потоком процесса, соединиться с ним и течь вместе с ним.*
>
> *— Первый закон ментата, из романа Фрэнка Герберта Дюна (1965)*

> *Вопреки ожиданиям, состояние потока обычно возникает не во время спокойного отдыха и развлечений, а когда мы активно заняты трудным делом, задачей, требующей напряжения умственных и физических способностей… Достичь состояния потока без усилий трудно. Состояние потока — не «пустая трата времени».*
>
> *— Михай Чиксентмихайи, Поток: психология оптимального переживания (1990)*

> *Есть разница между тем, чтобы знать путь, и тем, чтобы пройти его.*
>
> *— Морфеус [Лоренс Фишберн], Матрица (1999)*

В середине 1950-х годов исследователь ВВС США Теодор Э. Харрис и отставной генерал армии США Фрэнк С. Росс написали секретный отчёт об исследовании железнодорожной сети, соединявшей Советский Союз со странами-сателлитами в Восточной Европе. Сеть моделировалась графом с 44 вершинами, представлявшими географические регионы, и 105 рёбрами — железнодорожными связями между ними. Каждому ребру был назначен вес, обозначавший скорость перевозки грузов из одного региона в другой. По существу методом проб и ошибок они определили и максимальный объём грузов, который можно переместить из России в Европу, и наиболее дешёвый способ нарушить работу сети, удалив связи — или, говоря менее отвлечённо, взорвав железнодорожные пути. Последний они назвали «узким местом». Их отчёт, включавший изображённую на рисунке 10.1 схему сети, рассекретили лишь в 1999 году.[10-1]

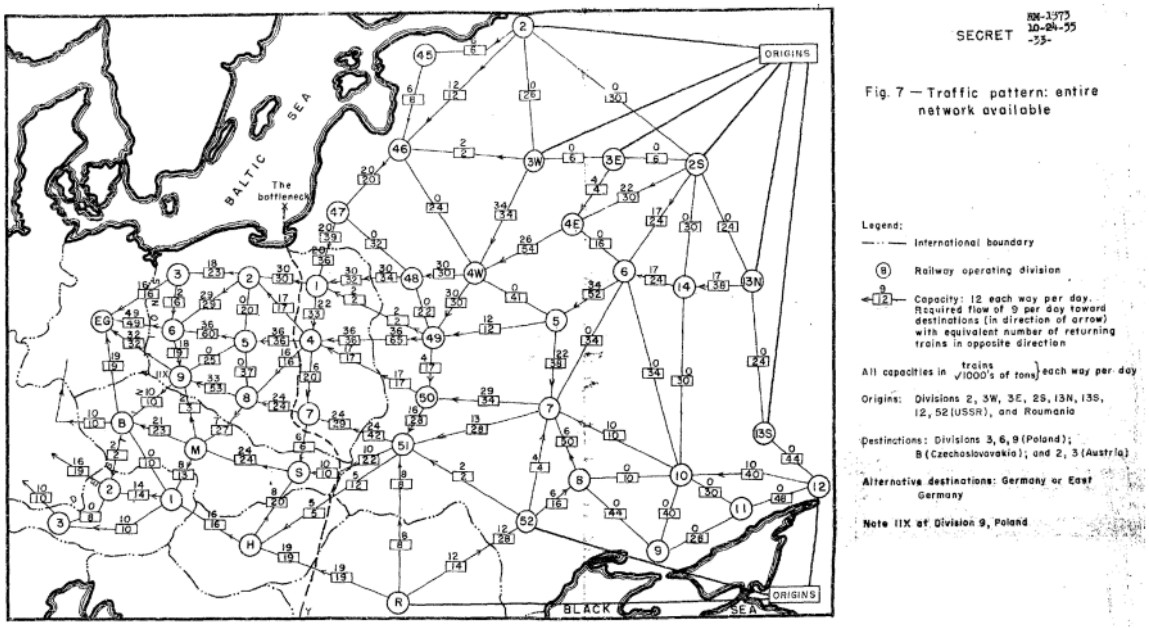


Рисунок 10.1. Карта железнодорожной сети стран Варшавского договора, составленная Харрисом и Россом. См. сведения об источниках иллюстраций в конце книги.

Это одно из первых зафиксированных применений задач о максимальном потоке и минимальном разрезе. В обеих задачах дан ориентированный граф $G = (V, E)$ с двумя особыми вершинами s и t, называемыми *источником* и *стоком*. Как и в предыдущих главах, через $u \to v$ обозначается направленное ребро из вершины u в вершину v. Интуитивно задача о максимальном потоке спрашивает, с какой наибольшей скоростью некоторый ресурс можно перемещать из s в t; задача о минимальном разрезе — какой минимальный ущерб необходимо нанести, чтобы отделить s от t.

> ***Примечание 10-1.*** *Об этой истории я узнал из увлекательного обзора Александера Схрейвера «Об истории комбинаторной оптимизации (до 1960 года)»; отчёт Харриса и Росса был рассекречен по просьбе Схрейвера. Форд и Фалкерсон, с которыми мы вскоре встретимся, приписывают постановку задачи о максимальном потоке Харрису, хотя точная хронология несколько запутанна: Харрис и Росс благодарят Джорджа Данцига «за помощь в формулировке задачи».*

10.1. Потоки

(s, t)-*поток* — или просто *поток*, если источник и сток ясны из контекста, — это функция $f : E \to \mathbb{R}$, удовлетворяющая следующему условию сохранения в каждой вершине v, кроме, возможно, s и t:

$$\sum_u f(u \to v) = \sum_w f(v \to w).$$

Иными словами, суммарный поток, входящий в v, равен суммарному потоку, выходящему из v. Для простоты обозначений положим $f(u \to v) = 0$, если ребра $u \to v$ в графе нет.

Величина потока f, обозначаемая $|f|$, — суммарный чистый поток, выходящий из источника s:

$$|f| := \sum_w f(s \to w) - \sum_u f(u \to s).$$

Нетрудно доказать, что $|f|$ также равно суммарному чистому потоку, входящему в сток t. Для упрощения обозначений пусть $\partial f(v)$ обозначает суммарный чистый поток, *выходящий* из вершины v:

$$\partial f(v) := \sum_w f(v \to w) - \sum_u f(u \to v).$$

> *Примечание переводчика: в формуле определения $\partial f(v)$ в исходном издании входящий и выходящий потоки переставлены местами. Исправленный выше знак согласуется со словами автора и последующими равенствами.*

Из условия сохранения следует, что $\partial f(v) = 0$ для каждой вершины v, кроме s и t, поэтому

$$\sum_v \partial f(v) = \partial f(s) + \partial f(t).$$

С другой стороны, любой поток, выходящий из одной вершины, входит в другую, поэтому $\sum_v \partial f(v) = 0$. Отсюда немедленно следует, что $|f| = \partial f(s) = -\partial f(t)$.

Теперь предположим, что дана ещё одна функция $c : E \to \mathbb{R}_{\geq 0}$, назначающая каждому ребру e неотрицательную *пропускную способность* $c(e)$. Поток f называется *допустимым* относительно c, если для каждого ребра e выполнено $0 \leq f(e) \leq c(e)$. В большинстве случаев мы рассматриваем только потоки, допустимые относительно некоторой фиксированной функции пропускных способностей c. Говорят, что поток f *насыщает* ребро e, если $f(e) = c(e)$, и *обходит* его, если $f(e) = 0$. Задача о максимальном потоке состоит в вычислении допустимого (s, t)-потока наибольшей возможной величины в заданном ориентированном графе с заданной функцией пропускных способностей.

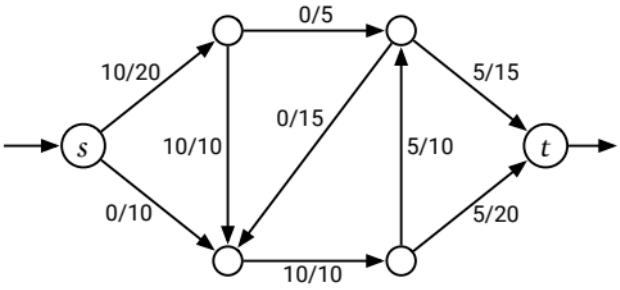


Рисунок 10.2. Допустимый (s, t)-поток величины 10. На каждом ребре указаны поток и пропускная способность.

10.2. Разрезы

(s,t)-*разрез* — или просто *разрез*, если источник и сток ясны из контекста, — это разбиение вершин на непересекающиеся подмножества S и T, то есть $S \cup T = V$ и $S \cap T = \varnothing$, причём $s \in S$ и $t \in T$.

Если задана функция пропускных способностей $c : E \to \mathbb{R}_{\geq 0}$, *пропускная способность разреза* — сумма пропускных способностей рёбер, которые начинаются в S и заканчиваются в T:

$$\|S,T\| := \sum_{v \in S} \sum_{w \in T} c(v \to w).$$

Как и прежде, если $v \to w$ не является ребром графа, полагаем $c(v \to w) = 0$. Заметим, что определение несимметрично: рёбра, начинающиеся в T и заканчивающиеся в S, не учитываются. Задача о минимальном разрезе состоит в вычислении (s,t)-разреза с наименьшей возможной пропускной способностью.

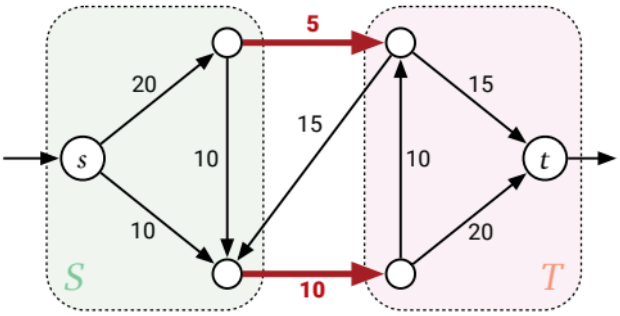


Рисунок 10.3. (s,t)-разрез пропускной способности 15. На каждом ребре указана его пропускная способность.

Интуитивно минимальный разрез — самый дешёвый способ прервать весь поток из s в t. Действительно, нетрудно показать следующую связь между потоками и разрезами.

Лемма 10.1. Пусть f — любой допустимый (s,t)-поток, а (S,T) — любой (s,t)-разрез. Величина f не превосходит пропускной способности (S,T). Более того, $|f| = \|S,T\|$ тогда и только тогда, когда f насыщает каждое ребро из S в T и обходит каждое ребро из T в S.

Доказательство. Выберите любой поток f и любой разрез (S,T) и проследите за цепочкой равенств и неравенств:

$$
\begin{aligned}
|f| &= \partial f(s) && \text{по определению} \\
&= \sum_{v \in S} \partial f(v) && \text{условие сохранения} \\
&= \sum_{v \in S} \sum_{w} f(v \to w) - \sum_{v \in S} \sum_{u} f(u \to v) && \text{определение } \partial \\
&= \sum_{v \in S} \sum_{w \notin S} f(v \to w) - \sum_{v \in S} \sum_{u \notin S} f(u \to v) && \text{исключаем рёбра внутри } S \\
&= \sum_{v \in S} \sum_{w \in T} f(v \to w) - \sum_{v \in S} \sum_{u \in T} f(u \to v) && \text{определение разреза} \\
&\le \sum_{v \in S} \sum_{w \in T} f(v \to w) && \text{поскольку } f(u \to v) \ge 0 \\
&\le \sum_{v \in S} \sum_{w \in T} c(v \to w) && \text{поскольку } f(v \to w) \le c(v \to w) \\
&= \|S, T\| && \text{по определению.}
\end{aligned}
$$

На втором шаге мы лишь прибавляем нули, поскольку $\partial f(v) = 0$ для каждой вершины $v \in S \setminus \{s\}$. На четвёртом шаге мы исключаем значения $f(x \to y)$, у которых и x, и y принадлежат S: они входят в обе суммы, с положительным знаком при $v = x$, $w = y$ и с отрицательным при $v = y$, $u = x$.

Первое неравенство в этом выводе является равенством тогда и только тогда, когда f обходит каждое ребро из T в S. Аналогично второе неравенство является равенством тогда и только тогда, когда f насыщает каждое ребро из S в T. $\square$

Из леммы сразу следует: если $|f| = \|S, T\|$, то f обязательно максимален, а (S, T) — минимальный разрез.

10.3. Теорема о максимальном потоке и минимальном разрезе

Удивительно, но в каждой транспортной сети существуют допустимый (s, t)-поток f и (s, t)-разрез (S, T) такие, что $|f| = \|S, T\|$. Это знаменитая теорема о максимальном потоке и минимальном разрезе, впервые доказанная Лестером Фордом, известным нам по кратчайшим путям, и Делбертом Фалкерсоном в 1954 году, а независимо от них — Питером Элиасом, Амиэлем Фейнстейном и Клодом Шенноном, известным по теории информации и роботу для решения лабиринтов, в 1956 году.

Теорема о максимальном потоке и минимальном разрезе. В любой транспортной сети с источником s и стоком t величина максимального (s, t)-потока равна пропускной способности минимального (s, t)-разреза.

Форд и Фалкерсон доказали теорему следующим образом. Зафиксируем граф G, вершины s и t и функцию пропускных способностей $c : E \to \mathbb{R}_{\ge 0}$. Доказательство будет проще, если предположить, что G *приведённый*: между любыми двумя вершинами u и v имеется не более одного ребра. В частности, либо $c(u \to v) = 0$, либо $c(v \to u) = 0$. Этого предположения легко добиться: разделим каждое ребро $u \to v$ графа G новой вершиной x, заменив его путём

$u \to x \to v$, и положим $c(u \to x) = c(x \to v) = c(u \to v)$. В полученном графе величина максимального потока и пропускная способность минимального разреза совпадают с исходными.

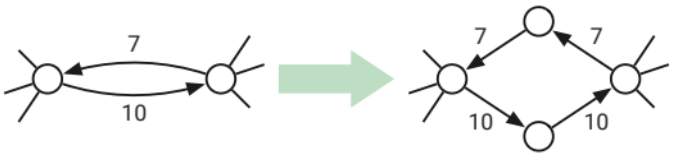


Рисунок 10.4. Обеспечение условия одного направления.

Пусть f — произвольный допустимый (s, t)-поток в G. Определим новую функцию пропускных способностей $c_f : V \times V \to \mathbb{R}$, называемую *остаточной пропускной способностью*:

$$c_f(u \to v) = \begin{cases} c(u \to v) - f(u \to v), & u \to v \in E, \\ f(v \to u), & v \to u \in E, \\ 0, & \text{иначе.} \end{cases}$$

Интуитивно остаточная пропускная способность ребра показывает, какой дополнительный поток можно по нему направить. Поскольку $f \geq 0$ и $f \leq c$, остаточные пропускные способности всегда неотрицательны. Может оказаться, что $c_f(u \to v) > 0$, хотя $u \to v$ не является ребром исходного графа G. Поэтому определим *остаточный граф* $G_f = (V, E_f)$, где E_f — множество рёбер с положительной остаточной пропускной способностью. Большинство остаточных графов не являются приведёнными. В частности, если $0 < f(u \to v) < c(u \to v)$, остаточный граф G_f содержит и $u \to v$, и обратное ему ребро $v \to u$.

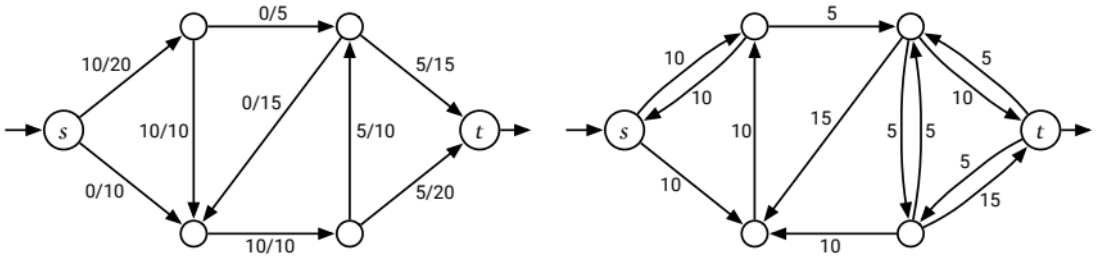


Рисунок 10.5. Поток f во взвешенном графе G и соответствующий остаточный граф G_f.

Теперь рассмотрим два случая: либо в остаточном графе G_f существует направленный путь от источника s до стока t, либо такого пути нет.

Сначала предположим, что G_f содержит направленный путь P из s в t; назовём P *увеличивающим путём*. Пусть $F = \min_{u \to v \in P} c_f(u \to v)$ — максимальный поток, который можно направить по P. Определим новый поток $f' : E \to \mathbb{R}$ в исходном графе:

$$f'(u \to v) = \begin{cases} f(u \to v) + F, & u \to v \in P, \\ f(u \to v) - F, & v \to u \in P, \\ f(u \to v), & \text{иначе.} \end{cases}$$

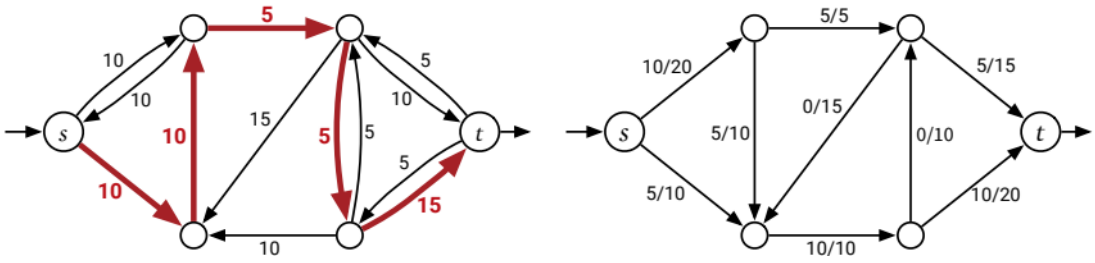


Рисунок 10.6. Увеличивающий путь величины $F = 5$ и полученный увеличенный поток f'.

Утверждается, что новый поток f' допустим относительно исходных пропускных способностей c, то есть всюду $f' \geq 0$ и $f' \leq c$. Рассмотрим ребро $u \to v$ исходного графа G. Возможны три случая.

- Если увеличивающий путь P содержит $u \to v$, то $f'(u \to v) = f(u \to v) + F > f(u \to v) \geq 0$, поскольку f допустим, и

$$\begin{aligned} f'(u \to v) &= f(u \to v) + F \\ &\leq f(u \to v) + c_f(u \to v) \\ &= f(u \to v) + c(u \to v) - f(u \to v) \\ &= c(u \to v). \end{aligned}$$

- Если увеличивающий путь P содержит обратное ребро $v \to u$, то $f'(u \to v) = f(u \to v) - F < f(u \to v) \leq c(u \to v)$, опять же благодаря допустимости f, и

$$\begin{aligned} f'(u \to v) &= f(u \to v) - F \\ &\geq f(u \to v) - c_f(v \to u) \\ &= f(u \to v) - f(u \to v) \\ &= 0. \end{aligned}$$

- Наконец, если увеличивающий путь не содержит ни $u \to v$, ни $v \to u$, то $f'(u \to v) = f(u \to v)$ и, поскольку f допустим, $0 \leq f'(u \to v) \leq c(u \to v)$.

Итак, f' действительно допустим. Более того, только первое ребро увеличивающего пути выходит из s, поэтому $|f'| = |f| + F > |f|$. Таким образом, f' — допустимый поток большей величины, чем f. Следовательно, если в остаточном графе G_f есть путь из s в t, поток f не максимален.

С другой стороны, предположим, что G_f не содержит направленного пути из s в t. Пусть S — множество вершин, достижимых из s в G_f, а $T = V \setminus S$. Разбиение (S, T), очевидно, является (s, t)-разрезом. Для любых $u \in S$ и $v \in T$ имеем

$$c_f(u \to v) = \big(c(u \to v) - f(u \to v)\big) + f(v \to u) = 0.$$

Поскольку f допустим, $c(u \to v) - f(u \to v) \geq 0$ и $f(v \to u) \geq 0$, а значит, оба слагаемых равны нулю. Иными словами, f насыщает каждое ребро из S в T и обходит каждое ребро из T

в S. По лемме 10.1 получаем $|f| = \|S, T\|$, следовательно, f — максимальный поток, а (S, T) — минимальный разрез.

Доказательство завершено! □

10.4. Алгоритм Форда — Фалкерсона с увеличивающими путями

Доказательство теоремы о максимальном потоке и минимальном разрезе сразу подсказывает алгоритм вычисления максимального потока: начав с нулевого потока, многократно увеличивать его вдоль любого пути из s в t в остаточном графе, пока такой путь существует.

Из алгоритма следует важное, хотя и несложное следствие.

Теорема о целочисленности. Если все пропускные способности транспортной сети целые, то существует максимальный поток, принимающий на каждом ребре целочисленное значение.

Доказательство. По индукции покажем, что после каждой итерации алгоритма увеличивающих путей все значения потока и остаточных пропускных способностей целые.

- До первой итерации все значения потока равны нулю, а все остаточные пропускные способности равны исходным, которые по условию целые.
- На каждой последующей итерации предположение индукции означает, что пропускная способность F увеличивающего пути целая; значит, увеличение меняет поток на каждом ребре, а следовательно и остаточную пропускную способность каждого ребра, на целое число.

В частности, каждая итерация увеличивает величину потока на положительное целое число. Следовательно, алгоритм в конце концов останавливается и возвращает максимальный поток. □

Если пропускная способность каждого ребра целая, то алгоритм Форда — Фалкерсона заведомо останавливается не более чем через $|f^*|$ итераций, где f^* — максимальный поток. На каждой итерации можно построить остаточный граф G_f и выполнить поиск в любом порядке, найдя увеличивающий путь за $O(E)$ времени. Следовательно, время работы в худшем случае равно $O(E|f^*|)$.

Джек Эдмондс и Ричард Карп заметили, что эта оценка по существу точна. Рассмотрим сеть из четырёх вершин на рисунке 10.7, где X — большое целое число. Максимальный поток в ней, очевидно, равен $2X$. Однако алгоритм Форда — Фалкерсона может попеременно направлять единицу потока по увеличивающему пути $s \to u \to v \to t$, а затем единицу по пути $s \to v \to u \to t$. При этом время работы составит $\Theta(X) = \Omega(|f^*|)$.

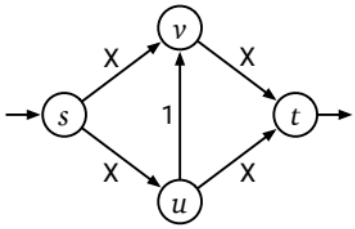


Рисунок 10.7. Плохой пример Эдмондса и Карпа для алгоритма Форда — Фалкерсона.

На практике алгоритм Форда — Фалкерсона обычно работает быстро и всегда быстр, когда величина максимального потока $|f^*|$ мала. Но без дополнительных ограничений на увеличивающие пути он неэффективен в худшем случае. Сеть Эдмондса и Карпа можно описать всего $O(\log X)$ битами; следовательно, время работы Форда — Фалкерсона экспоненциально по размеру входа.

♥ Иррациональные пропускные способности

Но что, если пропускные способности не целые? Если умножить все пропускные способности на одну и ту же положительную константу, максимальный поток всюду увеличится в такое же число раз. Следовательно, если пропускные способности всех рёбер рациональны, алгоритм Форда — Фалкерсона в конце концов останавливается, хотя его время работы по-прежнему экспоненциально по числу битов, описывающих вход.

Однако если разрешить иррациональные пропускные способности, алгоритм может работать бесконечно, каждый раз находя всё меньшие увеличивающие пути. Хуже того, эта бесконечная последовательность увеличений может даже не сходиться к максимальному потоку или к значительной его доле! Наименьшую сеть с таким плохим поведением обнаружил Ури Цвик в 1993 году.[10-2]

Рассмотрим сеть из шести вершин на рисунке 10.8. Шесть из девяти рёбер имеют некоторую большую целочисленную пропускную способность X, два — способность 1, а одно — способность $\varphi = (\sqrt{5} - 1)/2 \approx 0{,}618034$, выбранную так, что $1 - \varphi = \varphi^2$. Чтобы показать, что алгоритм Форда — Фалкерсона может зациклиться, проследим за остаточными пропускными способностями трёх горизонтальных рёбер по мере работы алгоритма. Остаточные пропускные способности шести остальных рёбер всегда будут не меньше $X - 3$.

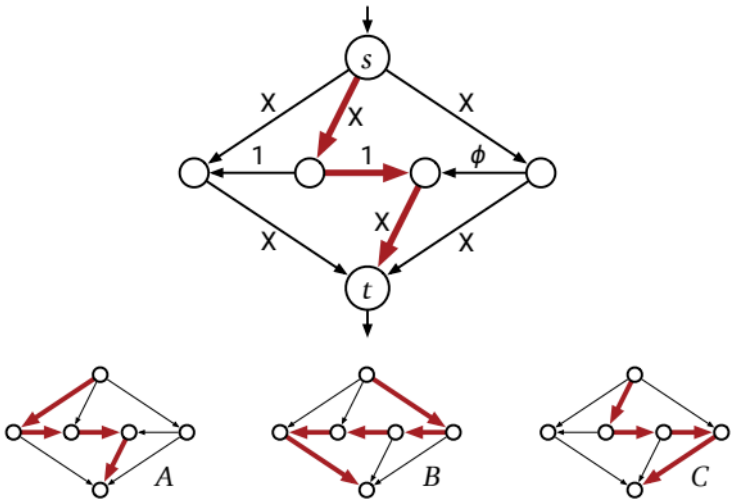


Рисунок 10.8. Пример Ури Цвика, на котором алгоритм не останавливается, и три увеличивающих пути.

Пусть алгоритм Форда — Фалкерсона начинает с выбора центрального увеличивающего пути, показанного в верхней части рисунка 10.8. Остаточные пропускные способности трёх горизонтальных рёбер слева направо становятся равны 1, 0 и φ. Предположим по индукции, что они равны φ^{k-1}, 0 и φ^k для некоторого неотрицательного целого k.

1. Увеличим поток вдоль пути B на φ^k; остаточные пропускные способности станут φ^{k+1}, φ^k, 0.
2. Увеличим поток вдоль пути C на φ^k; получим φ^{k+1}, 0, φ^k.
3. Увеличим поток вдоль пути B на φ^{k+1}; получим 0, φ^{k+1}, φ^{k+2}.
4. Увеличим поток вдоль пути A на φ^{k+1}; получим φ^{k+1}, 0, φ^{k+2}.

В исходном тексте далее указано, что после $4n+1$ шагов увеличения остаточные пропускные способности горизонтальных рёбер равны φ^{2n-2}, 0, φ^{2n-1} и что величина потока стремится к

$$1 + 2\sum_{i=1}^{\infty} \varphi^i = 1 + \frac{2}{1-\varphi} = 4 + \sqrt{5} < 7,$$

хотя величина максимального потока, очевидно, равна $2X + 1 \gg 7$.

> *Примечание переводчика: приведённые в оригинале показатели и равенство геометрической суммы содержат арифметические ошибки. После начального увеличения и n полных циклов из четырёх шагов получаем $\varphi^{2n}, 0, \varphi^{2n+1}$; сумма указанных приращений равна $1 + 2\sum_{i=1}^{\infty}\varphi^i = 1 + 2\varphi/(1-\varphi) = 2 + \sqrt{5}$. Это не меняет вывода о незавершении алгоритма и отсутствии сходимости к максимальному потоку при большом X.*

Читатель с практическим складом ума может спросить, зачем вообще беспокоиться об иррациональных пропускных способностях: ведь компьютеры могут точно представлять только небольшие целые числа или небольшие рациональные числа со степенью двойки в

знаменателе. Хороший вопрос! Ответ математика состоит в том, что ограничение целыми пропускными способностями буквально искусственно: это следствие устройства цифровой вычислительной техники или, возможно, не относящихся к делу законов физики, а не внутреннее свойство абстрактной вычислительной задачи.

Но есть и более практическая причина: поведение алгоритма на иррациональных входных данных кое-что говорит о его худшем поведении на практике при пропускных способностях с плавающей точкой — и это поведение ужасно! Даже при вполне разумных пропускных способностях небрежная реализация Форда — Фалкерсона может зациклиться просто из-за ошибок округления, так и не приблизившись к правильному ответу.

> ***Примечание 10-2.*** *В 1962 году Форд и Фалкерсон описали более сложную сеть из 10 вершин и 48 рёбер с таким же плохим поведением.*

10.5. Сложение и разложение потоков

Обычно потоки определяют как функции на рёбрах графа, удовлетворяющие определённым ограничениям в вершинах. Однако существует и второе описание потоков, в некоторых случаях более естественное и полезное.

Рассмотрим произвольный граф G с источником s и стоком t. Зафиксируем любые два (s,t)-потока f и g и любые два действительных числа α и β и рассмотрим функцию $h : E \to \mathbb{R}$, определённую для каждого ребра $u \to v$ равенством

$$h(u \to v) := \alpha \cdot f(u \to v) + \beta \cdot g(u \to v).$$

Короче эту запись можно представить как $h = \alpha f + \beta g$. Непосредственная проверка определений показывает, что h тоже является (s,t)-потоком величины $|h| = \alpha|f| + \beta|g|$. Вообще, любая линейная комбинация (s,t)-потоков тоже является (s,t)-потоком.

Оказывается, любой (s,t)-поток можно записать как взвешенную сумму потоков специального вида. Для любого направленного пути P из s в t определим соответствующий *поток по пути*:

$$P(u \to v) = \begin{cases} 1, & u \to v \in P, \\ -1, & v \to u \in P, \\ 0, & \text{иначе.} \end{cases}$$

Непосредственная проверка определений показывает, что функция $P : E \to \mathbb{R}$ действительно является (s,t)-потоком величины 1. Я намеренно использую обозначение P и для пути — последовательности вершин и направленных рёбер, — и для единичного потока вдоль этого пути.

Аналогично для любого направленного цикла C определим соответствующий *поток по циклу*:

$$C(u \to v) = \begin{cases} 1, & u \to v \in C, \\ -1, & v \to u \in C, \\ 0, & \text{иначе.} \end{cases}$$

Опять-таки легко проверить, что $C : E \to \mathbb{R}$ является (s,t)-потоком нулевой величины. Из предыдущего рассуждения следует, что любая линейная комбинация потоков по путям и потоков по циклам тоже является потоком; такую взвешенную сумму называют *разложением потока*. Более того, любой неотрицательный поток допускает разложение следующего специального вида.

Теорема о разложении потока. Любой неотрицательный (s,t)-поток f можно записать как положительную линейную комбинацию направленных (s,t)-путей и направленных циклов. Причём направленное ребро $u \to v$ встречается хотя бы в одном из этих путей или циклов тогда и только тогда, когда $f(u \to v) > 0$, а общее число путей и циклов не превышает числа рёбер сети.

Доказательство. Докажем теорему по индукции по числу рёбер с ненулевым потоком, интуитивно выполняя алгоритм Форда — Фалкерсона в обратном направлении. Пока хотя бы по одному ребру графа проходит положительный поток, можно найти либо (s,t)-путь, либо направленный цикл, по которому проходит поток. Вычитание максимально возможного потока вдоль этого пути или цикла опустошает хотя бы одно ребро, после чего Фея рекурсии может дать оставшуюся часть разложения.

Для формализации сначала рассмотрим частный случай *циркуляций* — потоков нулевой величины, сохраняющихся в каждой вершине. Зафиксируем произвольную циркуляцию f в произвольной транспортной сети; пусть $\#f$ обозначает число рёбер $u \to v$, для которых $f(u \to v) > 0$. По индукции по $\#f$ докажем, что f можно разложить в положительную линейную комбинацию не более чем $\max\{0, \#f - 1\}$ циклов. Возможны три случая.

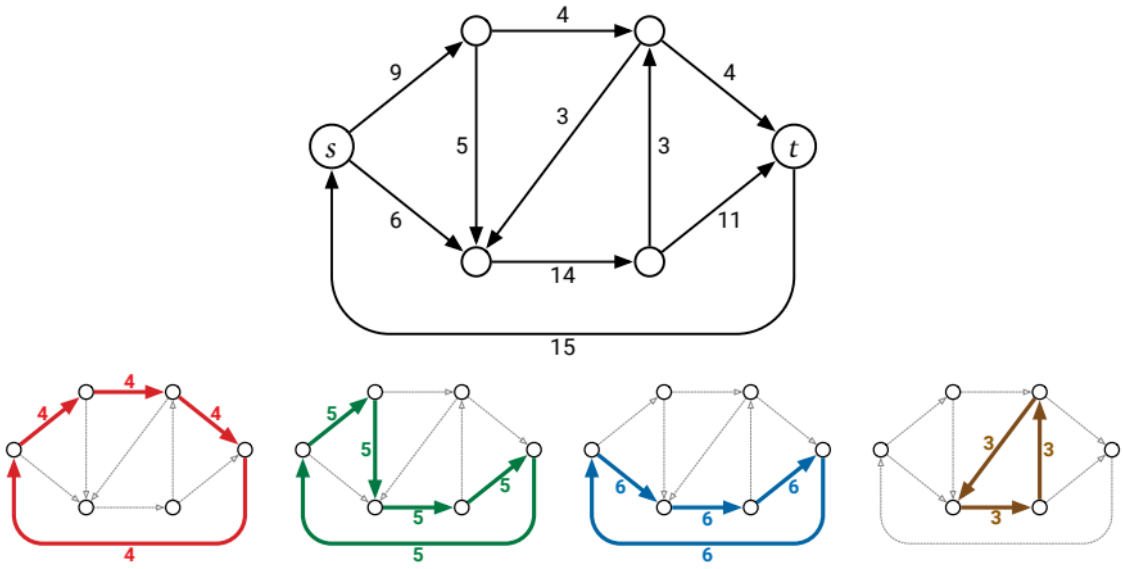


Рисунок 10.9. Разложение циркуляции на взвешенные направленные циклы.

- Если $\#f = 0$, то f по пустому условию является линейной комбинацией нуля циклов.
- Предположим, что $f(u \to v) > 0$ только на рёбрах одного направленного цикла. Тогда $\#f \geq 2$ и f тривиально является линейной комбинацией одного цикла.

- Иначе выберем произвольное ребро $u \to v$ с $f(u \to v) > 0$. Рассмотрим произвольный маршрут $v_0 \to v_1 \to v_2 \to \cdots$, где $v_0 = u$, $v_1 = v$, а $f(v_{i-1} \to v_i) > 0$ для каждого индекса i. Из условия сохранения следует, что у каждой вершины с входящим потоком есть и исходящий поток, поэтому маршрут можно продолжать сколь угодно долго; в частности, в конце концов он должен посетить какую-то вершину повторно. Пусть k — наименьший индекс, для которого $v_j = v_k$ при некотором $j < k$. Тогда подмаршрут $v_j \to v_{j+1} \to \cdots \to v_k$ является простым направленным циклом C.

  > *Примечание переводчика: в оригинале второй индекс в записи подмаршрута указан как $j - 1$; по смыслу маршрута исправлено на $j + 1$.*

  Положим $F := \min_{e \in C} f(e)$ и рассмотрим функцию $f' := f - F \cdot C$, или подробнее:

  $$f'(u \to v) := \begin{cases} f(u \to v) - F, & u \to v \in C, \\ f(u \to v), & \text{иначе.} \end{cases}$$

  Непосредственная проверка определений показывает, что f' — ещё одна допустимая циркуляция в G. Существует хотя бы одно ребро $e \in C$ с $f(e) = F$, и потому $f'(e) = 0$, откуда $\#f' \le \#f - 1$. Поскольку в f' поток проходит по меньшему числу рёбер, чем в f, Фея рекурсии может разложить f' не более чем на $\#f' - 1 \le \#f - 2$ циклов. Добавив F единиц потока вдоль цикла C, получаем разложение f; короче: $f = f' + F \cdot C$.

Теперь пусть f — произвольный (s, t)-поток в произвольной транспортной сети, причём $|f| > 0$. Добавим в сеть ребро $t \to s$ и определим циркуляцию f', положив $f'(t \to s) = |f|$ и $f'(u \to v) = f(u \to v)$ для каждого исходного ребра $u \to v$. Заметим, что $\#f' = \#f + 1 \ge 2$. Предыдущее рассуждение показывает, что циркуляция f' является положительной линейной комбинацией не более чем $\#f' - 1$ направленных циклов. Удалив ребро $t \to s$, получаем разложение исходного потока f не более чем на $\#f' - 1 = \#f$ путей и циклов. А именно: циклы f', содержащие $t \to s$, становятся (s, t)-путями в f, а циклы, не содержащие $t \to s$, остаются циклами. $\square$

Доказательство теоремы о разложении потока даёт более сильные результаты в двух интересных частных случаях:

- любую циркуляцию можно разложить во взвешенную сумму циклов; пути не нужны;
- любой ациклический (s, t)-поток можно разложить во взвешенную сумму (s, t)-путей; циклы не нужны.

Более того, последовательно устраняя циклы потока, пока их не останется, можно преобразовать любой поток в ациклический поток той же величины. В частности, любая транспортная сеть допускает ациклический максимальный (s, t)-поток.

Доказательство непосредственно превращается в алгоритм, похожий на алгоритм Форда — Фалкерсона, для разложения любого (s, t)-потока на пути и циклы. Алгоритм многократно ищет в оставшемся потоке направленный (s, t)-путь или направленный цикл, а затем вычитает

вдоль найденного пути или цикла максимально возможный поток, пока поток не иссякнет. Найти путь или цикл потока можно за $O(V)$ времени следующим образом:

- Если какое-либо ребро, выходящее из s, несёт положительный поток, следуем по произвольному маршруту из s в графе потока, пока не достигнем t — получив путь потока, — или не посетим какую-либо вершину во второй раз — получив цикл потока.
- Если ни одно ребро, выходящее из s, не несёт положительного потока, найдём любую другую вершину v с положительным исходящим потоком и следуем от неё по произвольному маршруту в графе потока, пока не посетим какую-либо вершину во второй раз; получим цикл потока.

В обоих случаях условие сохранения означает, что алгоритм никогда не застрянет. Каждая итерация занимает $O(V)$ времени и удаляет из графа потока хотя бы одно ребро, поэтому весь алгоритм разложения работает за $O(VE)$ времени.

Разложения потока дают естественную нижнюю оценку времени работы любого алгоритма поиска максимального потока, который строит поток по одному пути или циклу за раз. Каждый поток раскладывается не более чем на E путей и циклов, каждый из которых использует не более V рёбер, поэтому полная сложность разложения равна $O(VE)$. Более того, нетрудно построить потоки, для которых сложность любого разложения равна $\Omega(VE)$. Следовательно, любой алгоритм поиска максимального потока, который явно строит поток по одному пути или циклу за раз, в частности любая реализация алгоритма увеличивающих путей Форда — Фалкерсона, в худшем случае должен затратить $\Omega(VE)$ времени.

10.6. Алгоритмы Эдмондса и Карпа

Алгоритм Форда — Фалкерсона не уточняет, по какому пути в остаточном графе увеличивать поток; в плохом поведении алгоритма в худшем случае можно винить неудачный выбор увеличивающих путей. В начале 1970-х годов Джек Эдмондс и Ричард Карп опубликовали два естественных правила выбора увеличивающих путей, оба из которых привели к более эффективным алгоритмам.

Самые широкие увеличивающие пути

Первое правило Эдмондса и Карпа по существу задаёт жадный алгоритм:

> *Выбирайте увеличивающий путь с наибольшей пропускной способностью его узкого места.*

Нетрудно показать, что (s, t)-путь с максимальным узким местом в ориентированном графе можно вычислить за $O(E \log V)$ времени обходом «сначала лучший», похожим на алгоритм Ярника для минимального остовного дерева или алгоритм Дейкстры для кратчайших путей. Алгоритм выращивает ориентированное дерево T с корнем в s по одной вершине за раз, многократно добавляя к T выходящее из него ребро наибольшей пропускной способности, пока T не будет содержать путь из s в t. Можно также имитировать алгоритм Краскала:

добавлять рёбра по одному в порядке убывания пропускной способности, пока не появится путь из s в t; однако такой подход менее эффективен, по крайней мере для ориентированного графа.

Чтобы завершить анализ времени работы алгоритма поиска потока, нужна верхняя оценка числа итераций до остановки. Вообще, при произвольных действительных пропускных способностях алгоритм может никогда не остановиться; см. упражнение 18. Но при целочисленных пропускных способностях число итераций можно оценить через величину максимального потока $|f^*|$ следующим образом.

Пусть f — любой поток в G, а f' — максимальный поток в текущем остаточном графе G_f. В начале алгоритма $G_f = G$ и $f' = f^*$. Мы уже доказали, что f' можно разложить не более чем на E путей и циклов. Простое рассуждение об усреднении показывает, что хотя бы один из путей разложения должен нести не менее $|f'|/E$ единиц потока. Отсюда сразу следует, что самый широкий (s,t)-путь в G_f несёт не менее $|f'|/E$ единиц потока.

Поэтому увеличение f вдоль пути с максимальным узким местом в G_f умножает величину оставшегося максимального потока в G_f не более чем на $1 - 1/E$. Иными словами, величина остаточного максимального потока экспоненциально убывает с числом итераций. После $E \cdot \ln |f^*|$ итераций величина максимального потока в G_f не превосходит

$$|f^*|(1-1/E)^{E \ln |f^*|} < |f^*| e^{-\ln |f^*|} = 1.$$

Это константа Эйлера e, а не ребро e. Простите. В частности, после $E \cdot \ln |f^*|$ итераций величина остаточного максимального потока меньше 1. Если все пропускные способности целые, то и величина остаточного максимального потока целая, поэтому она обязана равняться нулю; иными словами, f — максимальный поток!

Мы заключаем, что для графов с целочисленными пропускными способностями алгоритм Эдмондса — Карпа с выбором самого широкого пути работает за $O(E^2 \log E \log |f^*|)$ времени. В отличие от оценки худшего случая для обычного алгоритма Форда — Фалкерсона, эта граница действительно является полиномиальной функцией размера входа.

Как и исходный алгоритм Форда — Фалкерсона, алгоритм самого широкого пути может зациклиться в сетях с произвольными действительными пропускными способностями. Однако наш анализ показывает, что даже если алгоритм никогда не остановится, поддерживаемый им поток f в пределе приближается к максимальному.

Кратчайшие увеличивающие пути

Второе правило Эдмондса — Карпа на самом деле было предложено Фордом и Фалкерсоном как практическая эвристика уже в их исходной работе о максимальных потоках; вариант этого правила независимо предложил в 1970 году российский математик Ефим Диниц.[10-3]

> *Выбирайте увеличивающий путь с наименьшим числом рёбер.*

Кратчайший увеличивающий путь можно найти за $O(E)$ времени с помощью поиска в ширину в остаточном графе. Удивительно, но полученный алгоритм останавливается после

полиномиального числа итераций независимо от конкретных пропускных способностей рёбер!

Доказательство этой полиномиальной верхней границы опирается на два наблюдения об изменении остаточного графа. Пусть f_i — текущий поток после i шагов увеличения, а G_i — соответствующий остаточный граф. В частности, f_0 всюду равен нулю и $G_0 = G$. Для каждой вершины v пусть $\mathrm{level}_i(v)$ обозначает невзвешенное расстояние кратчайшего пути от s до v в G_i или, что то же самое, уровень v в дереве поиска в ширину графа G_i с корнем s. В частности, если пути из s в v в G_i нет, то $\mathrm{level}_i(v) = \infty$, поскольку $\min \varnothing = \infty$.

Первое наблюдение: со временем уровень вершины может только увеличиваться.

Лемма 10.2. $\mathrm{level}_i(v) \geq \mathrm{level}_{i-1}(v)$ для всех вершин v и всех целых $i > 0$.

Доказательство. Зафиксируем произвольное положительное целое i и произвольную вершину v. Докажем утверждение по индукции по $\mathrm{level}_i(v)$, а не по целому i. В качестве предположения индукции допустим, что для каждой вершины u, для которой $\mathrm{level}_i(u) < \mathrm{level}_i(v)$, выполнено $\mathrm{level}_i(u) \geq \mathrm{level}_{i-1}(u)$. Возможны три случая.

- Если $v = s$, сразу получаем $\mathrm{level}_i(s) = \mathrm{level}_{i-1}(s) = 0$.
- Если пути из s в v в G_i нет, то $\mathrm{level}_i(v) = \infty \geq \mathrm{level}_{i-1}(v)$.
- Иначе пусть $s \to \cdots \to u \to v$ — любой невзвешенный кратчайший путь из s в v в графе G_i. Поскольку он кратчайший, $\mathrm{level}_i(v) = \mathrm{level}_i(u) + 1$, и из предположения индукции следует $\mathrm{level}_i(u) \geq \mathrm{level}_{i-1}(u)$. Осталось показать, что $\mathrm{level}_{i-1}(u) \geq \mathrm{level}_{i-1}(v) - 1$. Возможны два подслучая.
  - Если $u \to v$ является ребром G_{i-1}, то $\mathrm{level}_{i-1}(v) \leq \mathrm{level}_{i-1}(u) + 1$, поскольку уровни определены обходом в ширину.
  - Если же $u \to v$ не является ребром G_{i-1}, то обратное ребро $v \to u$ должно входить в i-й увеличивающий путь, который по определению является кратчайшим путём из s в t в G_{i-1}. Следовательно, $\mathrm{level}_{i-1}(v) = \mathrm{level}_{i-1}(u) - 1 \leq \mathrm{level}_{i-1}(u) + 1$.

В обоих подслучаях $\mathrm{level}_i(v) = \mathrm{level}_i(u) + 1 \geq \mathrm{level}_{i-1}(u) + 1 \geq \mathrm{level}_{i-1}(v)$. $\square$

> ***Примечание 10-3.*** *Точнее, Диниц открыл более сложный алгоритм поиска максимального потока, будучи студентом на курсе алгоритмов Георгия Адельсон-Вельского — первой букве в названии АВЛ-деревьев — и решая задачу, предложенную на занятии. Алгоритм Диница также направляет потоки по кратчайшим путям, но ведёт дополнительный учёт, позволяющий уменьшить время работы с $O(VE^2)$ до $O(V^2E)$.*

Всякий раз, увеличивая поток, мы удаляем из остаточного графа ребро, являющееся узким местом увеличивающего пути; при этом некоторые обратные рёбра пути могут появиться вновь. Второе наблюдение: ребро не может появляться и исчезать слишком много раз.

Лемма 10.3. Во время работы алгоритма Эдмондса — Карпа с кратчайшими увеличивающими путями каждое ребро $u \to v$ исчезает из остаточного графа G_f не более $V/2$ раз.

Доказательство. Предположим, что $u \to v$ входит в два остаточных графа G_i и G_{j+1}, но не входит ни в один промежуточный граф $G_{i+1}, \ldots, G_j$ при некоторых $i < j$. Тогда $u \to v$ должно находиться в увеличивающем пути, вдоль которого выполнялся переход от G_i к G_{i+1}, поэтому $\text{level}_i(v) = \text{level}_i(u) + 1$; а $v \to u$ должно находиться в увеличивающем пути от G_j к G_{j+1}, поэтому $\text{level}_j(v) = \text{level}_j(u) - 1$. По предыдущей лемме

$$\text{level}_j(u) = \text{level}_j(v) + 1 \geq \text{level}_i(v) + 1 = \text{level}_i(u) + 2.$$

Иными словами, между исчезновением и повторным появлением $u \to v$ расстояние от s до u увеличилось не менее чем на 2. Поскольку каждый уровень либо меньше V, либо бесконечен, число исчезновений не превосходит $V/2$. □

> *Примечание переводчика: в оригинале пути, используемые при переходах $G_i \to G_{i+1}$ и $G_j \to G_{j+1}$, названы i-м и j-м, хотя выше i-й шаг определён как переход $G_{i-1} \to G_i$. Здесь переходы записаны явно.*

Теперь можно получить верхнюю границу числа итераций. Поскольку каждое ребро исчезает не более $V/2$ раз, всего происходит не более $EV/2$ исчезновений. Но на каждой итерации исчезает хотя бы одно ребро, следовательно, алгоритм останавливается не более чем через $EV/2$ итераций. Каждая итерация требует $O(E)$ времени, поэтому полный алгоритм работает за $O(VE^2)$ времени.

10.7. Дальнейшие достижения

На этом история алгоритмов поиска максимального потока далеко не заканчивается. Десятилетия дальнейших исследований привели к нескольким более быстрым алгоритмам; некоторые из них перечислены на рисунке 10.10.[10-4] Все перечисленные алгоритмы вычисляют максимальный поток за несколько итераций. У большинства есть два варианта: более простой выполняет каждую итерацию полным перебором, а более быстрый использует сложные структуры данных для поддержания остовного дерева транспортной сети, благодаря чему каждую итерацию и обновление дерева можно выполнить за логарифмическое время. Нет оснований считать лучшие известные алгоритмы оптимальными; действительно, максимальные потоки по-прежнему являются очень активной областью исследований.

Метод	Прямой вариант	С динамическими деревьями	Источники
Блокирующий поток	$O(V^2E)$	$O(VE \log V)$	Диниц; Карзанов; Эвен и Итай; Слейтор и Тарьян
Сетевой симплекс-метод	$O(V^2E)$	$O(VE \log V)$	Данциг; Голдфарб и Хао; Голдберг, Григориадис и Тарьян

Метод	Прямой вариант	С динамическими деревьями	Источники
Проталкивание и переименование, общий вариант	$O(V^2E)$	—	Голдберг и Тарьян
Проталкивание и переименование, FIFO	$O(V^3)$	$O(VE\log(V^2/E))$	Голдберг и Тарьян
Проталкивание и переименование, наибольшая метка	$O(V^2\sqrt{E})$	—	Чериян и Махешвари; Тунчел
Игры проталкивания, переименования и сложения	—	$O(VE\log_{E/(V\log V)}V)$	Чериян и Хагеруп; Кинг, Рао и Тарьян
Псевдопоток	$O(V^2E)$	$O(VE\log V)$	Хохбаум
Псевдопоток, наибольшая метка	$O(V^3)$	$O(VE\log(V^2/E))$	Хохбаум и Орлин
Инкрементальный поиск в ширину	$O(V^2E)$	$O(VE\log(V^2/E))$	Голдберг, Хелд, Каплан, Тарьян и Вернек
Компактные сети	—	$O(VE)$	Орлин

Рисунок 10.10. Несколько чисто комбинаторных алгоритмов поиска максимального потока и время их работы.

Самый быстрый известный автору чисто комбинаторный алгоритм поиска максимального потока, анонсированный Джеймсом Орлином в 2012 году, работает за $O(VE)$ времени, в точности совпадая со сложностью разложения потока в худшем случае. Подробности алгоритма Орлина далеко выходят за рамки этой книги. Помимо собственных новых приёмов, Орлин использует несколько более ранних алгоритмов и структур данных как чёрные ящики, причём большинство из них сами по себе довольно сложны. В частности, его алгоритм не строит разложение потока явно; более того, для графов всего с $O(V)$ рёбер расширение его алгоритма работает всего за $O(V^2/\log V)$ времени!

Тем не менее именно эту оценку следует использовать при анализе алгоритмов, применяющих максимальные потоки. Поэтому запишите в шпаргалки и приводите в домашних заданиях следующее утверждение:

> *Максимальные потоки можно вычислить за $O(VE)$ времени.*

Наконец, для сетей с единичной пропускной способностью, где каждое ребро имеет способность 1, известны более быстрые алгоритмы. В 1973 году Александр Карзанов доказал, что алгоритм блокирующего потока Диница — первый алгоритм в приведённой выше таблице

— в этом случае работает за $O(\min\{V^{2/3}, E^{1/2}\}E)$ времени. Эта оценка, кажется, преодолевает барьер $\Omega(VE)$ для разложения потока, но на самом деле из анализа Карзанова следует, что любой поток в сети с единичными способностями можно разложить на пути общей сложности $O(\min\{V^{2/3}, E^{1/2}\}E)$. Четыре десятилетия это был самый быстрый известный алгоритм для данного случая. Рекорд Карзанова был побит в 2013 году, когда Александр Мондры объявил о поистине замечательном алгоритме вычисления максимального потока в таких сетях за $O(E^{10/7}\operatorname{polylog} E)$ времени. И снова подробности алгоритма Мондры далеко выходят за рамки этой книги и даже за пределы компетенции её автора.

> ***Примечание 10-4.*** *Чтобы таблица была короткой, я намеренно опустил алгоритмы, время работы которых зависит от пропускных способностей рёбер или величины максимального потока. Даже с этим ограничением список постыдно неполон!*

Упражнения

0. Дан ориентированный граф $G = (V, E)$, две вершины s и t, функция пропускных способностей $c : E \to \mathbb{R}_+$ и вторая функция $f : E \to \mathbb{R}$. Опишите алгоритм проверки, является ли f максимальным (s, t)-потоком в G.

1. Пусть f и f' — два допустимых (s, t)-потока в транспортной сети G, причём $|f'| > |f|$. Докажите, что в остаточной сети G_f существует допустимый (s, t)-поток величины $|f'| - |f|$.

2. Пусть $u \to v$ — произвольное ребро произвольной транспортной сети G. Докажите: если существует минимальный (s, t)-разрез (S, T), для которого $u \in S$ и $v \in T$, то не существует минимального разреза (S', T'), для которого $u \in T'$ и $v \in S'$.

3. Пусть (S, T) и (S', T') — минимальные (s, t)-разрезы некоторой транспортной сети G. Докажите, что $(S \cap S', T \cup T')$ и $(S \cup S', T \cap T')$ также являются минимальными (s, t) -разрезами в G.

4. Пусть транспортная сеть G содержит пару противоположных рёбер $u \to v$ и $v \to u$, оба с положительной пропускной способностью. Пусть G' — сеть, полученная из G уменьшением пропускной способности обоих рёбер на $\min\{c(u \to v), c(v \to u)\}$. Иначе говоря:

- если $c(u \to v) > c(v \to u)$, измените способность $u \to v$ на $c(u \to v) - c(v \to u)$ и удалите $v \to u$;
- если $c(u \to v) < c(v \to u)$, измените способность $v \to u$ на $c(v \to u) - c(u \to v)$ и удалите $u \to v$;
- наконец, если $c(u \to v) = c(v \to u)$, удалите оба ребра.

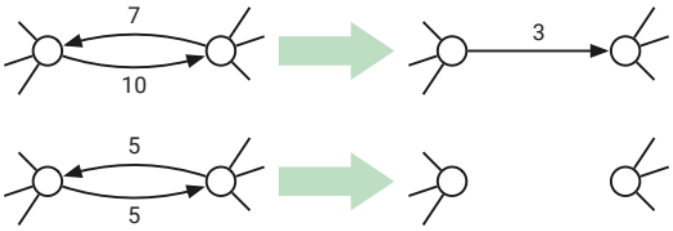


Рисунок 10.11. Обеспечение условия одного направления.

а. Докажите, что любой максимальный (s, t)-поток в G' является максимальным (s, t)-потоком и в G. Следовательно, упростив каждую пару противоположных рёбер в G, получаем приведённую сеть с той же величиной максимального потока.

б. Докажите, что любой минимальный (s, t)-разрез в G является минимальным (s, t)-разрезом в G' и наоборот.

в. Докажите, что существует хотя бы один максимальный (s, t)-поток в G, который не является максимальным (s, t)-потоком в G'.

5.

а. Опишите эффективный алгоритм проверки, содержит ли заданная транспортная сеть единственный максимальный (s, t)-поток.

б. Опишите эффективный алгоритм проверки, содержит ли заданная транспортная сеть единственный минимальный (s, t)-разрез.

в. Опишите сеть с единственным максимальным (s, t)-потоком, но не единственным минимальным (s, t)-разрезом.

г. Опишите сеть с единственным минимальным (s, t)-разрезом, но не единственным максимальным (s, t)-потоком.

6. (s, t)-поток в сети G называется ациклическим, если не существует направленных циклов, на каждом ребре которых поток положителен; иначе говоря, подграф рёбер с положительным потоком является ориентированным ациклическим графом.

а. Опишите и проанализируйте алгоритм вычисления ациклического максимального (s, t)-потока в заданной сети. Его асимптотическое время работы должно совпадать со временем алгоритма Форда — Фалкерсона.

б. Опишите и проанализируйте алгоритм проверки того, что каждый максимальный (s, t)-поток в заданной сети ацикличен.

7. Пусть $G = (V, E)$ — транспортная сеть, в которой каждое ребро имеет пропускную способность 1, а расстояние кратчайшего пути от s до t не меньше d.

а. Докажите, что величина максимального (s, t)-потока не превосходит E/d.

б. Теперь предположим, что G — простой граф: для любых вершин u и v имеется не более одного ребра из u в v. Транспортные сети могут содержать параллельные рёбра. Докажите, что величина максимального (s,t)-потока не превосходит $O(V^2/d^2)$. Подсказка: сколько вершин приходится в среднем на уровень дерева поиска в ширину с корнем s?

8. Дана транспортная сеть $G=(V,E)$ с единичной пропускной способностью каждого ребра и целое число k. Опишите и проанализируйте алгоритм поиска k рёбер G, после удаления которых величина максимального (s,t)-потока в оставшемся графе будет минимальной.

9. Оценку в доказательстве теоремы о разложении потока можно усилить. Пусть $G=(V,E)$ — произвольная транспортная сеть, а f — произвольный (s,t)-поток в G.

а. Докажите, что при $|f|=0$ поток f является взвешенной суммой не более $E-V+1$ направленных циклов, причём для каждого ребра e каждого из этих циклов $f(e)>0$.

б. Докажите, что при $|f|>0$ поток f является взвешенной суммой не более $E-V+2$ направленных путей и циклов, причём для каждого ребра e каждого из этих путей и циклов $f(e)>0$.

в. Докажите точность обеих оценок: существуют графы с циркуляциями, которые нельзя разложить менее чем на $E-V+1$ циклов, и потоками, которые нельзя разложить менее чем на $E-V+2$ путей и циклов. Подсказка: это просто.

10. ♣ Из наблюдения, что любая линейная комбинация (s,t)-потоков сама является (s,t)-потоком, следует, что множество всех не обязательно допустимых (s,t)-потоков в любом графе образует действительное векторное пространство, которое можно назвать *пространством потоков* графа.

а. Докажите, что размерность пространства потоков любого связного графа $G=(V,E)$ равна $E-V+2$.

б. Пусть T — любое остовное дерево G. Докажите, что следующий набор путей и циклов задаёт базис пространства потоков: единственный путь из s в t в T и единственный цикл в $T\cup\{e\}$ для каждого ребра $e\notin T$.

в. Пусть T — любое остовное дерево G, а F — лес, полученный удалением любого одного ребра из T. Докажите, что следующий набор путей и циклов задаёт базис пространства потоков: единственный путь из s в t в $F\cup\{e\}$ для каждого ребра $e\notin F$ с концами в разных компонентах F; единственный цикл в $F\cup\{e\}$ для каждого ребра $e\notin F$ с обоими концами в одной компоненте F.

г. Докажите или опровергните: любая связная транспортная сеть имеет базис пространства потоков, целиком состоящий из простых путей от s до t.

11. Разрезы иногда определяют как подмножества рёбер графа, а не как разбиения его вершин. В этой задаче докажите, что оба определения почти эквивалентны.

Скажем, что подмножество X направленных рёбер отделяет s от t, если каждый направленный путь из s в t содержит хотя бы одно направленное ребро из X. Для любого подмножества вершин S пусть δS обозначает множество направленных рёбер, выходящих из S:

$$\delta S := \{u \to v \mid u \in S,\ v \notin S\}.$$

а. Докажите: если $(S, T) - (s, t)$-разрез, то δS отделяет s от t.

б. Пусть X — произвольное множество рёбер, отделяющее s от t. Докажите, что существует (s, t)-разрез (S, T) такой, что $\delta S \subseteq X$.

в. Пусть X — минимальное по включению множество рёбер, отделяющее s от t; такое множество иногда называют *связкой*. Докажите, что существует (s, t)-разрез (S, T) такой, что $\delta S = X$.

12. Предположим, что вместо пропускных способностей мы рассматриваем сети, в которых каждое ребро $u \to v$ имеет неотрицательное требование $d(u \to v)$. Теперь (s, t)-поток f допустим тогда и только тогда, когда $f(u \to v) \geq d(u \to v)$ для каждого ребра $u \to v$. Величина допустимого потока теперь может быть сколь угодно большой. Естественная задача — найти допустимый (s, t)-поток наименьшей величины.

а. Опишите эффективный алгоритм вычисления допустимого (s, t)-потока по заданным графу, функции требований и вершинам s и t. Подсказка: найдите всюду ненулевой поток, затем увеличьте его масштаб, чтобы он стал допустимым.

б. Пусть доступна подпрограмма `MaxFlow`, вычисляющая максимальные потоки в сетях с пропускными способностями рёбер. Опишите эффективный алгоритм вычисления минимального потока в заданной сети с требованиями на рёбрах; он должен вызвать `MaxFlow` ровно один раз.

в. Сформулируйте и докажите аналог теоремы о максимальном потоке и минимальном разрезе для этой постановки. Соответствуют ли минимальным потокам максимальные разрезы?

13. Для любой транспортной сети G и любых вершин u и v пусть $\text{bottleneck}_G(u, v)$ обозначает максимум по всем путям π из u в v в G от минимальной пропускной способности ребра на π.

а. Опишите и проанализируйте алгоритм вычисления $\text{bottleneck}_G(s, t)$ за $O(E \log V)$ времени. Это количество потока, которое алгоритм Эдмондса — Карпа с самыми широкими увеличивающими путями пропускает на первой итерации.

б. Теперь предположим, что сеть G неориентирована или, что то же самое, $c(u \to v) = c(v \to u)$ для каждой пары вершин u, v. Опишите и проанализируйте алгоритм вычисления $\text{bottleneck}_G(s, t)$ за $O(V + E)$ времени. Подсказка: найдите медианную пропускную способность ребра. Почему это ускорение не работает для ориентированных графов?

в. ♥ Снова предположим, что сеть G неориентирована. Опишите и проанализируйте алгоритм построения остовного дерева T графа G, такого что $\text{bottleneck}_T(u, v) = \text{bottleneck}_G(u, v)$

для всех вершин u, v. Рёбра в T наследуют пропускные способности из G. Для полного балла алгоритм должен работать за $O(E)$ времени.

14. Даны транспортная сеть G с целочисленными пропускными способностями рёбер и целочисленный максимальный поток f^* в G. Опишите алгоритмы следующих операций:

а. `Increment(e)`: увеличить пропускную способность ребра e на 1 и обновить максимальный поток.

б. `Decrement(e)`: уменьшить пропускную способность ребра e на 1 и обновить максимальный поток.

Оба алгоритма должны изменять f^* так, чтобы он по-прежнему оставался максимальным потоком, быстрее, чем при повторном вычислении с нуля.

15. Пусть G — сеть с целочисленными пропускными способностями рёбер. Ребро G назовём *верхне-связывающим*, если увеличение его пропускной способности на 1 также увеличивает величину максимального потока в G. Аналогично ребро *нижне-связывающее*, если уменьшение его пропускной способности на 1 также уменьшает величину максимального потока в G.

а. Есть ли в каждой сети G хотя бы одно верхне-связывающее ребро? Докажите правильность ответа.

б. Есть ли в каждой сети G хотя бы одно нижне-связывающее ребро? Докажите правильность ответа.

в. Опишите алгоритм нахождения всех верхне-связывающих рёбер G по заданным G и максимальному потоку в G за $O(E)$ времени.

г. Опишите алгоритм нахождения всех нижне-связывающих рёбер G по заданным G и максимальному потоку в G за $O(EV)$ времени.

16. В транспортной сети G может быть более одного минимального (s, t)-разреза. Назовём *лучшим минимальным (s, t)-разрезом* любой минимальный разрез (S, T) с наименьшим числом рёбер, переходящих из S в T.

а. Опишите эффективный алгоритм нахождения лучшего минимального (s, t)-разреза при целочисленных пропускных способностях.

б. Опишите эффективный алгоритм нахождения лучшего минимального (s, t)-разреза при произвольных пропускных способностях рёбер.

в. Опишите эффективный алгоритм проверки, содержит ли заданная сеть единственный лучший минимальный (s, t)-разрез.

17. Новый преподаватель, впервые читающий курс о максимальных потоках, предлагает следующую жадную модификацию общего алгоритма увеличивающих путей Форда — Фалкерсона. Вместо поддержки остаточного графа просто[10-5] уменьшим пропускные

способности рёбер вдоль увеличивающего пути! В частности, всякий раз, когда ребро насыщается, просто удалим его из графа. Кому вообще нужна эта ерунда с остаточным графом?

```
GreedyFlow(G, c, s, t):
    для каждого ребра e в G
        f(e) ← 0
    пока существует путь из s в t
        π ← произвольный путь из s в t
        F ← минимальная пропускная способность ребра на π
        для каждого ребра e на π
            f(e) ← f(e) + F
            если c(e) = F
                удалить e из G
            иначе
                c(e) ← c(e) – F
    вернуть f
```

а. Покажите, что `GreedyFlow` не всегда вычисляет максимальный поток.

б. Покажите, что `GreedyFlow` даже не гарантирует хорошего приближения к максимальному потоку. А именно: для любой константы $\alpha > 1$ существует сеть G, величина максимального потока в которой более чем в α раз превышает величину потока, вычисленного `GreedyFlow`. Подсказка: предположите, что `GreedyFlow` на каждой итерации выбирает наихудший возможный путь π.

> ***Примечание 10-5.*** *Наречие «просто» почти всегда является подсознательным сокращением фразы «мне лень разбираться в деталях, но вы всё равно должны мне верить», или, ещё короче, «вероятно, это неверно». См. также слова «лишь», «несложно», «ясно» и «очевидно».*

18. В 1980 году Морис Кейранн опубликовал пример транспортной сети, показанный ниже, в которой эвристика Эдмондса и Карпа «самый широкий путь» не останавливается. Как и в плохом примере Цвика для исходного алгоритма Форда — Фалкерсона, φ обозначает обратное золотое сечение $(\sqrt{5}-1)/2$. Три вертикальных ребра играют по существу ту же роль, что горизонтальные рёбра в примере Цвика.

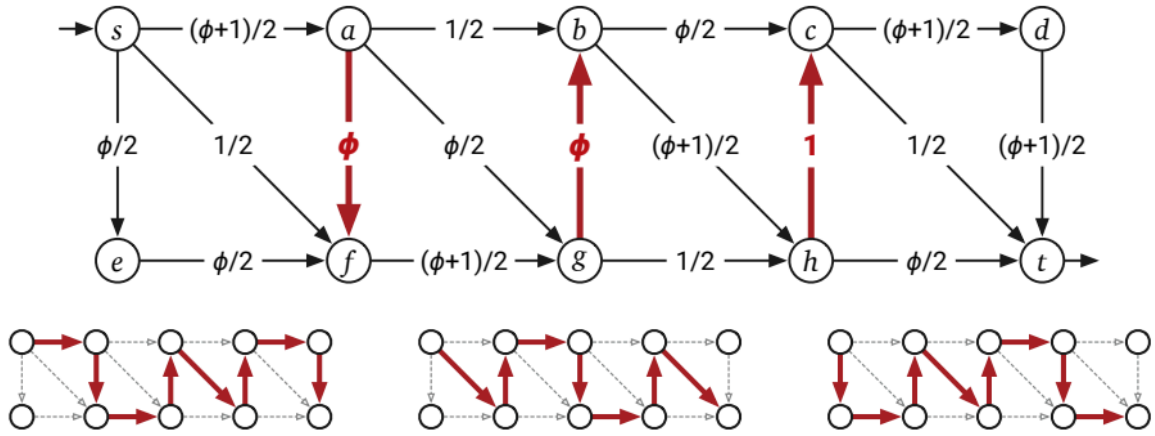


Рисунок 10.12. Сеть Кейранна и последовательность увеличений вдоль самых широких путей.

а. Покажите, что следующая бесконечная последовательность увеличений вдоль путей является допустимым выполнением алгоритма Эдмондса — Карпа «самый широкий путь». См. нижнюю часть рисунка 10.12.

```
QueyranneFatPaths:
    для i ← 1 до ∞
        пропустить ϕ^(3i−2) единиц потока по s→a→f→g→b→h→c→d→t
        пропустить ϕ^(3i−1) единиц потока по s→f→a→b→g→h→c→t
        пропустить ϕ^(3i) единиц потока по s→e→f→a→g→b→c→h→t
```

б. Опишите последовательность из $O(1)$ увеличений вдоль путей, которая даёт максимальный поток в сети Кейранна.

19. ♥ (s,t)-*последовательно-параллельный граф* — ориентированный ациклический граф с выделенными вершинами s и t, имеющий одну из следующих структур:

- **Базовый случай:** одно направленное ребро из s в t.
- **Последовательное соединение:** объединение (s,u)-последовательно-параллельного и (u,t)-последовательно-параллельного графов с общей вершиной u, но без других общих вершин или рёбер.
- **Параллельное соединение:** объединение двух меньших (s,t)-последовательно-параллельных графов с одинаковыми источником s и стоком t, но без других общих вершин или рёбер.

Любой (s,t)-последовательно-параллельный граф G можно представить *деревом разложения*: двоичным деревом с тремя типами узлов — листьями, соответствующими рёбрам G; последовательными узлами, соответствующими вершинам, отличным от s и t; и параллельными узлами. Один и тот же граф может представляться множеством разных деревьев разложения.

а. Дан ориентированный граф G с двумя выделенными вершинами s и t. Опишите и проанализируйте алгоритм, который либо строит дерево разложения для G, либо правильно сообщает, что G не является (s,t)-последовательно-параллельным. Подсказка: стройте дерево снизу вверх.

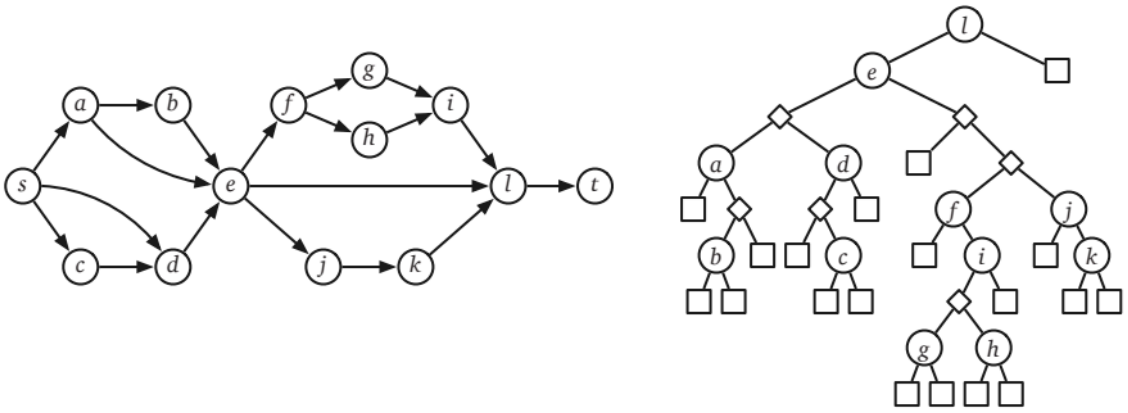


Рисунок 10.13. Последовательно-параллельный граф и соответствующее дерево разложения. Квадраты в дереве разложения — листья, ромбы — параллельные узлы.

б. Опишите и проанализируйте алгоритм вычисления максимального (s, t)-потока в заданной (s, t)-последовательно-параллельной транспортной сети с произвольными пропускными способностями рёбер. Подсказка: с учётом пункта а можно считать дерево разложения заданным. Сначала вычислите величину максимального потока, затем сам максимальный поток.

20. Алгоритм Эдмондса — Карпа «самый широкий путь» можно ускорить, по крайней мере для сетей с небольшими целочисленными пропускными способностями, ослабив требования к следующему увеличивающему пути. Вместо пути с максимальной пропускной способностью узкого места найдём путь, у которого эта величина составляет хотя бы половину максимальной, с помощью следующего алгоритма масштабирования пропускных способностей. Этот алгоритм действительно был предложен Эдмондсом и Карпом.

Предположим, что все пропускные способности рёбер — положительные целые числа, меньшие $U = 2^k$ для некоторого целого k. Алгоритм поддерживает порог узкого места Δ, первоначально равный U. В каждой фазе алгоритм увеличивает поток вдоль путей из s в t, в которых каждое ребро имеет остаточную пропускную способность не меньше Δ. Когда такого пути нет, фаза заканчивается, мы присваиваем $\Delta \leftarrow \lfloor \Delta/2 \rfloor$ и начинаем следующую фазу. Алгоритм заканчивается при $\Delta = 0$.

а. Сколько фаз выполнит алгоритм в худшем случае?

б. Пусть f — поток в конце фазы для некоторого значения Δ. Докажите, что пропускная способность минимального разреза в остаточном графе G_f не превосходит $E \cdot \Delta$.

в. Докажите, что в каждой фазе алгоритма происходит не более $2E$ увеличений.

г. Каково общее время работы алгоритма масштабирования пропускных способностей?

Глава 11. Применения потоков и разрезов

Долгое время меня озадачивало, как нечто настолько дорогое и передовое может быть настолько бесполезным. А потом я понял, что компьютер — глупая машина, способная делать невероятно умные вещи, а программисты — умные люди, способные делать невероятно глупые вещи. Короче говоря, они идеально подходят друг другу.

— Билл Брайсон, Notes from a Big Country (1999)

Вскоре после падения «железного занавеса» в 1990 году встретились американец и русский, оба работавшие над разработкой оружия. Американец спросил: «Когда вы создавали бомбу, как вам удалось выполнить столько вычислений на ваших слабых компьютерах?» Русский ответил: «Мы использовали более совершенные алгоритмы».

— Ефим Диниц, «Алгоритм Диница: исходная версия и версия Эвена» (2006)

Я слышал, теперь я плохой парень, потому что не плыву по течению. Никогда не плывите по течению — будьте течением.

— Jay-Z, Stream of Consciousness, 16 мая 2015 года

11.1. Пути без общих рёбер

Одно из простейших применений максимальных потоков — вычисление максимального числа путей без общих рёбер между двумя заданными вершинами s и t ориентированного графа G. Пути в G не имеют общих рёбер, если каждое ребро G встречается не более чем в одном из них; однако несколько таких путей могут проходить через одну и ту же вершину.

Если каждому ребру назначить пропускную способность 1, максимальный поток от s к t пропустит по каждому ребру либо 0, либо 1 единицу потока. Из теоремы о разложении потока следует, что подграф S насыщенных рёбер является объединением нескольких путей и циклов без общих рёбер. Более того, число путей в этом разложении в точности равно величине потока. Извлечь сами пути из S просто: пройдите по любому ориентированному пути в S от s к t, удалите этот путь из S и повторите действие рекурсивно.

И наоборот, любой набор из k путей без общих рёбер можно преобразовать в поток, пропустив единицу потока по каждому пути от s к t; величина полученного потока в точности равна k. Следовательно, любой алгоритм максимального потока фактически вычисляет наибольший возможный набор путей без общих рёбер.

Если вычислять максимальный поток алгоритмом Орлина, пути без общих рёбер можно найти за O(VE) времени, однако алгоритм Орлина для этого простого применения избыточен.

Пропускная способность разреза ({s}, V \ {s}) не превышает V − 1, поэтому и величина максимального потока не превышает V − 1. Таким образом, уже исходный алгоритм Форда — Фалкерсона с дополняющими путями работает за O(|f*| E) = O(VE) времени.

Этот же алгоритм можно использовать для поиска путей без общих рёбер в неориентированных графах. Сначала заменим каждое неориентированное ребро uv в G парой ориентированных рёбер u → v и v → u, каждое с единичной пропускной способностью, и назовём получившийся граф G′. Затем вычислим максимальный поток f* от s к t в G′ алгоритмом Форда — Фалкерсона. Если для ребра uv графа G поток f* насыщает оба ориентированных ребра u → v и v → u в G′, мы можем удалить оба ребра из потока, не меняя его величины. В более общем случае можно найти ациклический максимальный поток в G′, сократив все циклы f*, а не только циклы длины 2. Поэтому без ограничения общности f* задаёт единственное направление каждому насыщенному ребру. Наконец, пути без общих рёбер можно извлечь поиском в подграфе ориентированных рёбер, насыщенных f*.

11.2. Пропускные способности вершин и пути без общих вершин

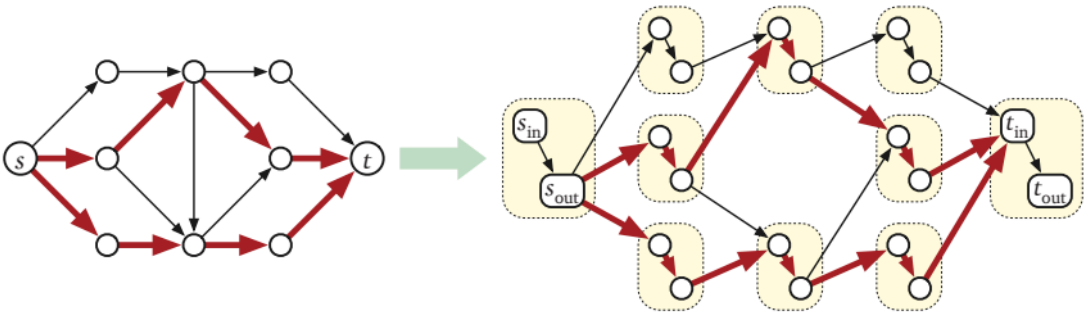
Теперь предположим, что пропускные способности заданы не только рёбрам, но и вершинам входного графа G. В дополнение к остальным ограничениям для каждой вершины v, кроме, возможно, s и t, потребуем, чтобы суммарный поток, входящий в v, а значит, и выходящий из v, не превышал некоторого неотрицательного значения c(v):

$$\sum_{u \to v} f(u \to v) \le c(v).$$

Можно ли по-прежнему вычислять максимальные потоки с этими новыми ограничениями на вершины?

В 1962 году Форд и Фалкерсон предложили следующее сведение к потоковой сети $\bar{G}$, в которой пропускные способности есть только у рёбер. Заменим каждую вершину v двумя вершинами v_in и v_out, соединёнными ребром v_in → v_out с пропускной способностью c(v), а затем заменим каждое ориентированное ребро u → v ребром u_out → v_in, сохранив прежнюю пропускную способность. Непосредственная проверка определений показывает, что каждому допустимому потоку от s_out к t_in в $\bar{G}$ соответствует допустимый поток от s к t той же величины в исходном графе G, и наоборот. В частности, каждому максимальному потоку в $\bar{G}$ соответствует максимальный поток в G. Сведение G к $\bar{G}$ занимает O(E) времени; после этого можно вычислить максимальный поток в $\bar{G}$ алгоритмом Орлина. Таким образом, вычисление максимального потока в G требует O(VE) времени.

Теперь легко вычислить максимальное число путей без общих вершин от s к t за O(VE) времени: назначьте каждой вершине пропускную способность 1 и вычислите максимальный поток!



11.3. Паросочетания в двудольном графе

Другое естественное применение максимальных потоков — поиск максимальных паросочетаний в двудольных графах. **Паросочетание** — подграф, в котором степень каждой вершины не превышает единицы, или, равносильно, набор рёбер, никакие два из которых не имеют общей вершины. Требуется найти паросочетание с наибольшим числом рёбер.

Например, предположим, что есть врачи, ищущие работу, и больницы, ищущие врачей. Каждый врач перечисляет больницы, в которых согласен работать, а каждая больница — врачей, которых согласна нанять. Наша задача — найти наибольшее множество назначений врачей в больницы, приемлемых для обеих сторон.[1] Эта задача равносильна поиску максимального паросочетания в двудольном графе, вершины которого — врачи и больницы, а ребро соединяет врача с больницей тогда и только тогда, когда они взаимно приемлемы.

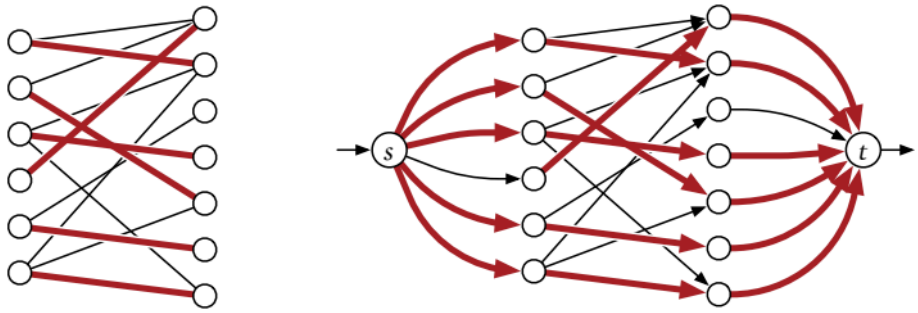
Задачу можно решить сведением к максимальному потоку. Пусть G — заданный двудольный граф с множеством вершин L ∪ R, причём каждое ребро соединяет вершину L с вершиной R. Построим новый ориентированный граф G′: (1) направим каждое ребро из L в R, (2) добавим новый исток s с рёбрами к каждой вершине L, (3) добавим новый сток t с рёбрами от каждой вершины R. Наконец, назначим каждому ребру G′ пропускную способность 1.

Любое паросочетание M в G можно преобразовать в поток f_M в G′: для каждого ребра uw из M направим единицу потока по пути s → u → w → t. Эти пути не пересекаются, кроме вершин s и t, поэтому получившийся поток удовлетворяет ограничениям пропускной способности. Более того, его величина равна числу рёбер M.

Наоборот, рассмотрим любой поток f от s к t в G′, вычисленный алгоритмом Форда — Фалкерсона с дополняющими путями. Поскольку пропускные способности рёбер целочисленны, алгоритм назначает каждому ребру целочисленный поток. Это легко проверить индукцией — подсказка, подсказка. Более того, поскольку каждое ребро имеет единичную пропускную способность, вычисленный поток либо насыщает каждое ребро G′, f(e) = 1, либо избегает его, f(e) = 0. Наконец, поскольку в каждую вершину L может входить не более единицы потока, а из каждой вершины R — выходить не более единицы, насыщенные рёбра из L в R образуют паросочетание в G. Размер этого паросочетания в точности равен |f|.

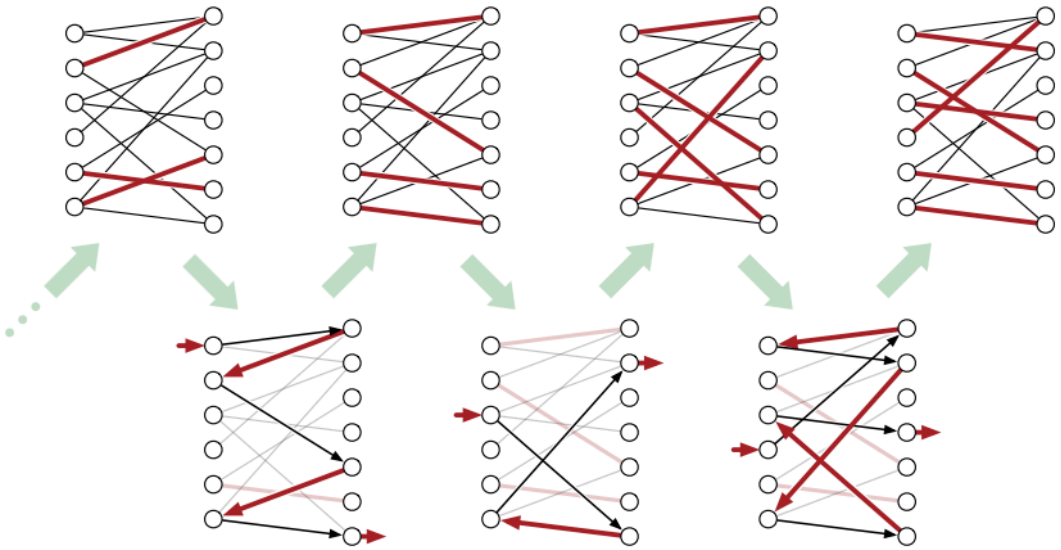
Следовательно, размер максимального паросочетания G равен величине максимального потока G′. Если максимальный поток вычислен с помощью дополняющих путей, сам поток можно преобразовать в максимальное паросочетание за O(E) времени. Максимальный поток можно

вычислить за O(VE) времени как алгоритмом Орлина, так и обычным алгоритмом Форда — Фалкерсона.



Полезно понять поведение алгоритма Форда — Фалкерсона в G′ через исходный двудольный граф G. Алгоритм поддерживает паросочетание M в G, изначально пустое; рёбрам M соответствуют рёбра G′, несущие поток. Назовём вершину G **сочетанной**, если она является концом некоторого ребра M, и **несочетанной** в противном случае. На каждой итерации мы ищем в G **чередующийся путь**: путь от несочетанной вершины L до несочетанной вершины R, в котором чередуются рёбра M и рёбра вне M. Такие пути в G в точности соответствуют дополняющим путям для текущего потока в G′. Найдя дополняющий путь P, обновляем M, заменяя его симметрической разностью $M \oplus P$; число рёбер M при этом увеличивается на 1. Затем переходим к следующей итерации.

Если чередующегося пути нет, теорема о максимальном потоке и минимальном разрезе гарантирует, что M — максимальное паросочетание, и алгоритм завершается. Поиск одного чередующегося пути требует O(E) времени, а алгоритм останавливается не более чем после V итераций, поэтому общее время равно O(VE). Работа алгоритма показана на рис. 11.3.



Эту характеристику максимальных паросочетаний двудольных графов через чередующиеся пути доказал Клод Берж в 1957 году независимо от теоремы о максимальном потоке и

минимальном разрезе, хотя она уже неявно содержалась в алгоритмах Гарольда Куна 1955 года, Денеша Кёнига 1916 года и Карла Якоби примерно 1836 года.

Более сложный алгоритм, предложенный Джоном Хопкрофтом и Ричардом Карпом в 1973 году, вычисляет максимальное паросочетание в двудольном графе всего за $O(\sqrt{V}\ E)$ времени, находя на каждой итерации несколько непересекающихся чередующихся путей.

[1] Эта задача значительно отличается от задачи устойчивого паросочетания из главы 4: мы не пытаемся сделать каждого врача и каждую больницу максимально довольными.

11.4. Выбор кортежей

Задача о максимальном паросочетании в двудольном графе — простейший пример более широкого класса задач, которые я называю **выбором кортежей**.[2] На вход такой задачи подаются несколько конечных множеств X_1, X_2, ..., X_d, каждое из которых представляет отдельный дискретный ресурс. Требуется выбрать как можно больше d-кортежей, каждый из которых содержит ровно один элемент каждого множества X_i, при соблюдении нескольких ограничений следующих видов:

- Для каждого индекса i каждый элемент $x \in X_i$ может встречаться не более чем в $c(x)$ выбранных кортежах.
- Для каждого индекса i любые два элемента $x \in X_i$ и $y \in X_{i+1}$ могут вместе встречаться не более чем в $c(x, y)$ выбранных кортежах.

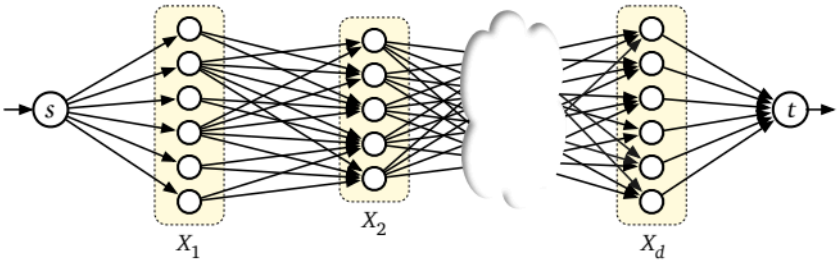
Каждая верхняя граница $c(x)$ и $c(x, y)$ является неотрицательным целым числом, обычно небольшим, либо ∞.

В задаче о максимальном паросочетании имеется $d = 2$ ресурса; каждый элемент x имеет пропускную способность $c(x) = 1$, а каждая пара (x, y) имеет пропускную способность $c(x, y) = 1$ или $c(x, y) = 0$ в зависимости от того, является ли xy ребром исходного двудольного графа.

Поскольку ресурсы упорядочены линейно и ограничения наложены только на пары объектов из соседних подмножеств X_i и X_{i+1},[3] задачу выбора кортежей можно свести к задаче максимального потока в ориентированном графе G, определённом следующим образом:

- G содержит вершину для каждого элемента каждого множества X_i, а также исток s и сток t. Каждая вершина x, кроме s и t, имеет пропускную способность $c(x)$.
- G содержит ребро $s \to w$ для каждого $w \in X_1$, ребро $z \to t$ для каждого $z \in X_d$ и ребро $x \to y$ с пропускной способностью $c(x, y)$ для каждой пары $x \in X_i$ и $y \in X_{i+1}$ при всех i. При желании можно опустить рёбра $x \to y$, для которых $c(x, y) = 0$.

Каждому пути от s к t в G соответствует d-кортеж, который можно выбрать; и наоборот, каждому выбираемому d-кортежу, удовлетворяющему заданным ограничениям, соответствует путь от s к t в G.



В более общем случае пусть f — произвольный допустимый целочисленный поток от s к t в G. Поскольку все пропускные способности являются целыми числами либо ∞, из теоремы о разложении потока следует, что f представляет собой сумму |f| путей от s к t, каждый из которых несёт ровно единицу потока. Непосредственная проверка определений показывает, что получившееся множество кортежей удовлетворяет всем ограничениям. И наоборот, для любого множества из k кортежей, удовлетворяющего ограничениям, сумма k соответствующих путей является допустимым целочисленным потоком от s к t величины k.

Следовательно, можно выбрать максимальное число кортежей, удовлетворяющих ограничениям, вычислив максимальный поток f* от s к t в G, а затем разложив его. Поскольку все конечные пропускные способности в G целочисленны, без ограничения общности f* можно считать целочисленным потоком и, по предыдущему абзацу, сопоставить ему допустимое множество из |f*| кортежей.

[2] Я не нашёл стандартного названия для этих задач и поэтому придумал собственное. Иногда их называют «задачами о назначениях», но чаще «задачей о назначениях» называют поиск паросочетания максимального веса во взвешенном двудольном графе.

[3] Если ограничения наложены и на пары объектов хотя бы из одной пары несоседних подмножеств X_i и X_j, где j > i + 1, задача становится NP-трудной, что следует из прямого сведения от Exact-3DimensionalMatching. NP-трудность рассматривается в следующей главе.

Расписание экзаменов

Следующая «реальная» задача составления расписания, возможно, поможет прояснить наше общее сведение.

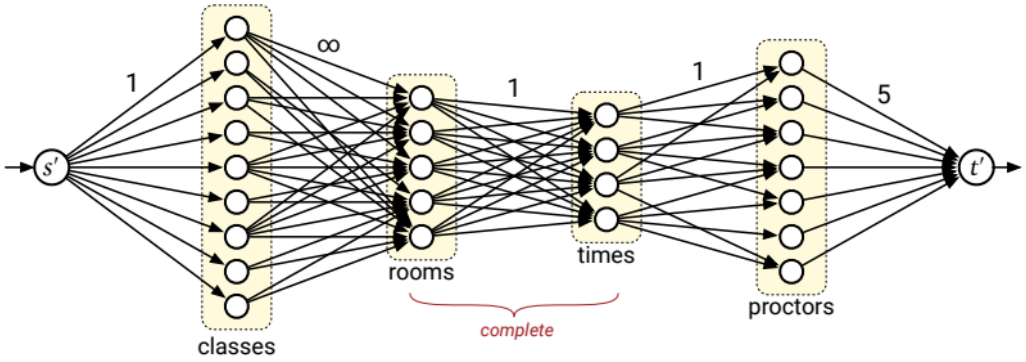
Университет Шам-Пубанана нанял вас написать алгоритм составления расписания выпускных экзаменов. Есть n различных курсов; каждому необходимо назначить экзамен в одной из r аудиторий и в один из t временных интервалов. В каждой аудитории на каждый интервал можно назначить экзамен не более чем одного курса; и наоборот, экзамен курса нельзя разбить между несколькими аудиториями или моментами времени. Кроме того, за каждым экзаменом должен следить один из p наблюдателей.[4] Каждый наблюдатель может следить не более чем за одним экзаменом одновременно; каждый доступен лишь в определённые интервалы времени; ни одному наблюдателю нельзя назначить больше пяти экзаменов в общей сложности. Вход задачи составляют три массива:

- Целочисленный массив E[1 .. n], где E[i] — число студентов на i-м курсе.

- Целочисленный массив S[1 .. r], где S[j] — число мест в j-й аудитории. Экзамен i-го курса можно провести в j-й аудитории тогда и только тогда, когда E[i] ≤ S[j].
- Булев массив A[1 .. t, 1 .. p], где A[k, ℓ] = True тогда и только тогда, когда ℓ-й наблюдатель доступен в k-й временной интервал.[5]

Пусть N = n + r + tp — общий размер входа. Требуется разработать алгоритм, который либо назначит аудиторию, временной интервал и наблюдателя для каждого экзамена, либо правильно сообщит, что такое расписание невозможно.

Это стандартная задача выбора кортежей с четырьмя ресурсами: курсами, аудиториями, временными интервалами и наблюдателями. Для её решения построим потоковую сеть G с шестью видами вершин: исток s′, вершина c_i для каждого курса, вершина r_j для каждой аудитории, вершина t_k для каждого временного интервала, вершина p_ ℓ для каждого наблюдателя и сток t′; а также с пятью видами рёбер, как показано на рис. 11.5.



- Ребро s′ → c_i с пропускной способностью 1 для каждого курса i: «Для каждого курса можно провести не более одного итогового экзамена».
- Ребро c_i → r_j с пропускной способностью ∞ для каждого курса i и аудитории j, таких что E[i] ≤ S[j]: «Экзамен курса i можно провести в аудитории j тогда и только тогда, когда в аудитории достаточно мест». Только здесь используются числа студентов E[i] и мест S[j].
- Ребро r_j → t_k с пропускной способностью 1 для каждой аудитории j и интервала k: «В аудитории j в момент k можно провести не более одного экзамена».
- Ребро t_k → p_ℓ с пропускной способностью 1 для интервала k и наблюдателя ℓ, таких что A[ℓ, k] = True: «Наблюдатель может следить не более чем за одним экзаменом в любой момент и только тогда, когда доступен».
- Ребро p_ℓ → t′ с пропускной способностью 5 для каждого наблюдателя ℓ: «Каждый наблюдатель может следить не более чем за пятью экзаменами».

Примечание переводчика. При определении входа автор использует A[k, ℓ] для доступности наблюдателя ℓ в интервал k. В пункте о ребре выше индексы в оригинале переставлены; для согласованности с определением нужно использовать A[k, ℓ].

Я обозначаю исток и сток s′ и t′ вместо s и t только потому, что в условии переменная t уже используется для числа временных интервалов. Всего в G имеется n + r + t + p + 2 = O(N) вершин и O(nr + rt + tp) = O(N²) рёбер.

Каждый путь от s' к t' в G представляет единственный допустимый выбор курса, аудитории, времени и наблюдателя для одного экзамена: в частности, курс помещается в аудиторию, а наблюдатель доступен в это время. И наоборот, каждому допустимому выбору соответствует путь от s' к t'. Следовательно, можно составить корректное расписание для максимального числа экзаменов, вычислив максимальный поток f^* от s' к t' в G, разложив f^* на пути от s' к t', а затем преобразовав каждый путь в назначение «курс — аудитория — время — наблюдатель». Если $|f^*| = n$, возвращаем полученное расписание; иначе можем правильно сообщить, что назначить все n экзаменов невозможно.

Построение G из заданных входных данных полным перебором занимает $O(E)$ времени. Максимальный поток вычисляется за $O(VE)$ времени алгоритмом Форда — Фалкерсона, поскольку $|f^*| \le n < V$, или алгоритмом Орлина; разложение потока также занимает $O(VE)$. Таким образом, общее время алгоритма равно $O(VE) = O(N^3)$.

[4] За пределами США таких наблюдателей чаще называют *invigilators*.

[5] Возможно, эти данные лучше представить в виде графа, но мне показалось, что это сделает сведение более запутанным.

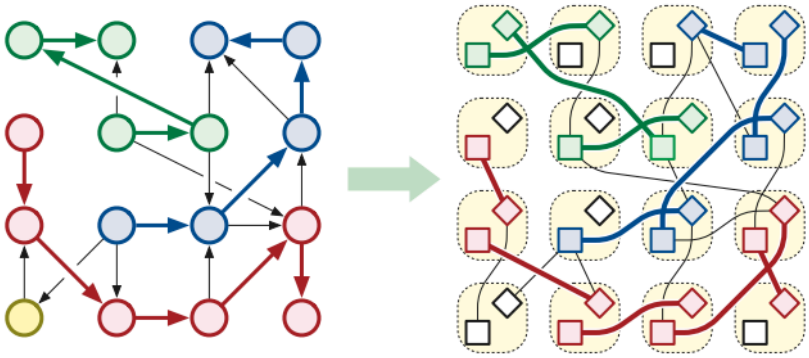
11.5. Покрытия непересекающимися путями

Покрытие путями ориентированного графа G — набор ориентированных путей в G, такой что каждая вершина G лежит хотя бы на одном пути. **Покрытие непересекающимися путями** — покрытие, в котором каждая вершина G лежит ровно на одном пути. Любой ориентированный граф имеет тривиальное покрытие непересекающимися путями из нескольких путей нулевой длины, но это скучно. Вместо этого будем искать покрытие, содержащее как можно меньше путей. В общем случае задача NP-трудна: граф имеет такое покрытие размера 1 тогда и только тогда, когда содержит гамильтонов путь. Однако для ориентированных ациклических графов существует эффективный алгоритм на основе потоков.

Чтобы решить задачу для заданного DAG $G = (V, E)$, построим новый двудольный граф $G' = (V', E')$:

- G' содержит две вершины $v♭$ и $v♯$ для каждой вершины v графа G.
- G' содержит неориентированное ребро $u♭v♯$ для каждого ориентированного ребра $u \to v$ в G.

Если G задан матрицей смежности, то G' — двудольный граф, задаваемый той же матрицей!



Теперь утверждается, что G можно покрыть k непересекающимися путями тогда и только тогда, когда новый граф G′ имеет паросочетание размера V − k. Как обычно, докажем эквивалентность в двух направлениях.

Пусть G имеет покрытие P из k непересекающихся путей; будем рассматривать P как подграф G. Полустепень захода каждой вершины P равна 0 или 1; кроме того, в каждом пути P ровно одна вершина имеет полустепень захода 0. Следовательно, P содержит ровно V − k рёбер. Определим подмножество рёбер G′:

$$M := \{u^\flat v^\sharp \in E' \mid u \to v \in P\}.$$

По определению покрытия непересекающимися путями у каждой вершины G есть не более одного входящего и не более одного исходящего ребра в P. Следовательно, каждая вершина G′ инцидентна не более чем одному ребру M; то есть M — паросочетание размера V − k.

И наоборот, пусть G′ имеет паросочетание M размера V − k. Спроецируем M обратно на G, определив подграф P = (V, M′), где

$$M' := \{u \to v \in E \mid u^\flat v^\sharp \in M\}.$$

По определению паросочетания у каждой вершины G есть не более одного входящего и не более одного исходящего ребра в P. Отсюда P состоит из непересекающихся ориентированных путей в G; поскольку P содержит каждую вершину, оно определяет покрытие непересекающимися путями с V − k рёбрами. Число путей в P равно числу вершин G, не имеющих входящего ребра в M′. Следовательно, P содержит ровно k путей.

Отсюда непосредственно следует, что минимальное покрытие G непересекающимися путями можно найти, вычислив максимальное паросочетание в G′ с помощью алгоритма максимального потока Форда — Фалкерсона за O(V′E′) = O(VE) времени.

Несмотря на формулировку в терминах DAG и путей, на самом деле это задача о максимальном паросочетании: мы хотим сопоставить как можно большему числу вершин различные последующие вершины графа. Число путей, необходимых для покрытия DAG, равно числу вершин без последователя. И, конечно, любая задача о максимальном паросочетании в двудольном графе на самом деле является задачей о потоке.

Минимальный штат преподавателей

Вернёмся в университет Шам-Пубанана ради ещё одной «реальной» задачи составления расписания.[6] Каждый день там проходят несколько тысяч занятий. Из-за крайнего сокращения бюджета университет должен значительно уменьшить штат преподавателей. Однако, поскольку студенты платят за обучение, а университет не может позволить себе адвокатов, необходимо сохранить достаточно преподавателей, чтобы гарантировать проведение каждого занятия, объявленного в каталоге курсов. Каким минимальным числом преподавателей может обойтись университет? Каждому оставшемуся преподавателю назначается последовательность занятий на день. Занятия одного преподавателя не должны пересекаться; кроме того, в расписании должно хватать времени, чтобы перейти от одного занятия к следующему. Для этой задачи предположим, что любой преподаватель способен вести любое занятие и что у преподавателей не будет часов консультаций, обеда или перерывов в туалет.[7]

Конкретнее, пусть есть n занятий в m различных местах. Вход задачи состоит из следующих данных:

- Массив занятий C[1 .. n], где у каждого занятия C[i] три поля: время начала C[i].start, время окончания C[i].end и место проведения C[i].loc.
- Двумерный массив T[1 .. m, 1 .. m], где T[u, v] — время, необходимое для перехода из места u в место v.

Нужно найти минимальное число преподавателей, способных вместе провести все занятия так, чтобы всякий раз, когда одному преподавателю назначены два занятия i и j и C[j].start ≥ C[i].start, выполнялось

$$C[j].\text{start} \geq C[i].\text{end} + T[C[i].\text{loc}, C[j].\text{loc}].$$

Задачу можно свести к покрытию непересекающимися путями. Построим ориентированный ациклический граф G = (V, E), вершины которого соответствуют занятиям, а рёбра — парам занятий, достаточно разнесённых по времени, чтобы их мог вести один преподаватель. В частности, ориентированное ребро i → j означает, что один преподаватель может провести сначала занятие i, а затем j. Этот DAG легко построить полным перебором за $O(n^2)$ времени. Затем найдём покрытие G непересекающимися путями с помощью описанного выше алгоритма паросочетания; каждый ориентированный путь G представляет допустимое расписание одного преподавателя. Весь алгоритм работает за $O(n^2 + VE) = O(n^3)$ времени.[8]

Несмотря на первоначальное описание через интервалы и расстояния, на самом деле это задача о максимальном паросочетании, а значит, в действительности задача о максимальном потоке. Мы хотим сопоставить как можно большему числу занятий следующее занятие того же преподавателя. Необходимое число преподавателей равно числу занятий без назначенного последующего занятия: каждое из них является последним занятием некоторого преподавателя.

[6] Более реалистичный и менее удручающий вариант этой задачи использует самолёты и рейсы или автобусы и автобусные маршруты вместо преподавателей и занятий.

[7] Тем не менее предполагается, что они будут отвечать на электронные письма студентов по пути между занятиями.

[8] Если считать, что все интервалы перехода T[u, v] одинаковы,[9] эту задачу можно решить за O(n log n) времени простым жадным алгоритмом.

[9] Многие американские университеты устанавливают десятиминутные перерывы между занятиями, удивительным образом считая, что за десять минут человек может дойти от любой аудитории до любой другой на том же кампусе. Приглашаю каждого, кто считает это реалистичным, посетить мой кампус и пройти от одного Siebel Center до другого.

11.6. Выбывание из бейсбольного турнира

Каждый год миллионы американских болельщиков следят за любимой бейсбольной командой в надежде, что она выйдет в плей-офф и в конечном счёте попадёт в Мировую серию. К сожалению, большинство команд «математически выбывает» за несколько дней или даже недель до конца регулярного сезона. Часто это легко заметить: команда уже не может выиграть достаточно матчей, чтобы догнать лидера своего дивизиона. Но иногда ситуация сложнее. Вот реальная турнирная таблица Восточного дивизиона Американской лиги на 30 августа 1996 года.

Команда	Победы–поражения	Осталось	NYY	BAL	BOS	TOR	DET
New York Yankees	75–59	28	—	3	8	7	3
Baltimore Orioles	71–63	28	3	—	2	7	4
Boston Red Sox	69–66	27	8	2	—	0	0
Toronto Blue Jays	63–72	27	7	7	0	—	0
Detroit Tigers	49–86	27	3	4	0	0	—

«Детройт» явно отстаёт, но самые преданные болельщики Tigers могут ещё надеяться на победу. Ведь если команда выиграет все 27 оставшихся матчей, она закончит сезон с 76 победами — больше, чем сейчас у любой другой команды. Так что достаточно, чтобы все остальные проиграли все матчи… но это невозможно, поскольку некоторым ещё предстоит играть друг с другом. Вот полное рассуждение.[10]

> *Выиграв все оставшиеся матчи, «Детройт» может закончить сезон с результатом 76–86. Если Yankees выиграют хотя бы два матча, их результат составит 77–85 и они опередят «Детройт». Поэтому предположим, что Tigers больше не проигрывают, а Yankees больше не побеждают. Но у Нью-Йорка*

осталось восемь игр с Бостоном. Если Red Sox выиграют их все, у них будет не менее 77 побед и они опередят Tigers. Значит, единственный шанс «Детройта» занять первое место — чтобы Нью-Йорк выиграл ровно одну из восьми игр с Бостоном и проиграл все остальные. При этом Red Sox должны проиграть все игры с другими командами. Тогда три команды разделят первое место.

Посмотрим теперь, что произойдёт с Orioles и Blue Jays при этом сценарии. У Балтимора осталось две игры с Бостоном и три с Нью-Йорком. Если всё произойдёт так, как описано выше, у Orioles будет не менее 76 побед. Значит, «Детройт» сможет догнать Балтимор, только если Orioles проиграют все игры командам, кроме Нью-Йорка и Бостона. В частности, Балтимор должен проиграть все семь оставшихся игр с Торонто. У Blue Jays осталось и семь игр с Yankees, и мы уже знаем, что для первого места «Детройта» Торонто должен выиграть все эти игры. Но тогда Blue Jays одержат не менее 14 новых побед и закончат с результатом 77–85 или лучше, опередив Tigers. Поэтому, что бы ни происходило дальше, «Детройт» не сможет занять первое место в Восточном дивизионе Американской лиги.

Должен быть более простой способ это выяснить!

Абстрактно задача формулируется так. На входе даны массивы W[1 .. n] и G[1 .. n, 1 .. n], где W[i] — число уже выигранных командой i матчей, а G[i, j] — число предстоящих игр между командами i и j. Нужно определить, может ли команда n закончить сезон с наибольшим числом побед, возможно разделив первое место с другими командами.[11]

В середине 1960-х Бенджамин Шварц заметил, что вопрос можно представить как задачу о максимальном потоке; примерно двадцать лет спустя Дэн Гасфилд, Чарльз Мартел и Давид Фернандес-Бака упростили эту формулировку до задачи выбора из пар. А именно, мы хотим узнать, можно ли назначить победителя каждой игры так, чтобы команда n заняла первое место. Пусть $R[i] = \sum_j G[i, j]$ — число оставшихся игр команды i. Будем считать, что команда n выигрывает все R[n] оставшихся матчей. Тогда она сможет занять первое место тогда и только тогда, когда каждая другая команда i выиграет не более $W[n] + R[n] - W[i]$ из своих R[i] оставшихся игр.

Поскольку мы выбираем победителя для каждой игры, начнём с построения двудольного графа, вершины которого обозначают игры и команды. Для каждой пары $1 \le i < j < n$ создаём игровую вершину $g_{i,j}$, а для каждого $1 \le i < n$ — вершину команды t_i. Для каждой пары i, j добавляем рёбра $g_{i,j} \to t_i$ и $g_{i,j} \to t_j$ бесконечной пропускной способности. Добавляем источник s и рёбра $s \to g_{i,j}$ с пропускной способностью G[i, j]. Наконец, добавляем сток t и рёбра $t_i \to t$ с пропускной способностью $W[n] - W[i] + R[n]$. Если эта величина для какой-либо команды отрицательна, команда n уже выбыла: даже выиграв все оставшиеся игры, она не догонит эту команду.

На рисунке 11.7 показан граф, построенный по таблице Восточного дивизиона 1996 года, где «команда n» — Detroit Tigers. Все рёбра без указанной пропускной способности имеют бесконечную ёмкость.

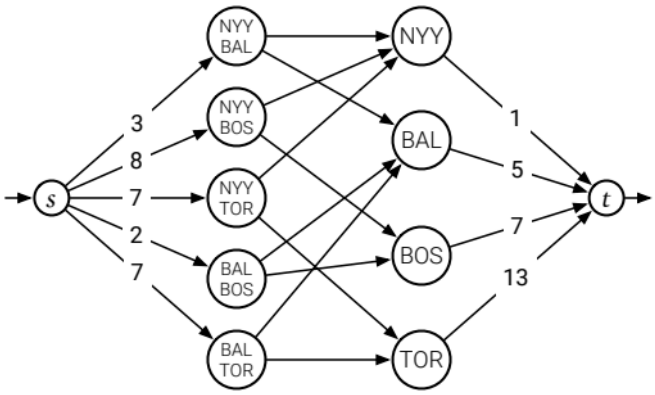


Рис. 11.7. Cubs побеждают! Cubs побеждают!

Теорема. Команда n может закончить сезон на первом месте тогда и только тогда, когда в этом графе существует допустимый поток, насыщающий каждое ребро, выходящее из s.

Доказательство. Предположим, команда n может закончить сезон на первом месте. Тогда каждая команда i < n выигрывает не более W[n] + R[n] – W[i] оставшихся игр. Для каждой победы команды i над командой j добавим единицу потока вдоль пути $s \to g_{i,j} \to t_i \to t$. Поскольку команды i и j играют ровно G[i, j] раз, каждое ребро из s насыщено. Поскольку каждая команда i выигрывает не более W[n] + R[n] – W[i] игр, полученный поток допустим.

Обратно, пусть f — целочисленный допустимый поток, насыщающий все рёбра из s. Такой поток существует всякий раз, когда существует насыщающий поток: все пропускные способности целочисленны, поэтому можно выбрать целочисленный максимальный поток. Предположим, что команда i выигрывает ровно $f(g_{i,j} \to t_i)$ игр против команды j для каждой пары i, j. Тогда команды i и j играют $f(g_{i,j} \to t_i) + f(g_{i,j} \to t_j) = f(s \to g_{i,j}) = G[i, j]$ матчей, поэтому все предстоящие игры сыграны. Более того, каждая команда i выигрывает в сумме $\sum_j f(g_{i,j} \to t_i) = f(t_i \to t) \le W[n] + R[n] - W[i]$ предстоящих игр, то есть не более W[n] + R[n] игр за сезон. Следовательно, если команда n выиграет все свои будущие игры, она закончит сезон на первом месте. □

Итак, чтобы определить, может ли наша любимая команда победить, строим сеть, вычисляем максимальный поток и проверяем, насыщает ли он каждое ребро из s. Например, в графе на рисунке 11.7 общая ёмкость рёбер из s равна 27, поскольку осталось 27 игр. Но общая ёмкость рёбер в t равна лишь 26, поэтому значение максимального потока не превышает 26. Значит, «Детройт» математически выбыл.[12]

Сеть имеет $O(n^2)$ вершин и $O(n^2)$ рёбер; построить её можно за $O(n^2)$. Алгоритм Орлина вычисляет максимальный поток за $O(VE) = O(n^4)$. Это не самый быстрый алгоритм для задачи о выбывании: в 2001 году Кевин Уэйн доказал, что все математически выбывшие команды можно определить всего за $O(n^3)$, по существу вычислив один максимальный поток.

[10] Пример и рассуждение взяты с сайта Эли Олиника https://s2.smu.edu/~olinick/riot/detroit.html и основаны на его совместных исследованиях с Иланом Адлером, Аланом Эрерой и Дорит Хохбаум.

[11] Мы неявно предполагаем, что ничьих не бывает и каждая игра действительно состоится. Оба предположения согласуются с правилами Главной бейсбольной лиги, по крайней мере для матчей, влияющих на выход в плей-офф, если не случатся войны, стихийные бедствия или нашествия пчёл.

[12] Здесь нам повезло или не повезло: команда может выбыть и тогда, когда суммарная ёмкость всех рёбер в t не меньше суммарной ёмкости рёбер из s.

11.7. Выбор проектов

В последнем примере предположим, что нам дано множество из n проектов, которые мы могли бы выполнить. Некоторые проекты нельзя начинать, пока не завершены определённые другие. Проекты и зависимости описаны ориентированным ациклическим графом G: его вершины — проекты, а ребро u → v означает, что проект u нельзя выполнить до проекта v. Это именно такой граф зависимостей, который мы рассматривали в разделе 6.4. Наконец, с каждым проектом v связана прибыль $(v), получаемая при его завершении; для некоторых проектов прибыль отрицательна, что мы считаем положительными затратами. Мы можем выбрать для выполнения любое подмножество X проектов, содержащее все их зависимости: если x ∈ X, любой проект, от которого зависит x, тоже принадлежит X. Цель — найти допустимое подмножество с наибольшей суммарной прибылью. В частности, если прибыль от всех работ отрицательна, правильный ответ — не делать ничего.

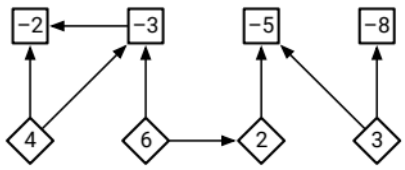


Рис. 11.8. Граф зависимостей восьми проектов. Ромбами обозначены прибыльные проекты, квадратами — затратные. Ребро u → v означает, что u зависит от v.

На высоком уровне нужно разбить проекты на два подмножества S и T: работы, которые мы выбираем (Select), и работы, от которых отказываемся (Turn down). Интуитивно хотелось бы свести задачу к минимальному разрезу некоторого графа. Но какого? Как обеспечить выполнение предварительных условий? Мы хотим максимизировать прибыль, а умеем находить только минимальные разрезы. И как превратить отрицательную прибыль в положительную пропускную способность?

Чтобы преобразовать заданный граф ограничений G в сеть G′, добавим источник s и сток t, ребро s → v для каждой прибыльной работы v с $(v) > 0 и ребро u → t для каждой затратной работы u с $(u) < 0. Интуитивно можно представить s как новую работу «Спать!» с нулевой прибылью и затратами, которую необходимо выполнить последней. Назначим рёбрам G′ следующие пропускные способности:

- c(s → v) = $(v) для каждой прибыльной работы v;
- c(u → t) = −$(u) для каждой затратной работы u;
- c(u → v) = ∞ для каждого ребра зависимости u → v.

Все пропускные способности положительны, поэтому это допустимый вход задачи о минимальном разрезе.

Рассмотрим произвольный (s, t)-разрез (S, T) в G′. Для любого ребра u → v исходного графа зависимостей, если u ∈ S и v ∈ T, то ёмкость разреза ‖S, T‖ = ∞. Следовательно, работы из S можно выбрать допустимым образом тогда и только тогда, когда ёмкость разреза конечна.

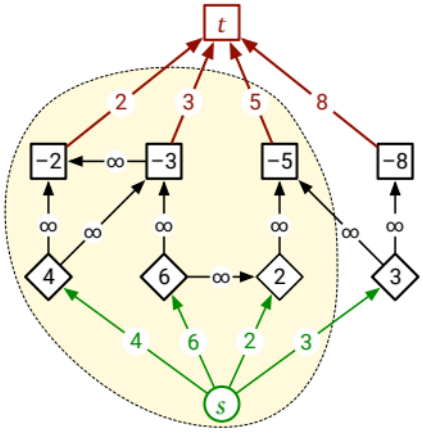


Рис. 11.9. Сеть для примера с графом зависимостей и её минимальный разрез. Ёмкость разреза равна 13, а P = 15, поэтому суммарная прибыль выбранных работ равна 2.

Оказывается, разрезам меньшей ёмкости соответствуют наборы работ с большей прибылью. Точнее, утверждается, что выбор работ из S приносит прибыль P − ‖S, T‖, где P — сумма всех положительных прибылей:

$$P = \sum_{v} \max\{0, \$(v)\} = \sum_{\$(v) > 0} \$(v).$$

Это утверждение доказывается прямой подстановкой определений. Для любого подмножества X проектов определим три величины. Как обычно, c(u → v) = 0, если u → v не является ребром.

$$\text{cost}(X) := \sum_{u \in X,\, \$(u) < 0} -\$(u) = \sum_{u \in X} c(u \to t).$$

$$\text{yield}(X) := \sum_{v \in X,\, \$(v) > 0} \$(v) = \sum_{v \in X} c(s \to v).$$

$$\text{profit}(X) := \sum_{v \in X} \$(v) = \text{yield}(X) - \text{cost}(X).$$

По определению P = yield(V) = yield(S) + yield(T). Поскольку разрез (S, T) имеет конечную ёмкость, его могут пересекать только рёбра вида s → v и u → t. По построению каждое ребро s → v ведёт к прибыльной работе, а каждое u → t выходит из затратной. Поэтому ‖S, T‖ = cost(S) + yield(T). Сразу получаем P − ‖S, T‖ = yield(S) − cost(S) = profit(S), что и требовалось.

Следовательно, максимальную прибыль можно получить, вычислив минимальный разрез в G′. Граф G′ легко построить по G за O(V + E), а минимальный (s, t)-разрез в G′ можно вычислить за O(VE) алгоритмом Орлина. Таким образом, весь алгоритм выбора проектов работает за O(VE).

Упражнения

1. Пусть G = (V, E) — ориентированный граф, в котором входящая и исходящая степени каждой вершины равны. Предположим, в G есть k путей из вершины u в вершину v, не имеющих общих рёбер. Обязательно ли при этих условиях существуют k путей из v в u, не имеющих общих рёбер? Приведите доказательство или контрпример с объяснением.

2. Дан неориентированный граф G = (V, E) с тремя вершинами u, v и w. Опишите и проанализируйте алгоритм, определяющий, существует ли путь из u в w, проходящий через v. Подсказка: если бы G был ориентированным графом, эта задача была бы NP-трудной!

3. Рассмотрим ориентированный граф G = (V, E) с несколькими источниками $s_1, s_2, \ldots, s_\sigma$ и стоками $t_1, t_2, \ldots, t_\tau$; ни одна вершина не является одновременно источником и стоком. Многотерминальный поток — функция $f: E \to \mathbb{R}_{\geq 0}$, удовлетворяющая условию сохранения потока в каждой вершине, не являющейся ни источником, ни стоком. Значение $|f|$ многотерминального потока — суммарный избыток потока, выходящего из всех источников:

$$|f| := \sum_{i=1}^{\sigma} \left(\sum_{w} f(s_i \to w) - \sum_{u} f(u \to s_i) \right).$$

   Как обычно, нас интересует поток максимального значения с учётом ограничений пропускной способности рёбер. В частности, нас не интересует, сколько потока переходит из конкретного источника в конкретный сток.

   (a) Рассмотрим следующий алгоритм вычисления многотерминальных потоков. Переменные f и f′ представляют функции потока. Подпрограмма `MaxFlow(G, s, t)` решает обычную задачу максимального потока с источником s и стоком t.

```
MaxMultiFlow(G, s[1 .. σ], t[1 .. τ]):
    f ← 0                       ⟨⟨Инициализировать поток⟩⟩
    для i ← 1 до σ
        для j ← 1 до τ
            f′ ← MaxFlow(G_f, s[i], t[j])
            f ← f + f′          ⟨⟨Обновить поток⟩⟩
    вернуть f
```

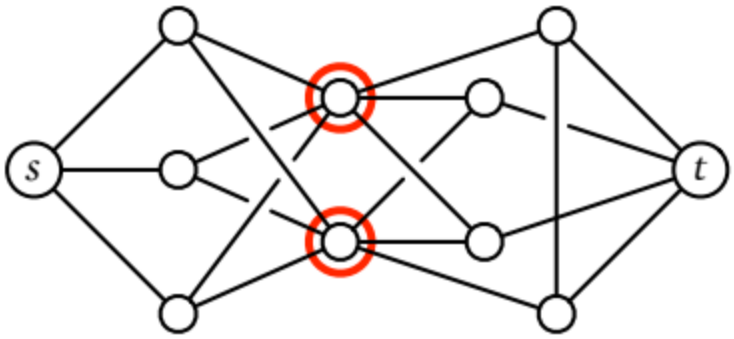
Докажите, что этот алгоритм правильно вычисляет максимальный многотерминальный поток в G.

(b) Опишите более эффективный алгоритм вычисления максимального многотерминального потока в G.

4. На острове Содор много городов и деревень, соединённых обширной железнодорожной сетью. Недавно в деревне Скарлуи обнаружили несколько случаев смертельно опасной заразной болезни: то ли свиного гриппа, то ли зомби; сообщения противоречивы. Начальник железной дороги Содора планирует закрыть некоторые станции, чтобы болезнь не распространилась в его родной город Тидмут. Через закрытую станцию поезда

не проходят. Чтобы сократить расходы и не привлекать внимания, он хочет закрыть как можно меньше станций. Однако он не может закрыть станцию Скарлуи, поскольку тогда подвергнется опасности заражения, и не может закрыть станцию Тидмут, поскольку тогда не сможет посещать любимый паб.

Опишите и проанализируйте алгоритм поиска минимального числа станций, которые нужно закрыть, чтобы прекратить любое железнодорожное сообщение от Скарлуи до Тидмута. Сеть представлена неориентированным графом: каждая станция — вершина, каждое железнодорожное соединение — ребро. Две специальные вершины s и t обозначают станции Скарлуи и Тидмут. Например, для следующего графа алгоритм должен вернуть 2.



5. Решётка n × n — неориентированный граф с n^2 вершинами, расположенными в n строках и n столбцах. Вершина в строке i и столбце j обозначается (i, j). Каждая вершина (i, j) имеет ровно четырёх соседей: (i − 1, j), (i + 1, j), (i, j − 1) и (i, j + 1), за исключением граничных вершин, для которых i = 1, i = n, j = 1 или j = n.

   Пусть (x_1, y_1), (x_2, y_2), …, (x_m, y_m) — различные вершины решётки, называемые терминалами. Задача побега состоит в том, чтобы определить, существуют ли m путей без общих вершин, соединяющих терминалы с m различными граничными вершинами.

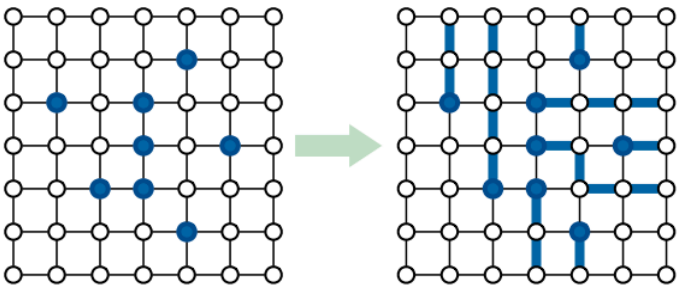


Рис. 11.10. Положительный пример задачи побега и его решение.

(a) Опишите и проанализируйте эффективный алгоритм решения задачи побега. Его время работы должно быть полиномом небольшой степени от n.

(b) Теперь предположим, что на вход подаются только целое число n и список из m терминалов. Если m очень мало, предыдущее время работы на самом деле

экспоненциально относительно размера входа! Опишите и проанализируйте алгоритм с временем работы, полиномиальным от m.

(c) ♥ Измените предыдущий алгоритм так, чтобы он явно выдавал пути побега, если они существуют, по-прежнему за время, полиномиальное от m.

6. Кафедра пригородной тишины SPU устраивает поле для мини-гольфа в подвале центра See-Bull! Игровое поле — замкнутый многоугольник, ограниченный m горизонтальными и вертикальными отрезками, сходящимися под прямыми углами. На поле есть n стартовых точек и n лунок, между которыми задано взаимно однозначное соответствие. Из каждой стартовой точки можно ударить по мячу прямо к соответствующей лунке по прямой, не касаясь границы поля. Отбивать мячи от стен игрокам нельзя: слишком много стекла. Все n стартовых точек и n лунок расположены в разных местах.

   К сожалению, компьютер архитектора вышел из строя как раз перед началом строительства. Благодаря героическим усилиям системных администраторов удалось восстановить координаты стартовых точек и лунок, но сведения о соответствии между ними потерялись!

   Опишите и проанализируйте алгоритм, находящий взаимно однозначное соответствие между стартовыми точками и лунками, удовлетворяющее условию прямолинейного удара, либо сообщающий, что такого соответствия нет. На входе даны координаты x и y всех m углов поля, n стартовых точек и n лунок. Считайте, что по координатам концов двух отрезков можно за постоянное время определить, пересекаются ли они.

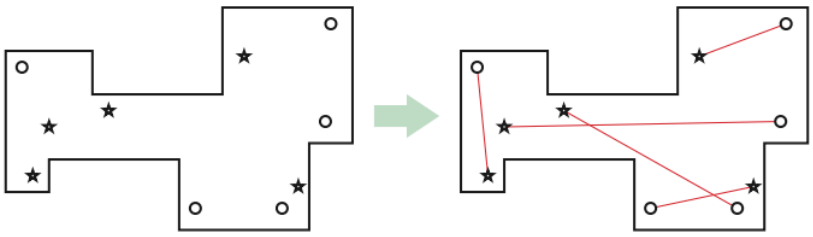


Рис. 11.11. Поле для мини-гольфа с пятью стартовыми точками и лунками и допустимое соответствие между ними.

7. Циклическое покрытие ориентированного графа G = (V, E) — множество циклов без общих вершин, покрывающих все вершины G. Опишите и проанализируйте эффективный алгоритм поиска циклического покрытия заданного графа или правильного сообщения, что его нет. Подсказка: используйте двудольное паросочетание!

8. Дана шахматная доска n × n, из которой удалены некоторые клетки, и большой набор костяшек домино размером ровно в две клетки. Опишите и проанализируйте алгоритм, определяющий, можно ли замостить доску домино: каждая костяшка должна покрывать ровно две неудалённые клетки, а каждая неудалённая клетка — быть покрыта ровно одной костяшкой.

   На входе дан булев массив Deleted[1 .. n, 1 .. n], где Deleted[i, j] = True тогда и только тогда, когда клетка в строке i и столбце j удалена. Выход — одно булево значение; вычислять

само расположение домино не требуется. Например, для доски на рисунке 11.12 алгоритм должен вернуть True.

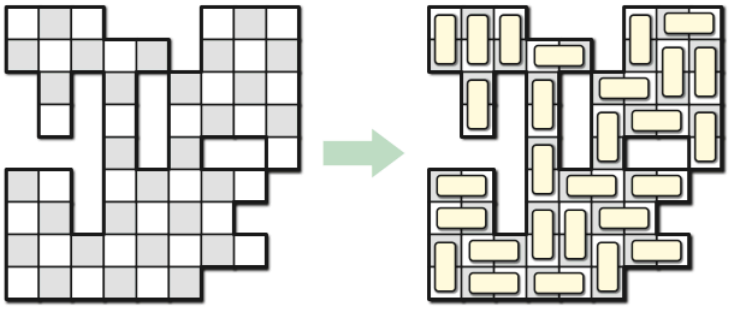


Рис. 11.12. Покрытие частичной шахматной доски костяшками домино.

9. Дана квадратная решётка n × n, часть клеток которой чёрные, а остальные белые. Опишите и проанализируйте алгоритм, определяющий, можно ли расположить фишки так, чтобы:

   - каждая фишка находилась на белой клетке;
   - в каждой строке была ровно одна фишка;
   - в каждом столбце была ровно одна фишка.

   На входе дан двумерный булев массив IsWhite[1 .. n, 1 .. n], указывающий белые клетки. Выход — одно булево значение. Например, для решётки на рисунке 11.13 алгоритм должен вернуть True.

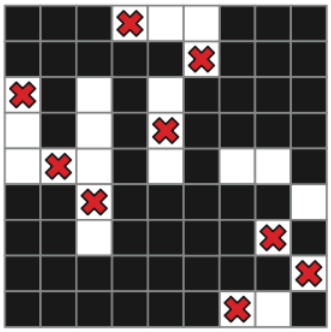


Рис. 11.13. Отметки в каждой строке и каждом столбце решётки.

10. Дан набор коробок, каждая из которых задана высотой, шириной и глубиной в сантиметрах. Все три размера каждой коробки строго больше 10 см и строго меньше 20 см. Как и следовало ожидать, одну коробку можно поместить в другую, если первую можно повернуть так, чтобы её высота, ширина и глубина были соответственно меньше размеров второй. Коробки можно вкладывать друг в друга рекурсивно. Коробка называется видимой, если не находится внутри другой коробки. Опишите и проанализируйте алгоритм вложения коробок, минимизирующий число видимых коробок.

11. Дана решётка n × n, некоторые клетки которой отмечены. Она представлена булевым массивом M[1 .. n, 1 .. n], где M[i, j] = True тогда и только тогда, когда клетка (i, j) отмечена. Монотонный путь по решётке начинается в верхней левой клетке, на каждом шаге движется только вправо или вниз и заканчивается в нижней правой клетке. Наша цель — покрыть отмеченные клетки как можно меньшим числом монотонных путей.

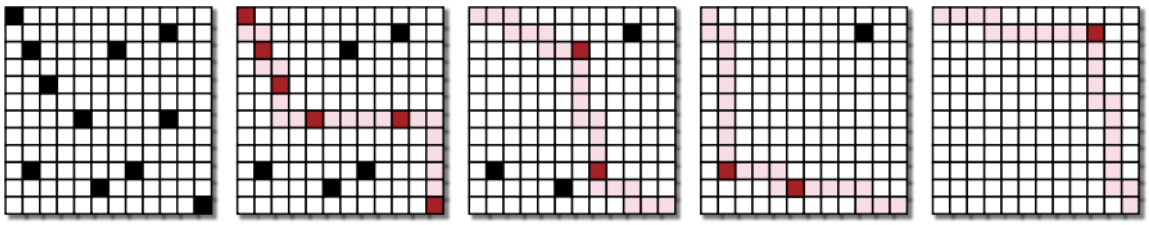


Рис. 11.14. Жадное покрытие отмеченных клеток четырьмя монотонными путями.

(a) Опишите алгоритм поиска монотонного пути, покрывающего наибольшее число отмеченных клеток.

(b) Есть естественная жадная эвристика поиска небольшого покрытия: если отмеченные клетки остались, найти монотонный путь π, покрывающий их максимальное число, снять отметки со всех покрытых им клеток и повторить. Покажите, что этот алгоритм не всегда находит оптимальное решение.

(c) Опишите и проанализируйте эффективный алгоритм вычисления наименьшего множества монотонных путей, покрывающих все отмеченные клетки.

12. Учёный совет университета Шам-Пубанана решил созвать комитет, чтобы определить, кто должен заменить недавно опозоренного барона Фактотума в качестве официального талисмана спортивных команд SPU «Боевые Пу-ба»: дядя Габби, профессор Бобо Корнелиус или горилла-экстрасенс Мофо. От каждой кафедры в комитет необходимо выбрать ровно одного преподавателя. Некоторые преподаватели работают на нескольких кафедрах, но каждый член комитета может представлять только одну. Например, если профессор Благоевич состоит на кафедрах коррупции и глупости и выбран представителем кафедры глупости, коррупцию должен представлять кто-то другой. Кроме того, правила университета требуют одинакового числа ассистент-профессоров, ассоциированных профессоров и полных профессоров в каждом преподавательском комитете. К счастью, число кафедр кратно трём.

Опишите и проанализируйте алгоритм выбора преподавателей SPU для комитета по выбору обезьяньего талисмана после Фактотума либо корректного сообщения о невозможности сформировать такой комитет. На входе дан двудольный граф, показывающий принадлежность преподавателей к кафедрам; каждая вершина преподавателя помечена его должностью: ассистент-профессор, ассоциированный или полный профессор.

13. На кафедре пригородной тишины университета Шам-Пубанана действует гибкая учебная программа со сложным набором требований для выпуска. Кафедра предлагает n различных курсов; имеется m требований. Каждое требование задаёт подмножество из n

курсов и количество курсов, которое необходимо пройти из этого подмножества. Подмножества разных требований могут пересекаться, но каждый курс можно использовать для выполнения не более одного требования.

Например, пусть n = 5 курсов A, B, C, D, E и m = 2 требования:

- пройти не менее двух курсов из {A, B, C};
- пройти не менее двух курсов из {C, D, E}.

Тогда студент, прошедший только B, C, D, не может выпуститься, а студент, прошедший A, B, C, D или B, C, D, E, может.

Опишите и проанализируйте алгоритм, определяющий, может ли данный студент выпуститься. На входе — список m требований, каждое из которых задаёт подмножество n курсов и необходимое количество курсов из него, а также список пройденных студентом курсов.

14. Вы организуете первый ежегодный 72-часовой танцевальный обмен кафедры пригородной тишины SPU, который продлится всю пятницу, субботу и воскресенье. Во время мероприятия прозвучит несколько получасовых музыкальных сетов; на выступление подало заявки много диджеев. Нанять диджеев нужно при следующих ограничениях:

- Каждый день должно прозвучать ровно k сетов, всего 3k.
- Каждый сет играет один диджей в одном музыкальном жанре: эмбиент, баблгам, дабстеп, хорроркор, K-pop, квайто, мариачи, традиционный джаз, трип-хоп, нэшвиллское кантри, парапара, ска и так далее.
- Каждый жанр может звучать не более одного раза в день.
- Каждый кандидат предоставил список жанров, которые готов играть.
- Каждый диджей может сыграть не более трёх сетов за всё мероприятие.

Пусть имеется n кандидатов в диджеи и g музыкальных жанров. Опишите и проанализируйте эффективный алгоритм, который назначает диджея и жанр каждому из 3k сетов или правильно сообщает, что такое назначение невозможно.

15. Предположим, вы управляете сайтом, который каждый день посещает один и тот же набор людей. Каждый посетитель относит себя к одной или нескольким демографическим группам: например, мужчина 40–50 лет, отец, житель Иллинойса, учёный, блогер и поклонник Гилберта и Салливана.[13] Сайт финансируется рекламодателями. Каждый рекламодатель указал, каким демографическим группам следует показывать рекламу и сколько показов в день необходимо. Всего имеется n посетителей, k демографических групп и m рекламодателей.

Опишите эффективный алгоритм, который по всем этим данным определяет, можно ли показывать каждому посетителю ровно одно объявление в день так, чтобы каждый рекламодатель получил желаемое число показов и каждое объявление увидел человек из подходящей демографической группы.

[13] Я очень хороший теоретик информатики, а именно специалист по геометрическим алгоритмам.

16. Дан массив A[1 .. m][1 .. n] неотрицательных действительных чисел. Требуется округлить A до целочисленной матрицы, заменив каждый элемент x на ⌊x⌋ либо ⌈x⌉, не изменяя суммы элементов ни в одной строке и ни в одном столбце. Например:

$$\begin{bmatrix} 1.2 & 3.4 & 2.4 \\ 3.9 & 4.0 & 2.1 \\ 7.9 & 1.6 & 0.5 \end{bmatrix} \to \begin{bmatrix} 1 & 4 & 2 \\ 4 & 4 & 2 \\ 8 & 1 & 1 \end{bmatrix}.$$

(a) Опишите и проанализируйте эффективный алгоритм, который либо выполняет такое округление, либо правильно сообщает, что оно невозможно.

(b) Докажите, что допустимое округление возможно тогда и только тогда, когда суммы элементов каждой строки и каждого столбца являются целыми числами. Иными словами, докажите, что алгоритм из пункта (a) либо возвращает допустимое округление, либо его невозможность очевидна.

(c) ♥ Предположим, гарантируется, что ни один элемент входной матрицы A не является целым числом. Опишите и проанализируйте ещё более быстрый алгоритм, который округляет A или правильно сообщает, что округление невозможно. Для полного зачёта его время работы должно быть $O(mn)$. Подсказка: не используйте потоки.

17. Одноранговые сети состоят из маломощных беспроводных устройств. В принципе[14] такие сети можно использовать на поле боя, в районах недавних стихийных бедствий и других труднодоступных местах. Идея состоит в том, чтобы распределить большое число дешёвых простых устройств по нужной территории, например сбросив их с самолёта; затем устройства автоматически настроят работающую беспроводную сеть.

Устройства могут связываться только на ограниченном расстоянии. Предположим, все устройства одинаковы и существует расстояние D такое, что два устройства могут связываться тогда и только тогда, когда расстояние между ними не превышает D.

Нам нужна надёжная сеть, но дешёвые маломощные устройства часто выходят из строя. Если устройство обнаружит, что скоро откажет, оно должно передать свои данные другому резервному устройству в пределах связи. Требуется, чтобы каждое устройство x имело k потенциальных резервных устройств на расстоянии не более D; эти k устройств называются резервным множеством x. Кроме того, нежелательно, чтобы какое-либо устройство входило в резервные множества слишком многих других: иначе один отказ может повлиять на большую часть сети.

Пусть даны радиус связи D, параметры b и k и массив расстояний d[1 .. n, 1 .. n], где d[i, j] — расстояние между устройствами i и j. Опишите алгоритм, который либо вычисляет резервное множество размера k для каждого из n устройств так, что ни одно устройство не входит более чем в b резервных множеств, либо правильно сообщает, что такого набора множеств не существует.

[14] Но на практике не так уж часто.

18. Столкнувшись с угрозой крайне жёсткого сокращения бюджета, Потёмкинский университет решил нанять актёров, которые будут сидеть на занятиях в роли «студентов», чтобы каждое предлагаемое занятие было полностью заполнено. Поскольку актёры дороги, университет хочет нанять как можно меньше людей.

Опираясь на прежний опыт управления ныне закрытым университетом Шам-Пубанана, администраторы Потёмкинского университета предоставили вам ориентированный ациклический граф $G = (V, E)$, вершины которого обозначают занятия, а ребро $i \to j$ означает, что один «студент» может посетить занятие i, а затем занятие j. Кроме того, дан массив $cap[1 .. V]$ максимального числа «студентов» на каждом занятии. Опишите и проанализируйте алгоритм вычисления минимального числа «студентов», позволяющего полностью заполнить все занятия.

19. Квентин, Алиса и другие физики из Брейкбиллса планируют путешествие через Междумирье в Филлори. Междумирье — огромный безлюдный город, состоящий из площадей, на каждой из которых есть фонтан, способный магически переносить людей в другой мир. Соседние площади соединены воротами, проклятыми Чудовищем. Ворота открываются одновременно лишь на пять минут каждый час — с 12:00 до 12:05, затем с 1:00 до 1:05 и так далее; в остальное время они заперты. Если за эти пять минут через одни ворота пройдёт более одного человека, Чудовище заметит их присутствие.[15] Кроме того, любой, кто попытается открыть запертые ворота или пройти более чем через одни ворота за один пятиминутный интервал, превратится в ниффина.[16] Однако через разные ворота одновременно и через одни ворота в разное время может безопасно проходить любое число людей.

Дана карта Междумирья в виде графа G с вершиной для каждого фонтана и ребром для каждых ворот; фонтаны на Землю и в Филлори отмечены.

(a) Пусть также дано положительное целое число h. Опишите и проанализируйте алгоритм вычисления максимального числа людей, которые могут пройти от земного фонтана к фонтану Филлори не более чем за h часов, то есть после не более чем h открытий ворот, никого не предупредив и не превратившись в ниффина. Время работы алгоритма должно зависеть от h. Подсказка: постройте другой граф.

(b) ♣ ♥ Опишите и проанализируйте алгоритм для пункта (a), время работы которого полиномиально от V и E и не зависит от h.

(c) Пусть вместо этого дополнительно дано целое число k. Опишите и проанализируйте алгоритм вычисления минимального числа часов, достаточного для прохода k человек от земного фонтана к фонтану Филлори без предупреждения Чудовища и превращения в ниффина. Подсказка: используйте пункт (a).

[15] Это очень плохо.

[16] Это очень-очень плохо.

20. ♥ Пусть $G = (L \sqcup R, E)$ — двудольный граф, левые вершины L которого пронумерованы $\ell_1, \ell_2, \ldots, \ell_n$, а правые вершины R — $r_1, r_2, \ldots, r_n$. Паросочетание M в G называется

непересекающимся, если для каждой пары его рёбер $\ell_i r_j$ и $\ell_{i'} r_{j'}$ выполнено: $i < i'$ тогда и только тогда, когда $j < j'$.

(a) Опишите и проанализируйте алгоритм поиска наибольшего непересекающегося паросочетания в G. Подсказка: на самом деле это не задача о потоке.

(b) Опишите и проанализируйте алгоритм поиска наименьшего числа непересекающихся паросочетаний M_1, M_2, ..., M_k, таких что каждое ребро G принадлежит ровно одному из них. Подсказка: а это действительно задача о потоке.

21. ♥ Пусть $G = (L \sqcup R, E)$ — двудольный граф, левые вершины которого L пронумерованы ℓ_1, ℓ_2, ..., ℓ_n в произвольном порядке.

(a) Паросочетание M называется плотным, если в L нет двух подряд идущих несочетанных вершин: для каждого i хотя бы одна из ℓ_i и ℓ_{i+1} инцидентна ребру M. Опишите алгоритм определения, есть ли в G плотное паросочетание.

(b) Паросочетание M называется разреженным, если в L нет двух подряд идущих сочетанных вершин: для каждого i хотя бы одна из ℓ_i и ℓ_{i+1} не инцидентна ребру M. В частности, пустое паросочетание разрежено. Опишите алгоритм поиска наибольшего разреженного паросочетания в G.

(c) Паросочетание M называется палиндромным, если для каждого i вершины ℓ_i и ℓ_{n-i+1} либо обе инцидентны рёбрам M, либо обе не инцидентны. В частности, пустое паросочетание палиндромно. Опишите алгоритм поиска наибольшего палиндромного паросочетания в G.

Ни в одной из этих задач не ограничивается, какие вершины R сочетаются, а какие нет.

22. ♥ Корневое дерево — ориентированный ациклический граф, в котором у каждой вершины ровно одно входящее ребро, кроме корня, в который не входит ни одно ребро. Эквивалентно, корневое дерево состоит из корневой вершины с рёбрами к корням нуля или более меньших корневых деревьев. Опишите эффективный алгоритм вычисления по двум корневым деревьям A и B наибольшего корневого дерева, изоморфного одновременно некоторому подграфу A и некоторому подграфу B. Короче говоря, опишите алгоритм поиска наибольшего общего поддерева двух корневых деревьев.

Подсказка: это была бы сравнительно простая задача динамического программирования, если бы у каждой вершины было O(1) потомков или если бы потомки каждой вершины были упорядочены слева направо. Но для неупорядоченных деревьев с большой степенью нужна иная техника эффективного объединения рекурсивных подзадач.

Глава 12. NP-трудность

В своём кратком и отрывочном трактате он являет вечный пример — не законов и даже не метода, поскольку нет иного метода, кроме способности быть очень умным, — но самого разума, который быстро ведёт анализ ощущений к принципу и определению.

— Т. С. Элиот об Аристотеле, «Совершенный критик», The Sacred Wood (1921)

Прелесть стандартов в том, что их так много и можно выбирать; более того, если ни один вам не нравится, можно просто подождать модель следующего года.

— Эндрю С. Таненбаум, Computer Networks (1981)

Поистине редкий ум способен сделать до сих пор несуществовавшее ослепительно очевидным. Возглас «Я и сам мог бы до этого додуматься» очень популярен и обманчив: дело в том, что человек этого не сделал, и этот факт весьма значителен и показателен.

— Дирк Джентли — Ричарду Макдаффу в романе Дугласа Адамса «Детективное агентство Дирка Джентли» (1987)

Если задача не имеет решения, возможно, это не задача, а факт, с которым со временем нужно научиться жить.

— Шимон Перес, цитируется Дэвидом Рамсфелдом в Rumsfeld's Rules (2001)

12.1. Игра, в которой не выиграть

Представьте, что продавец в красном костюме, подозрительно похожий на Тома Уэйтса, предъявляет вам чёрный стальной ящик с n двоичными переключателями на передней панели и лампочкой сверху. Продавец говорит, что состоянием лампочки управляет сложная булева схема — набор элементов And, Or и Not, соединённых проводами, с одним входным проводом для каждого переключателя и единственным выходным проводом для лампочки. Затем он задаёт простой вопрос: можно ли установить переключатели так, чтобы лампочка загорелась? Если вы ответите правильно, он подарит вам сто миллиардов долларов; если ошибётесь или умрёте, не ответив, он заберёт вашу душу.



Рис. 12.1. Элемент AND, элемент OR и элемент NOT.

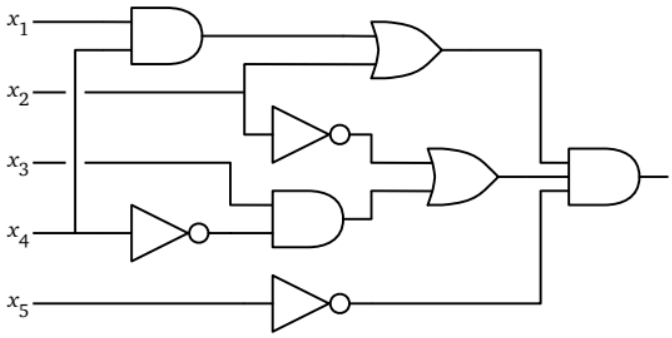


Рис. 12.2. Булева схема. Входы расположены слева, выход — справа.

Насколько вы можете судить, Противник вообще не подключил переключатели к лампочке, поэтому как бы вы их ни устанавливали, она останется выключенной. Если вы объявите, что включить лампочку можно, Противник откроет ящик и покажет, что никакой схемы внутри нет. Но если вы объявите, что включить лампочку нельзя, не проверив все 2^n положений переключателей, Противник волшебным образом создаст внутри ящика схему, которая включает лампочку тогда и только тогда, когда переключатели находятся в одном из непроверенных вами положений, а затем установит их в это положение и включит лампочку. Вы не сможете обнаружить обман, поскольку до конца игры не видите содержимого ящика. Единственный способ гарантированно правильно ответить на вопрос Противника — попробовать все 2^n возможных положений. Вы быстро понимаете, что это займёт гораздо больше времени, чем вам осталось жить, и вежливо отклоняете предложение.

Противник улыбается и рычит голосом Джокера Хита Леджера, выкурившего блок «Мальборо»: «Ах да, конечно, у вас нет причин мне доверять. Но, возможно, я смогу вас успокоить». Он протягивает вам большой свиток пергамента — вы надеетесь, что он сделан из овечьей кожи, — на котором нарисована или, возможно, вытатуирована принципиальная схема. «Вот полные чертежи схемы внутри ящика. Можете заглянуть внутрь и проверить соответствие чертежей. Или собрать собственную схему по этим чертежам. Или написать компьютерную программу для моделирования схемы. Как пожелаете. Если выяснится, что чертежи не соответствуют схеме в ящике, вы выиграете сто миллиардов долларов». Несколько выборочных проверок убеждают вас, что явных недостатков в чертежах нет; скрытый обман кажется невозможным.

Но вам всё равно следует отклонить «щедрое» предложение Противника. Задача, которую он ставит, называется **выполнимостью булевой схемы**, или CircuitSat: дана булева схема; существует ли набор входов, при котором схема выдаёт True, или же она всегда выдаёт False? Для любого конкретного набора входов мы можем вычислить выход за полиномиальное, точнее линейное, время с помощью поиска в глубину. Но никто не знает, как решить CircuitSat быстрее полного перебора всех 2^n возможных входов схемы, требующего экспоненциального времени. Надо признать, никто не доказал строго, что полный перебор нельзя улучшить: возможно, просто ещё не найден какой-то хитрый алгоритм. Но ведь никто строго не доказал и отсутствие антигравитационных единорогов. Для практических целей можно считать, что быстрого алгоритма для CircuitSat нет.

Вы отвечаете продавцу отказом. Он улыбается и говорит: «Ты умнее, чем выглядишь, малыш», — а затем улетает на своём антигравитационном единороге.

12.2. P против NP

Минимальное требование, позволяющее считать алгоритм «эффективным», — ограничение времени работы полиномиальной функцией размера входа: $O(n^c)$ для некоторой константы c, где n — размер входа.[1] Исследователи рано поняли, что не все задачи можно решить так быстро, но с трудом определяли, какие именно можно, а какие нельзя. Существует несколько так называемых NP-трудных задач, для которых большинство людей считает полиномиальное решение невозможным, хотя никто не умеет доказать сверхполиномиальную нижнюю границу.

Задача принятия решения — задача, результат которой представляет собой единственное булево значение: «да» или «нет». Определим три класса таких задач:

- **P** — множество задач принятия решения, разрешимых за полиномиальное время. Интуитивно P — множество задач, которые можно решить быстро.
- **NP** — множество задач принятия решения со следующим свойством: если ответ «да», существует доказательство этого факта, которое можно проверить за полиномиальное время. Интуитивно NP — множество задач, для которых ответ «да» можно быстро проверить, если перед нами есть решение.
- **co-NP** в некотором смысле противоположен NP. Если ответ задачи из co-NP — «нет», существует доказательство этого факта, проверяемое за полиномиальное время.

Например, задача выполнимости схемы принадлежит NP. Если булева схема выполнима, любой набор из m входных значений, при котором она выдаёт True, служит доказательством её выполнимости; проверить доказательство можно, вычислив результат схемы за полиномиальное время. Широко распространено убеждение, что выполнимость схемы не принадлежит ни P, ни co-NP, но никто этого точно не знает.

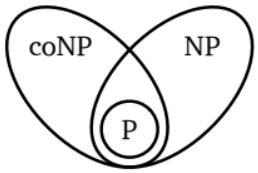
Каждая задача принятия решения из P также принадлежит NP. Если задача принадлежит P, ответ «да» можно проверить за полиномиальное время, вычислив ответ заново! Аналогично, каждая задача из P принадлежит и co-NP.

Возможно, самый важный нерешённый вопрос теоретической информатики — если не всей информатики и даже не всей науки — состоит в том, различны ли классы сложности P и NP. Интуитивно большинству кажется очевидным, что $P \neq NP$; домашние задания и экзамены по алгоритмам и структурам данных, надеюсь, убедили вас, что задачи могут быть невероятно сложными, даже когда их решения задним числом кажутся простыми. Это совершенно очевидно: конечно, решать задачи с нуля сложнее, чем проверять правильность готового решения. Мы можем разумно принять утверждение $P \neq NP$, и большинство разработчиков алгоритмов так и поступает, как закон природы — подобно другим законам, таким как уравнения Максвелла, общая теория относительности и завтрашний восход солнца, — которые убедительно подтверждаются свидетельствами, но не имеют математического доказательства.

Однако, соблюдая математическую строгость, нужно признать: никто не знает, как доказать $P \neq NP$. Более того, за десятилетия реального прогресса в доказательстве было мало или не было

вовсе.[2] Математический институт Клэя ставит вопрос о P и NP первым среди семи задач тысячелетия и предлагает миллион долларов за его решение. И да, некоторые люди действительно потеряли души или по крайней мере рассудок, пытаясь решить эту задачу.

Более тонкий, но тоже открытый вопрос — различаются ли классы NP и co-NP. Даже если мы умеем быстро проверять каждый ответ «да», нет оснований полагать, что мы столь же быстро можем проверять и ответы «нет». Например, насколько известно, короткого доказательства невыполнимости булевой схемы не существует. Обычно считают, что NP ≠ co-NP, но и это никто не умеет доказать.



[1] Это понятие эффективности независимо формализовали Алан Кобэм в 1965 году, Джек Эдмондс в 1965 году и Майкл Рабин в 1966 году, хотя Курт Гёдель, Джон Нэш и Джон фон Нейман рассматривали похожие понятия более чем за десять лет до этого.

[2] Возможно, наиболее существенный прогресс состоит в результатах о барьерах, показывающих, что целые направления поиска обречены на неудачу. В очень реальном смысле мы не только не представляем, как доказать P ≠ NP, но и можем доказать, что не представляем, как это доказать!

12.3. NP-трудные, принадлежащие NP и NP-полные задачи

Задача П является **NP-трудной**, если полиномиальный алгоритм для П влёк бы существование полиномиального алгоритма для любой задачи из NP. Иными словами:

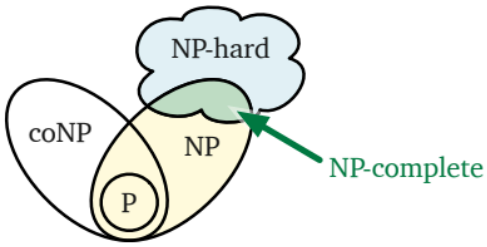
> *П является NP-трудной ⇐⇒ если П решается за полиномиальное время, то P = NP.*

Интуитивно, если бы мы могли быстро решить одну конкретную NP-трудную задачу, то могли бы быстро решить любую задачу, решение которой легко понять, используя решение этой одной специальной задачи как подпрограмму. NP-трудные задачи по меньшей мере столь же трудны, как любая задача из NP.

Наконец, задача называется **NP-полной**, если она одновременно NP-трудна и принадлежит NP, то есть «не сложнее NP». Неформально NP-полные задачи — самые трудные задачи в NP. Полиномиальный алгоритм хотя бы для одной NP-полной задачи немедленно означал бы существование полиномиального алгоритма для каждой NP-полной задачи. Доказана NP-полнота буквально тысяч задач, поэтому полиномиальный алгоритм для одной из них, а значит и для всех, кажется невероятно маловероятным.

Назвать задачу NP-трудной — всё равно что сказать: «Если у меня есть собака, она свободно говорит по-английски». Вы, вероятно, не знаете, есть ли у меня собака, но, думаю, почти

уверены, что говорящей собаки у меня нет. Никто математически не доказал, что собаки не умеют говорить по-английски: отсутствие людей, слышавших такую речь, и сотни обследований собак без подходящего строения пасти и умственных способностей служат свидетельствами, но не математическим доказательством. Тем не менее ни один здравомыслящий человек не поверил бы мне, заяви я, что у меня есть собака, свободно говорящая по-английски.[3] Поэтому из утверждения «Если у меня есть собака, она свободно говорит по-английски» естественно следует: никто в здравом уме не должен верить, что у меня есть собака! Точно так же, если задача NP-трудна, никто в здравом уме не должен верить, что её можно решить за полиномиальное время.



Далеко не очевидно, что NP-трудные задачи вообще существуют. Следующую замечательную теорему впервые опубликовал Стивен Кук в 1971 году, а Леонид Левин независимо доказал её в 1973 году.[4]

Теорема Кука — Левина. Задача выполнимости булевой схемы NP-трудна.

Я даже не буду набрасывать здесь доказательство, поскольку намеренно оставил определения расплывчатыми.[5]

[3] ...Заведующий кафедрой печально качает головой: «Да бросьте, это просто лай. Позвольте мне задать вопрос. Кто был величайшим теоретиком сложности всех времён?» Собака наклоняет голову, на несколько секунд задумывается и говорит: «Карп!» Когда заведующий выгоняет их из кабинета, собака поворачивается к хозяину и спрашивает: «Может, надо было сказать Импальяццо?»

[4] Левин впервые сообщил о своих результатах на семинарах в Москве в 1971 году, ещё будучи аспирантом. Весть о результате Кука дошла до Советского Союза не раньше 1973 года, уже после публикации сообщения Левина; согласно закону Стиглера, этот результат часто называют «теоремой Кука». По политическим причинам Левину отказали в присуждении степени в Московском университете; в 1978 году он эмигрировал в США и через год защитился в Массачусетском технологическом институте. Математический факультет Беркли отказал Куку в постоянной должности в 1970 году, за год до публикации его основополагающей статьи; позднее именно он, но не Левин, получил за это доказательство премию Тьюринга.

[5] Заинтересованные читатели найдут доказательство в моих лекциях о недетерминированных машинах Тьюринга на http://algorithms.wtf или в превосходной книге Боаза Барака *Introduction to Theoretical Computer Science*.

♥ 12.4. Формальные определения (HC SVNT DRACONES)

Формально классы сложности P, NP и co-NP определяются через языки и машины Тьюринга. **Язык** — множество строк над некоторым конечным алфавитом Σ; без ограничения общности можно считать Σ = {0, 1}. **Машина Тьюринга** — очень ограниченный вид компьютера, грубо говоря конечный автомат с неограниченной лентой памяти; её точное определение на удивление несущественно. P — множество языков, принадлежность которым детерминированная одноленточная машина Тьюринга может определить за полиномиальное время. Аналогично, NP — множество языков, принадлежность которым недетерминированная машина Тьюринга может определить за полиномиальное время; NP — сокращение от *Nondeterministic Polynomial-time*, то есть «недетерминированное полиномиальное время».

Требование полиномиального времени достаточно грубо, чтобы не уточнять точный вид машины Тьюринга: количество лент, головок, дорожек, размер алфавита ленты и так далее. Фактически любой алгоритм, работающий на машине с произвольным доступом[6] за время T(n), можно смоделировать одноленточной машиной Тьюринга с одной дорожкой и одной головкой за $O(T(n)^4)$ времени. Этот результат позволяет строго рассуждать о вычислительной сложности через стандартные высокоуровневые конструкции, такие как массивы, циклы и рекурсия, вместо непосредственного описания всего в терминах машин Тьюринга.

Формально задача Π является NP-трудной тогда и только тогда, когда для каждого языка $\Pi' \in$ NP существует полиномиальное **сведение Тьюринга** от Π′ к Π. Сведение Тьюринга — сведение, которое можно выполнить на машине Тьюринга: это машина M, решающая Π′ с помощью другой машины M′ для Π как подпрограммы-чёрного ящика. Сведения Тьюринга также называют сведениями к оракулу; полиномиальные сведения Тьюринга — сведениями Кука.

Исследователи теории сложности предпочитают определять NP-трудность через полиномиальные **сведения по многим к одному**, называемые также сведениями Карпа. Сведение по многим к одному языка $L' \subseteq \Sigma^*$ к языку $L \subseteq \Sigma^*$ — функция $f: \Sigma^* \to \Sigma^*$, такая что $x \in L'$ тогда и только тогда, когда $f(x) \in L$. Тогда язык L можно определить как NP-трудный тогда и только тогда, когда для любого языка $L' \in$ NP существует вычислимое за полиномиальное время сведение по многим к одному от L′ к L.

Каждое сведение Карпа является сведением Кука, но не наоборот. В частности, любое сведение Карпа одной задачи принятия решения Π к другой Π′ равносильно преобразованию входа Π во вход Π′, вызову оракула, то есть подпрограммы, для Π′ и непосредственному возврату ответа. Однако, насколько нам известно, не всякое сведение Кука можно смоделировать сведением Карпа.

Теоретики сложности предпочитают сведения Карпа прежде всего потому, что NP замкнут относительно сведений Карпа, но не замкнут относительно сведений Кука, если только NP не равен co-NP, что считается маловероятным. Существуют естественные задачи, которые (1) NP-трудны относительно сведений Кука, но (2) NP-трудны относительно сведений Карпа лишь при P = NP. Тривиальный пример — UnSat: дана булева формула; всегда ли она ложна? С другой стороны, сведения по многим к одному применимы только к задачам принятия решения,

формально — к языкам; формально ни одна задача оптимизации или построения не является NP-трудной по Карпу.

Для ещё большей путаницы Кук и Карп изначально определяли NP-трудность через сведения с логарифмическим объёмом памяти. Любое такое сведение является полиномиальным по времени, но, насколько известно, не наоборот. Остаётся открытым вопрос, меняется ли множество NP-трудных задач, если расширить допустимые сведения Кука или Карпа с логарифмической памяти до полиномиального времени.

К счастью, ни одна из этих тонкостей не высовывает свою безобразную голову на практике: в частности, каждое сведение этой главы можно формализовать как сведение по многим к одному с логарифмической памятью. Так что теперь можете проснуться.

[6] Машина с произвольным доступом — модель вычислений, более точно отражающая реальные компьютеры. Стандартная такая машина имеет неограниченную память с произвольным доступом в виде неограниченного массива M[0 .. ∞], где по каждому адресу M[i] хранится одно w-битное целое число при некотором фиксированном w, и может за постоянное время читать и записывать любое место памяти. Алгоритмы для такой машины формально записывают на ассемблероподобном языке с инструкциями вроде ADD i, j, k, означающей «M[i] ← M[j] + M[k]»; INDIR i, j, означающей «M[i] ← M[M[j]]»; и IF0GOTO i, ℓ, означающей «если M[i] = 0, перейти к строке ℓ». Однако точный набор инструкций на удивление несущественен. По определению каждая инструкция выполняется за единицу времени. На практике такие алгоритмы можно точно описывать более высокоуровневым псевдокодом, если внимательно следить за точностью арифметики.

12.5. Сведения и выполнимость формул

Чтобы доказать NP-трудность любой задачи, кроме выполнимости схемы, мы используем сведение. Свести задачу A к другой задаче B — значит описать алгоритм решения A, предполагая, что алгоритм для B уже существует. Вы занимаетесь сведениями уже много лет, ещё до начала чтения этой книги, только, вероятно, называли их как-то иначе: подпрограммами, служебными функциями, модульным программированием или использованием калькулятора.

Чтобы доказать NP-трудность, мы описываем похожее преобразование задач, но направление оказывается противоположным тому, которого ожидает большинство людей. Вам следует вытатуировать следующее правило на тыльной стороне ладони рядом с датой рождения мамы и настоящими правилами «Монополии».[7]

> *Чтобы доказать NP-трудность задачи A, сведите известную NP-трудную задачу к A.*

Иными словами, чтобы доказать трудность своей задачи, нужно описать эффективный алгоритм решения *другой* задачи, о которой вы уже знаете, что она трудна, используя гипотетический эффективный алгоритм вашей задачи как подпрограмму-чёрный ящик. По сути это доказательство от противного: сведение показывает, что если бы ваша задача была лёгкой, то и другая задача была бы лёгкой, а это не так. Равносильно, поскольку другая задача

известна как трудная, из сведения следует трудность вашей задачи: предполагаемого эффективного алгоритма на самом деле не существует.

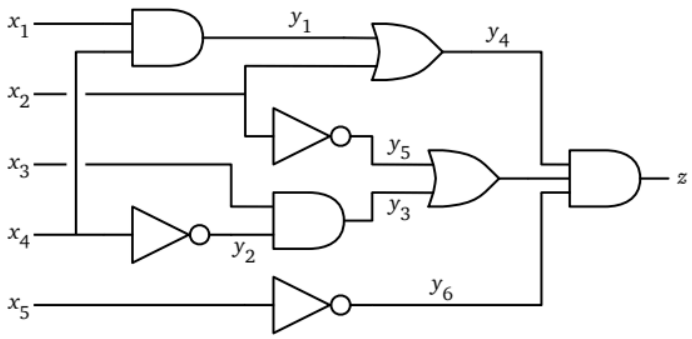
В качестве классического примера рассмотрим задачу выполнимости формулы, обычно называемую просто Sat. На вход Sat подаётся булева формула, например

$$(a \vee b \vee c \vee \bar{d}) \Leftrightarrow ((b \wedge \bar{c}) \vee (\bar{a} \Rightarrow d) \vee (c \neq a \wedge b)),$$

и требуется выяснить, можно ли назначить булевы значения переменным a, b, c и так далее, чтобы значение всей формулы было True.

Чтобы доказать NP-трудность Sat, нужно построить сведение от известной NP-трудной задачи. Пока единственная известная нам NP-трудная задача — CircuitSat, поэтому начнём с неё.

Пусть K — произвольная булева схема. Преобразуем, или точнее перепишем, K в булеву формулу Ф следующим образом. Сначала пометим каждый внутренний провод новой переменной y_j, а выходной провод — новой переменной z. Формула Ф состоит из списка равенств, по одному для каждого логического элемента, соединённых операциями And, а в конце добавляется ∧ z. Рисунок 12.5 показывает результат такой записи для нашей схемы из примера.



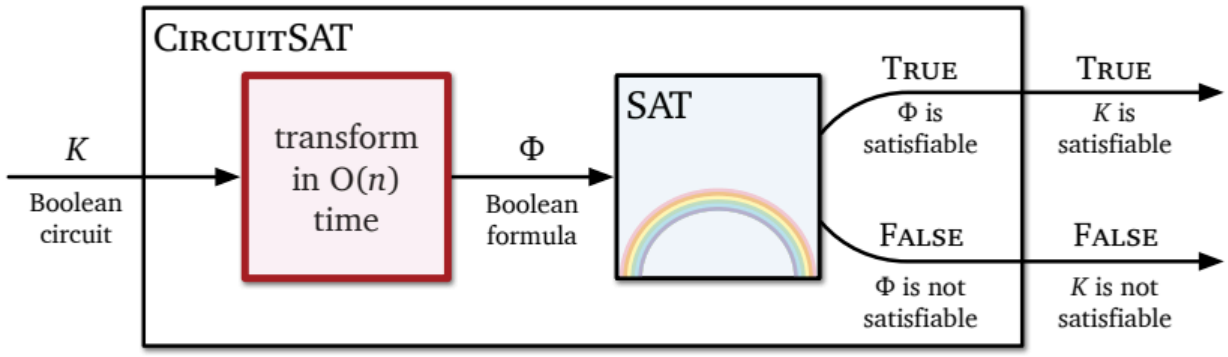
$$(y_1 = x_1 \wedge x_4) \wedge (y_2 = \overline{x_4}) \wedge (y_3 = x_3 \wedge y_2) \wedge (y_4 = y_1 \vee x_2) \wedge$$
$$(y_5 = \overline{x_2}) \wedge (y_6 = \overline{x_5}) \wedge (y_7 = y_3 \vee y_5) \wedge (z = y_4 \wedge y_7 \wedge y_6) \wedge z$$

Утверждается, что исходная схема K выполнима тогда и только тогда, когда выполнима получившаяся формула Ф. Как и любое утверждение «тогда и только тогда», доказываем это в два этапа:

- Имея набор входов, выполняющий схему K, можно получить выполняющий формулу Ф набор значений, вычислив выход каждого элемента K.
- Имея выполняющий формулу Ф набор значений, можно получить выполняющий схему набор входов, просто отбросив переменные внутренних проводов y_i и выходную переменную z.

Всё преобразование схемы в формулу можно выполнить за линейное время. Кроме того, размер полученной формулы лишь в постоянное число раз больше любого разумного представления схемы.

Предположим для рассуждения, что существует алгоритм, определяющий за полиномиальное время, выполнима ли заданная булева формула. Тогда для любой булевой схемы K можно решить вопрос о её выполнимости: сначала преобразовать K в формулу Φ описанным способом, а затем спросить наш загадочный волшебный алгоритм Sat, выполнима ли Φ, как показано на следующем рисунке. Каждый блок изображает алгоритм. Красный блок слева — подпрограмма преобразования. Блок справа — гипотетический волшебный алгоритм Sat. Он обязательно волшебный, потому что на нём радуга.[8]



Если вы предпочитаете волшебный псевдокод волшебным блокам:

```
CircuitSat(K):
    переписать K в булеву формулу Φ
    вернуть Sat(Φ)   ⟨⟨????ВОЛШЕБСТВО????⟩⟩
```

Перевод K в Φ требует только полиномиального, на самом деле линейного, времени, поэтому весь алгоритм CircuitSat также работает за полиномиальное время:

$$T_{\text{CircuitSat}}(n) \leq O(n) + T_{\text{Sat}}(O(n)).$$

Мы заключаем, что любой полиномиальный алгоритм для Sat дал бы полиномиальный алгоритм для CircuitSat, а тот, в свою очередь, означал бы P = NP. Следовательно, Sat NP-трудна!

[7] Если игрок попадает на свободную собственность и отказывается или не может её купить, собственность немедленно выставляется на аукцион; отказавшийся игрок тоже может участвовать, а ставки могут быть как угодно выше или ниже указанной цены. Игроки в тюрьме по-прежнему могут покупать и продавать собственность, дома и отели и получать ренту. В игре 32 дома и 12 отелей; когда они закончились, они закончились. В частности, если все дома уже на поле, нельзя заменить отель четырьмя домами: придётся снести все отели группы до основания. Игроки могут продавать или обменивать незастроенную собственность, но не продавать её банку; здания можно продавать банку за полцены, но нельзя продавать или обменивать друг с другом. Все штрафы платятся непосредственно банку. Игрок на бесплатной стоянке ничего не выигрывает. Игрок, попавший на поле Go, получает ровно 200 долларов и не больше. Железные дороги — не волшебные телепорты. И наконец, Джефф всегда берёт ботинок. Нет, не тираннозавра и не пингвина — именно ботинок, чёрт возьми.

[8] Кей Эриксон, личное сообщение, 2011. Для читателей чёрно-белого печатного издания: да, эта круглая штука — радуга.

12.6. 3SAT (сведение из CIRCUITSAT)

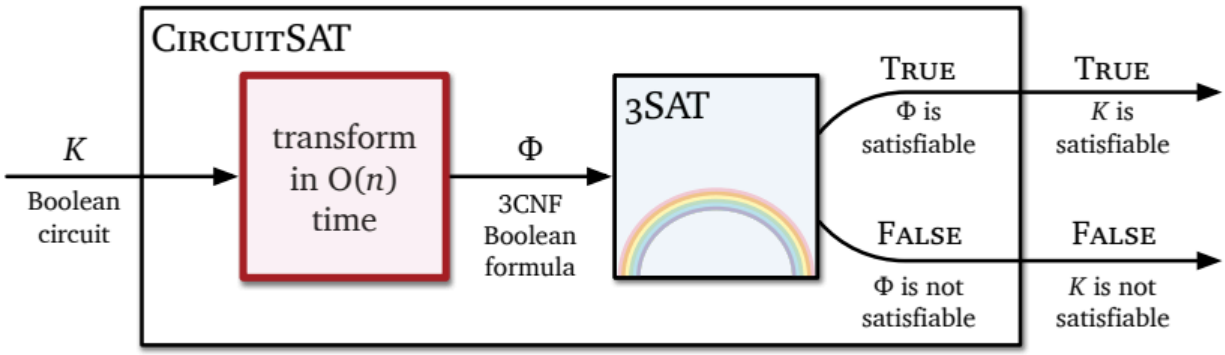
Частный случай `Sat`, особенно полезный при доказательстве NP-трудности, называется `3CNF-Sat`, или чаще просто `3Sat`.

Булева формула имеет *конъюнктивную нормальную форму* (КНФ), если она является конъюнкцией нескольких дизъюнктов, каждый из которых — дизъюнкция нескольких литералов, представляющих собой переменные или их отрицания. Например:

$$\underbrace{(a \vee b \vee c \vee d)}_{\text{дизъюнкт}} \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b}).$$

Формула в 3-КНФ — это КНФ с ровно тремя литералами в каждом дизъюнкте. Предыдущий пример не является 3-КНФ: в первом дизъюнкте четыре литерала, а в последнем всего два. `3Sat` — ограничение задачи `Sat` формулами в 3-КНФ: для данной формулы в 3-КНФ существует ли набор значений переменных, обращающий её в истину?

Можно доказать NP-трудность `3Sat` сведением из более общей задачи `Sat`, но на самом деле проще начать сначала и сделать сведение непосредственно из `CircuitSat`.



Данную булеву схему K преобразуем в эквивалентную по выполнимости формулу в 3-КНФ в несколько этапов. За исключением самого последнего, это сведение описал Григорий Цейтин в 1966 году, за пять лет до доказательств теоремы Кука — Левина. (В той же статье Цейтин описал задачу, которую мы теперь называем `CNF-Sat`, возможно впервые.) Описывая этапы, будем также доказывать их корректность.

- Убедимся, что каждый элемент И и ИЛИ в K имеет ровно два входа. Если элемент имеет $k > 2$ входов, заменим его бинарным деревом из $k - 1$ двухвходовых элементов. Обозначим результат K'. Схемы K и K' логически эквивалентны, поэтому любой удовлетворяющий вход K удовлетворяет K' и наоборот.
- Перепишем K' в булеву формулу Φ_1 с одним условием на каждый элемент, как в предыдущем сведении к `Sat`. Мы уже доказали, что любой удовлетворяющий вход K' преобразуется в удовлетворяющий набор Φ_1 и наоборот.

- Заменим каждое условие Φ_1 формулой в КНФ. Существуют только три вида условий — по одному для каждого типа элементов K':

$$\begin{aligned} a = b \wedge c &\longmapsto (a \vee \bar{b} \vee \bar{c}) \wedge (\bar{a} \vee b) \wedge (\bar{a} \vee c), \\ a = b \vee c &\longmapsto (\bar{a} \vee b \vee c) \wedge (a \vee \bar{b}) \wedge (a \vee \bar{c}), \\ a = \bar{b} &\longmapsto (a \vee b) \wedge (\bar{a} \vee \bar{b}). \end{aligned}$$

Обозначим полученную формулу в КНФ через Φ_2. Формулы Φ_1 и Φ_2 логически эквивалентны, поэтому каждый удовлетворяющий набор одной удовлетворяет и другой.

- Заменим каждый дизъюнкт Φ_2 формулой в 3-КНФ. В каждом дизъюнкте Φ_2 не более трёх литералов, а нам нужно ровно три. Каждый двухлитеральный дизъюнкт разворачиваем в два трёхлитеральных, вводя одну новую переменную, а последний однолитеральный дизъюнкт — в четыре трёхлитеральных, вводя две новые переменные:

$$\begin{aligned} a \vee b &\longmapsto (a \vee b \vee x) \wedge (a \vee b \vee \bar{x}), \\ z &\longmapsto (z \vee x \vee y) \wedge (z \vee \bar{x} \vee y) \wedge (z \vee x \vee \bar{y}) \wedge (z \vee \bar{x} \vee \bar{y}). \end{aligned}$$

Обозначим результат Φ_3. Каждый удовлетворяющий набор Φ_2 можно преобразовать в удовлетворяющий набор Φ_3, присвоив новым переменным x, y произвольные значения. И наоборот, из любого удовлетворяющего набора Φ_3 получается удовлетворяющий набор Φ_2, если отбросить новые переменные.

Например, рассмотренная ранее схема преобразуется в следующую формулу в 3-КНФ; сравните с рисунком 12.5.

$$\begin{gathered} (y_1 \vee \overline{x_1} \vee \overline{x_4}) \wedge (\overline{y_1} \vee x_1 \vee z_1) \wedge (\overline{y_1} \vee x_1 \vee \overline{z_1}) \wedge (\overline{y_1} \vee x_4 \vee z_2) \wedge (\overline{y_1} \vee x_4 \vee \overline{z_2}) \\ \wedge (y_2 \vee x_4 \vee z_3) \wedge (y_2 \vee x_4 \vee \overline{z_3}) \wedge (\overline{y_2} \vee \overline{x_4} \vee z_4) \wedge (\overline{y_2} \vee \overline{x_4} \vee \overline{z_4}) \\ \wedge (y_3 \vee \overline{x_3} \vee \overline{y_2}) \wedge (\overline{y_3} \vee x_3 \vee z_5) \wedge (\overline{y_3} \vee x_3 \vee \overline{z_5}) \wedge (\overline{y_3} \vee y_2 \vee z_6) \wedge (\overline{y_3} \vee y_2 \vee \overline{z_6}) \\ \wedge (\overline{y_4} \vee y_1 \vee x_2) \wedge (y_4 \vee \overline{x_2} \vee z_7) \wedge (y_4 \vee \overline{x_2} \vee \overline{z_7}) \wedge (y_4 \vee \overline{y_1} \vee z_8) \wedge (y_4 \vee \overline{y_1} \vee \overline{z_8}) \\ \wedge (y_5 \vee x_2 \vee z_9) \wedge (y_5 \vee x_2 \vee \overline{z_9}) \wedge (\overline{y_5} \vee \overline{x_2} \vee z_{10}) \wedge (\overline{y_5} \vee \overline{x_2} \vee \overline{z_{10}}) \\ \wedge (y_6 \vee x_5 \vee z_{11}) \wedge (y_6 \vee x_5 \vee \overline{z_{11}}) \wedge (\overline{y_6} \vee \overline{x_5} \vee z_{12}) \wedge (\overline{y_6} \vee \overline{x_5} \vee \overline{z_{12}}) \\ \wedge (\overline{y_7} \vee y_3 \vee y_5) \wedge (y_7 \vee \overline{y_3} \vee z_{13}) \wedge (y_7 \vee \overline{y_3} \vee \overline{z_{13}}) \wedge (y_7 \vee \overline{y_5} \vee z_{14}) \wedge (y_7 \vee \overline{y_5} \vee \overline{z_{14}}) \\ \wedge (y_8 \vee \overline{y_4} \vee \overline{y_7}) \wedge (\overline{y_8} \vee y_4 \vee z_{15}) \wedge (\overline{y_8} \vee y_4 \vee \overline{z_{15}}) \wedge (\overline{y_8} \vee y_7 \vee z_{16}) \wedge (\overline{y_8} \vee y_7 \vee \overline{z_{16}}) \\ \wedge (y_9 \vee \overline{y_8} \vee \overline{y_6}) \wedge (\overline{y_9} \vee y_8 \vee z_{17}) \wedge (\overline{y_9} \vee y_6 \vee z_{18}) \wedge (\overline{y_9} \vee y_6 \vee \overline{z_{18}}) \wedge (\overline{y_9} \vee y_8 \vee \overline{z_{17}}) \\ \wedge (y_9 \vee z_{19} \vee z_{20}) \wedge (y_9 \vee \overline{z_{19}} \vee z_{20}) \wedge (y_9 \vee z_{19} \vee \overline{z_{20}}) \wedge (y_9 \vee \overline{z_{19}} \vee \overline{z_{20}}) \end{gathered}$$

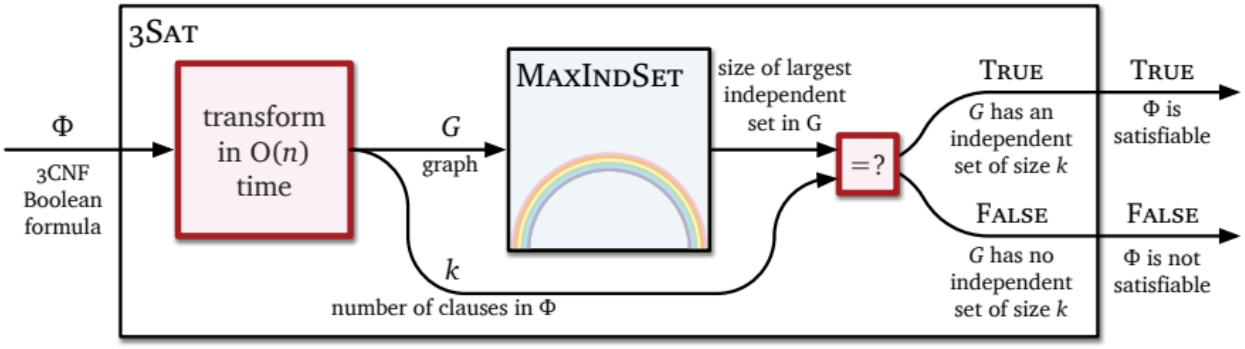
Ух! На первый взгляд эта формула может выглядеть куда сложнее исходной схемы, но на самом деле она больше только в постоянное число раз. В частности, упрощённая схема K' содержит не более чем вдвое больше проводов, чем исходная K, а каждый двухвходовый элемент K' преобразуется не более чем в пять дизъюнктов Φ_3. Даже если бы размер формулы был большим полиномом размера схемы, например n^{374}, сведение оставалось бы корректным.

Наше сведение преобразует произвольную булеву схему K в формулу 3-КНФ Φ_3 за полиномиальное, фактически линейное, время. Более того, любой удовлетворяющий вход K преобразуется в удовлетворяющий набор Φ_3, и наоборот. Иными словами, K выполнима тогда и только тогда, когда выполнима Φ_3. Поэтому полиномиальный алгоритм `3Sat` дал бы полиномиальный алгоритм `CircuitSat`, что означало бы $P = NP$. Следовательно, `3Sat` NP-трудна.

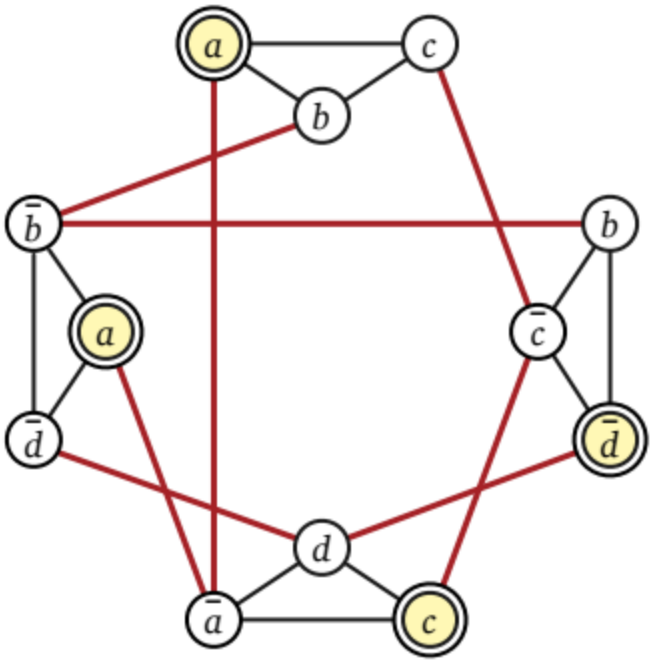
12.7. Максимальное независимое множество (сведение из 3SAT)

В следующих нескольких задачах входом служит простой невзвешенный неориентированный граф, а результатом — размер наибольшего или наименьшего подграфа с некоторым структурным свойством.

Пусть G — произвольный граф. *Независимое множество* в G — подмножество вершин, между которыми нет рёбер. Задача о максимальном независимом множестве, которую я сокращённо называю `MaxIndSet`, спрашивает размер наибольшего независимого множества заданного графа. Я докажу NP-трудность `MaxIndSet` сведением из `3Sat`, как подсказывает рисунок 12.7.



По произвольной формуле Φ в 3-КНФ построим граф G. Пусть k — число дизъюнктов Φ. Граф содержит ровно $3k$ вершин, по одной на каждое вхождение литерала в Φ. Две вершины соединены ребром тогда и только тогда, когда они либо (1) соответствуют литералам одного дизъюнкта, либо (2) соответствуют переменной и её отрицанию. Например, формула $(a \vee b \vee c) \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b} \vee \bar{d})$ преобразуется в граф на рисунке 12.8.



$$(a \vee b \vee c) \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b} \vee \bar{d})$$

Каждое независимое множество G содержит не более одной вершины из каждого треугольника дизъюнкта, поскольку любые две вершины треугольника соединены. Поэтому максимальное независимое множество имеет размер не больше k. Утверждается, что G содержит независимое множество размера ровно k тогда и только тогда, когда исходная формула Φ выполнима. Как обычно для утверждения «тогда и только тогда», доказательство состоит из двух частей.

⇒ Пусть Φ выполнима. Зафиксируем произвольный удовлетворяющий набор. По определению каждый дизъюнкт Φ содержит хотя бы один истинный литерал. Значит, можно выбрать множество S из k вершин G, по одной на каждый треугольник, причём соответствующие литералы будут истинны. Поскольку в каждом треугольнике не более одной вершины S, никакие две вершины S не соединены ребром треугольника. Поскольку все соответствующие литералы истинны, никакие две вершины S не соединены ребром между противоречащими литералами. Следовательно, S — независимое множество размера k.

⇐ Пусть, наоборот, в G есть независимое множество S размера k. Каждая вершина S должна находиться в отдельном треугольнике. Присвоим значение «истина» каждому литералу из S; поскольку противоречащие литералы соединены рёбрами, такое присваивание непротиворечиво. Могут оставаться переменные x, для которых ни x, ни $\bar{x}$ не соответствуют вершине S; им можно присвоить произвольные значения. Поскольку S содержит вершину из каждого треугольника, каждый дизъюнкт Φ содержит хотя бы один истинный литерал. Следовательно, Φ выполнима.

Преобразование Φ в G занимает полиномиальное время, даже при полном переборе. Поэтому если бы мы могли решить `MaxIndSet` за полиномиальное время, то могли бы за полиномиальное время решить и `3Sat`, построив граф G и сравнив размер его наибольшего независимого

множества с числом дизъюнктов Φ. Но это означало бы $P = NP$, что нелепо! Следовательно, `MaxIndSet` NP-трудна.

12.8. Общая схема

Все доказательства NP-трудности и, более широко, все полиномиальные сведения следуют одной общей схеме. Чтобы свести задачу X к задаче Y за полиномиальное время, нужно сделать три вещи:

1. Описать полиномиальный алгоритм преобразования произвольного экземпляра x задачи X в специальный экземпляр y задачи Y.
2. Доказать, что если x — «хороший» экземпляр X, то y — «хороший» экземпляр Y.
3. Доказать, что если y — «хороший» экземпляр Y, то x — «хороший» экземпляр X. (Обычно именно эта часть доставляет больше всего трудностей.)

Разумеется, разработать корректное сведение не означает выполнять эти три задачи по очереди. Если сначала записать алгоритм, который кажется работающим, а потом пытаться доказать его корректность, это редко приводит к успеху, особенно при ограниченном времени, например на экзамене. Алгоритм и доказательства обоих направлений нужно разрабатывать одновременно.

Цитируя покойного великого Рики Джея:[9] «Этот навык приобретается».

Многих студентов смущает, что алгоритм сведения работает только в одну сторону — от X к Y, — а доказательство корректности должно работать в обе. Однако доказательства двух направлений не симметричны. Одно должно охватывать произвольные экземпляры X, а другое — только специальные экземпляры Y, полученные алгоритмом сведения. Использование этой асимметрии — ключ к построению корректных сведений.

Мне полезно думать о преобразовании *сертификатов* — доказательств того, что экземпляр «хорош», — вместе с самими экземплярами. Например, сертификат `CircuitSat` — набор входов, зажигающий лампочку; сертификат `Sat` или `3Sat` — удовлетворяющий набор значений; сертификат `MaxIndSet` — большое независимое множество. Для сведения X к Y нужно разработать три алгоритма:

- преобразовать произвольный экземпляр x задачи X в специальный экземпляр y задачи Y за полиномиальное время;
- преобразовать произвольный сертификат для x в сертификат для y;
- преобразовать произвольный сертификат для y в сертификат для x.

Вторая и третья задачи относятся к входу и выходу первого алгоритма. Обратимым должно быть преобразование сертификатов, а не экземпляров. Нам никогда не нужно преобразовывать экземпляры Y или вообще рассматривать произвольные экземпляры Y. Полиномиальное время требуется только первому алгоритму, хотя на практике второй и третий почти всегда проще первого.

Например, наше сведение `CircuitSat` к `3Sat` состоит из трёх алгоритмов:

- Первый за полиномиальное время преобразует произвольную булеву схему K в специальную формулу Φ_3 в 3-КНФ: кодирует каждый провод переменной, каждый элемент — подформулой и разворачивает подформулы в 3-КНФ.
- Второй преобразует произвольный удовлетворяющий вход K в удовлетворяющий набор Φ_3: проводит вход через схему, переносит значение каждого провода в соответствующую переменную и произвольно задаёт дополнительные переменные.
- Третий преобразует произвольный удовлетворяющий набор Φ_3 в удовлетворяющий вход K: переносит значения переменных проводов на соответствующие провода K.

Сведение работает, поскольку первый алгоритм кодирует любую схему K высокоструктурированной формулой Φ_3. Её особая структура ограничивает способы удовлетворения: каждый удовлетворяющий набор Φ_3 должен «происходить» из некоторого удовлетворяющего входа K. Произвольные формулы в 3-КНФ нам рассматривать не нужно.

Аналогично сведение `3Sat` к `MaxIndSet` состоит из трёх алгоритмов:

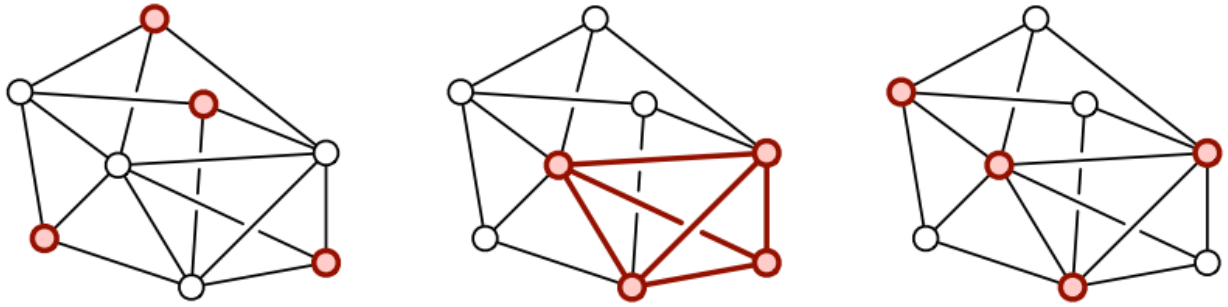
- Первый за полиномиальное время преобразует произвольную формулу Φ в 3-КНФ в специальный граф G и конкретное целое число k.
- Второй преобразует произвольный удовлетворяющий набор Φ в независимое множество размера k в G.
- Третий преобразует произвольное независимое множество размера k в G в удовлетворяющий набор Φ.

И вновь первое преобразование кодирует входную формулу Φ высокоструктурированным графом G и конкретным числом k. Структура G обеспечивает, что каждое независимое множество размера k «происходит» из удовлетворяющего набора Φ. Произвольные графы и размеры независимых множеств мы не рассматриваем.

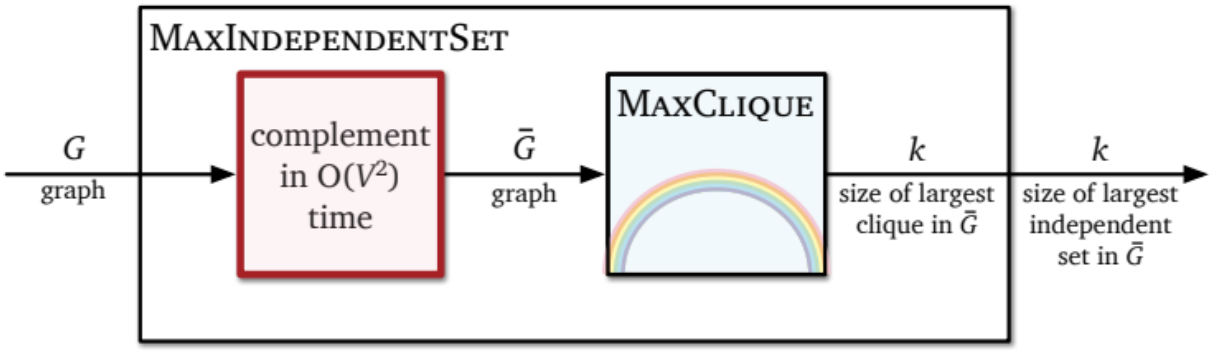
[9] Из его офф-бродвейского шоу 1996 года «Рики Джей и его 52 помощника».

12.9. Клика и вершинное покрытие (сведение из независимого множества)

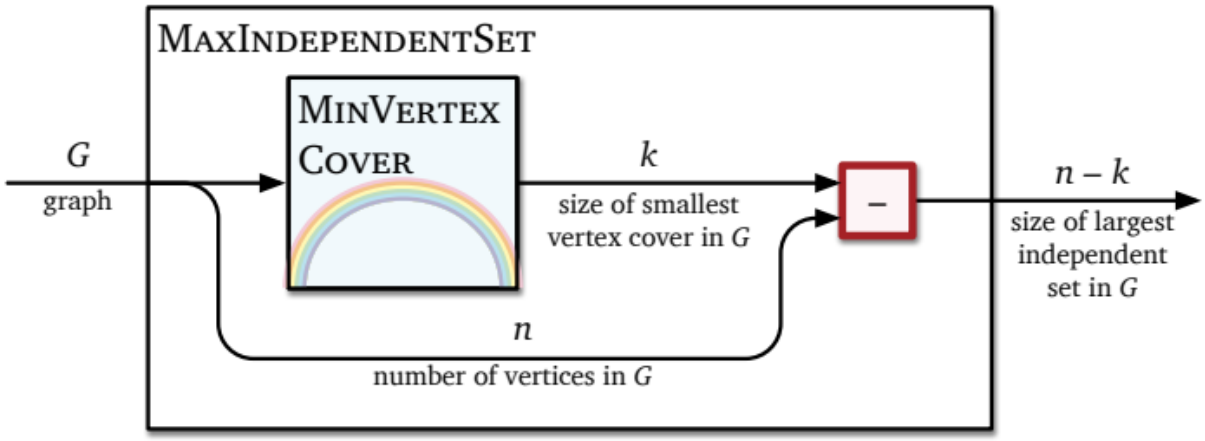
Клика — другое название полного графа, то есть графа, в котором любые две вершины соединены ребром. Задача `MaxClique` спрашивает число вершин в наибольшем полном подграфе заданного графа. *Вершинное покрытие* графа — множество вершин, содержащее хотя бы один конец каждого ребра. Задача `MinVertexCover` спрашивает размер наименьшего вершинного покрытия заданного графа.



Докажем NP-трудность `MaxClique` следующим простым сведением из `MaxIndSet`. Для любого графа G существует дополнение по рёбрам $\bar{G}$ с теми же вершинами, но противоположным множеством рёбер: uv — ребро $\bar{G}$ тогда и только тогда, когда uv не является ребром G. Множество вершин независимо в G тогда и только тогда, когда те же вершины образуют клику в $\bar{G}$. Значит, максимальное независимое множество G и максимальная клика дополнения G состоят из одних и тех же вершин и имеют одинаковый размер.



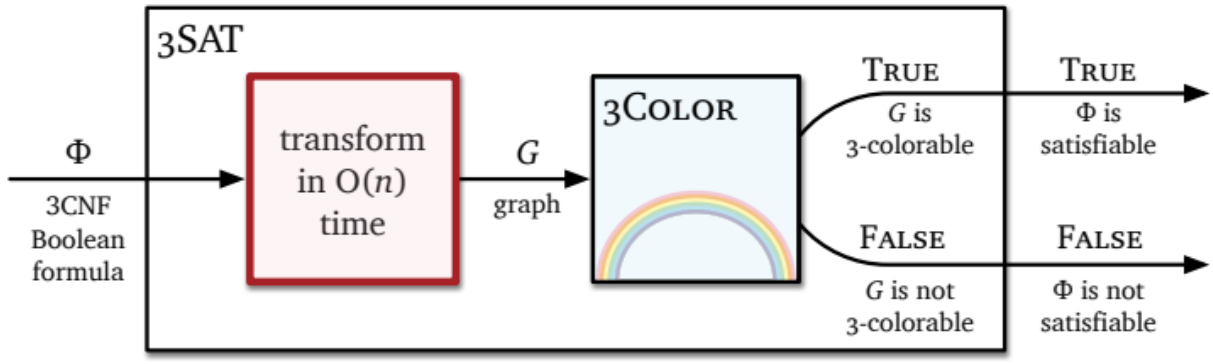
Доказательство NP-трудности `MinVertexCover` ещё проще, поскольку основано на следующем наблюдении: I — независимое множество графа $G = (V, E)$ тогда и только тогда, когда его дополнение $V \setminus I$ — вершинное покрытие того же графа G. Значит, максимальное независимое множество любого графа — дополнение его минимального вершинного покрытия! Поэтому если минимальное вершинное покрытие графа с n вершинами имеет размер k, то максимальное независимое множество имеет размер $n - k$.



12.10. Раскраска графа (сведение из 3SAT)

Правильная k*-раскраска* графа $G = (V, E)$ — функция $C : V \to \{1, 2, \ldots, k\}$, присваивающая каждой вершине один из k «цветов» так, что концы каждого ребра имеют разные цвета. (На самом деле цвета — произвольные метки, которые для простоты мы представляем небольшими положительными целыми числами, а не электромагнитными частотами, векторами CMYK или номерами Pantone.) Задача раскраски графа спрашивает наименьшее возможное число цветов в правильной раскраске заданного графа.

Для доказательства NP-трудности достаточно рассмотреть задачу принятия решения `3Color`: можно ли правильно раскрасить данный граф в три цвета? Докажем NP-трудность `3Color` сведением из `3Sat`. (Почему `3Sat`? Потому что в названии есть тройка. Вам, наверное, кажется, что я шучу, но нет.) По формуле Φ в 3-КНФ построим граф G, раскрашиваемый в три цвета тогда и только тогда, когда Φ выполнима, как показывает обычная схема.



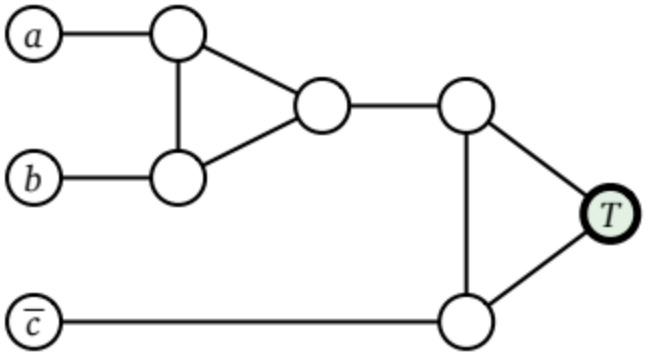
Мы описываем сведение с помощью стандартной стратегии разбиения выходного графа G на *гаджеты* — подграфы, которые выражают различные смысловые свойства входной формулы Φ на языке раскраски графов. Разбиение на отдельные гаджеты полезно не только для понимания существующих сведений и доказательства их корректности, но и для построения новых доказательств NP-трудности.[10]

Сведение формулы к графу использует три типа гаджетов:

- Есть единственный *гаджет истинности*: треугольник с вершинами T, F, X, интуитивно обозначающими «истина», «ложь» и «другое». Поскольку все эти вершины соединены, в любой правильной трёхцветной раскраске их цвета различны. Для удобства назовём эти цвета «истина», «ложь» и «другое». Следовательно, говоря, что вершина окрашена в цвет «истина», мы имеем в виду её совпадение по цвету с вершиной T.
- Для каждой переменной a граф содержит *гаджет переменной*: треугольник, соединяющий две новые вершины a и $\bar{a}$ с вершиной X гаджета истинности. Вершина a должна быть окрашена либо в «истину», либо в «ложь», поэтому $\bar{a}$ должна быть окрашена соответственно в «ложь» или «истину».



- Наконец, для каждого дизъюнкта Φ граф содержит *гаджет дизъюнкта*. Каждый такой гаджет соединяет три вершины литералов из соответствующих гаджетов переменных с вершиной T из гаджета истинности с помощью пяти новых непомеченных вершин и десяти рёбер, как показано ниже.



По существу каждый треугольник гаджета дизъюнкта действует как «мажоритарный элемент». В любой правильной трёхцветной раскраске, если две вершины слева от треугольника имеют одинаковый цвет, крайняя правая вершина треугольника должна иметь тот же цвет; если же две левые вершины имеют разные цвета, цвет правой можно выбрать произвольно. См. рисунок 12.14.

Отсюда следует, что правильной трёхцветной раскраски гаджета дизъюнкта, в которой все три вершины литералов окрашены в «ложь», не существует. С другой стороны, любую раскраску вершин литералов более чем одним цветом можно продолжить до правильной раскраски всего гаджета. Гаджеты переменных заставляют каждую вершину литерала иметь цвет «истина» или «ложь»; поэтому в любой правильной раскраске гаджета дизъюнкта хотя бы одна вершина литерала окрашена в «истину».

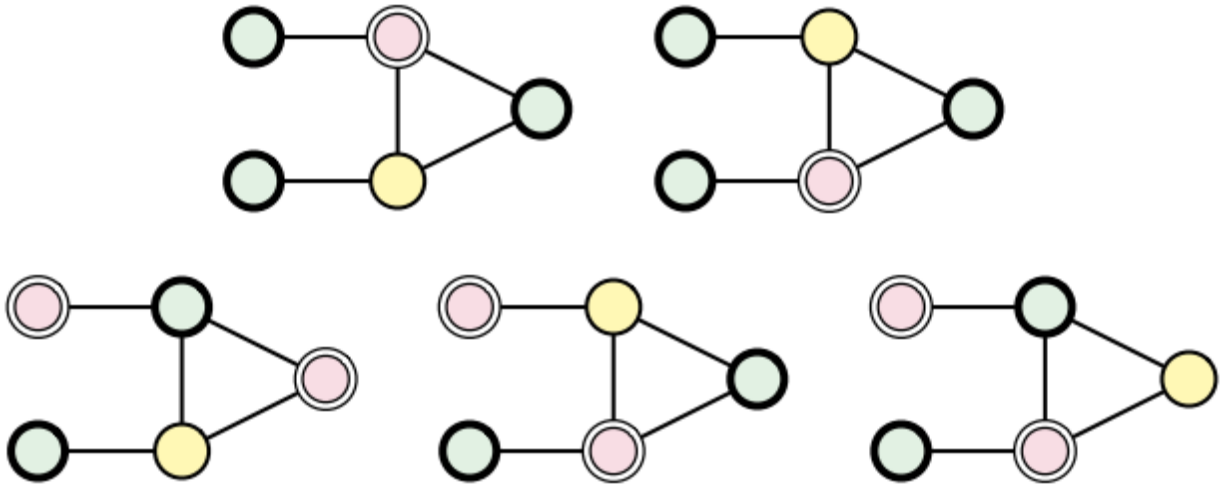
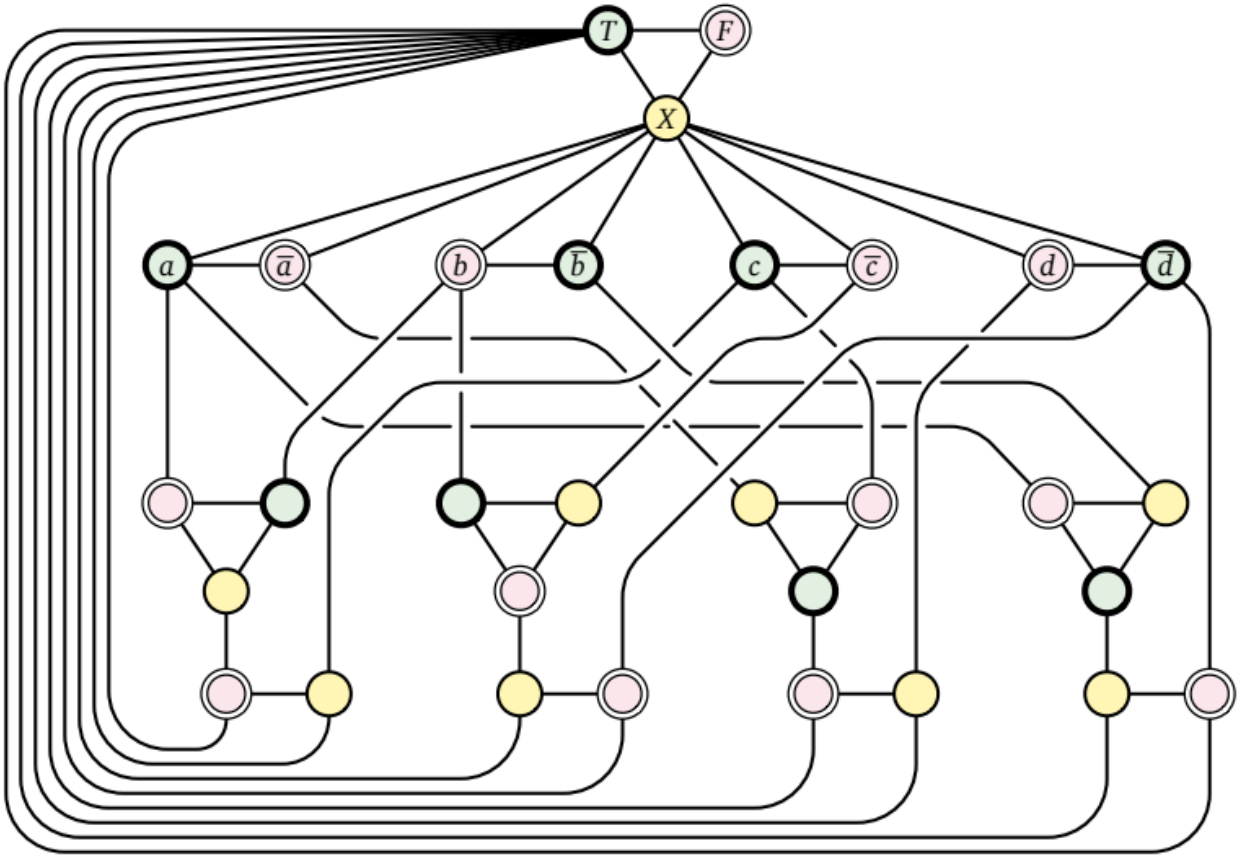


Рисунок 12.14. Все правильные трёхцветные раскраски половины гаджета с точностью до перестановки цветов.

Итоговый граф G содержит ровно одну вершину T, ровно одну вершину F и ровно две вершины $a, \bar{a}$ для каждой переменной. На рисунке 12.15 показан граф для той же формулы $(a \vee b \vee c) \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b} \vee \bar{d})$, которая иллюстрировала сведение к `MaxIndSet` на рисунке 12.8. Показанная трёхцветная раскраска — одна из нескольких, соответствующих набору $a = c = \mathrm{True}, b = d = \mathrm{False}$.



$$(a \vee b \vee c) \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b} \vee \bar{d})$$

Большую часть доказательства корректности мы уже выполнили. Если формула выполнима, можно раскрасить вершины литералов согласно любому удовлетворяющему набору, а затем, поскольку каждый дизъюнкт удовлетворён, продолжить раскраску на каждый его гаджет. С другой стороны, если граф раскрашивается в три цвета, из любой такой раскраски можно извлечь удовлетворяющий набор: хотя бы одна из трёх вершин литералов в каждом гаджете дизъюнкта окрашена в «истину».

Поскольку `3Color` — частный случай более общей задачи раскраски графа «каково минимальное число цветов?», эта более общая оптимизационная задача также NP-трудна.

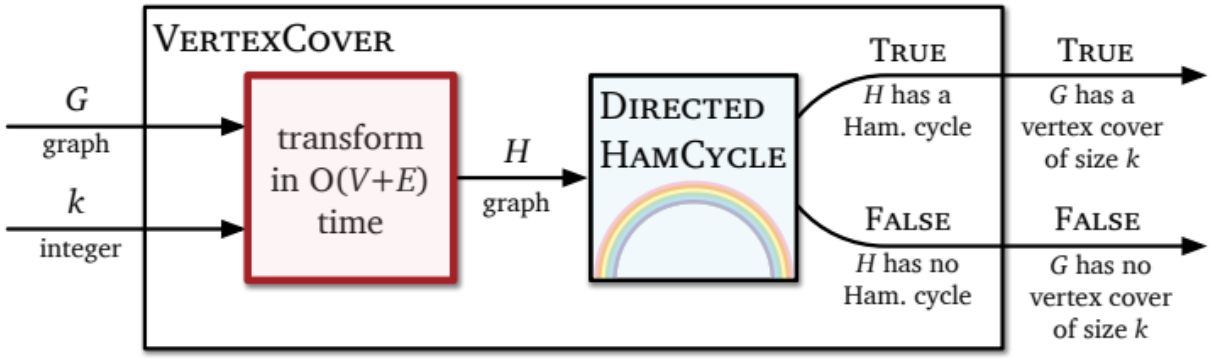
[10] В сведении `CircuitSat` к `Sat` каждый элемент входной схемы кодировался отдельным условием выходной формулы; эти условия — «гаджеты элементов». Аналогично сведение `3Sat` к `MaxIndSet` использовало два вида гаджетов: «гаджеты дизъюнктов» (треугольники) и «гаджеты переменных» (рёбра между противоречащими литералами).

12.11. Гамильтонов цикл

Гамильтонов цикл графа — цикл, посещающий каждую вершину ровно один раз. (Он сильно отличается от эйлерова цикла — замкнутого маршрута, проходящего каждое ребро ровно один раз; эйлеровы циклы легко находить и строить за линейное время с помощью поиска в глубину.) Рассмотрим два доказательства NP-трудности задачи о гамильтоновом цикле в ориентированных графах.

Сведение из вершинного покрытия

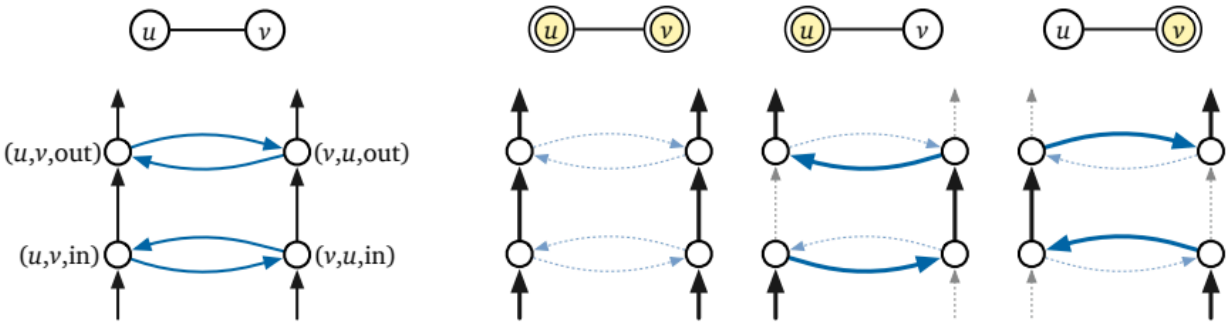
Первое доказательство использует задачу принятия решения о вершинном покрытии. По неориентированному графу G и целому k построим ориентированный граф H, который имеет гамильтонов цикл тогда и только тогда, когда G имеет вершинное покрытие размера k. Как и раньше, выходной граф H состоит из нескольких гаджетов, каждый из которых соответствует определённым особенностям входов G и k.



- Для каждого неориентированного ребра uv в G ориентированный граф H содержит *гаджет ребра* из четырёх вершин (u, v, in), (u, v, out), (v, u, in), (v, u, out) и шести ориентированных рёбер:

$$(u, v, \text{in}) \to (u, v, \text{out}), \quad (u, v, \text{in}) \to (v, u, \text{in}), \quad (v, u, \text{in}) \to (u, v, \text{in}),$$
$$(v, u, \text{in}) \to (v, u, \text{out}), \quad (u, v, \text{out}) \to (v, u, \text{out}), \quad (v, u, \text{out}) \to (u, v, \text{out}).$$

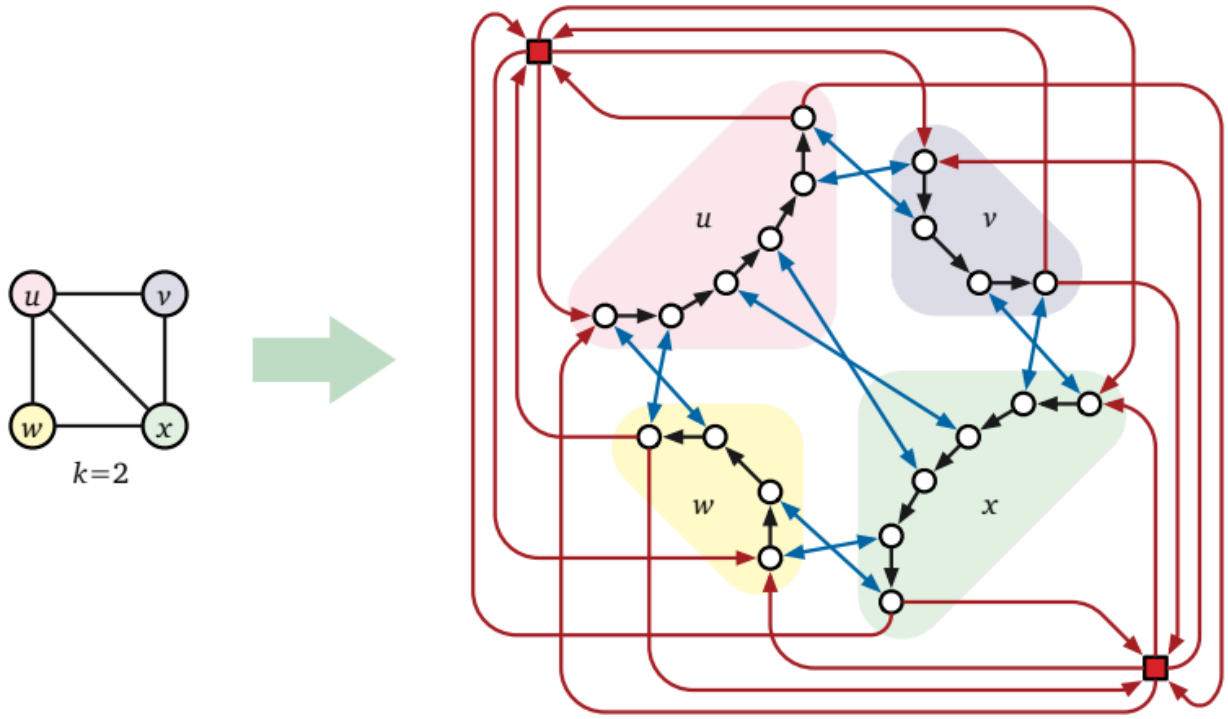
Гаджет показан на рисунке 12.16. Каждая вершина `in` имеет дополнительное входящее ребро, а каждая вершина `out` — дополнительное исходящее. Любой гамильтонов цикл в H должен проходить через гаджет одним из трёх способов: прямо по обеим сторонам либо с переходом с одной стороны на другую и обратно. Впоследствии эти варианты будут соответствовать включению в вершинное покрытие обеих вершин u, v, только u или только v.



- Для каждой вершины u в G все гаджеты инцидентных рёбер uv соединяются в H в один ориентированный путь, называемый *цепочкой вершины*. А именно, пусть у u соседи $v_1, \ldots, v_d$. Тогда H содержит $d - 1$ дополнительных рёбер $(u, v_i, \text{out}) \to (u, v_{i+1}, \text{in})$ для каждого i от 1 до $d - 1$.

- Наконец, H содержит k *вершин покрытия* $x_1, \ldots, x_k$. От каждой из них идёт ориентированное ребро к первой вершине каждой цепочки и ориентированное ребро от последней вершины каждой цепочки к ней.

На рисунке 12.17 приведён полный пример преобразования; каждый синий отрезок с двумя стрелками представляет пару ориентированных рёбер.

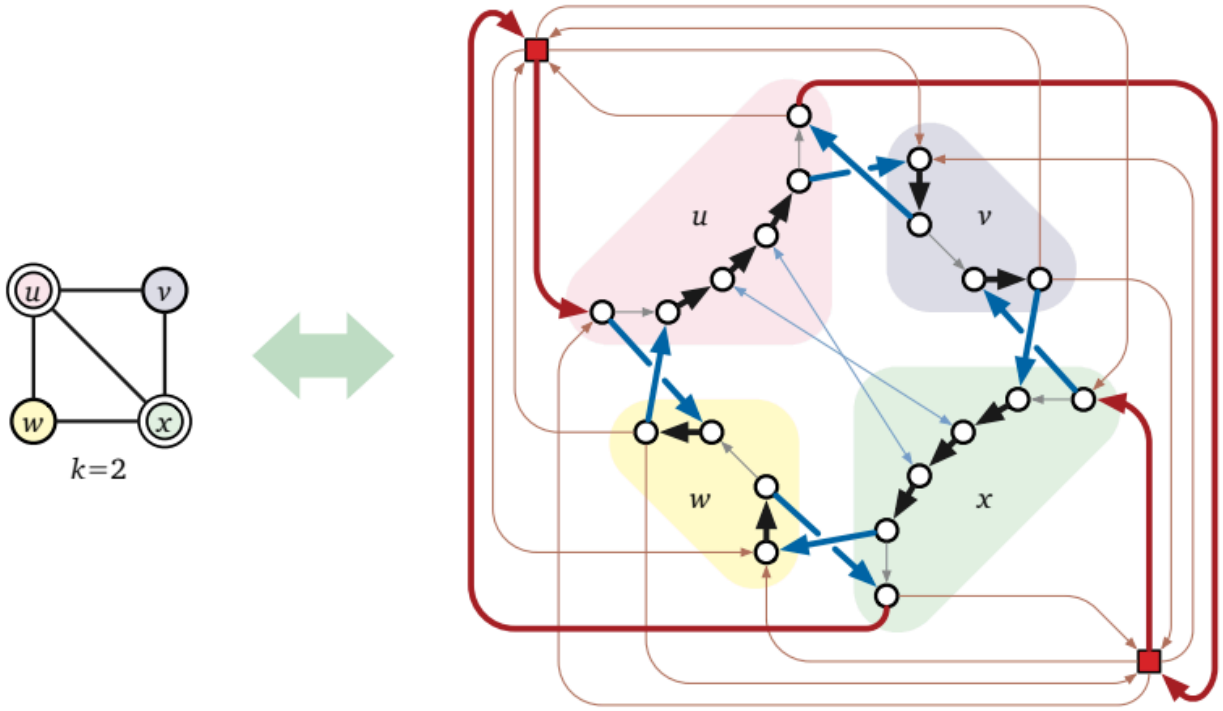


Как обычно, докажем корректность в два этапа.

⇒ Пусть $C = \{u_1, u_2, \ldots, u_k\}$ — вершинное покрытие G размера k. Построим гамильтонов цикл в H, «кодирующий» C. Для каждого i от 1 до k пройдём путь от вершины покрытия x_i через цепочку вершины u_i к вершине покрытия x_{i+1}, а при $i = k$ — к x_1. Проходя цепочку u_i, для каждой вершины (u_i, v, in) выбираем продолжение так:

- если $v \in C$, идём по ребру $(u_i, v, \text{in}) \to (u_i, v, \text{out})$;
- если $v \notin C$, делаем обход $(u_i, v, \text{in}) \to (v, u_i, \text{in}) \to (v, u_i, \text{out}) \to (u_i, v, \text{out})$.

Таким образом, для каждого ребра uv графа G гамильтонов цикл посещает (u, v, in) и (u, v, out) как часть цепочки u, если $u \in C$, и как часть цепочки v в противном случае. См. рисунок 12.18.



$\Leftarrow$ Пусть, наоборот, H содержит гамильтонов цикл C. Этот цикл должен содержать ребро от каждой вершины покрытия к началу некоторой цепочки. Разбор вариантов гаджета ребра по индукции показывает, что после входа в цепочку вершины u цикл должен пройти всю цепочку. А именно, в каждой вершине (u, v, in) цикл содержит либо одно ребро $(u, v, \text{in}) \to (u, v, \text{out})$, либо обход $(u, v, \text{in}) \to (v, u, \text{in}) \to (v, u, \text{out}) \to (u, v, \text{out})$, а затем ребро к следующему гаджету в цепочке u либо к вершине покрытия, если гаджет последний. В частности, если C содержит ребро обхода $(u, v, \text{in}) \to (v, u, \text{in})$, он не может содержать рёбра между какой-либо вершиной покрытия и цепочкой v. Следовательно, C проходит ровно k цепочек. Более того, эти цепочки задают вершинное покрытие исходного графа G, поскольку C посещает (u, v, in) для каждого ребра uv в G.

Итак, G имеет вершинное покрытие размера k тогда и только тогда, когда H содержит гамильтонов цикл. Преобразование G в H занимает не более $O(V^2)$; следовательно, задача о гамильтоновом цикле в ориентированном графе NP-трудна.

Сведение из 3SAT

NP-трудность ориентированного гамильтонова цикла можно доказать и непосредственным сведением из `3Sat`. По произвольной формуле Φ в 3-КНФ с n переменными $x_1, \ldots, x_n$ и k дизъюнктами $c_1, \ldots, c_k$ строим ориентированный граф H, содержащий гамильтонов цикл тогда и только тогда, когда Φ выполнима.

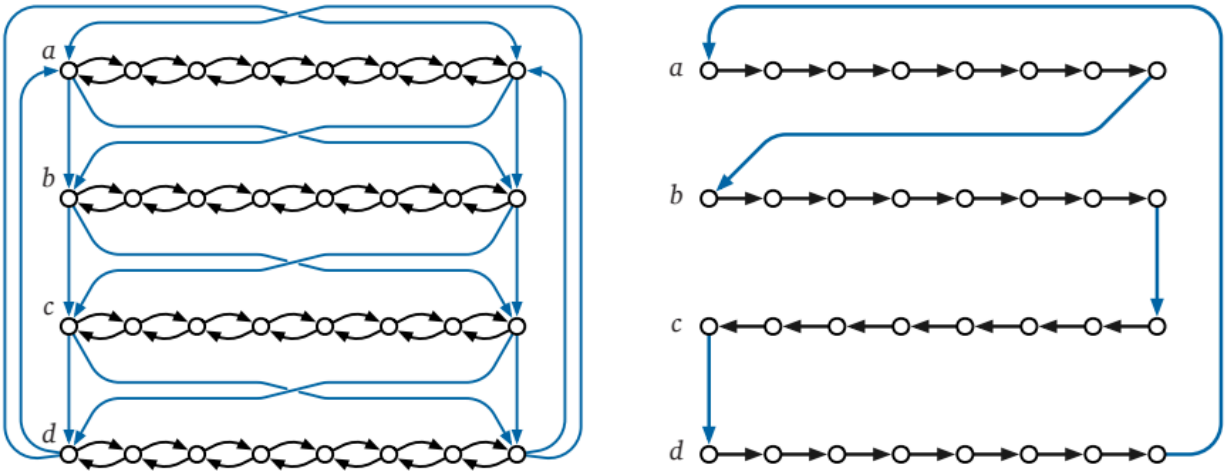
Для каждой переменной x_i построим *гаджет переменной* — двусвязный список вершин $(i, 0), (i, 1), \ldots, (i, 2k)$, соединённых рёбрами $(i, j-1) \to (i, j)$ и $(i, j) \to (i, j-1)$ для каждого j. Первую и последнюю вершины каждой пары соседних гаджетов соединяем рёбрами

$$(i,0) \to (i+1,0), \quad (i,2k) \to (i+1,0), \quad (i,0) \to (i+1,2k), \quad (i,2k) \to (i+1,2k)$$

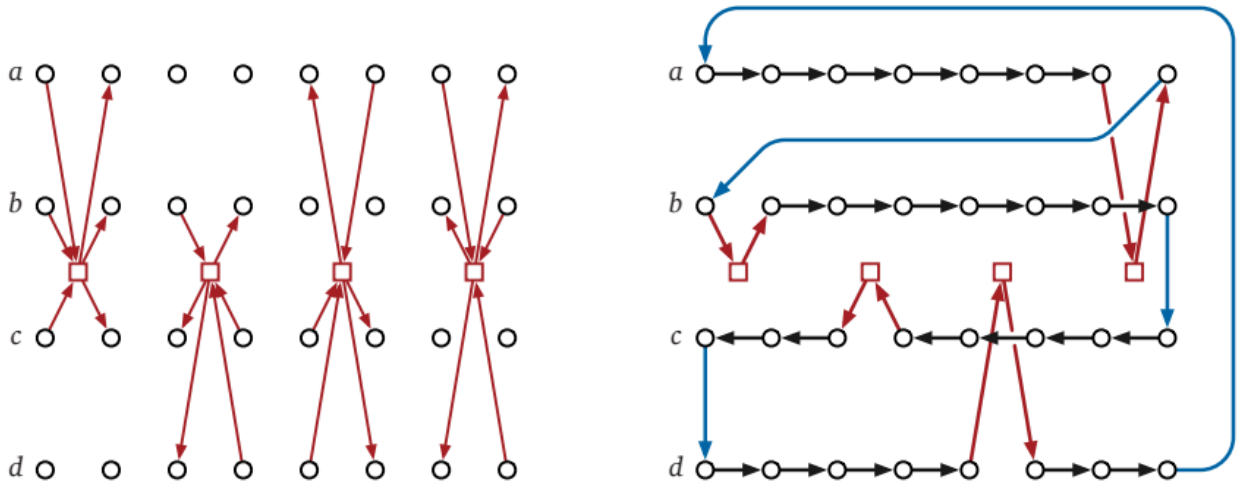
для каждого i; концы первого и последнего гаджетов также соединяем рёбрами

$$(n,0) \to (1,0), \quad (n,2k) \to (1,0), \quad (n,0) \to (1,2k), \quad (n,2k) \to (1,2k).$$

Полученный граф G имеет ровно 2^n гамильтоновых циклов, по одному для каждого набора булевых значений n переменных Φ. А именно, для каждого i мы проходим i-й гаджет слева направо, если $x_i = \text{True}$, и справа налево, если $x_i = \text{False}$. См. рисунок 12.19.



Теперь расширим G до графа H, добавив вершину дизъюнкта $[j]$ для каждого c_j и соединив её с гаджетами переменных шестью рёбрами, как на рисунке 12.20. Для каждого положительного литерала x_i в c_j добавим рёбра $(i,2j-1) \to [j] \to (i,2j)$, а для каждого отрицательного $\bar{x}_i$ — рёбра $(i,2j) \to [j] \to (i,2j-1)$. Эти соединения гарантируют, что гамильтонов цикл G можно расширить до гамильтонова цикла H тогда и только тогда, когда соответствующий набор переменных удовлетворяет Φ. Полный разбор случаев показывает, что H имеет гамильтонов цикл тогда и только тогда, когда Φ выполнима.



Преобразование Φ в H занимает $O(kn)$, то есть не более квадратичного времени от общей длины формулы; следовательно, задача об ориентированном гамильтоновом цикле NP-трудна.

Варианты и расширения

Тривиальные изменения предыдущих сведений показывают, что задача о гамильтоновом пути в ориентированных графах тоже NP-трудна. Гамильтонов путь в G — простой путь, посещающий каждую вершину G ровно один раз. На самом деле существуют простые полиномиальные сведения от задачи о гамильтоновом цикле к задаче о гамильтоновом пути и обратно. Подробности я оставляю в упражнениях.

Оба предыдущих сведения относятся к ориентированным графам, но соответствующая задача для неориентированных графов тоже NP-трудна. Существует сравнительно простое сведение ориентированного варианта к неориентированному; его подробности я тоже оставляю в упражнениях.

Наконец, печально известная *задача коммивояжёра* требует найти кратчайший гамильтонов цикл или путь в графе со взвешенными рёбрами. Поскольку найти кратчайший цикл или путь очевидно не проще, чем определить само его существование — рассмотрите граф, где каждое ребро имеет вес 1! — `TravelingSalesman` также NP-трудна.

12.12. Сумма подмножества (сведение из вершинного покрытия)

Следующая NP-трудная задача — `SubsetSum`, рассмотренная в главе 2: по множеству X положительных целых чисел и целому числу T определить, есть ли подмножество X, сумма элементов которого равна T.

Снова используем сведение из `VertexCover`. По графу G и числу k нужно вычислить множество X положительных целых и число T так, чтобы подмножество X с суммой T существовало тогда и только тогда, когда у G есть вершинное покрытие размера k. Преобразование использует всего два типа «гаджетов» — целые числа, представляющие вершины и рёбра G.

Произвольно пронумеруем рёбра G от 0 до $E-1$. Множество X содержит целое $b_i := 4^i$ для каждого ребра i и целое

$$a_v := 4^E + \sum_{i \in \Delta(v)} 4^i$$

для каждой вершины v, где $\Delta(v)$ — множество рёбер, имеющих v одним из концов. Или же можно представить каждое число из X как $(E+1)$-разрядное число в системе счисления с основанием 4. Разряд E равен 1, если число представляет вершину, и 0 в противном случае; для каждого $i < E$ разряд i равен 1, если число представляет ребро i либо один из его концов, и 0 иначе. Наконец, зададим целевую сумму

$$T := k \cdot 4^E + \sum_{i=0}^{E-1} 2 \cdot 4^i.$$

Докажем корректность сведения.

⇒ Пусть у G есть вершинное покрытие C размера k. Рассмотрим подмножество

$$X' := \{a_v \mid v \in C\} \cup \{b_i \mid \text{ребро } i \text{ имеет ровно один конец в } C\}.$$

Сумма элементов X', записанная по основанию 4, имеет старший разряд k, а все остальные разряды равны 2. Поэтому сумма элементов X' равна ровно T.

⇐ Пусть, наоборот, существует подмножество $X' \subseteq X$ с суммой T. Тогда

$$\sum_{v \in V'} a_v + \sum_{i \in E'} b_i = T$$

для некоторых $V' \subseteq V$ и $E' \subseteq E$. При сложении этих чисел по основанию 4 в первых E разрядах нет переносов, поскольку для каждого i в X только три числа имеют единицу в разряде i. Число ребра b_i даёт в этот разряд только одну единицу, а соответствующий разряд T равен 2. Следовательно, для каждого ребра G хотя бы один его конец должен принадлежать V'. Иными словами, V' — вершинное покрытие. С другой стороны, только числа вершин не меньше 4^E, а $\lfloor T/4^E \rfloor = k$, поэтому V' содержит не более k элементов. (Несложно увидеть, что на самом деле ровно k.)

Например, для графа $G = (V, E)$ с четырьмя вершинами, где $V = \{u, v, w, x\}$ и $E = \{uv, uw, vw, vx, wx\}$, множество X может содержать следующие числа по основанию 4:

$$\begin{array}{ll} a_u := 111000_4 = 1344 & b_{uv} := 010000_4 = 256 \\ a_v := 110110_4 = 1300 & b_{uw} := 001000_4 = 64 \\ a_w := 101101_4 = 1105 & b_{vw} := 000100_4 = 16 \\ a_x := 100011_4 = 1029 & b_{vx} := 000010_4 = 4 \\ & b_{wx} := 000001_4 = 1 \end{array}$$

Если нужно вершинное покрытие размера $k = 2$, целевая сумма равна $T := 222222_4 = 2730$. Действительно, покрытию $\{v, w\}$ соответствует подмножество $\{a_v, a_w, b_{uv}, b_{uw}, b_{vx}, b_{wx}\}$ с суммой $1300 + 1105 + 256 + 64 + 4 + 1 = 2730$.

Сведение очевидно выполняется за полиномиальное время. Мы уже доказали NP-трудность `VertexCover`, следовательно, `SubsetSum` тоже NP-трудна.

Осторожно, сведение!

Здесь необходимо подчеркнуть один тонкий момент. Триста с лишним страниц назад, в главе 3, мы разработали алгоритм динамического программирования для `SubsetSum` со временем $O(nT)$. Разве это не полиномиальный алгоритм? Разве мы только что не доказали $P = NP$? Эй, где мой миллион долларов?!

Увы, жизнь не так проста. Да, время работы полиномиально зависит от n и T, но для настоящей полиномиальности оно должно быть полиномиальной функцией *размера входа* — числа битов, необходимых для его представления. Значения элементов X и целевая сумма T могут быть экспоненциально больше числа входных битов. Именно описанное сведение

порождает T, экспоненциально большее размера исходного графа, из-за чего алгоритм динамического программирования работает за экспоненциальное время.

Говорят, что такие алгоритмы работают за *псевдополиномиальное время*, а любая NP-трудная задача с таким алгоритмом называется *слабо NP-трудной*. Эквивалентно, слабо NP-трудную задачу можно решить за полиномиальное время, когда все входные числа представлены унарно как суммы единиц, но она становится NP-трудной при двоичном представлении. Если задача NP-трудна даже при унарном представлении всех входных чисел, её называют *сильно NP-трудной*. Хороший пример — `TravelingSalesman`, которая остаётся NP-трудной даже для полного входного графа, где каждый вес ребра равен 1 или 2.

12.13. Другие полезные NP-трудные задачи

Доказана NP-трудность буквально тысяч задач. Здесь перечислю несколько NP-трудных задач, полезных при построении сведений.[11] Я не буду подробно излагать доказательства их NP-трудности, но большинство можно найти в классической «Страшной чёрной книге NP-полноты» Гэри и Джонсона.[12] Все рассмотренные до сих пор задачи и большинство перечисленных ниже впервые были доказаны NP-трудными в одной основополагающей статье Ричарда Карпа 1972 года.[13]

- `PlanarCircuitSat`: дана булева схема, которую можно уложить на плоскости без пересечения проводов. Существует ли вход, при котором схема выдаёт истину? NP-трудность доказывается сведением из общей выполнимости схем заменой каждого пересечения небольшим набором элементов.
- `1-in-3Sat`: дана формула в 3-КНФ. Существует ли набор значений, при котором каждый дизъюнкт содержит ровно один истинный литерал? NP-трудность доказывается сведением из обычной `3Sat`.
- `NotAllEqual3Sat`: дана формула в 3-КНФ. Существует ли набор значений, при котором каждый дизъюнкт содержит хотя бы один истинный и хотя бы один ложный литерал? Сведение — из обычной `3Sat`.
- `Planar3Sat`: для формулы в 3-КНФ рассмотрим двудольный граф, вершины которого — дизъюнкты и переменные, а ребро означает вхождение переменной или её отрицания в дизъюнкт. Если граф планарен, формула также называется планарной. Задача спрашивает, выполнима ли заданная планарная формула в 3-КНФ. NP-трудность доказывается сведением из `PlanarCircuitSat`.[14]
- `Exact3DimensionalMatching`, или `X3M`: даны множество S и коллекция его трёхэлементных подмножеств, называемых тройками. Есть ли подколлекция непересекающихся троек, в точности покрывающая S? NP-трудность доказывается сведением из `3Sat`, потому что в названии есть тройка.
- `Partition`: дано множество S из n целых чисел. Существуют ли подмножества A, B с $A \cup B = S$, $A \cap B = \varnothing$ и $\sum_{a \in A} a = \sum_{b \in B} b$? NP-трудность следует из простого сведения из `SubsetSum`. Как и `SubsetSum`, эта задача лишь слабо NP-трудна.
- `3Partition`: дано множество S из $3n$ целых чисел. Можно ли разбить его на n непересекающихся троек с одинаковой суммой элементов каждой? Несмотря на похожие названия, эта задача сильно отличается от `Partition`; извините, не я придумал названия.

NP-трудность следует из сведения из `X3M`, потому что там есть тройка. В отличие от `Partition`, задача сильно NP-трудна: она остаётся NP-трудной, даже если каждое входное число не превосходит n^3.

- `SetCover`: дана коллекция множеств $\mathcal{S} = \{S_1, \ldots, S_m\}$. Найти наименьшую подколлекцию, содержащую все элементы объединения $\bigcup_i S_i$. Это обобщение и `VertexCover`, и `X3M`.
- `HittingSet`: дана коллекция $\mathcal{S} = \{S_1, \ldots, S_m\}$. Найти минимальное число элементов объединения $\bigcup_i S_i$, пересекающих каждое множество коллекции. Это тоже обобщение `VertexCover`.
- `LongestPath`: дан граф G с неотрицательными весами рёбер, ориентированный или неориентированный, и две вершины u, v. Найти длиннейший простой путь из u в v. Путь прост, если посещает каждую вершину не более одного раза. Задача обобщает соответствующую задачу о гамильтоновом пути. Разумеется, соответствующая задача о кратчайшем пути решается за полиномиальное время.
- `SteinerTree`: дан взвешенный неориентированный граф G, некоторые вершины которого помечены. Найти поддерево минимального веса, содержащее все помеченные вершины. Если помечены все вершины, минимальное дерево Штейнера — минимальное остовное дерево; если ровно две — кратчайший путь между ними. NP-трудность доказывается сведением из `VertexCover`.
- `Max2Sat`: дана булева формула в КНФ, где каждый дизъюнкт содержит ровно два литерала. Найти набор переменных, максимизирующий число дизъюнктов хотя бы с одним истинным литералом. NP-трудность следует из сведения из `3Sat` — да, хотя тройки в названии нет. Более простая задача принятия решения `2Sat`, спрашивающая, можно ли удовлетворить все дизъюнкты, решается за полиномиальное время.
- `MaxCut`: дан неориентированный граф $G = (V, E)$. Найти подмножество $S \subset V$, максимизирующее число рёбер, ровно один конец которых принадлежит S. Равносильно требуется найти наибольший двудольный подграф G. NP-трудность следует из сведения из `Max2Sat`.

Помимо этих сухих, но полезных задач, для большинства интересных головоломок и пасьянсов доказана NP-трудность их самих или обобщений. (Можно утверждать, что если игра или головоломка хотя бы не NP-трудна, она неинтересна!) Вот знакомые вам примеры:

- «Сапёр» (сведение из `CircuitSat`);[15]
- судоку (в конечном счёте из `3Sat`);[16]
- тетрис (из `3Partition`);[17]
- пасьянс «Косынка» (из `3Sat`);[18]
- Pac-Man (из `HamiltonianCycle`);[19]
- Super Mario Brothers (из `3Sat`);[20]
- Candy Crush Saga (из варианта `3Sat`);[21]
- Threes/2048 (конечно, из `3Sat`);[22]
- Trainyard (из `DominatingSet`; см. упражнение 26);[23]
- кратчайшее решение кубика Рубика $n \times n \times n$ (из `3Sat` через частный случай `PlanarUndirectedHamCycle`);[24]

- Cookie Clicker (из `Partition` или `3Partition`).[25]

Этот список неизбежно неполон из-за ограниченного бюджета сносок.[26] По состоянию на июнь 2019 года никто не опубликовал доказательства NP-трудности обобщений Ultimate Paperclips, Line Rider, Twister или Cards Against Humanity, но, уверен, это лишь вопрос времени.

[11] Когда-нибудь придётся вам сведение найти; у меня есть небольшой список — список, который поможет тяжёлые задачи снизу оценить. Быстрых решений нет; доказательствам он даст ответ.

[12] Michael Garey, David Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Co., 1979. И да, она действительно чёрная.

[13] Позже поставлена вне Бродвея как «Ричард Карп и его 21 помощник», за что Карп заслуженно получил премию Тони — Тьюринга.

[14] Удивительно, но `PlanarNotAllEqual3Sat` решается за полиномиальное время!

[15] Richard Kaye. *Minesweeper is NP-complete*. Mathematical Intelligencer 22(2):9–15, 2000; см. также Allan Scott, Ulrike Stege, Iris van Rooij. *Minesweeper may not be NP-complete but is hard nonetheless*. Mathematical Intelligencer 33(4):5–17, 2011.

[16] Takayuki Yato, Takahiro Seta. *Complexity and completeness of finding another solution and its application to puzzles*. IEICE Transactions E86-A(5):1052–1060, 2003. http://www-imai.is.s.u-tokyo.ac.jp/~yato/data2/MasterThesis.pdf

[17] Ron Breukelaar et al. *Tetris is hard, even to approximate*. International Journal of Computational Geometry and Applications 14:41–68, 2004.

[18] Luc Longpré, Pierre McKenzie. *The complexity of Solitaire*. Proceedings of the 32nd International Mathematical Foundations of Computer Science, 182–193, 2007.

[19] Giovanni Viglietta. *Gaming is a hard job, but someone has to do it!* Theory of Computing Systems 54(4):595–621, 2014. http://giovanniviglietta.com/papers/gaming2.pdf

[20] Greg Aloupis, Erik D. Demaine, Alan Guo, Giovanni Viglietta. *Classic Nintendo games are (computationally) hard*. Theoretical Computer Science 586:135–160, 2015. https://arxiv.org/abs/1203.1895

[21] Luciano Gualà, Stefano Leucci, Emanuele Natale. *Bejeweled, Candy Crush and other match-three games are (NP-)hard*. Proc. 2014 IEEE Conference on Computational Intelligence and Games, 2014. https://arxiv.org/abs/1403.5830

[22] Stefan Langerman, Yushi Uno. *Threes!, Fives, 1024!, and 2048 are Hard*. Proc. 8th International Conference on Fun with Algorithms, 2016. https://arxiv.org/abs/1505.04274

[23] Matteo Almanza, Stefano Leucci, Alessandro Panconesi. *Trainyard is NP-Hard*. Proc. 8th International Conference on Fun with Algorithms, 2016. https://arxiv.org/abs/1603.00928

[24] Erik D. Demaine, Sarah Eisenstat, Mikhail Rudoy. *Solving the Rubik's Cube optimally is NP-complete*. Proc. 35th Symposium on Theoretical Aspects of Computer Science, 2018.

https://arxiv.org/abs/1706.06708

[25] Erik D. Demaine, Hiro Ito, Stefan Langerman, Jayson Lynch, Mikhail Rudoy, Kai Xiao. *Cookie Clicker*. Препринт, август 2018. https://arxiv.org/abs/1808.07540

[26] См. https://xkcd.com/1208/

12.14. Как выбрать подходящую задачу

Один из самых сложных шагов доказательства NP-трудности — выбор задачи, из которой удобнее всего делать сведение. Теорема Кука — Левина означает: если существует сведение из какой-либо NP-трудной задачи к задаче X, то существует сведение из каждой NP-полной задачи к X. Но с одними задачами работать проще, чем с другими. Систематического метода выбора нет, однако полезны следующие практические правила.

- Если задача требует присвоить объектам биты, выбрать подмножество объектов или разбить их на два подмножества, попробуйте сведение из какого-либо варианта `Sat` или из `Partition`.
- Если нужно присвоить объектам метки из небольшого фиксированного множества или разбить объекты на небольшое число подмножеств, попробуйте `kColor` или даже `3Color`.
- Если нужно расположить объекты в определённом порядке, попробуйте `DirectedHamCycle`, `DirectedHamPath` или `TravelingSalesman`.
- Если нужно найти малое подмножество, удовлетворяющее ограничениям, попробуйте `MinVertexCover`.
- Если нужно найти большое подмножество, удовлетворяющее ограничениям, попробуйте `MaxIndSet`, `MaxClique` или `Max2Sat`.
- Если нужно разбить объекты на большое число небольших подмножеств, попробуйте `3Partition`.
- Если в задаче естественно возникает число 3, попробуйте `3Sat`, `3Color`, `X3M` или `3Partition`. (Нет, это не шутка.)
- Если ничто другое не помогает, попробуйте `3Sat` или даже `CircuitSat`!

Не рекомендую начинать сведение с тетриса, SuperMarioBros или Trainyard. Лучше выбрать как можно более простую исходную задачу, которая всё же отражает некоторую особенность, делающую вашу задачу трудной.

12.15. Несерьёзный пример из реальной жизни

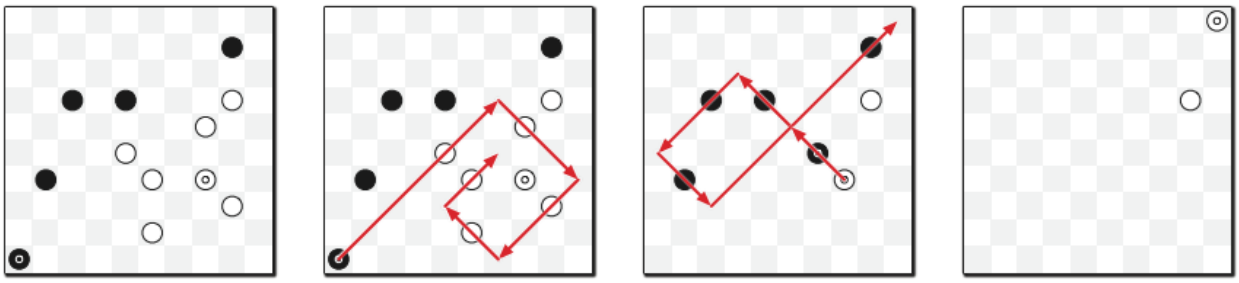
Шашки — семейство настольных игр, в которые играют тысячи лет. Большинство американцев знакомы с английскими шашками, но наиболее распространённый в мире вариант — международные, или польские, шашки — возник в Нидерландах в XVI веке. За полными правилами обратитесь к Википедии; здесь приведены несколько важных отличий от англо-американской игры:

- **Дальнобойные дамки.** Как и в английских шашках, шашка, закончившая ход на ближайшей к противнику горизонтали, становится дамкой и получает возможность

двигаться назад. Но в международных шашках дамка за один ход может пройти любое расстояние по диагонали, если промежуточные клетки пусты или содержат ровно одну шашку противника, которую она берёт.

- **Обязательное максимальное взятие.** Каждый ходящий игрок обязан взять как можно больше шашек противника. Это отличается от обязательного взятия в английских шашках, где игрок обязан брать при возможности, а ход со взятием заканчивается, только когда ходящая шашка больше не может брать. Иначе говоря, английские шашки требуют на каждом ходу брать локально максимальное множество шашек, а международные — глобально наибольшее.
- **Тонкости взятия.** Как и в английских шашках, взятые шашки снимаются с доски только в конце хода. Каждую шашку можно взять не более одного раза. Следовательно, после перепрыгивания шашка противника остаётся на доске, но до конца хода перепрыгнуть её повторно нельзя.

Например, в первой позиции ниже кружки обозначают шашки, а двойные кружки — дамки. Чёрные обязаны сделать первый указанный ход и взять пять белых шашек: взять больше пяти невозможно, а другого хода со взятием пяти нет. Продолжить взятие на северо-восток или юго-восток чёрные не могут, поскольку взятые белые шашки остаются на доске до окончания хода. Затем белые обязаны сделать второй указанный ход и выиграть.

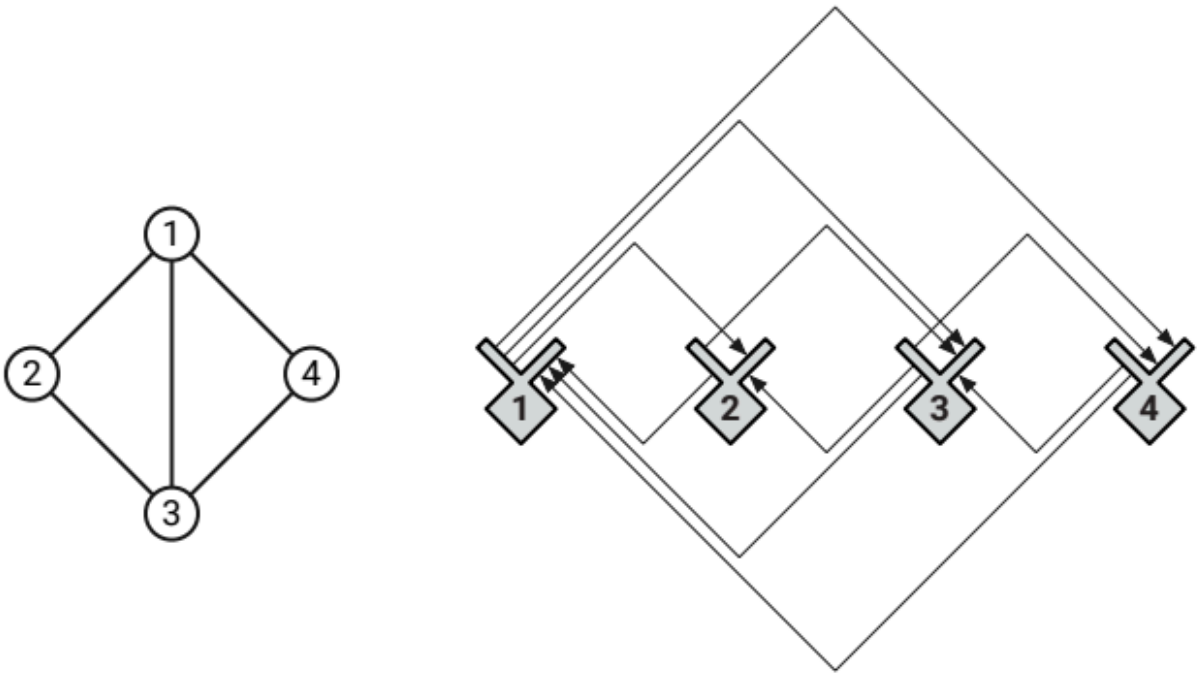


Обычная игра на доске 10×10 с 20 шашками каждого цвета вычислительно тривиальна: можно заранее вычислить оптимальный ход обоих игроков во всех возможных позициях и жёстко записать результаты в таблицу постоянного размера. Да, константа большая, но всё равно константа!

Но рассмотрим естественное обобщение международных шашек на доску $n \times n$. В этом случае даже поиск допустимого хода NP-труден! Следующее сведение из задачи о гамильтоновом цикле в графе нашёл Боб Херн в 2010 году.[27] В большинстве игр для двух игроков поиск лучшего хода NP-труден или ещё сложнее. Это единственный известный мне пример игры, причём настоящей игры, в которую веками играют миллионы людей, где NP-трудно просто следовать правилам!

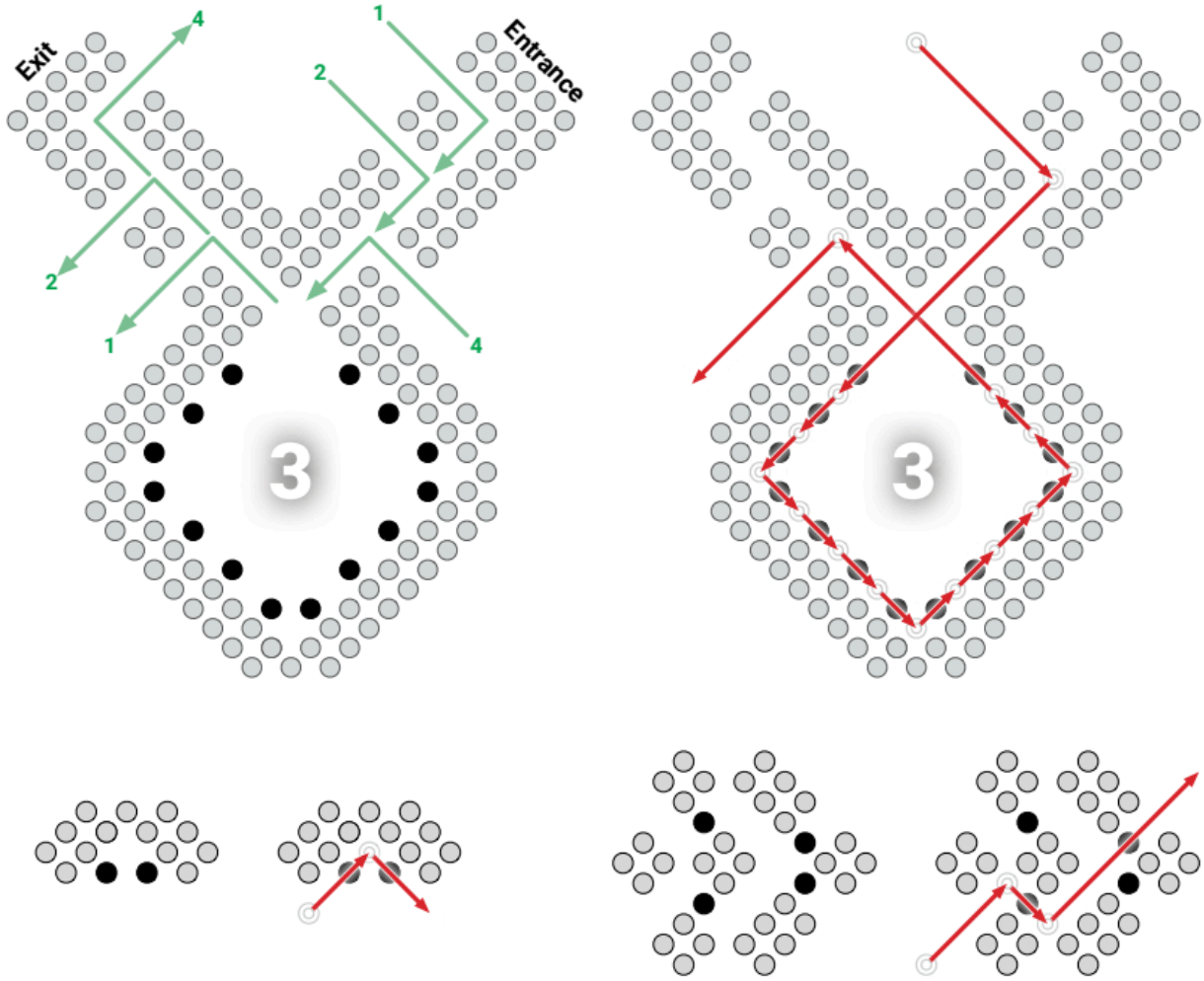
По неориентированному графу G с n вершинами построим позицию международных шашек, в которой белые могут взять за один ход определённое количество чёрных шашек тогда и только тогда, когда у G есть гамильтонов цикл. Будем рассматривать G как ориентированный граф с двумя дугами $u \to v$ и $v \to u$ вместо каждого неориентированного ребра uv. Произвольно пронумеруем вершины от 1 до n. Построенная позиция содержит несколько гаджетов:

- Вершины G представлены похожими на кроликов гаджетами вершин, равномерно расположенными на горизонтальной линии. Каждой дуге $i \to j$ соответствует путь из двух диагональных отрезков от «левого уха» гаджета вершины i к «правому уху» гаджета вершины j. Путь расположен над гаджетами, если $i < j$, и под ними, если $i > j$.



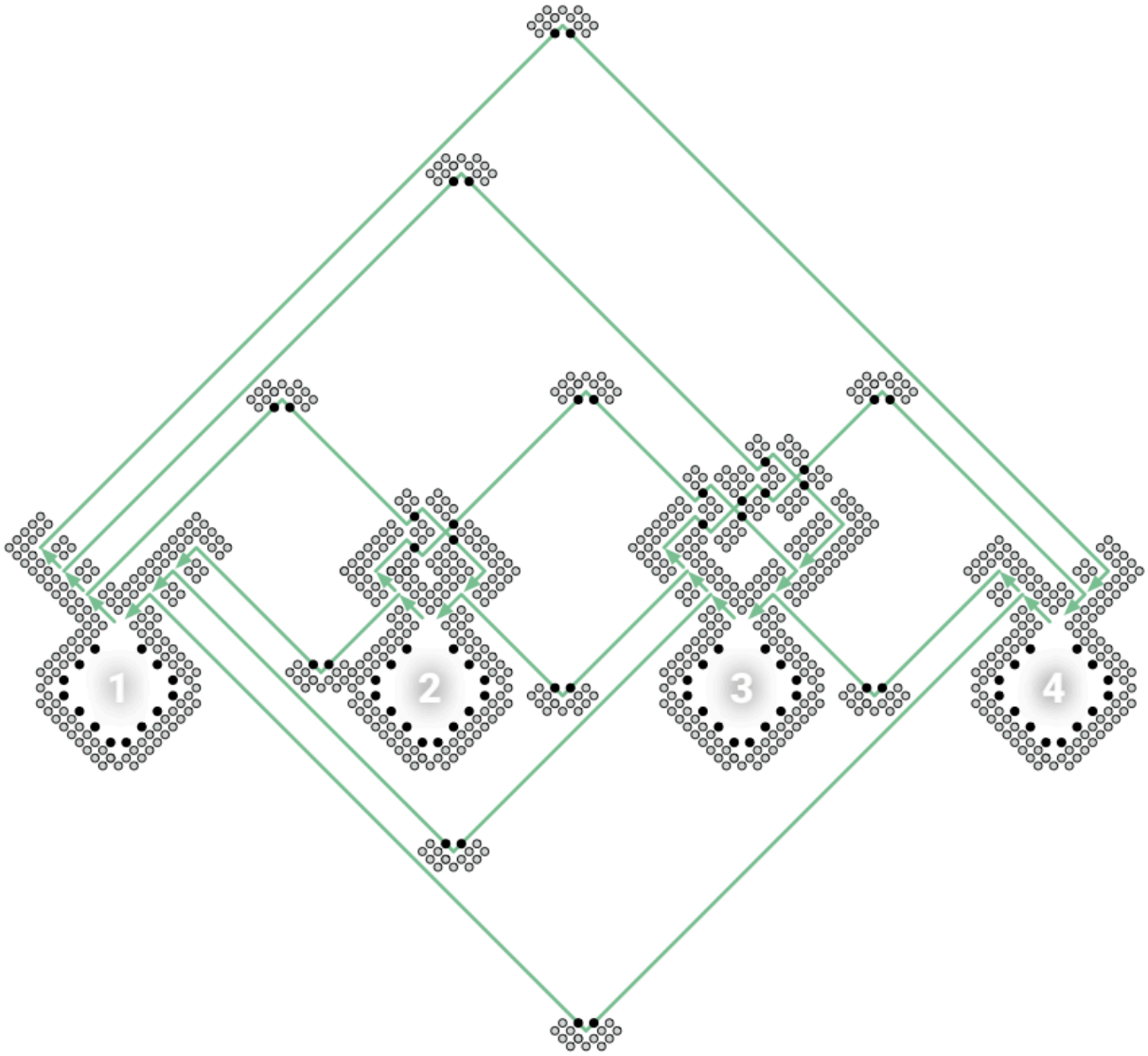
- Основная часть каждого гаджета вершины — ромбовидная область, называемая хранилищем. Его стены состоят из двух сплошных слоёв чёрных шашек, которые нельзя взять; на рисунках они обозначены серыми кружками. Внутри каждого хранилища находятся N доступных для взятия чёрных шашек для некоторого большого целого N, которое мы выберем позже. Белая дамка может войти через «правое ухо», взять все внутренние шашки и выйти через «левое ухо». Оба уха — коридоры со стенами толщиной в две шашки; там, где заканчиваются пути дуг, оставлены проходы для входа и выхода. Длины ушей легко подобрать в соответствии с другими гаджетами.
- Для каждой дуги $i \to j$ есть *угловой гаджет*, направляющий белую дамку из гаджета вершины i в гаджет вершины j.
- Наконец, в каждом месте пересечения двух путей дуг есть *гаджет пересечения*, позволяющий дамке следовать по любому из двух путей, но запрещающий переключаться с одного на другой.

Одна белая дамка начинает ход в нижнем углу одного из хранилищ. При любом допустимом ходе она должна поочерёдно полностью проходить пути дуг и очищать хранилища. Дамка может проходить гаджеты и в обратном направлении, входя в хранилище через выход и наоборот. Но обращение гамильтонова цикла в G тоже является гамильтоновым циклом, поэтому движение назад допустимо.



Если в G есть гамильтонов цикл, белая дамка может взять не менее nN чёрных шашек, посетив остальные хранилища и вернувшись в начальное. С другой стороны, если гамильтонова цикла нет, она может взять не более половины шашек начального хранилища и, таким образом, не более $(n - \frac{1}{2})N + O(n^3)$ шашек противника в общей сложности. Слагаемое $O(n^3)$ учитывает угловые гаджеты и гаджеты пересечений: каждое ребро проходит через один угловой гаджет и не более $n^2/2$ гаджетов пересечений.

Чтобы завершить сведение, положим $N = n^4$. В итоге получаем позицию на доске размера $O(n^5) \times O(n^5)$ с $O(n^5)$ чёрными шашками и одной белой дамкой. Очевидно, эту позицию можно построить перебором за полиномиальное время. Полный пример приведён на рисунке 12.25.



Остаётся открытым вопрос о NP-трудности близкой задачи: по позиции международных шашек на доске $n \times n$ определить, могут ли, а значит, обязаны ли белые взять все чёрные шашки и выиграть за один ход.

[27] См. Theoretical Computer Science Stack Exchange: http://cstheory.stackexchange.com/a/1999/111

♥ 12.16. И за пределами зебры

Классы P и NP — только первые две ступени огромной иерархии классов сложности. В завершение главы и книги опишем ещё несколько интересных классов.

Полиномиальная память

PSPACE — множество задач принятия решения, решаемых с использованием полиномиального объёма памяти. Каждая задача из NP, а значит, и из P, принадлежит

PSPACE. Обычно считается, что $NP \neq$ PSPACE, но никто не может доказать даже $P \neq$ PSPACE.

Задача Π называется PSPACE-*трудной*, если для любой задачи Π', решаемой с использованием полиномиальной памяти, существует полиномиальное многие-к-одному сведение из Π' в Π. Если какая-либо PSPACE-трудная задача принадлежит NP, то PSPACE $= NP$; аналогично, если такая задача принадлежит P, то PSPACE $= P$.

Каноническая PSPACE-трудная задача — задача об *истинности кванторной булевой формулы*, или `QBF`: дана булева формула с произвольным числом кванторов всеобщности и существования, но без свободных переменных; эквивалентна ли она истине? Например, допустимым входом `QBF` является выражение

$$\exists a:\ \forall b:\ \exists c:\ (\forall d : a \vee b \vee c \vee \bar{d}) \Leftrightarrow ((b \wedge \bar{c}) \vee (\exists e : (\bar{a} \Rightarrow e) \vee (c \neq a \wedge e))).$$

`Sat` эквивалентна частному случаю `QBF`, где формула содержит только кванторы существования $\exists$. Задача `QBF` остаётся PSPACE-трудной, даже если все кванторы стоят в начале, строго чередуются $\exists$ и $\forall$, а выражение после них записано в КНФ с ровно тремя литералами в каждом дизъюнкте, например:

$$\exists a:\ \forall b:\ \exists c:\ \forall d : \big((a \vee b \vee c) \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b} \vee \bar{d})\big).$$

Этот ограниченный вариант `QBF` можно сформулировать и как вопрос о стратегии двух игроков. Пусть Алисе и Бобу дана формула в 3-КНФ со свободными переменными $x_1, x_2, \ldots, x_n$. Игроки по очереди задают значения переменных в порядке индексов: Алиса — x_1, Боб — x_2 и так далее. В итоге Алиса задаёт все переменные с нечётными индексами, а Боб — с чётными. Алиса стремится сделать выражение истинным, Боб — ложным. Кто выиграет при идеальной игре обоих? Неудивительно, что большинство игр для двух игроков,[28] например крестики-нолики, реверси, шашки, го, шахматы и манкала, а точнее подходящие обобщения этих игр постоянного размера на доски произвольного размера, являются PSPACE-трудными.

Другой канонический пример — *тотальность недетерминированного конечного автомата*: дан НКА M над алфавитом Σ; допускает ли M каждую строку из Σ^*? Близкие задачи об эквивалентности НКА (допускают ли два автомата один язык?) и минимизации НКА (найти наименьший автомат, допускающий тот же язык) тоже PSPACE-трудны, как и соответствующие вопросы о регулярных выражениях. (Соответствующие задачи для детерминированных конечных автоматов решаются за полиномиальное время.)

[28] Обзор известных результатов о вычислительной сложности игр и головоломок, хороший, хотя неизбежно устаревающий, см. в монографии Erik Demaine и Bob Hearn *Games, Puzzles, and Computation* (CRC Press, 2009).

Экспоненциальное время

Следующий существенно больший класс сложности, EXP, также называемый EXPTIME, — множество задач принятия решения, решаемых за экспоненциальное время, то есть не более чем за 2^{n^c} шагов при некоторой константе $c > 0$. Каждая задача из PSPACE, а значит, и из NP и P, принадлежит EXP. Обычно считается, что PSPACE $\subsetneq$ EXP, но никто не может

доказать даже $NP \neq \mathrm{EXP}$. Задача Π называется EXP-трудной, если любую задачу, решаемую за экспоненциальное время, можно свести к ней за полиномиальное время. Если какая-либо EXP-трудная задача принадлежит PSPACE, то $\mathrm{EXP} = \mathrm{PSPACE}$; аналогично принадлежность NP означала бы $\mathrm{EXP} = NP$. Известно, что $P \neq \mathrm{EXP}$; в частности, никакая EXP-трудная задача не принадлежит P.

Естественные обобщения многих интересных игр для двух игроков, включая шашки, шахматы, манкалу и го, на самом деле EXP-трудны. Граница между PSPACE-полными и EXP-трудными играми довольно тонка. Например, в стандартных шахматах 8×8 есть три способа завершить партию вничью: пат (игроку, который ходит, не поставлен шах, но у него нет допустимых ходов), троекратное повторение позиции или пятьдесят ходов без взятия и движения пешки. Обобщение шахмат на доску $n \times n$ принадлежит PSPACE или является EXP-трудным в зависимости от обобщения этих правил. Если объявлять ничью, скажем, после n^3 ходов без взятия, любая партия завершается за полиномиальное число ходов, поэтому все партии из заданной позиции можно перебрать с полиномиальной памятью. Если же полностью игнорировать правило ходов без взятия, партия может длиться экспоненциальное число ходов и очевидного способа обнаружить повторение позиции с полиномиальной памятью нет; действительно, этот вариант шахмат $n \times n$ является EXP-трудным.

Всё выше!

Разумеется, даже экспоненциальное время — не конец истории. NEXP — класс задач принятия решения, решаемых за недетерминированное экспоненциальное время; равносильно, задача принадлежит NEXP тогда и только тогда, когда для каждого экземпляра с ответом «да» существует доказательство, проверяемое за экспоненциальное время. $\mathrm{EXPSPACE}$ — множество задач принятия решения, решаемых с использованием экспоненциальной памяти. Даже в этих более крупных классах есть трудные задачи: например, если добавить в синтаксис регулярных выражений операцию пересечения $\cap$, определение того, задают ли два выражения один язык, становится $\mathrm{EXPSPACE}$-трудным. За $\mathrm{EXPSPACE}$ следуют классы с двукратно экспоненциальными ограничениями ресурсов (EEXP, NEEXP, $\mathrm{EEXPSPACE}$), затем с трёхкратно экспоненциальными (EEEXP, NEEEXP, $\mathrm{EEEXPSPACE}$) и так далее до бесконечности.

Все эти классы можно упорядочить по включению:

$$P \subseteq NP \subseteq \mathrm{PSPACE} \subseteq \mathrm{EXP} \subseteq \mathrm{NEXP} \subseteq \mathrm{EXPSPACE} \subseteq \mathrm{EEXP} \subseteq \mathrm{NEEXP} \subseteq \cdots .$$

Большинство специалистов по сложности твёрдо убеждены, что каждое включение строгое, то есть никакие два из этих классов не равны. Но сильнейший доказанный результат утверждает, что каждый класс в цепочке строго содержится в классе, находящемся на три шага дальше. Например, доказано $P \neq \mathrm{EXP}$ и $\mathrm{PSPACE} \neq \mathrm{EXPSPACE}$, но не известно, отличаются ли P и PSPACE либо NP и EXP.

Пределом этой последовательности всё более экспоненциальных классов является класс $\mathrm{ELEMENTARY}$ задач принятия решения, решаемых с ограничением времени или памяти функцией вида $2 \uparrow^k n$ для некоторого постоянного целого k, где

$$2 \uparrow^k n := \begin{cases} n, & k = 0, \\ 2^{2 \uparrow^{k-1} n}, & \text{иначе.} \end{cases}$$

Например, $2 \uparrow^1 n = 2^n$, а $2 \uparrow^2 n = 2^{2^n}$.

Может показаться, что всякую естественную разрешимую задачу можно решить за элементарное время, но это предположение неверно. Рассмотрим обобщённые регулярные выражения, рекурсивно построенные из возможно пустых строк над конечным алфавитом с помощью конкатенации (xy), объединения $(x + y)$, замыкания Клини (x^*) и отрицания $(\bar{x})$. Например, $\overline{(0+1)^*00(0+1)^*}$ задаёт множество строк из $\{0,1\}^*$, не содержащих двух нулей подряд. Можно алгоритмически определить, описывают ли два таких выражения один язык: рекурсивно преобразовать каждое выражение в эквивалентный НКА, затем НКА в ДКА и минимизировать ДКА. Однако время этого алгоритма имеет неэлементарную оценку $2 \uparrow^{\Theta(n)} 2$, поскольку каждый уровень рекурсивного отрицания может экспоненциально увеличить число состояний. В 1974 году Ларри Стокмайер доказал, что эту задачу нельзя решить за элементарное время, даже если запретить замыкание Клини.

Упражнения

1. (а) Опишите и проанализируйте алгоритм решения `Partition` за $O(nM)$, где n — размер входного множества, а M — сумма абсолютных значений его элементов.

(б) Почему из существования этого алгоритма не следует $P = NP$?

2. Рассмотрим задачу `BoxDepth`: дано множество из n прямоугольников на плоскости со сторонами, параллельными осям. Каков размер наибольшего подмножества прямоугольников, содержащих общую точку?

(а) Опишите полиномиальное сведение `BoxDepth` к `MaxClique`.

(б) Опишите и проанализируйте полиномиальный алгоритм для `BoxDepth`. [Подсказка: время $O(n^3)$ должно получиться легко, но возможно и $O(n \log n)$.]

(в) Почему из этих двух результатов не следует $P = NP$?

3. Булева формула находится в *дизъюнктивной нормальной форме* (ДНФ), если она состоит из дизъюнкции нескольких термов, каждый из которых — конъюнкция одного или нескольких литералов. Например, формула $(\bar{x} \wedge y \wedge \bar{z}) \vee (y \wedge z) \vee (x \wedge \bar{y} \wedge \bar{z})$ находится в ДНФ. Задача `DNF-Sat` спрашивает, выполнима ли заданная формула в ДНФ.

(а) Опишите полиномиальный алгоритм решения `DNF-Sat`.

(б) В чём ошибка следующего рассуждения, что $P = NP$?

Пусть дана булева формула в КНФ с не более чем тремя литералами в каждом дизъюнкте и нужно выяснить её выполнимость. По закону дистрибутивности можно построить эквивалентную формулу в ДНФ. Например,

$$(x \vee y \vee \bar{z}) \wedge (\bar{x} \vee \bar{y}) \iff (x \wedge \bar{y}) \vee (y \wedge \bar{x}) \vee (\bar{z} \wedge \bar{x}) \vee (\bar{z} \wedge \bar{y}).$$

Теперь алгоритм из пункта (а) за полиномиальное время определяет выполнимость полученной ДНФ. Мы решили `3Sat` за полиномиальное время; поскольку `3Sat` NP-трудна, заключаем, что $P = NP$!

4. Задача `AllOrNothing3Sat` спрашивает, существует ли для заданной булевой формулы в 3-КНФ набор переменных, при котором каждый дизъюнкт содержит либо три истинных, либо три ложных литерала.

(а) Опишите полиномиальный алгоритм для `AllOrNothing3Sat`.

(б) Но `3Sat` NP-трудна! Почему существование этого алгоритма не доказывает $P = NP$?

5. (а) Дан волшебный чёрный ящик, который за полиномиальное время определяет длину кратчайшего гамильтонова цикла произвольного взвешенного графа G. Опишите и проанализируйте полиномиальный алгоритм, который с помощью этого ящика как подпрограммы вычисляет сам кратчайший гамильтонов цикл заданного взвешенного графа.

(б) Дан волшебный чёрный ящик, за полиномиальное время определяющий число вершин наибольшего полного подграфа произвольного графа G. Опишите и проанализируйте полиномиальный алгоритм, который с помощью ящика вычисляет полный подграф максимального размера.

(в) Дан волшебный чёрный ящик, за полиномиальное время определяющий, можно ли раскрасить произвольный граф G в три цвета. Опишите и проанализируйте полиномиальный алгоритм, который с помощью ящика строит правильную трёхцветную раскраску заданного графа либо правильно сообщает, что её нет. [Подсказка: вход чёрного ящика — граф. Только граф. Вершины и рёбра. Ничего больше.]

(г) Дан волшебный чёрный ящик, за полиномиальное время определяющий выполнимость произвольной булевой формулы Φ. Опишите и проанализируйте полиномиальный алгоритм, который с помощью ящика находит удовлетворяющий набор заданной формулы либо правильно сообщает, что его нет.

(д) Дан волшебный чёрный ящик, за полиномиальное время определяющий, можно ли произвольное множество X положительных целых разбить на множества A, B с $\sum A = \sum B$. Опишите и проанализируйте полиномиальный алгоритм, который с помощью ящика строит разбиение заданного множества на части с равными суммами либо правильно сообщает, что его нет.

♣♥ (е) Дан волшебный чёрный ящик, за полиномиальное время определяющий, соответствует ли произвольному обобщённому регулярному выражению R, определённому непосредственно перед упражнениями, хотя бы одна строка. Опишите и проанализируйте полиномиальный алгоритм, который с помощью ящика находит одну соответствующую строку либо правильно сообщает, что такой строки нет.

6. В числе 3 есть нечто особенное.

(а) Опишите и проанализируйте полиномиальный алгоритм для `2Partition`: по множеству S из $2n$ положительных целых определить, можно ли разбить элементы на n непересекающихся

пар с одинаковыми суммами.

(б) Опишите и проанализируйте полиномиальный алгоритм для `2Color`: по неориентированному графу G определить, допускает ли он правильную раскраску всего двумя цветами.

(в) Опишите и проанализируйте полиномиальный алгоритм для `2Sat`: по булевой формуле Φ в КНФ с ровно двумя литералами в каждом дизъюнкте определить её выполнимость. [Подсказка: задача тесно связана с темами предыдущей главы.]

7. В числе 3 нет ничего особенного.

(а) `12Partition`: по множеству S из $12n$ положительных целых определить, можно ли разбить элементы на n подмножеств по 12 элементов с одинаковыми суммами. Докажите NP-трудность. [Подсказка: сведите из `3Partition`; возможно, сначала проще рассмотреть мультимножества.]

(б) `12Color`: по неориентированному графу G определить, можно ли раскрасить вершины в 12 цветов так, чтобы концы каждого ребра имели разные цвета. Докажите NP-трудность. [Подсказка: сведите из `3Color`.]

(в) `12Sat`: по булевой формуле Φ в КНФ с ровно 12 литералами в каждом дизъюнкте определить её выполнимость. Докажите NP-трудность. [Подсказка: сведите из `3Sat`.]

8. Есть два варианта задачи о гамильтоновом цикле: для ориентированных и неориентированных графов. Выше приведены два доказательства NP-трудности ориентированного варианта.

(а) Опишите полиномиальное сведение неориентированного варианта к ориентированному. Докажите его корректность.

(б) Опишите полиномиальное сведение ориентированного варианта к неориентированному. Докажите его корректность.

(в) Какое из этих сведений показывает NP-трудность неориентированного варианта?

9. (а) Опишите полиномиальное сведение `UndirectedHamiltonianCycle` к `DirectedHamiltonianCycle`.

(б) Опишите полиномиальное сведение `DirectedHamiltonianCycle` к `UndirectedHamiltonianCycle`.

10. (а) Опишите полиномиальное сведение `HamiltonianPath` к `HamiltonianCycle`.

(б) Опишите полиномиальное сведение `HamiltonianCycle` к `HamiltonianPath`. [Подсказка: полиномиальное сведение может вызывать подпрограмму-чёрный ящик более одного раза, но не обязано.]

11. Рассмотрим следующие тонко различающиеся варианты `CNFSat`. Во всех входом служит булева формула Φ в КНФ, а задача — определить её выполнимость.

(а) Пусть каждый дизъюнкт Φ содержит не более трёх литералов, а каждая переменная входит не более чем в три дизъюнкта. Докажите NP-трудность этого варианта.

(б) Пусть каждый дизъюнкт содержит ровно три литерала, а каждая переменная входит не более чем в четыре дизъюнкта. Докажите NP-трудность этого варианта `3Sat`. [Подсказка: сначала решите пункт (а).]

♥ (в) Пусть каждый дизъюнкт может содержать произвольное число литералов, но каждая переменная входит не более чем в два дизъюнкта. Опишите полиномиальный алгоритм этого варианта `CNFSat`.

♥ (г) Пусть каждый дизъюнкт содержит ровно три литерала, а каждая переменная входит не более чем в три дизъюнкта. Докажите, что Φ обязательно выполнима. (Таким образом, этот вариант `3Sat` совершенно тривиален!)

12. (а) Докажите NP-трудность `NotAllEqual3Sat`.

(б) Докажите NP-трудность `1-in-3Sat`.

13. Булева формула в *исключающей конъюнктивной нормальной форме* (ХКНФ) — конъюнкция нескольких дизъюнктов, каждый из которых является исключающим ИЛИ нескольких литералов; иными словами, дизъюнкт истинен тогда и только тогда, когда содержит нечётное число истинных литералов. Задача `XCNF-Sat` спрашивает, выполнима ли заданная формула в ХКНФ. Либо опишите полиномиальный алгоритм для `XCNF-Sat`, либо докажите её NP-трудность. [Подсказка: не пытайтесь делать и то и другое.]

♥ **14.** Рассмотрим вариант `3Sat`, называемый `Majority3Sat`. Как и в `3Sat`, входом является булева формула Φ в КНФ с ровно тремя литералами на дизъюнкт. Требуется определить, существует ли набор значений переменных, при котором каждый дизъюнкт содержит хотя бы два истинных литерала.

Либо опишите полиномиальный алгоритм для `Majority3Sat`, либо докажите NP-трудность. [Подсказка: не пытайтесь делать и то и другое.]

♠♥ **15.** Для любого $X \subseteq \{0,1,2,3\}$ рассмотрим задачу `X-3Sat`. Вход — булева формула Φ в КНФ с ровно тремя литералами в каждом дизъюнкте. Нужно определить, существует ли набор значений переменных, при котором число истинных литералов каждого дизъюнкта принадлежит X. Например:

- $\{1,2,3\}$-`3Sat` — обычная `3Sat`;
- $\{0,3\}$-`3Sat` совпадает с `AllOrNothing3Sat` (см. упражнение 4);
- $\{1,2\}$-`3Sat` обычно называется `NotAllEqual3Sat` (см. упражнение 12(а));
- $\{1\}$-`3Sat` обычно называется `1-in-3Sat` (см. упражнение 12(б));
- $\{1,3\}$-`3Sat` обычно называется `XCNF-3Sat` (см. упражнение 13);
- $\{2,3\}$-`3Sat` обычно называется `Majority3Sat` (см. упражнение 14).

Перечислите все подмножества $X \subseteq \{0,1,2,3\}$, для которых `X-3Sat` решается за полиномиальное время, считая, что $P \neq NP$. [Подсказка: не приводите 16 отдельных рассуждений.]

16. Докажите NP-трудность следующих задач.

(а) Содержит ли заданный неориентированный граф G простой путь, посещающий все вершины, кроме 17?

(б) Есть ли у заданного неориентированного графа G остовное дерево, каждая вершина которого имеет степень не более 23?

(в) Есть ли у заданного неориентированного графа G остовное дерево не более чем с 42 листьями?

(г) Дан неориентированный граф $G = (V, E)$. Каков максимальный размер подмножества $S \subseteq V$, такого что не более 374 рёбер E имеют оба конца в S?

(д) Дан неориентированный граф $G = (V, E)$. Каков максимальный размер подмножества $S \subseteq V$, такого что каждая вершина S имеет не более 473 соседей в S?

(е) Можно ли раскрасить вершины заданного неориентированного графа G тремя цветами так, чтобы не более 31337 рёбер имели концы одного цвета?

17. Докажите NP-трудность следующих вариантов задачи о минимальном остовном дереве.

(а) По графу G вычислить остовное дерево максимального диаметра. (Диаметр дерева T — длина наибольшего пути в T.)

(б) По графу G со взвешенными рёбрами вычислить остовное дерево поиска в глубину минимального веса.

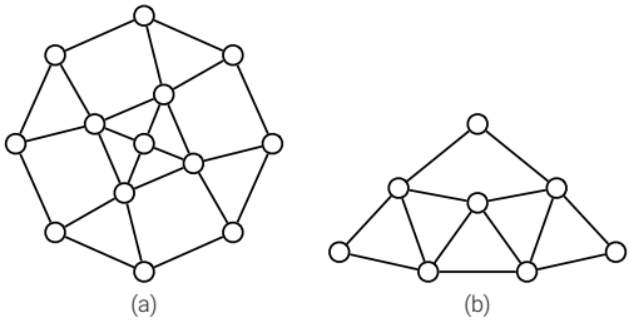
(в) По графу G со взвешенными рёбрами и подмножеству вершин S вычислить остовное дерево минимального веса, все листья которого принадлежат S.

(г) По графу G со взвешенными рёбрами и целому ℓ вычислить остовное дерево минимального веса не более чем с ℓ листьями.

(д) По графу G со взвешенными рёбрами и целому Δ вычислить остовное дерево минимального веса, все вершины которого имеют степень не более Δ.

18. (а) Используя гаджет на рисунке 12.26(а), докажите NP-трудность определения, раскрашивается ли заданный планарный граф в три цвета. [Подсказка: покажите, что гаджет можно раскрасить в три цвета, затем соответствующим образом замените гаджетами пересечения в плоском рисунке графа.]

(б) Используя пункт (а) и гаджет на рисунке 12.26(б), докажите NP-трудность определения, раскрашивается ли планарный граф максимальной степени 4 в три цвета. [Подсказка: замените каждую вершину степени более 4 набором соединённых гаджетов так, чтобы все степени были не больше четырёх.]



19. Докажите NP-трудность `PlanarCircuitSat`. [Подсказка: постройте гаджет пересечения проводов.]

20. (а) Опишите полиномиальное сведение `3Sat` к `4Sat`.

(б) Опишите полиномиальное сведение `4Sat` к `3Sat`.

♥ **21.** Опишите прямое полиномиальное сведение `4Color` к `3Color`. (Это гораздо сложнее сведения в обратную сторону.)

22. Костяшка домино — прямоугольник 1×2, разделённый на два квадрата, на каждом из которых записано целое число.[29] При допустимом размещении костяшки выложены в линию торцами друг к другу так, что числа на соседних торцах совпадают.

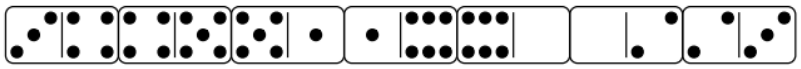


Рисунок 12.27. Допустимое размещение домино, в котором каждое целое число от 0 до 6 встречается дважды.

Для каждой задачи либо опишите полиномиальный алгоритм, либо докажите NP-трудность:

(а) Дан произвольный набор D костяшек. Можно ли допустимо разместить все костяшки из D?

(б) Дан произвольный набор D костяшек. Существует ли допустимое размещение костяшек из D, в котором каждое целое число от 1 до n встречается ровно дважды?

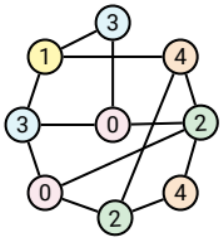
[29] Обычно вместо чисел на костяшках изображены точки, как на игральных костях. В стандартном домино на каждой стороне от 0 до 6 точек, хотя можно купить наборы с 9 или даже 12; мы допускаем произвольные целочисленные метки. Стандартный набор содержит ровно одну костяшку для каждой неупорядоченной пары меток; мы не предполагаем этого свойства у входных наборов.

♥ (в) Дан произвольный набор D костяшек. Какое наибольшее число костяшек из D можно использовать для допустимого размещения?

23. *Игра с камешками* — игра для одного человека на неориентированном графе G, где на каждой вершине лежит ноль или больше камешков. Один ход состоит в удалении двух камешков из вершины v и добавлении одного камешка произвольному соседу v. (Очевидно, перед ходом в v должно лежать хотя бы два камешка.)

Задача `PebbleDestruction` спрашивает, существует ли для заданного графа $G = (V, E)$ и числа камешков $p(v)$ на каждой вершине последовательность ходов, после которой останется всего один камешек. Докажите NP-трудность `PebbleDestruction`.

24. Напомним, что 5-раскраска графа G присваивает каждой вершине «цвет» из $\{0, 1, 2, 3, 4\}$ так, что для каждого ребра uv цвета u и v различаются. 5-раскраска называется *осторожной*, если цвета соседних вершин не только различны, но и отличаются более чем на 1 по модулю 5. Докажите NP-трудность определения, имеет ли заданный граф осторожную 5-раскраску. [Подсказка: сведите из обычной задачи `5Color`.]

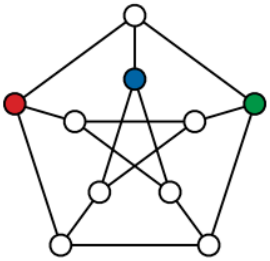


25. (а) Подмножество вершин S неориентированного графа G называется *полунезависимым*, если каждая вершина S смежна не более чем с одной другой вершиной S. Докажите NP-трудность поиска размера наибольшего полунезависимого множества заданного неориентированного графа.

(б) Подмножество S называется *отчасти независимым*, если каждая его вершина смежна не более чем с 374 другими вершинами S. Докажите NP-трудность поиска размера наибольшего такого множества.

(в) Подмножество S называется *почти независимым*, если не более 374 рёбер G имеют оба конца в S. Докажите NP-трудность поиска размера наибольшего такого множества.

26. Пусть $G = (V, E)$ — граф. *Доминирующее множество* в G — подмножество вершин S, такое что каждая вершина G либо принадлежит S, либо смежна с вершиной S. Задача `DominatingSet` спрашивает, содержит ли заданный граф G доминирующее множество размера k. Докажите NP-трудность этой задачи.



27. Подмножество вершин S неориентированного графа G называется *свободным от треугольников*, если для любых трёх вершин $u, v, w \in S$ хотя бы одно из рёбер uv, uw, vw отсутствует в G. Докажите NP-трудность поиска размера наибольшего свободного от треугольников подмножества вершин заданного неориентированного графа.

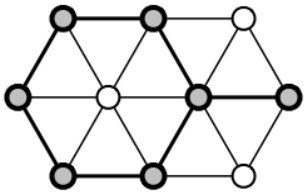
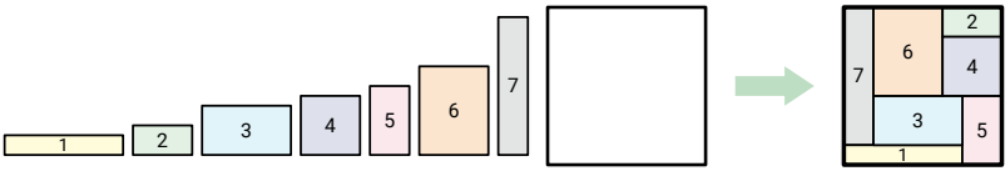


Рисунок 12.30. Свободное от треугольников множество из 7 вершин. Оно не является наибольшим в этом графе.

28. Задача `RectangleTiling`: даны один большой прямоугольник и несколько меньших; определить, можно ли разместить меньшие внутри большого без пробелов и наложений.

(а) Докажите NP-трудность `RectangleTiling`.

(б) Докажите сильную NP-трудность `RectangleTiling`.



29. (а) Подмножество вершин B графа G называется *множеством Бёрра*, если после удаления из G всех вершин B остаётся подграф, не содержащий гамильтонова пути. Докажите NP-трудность поиска наименьшего множества Бёрра в заданном графе.

(б) Подмножество вершин S графа G называется *множеством Скайлера*, если после удаления из G всех вершин S остаётся подграф, содержащий гамильтонов путь. Докажите NP-трудность поиска наименьшего множества Скайлера в заданном графе.

30. (а) *Тонов путь* в графе G — путь, проходящий хотя бы через половину вершин G. Докажите NP-трудность определения существования такого пути.

(б) *Тонов цикл* в графе G — цикл, проходящий хотя бы через половину вершин G. Докажите NP-трудность определения существования такого цикла. [Подсказка: используйте пункт (а). Или нет.]

31. Пусть G — неориентированный граф со взвешенными рёбрами. *Тяжёлый гамильтонов цикл* — цикл C, проходящий через каждую вершину G ровно один раз, причём суммарный вес его рёбер больше половины суммарного веса всех рёбер G. Докажите NP-трудность определения, есть ли у графа тяжёлый гамильтонов цикл.

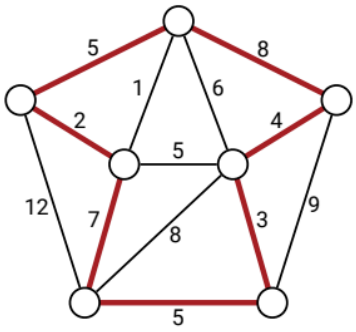


Рисунок 12.32. Тяжёлый гамильтонов цикл. Суммарный вес цикла — 34, всего графа — 67.

32. Для каждой задачи либо опишите полиномиальный алгоритм, либо докажите NP-трудность.

(а) *Дважды эйлеров обход* неориентированного графа G — замкнутый маршрут, проходящий каждое ребро G ровно два раза. По заданному G определить, есть ли такой обход.

(б) *Дважды гамильтонов обход* неориентированного графа G — замкнутый маршрут, посещающий каждую вершину G ровно два раза. По заданному G определить, есть ли такой обход.

(в) *Дважды гамильтонов контур* неориентированного графа G — замкнутый маршрут, посещающий каждую вершину ровно два раза и проходящий каждое ребро не более одного раза. По заданному G определить, есть ли такой контур.

(г) *Трижды эйлеров обход* неориентированного графа G — замкнутый маршрут, проходящий каждое ребро ровно три раза. По заданному G определить, есть ли такой обход.

(д) *Трижды гамильтонов обход* неориентированного графа G — замкнутый маршрут, посещающий каждую вершину ровно три раза. По заданному G определить, есть ли такой обход.

33. В этом упражнении нужно доказать корректность сведения `VertexCover` к `SteinerTree`. Пусть требуется найти наименьшее вершинное покрытие неориентированного графа $G = (V, E)$. Построим граф $H = (V', E')$:

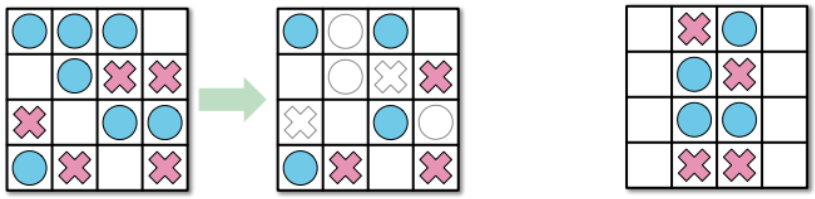
- $V' = V \cup E \cup \{z\}$;
- $E' = \{ve \mid v \in V$ — конец ребра $e \in E\} \cup \{vz \mid v \in V\}$.

Иными словами, построим H, подразделив каждое ребро G новой вершиной, а затем соединив все исходные вершины G с новой вершиной-апексом z.

Докажите, что G имеет вершинное покрытие размера k тогда и только тогда, когда у H есть поддерево из $k + |E| + 1$ вершин, содержащее все вершины $E \cup \{z\}$.

34. Рассмотрим игру для одного человека. Головоломка представляет собой сетку $n \times m$, каждая клетка которой либо пуста, либо занята красным или синим камнем. Нужно удалить некоторые камни так, чтобы оставшиеся удовлетворяли двум условиям: (1) в каждой строке

есть хотя бы один камень; (2) ни один столбец не содержит камней обоих цветов. Некоторые начальные позиции неразрешимы.



Слева — разрешимая головоломка и одно из её решений; справа — неразрешимая головоломка.

Докажите NP-трудность определения разрешимости заданной начальной позиции красных и синих камней.

35. Каждая из следующих игр использует сетку $n \times m$, в которой каждая клетка пуста либо содержит камень. За один ход можно удалить все камни произвольного столбца.

(а) Докажите NP-трудность поиска наименьшего подмножества столбцов, очистка которых оставляет не более одного камня в каждой строке.

(б) Докажите NP-трудность поиска наибольшего подмножества столбцов, очистка которых оставляет хотя бы один камень в каждой строке.

♥ (в) Докажите NP-трудность определения, существует ли подмножество столбцов, очистка которых оставляет ровно один камень в каждой строке.

36. Джефф старается сделать студентов счастливыми. В начале курса он раздаёт анкету с возможными правилами курса по вопросам, в которых готов проявить гибкость. Каждый студент отвечает на каждый вопрос одним из трёх вариантов: «полностью поддерживаю», «в целом безразлично», «категорически против». Каждый выбирает «полностью поддерживаю» или «категорически против» не более чем для пяти вопросов. Поскольку студенты Джеффа очень понимающие, студент счастлив тогда и только тогда, когда хотя бы одно из его сильных предпочтений удовлетворено. Либо опишите полиномиальный алгоритм установления правил курса, максимизирующего число счастливых студентов, либо докажите NP-трудность задачи.

37. Вы отвечаете за хореографию мюзикла местного любительского театра, и пришло время придумать финальную позу большого заключительного номера. («Трамвай!») Вы решили, что все n актёров в конце песни выстроятся в одну большую линию, вытянув руки и эффектно растопырив пальцы.

Режиссёр постановил, что в финале каждый актёр должен направить обе руки либо вверх, либо вниз; вам нужно определить, кто куда их направит. Режиссёр также дал список расстановок, которые расстроят его тонкую художественную натуру. Каждая запрещённая расстановка задаётся подмножеством актёров и положениями их рук; например: «Мардж не может направлять руки вверх, когда Нед, Апу и Смитерс направляют руки вниз».

Докажите NP-трудность поиска допустимой расстановки рук.

38. На следующей вечеринке один из гостей предложит сыграть в трёхсторонний мамблтипег — игру на ловкость, для которой нужны три команды и нож. Официальные правила трёхстороннего мамблтипега, утверждённые на Священном Римском соборе трёхстороннего мамблтипега в 1625 году, требуют: (1) в каждой команде есть хотя бы один человек; (2) любые два человека одной команды знакомы; (3) каждый зритель состоит в одной из трёх команд. Разумеется, вечеринка будет очень весёлой и никто не захочет уходить. На вечеринке найдётся несколько пар незнакомых друг с другом людей. Хозяин, наслышанный о ваших выдающихся алгоритмических способностях, вручит вам список знакомых пар и попросит выбрать команды, пока сам точит нож.

Либо опишите и проанализируйте полиномиальный алгоритм определения, можно ли разделить гостей на три допустимые команды, либо докажите NP-трудность задачи.

39. Вечеринка идёт великолепно, но теперь пора построиться для «Алгоритмического марша» (アルゴリズムこうしん)! Этот танец придумал японский комический дуэт Itsumo Kokokara (いつもここから) для детской телепередачи PythagoraSwitch (ピタゴラスイッチ). Марш исполняется цепочкой людей; каждый начинает определённую последовательность движений на один такт позже стоящего непосредственно перед ним. Таким образом, это танцевальный эквивалент музыкального канона, например «Row Row Row Your Boat» или «Frère Jacques».

Правила приличия требуют, чтобы каждый участник был знаком с человеком непосредственно перед ним: иначе маленькая ошибка может привести к ужасному конфузу между незнакомцами. Пусть дан полный список пар знакомых гостей. Докажите NP-трудность определения наибольшего числа гостей, которые могут участвовать в «Алгоритмическом марше». Без ограничения общности можно считать, что на вечеринке нет ниндзя.

♣ **40.** Докажите NP-трудность следующих задач о недетерминированных конечных автоматах и регулярных выражениях:

(а) Дан НКА M над алфавитом $\Sigma = \{0, 1\}$. Есть ли строка из Σ^*, которую M не допускает?

(б) Дан ациклический НКА M над алфавитом $\Sigma = \{0, 1\}$. Какова длина самой короткой строки из Σ^*, которую M не допускает?

(в) Дано регулярное выражение R над алфавитом $\Sigma = \{0, 1\}$. Есть ли строка из Σ^*, которой R не соответствует?

(г) Дано регулярное выражение R без звезды Клини над алфавитом $\Sigma = \{0, 1\}$. Какова длина самой короткой строки из Σ^*, которой R не соответствует?

(На самом деле задачи (а) и (в) PSPACE-полны; даже доказательство их принадлежности PSPACE нетривиально.)

♣ **41.** (а) Опишите полиномиальный алгоритм: по НКА M над алфавитом $\Sigma = \{0, 1\}$ определить, существует ли строка из Σ^*, которую M допускает.

(б) Опишите полиномиальный алгоритм: по регулярному выражению R над алфавитом $\Sigma = \{0, 1\}$ определить, существует ли строка из Σ^*, которой соответствует R.

(в) Дополнение любого регулярного языка тоже регулярно. Почему эти два алгоритма вместе с результатами об NP-трудности из упражнения 40 не доказывают, что $P = NP$?

42. Харону нужно перевезти n недавно умерших через реку Ахерон в Аид. Некоторые пары этих людей — заклятые враги, которым нельзя оставаться вместе ни на одном берегу без Харона. (Если враги останутся одни, один украдёт обол изо рта другого, обречя того вечно скитаться призраком по берегам Ахерона. Скажем просто: это очень плохо.) Паром вмещает не более k человек, включая Харона, и только Харон умеет им управлять.[30]

Докажите NP-трудность определения, может ли Харон перевезти всех n человек через Ахерон невредимыми, не считая того, что они, ну, умерли. На вход подаются целые k, n и граф G из n вершин, описывающий пары врагов. Результат — `True` или `False`. Пожалуйста, не пишите решение на классической латыни.

[30] Эта задача обобщает известную задачу о волке, козе и капусте, впервые известную по замечательной средневековой рукописи *Propositiones ad Acuendos Juvenes* («Задачи для развития юных умов»).

XVIII. Задача о человеке, козе и волке. Человеку нужно было перевезти через реку волка, козу и связку капусты. Однако он нашёл лишь лодку, способную вмещать двух из них, включая самого человека. Ему было велено перевезти всё невредимым. Кто может, пусть скажет, как они смогли переправиться целыми?

Решение. Сначала я перевёз бы козу, оставив волка и капусту на другом берегу. Затем вернулся бы и перевёз волка; оставив его на берегу, взял бы козу в лодку и отвёз обратно. Оставив козу, перевёз бы капусту. Затем снова переправился бы и привёз козу. Так переправа прошла бы благополучно, без угрозы расправы.

Вероятнее всего, автор *Propositiones* — плодовитый английский учёный VIII века Алкуин Йоркский. Свидетельства его авторства несколько косвенны; однако из его переписки с Карлом Великим известно, что он послал императору несколько «простых арифметических задач для забавы». Большинство современных исследователей считает, что даже если Алкуин и написал *Propositiones*, он не придумал все задачи сам, а собрал их из ещё более ранних источников.

Некоторые вещи никогда не меняются.

Источники иллюстраций

> *Умело выбранная иллюстрация почти необходима, чтобы закрепить истину в обычном сознании, и ни один преподаватель не может пренебрегать этой частью подготовки.*
>
> *— Говард Кросби (около 1880 года)*
>
> *Один показ стоит сотни рассказов.*
>
> *— Алан Уоттс, неточно цитируя китайскую пословицу, «Путь дзэн» (1957)*
>
> *Пожалуйста, не думайте, что это нейтральный вопрос и что единственное преимущество отказа от картинок — экономия места. Иллюстрации в учебниках на самом деле мешают процессу обучения.*
>
> *— Невилл Мартин Гуинн, Gwynne's Grammar (2013)*

Все рисунки этой книги, включая обложку оригинала, являются работами автора, кроме перечисленных ниже. Все перечисленные произведения находятся в общественном достоянии, если не указано иное. Номера страниц относятся к английскому оригиналу.

- Рис. 0.1 (стр. 5) — Biblioteca nazionale Braidense, Милан: http://atena.beic.it/webclient/DeliveryManager?pid=2953344.
- Рис. 0.2 (стр. 5) — Internet Archive: https://archive.org/details/archimedisopera05eutogoog/page/n377.
- Рис. 1.16 (стр. 45) — Internet Archive: https://archive.org/details/p1rcrationsm00lucauoft/page/162.
- Рис. 1.25 (стр. 61) — создан по портрету автора, нарисованному мелками Тиной Эриксон в 2000 году; включён с разрешения художницы.
- Рис. 5.1 (стр. 188) — Wikimedia Commons: https://commons.wikimedia.org/wiki/File:Tabula_Peutingeriana_-_Miller.jpg.
- Рис. 5.2 (стр. 189) — галерея «юридических деревьев», опубликованная юридической библиотекой Йельского университета по лицензии Creative Commons: https://www.flickr.com/photos/yalelawlibrary/albums/72157621954683764.
- Рис. 5.3 (стр. 189) — Internet Archive: https://archive.org/details/A077240124/page/n261.
- Упражнения 5.20 (стр. 216) и 8.22 (стр. 304) — оригинальные головоломки автора, вдохновлённые книгой Jason Batterson и Shannon Rogers, *Beast Academy Math: Practice 3A*, 2012: https://beastacademy.com/pdf/3A/printables/AngleMazes.pdf, https://www.beastacademy.com/resources/printables.php.
- Рис. 10.1 (стр. 328) — T. E. Harris, F. S. Ross. *Fundamentals of a method for evaluating rail net capacities*. RAND Corporation, Research Memorandum RM-1517, 24 октября 1955 года. Произведение правительства США, находящееся в общественном достоянии: http://www.dtic.mil/dtic/tr/fulltext/u2/093458.pdf.

Сведения об оригинальном издании

> *1. Имейте что сказать.*
> *2. Скажите это.*
> *3. Остановитесь, сказав всё необходимое.*
> *4. Дайте работе подходящее название.*
>
> *— Джон Шоу Биллингс, выступление о медицинской литературе, Международный медицинский конгресс, Лондон (1881)*
>
> *Знаете, я мог бы написать книгу. И она была бы достаточно толстой, чтобы оглушить быка.*
>
> *— Лори Андерсон, Let X=X, альбом Big Science (1982)*

Оригинал книги редактировался в TeXShop версии 4.27 и был набран с помощью pdfLaTeX из MacTeX-2018, с использованием класса документа memoir, стиля глав madsen, стиля заголовков komalike и стиля страниц Ruled, нескольких стандартных пакетов, включая amsmath, babel, enumitem, imakeidx, mathdesign, microtype и standalone, а также смущающего количества собственных настроек и ухищрений с TeX. Текст набран шрифтами Bitstream Charter, Ἀρτεμισία, Roboto и Inconsolata. За исключением перечисленного в разделе об источниках иллюстраций, все рисунки автор создал в OmniGraffle Pro, экспортировал в PDF и включил с помощью пакета LaTeX graphicx.

Часть нашей программы была воспроизведена механически, и на этом мы завершаем эфирный день.

Указатели оригинального издания

На следующих страницах воспроизведены указатели из английского издания. Их номера страниц относятся к оригиналу; имена, термины и названия алгоритмов сохранены на английском языке.

If we had an index file, we could look it up in the index file under "index file".

— Tegan Jovanka [Janet Fielding], "Castrovalva (Part 1)", *Doctor Who*, Season 19 (January 4, 1982)

I started with the phone book. Looking up "mensa" was not going to be easy, what with having to follow the strict alphabetizing rules that are so common nowadays. I prefer a softer, more fuzzy alphabetizing scheme, one that allows the mind to float free and "happen" upon the word. There is pride in that. The dictionary is a perfect example of over-alphabetization, with its harsh rules and every little word neatly in place. It almost makes me never want to eat again.

— Steve Martin, "How I Joined Mensa", *The New Yorker*, July 21, 1997.

Index

For some topics with multiple references, bold page numbers indicate the primary reference. Humans and pseudocode are indexed separately.

1-IN-3SAT, 405
2048 (game), 407
2COLOR, 417
2PARTITION, 417
2SAT, 406, 417
3CNF formula, 388
3COLOR, 395
  reduction from 3SAT, 395
3PARTITION, 405
3SAT, 388
  reduction from CIRCUITSAT, 388, 393, 395
  reduction to 3COLOR, 395
  reduction to DIRECTEDHAMCYCLE, 400
  reduction to MAXINDSET, 390, 393, 395
  rule of three, 395, 405, 408
4:20, 161

academic job market, 170
active vertex (depth-first search), 228
acyclic graph (= forest), 191
acyclic maximum flow, 339, 345

ad-hoc networks, 375
addition chains, 94
  increment and double only, 181
additional recurrence parameter, 295, 313, 318
adjacency matrix, 361
adjacent vertices, 191
airline scheduling, 362
alternating path, 356
amortized analysis, 264, 267
The Announcer's Test, 17
antanairesis, *see* Euclid's algorithm
Antarctica, 123, 143, 181
APSP, *see* shortest paths, all-pairs
arbitrage, 321
arithmetic takes time, 104, 325
arpedonaptai, 7, 317
arrow notation ($a \uparrow^b c$), 415
articulation point, *see* cut vertex
artificial source vertex, 227, 312, 355, 367
Āryabhaṭa's pulverizer, *see* Euclid's algorithm
assignment, *see* matching, tuple selection
augmenting path, 332
"average case" analysis, 31

B-tree, 147
back edge (depth-first search), 229
backtracking, **71**
  recursive brute force, 80
  sequence of decisions, 79
  summary of past decisions, 79
backward induction, *see* dynamic programming
Baguenaudier, 45
balanced brackets, 148, 184
"The Barley Mow", 18
base case, 23
baseball elimination, 363
BEAMILLIONAIREANDNEVERPAYTAXES
  reduction from BEAMILLIONAIRE, 10
Bellman-Ford, 289
  as dynamic programming, 294
  Moore's variant, 292
Bellman-Kalaba, *see* Bellman-Ford
Bellman-Shimbel, *see* Bellman-Ford
Bellman's equation, *see* recurrence
"best case" analysis, 31
best-first search, 202
  Dijkstra's algorithm, 288
  Jarník's algorithm, 264
  widest-path algorithm (Edmonds-Karp), 340, 348
BFS, *see* breadth-first search
binary search trees, 67
  AA trees, 96, 145
  AVL trees, 96, 144, 341
  optimal, 91
  reconfiguration, 68
  red-black trees, 96, 144
    left-leaning, *see* AA trees
binary to decimal conversion, 56
bipartite graph, 207
bipartite maximum matching, 355
bitonic, 60
black box, 12, 21, 22, 416
  see also none of your business
Bob's mama sees a ukulele, 129
bond (minimal edge cut), 347
boolean circuits, 190, 379
boolean formula, 386
boolean matrix multiplication, 323
Borůvka's algorithm, 261
  advantages, 262
Boston Pool algorithm, 172
bottleneck, *see also* minimum cut
bottleneck distance, 270, 347
bottleneck spanning tree, 348
breadth-first search, 202, **278**, 341
Bridges of Königsburg, 190
  see also Euler Tour

BST, *see* binary search trees
Bubba sees a banana, 128
bus scheduling, 362

c_f (residual capacity), 331
Camelot, 185
Candy Crush Saga, 407
capacity scaling, 351
careful graph coloring, 422
central vertex of a tree, 64
checkerboard, 59, 222, 371
checkers, *see* draughts
children's songs, 16
choosing the right problem to reduce from, 407
circuit satisfiability, *see* CircuitSat
CircuitSat, 380
  reduction to 3Sat, 388, 393, 395
  reduction to Sat, 386
circulation, 337
clause, 388
clause gadget, 390, 395, 396, 401
clique, 394
closed walk in a graph, 191
CNF, *see* conjunctive normal form
CNF-Sat, *see* Sat
co-NP, 381
compass and straightedge, 7
component, 191
computationes canonica et legalis, 188
condensation, *see* strong component graph
configuration graph, 194, 220, 221
conjunctive normal form, 388
connected component, *see* component
connected graph, 191
conservation constraint, 328
convenience, 32, 79, 227, 237, 279, 396
Cook reduction, 384
Cookie Clicker, 407
counting graph components, 204
cover gadget, 399
cross edge (depth-first search), 229
crossing gadget, 410
cursus publicus, 188
cut (vertex partition), 329
cut capacity $\|S, T\|$, 329
cut vertex, 245
cycle cover, 371
cycle flow, 337
cycle in a graph, 191

dag, *see* directed acyclic graph
Dance Dance Revolution, 133
data structures for graphs, *see* graphs
decision problem, 381
decision tree, 53
decision versus optimization, 85
degree of a vertex, 191
DeNile, 146
dependency graph, 107, 109, 114, 119, 193, 234, 366
depth-first order, *see* preorder, postorder
depth-first search, 76, 201, **225**, 282, 380
DFA, *see* finite-state automaton
DFS, *see* depth-first search
Dijkstra's algorithm, 203, 284
  with negative edges, 288
    exponential running time, 289, 300
  with no negative edges, 286
Dinic's [Dinitz's] algorithm, 341
directed acyclic graph, 192, 231
directed cycle, 192
directed graph, 190
DirectedHamCycle, 398
  reduction from 3Sat, 400

reduction from VERTEXCOVER, 398
DIRECTEDHAMPATH, 401
in a directed acyclic graph, 250
in a tournament, 208
reduction to shortest simple path, 275
directed path, 192
directed walk, 192
disconnected graph traversal, 203
disjoint paths
edge-disjoint, 353
vertex-disjoint, 354
disjoint-path cover, 253
in directed acyclic graphs, 360
NP-hard in general graphs, 360
disjoint-set data structure, 267, 271
disjunctive normal form, 416
$dist_{\leq i}(v)$ (length of shortest walk to v with at most i edges), 291
$dist(u, v)$, 309
$dist(v)$ (tentative distance), 276
distance multiplication, *see* min-plus matrix multiplication
distance tables, 310
divide and conquer, 26, 29, **31**, 37, 91, 315, 323
domain transformation, 34, 56
DOMINATINGSET, 407, 422
in interval graphs, 178
dominos, 371, 421
Don't try to be clever, 80, 82, 87, 89
DP, *see* Deadpool, dynamic programming
Dr. Seuss [Theodor Suess Giesel]
On Beyond Zebra, 167
Dr. Seuss [Theodore Suess Giesel]
The Cat in the Hat Comes Back, 21
draughts, 100, 408
English ("checkers"), 222
international, 408
reduction from UNDIRECTEDHAMCYCLE, 409
drinking songs, 1, 16, 18
duplation and mediation, 5, 21, 23, 43
dynamic programming, **97**, **101**, 178, 183, 185, 222, 282
as postorder traversal, 234, 255
before Bellman, 101
boilerplate, 106
in directed acyclic graphs, 235, 250
in trees, 120
not *always* better than memoization, 117
sequential, 105
space optimization, 102
tree-shaped, 117

edge (pair of vertices), 187
edge capacity, 329
edge contraction, 269
edge demands, 347
edge gadget, 398, 402, 410
edge reweighting, 311
edge-complement $\overline{G}$, 394
edge-disjoint paths, 353
edit distance, 111, 139, 149, 193, 235, 237
Edmonds-Karp algorithms
fattest augmenting paths, 340
shortest augmenting paths, 341
EDVAC, 27
Egyptian multiplication, *see* duplation and mediation
ELEMENTARY, 415
elves, *see* Recursion Fairy
empty edge (flows), 329

endpoints of an edge, 191
epiphany, 166, 242
errors, vii
escape problem, 369
Ethiopian peasant multiplication, *see* duplation and mediation
Euclid's algorithm, 57
Euler tour, 190, 208, 223, 398
evaluation order, 107, 109, 114, 119
  as postorder, 235
  single and double arrows, 110, 120
Exact3DimensionalMatching, 405
EXP (exponential time), 413
EXP-hard, 414
exponential decay, 336, 340, 349, 351
exponentiation, 42

$|f|$ (flow value), 328
factorial, 57
fake-sugar-packet game, 74
Fantastic Mr. Fox, 133
fast Fourier transform, 42
feasible flow, 329
FFT, *see* fast Fourier transform
Fibonacci heaps, 264, 288
Fibonacci numbers, 98, 193, 249
Fight Club, 180
finished vertex (depth-first search), 228
finite-state automaton, 194, 253, 255
  non-deterministic, 195
    PSPACE-hard problems, 413
First make it work, then make it fast, 22, 26, 40, 80, 84, 95, 96, 99, 106, 108, 109, 123, 289
First what, then how, 106
Fizzbuzz (standard interview question), 17
flood fill, 205
flow, 328
flow decomposition, 336, 358
  algorithm, 339
flow value $|f|$, 328
flow vector space, 346
flying kings, 223, 408
Ford-Fulkerson, 334
  can run forever, 335, 340, 349
  exponential running time, 335
  fattest augmenting paths, 340
  shortest augmenting paths, 341
Ford's relaxation algorithm, 276
  exponential running time, 300
forest (= acyclic graph), 191
formula satisfiability, *see* Sat
forward edge (depth-first search), 229
French flag walk, 303
French invasion of Indochina, 24
funny matrix multiplication, *see* min-plus matrix multiplication

G_f (residual graph), 332
gadgets, 395
Gale-Shapley algorithm, 173
game state, 75, 194
game trees, 74, **75**, 79, 80, 100
garbage collection, 210
gate gadgets, 395
general patterns
  backtracking, 79
  divide and conquer, 31
  dynamic programming, 105
  graph traversal, 199
  greedy exchange arguments, 164
  minimum-spanning-tree algorithms, 259

NP-hardness proofs, 392
shortest-path algorithms, 276
generic graph traversal, *see* whatever-first search
George of the Jungle, 65
Giggle, 151, 301
Gilbert and Sullivan
HMS Pinafore, 108
The Mikado, 404
The Pirates of Penzance, 374
golden ratio, 99, 335, 349
good pivot, 33, 36
median of medians, 37
graph coloring, 395
in interval graphs, 178
graph embedding, 192
graph reduction, 205, 212
graph traversal, 199, 209, 247
disconnected graphs, 203, 205, 227
graphical statics, 189
graphs
data structures
adjacency list, 195, 198
adjacency matrix, 196, 198
comparison, 198
implicit, 198, 207, 235
historical examples, 187
modern examples, 192
terminology, 190
greatest common divisor, 57
greedy algorithms, **159**
are usually wrong, 107
that don't work, 172, 177, 183, 185, 223, 349, 373
try dynamic programming first, 108
greedy exchange arguments, 160, 161, 163, **164**, 164, 168, 258, 260
guillotine subdivision, 145
see also kd-tree

Gulliver's Travels, 2, 3, 96, 98, 145

Hamiltonian cycle, *see* DIRECTEDHAMCYCLE, UNDIRECTEDHAMCYCLE
definition, 208, 398
Hamiltonian path, *see* DIRECTEDHAMPATH, UNDIRECTEDHAMPATH
definition, 401
Handshake Lemma, 218
hashtags, 80
head of an edge, 191
Hellenistic snobbery, 57
helpful drawings
evaluation order arrows, 110, 120
NP-hardness reduction, 387
recursion trees, 32
heuristic, 30, 76
see also algorithm that doesn't work
HITTINGSET, 406
How do I. . .
choose the right problem to reduce from?, 407
derive a dynamic programming algorithm?, 106
design a backtracking algorithm?, 79
prove that a greedy algorithm is correct?, 164
prove that a problem is NP-hard?, 386, 392
Huffman codes, 165, 179
Huntington-Hill algorithm, 22
Hyperbole and a Half, 261, 291
hypercube, 208

IBM, 26
implicit graph representation, 195
in-degree of a vertex, 191

incorrect proofs that P=NP, 404, 415, 416
independent set, *see* MaxIndSet, 390
index formulation, 77, 83, 88
indice, *see* index (dammit)
induction, ii, 23, 28, 29, 40, 75, 77, 100, 103, 163–165, 175, 200, 201, 230, 242, 260, 277, 282, 284, 287, 291, 318, 334, 336–338, 341
  backward, *see* dynamic programming
  see also recursion
induction hypothesis, *see* Recursion Fairy
infinite loop, 16, 172, 277, 294, 313, 335, 349
input size, 404
integer maximum flow, 334, 348, 358
integer multiplication
  divide-and-conquer, 40
  duplation and mediation, 5
  Karatsuba's algorithm, 41
  Toom-Cook algorithm, 42
  via fast Fourier transform, 42
integer multiplication lattice algorithm, 3
Integrality Theorem (maximum flows), 334
international draughts, *see* draughts
interpuncts (word·spacing), 80
intersection graph, 192
interval graph, 192
interview questions, 124, 143, 156
inverse Ackerman function $\alpha(n)$, 267
inversion counting, 51

Jarník's algorithm, 203, 263
Jeff actually did this, 249, 349
Johnson's algorithm, 312
jump in the middle, 80, 81, 86, 89

Kaniel the Dane, 218
Karp reduction, 385
kd-tree, 65
  see also guillotine subdivision
Klondike, 407
knights and knaves, 59
Kosaraju-Sharir algorithm, 240
Kruskal's algorithm, 265
Kubla Khan, 134
kuṭṭaka, *see* Euclid's algorithm

label of a path, 251–253
labeling graph components, 204
language (set of strings), 384
largest common subtree, 377
Latin, 2, 80, 428
lattice multiplication, 3
laws of physics, irrelevance of, 336
Let that which does not matter truly slide, 12, 14, 80, 82
level of a vertex, 341
Levenshtein distance, *see* edit distance
line breaking, 130
linear-time selection, 37, 53, 55
list of NP-hard problems, 404
literal, 388
local maximum, 63
local minimum, 61
logarithmic-space reduction, 385
logic gates, 379
longest common increasing subsequence, 126
longest common subsequence, 94, 115, 125
longest increasing digital subsequence, 140
longest increasing subsequence, 86, 109, 237

LONGESTPATH, 406
in directed acyclic graphs, 86, 88, 90, **236**
reduction from TRAVELINGSALESMAN, 236
loop invariant, *see* induction hypothesis
low(*v*), 242
lower bound via adversary argument, 380

magnetic tape, 159
MAJORITY3SAT, 419
majority gate, 396
many-one reduction, 385
marketing buzzwords, 102
Master Theorem, *see* recursion trees
matching, 355
non-crossing, 377
other special cases, 377
mātrāvṛtta, 97
matrice, *see* matrix (dammit)
matrix multiplication
boolean, 323
in sub-cubic time, 324
min-plus, **316**, 323, 325
standard, 323, 325
matrix rounding, 182, 374
MAX2SAT, 406
MAXCLIQUE, 394
reduction from MAXINDSET, 394
MAXCUT, 406
MAXINDSET, 390
in circular arc graphs, 185
in interval graphs, 162
in trees, 120
reduction from 3SAT, 390, 393, 395
reduction to MAXCLIQUE, 394
reduction to MINVERTEXCOVER, 394
maximum flows, **329**
acyclic, 339, 345
integer, 334
multiple sources and targets, 368
with vertex capacities, 354
maximum independent set, *see* MAXINDSET
maximum matching in bipartite graphs, 355
maximum subarray problem, 124
two-dimensional, 138
Maxwell-Cremona diagrams, 189
mazes, 190, 247
acute-angle, 216, 304
number, 212
median, *see* selection
median-of-medians selection, 37, 53
median-of-medians-of-medians selection, 55
median-of-three heuristic, 30, 50
memoization, 99, 194, 234, 271, 278
see also dynamic programming
memoized recursion is depth-first search, 234
mergesort, 26
mergesort recurrence, 28, 30, 33
metagraph, *see* strong component graph
methodisches Tatonnieren, 72
MINVERTEXCOVER, 394
reduction from MAXINDSET, 394
reduction to DIRECTEDHAMCYCLE, 398
reduction to SUBSETSUM, 402
min-plus matrix multiplication, **316**, 323, 325
Minesweeper, 406
minimum clique cover
in circular arc graphs, 186
in interval graphs, 178

minimum cuts, 330
minimum spanning trees, 203, **257**
  uniqueness, 258, 268
Minty's algorithm, *see* Dijkstra's algorithm
mom, *see* median of medians
mondegreen, 18
Monopoly, actual rules of, 386
Moore's algorithm, 292
Morse code, 97, 165
mountain climbing problem, 217
MST, *see* minimum spanning trees
multigraph, 191

n queens, 71, 79, 80
Nadirian Dream-Dollars, 123, 181
Napoleon Dynamite, 131
National Resident Matching Program, 171
"Needleman-Wunch" algorithm, 115
negative cycle detection, 292, 294, 299
negative cycles, 275, 277
negative edges, 274
neighbor, 191
Neitherlands (*The Magicians*), 376
nesting boxes, 372
new vertex (depth-first search), 228
NFA, *see* finite-state automaton
Nobel Prize in ~~Algorithms~~ Economics, 173
node, *see* vertex
none of your business, 21, 23, 25, 80, 206
  see also black box
NotAllEqual3Sat, 405
NP (nondeterministic polynomial time), 381
NP versus co-NP, 382
NP versus EXP, 414
NP versus PSPACE, 412
NP-complete, 383
NP-hard, 121, 138, 140, 235, 275, 358, 360, 381, **382**
  formal definition, 384
  weakly, 117, 404

obvious, 14, 76, 80, 317, 382, 383
Oh yeah, we already did this, 30, 78
one-armed quicksort, *see* quickselect
open problems
  all-pairs shortest paths, 317
  matrix multiplcation, 317
  optimal addition chains, 44
  optimal pancake flipping, 49
  P versus NP, 381
  sorting binary trees by swaps and rotations, 69
  winning international draughts in one turn, 411
open-pit mining
  see project selection, 366
optimal binary trees
  binary search trees, 91, 117
    variants, 96, 144
  expression trees, 141, 142
  prefix-free binary codes, 165
optimal substructure, *see also* correct recurrence, 112
ordered subtree, 156
Orlin's algorithm, 344
out-degree of a vertex, 191

P (polynomial time), 381
P versus NP, 382
P versus PSPACE, 412
P≠NP as a law of nature, 382
Pac-Man, 407
palindrome, 81, 128, 218, 222, 252, 377
pancake sorting, 49, 151
parallel assignment, 249
parent, 200, 276

see also mom
PARTITION
NP-hard problem, 405
subroutine in quicksort and quickselect, 29
party planning, 182
path compression, 271
path flow, 337
path in a graph, 191
peasant multiplication, *see* duplation and mediation
pebbling, 422
pecking order, 208
pixels, 206
PLANAR3SAT, 405
PLANARCIRCUITSAT, 405
planar graph, 192
PLANARNOTALLEQUAL3SAT, 405
Plankalkül, 205, 278
plumbus, 322
politics
academic, 59, 373, 376, 384
Illinois, 373
Renaissance Italian, 47, 59
Soviet, 383
postorder, 227
tree traversal, 65, 66, 122
power, *see* exponentiation
$pred(u, v)$, 309
$pred(v)$ (tentative predecessor), 276
predecessor of a vertex, 191
prefix, 77
prefix-free binary code, 165
preorder, 227
tree traversal, 65, 66
prerequisites, i
references, ii
Prim's algorithm, *see* Jarník's algorithm
project selection, 366
proper k-coloring, 395
proper subgraph, 191
Propositiones ad Acuendos Juvenes, 428
prosody, 97
see also Fibonacci nunbers
see also Morse code
pseudo-polynomial time, 117, 404
PSPACE (polynomial space), 412
PSPACE versus EXP, 414
PSPACE-hard, 412
punched cards, 26

QBF (quantified boolean formula), 413
quickselect, 35
quicksort, 29
quicksort recurrence, 33, 34

Racetrack, 220
rainbow, 387
RAND Corporation, 101
random-access machine, 384
$reach(v)$, 226
$reach^{-1}(v)$, 238
reachability, 191, 199
directed, 226
reciprocal diagrams, 189
recommended course policies, 108, 123
reconfiguration problems, 24, 45, 49, 68, 213, 215, 216, 219, 221, 428
recurrences
full history, 85, 92
removing floors and ceilings, 34
scary, 84, 92
solving with recursion trees, 31
recursion, **22**, 261, 291, 353
backtracking, 71
depth-first search, 225
divide and conquer, 26
smart, *see* dynamic programming

see also induction
Recursion Fairy, 22, 25, 26, 28, 37, 337
recursion trees, **31**, 49, 99
all levels equal, 28, 30, **33**, 33, 39, 49
backtracking, 72, 76
exponential decay, **33**, 37–39, 43, 49
exponential growth, **33**, 40, 49, 78, 85
path, 36, 37
weird, 30, 49
recursive brute force, *see* backtracking
reduced flow network, 331, 345
reduction, 21, 56, 205, 212, 385
regular expressions, 151
generalized, 414, 415, 417
PSPACE-hard problems, 413
relaxing a tense edge, 276
repeated squaring, 43, 44, 103, 317
replacement paths, 298
repricing, *see* vertex reweighting
residual capacity c_f, 331
residual graph G_f, 332
results by RAND researchers, 101, 276, 284, 331
results by students, 35, 40, 42, 166, 278, 324, 341, 383, 407
Revelation 13:15–18, 161
reversal $rev(G)$ of directed graph G, 234
reverse topological order, *see* postorder
Rick and Morty, 322
road maps, 187, 274, 310
rock climbing, 153, 184, 253
rooted subtree, 121
Rubik's Cube, 407
rule of three, 392, 393, 395, 405, 407, 408, 417
ruler function, 45, 46
Russian peasant multiplication, *see* duplation and mediation

$\|S, T\|$ (cut capacity), 329
(s, t)-cut, 329
(s, t)-flow, 328
safe edge, 259
SAT, 386
reduction from CIRCUITSAT, 386
satisfiability
circuit, *see* CIRCUITSAT
formula, *see* SAT
saturated edge (flows), 329
$scc(G)$ (strong component graph), 237
scheduling, 161
greedy algorithms that don't work, 177
via dynamic programming, 162
via greedy algorithm, 162, 363
via maximum flows, 358, 362
Scrabble, 134
scriptio continua, 80, 166
Seidel's algorithm, 317, 324
selection, 35, 52
median-of-medians, 37, 53
median-of-medians-of-medians, 55
quickselect, 35
self-descriptive sentence, 166
self-reduction, 416
semi-connected graph, 244
sequence alignment, *see* edit distance
sequence of decisions, 79, 112
series-parallel graph, 350
SETCOVER, 406
Sham-Poobanana University, 142, 161, 359, 362, 373

Shimbel's algorithm, *see* Bellman-Ford
shortest common supersequence, 95, 125
shortest path tree, 274
shortest paths, 203
  all-pairs, **309**
  analog algorithms, 278
  in directed acyclic graphs, 282
  in unweighted graphs, 278
  single-source, **273**
  versus shortest walks, 275
  with negative edges, 274, 288
    in undirected graphs, 275
  with non-negative edges, 284
shortest simple path
  reduction from DIRECTEDHAMPATH, 275
shuffle, 126
simple graph, 191
sink (vertex with out-degree 0), 231
sink component, 239, 242
snails, 143
Snakes and Ladders, 212
soapbox, vi, 12, 106
Sollin's algorithm, *see* Borůvka's algorithm
solving a more general problem, 36, 80
solving the right problem, 83, 87, 88, 90, 91
sorting algorithms
  mergesort, 26
  quicksort, 29
source (in a flow network), 328
source (vertex with in-degree 0), 231
source component, 240
spanning forest, 192
spanning tree, 192
squaring and mediation, 44
SSSP, *see* shortest paths, single-source
stable matching, 170, 179
starting time of a vertex (depth-first search), 228, 242
STEINERTREE, 406
Stigler's Law of Eponymy, 42, **98**, 114, 189, 261, 263, 284, 289, 383
Strassen's algorithm, 317
strong component graph, 237
strong components, 237
  connected in depth-first forest, 239
  in linear time, 238
    Kosaraju-Sharir, 240
    Tarjan's algorithm, 242
strong connectivity, 237
strongly connected components, *see* strong components
strongly connected graph, 192
subgraph, 191
subsequence, 86
subset construction, 195
SUBSETSUM, 76, 79, 80, 93, 116, 237, 402
  dynamic programming algorithm, 404
  in pseudo-polynomial time, 117
  reduction from VERTEXCOVER, 402
successor of a vertex, 191
Sudoku, 406
suffix, 83
Sumerian clay tablets, 56
summary of past decisions, 79
Super Mario Brothers, 407

Tabula Peutingeriana, 187
tail of an edge, 191
talking dog joke, 318, 383
tape sorting, 159

target (in a flow network), 328
Tarjan's algorithm, 242
tâtonner, 72
tense edge, 276, 312
Tetris, 407
text segmentation, **80**, 94, 105, 124, 130, 237
Theseus (maze-solving robot), 100, 278
Threes (game), 407
Tibetan Memory Trick, *see* The Announcer's Test
token (breadth-first search), 279
token (Moore's algorithm), 293
topological order, *see* reverse postorder
topological sort, **232**
  implicit, 232
Tower of Hanoi, 24, 44
  configuration graph, 194
  non-recursive solutions, 44
  recurrence, 26, 78, 85, 89
  variants, 46–48, 140
Trainyard, 407
transforming certificates, 392
transitive closure, 246, 323
transitive reduction, 246
TravelingSalesman, 402
  dynamic programming, 139
  Euclidean, convex position, 139
  reduction from DirectedHamCycle, 402
  reduction to LongestPath, 236
tree (connected acyclic graph), 191
  equivalent definitions, 207
tree edge (depth-first search), 229
tree traversal, 66, 227
  postorder, 122
trivial but useless $O(1)$-time algorithms, 17, 409
truth gadget, 396
TSP, *see* TravelingSalesman
tuple selection, 357
Turing machines, 384
Turing reduction, 384
Twitbook, 152, 301
typography, 80, 130

Ulam distance, *see* edit distance
undecided edge, 260
undirected graph, 190
UndirectedHamCycle, 402
  in a hypercube, 208
  reduction to international draughts, 409
UndirectedHamPath, 402
union-find, *see* disjoint-set data structure
unordered subtree, 156
UnSat, 385
useful deliberate ignorance, 22, 26, 35
useless edge, 259

vacuous base case, 25, 76, 87, 92
value of a node in a recursion tree, 31
Vankin's Kilometer, 138
Vankin's Mile, 137
variable gadget, 390, 395, 396, 400
vertex, 187
vertex cover, 394
vertex gadget, 398, 403, 410
vertex-disjoint paths, 354
vertice, *see* vertex (dammit)
Vidrach Itky Leda, 213

walk in a graph, 191
wavefront, 281, 284, 286
weakly NP-hard, 117, 404
weighted median, 53
WFS, *see* whatever-first search
Whackbat, 133
whatever-first search, 199

best-first (priority queue), *see also* best-first search, 202
breadth-first (queue), 202
depth-first (stack), 201
widest paths, 203, 270
word RAM model, 384
X3M, 405
XCNF-SAT, 419
xkcd, 407

zero cycles, 321

Dicebat Bernardus Carnotensis nos esse quasi nanos gigantium humeris insidentes, ut possimus plura eis et remotiora videre, non utique proprii visus acumine, aut eminentia corporis, sed quia in altum subvehimur et extollimur magnitudine gigantea.

[Bernard of Chartres used to say that we were like dwarfs seated on the shoulders of giants. He pointed out that we see more and farther than our predecessors, not because we have keener vision or greater height, but because we are lifted up and borne aloft on their gigantic stature.]

— John of Salisbury, *Metalogicon* (1159), translated by Daniel D. McGarry (1955)

The secret to productivity is getting dead people to do your work for you.

— Robert J. Lang (2009)

Index of People

Adelson-Velsky, Georgy, 96, 144, 341
Adler, Ilan, 364
al-Adli ar-Rumi, 190
Adversary, All-Powerful Malicious, 31, 161, 376, 379
Alcuin of York, 428
Alice, 217
Alighieri, Dante, 2
Alon, Noga, 317
Andersson, Arne, 96, 145
Apollonius of Perga, 3
Approximate Median Fairy, 33, 37
Archimedes, 3
Atlas, Charles, 102
St. Augustine of Hippo, 81

Bayer, Rudolf, 96, 144
Bellman, Richard, 101, 289
Berge, Claude, 356
Blagojevich, Rod, 373
Blum, Manuel, 35
Bob, 217
Borůvka, Otakar, 261
Brahmagupta, 3
Brosh, Allie, 261, 291

Cayley, Arthur, 190
Cegłowski, Maciej, 182

Chaucer, Geoffrey, 2
Chazelle, Bernard, 217
Chazelle, Damien, 217
Choquet, Gustav, 261
Chowdhury, Rezaul, 324
Cicero, Marcus Tullius, 80
Claus, N. (de Siam), *see* Lucas, Édouard
Clifford, William, 190
Cobham, Alan, 381
Cook, Stephen, 42, 383
Couper, Archibald, 190
Cremona, Luigi, 189
Culmann, Carl, 189

Dantzig, George, 276, 284, 289, 328
Demaine, Erik, 413
Dijkstra, Edsger, 210, 263, 269, 284
DiMaggio, Joe, 318
Dinitz, Yefim, 341
Durden, Tyler, 180
Dweighter, Harry (pseudonym of Jacob Goodman), 49

Edmonds, Jack, 311, 334, 340, 381
Elias, Peter, 331
"Engine Charlie", *see* Wilson, Charles Erwin
Eppstein, David, 209
Erera, Alan, 364
Erickson, Hannah, 216, 252
Erickson, Kay, 387
Euclid, 7, 57
Euler, Leonhard, 190, 247
Eutocius of Ascalon, 3

Fürer, Martin, 42
Fahlberg, Constantin, 74
Fano, Robert, 166
Feinstein, Amiel, 331
Fernández-Baca, David, 364
Fibonacci, *see* Leonardo of Pisa
Fischer, Michael, 114, 315
Floyd, Robert, 35, 318
Fontana, Giovanni, 190
Ford, Lester, 276, 331
Frederick II, Holy Roman Emperor, 46
Fredman, Micheal, 264
Frisius, Gemma, 190
Fulkerson, Delbert, 311, 331

Gödel, Kurt, 381
Gale, David, 172
Galil, Zvi, 317
Garey, Michael, 404
Gates, Bill, 49
Gauß, Karl Friedrich, 42, 72
Goldstine, Herman, 26
Goodrich, Michael, 209
Gregory IX, Pope, 47
Grimm, Jacob and Wilhelm, 23
Guibas, Leonidas, 96, 144
Gusfield, Dan, 364

Harris, Theodore, 327
Harvey, David, 42
Hearn, Robert, 409, 413
Herotodus, 2
Hierholzer, Carl, 190, 247
Hillier, John, 284
Hoare, Tony, 29, 35
Hochbaum, Dorit, 364
van der Hoeven, Joris, 42
Hopcroft, John, 356
Huffman, David, 166

Ingerman, Peter, 318

Jacobi, Carl, 356
Jarník, Vojtěch, 263
Jay, Ricky, 392
Johnson, David, 404
Johnson, Donald, 288, 311

Kőnig, Dénes, 356

Kalaba, Robert, 291
Kane, Daniel, 218
Karatsuba, Anatoliĭ, 40
Karp, Richard, 311, 334, 340, 356, 405
Karzanov, Alexander, 344
Kekulé, August, 190
al-Khwārizmī, Muḥammad ibn Mūsā, 2
Kirchhoff, Gustav, 190
Kleene, Stephen, 318
Kolmogorov, Andrei, 40
Kosaraju, Rao, 240
Kruskal, Joseph, 263
Kuhn, Harald, 356

Lamport, Leslie, 210
Landis Evgenii, 96, 144
Laquière, Emmanuel, 72
Ledger, Heath, 380
Lee, Chin Yang, 278
Leonardo of Pisa, 2, 4, 46, 97, 98, 101
Levin, Leonid, 383
Leyzorek, Michael, 284, 316
Loberman, Harry, 263, 266
Lucas, Édouard, 24, 72, 103
Łukaszewicz, Józef, 261

Mądry, Aleksander, 344
Margalit, Oded, 317
Marston, John, 18
Martel, Charles, 364
Martin, Alain J., 210
Martin, Steve, 10
Massé, Pierre, 101
Maxwell, James Clerk, 189
McKenna, Terence, 23
Meyer, Albert, 315
Michie, Donald, 100, 117
Miller, Gary, 50
Minty, George, 276, 284, 289
Mom, 38
Moore, Edward, 205, 273, 278, 289
Moreno, Jacob, 190
Morgenstern, Oskar, 101
Murena, Lucius Licinius, 80
Musk, Elon, 182

Nash, John, 381
Nauck, Franz, 72

Okasaki, Chris, 13
Olinick, Eli, 364
Orlin, James, 343

Pacioli, Luca, 45
Papadimitriou, Christos, 49
Pappus of Alexandria, 3
Park, Joon-Sang, 324
Peirce, Charles Sanders, 190
Penner, Michael, 324
Peranson, Elliott, 172
Piṅgala, 43, 97, 103
Pinker, Steven, 13
Pitt, Lenny, 74
Prasanna, Viktor, 324
Pratt, Vaughan, 35
Prim, Robert, 260, 263

Queyranne, Maurice, 349

Rabin, Michael, 381
Ramachandran, Vijaya, 324
Rebaudi, Ovidio, 74
Recursion Fairy, ii, 22, 77, 80, 82, 89, 164, 169
Remsen, Ira, 74
Rivest, Ronald, 35
Ross, Frank, 327
Roy, Bernard, 318
Rudraṭa, 190

Sainte-Laguë, André, 190
Sallows, Lee, 166

Samuel, Arthur, 100
Saxel, Jindřich, 261
Schönhage, Arnold, 42
Scholten, Carel S., 210
Schrijver, Lex, 327
Schumacher, Heinrich, 72
Schwartz, Benjamin, 364
Sedgewick, Robert, 96, 144, 145
Shannon, Claude, 100, 166, 190, 278, 331
Shapley, Lloyd, 172
Sharir, Micha, 240
Shier, Douglas, 289
Shimbel, Alfonso, 289, 314
Siedel, Raimund, 317
Skiena, Steve, vii
Smullyan, Raymond, 59
Snell, Willebrod, 190
Sollin, George, 261
Steele, Guy, 16
Steffens, Elisabeth, 210
Stevin, Simon, 189
Stigler, Stephen, 98
Stockmeyer, Larry, 415
Strassen, Volker, 42, 317
al-Suli, Abu Bakr Muhammad bin Yahya, 190
Sulpicius Rufus, Servius, 81
Sylvester, James, 190

Tarjan, Robert, 35, 242, 264
Tarry, Gaston, 247
Tomizawa, Nobuaki, 311
Toom, Andrei, 42
Trémaux, Charles, 247
Tseitin, Grigorii, 389
Turing, Alan, 101

Varignon, Pierre, 189
Virahāṇka, 98, 101
von Neumann, John, 26, 101, 381
von Staudt, Karl, 190

Wagner, Robert, 114
Waits, Tom, 379
Warshall, Stephen, 318
Wayne, Kevin, 366
Weinberger, Arnold, 263, 266
Weiss, Mark Allen, 96, 145
Whiting, Peter, 284
Whittlesey, Kim, 273
Wiener, Christian, 247
Wilson, Charles Erwin, 102
Witzgall, Christoph, 289
Woodbury, Max, 289

Yuval, Gideon, 325

Zermelo, Ernst, 75
Zuse, Konrad, 205, 278
Zwick, Uri, 335

We should explain, before proceeding, that it is not our object to consider this program with reference to the actual arrangement of the data on the Variables of the engine, but simply as an abstract question of the nature *and* number *of the* operations *required to be performed during its complete solution.*

— Ada Augusta Byron King, Countess of Lovelace,
translator's notes for Luigi F. Menabrea,
"Sketch of the Analytical Engine invented by Charles Babbage, Esq." (1843)

How to play the flute. [picks up a flute] Well, here we are.
You blow there and you move your fingers up and down here.

— Alan [John Cleese], "How to Do It",
Monty Python's Flying Circus, episode 28 (aired October 26, 1972)

Index of Pseudocode

This index includes only algorithms with explicit pseudocode; see the main index for other named algorithms.

AddAllSafeEdges, 262
AddSafeEdges, 272
AllPairsBellmanFord, 314
Alouette, 16
ApportionCongress, 9

BeAMillionaireAndNeverPayTaxes, 10
BellmanFord, 291, 292
BellmanFordDP, 295
BellmanFordDP2, 296
BellmanFordDP3, 296
BellmanFordFinal, 296
BFS, 279
BFSWithToken, 279
BinaryGCD, 58
Borůvka, 262, 272
BottlesOfBeer, 1

CircuitSat, 388
CollectStep, 211
ComputeOptCost, 119
ConstructSubset, 79
CountAndLabel, 204
CountComponents, 204
Cruel, 51

DAGSSSP, 283
DFS, 225, 226, 228, 235
DFSALL, 227, 228
DIJKSTRA, 285
DYNAMICPROGRAMMING, 235

EAGERWFS, 209
EUCLIDGCD, 58

FACTORIAL, 57
FALLING, 57
FASTEUCLIDGCD, 58
FASTLIS, 110
FASTLIS2, 111
FASTMULTIPLY, 41
FASTRECFIBO, 104
FASTSPLITTABLE, 106
FASTSUBSETSUM, 117
FELLMANBORED, 299
FETCHBIT, 63
FIBONACCIMULTIPLY, 4
FINDLOW, 243
FINDLOWDFS, 243
FINDSAFEEDGES, 272
FISCHERMEYERAPSP, 315
FLOYDWARSHALL, 319
FORDSSSP, 277

GARBAGECOLLECT, 211
GREEDYFLOW, 349
GREEDYSCHEDULE, 163

HANOI, 26
HHGUESS, 19

INITF, 118
INITSSSP, 276
ISACYCLIC, 231
ISACYCLICDFS, 231
ITERATIVEDFS, 199
ITERFIBO, 101
ITERFIBO2, 103

JARNÍK, 265
JARNÍKINIT, 265
JARNÍKLOOP, 265
JOHNSONAPSP, 313

KLEENEAPSP, 319
KOSARAJUSHARIR, 241
KRUSKAL, 267

LABELONE, 204
LEYZOREKAPSP, 316
LIS, 89, 90
LISBIGGER, 88
LISFIRST, 90
LONGESTPATH, 236, 237

MARKEVERYVERTEXDUH, 203
MEMFIBO, 100
MEMOIZE, 235
MERGE, 27
MERGESORT, 27
MOMOMSELECT, 55
MOMSELECT, 37
MOM_bSELECT, 53
MOORE, 293
MULTIPLYORDIVIDE, 8
MUTATE, 212

NONNEGATIVEDIJKSTRA, 288

OBVIOUSAPSP, 310
OPTIMALBST, 119
OPTIMALBST2, 120
OPTIMALBST3, 120

PARTITION, 29
PEASANTMULTIPLY, 6, 23
PEASANTPOWER, 44
PIŃGALAPOWER, 43
PLACEQUEENS, 73
PLAYANYGAME, 76
POSTPROCESS, 233
POSTPROCESSDAG, 233
POSTPROCESSDAGDFS, 233
POSTPROCESSDFS, 233
POSTVISIT, 227

Preprocess, 227
PreVisit, 227
PushDagSSSP, 284

QueyranneFatPaths, 350
QuickSelect, 36
QuickSort, 29

RecFibo, 99
RecTarry, 247
RecTarry2, 248
RecursiveDFS, 199
Relax, 277
RightAngle, 8
RulerHanoi, 45

ShimbelAPSP, 314
ShorterEdge, 259
SlowPower, 43
SplitMultiply, 40
Splittable, 83
SqrtSort, 52
StoogeSort, 50
StrongComponents, 239
SubsetSum, 77, 78

Tarjan, 244
TarjanDFS, 244
Tarry, 247
Tarry2, 248
ThreeColorQueueSearch, 211
ThreeColorQueueStep, 211
ThreeColorSearch, 210
ThreeColorStackSearch, 210
ThreeColorStackStep, 210
ThreeColorStep, 210
TopologicalSort, 233, 234
TopSortDFS, 233
TreeMIS, 122

Unusual, 51

WFSAll, 203
WhateverFirstSearch, 200, 205
WhoTargetsWhom, 62